

PIC-tutor v0.50

Contents

PIC-tutor v0.50	6
定位	11
源码基线	13
v0.40 章节范围	13
v0.40 已完成的增量	15
成书前必须补齐	16
v0.40 构建方式	16
0. 写作说明	17
1. 动理学模型与 PIC 的基本思想	18
1.1 Vlasov 方程首先是相空间守恒律	18
1.2 碰撞项不是 Vlasov 的一部分, 但必须保留边界	19
1.3 Vlasov-Maxwell: 源项、约束与闭合	19
1.4 Vlasov-Maxwell 的能量与动量守恒边界	20
1.5 Vlasov-Poisson / electrostatic 极限不是另一套世界	20
1.6 宏粒子不是假粒子, 而是 coarse-grained 分布函数载体	20
1.7 权重可以不同, 但不是没有代价	21
1.8 shape factor 不是插值细节, 而是粒子-网格合同	21
1.9 PIC 的噪声不是 bug, 而是模型代价	22
1.10 Debye 长度、粒子数与统计时间尺度	22
1.11 从连续模型到 PIC 离散变量	22
1.13 本章当前文献边界	24
1.14 练习与源码定位	24
2. PIC 总循环: 从 Vlasov-Maxwell 到离散时间推进	24
2.1 连续模型: Vlasov-Maxwell 系统	24
2.2 宏粒子表示与形函数	25
2.3 leapfrog 时间层	26
2.3.1 ω_p 不是背景常数, 而是时间离散必须尊重的最快等离子体尺度	26
2.3.2 λ_D 不只是一条长度定义, 它直接约束 Δx	27
2.4 FDTD 场更新的数学骨架	27
2.4.1 CFL 不是经验参数, 而是离散 Maxwell 更新的因果上界	28
2.4.2 数值色散从这里开始: 连续光锥被离散 stencil 改写	29
2.4.3 Yee / Nodal / CKC 的差别本质上是离散色散合同不同	30
2.5 PSATD 与 FDTD 的主循环差异	31
2.6 一个真实 PIC step 的工程层次	31
2.6.1 AMR subcycling: 两个时间步不是同一个时间步的重复调用	32
2.6.2 JRhom 与 implicit: 同一个外层 step 内部也可能有不同的时间合同	32
2.7 本章后的源码阅读入口	34
2.8 参数示例与最小运行案例	34
2.9 本章当前基础文献清单	35
2.10 进一步阅读与练习	35
3. WarpX 主演化路径: 生命周期、初始化与 Evolve()	35

3.1 顶层入口: main.cpp	36
3.2 WarpX 类与单例构造	37
3.3 ReadParameters(): 主循环分支的来源	37
3.3.1 构造函数只建“跨 level 外壳”, 不直接建完整网格数据	37
3.3.2 AllocLevelData() / AllocLevelMFs() 里, 四条 solver 分支的落点不同	38
3.4 InitData(): 把状态准备到可推进	40
3.5 ComputeDt(): 步长不是一个固定常数	41
3.6 Evolve() 外层时间步	42
3.6.1 步末 moving window: 连续坐标与整数网格平移	42
3.7 OneStep(): 求解器分派器	44
3.8 OneStep_nosub(): 显式电磁标准路径	45
3.9 SyncCurrentAndRho(): 源项不是沉积完就可用	47
3.10 PushParticlesandDeposit(): 进入粒子容器	47
3.11 OneStep_sub1() 与 JRhom 的位置	47
3.11.1 implicit 分支: 一次物理步包含多次试探性 source 重建	49
3.11.2 nonlinear solver、JFNK 与 mass-matrix: J 的三种构造层	49
3.12 参数示例与最小运行闭环	50
3.13 进一步阅读与练习	51
3.14 本章小结	51
3A. WarpX 初始化链: 从 InitData() 到初始粒子和外部场	52
3A.1 初始化链为什么值得单独成章	52
3A.2 启动层先于 InitData(): MPI、AMReX、FFT、PETSc 与运行时契约	53
3A.2.1 参数系统先分命名空间, 再分读取语义	53
3A.2.2 文档 alias、AMReX-owned 参数与 WarpX 本地 parser 不是一回事	54
3A.3 顶层入口: fresh run 与 restart 分叉	59
3A.4 InitFromScratch(): AMReX level 与粒子初始化	60
3A.5 外部场初始化: grid field 与 particle external field	61
3A.6 species 初始化: PlasmaInjector 是参数总容器	61
3A.7 密度和动量分布: 从文本参数到 functor	62
3A.7 AddParticles(): 按注入类型进入创建函数	63
3A.8 体注入 AddPlasma(): 候选粒子、密度、动量和权重	64
3A.9 Gaussian beam: 显式束流列表注入	65
3A.10 openPMD 粒子文件: 文件粒子列表注入	66
3A.11 Projection divergence cleaning: 外部 A/B 场的初始约束修正	67
Laser antenna 与 profile 分派: laser 初始化并不走 PlasmaInjector	68
3A.12 初始化验证入口: 哪些 regressions 真正在兜底	74
3A.13 本章小结: 初始化状态怎样进入第一步推进	80
3A.14 练习与最小复现	80
4. 粒子推进器: 从 Lorentz 方程到 PushPX()	81
4.1 连续 Lorentz 方程	81
4.2 Boris 推进的结构	82
4.3 WarpX 如何选择 pusher	83
4.4 Vay pusher: 相对论速度变换一致性的更新	84
4.5 Higuera-Cary pusher: Boris-like 结构的相对论修正	86
4.6 从 MultiParticleContainer 到 PhysicalParticleContainer	88
4.7 PhysicalParticleContainer::Evolve() 的 tile loop	89
4.8 PushPX(): gather 和 push 的融合 kernel	90
4.9 doGatherShapeN(): 从网格场到粒子场	91
4.10 位置推进与无质量粒子	97
4.11 RR、implicit 与 photon path	98
4.12 Field Ionization: ADK 不是附加后处理, 而是粒子属性和沉积权重一起改写	99
4.13 Collisions: 不是旁路模块, 而是和 momentum push 强耦合的时间调度层	101
4.13.1 BinaryCollision 先产出事件表, ParticleCreationFunc 再真正创建 products	102
4.13.2 BackgroundMCC、PulsedDecay 与 DSMC: collision 还有三条完全不同的后半段	103
4.13.3 pairwisecoulomb、bremsstrahlung、background_stopping 与 nuclearfusion	104

4.13.4	kernel 细节层: Perez 有效碰撞密度、Bremsstrahlung photon 写回、fusion products 复制语义	106
4.13.5	更深一层的 event kernel: Perez 角分布、Bremsstrahlung 能谱反演与 fusion 概率控制	107
4.13.6	性能与数值后处理层: resampling、thermalizer 与 sorting	108
4.13.8	particle_pusher、single_particle、larmor、photon_pusher 的真实验证边界	109
4.13.9	particle_fields_diags 与 plasma_lens: 粒子 diagnostics 和粒子侧外场的两类强验证	112
4.13.10	particle_boundary_scrape、particle_data_python 与 single-precision diagnostics: 粒子 Python 接口的三类合同	113
4.13.11	particle_boundary_interaction、particle_boundary_process、particle_thermal_boundary 与 plasma_lens_python	114
4.13.12	point_of_contact_eb、particles_in_pml、subcycling_mr 与 Langmuir multi_mr	117
4.13.13	余下 embedded_boundary、electrostatic_sphere_eb 与辅助绘图脚本的真实边界	118
4.14	QED: 不是一个统一开关, 而是三条不同的 product-species 事件链	120
4.14.1	runtime attribute 和 product species 在构造期就锁定	120
4.14.2	InitQED() 真正做的是 engine/table 装配, 不是事件执行	121
4.14.3	主循环里的插入顺序: field ionization 后, particle injection 前	121
4.14.4	Quantum Synchrotron 与 Breit-Wheeler 都走 filterCopyTransformParticles	121
4.14.5	Schwinger 是第三条完全不同的创建路径	122
4.14.6	regression 的三组 strongest evidence	122
4.14.7	kernel 层真正的触发条件: 先推进 optical depth, 再跑 event pass	122
4.14.8	wrapper 的职责边界: WarpX 管场与属性, PICSAR-QED 管采样	123
4.14.9	Quantum Synchrotron 与 Breit-Wheeler 的 source 语义不同	123
4.14.10	QED table 不是附属数据, 而是 kernel 可执行性的前提	123
4.14.11	QedChiFunctions、virtual photons 与 linear_breit_wheeler 其实属于两棵不同的树	124
4.15	本章结论	126
4.16	练习与复现实验	126
5.	电荷、电流沉积与形函数: 源项如何回到网格	126
5.1	电荷沉积的基本形式	127
5.2	ShapeFactors.H: WarpX 实际使用的 0 到 4 阶形函数	128
5.3	电流沉积的守恒要求	130
5.3.1	五条路径的责任边界总览	131
5.4	WarpX 的旧电荷、新电荷和半步电流	132
5.5	多物种层如何清零和汇总源项	133
5.6	AMR coarse-fine interface: 粒子怎样切到 aux/cax/buf 路径	134
5.7	WarpXParticleContainer::DepositCurrent() 分派	136
5.8	WarpXParticleContainer::DepositCharge() 入口	141
5.8.1	icomp 不只是分量号, 而是旧/新 rho 的时间层合同	142
5.8.2	普通 charge deposition 的桥接合同在 ABLASTR, 而不在 kernel 本体	143
5.8.3	shared-memory charge deposition 不是走 ABLASTR, 而是先变成 tile-binned 执行合同	143
5.9	ChargeDeposition.H: 电荷沉积 kernel 的逐项结构	144
5.10	Direct current deposition: 非守恒但直观的速度加权沉积	149
5.11	Esirkepov current deposition: 用新旧形函数差构造连续性方程	151
5.11.1	Villasenor-Buneman 公式级审计: 四边界、重复分段与三维交叉项	160
5.11.2	Esirkepov 运行级维度证据: 1D、2D 与 3D Langmuir	163
5.12	沉积不只回 rho/J: WarpX 还把线性响应矩阵和统计矩交回网格	164
5.12.1	mass matrices: 沉的是 J(E) 的线性响应, 不是当前步的 Maxwell 源项	164
5.12.2	temperature / variance deposition: 沉的是样本数、加权均值和去均值二次矩	165
5.12.3	这一节回头修正了本章的总边界	166
5.13	沉积后为什么还要同步	166
5.13.1	PSATD 与 FDTD 的同步时序不同	166
5.13.2	SyncCurrent() 的骨架: coarsen、buffer、owner mask、filter、SumBoundary	167
5.13.3	SyncRho() 平行但不完全等于 SyncCurrent()	168
5.13.4	SyncCurrentAndRho() 的最后一步其实是边界条件	168
5.13.5	本章对应的两条关键 regression, 分别在验证哪一层 source-synchronization 合同	169
5.14	形函数、guard cells 与稳定性	170
5.15	本章结论	172
5.16	练习与源码定位	172

6. 电磁场求解器	172
6.1 FDTD 差分算子: Yee, Nodal 与 CKC	176
6.2 FDTD PML split-field 更新	176
6.3 PML sigma profile、damping 与电流源项	177
6.4 非 Cartesian FDTD: RZ、RCYLINDER 与 RSPHERE	178
6.5 PSATD 谱求解主流程	179
6.6 标准/Galilean PSATD 系数和 current correction	180
6.6.1 v0.26 X1-X4 系数的源码公式闭环	181
6.6.2 v0.27 time-averaging Psi/Y 系数的源码公式闭环	182
6.6.3 v0.17 文献闭环: Lehe et al. 2016 的 Galilean PSATD	184
6.6.4 v0.18 文献闭环: Kirchen et al. 2016 的 boosted-frame 应用证据	184
6.6.5 v0.19 文献闭环: Godfrey et al. 2014 的 PSATD NCI 策略谱系	185
6.6.6 v0.20 源码闭环: WarpX PSATD/NCI 机制对照表	186
6.7 PSATD-JRhom: 多次源项沉积与一阶/二阶谱更新	188
6.7.1 v0.28 JRhom second-order Y1-Y8 系数的源码公式闭环	190
6.8 RZ PSATD: Hankel transform、azimuthal modes 与 E_p/E_m	192
6.8.1 v0.29 RZ/Galilean RZ PSATD 系数边界	194
6.8.2 v0.30 comoving PSATD 系数与验证边界	196
6.8.3 v0.31 comoving PSATD regression analysis 方案	198
6.8.4 comoving PSATD reference 标定与 patch 收口条件	199
6.8.5 PSATD/场求解器算法族决策矩阵	203
6.9 静电与静磁求解器	204
6.9.1 Poisson 边界条件	206
6.9.2 Lab-frame 静电	206
6.9.3 Relativistic self fields	206
6.9.4 Effective potential	207
6.9.5 Magnetostatic vector Poisson	208
6.10 Hybrid PIC: 广义 Ohm 定律与 B 场 RK 子步	209
6.10.1 参数与辅助场	210
6.10.2 顶层 field update	210
6.10.3 Ampere 电流与 Ohm kernel	211
6.10.4 电子压力与外部矢势 split field	213
6.10.5 Fluids/ 与 HybridPICModel 不是同一条流体链	213
6.11 FieldSolver regression 判据: 从 analysis 脚本反读物理检查量	214
6.11.1 NCI FDTD: 场能增长是否被 corrector 压住	214
6.11.2 NCI PSATD: 电场能量比与 Gauss law	215
6.11.3 Maxwell hybrid QED: 真空修正后的相速度偏移	215
6.11.4 静电球: 解析场 L2 误差与能量守恒	216
6.11.5 隐式 EM: 能量、Gauss law 与求解器迭代数	219
6.11.6 Hybrid Ohm solver: 哪些是强判据, 哪些只是输出回归	223
6.11.7 具体 regression 入口索引	224
6.12 练习与运行验证	226
6.12.1 本章验证链的结论	226
7. 边界条件、PML 与 AMR	226
7.0 v0.8 源码入口地图	227
7.1 field / particle 边界不是两套彼此独立的输入	228
7.2 电磁 field boundary 的顶层分派	228
7.3 PEC / PMC 不只是场边界, 也是沉积对称性	229
7.4 PML 在 WarpX 里是独立子域, 不是单个边界公式	230
7.5 PML split field 与 PML 电流	230
7.5.1 v0.9 边界 regression 入口索引	231
7.5.2 v0.10 四条强 analysis 路径的正文化	238
7.5.3 v0.21 PSATD PML 源码闭环: 普通 PML、RZ PML 与 regression 边界	240
7.5.4 v0.22 PML 理论文献闭环: 从 Berenger/APML 到 PSATD split-field 系数	243
7.5.5 v0.23 LeeCPC2015 获取审计与公式映射准备	244
7.5.6 v0.24 PsatdAlgorithmPml.cpp 的 C1-C25 系数图谱	245

7.5.7 v0.25 LeeCPC2015 的论文-源码公式核对清单	246
7.5.8 v0.40 LeeCPC2015 accepted-manuscript 资产与源码映射	247
7.5.9 v0.40 PSATD-PML 的 Cartesian/RZ 并列复现	247
7.6 Embedded boundary 先是几何初始化和辅助标记系统	248
7.7 Embedded boundary 的 face extension: 把不稳定 cut face 变成 enlarged face	249
7.8 Embedded boundary 的粒子侧: signed-distance 判定、默认吸收与 scraped buffer	250
7.7.1 v0.11 guard-cell、通信和 regrid 的闭环	254
7.7.2 v0.12 coarse-fine substitution 与 transition zone 的证据图	258
7.7.3 v0.14 transition-zone validation 应该直接检查什么	259
7.7.4 v0.15 dedicated transition-zone 测试草案	261
7.7.5 v0.16 transition-zone regression patch 计划	263
7.7.6 练习与源码定位	266
7.7.7 v0.40 transition-zone 当前证据状态	266
8. 诊断、验证与案例	267
Langmuir wave	268
Uniform plasma	270
2026-07-12: checkpoint/restart 的真实证据	271
LWFA/PWFA	272
LWFA: laser_acceleration 是 runtime matrix, 不是统一 wake benchmark	273
PWFA: plasma_acceleration 是 workflow matrix, 不是解析 wake benchmark	274
当前 worktree 中这条应用主线真正成立的结论	274
Laser ion / plasma mirror / RPA/TNSA	275
laser_ion: 最强的 laser-target application entry	275
plasma_mirror: laser-solid surface-plasma workflow baseline	275
RPA/TNSA: 当前属于物理解释层, 不属于本地应用目录层	276
Capacitive discharge	276
1D PICMI 是当前最强的 Turner benchmark 入口	276
当前最硬断言是 case-1 ion-density profile	277
DSMC 版不是孤立小 test, 而是同一 benchmark scaffold 的分支	277
2D native / PICMI 当前仍主要是 workflow baseline	277
既有 plasma_acceleration 目录边界	277
Magnetic reconnection	278
force-free sheet + reduced FieldProbe	278
当前 analysis 是 observable extraction, 不是强 assert	279
checksum 仍是 active coverage 的另一半	279
Beam-beam / luminosity / FEL / ion extraction	279
DifferentialLuminosity: reduced-diagnostic 强谱基准	280
beam_beam_collision: collider-QED 应用骨架	280
free_electron_laser: boosted rigid-beam + undulator + BTD 的强 benchmark	280
ion_beam_extraction: EB electrostatic extraction 的强应用入口	281
accelerator_lattice: beamline optics 的强回归层	282
诊断在源码中的位置	282
BoundaryScrapingDiagnostics 与 Python scraped-particle buffer	287
固定模板案例页	288
模板 A: plotfile	288
模板 B: openPMD	289
模板 C: checkpoint	289
模板 D: BoundaryScraping/openPMD	290
Python 边界 buffer 的最小消费模板	291
模式 A: 运行结束后统一检查	291
模式 B: callback 或在线控制里的即时事件流	291
这和 BoundaryScrapingDiagnostics 的分工	292
Python field wrapper 的最小强回归	292
本地运行注意	293
2026-07-11 reader-side 验证增量	293
2026-07-12: reduced diagnostics 与 full-state reference 对照	294

LoadBalanceCosts: 性能诊断的 efficiency gate	295
ColliderRelevant: 束流统计与 luminosity-rate 聚合合同	296
DifferentialLuminosity: 1D/2D 解析谱与 AMR 对照	297
ParticleHistogram2D: 二维 openPMD writer 合同	298
BeamRelevant: 束流矩与截断高斯束合同	300
Native external-file Gaussian beam: 输入文件路径与束斑物理合同	301
第 8 章验证矩阵: 命令、产物与 gate	301
8.15 练习与复现实验	303
本章后续扩写	303
9. 文献路线与后续扩写计划	303
9.1 当前项目的文献证据分层	303
9.2 当前已 materialize 的核心文献树	304
9.2.1 PIC foundations	304
9.2.2 Particle pusher	304
9.2.3 PSATD / Galilean / boosted-frame / NCI	304
9.3 当前最突出的未闭环文献缺口	305
9.4 各章当前的文献成熟度	305
9.5 acquisition 优先级的重新排序	306
9.6 对 docs/literature-map.md 的使用边界	306
9.7 下一轮最合理的文献推进目标	306
方案 A: 第 5 章沉积文献闭环	306
方案 B: 第 7 章 PML 论文闭环	307
9.8 本章结论	307
附录 A: 符号、时间层与源码变量	307
A.1 连续模型符号	307
A.2 网格、时间层与离散量	308
A.3 沉积、推进与场求解记号	308
A.4 诊断、文件与验证术语	308
A.5 常用缩写	309
A.6 使用规则	309

PIC-tutor v0.50

当前合订 PDF 为 309 页; 本 v0.50 版本在 v0.49 基础上补入 Esirkepov paper-to-WarpX notation matrix, 明确论文方向分解、源码前缀累加、横向混合平均和电流归一化之间的层级边界。

版本日期: 2026-07-12

本次新增 Esirkepov notation matrix: 对当前 CurrentDeposition.H 的 14 个源码锚点全部通过, 固定 $W^1/W^2/W^3$ 到 $sdxi/sdyj/sdzk$ 、 $Jx/Jy/Jz$ 的方向映射, 记录 one_third/one_sixth 的 old/new tensor-product mixed average 和 invdtd 的 transverse inverse cell area / dt 归一化。该结果支持 preprint-backed + source-grounded 的记号说明, 不替代 CPC publisher-PDF line-by-line compare。

本次新增 3D AMR particles-in-PML 运行级分解: 初始 diag1000000 的 $Ex/Ey/Ez$ 全零; 末态 diag1000200 finest covering grid 上, 官方 signed max 为 $106.4354 < 110$, 严格 absolute max 为 $110.3994 > 110$, 唯一越界项是边界侧 PML 区域的负向 Ex , 而 Ey/Ez 的正负峰值绝对值均约 102.5971。level 0/1 读取给出相同极值。该结果只定位 signed consumer 的判据盲区, 不修改 ../warpx、阈值或上游 analysis。

本次新增 RZ secondary-emission resolution-aware boundary: 默认 64x64 case 的最大 EB impact-point 回溯误差为 3.6038% > 2%, 仍保持 BOUNDARY; 同一输入的 128x128/256x256 controls 分别为 0.9977%/0.6646%, 均通过官方 2% gate。该结果支持分辨率敏感性诊断, 不修改默认输入、不放宽 tolerance, 也不外推为 upstream baseline 已修复。

本次新增完整高阶 shape family: 在 128x256 下 shape=2/3/4 的 correction-on/off field gate 全部通过; correction-off charge residual 为 $9.644e-12/6.086e-12/6.724e-12$, correction-on axis charge residual 为 $2.177e-3/2.353e-3/2.552e-3$ 。该结果没有把 correction-on residual 伪装成已闭合, 也没有建议修改全局默认值。

本次新增 rho-side observable: 对 shape=2/3/4 高分辨率 correction-on sibling 直接读取 rho、rho_electrons 和 rho_ions,

末态 $\rho - (\rho_{\text{electrons}} + \rho_{\text{ions}})$ 相对差为 $1.303\text{e-}14/1.228\text{e-}14/1.343\text{e-}14$; 同时记录 integrated- ρ 时间序列漂移 $2.371\text{e-}6/2.729\text{e-}6/3.354\text{e-}6$ 。后者只作为 ρ -side 观测, 不替代 $\text{divE-}\rho$ 、current closure 或完整守恒证明; 既有 $\text{divE-}\rho$ axis residual $2.177\text{e-}3/2.353\text{e-}3/2.552\text{e-}3$ 继续保留为独立边界。

本次新增 shape=2/3/4 的 64x128 $\rightarrow$ 128x256 correction-off refined siblings: 粗网格 Er error 为 0.1323/0.1734/0.2134, 细网格降至 $9.318\text{e-}3/1.113\text{e-}2/1.365\text{e-}2$; 三档 charge residual 均低于 $1\text{e-}11$, field/charge 双 gate 全部通过。该族结果仍限定为 RZ、单层、固定 Langmuir 输入, 不改变全局轴修正默认值。

本次新增 shape=2 的 64x128/128x256 paired runtime: correction-off 的 Er field error 从 0.1323 降至 0.0093, charge residual 为 $2.202\text{e-}12/9.644\text{e-}12$, 高分辨率 field/charge 双 gate 通过; correction-on 高分辨率 field error 为 $9.321\text{e-}3/5.154\text{e-}3$, axis charge residual 为 $2.177\text{e-}3$, 因此仍保留 charge boundary。该结果仅支持 shape/resolution/axis-correction 交互解释。

本次新增 RZ Esirkepov particle_shape=1 的 64x128/128x256 paired runtime: 默认 correction-on 的 axis charge residual 从 $3.593\text{e-}3$ 降至 $1.520\text{e-}3$, correction-off 在两档分辨率均通过 field/charge 双 gate, 128x256 的 charge residual 为 $9.353\text{e-}12$ 。既有 shape=2/3/4 correction-off 的 Er field 边界仍保留, 不能把 shape=1 结果外推为全局默认建议; 未修改 ../warpx。

本次新增 particles_in_pml analysis source contract: 审计确认上游 analysis_particles_in_pml.py 使用 $\max(\text{Ex.max()}, \text{Ey.max()}, \text{Ez.max}())$, 而项目独立 contract 使用逐分量 $\max(\text{abs}(\text{field}))$; 3D AMR sibling 的官方 $106.43539539129057 < 110$ 与强化 $110.3993781372607 > 110$ 因此不能合并成同一个“通过”结论。该审计只修改 PIC-tutor 文档和脚本, 不修改 ../warpx。

本次增量完成 RZ JRhom first-stage handoff asset 的可重复重建与目标 checkout dry-run 验收: bundle、helper、unified diff、provenance、submission packet 和 PR draft 均由 MPI=2 ledger 重建; 目标 WarpX 保持 unstaged, 未写入上游源码。baseline helper 通过, ll2-no-timeavg-cleaning 负对照按预期拒绝, 独立 contract 继续为 passed=true。

同一增量还完成 comoving PSATD first-stage handoff asset 的可重复重建与正负复核: stable helper 通过, no-comoving reference 被 spike ceiling 拒绝, 独立 contract 为 passed=true; 目标 WarpX 仍保持 unstaged, energy gate 继续明确关闭。

本次还补做 RZ Galilean current_correction paired runtime: 非 PSB 2-rank 仅 charge gate 略超阈值, PSB single-box single-rank 同时通过 energy 与严格 $1\text{e-}9$ charge gate; 独立 paired contract 已归档, 分别保留 CHARGE_BOUNDARY 与 PASS。

本次继续补做 RZ Langmuir PSATD current_correction sibling: 官方解析场与 charge analysis 通过, 独立 Er/Ez 与同面 $\text{divE-}\rho/\epsilon_0$ contract 也通过。

本次又补做 RZ Langmuir PSATD-JRhom CL4 2-rank sibling: 官方与独立解析场 contract 通过; 由于 current_correction=0, 正文明确保留为 field/filter evidence, 不宣称 charge-conservation gate。

本次继续补做标准 RZ Langmuir PSATD 2-rank sibling: 官方与独立解析场 contract 通过, 形成 standard / current-correction / JRhom CL4 三条 RZ Langmuir 对照; standard sibling 同样不适用 charge gate。

本次又完成官方 RZ PSATD-PML 2-rank 复现: 独立 residual-field contract 得到 $\max|\text{Er}|/\max|\text{Ez}|=1.0316/0.5695$, 通过 < 2.0 gate; RZ PML 证据从历史 1-rank 项目级结果提升为官方 2-rank + 独立复核。

本次还新增 RZ secondary-emission EB source contract: 10/10 源码锚点通过, 确认 callback/source wiring 完整; 64×64 impact-point runtime gate 仍保持失败, $128 \times 128/256 \times 256$ 仅作为分辨率敏感性对照。

本次又新增 RZ Langmuir PSATD family matrix: standard、current-correction 和 JRhom CL4 三条 official-input contract 的解析场 gate 全部通过, charge gate 的适用范围单独保留在 current-correction sibling。

本次还生成 v0.41 public-release manifest: 484 个项目文件、20,385,796 bytes, allowlist 排除 runs/、references/ 和历史生成物; 该 manifest 只服务发布审计, 不代表已经完成 Git staging 或 push。

2026-07-12 增补 Esirkepov 1D/2D/3D Langmuir runtime evidence, 并将 2D 扩展到当前支持的 shape=1/2/3/4: 2D 理论场最大误差为 $1.2201\text{e-}2/3.4096\text{e-}2/4.6336\text{e-}2/6.0165\text{e-}2$, 分别通过 0.0503/0.0503/0.0503/0.07 官方阈值; charge residual 为 $3.5650\text{e-}12/3.1326\text{e-}12/4.5607\text{e-}12/2.8977\text{e-}12$, 均低于独立 $1\text{e-}11$ gate。shape=0 在 WarpX.cpp:1450 初始化断言处拒绝; 新增 2D MR overlay 的逐层 charge contract 为 L0/L1 0.8828/1.2005, 因此标记为 BOUNDARY, 不升级为 AMR 强守恒结论。

2026-07-12 增补 Esirkepov AMR route/source-sync source contract: 当前 WarpX checkout 的 15 个源码锚点全部通过, 覆盖粒子路由到 current_fp/current_buf、depos_lever=lev-1 coarse 几何、buffer masks、SyncCurrent 与 fine-to-coarse merge; 该证据与 2D MR runtime 的 L0/L1 BOUNDARY 结果并列, 明确不替代 dedicated intermediate-field diagnostics。

2026-07-12 增补 Python MR intermediate-field observability audit: MultiFabRegister 的 list/has/_get、PICMI sim.fields、现有 current_fp Python regression 以及 current_buf/rho_buf allocation 的 7 个锚点全部通过; 当前分类为 INTERFACE_PRESENT_RUNTIME_LEDGER_UNPROVEN, 不把 generic field-register API 升级为 MR 中间场 runtime 证据。

2026-07-12 增补 transition-zone route-count implementation packet: 固定 PartitionParticlesInBuffers、PhysicalParticleContainer::Evolve、SyncCurrent/SyncRho 的源码插入点, 定义 nfine/nbuffer、weight、rho/J、coarsened fine、owner-mask 和 post-sync 的最小 reduced schema 与 gates; 该设计尚未写入 ../warpx。

2026-07-12 增补 charge deposition bridge source contract, 以 13 个源码锚点验收时间层偏移、ABLASTR 暂存、形状函数分派、RZ 分支和原子写回。

2026-07-12 增补 deposition geometry/order source contract, 以 53 个源码锚点验收 charge ordinary/shared、Direct、Esirkepov、Villasenor、Vay、implicit 的 shape=1/2/3/4 分派和六类几何分支; 该证据不替代全组合 runtime regression。

2026-07-12 补入 RCYLINDER/RSPHERE Esirkepov Langmuir 2-rank runtime evidence; 独立径向 Er contract 的相对误差为 $2.174e-2/5.405e-3 < 0.12$, 范围限定为两条径向场合同。

2026-07-12 将 RCYLINDER/RSPHERE Esirkepov radial Er runtime coverage 扩展到 shape=1/2/3/4, 8 个 geometry×shape contract 全部通过; 范围仍限定为径向场, 不替代 charge/Gauss-law 验证。

2026-07-12 补入 RCYLINDER/RSPHERE shape=1 rho/divE charge 对照: 关闭 axis correction 后 RCYLINDER 通过 $1e-11$, RSPHERE residual 降至 $2.420e-11$ 但仍保留边界。

2026-07-12 增补 RSPHERE 64/128 resolution 与 axis-correction paired control; field gate 全通过, charge residual 保留为 resolution-sensitive boundary, 不宣称收敛阶。

2026-07-12 增加 radial axis-volume correction source contract, 10 个参数、RZ/RCYLINDER/RSPHERE 因子、charge scaling 和 rho buffer 调用锚点全部通过。

2026-07-12 复核 Esirkepov CPC 定稿的替代索引和 publisher endpoint: 元数据状态已加强, 但 PDF 仍返回 HTTP 403; publisher-PDF line-by-line compare 继续未完成。

2026-07-12 补入 RZ Esirkepov Langmuir 2-rank runtime evidence; 独立 Er/Ez field contract 通过, 误差为 $1.075e-2/8.240e-3$, 但同面 charge residual $3.593e-3 > 1e-11$, 保留为 charge BOUNDARY。

2026-07-12 补入 RZ Esirkepov charge boundary 源码语义 note, 核对官方 RZ 排除条件以及 ComputeDivE、DivEFunctor、RhoFunctor、FullDiagnostics 的诊断分叉; 当前 residual 只作为诊断合同边界, 不归因成单一 kernel 失败。

2026-07-12 增加 RZ charge diagnostic source contract, 11 个官方 exclusion、ComputeDivE/FDTD、DivE functor、rho functor 和 FullDiagnostics 锚点全部通过。

2026-07-12 增补 RZ Esirkepov do_dive_cleaning=1/0 paired control: 全局 charge residual $3.593e-3 \rightarrow 9.693e-2$, 约 26.98x, 且最大值由 axis cell 主导; 当前分类为 AXIS_DOMINATED_CLEANING_SENSITIVE_DIAGNOSTIC_BOUNDARY。

2026-07-12 增补 boundary.verboncoeur_axis_correction=true/false RZ paired control: 关闭 correction 后 charge residual 为 $5.513e-12 \leq 1e-11$, field gate 仍通过; 当前分类为 AXIS_CORRECTION_OFF_RESTORES_CHARGE_GATE, 不修改全局默认值。

2026-07-12 将 RZ Esirkepov Langmuir 默认轴修正 runtime shape coverage 扩展到 shape=1/2/3/4; field gate 全部通过, charge residual 仍由 axis cell 主导并保持 BOUNDARY。

2026-07-12 增补 RZ shape=2/3/4 的 axis-correction-off 交叉对照: charge gate 恢复但 Er field gate 失败, 确认存在 shape-dependent charge/field tradeoff。

2026-07-12 增补 2D AMR subcycling workflow 证据: 官方 2-rank、250 步 producer 与独立 contract 均通过, 最终两层 64x256、moving-window shift error $7.617e-8 \text{ m} \leq \text{coarse dz}$ 、四 species 存在; 证据范围限定为输出完整性与几何时间合同, 不替代 transition-zone route-count/守恒验证。

2026-07-12 增补 Cartesian 1D Silver-Mueller 证据: 官方 2-rank、500 步 producer 与独立 contract 均通过, Ex/Ey/Ez 最大绝对值为 $3.887e-8/3.887e-8/0 \text{ V/m}$, 低于 0.01 V/m ; Silver-Mueller 当前已覆盖 1D、2D x/z 与 RZ sibling 的短时 residual-field 合同。

2026-07-12 增补 Cartesian 2D Silver-Mueller z 证据: 官方 2-rank、500 步 producer 与独立 contract 均通过, Ex/Ey/Ez 最大绝对值为 $3.912e-3/3.516e-3/9.149e-4 \text{ V/m}$, 与 x 向 sibling 形成分量置换对照; 证据仍限定为短时低残余场。

2026-07-12 增补 Cartesian 2D Silver-Mueller x 证据: 官方 2-rank、500 步 producer 与独立 contract 均通过, Ex/Ey/Ez 最大绝对值为 $9.149e-4/3.516e-3/3.912e-3 \text{ V/m}$, 均低于 0.01 V/m ; 该证据与 RZ Silver-Mueller 短时残余场合同并列, 不替代系统反射率扫描。

2026-07-12 增补 3D PSATD-PML cleaning diagnostic semantics note: 把 ComputeDivE、DivBFuncor、DivEFuncor 和 PML F/G 输出边界接回 clean/control report, 明确本版不把 cell-centered native divB 对照升级为 strong cleaning physics gate.

2026-07-12 增补 3D PSATD-PML cleaning diagnostic semantics note: 把 ComputeDivE、DivBFuncor、DivEFuncor 和 PML F/G 输出边界接回 clean/control report, 明确本版不把 cell-centered native divB 对照升级为 strong cleaning physics gate.

2026-07-12 增补显式 PECInsulator 2D 证据: 官方 2-rank、10 步 producer 后, 独立 contract 验证初态场全零、上边界中段 By 相对 $3.3e-3$ 输入误差 $1.54\% < 5\%$ 、下边界 By=0; 本版将其与 implicit Poynting ledger 分开归类。

2026-07-12 增补 test_3d_pec_particle 的 2-rank PEC-vs-periodic control 证据: 全场 $\max|E|$ 比值 $0.0070491 < 0.01$ 、Ey 比值 $0.0022796 < 0.01$, 两 species 粒子状态一致; 证据范围限定为近 PEC 场抑制, 不替代直接粒子 gather instrumentation。

2026-07-12 增补 particles_in_pml 2D AMR sibling: 官方 test_2d_particles_in_pml_mr 2-rank、300 步运行与独立绝对值 contract 均通过, $\max|E|=3.661057413095795e-4 < 6e-4$; 2D 两档和 3D 单层已形成正向证据, 3D AMR 的 signed-vs-absolute 判据分歧继续保留。

2026-07-12 增补 particles_in_pml 3D 证据: 单层 2-rank 官方与独立 contract 通过, $\max|E|=4.325973103094924 < 10$; 3D AMR sibling 的官方有符号 gate 通过, 但独立全场绝对值 gate 为 $110.3993781372607 > 110$, 本版将其记录为判据边界而非强通过。

2026-07-12 增补官方 test_2d_particles_in_pml 2-rank 证据: 180 步后官方 residual-field analysis 通过, 独立全场绝对值 contract 得到 $\max|E|=2.5542538436684726e-4 < 3e-4$; 2D AMR、3D 单层随后补齐, 3D AMR 的 signed-vs-absolute 判据分歧与 upstream checksum 边界继续保留。

2026-07-12 增补 RZ 三模态 case-local sibling: n_rz_azimuthal_modes=3、dump_rz_modes=1, 验证 $m=1/2$ 实虚分量非零, 且 native theta=0 Er/Ez 与实模态重建误差分别为 $3.05e-16/2.53e-16$ 。官方 analysis_rz.py 仍是单模解析消费者, 不用于该三模 sibling。

2026-07-12 又补入第 5 章 Esirkepov/Villasenor 出版级对照表: 将论文第一性对象、现代 WarpX 入口和循环骨架并排固定, 并把 1/3/1/6 的共享系数与不同算法语义区分开; 本版仍不把公式级检查升级为所有分支的端到端等价证明。

2026-07-12 又补入第 7 章 PSATD-PML Cartesian/RZ 并列运行证据: Cartesian 低反射率为 $9.4704e-7$, RZ 径向 PML 末态残余场最大值为 1.4793; 新增独立 reader-side 报告, 并明确 1-rank 项目级复现不替代官方 2-rank CMake regression。

2026-07-12 又补入 transition-zone live source audit: 在 WarpX commit 8c488b1a9 上复核 BuildBufferMasks、stablePartition、nfine_gather/nfine_deposit、Efield_cax/current_buf/rho_buf 和 SyncCurrent/SyncRho 五组锚点全部通过, 同时确认 dedicated route-count 实现仍不存在; 本版不将该 audit 升级为 runtime regression。

2026-07-12 又补入 comoving 第一阶段 finite + spike 正/负 contract: stable baseline 的 spike_ratio=1.1103719982074416 通过阈值, no-comoving reference 的 1.1119614945212388 被阈值 1.1114823702056489 拒绝; energy gate 仍不纳入本版, 因为 local calibration 尚未形成可靠 energy oracle。

2026-07-12 又补入 RZ JRhom LL2 first-stage repeated/MPI 正/负 contract: baseline energy ratio 0.9770894022295227 通过 0.9780664916317521 ceiling, no-time-averaging reference ratio 1.0 被拒绝; 该证据仍属于 project-level helper validation, 目标 WarpX checkout 仍保持 analysis=OFF。

2026-07-12 又补入 ParticleHistogram2D BP5 weighted-moment sanity: 记录 ions/electrons 在 iteration 0/100 的总权重、z/uz 均值和标准差, 并明确该 reader-side 统计不等于更高分辨率/粒子数 convergence proof。

2026-07-12 又补入 ParticleHistogram2D 匹配物理时间的网格敏感性 contract: 384x512 baseline 与 768x1024 refined producer 的总权重、std(z)、std(uz) 局部稳定性 gate 通过; 1x1 particles-per-cell 负对照因电子总权重差 $1.9471e-3 > 1e-3$ 被拒绝, 粒子数收敛仍未完成。

2026-07-12 又将 ParticleHistogram2D 粒子数敏感性扩展为 1x1/2x2/4x4 pairwise contract: 1x1 -> 2x2 电子总权重差 $1.9471e-3$ 未通过, 2x2 -> 4x4 降至 $4.2685e-4$ 并通过总权重/std(z)/std(uz) 局部 gate; 本版仍不声称正式 convergence order。2026-07-12 又补做 8x8 particles-per-cell sibling: 4x4 -> 8x8 电子总权重差进一步降至 $3.6534e-4$, 四档相邻比较的总权重、std(z)、std(uz) 局部 gate 均通过; 8x8 producer 的 BP5 输出完整, MPI finalize 尾噪声仅限本机 OFI 收尾环境; 本版仍不声称正式 convergence order 或 upstream regression gate。2026-07-12 又重新完成 comoving PSATD 第一阶段目标 checkout 只读 preflight: audit/report/preview/dry-run 均成功, 目标树仍为 unstaged, helper 缺失、CMake analysis 仍为 OFF, 精确计划改动只有新增 helper 与一处 analysis wiring; 本轮未修改 ../warpX。详细证据见 notes/code-reading/fieldsolver/34-comoving-target-checkout-preflight.md。2026-07-12 又为第 8 章新增 scripts/plot_particle_histogram2d_particle_count.py 与 assets/figures/particle-histogram2d-particle-count.{p 生成图 8-12 展示四档 ParticleHistogram2D 相邻 pairwise 结果相对局部 gate 的归一化趋势; 证据仍限定为单 case reader-side 统计。

2026-07-12 又为第 5 章补入 four/seven/ten-boundary 到 WarpX earliest-crossing segment loop 的 Mermaid 源码映射示意; 该图明确是读者侧源码映射, 不是论文原图, 不替代 publisher figure-by-figure compare. 2026-07-12 又补入 implicit Villasenor 2-rank 运行级证据: 2D JFNK shape=2 case 的 energy/Gauss-law 官方与独立 contract 均通过, 2D shape=4 boundary-cropping sibling 的 Gauss-law contract 通过; RZ/1D sibling 的 PETSc 缺失与 SIGILL 边界均如实保留. 2026-07-12 又补入 implicit Villasenor filtered sibling: 官方与独立 contract 均通过, 显式确认 warpx.use_filter=1, 能量相对变化 $3.8931\text{e-}15$ 、Gauss-law RMS $5.1401\text{e-}16$; 该证据仍限定为 2D JFNK/filter 组合路径. 2026-07-12 又补入 implicit Villasenor PICMI sibling: Python-enabled build_py 下官方 2-rank producer 与独立 contract 均通过, 生成输入确认 Villasenor/theta-implicit/shape=2, energy 相对变化 $4.0980\text{e-}15$ 、Gauss-law RMS $9.5730\text{e-}16$; 保留 newton.liner_solver unused-input warning. 2026-07-12 又进一步定位 RZ implicit Villasenor blocker: 官方 PETSc 路径因 AMREX_USE_PETSC 缺失拒绝, amrex_gmres control 又在 ThetaImplicitEM::InitializeCurlCurlBCMasks() 触发 SIGILL; RZ 未进入物理计算, 边界记录见 notes/code-reading/particles/47-rz-implicit-villasenor-build-boundary.md.

2026-07-12 又补入 Villasenor-Buneman 可执行公式级 bounded check: Eq. (6)-(9) 四边界 flux、任意 crossing 分段位移以及 Eq. (36) 三维交叉项/体积闭合在 10000 组确定性样本上通过, 二维最大残差 $4.4409\text{e-}16$ 、三维最大残差 $1.7764\text{e-}15$; 证据等级仍是 paper-backed + source-grounded + formula-audited, 不升级为所有 WarpX 分支的端到端等价证明。

2026-07-12 又重新核查 Esirkepov 2001 CPC access: 官方 Elsevier API 确认发表 PII/cover date 并报告 openaccess=0, PDF 请求返回 HTTP 406 未授权最小响应; publisher-PDF line-by-line compare 仍是权限缺口。

2026-07-12 又补入 3D PSATD-PML divergence-cleaning reader-side 对照: 原生 divE/divB 输出与关闭 PML cleaning 的 control 均为 finite, 但 core divB clean/control 比值 12.7633, 因此只作为负/边界证据, 不升级为强 cleaning physics gate. - 2026-07-12 又补入官方 3D PSATD-PML 2-rank launcher 证据: FI_PROVIDER=tcp + MPICH -n 2 成功写出 native divE/divB; 与 1-rank 对照时 rho 相对差 $9.31\text{e-}13$ 、Ex 最大相对差 6.38%, 本版只收录 MPI producer coverage, 不宣称 rank-invariant field contract. - 2026-07-12 又补入同一 2-rank 分解上的 PML cleaning/control 对照: clean/control core Gauss residual $0.764365/1.274899$, core divB 比值 12.8282, 与单进程方向一致; 本版继续关闭强 cleaning physics gate. - 2026-07-12 又补入官方 2D Cartesian PSATD-PML 2-rank 证据: 初始能量相对误差 $1.45\text{e-}15$ 、未态反射率 $9.4704449758\text{e-}7 < 1\text{e-}6$, 新增独立 reader-side contract; 仍不把本地复现写成 upstream CMake checksum 的替代品. - 2026-07-12 又补入官方 2D Cartesian PSATD-PML 2-rank restart 证据: 从 chk000150 接续到 diag1000300, 官方与独立 reader-side 对 Bx/By/Bz/Ex/Ey/Ez/divE/rho 的最大绝对/相对误差均为 0.0, 通过 $1\text{e-}12$ restart gate; 该证据只支撑状态恢复一致性. - 2026-07-12 又补入官方 uniform_plasma 3D 2-rank restart 证据: 从 chk000006 接续到 diag1000010, 官方与独立 reader-side 对 37 个 field 的最大相对误差为 $2.8631\text{e-}16 < 1\text{e-}12$; 仓库 checksum 的 rank-specific 参考与本地 2-rank producer 最大相对差 $3.20\text{e-}2$, 本版明确保留该边界, 不把 restart field pass 写成 checksum pass.

2026-07-12 增补第 8 章可复现性矩阵: 统一整理 Langmuir、uniform-plasma、FieldProbe、reduced diagnostics、ColliderRelevant、DifferentialLuminosity、ParticleHistogram2D 和 BeamRelevant 的 producer/MPI、项目级脚本、主要 gate 与 case-local 证据目录, 并显式保留 coarse failure 与 binary mismatch 边界。

2026-07-12 增补 BeamRelevant 最小 3D 统计合同: 保留官方 initial_distribution 的 beam 参数, 在 case-local sibling 中只启用 beam 与 BeamRelevant, 验证 24 列输出、截断高斯束 charge、横向 rms 和纵向截断 rms; 新增 scripts/analyze_beam_relevant_contract.py. 完整官方 input 因预编译 binary 与当前源码对 maxwellian/maxwell_boltzmann 的语义不一致而未作为通过证据。

2026-07-12 增补 ParticleHistogram2D writer 证据: 运行 laser-ion 官方 2-rank application, 官方 time-average analysis 通过; PhaseSpaceIons/PhaseSpaceElectrons 均写出 0/100 两个 BP5 openPMD iteration、 1000×1000 uz-z 网格和有限非零数据. 新增 scripts/analyze_particle_histogram2d_contract.py, 明确二维 reduced diagnostic 不走普通文本 schema, 空 .txt sidecar 是预期边界。

2026-07-12 增补 diff_lumi_diag 解析谱证据: 按官方 2-rank 配置完成 leptons、leptons+AMR 和 photons 三组 sibling, 1D/2D differential luminosity analysis 均通过; 新增 scripts/analyze_diff_lumi_contract.py, 记录 1D/2D 误差、容差、最终 step、128 个能量 bin、 128×128 openPMD 网格和 AMR level。

2026-07-12 增补 collider_relevant_diags 束流诊断合同: 按官方 2-rank 配置真实运行并通过官方 analysis.py, 确认 ColliderRelevant 的 chi/角度统计、ParticleExtrema 和 dL/dt 聚合均成立; 新增项目脚本从 openPMD rho_beam_e/rho_beam_p 独立重建 dL/dt, 与 reduced 输出的两个 iteration 均为零相对误差。

2026-07-12 增补 LoadBalanceCosts 并行诊断证据: 按官方 2-rank 配置运行 Heuristic/Timers 两个 sibling, 并复现 LBC.txt efficiency improvement gate; Heuristic 为 $0.625252 \rightarrow 1.000000$, Timers 为 $0.744780 \rightarrow 0.996162$. 新增 scripts/analyze_load_balance_costs_contract.py 和 case-local 报告。

2026-07-12 增补第 8 章 reduced diagnostics 强对照: 按官方 2-rank 配置真实运行 test_3d_reduced_diags, 用官方 analysis_reduced_diags.py 比较 60 个 reduced observable 与 diag1000200 plotfile 重算值; 全部通过, 非 field-energy 最

大相对误差 $4.125\text{e-}13$, field-energy 相对误差 $2.483\text{e-}1$, 低于官方专用 0.3 容差。

2026-07-12 增补第 8 章 restart 证据: 真实运行 uniform_plasma 3D 2-rank 基线 with chk000006 restart sibling, 均接续到第 10 步; 官方 analysis_default_restart.py 和项目脚本对最终 diag1000010 做 37 个 field 的逐字段比较, 最大相对误差为 $2.8631\text{e-}16$, 通过 $1\text{e-}12$ gate。仓库 checksum API 的 rank-specific 参考与本地 2-rank producer 最大相对差为 $3.20\text{e-}2$, 因此本版只宣称 restart field reproducibility, 不宣称 checksum 通过。

2026-07-12 又补入 uniform-plasma 1-rank/2-rank consistency 负/边界证据: 粒子总权重一致, 但 field/particle/total energy 相对差为 $1.9379\text{e-}2/8.9170\text{e-}4/6.2269\text{e-}4$, physical-field 最大 L2 相对差 1.0185; 本版关闭该 case 的 rank-invariant field contract。

2026-07-12 又将 Esirkepov Eq.(23) density-decomposition 检查扩展为可归档 contract: 固定 seed 2001、10000 组样本最大残差 $8.8818\text{e-}16 \leq 2\text{e-}15$; 本版仍把它限定为 paper-formula/algebra evidence, 不把它写成 WarpX kernel regression。

2026-07-12 又补入 Esirkepov 当前 checkout source audit: CurrentDeposition.H 的 14 个 entrypoint、shifted-shape、归一化、三方向 prefix/difference/writeback 锚点全部通过; 证据等级为 read-only source audit, 不替代数值 kernel regression。

2026-07-12 又补入 Villasenor 当前 checkout source audit: CurrentDeposition.H 的 16 个 crossing/segment/fraction/this_J* writeback 锚点全部通过; 证据等级为 read-only source audit, 不替代数值 kernel regression。

2026-07-12 增补第 8 章 FieldProbe 诊断审计: 按 WarpX 官方 2-rank CMake 配置重新运行 field_probe 2D 单缝衍射, 并与 1-rank 结果对照; 两种配置的 FP_line.txt 完全一致, 但官方 analysis.py 的平均误差为 3.6703%, 超过 2.5% gate。新增 scripts/analyze_field_probe_diffraction.py 和失败报告, 当前不把该案例写成通过的 physics benchmark。

随后做了 FieldProbe 分辨率对照: 将网格从 $\lambda/16$ 加密到 $\lambda/32$, 并把取样步从 500 调到 1000 以保持相同物理时间; 官方口径误差降至 0.3533%, 最大选点误差 1.0414%, 通过 2.5% gate。该结果支持 coarse-grid 离散误差是原始失败的主因, 不能把 refined case 的通过反写成原始官方 coarse case 已通过。

2026-07-11 又推进了第 8 章的 reader-side 验证: 在项目内生成逐步诊断的 Langmuir 本地副本, 得到 81 个 plotfile 快照, 并用新增的 scripts/analyze_langmuir_frequency_fit.py 拟合目标模式频率; 同时用 scripts/analyze_uniform_plasma_conservation.py 对 uniform-plasma 初末状态做粒子数、粒子权重、场能、粒子动能和总能量统计, 并将 uniform plasma 延长到 100 步、形成 11 帧时间序列。随后真实运行 energy_conserving_thermal_plasma 1D/2D 两个 sibling 各 500 步并复现官方 reduced-energy 强 analysis, 1D/2D 最大 EF+EP 漂移分别为 $3.009\text{e-}4$ 和 $1.031\text{e-}4$, 均小于 $3.000\text{e-}3$ 。结果已归档到 runs/stage-c-validation/, 其中 Langmuir 的频率相对误差为 $3.595\text{e-}4$, uniform plasma 的粒子权重全程不变但长序列总能量末态变化 $1.387\text{e-}2$ 、最大绝对偏差 $2.518\text{e-}2$, 所以后者仍是 reader-side 统计, 而 thermal-plasma family 结果才是本轮可升级的强能量 gate。

定位

v0.40 是 PIC-tutor 的 deposition runtime-contract closure 版。它继承 v0.39 已经完成的 deposition paper-backed assets 基线, 但这次不再把第 5 章停在“已有论文资产、正文仍需继续从 notes 吸收实现合同”的状态, 而是进一步把 SyncCurrentAndRho()、boundary/source synchronization、Langmuir + current_correction 与 vay_deposition 的 regression 分工, 以及 Esirkepov / Villasenor 在公式到 kernel 之间的几层关键 runtime contract 更直接地压回正文。当前模块的闭合点已经从“主文献到位”推进成“current deposition 主线的源码、论文与验证分层已经能在正文内部自治闭合”。

本版仍不修改 ../warpX。新增的核心事实是: 第 5 章不再只是在源码层说明 Direct / Esirkepov / Villasenor / Vay 四条路径各自做什么, 而是把 source-synchronization 的顶层分叉、fine/coarse 同步、PEC source-boundary、specialized current_fp_vay 路径、以及两条关键 regression 分别验证哪一层 contract 这些原先更依赖配套 notes 才完整成立的实现边界, 也收进了正文。同时, Esirkepov Eq.(23) 与 Villasenor Eq.(6)-(9) 在 WarpX 当前 prefix-loop / segment-loop 结构里的程序化对应也进一步压细, 从而把第 5 章从“paper-backed + source-backed”推进到更接近“runtime-backed”的可审读状态。

在当前这轮 v0.40 后续推进里, 第 5 章又往前压了一小步: Villasenor Eq.(6)-(9) 不再只停在“Jx/Jz 可读成主方向输运乘横向平均”这种总括说法, 而是补清了 Jx1/Jx2 与 Jy1/Jy2 的镜像角色; 同时, 论文 Eq.(36) 那组四项 x-face contribution 与现代 3D kernel 的 old*old / old*new / new*old / new*new 混合平均之间, 也已经从“存在对应”推进到“可以解释局部耦合怎样分配到四个横向角点”的层次。这还不等于完成逐项核对, 但它确实把 Villasenor 这条线从抽象结构映射推进到了更贴近论文公式本体的成书密度。

与此同时, charge deposition 这条线也补上了一层此前仍容易混淆的时间语义: 现在正文已经明确区分 MultiParticleContainer::DepositCharge(rela 的外层 PushX() 粒子位置平移, 和 WarpXParticleContainer::DepositCharge(..., icomp, ...) 里 time_shift_delta -> LowerCorner(...) 的内层 Galilean 参考框架校正; 并且补回了独立 DepositCharge() 在 RZ / RCYLINDER / RSPHERE 下还会在 species 汇总后统一做 ApplyInverseVolumeScalingToChargeDensity(...)。这使第 5 章对 charge deposition 的说明不再只是 bridge contract 摘要, 而是开始把“什么时候改粒子位置、什么时候改网格参考原点、什么时候再乘几何体积因子倒数”讲成一条完整实现链。

这一轮继续往前压以后, charge deposition 的 shared-memory 特化路径也不再只是“知道有一条单独分支”。正文现在已经明确写清: do_shared_mem_charge_deposition 不走 ABLASTR deposit_charge(...), 而是先把粒子组织成 DenseBins +

shared_tilesize + max_tbox_size 的 tile-binned 执行合同, 再进入 doChargeDepositionSharedShapeN<1..4>(); 与此同时, max_tbox_size -> sample_tbox -> shared_mem_bytes <= sharedMemPerBlock 这条 GPU shared-memory 容量检查也被压回了主文。这样第 5 章对 charge deposition 的覆盖开始同时包含普通 bridge contract 与 shared-memory 特化执行拓扑, 而不再只偏向普通路径。

再往下走一步, 这条线的低维几何语义也开始从“隐含在源码条件编译里”变成正文里的显式事实: ChargeDeposition.H 在 1D_Z / RCYLINDER / RSPHERE 下并不是简单沿用 3D 多维 shape 再少几层循环, 而是先把粒子位置压成轴向或径向单变量, 再只构造一维 sz 或 sx support; shared-memory 版本在这件事上与普通版本保持同构, 改变的是执行拓扑, 不改变这些几何分支本身的物理语义。这样第 5 章对 charge deposition 的描述就不再只覆盖时间层、bridge contract 和 shared-memory 拓扑, 也开始覆盖低维 kernel 如何直接重写几何变量。

与此同时, DepositCharge() 的容器层接口边界也比之前更清楚了。正文现在已经把 reset / local / apply_boundary_and_scale_volume / interpolate_across_levels 四个开关分别钉回源码职责: 它们控制的是清零、guard-cell 通信、沉积后几何/边界整理, 以及多层 fine -> coarse average-down, 而不是粒子到网格的局部 shape kernel 本身。这样第 5 章对 charge deposition 的叙述开始明确区分“局部 rho 怎么写出来”和“最终可供 Maxwell/diagnostics 使用的 rho 怎么经通信、边界和 AMR 整理完成”。

再推进一步以后, charge deposition 与 coarse-buffer/AMR 邻接的分层也更清楚了。正文现在已经明确写清: PartitionParticlesInBuffers(...) 会先把粒子切成 fine rho_fp 与 coarse-buffer rho_buf 两段, 而 buffer 粒子不是先沉到 fine 再 restrict, 而是一开始就通过 DepositCharge(..., depos_lev = lev-1) 直接按 coarse patch 几何沉积; 相对地, 多层接口里的 interpolate_across_levels 只是函数尾的 average-down 选项, 运行时真正的 coarse/fine/buffer source 整理仍要依赖后面的 SyncRho() / AddRhoFromFineLevelandSumBound 这样第 5 章开始把“直接沉积在哪一层几何上”和“沉积完成后怎样做 coarse/fine/source synchronization”明确拆成两层。

这一部分现在已经补上最后一层负面边界: ChargeDeposition.H 不是 current-deposition mover, 它不恢复一步轨迹、不构造 old/new shape difference, 也不选择 Esirkepov / Villasenor / Direct / Vay. DepositCharge() 这一支现在在正文里被收束为“单时间层 ρ 采样 + 时间层/AMR/几何桥接 + 外层同步整理”的合同: icode/time_shift_delta 负责 old/new 网格参考原点, relative_time PushX 负责外层粒子位置取样, depos_lev=lev-1 负责 coarse-buffer 直接沉积, shared-memory 只改变执行拓扑, 而 PEC、guard exchange、inverse-volume scaling 与 AMR average-down 都留在容器/通信层。这使 DepositCharge() / ABLASTR / ChargeDeposition.H 的主线达到 v0.40 内部阶段闭合。

在 Esirkepov 2001 这条线上, 当前回合也把正文从“Eq. (23) 的局部公式映射”往“论文结构映射”推进了一步: 本地预印本已经足以确认预印本标题与 CPC 发表版标题并不完全相同, 也足以把预印本 Section 2/3/4 分别稳定对回 WarpX 的 continuity contract、density decomposition 三方向分解和二阶 spline 算法骨架。这样第 5 章对 Esirkepov 的依赖不再只是单条公式或单句唯一性 claim, 而开始具备更完整的问题设定到 kernel skeleton 的 paper-backed 结构支撑。

2026-07-11 又对这条线做了一次 bounded compare: arXiv 页面核实了预印本提交日期、题名、13 页/无图/10 条参考文献等元信息, 公开 CPC 书目信息核实了发表版题名、卷期、页码和 DOI, 并将结果独立记录到 notes/code-reading/particles/44-esirkepov-cpc-bounded-comparison 这一步把 publication metadata 从“已知但散落在 access audit”提升为可直接引用的项目资产; 但由于当前环境仍拿不到 publisher-formatted PDF, abstract、section numbering、Eq. (23) 排版和二阶 spline 段落的逐页 compare 仍保持未完成状态。

本轮又对 ScienceDirect 的直接 PDF 地址做了 browser-like 下载复核: 搜索结果可发现 publisher endpoint, 但浏览器侧进入 download-preparation error, 本机 curl -L 返回 HTTP 403。因此第 5 章新增了证据矩阵, 把 Section 2/3/4 的预印本全文与 WarpX 源码映射、CPC 书目/摘要元数据和仍待 publisher PDF 的四项正文 compare 分开列出; 这提升了成书表达的可审计性, 但不改变“publisher-PDF line-by-line compare 未完成”的状态。

在同一轮 publication-grade 精修中, 第 5 章又新增了五条路径的责任边界矩阵: Direct、Esirkepov、Villasenor、Vay 和 DepositCharge() 分别按第一性对象、离散连续性目标、轨迹处理、WarpX 入口和外层职责并排对照。这个矩阵不是新的算法断言, 而是把已经在各分节分别成立的 runtime contract 压成一个可复查的导航表, 并同步修正 5.15 的剩余工作描述: charge deposition bridge 不再作为主要缺口, 后续优先级回到论文证据、图表和出版级文字精修。

随后又把 Villasenor 的 crossing 语义图示化: 正文新增 Mermaid 流程图和 four-/seven-/ten-boundary 对照表, 把论文中的固定几何命名与 WarpX 当前 cell_crossings_* -> num_segments -> earliest-crossing -> local this_J* 的动态循环分开。图明确表达了一个重要边界: seven-/ten-boundary 不是现代源码中的固定 case label, 而是 repeated segmentation 可能产生的论文级几何结果; 源码真正的扩展性来自 crossing 计数和 segment loop。

在同一轮继续推进第 6 章后, 又新增了 PSATD/场求解器算法族决策矩阵。矩阵把 FDTD、Cartesian standard/Galilean/comoving、JRhomb、RZ standard/Galilean 与 PML 按谱基、源项时间模型、沉积/组合限制、系数族和当前验证证据并排组织, 特别固定了两条防混写规则: Cartesian Galilean/average-field 的同名 Y 系数不能与 JRhomb Y1-Y8 合并; RZ Ep/Em 也不能当作 Cartesian Ex/Ey 的简单别名。同时, comoving 和 RZ JRhomb 的本地 helper/ledger 仍被明确写成 local validation 或 handoff 证据, 不提前升级为 upstream 强物理 regression。

随后对 RZ JRhomb first-stage bundle 的目标 checkout 做了只读 audit/report/preview: 当前 ../warpx 为 unstaged, helper 文件缺失, CMake analysis 仍为 OFF, 精确 preview diff 只包含新增 analysis_rz_jrhomb.py 和一行 analysis wiring。该

结果写入 `notes/code-reading/fieldsolver/rz_jrhom_first_stage_target_report.md`, 把“bundle 已准备好”和“已经写入 WarpX”明确拆开; 本轮仍不修改 `../warpx`。

第 7 章也新增了 transition-zone 当前证据状态表: `BuildBufferMasks()`、`PartitionParticlesInBuffers()`、`E/Bfield_aux/cax` 和 `rho/current_fp/buf` 的源码合同已核对, 现有 MR/Langmuir/PML regression 属于间接整体证据, 而 `TransitionZoneRoutes/route-count` dedicated test 仍停留在 v0.15/v0.16 的 WarpX patch 设计, 目标 checkout 中尚未实现。由此把第 7 章的准确边界固定为“源码已核、间接验证已核、专门 route proof 待实现”。

同一轮还完成了 LeeCPC2015 accepted-manuscript 的第一轮 materialization: eScholarship 49m2k3vj 提供的 7 页 PDF 已进入论文专属目录, 并经 MinerU 生成 Markdown、13 张图片和论文顺序中文讲解; 第 7 章现在可以在 accepted-manuscript-backed + source-grounded 证据等级下解释 PML split-field、PSTD staggered shift、反射系数递推和高阶/PSTD 反射率结论。由于本地版本不是 publisher-formatted CPC PDF, C1-C25、Galilean T2、cleaning F/G 和 RZ PML 仍保持 WarpX 实现侧/专用扩展边界, 不能直接归因给论文。

本版仍不是出版终稿。它继续保留 Esirkepov 2001 的 CPC 定稿 PDF 对照, 也继续保留第 5 章后续 publication-grade 精修的现实边界。第 6 章 upstream handoff 和第 7 章 Lee/Vay 论文闭环同样未结束。后续应优先补齐 Esirkepov 2001 的发表版对照, 并在不破坏当前主线闭合的前提下继续压缩第 5 章冗余、补图表和逐式核对, 再考虑切往下一个成书主模块。

源码基线

- 本书项目仓库: `/Volumes/PHILIPS/programs/PIC/PIC-tutor`
- WarpX 只读源码: `../warpx`
- 当前 WarpX 分支: `pkuHEDPbranch`
- 当前 WarpX commit: `8c488b1a9`

v0.40 只修改 PIC-tutor 书稿项目, 不修改 `../warpx` 原仓库。本版继续以 v0.39 已核定的第 5 章 paper-backed 资产为基础, 推进 `manuscript/chapters/05-deposition-shapes.md`、`README.md`、`TODO.md`、`manuscript/README.md` 与版本说明的统一收口, 并重建当前合订稿。后续若 WarpX 更新, 必须重新校准源码行号、沉积入口和 regression 锚点后再发布新版。

v0.40 章节范围

章节	文件	v0.40 状态	下一步缺口
写作说明	<code>chapters/00-preface.md</code>	沿用 v0.1	需同步最终出版路线
动理学模型	<code>chapters/01-kinetic-models.md</code>	v0.40 增补连续模型到 PIC 离散变量桥	Hockney-Eastwood、Yee 等一手文献闭环未完成; 仍需继续压紧与第 2 章的 leapfrog/CFL/色散衔接
PIC 总循环	<code>chapters/02-pic-loop.md</code>	v0.40 增补 AMR subcycling、JRhom 与 implicit 时间合同	仍需把基础文献和公式变量定义做出版级补齐, 并继续压紧与第 3 章的调用链衔接
WarpX 主演化路径	<code>chapters/03-warpx-evolve.md</code>	v0.40 增补 <code>OneStep_sub1()</code> 、JRhom、implicit 主调用链及 nonlinear/JFNK/mass-matrix J 构造边界	更细的 mass-matrix kernel 和场算法公式仍需分章展开
WarpX 初始化链	<code>chapters/03a-warpx-initialization.md</code>	已做 v0.20 草稿收束; v0.40 补入 native external-file Gaussian beam 的 1-rank 独立束斑合同	需要拆短小节、补流程图、压缩过长审计段落; 官方 native <code>analysis.py</code> 缺失和完整 <code>initial_distribution</code> binary mismatch 仍需保留边界
粒子推进器	<code>chapters/04-particle-pusher.md</code>	v0.40 增补 Higuera-Cary 运行级合同报告	仍需压缩多物理长段, 并把 Boris/Vay 对照和更多 validation 表格图形化

章节	文件	v0.40 状态	下一步缺口
沉积与形函数	chapters/05-deposition-shapes	把 current deposition 的 runtime/source-validation contract 进一步压回正文, 并把 DepositCharge() / ABLASTR / ChargeDeposition.H 的 ordinary/shared-memory、time-level、low-dimensional geometry、coarse-buffer 与外层同步边界阶段性收口; Villasenor-Buneman 已完成论文公式级审计, 并补入 2D implicit JFNK 的普通、filtered、shape=4 cropping 与 PICMI 运行级证据; RZ charge/inverse-volume 已有官方场/能量与独立全域电荷闭合证据	仍需为 Esirkepov 2001 补齐 CPC 定稿对照, 并继续做记号统一、出版级表格精修和现代 geometry/order 分支逐项核对
场求解器	chapters/06-field-solvers	Lehe/Kirchen/Godfrey 文献闭环、PSATD/NCI 源码机制对照表, 并在 v0.26-v0.36 连续收口 comoving 与 RZ validation; v0.40 新增 1D semi-implicit/theta-implicit Picard sibling 运行级总能量守恒、comoving 正/负 spike contract 和 RZ JRhom 2-rank 正/负 energy contract	仍需决定是否真正在目标 WarpX checkout 上 staging 并上提 RZ JRhom helper; 更细 solver family 仍需继续扩展
边界、PML 与 AMR	chapters/07-boundaries-amr	在 v0.25 增补 LeeCPC2015 论文-源码公式核对清单, 并在 v0.40 完成 eScholarship accepted/submitted manuscript 的 MinerU、中文讲解和源码映射	仍需取得 publisher-formatted CPC PDF 做版本对照, 并实现 dedicated transition-zone regression

章节	文件	v0.40 状态	下一步缺口
诊断、验证与案例	chapters/08-diagnostics-cases.md	完成 reader-side、reduced diagnostics、多案例 physics/writer/performance gates、统一复现矩阵，并嵌入 Langmuir、energy-conserving thermal plasma、FieldProbe、reduced diagnostics、DifferentialLuminosity、LoadBalanceCosts、ColliderRelevant、ParticleHistogram2D 和 BeamRelevant 九组真实验证图；另补 native Gaussian external-file 项目级束斑 gate，并补齐 uniform-plasma 3D 2-rank restart field reproducibility	仍需补更多可视化图表、完整 initial_distribution binary 匹配复现、ParticleHistogram2D 物理收敛性和更多边界/应用案例
文献路线	chapters/09-literature-roadmap.md	roadmap 路线图，并把第 5 章最关键两篇 deposition 主文献更新到 paper-backed 状态	仍需继续消化 docs/literature-map.md 中可并入正文的旧条目，并推动下一批优先论文 materialize
符号表	appendices/A-symbols.md	v0.40 已扩展为连续模型、离散时间层、源码变量、诊断术语和缩写速查表	后续随新章节继续补充专用符号和单位

v0.40 已完成的增量

- 冻结 manuscript/VERSION-v0.39.md，避免重建 v0.39 时误用 v0.40 版本说明。
- 新增 scripts/build_v40.py，生成 dist/pic-tutor-v0.40.md、自包含 MathJax 的 dist/pic-tutor-v0.40.html 与 dist/pic-tutor-v0.40.pdf；该条记录对应 279 页历史构建快照，当前页数由 scripts/verify_v40_build.py 固定验收，HTML 数学转换警告为 0。
- 把 SyncCurrentAndRho() 的 solver/algorithm 分叉、current_fp_vay 专用路径、fine/coarse source synchronization、PEC source-boundary 和两条关键 regression 的 contract 分层直接收回第 5 章主文，减少这部分解释对配套 notes 的依赖。
- 继续压细 Esirkepov Eq.(23) 与 Villasenor Eq.(6)-(9) 在 WarpX 当前 prefix-loop / segment-loop 结构中的 programmatic mapping，使论文公式、源码循环与 regression 语义三者在正文中能够直接互相对照。
- 继续把 DepositCharge() / ABLASTR / ChargeDeposition.H 的 bridge contract 压回第 5 章主文，补齐 ordinary/shared-memory、time-level、low-dimensional geometry、coarse-buffer 和外层同步职责边界。
- 新增第 6 章 PSATD/场求解器算法族决策矩阵，统一导航 FDTD、Cartesian standard/Galilean/comoving、JRhom、RZ standard/Galilean 与 PML 的谱基、源项时间模型、组合限制、系数族和验证证据等级，明确同名系数与 checksum 证据的边界。
- 第 8 章完成 Langmuir、uniform-plasma、FieldProbe、reduced diagnostics、ColliderRelevant、DifferentialLuminosity、ParticleHistogram2D 和 BeamRelevant 的运行级证据整理，新增项目级分析脚本、case-local JSON/Markdown 报告和统一复现矩阵；保留 coarse-grid physics failure、writer-only contract、binary mismatch 等边界。
- 第 1 章新增“从连续模型到 PIC 离散变量”桥接小节，将 $f/\rho/J/E/B$ 映射到粒子时间层、网格字段、fine/coarse source buffer 与 SyncCurrentAndRho()，并明确 DepositCharge()、current deposition、source synchronization 的三层职责。
- 第 2 章新增 AMR subcycling 时间合同，基于 OneStep_sub1() 固定两级/比例 2、细层两次推进、粗层一次推进、fine-to-coarse current/rho 合成、guard/auxiliary 可见性和 electrostatic 禁止组合边界。
- 第 2 章新增标准显式、PSATD-JRhom 与 implicit 时间推进对照，明确 OneStep_JRhom() 的多次相对时间沉积、m_JRhom_subintervals/time averaging、implicit PreRHSOp() 的粒子/source 重算，以及 current_correction、collision split push 和 RHS/物理时间步的边界。
- 第 3 章新增 implicit 主调用链：从 WarpX::OneStep() 进入 SemiImplicitEM::OneStep()、m_nlsolver->Solve()、ComputeRHS()、PreRHSOp()、CumulateJ()/mass matrices、SyncCurrentAndRho() 到最终粒子和磁场提交，明确 non-linear iteration 不等于额外物理时间步。
- 第 3 章新增 implicit nonlinear/JFNK/mass-matrix 合同：将 picard/newton/petsc_snes、J_suborbit + J0 +

- M.deltaE、CumulateJ()、ComputeJfromMassMatrices()、staggering offset 以及 3D/RSPHERE 限制接回源码。
- 第 4 章新增官方 particle_pusher Higuera-Cary 运行级证据: 10000 步末态 $\max|x| = 1.1430664e-4 < 1e-3$, 新增 scripts/analyze_particle_pusher_contract.py 与 case-local JSON/Markdown 报告, 并明确该证据不等价于三种 pusher 的完整 benchmark; 同时新增 single_particle velocity synchronization 运行级证据, u_z 相对误差为 $1.3237889e-16 < 1e-15$, 完成 Boris/Vay/Higuera-Cary pusher-only sibling 对照, 补入 photon_pusher 16-species 无质量传播/动量保持证据, 并完成 larmor continuum orbit audit 但保留 checksum-only 边界。
- 第 8 章新增第一张真实验证图 assets/figures/langmuir-field-vs-theory.png, 由 Langmuir 81 快照验证树生成, 并在正文中明确区分波形图的 reader-side sanity check 与数值 gate。
- 第 8 章新增 scripts/plot_energy_conserving_thermal_plasma.py 和 assets/figures/energy-conserving-thermal-plasma.png, 由 1D/2D case-local JSON 可重建 EF+EP 总能量/相对漂移图, 并明确共同 0.003 gate。
- 第 8 章新增 scripts/plot_field_probe_resolution.py 和 assets/figures/field-probe-resolution-comparison.png, 由 resolution comparison JSON 可重建 coarse/refined 平均误差与最大选点误差图, 并保留 2.5% physics gate、coarse failure 和 refined pass 的边界。
- 第 8 章新增 scripts/plot_reduced_diags_error_layers.py 和 assets/figures/reduced-diags-error-layers.png, 由 reduced-diags contract JSON 可重建普通 observable 与 field-energy 特殊容差的误差分层图, 并保留 $1e-12 / 0.3$ 两组 gate。
- 第 8 章新增 scripts/plot_diff_lumi_errors.py 和 assets/figures/diff-lumi-errors.png, 由三份 differential-luminosity contract JSON 可重建 1D/2D 误差与 case-specific gate 图, 并保留 photons 独立容差。
- 第 8 章新增 scripts/plot_load_balance_efficiency.py 和 assets/figures/load-balance-efficiency.png, 由 Heuristic/Timers 两份 LoadBalanceCosts JSON 可重建 before/after rank-level efficiency 图, 并明确其属于性能 gate 而非物理精度 gate。
- 第 8 章新增 scripts/plot_collider_luminosity_consistency.py 和 assets/figures/collider-dldt-consistency.png, 由 ColliderRelevant contract JSON 可重建两个 openPMD iteration 的 dL/dt reduced/full-state consistency 图, 并保留“聚合定义一致性, 不等于动力学 benchmark”的边界。
- 第 8 章新增 scripts/plot_particle_histogram2d.py 和 assets/figures/particle-histogram2d-phase-space.png, 从 laser-ion BP5 series 读取真实 PhaseSpaceIons/PhaseSpaceElectrons iteration 0/100, 生成 u_z - z 相空间图, 并保留独立颜色归一化、1000×1000 writer 和空文本 sidecar 边界。
- 第 8 章新增 scripts/plot_beam_relevant_contract.py 和 assets/figures/beam-relevant-contract.png, 由 BeamRelevant contract JSON 可重建截断高斯束 charge 比值与 $x/y/z$ rms 对照图, 并保留初始化-only 和完整 binary mismatch 边界。
- 构建链修复 12 张图表的资源路径: 章节源文件使用相对 manuscript/assets/figures/, 合订 Markdown 在构建时映射到项目根 manuscript/assets/figures/, 不再依赖本机绝对路径。
- scripts/build_v40.py 的 Pandoc/ XeLaTeX 子进程固定在项目根 $cwd=ROOT$; 已从项目外 /tmp cwd 实测重建, 保证绝对脚本调用不会因调用方目录变化而丢失图表资源。
- 新增 scripts/verify_v40_build.py 作为构建后验入口, 统一检查 PDF 页数/关键章节、12 张图表链接与 HTML 内嵌资源、图 8-1 至图 8-12、附录 A 和构建警告; 当前 8 项断言全部通过。
- 更新 README.md、TODO.md、manuscript/README.md 与版本说明, 把第 5 章当前状态统一标记为“deposition runtime-contract 已阶段性收口, 后续重点切回 CPC 定稿对照、图表化和 publication-grade 精修”。

成书前必须补齐

- 每章记录最终采用的 WarpX commit, 并避免同章混用未说明的历史行号。
- 每章补齐公式变量定义、参数入口、源码路径、行号和真实源码块。
- 至少为核心章节绑定一个 Example 或 Regression。
- 把 docs/parameter-map.md 和 docs/example-regression-map.md 中的资料条目回填为正文叙述。
- 按 docs/paper-reading-workflow.md 完成核心论文 MinerU 转换和中文讲解笔记。
- 建立稳定的 Markdown/HTML/PDF 构建流程; v0.40 已在本机 Pandoc + XeLaTeX + CJK 字体环境生成并审阅 PDF, 其他环境仍需满足对应工具链条件。
- 继续清理历史 TeX 数学标记和 XeLaTeX 字体回退警告; 当前 HTML 已通过嵌入式 MathJax 消除转换警告, PDF 中普通段落误用 $\backslash\rho$ 导致的两条赤字提示也已修复, 当前构建日志无数学转换或赤字警告。
- 对 public GitHub 仓库中的 PDF 和运行产物做版权与体积审计。
- 已新增 docs/public-repo-release-audit.md 记录当前体积和发布边界; 第三方论文 PDF 的逐项公开许可确认仍未完成。

v0.40 构建方式

生成合订 Markdown、HTML 预览和 PDF:


```
python scripts/build_v40.py
```

构建后运行产物验收:

```
python scripts/verify_v40_build.py --build-log /tmp/pic-tutor-build-v40.log
```

生成的文件:

- dist/pic-tutor-v0.40.md
- dist/pic-tutor-v0.40.html (若本机存在 pandoc)
- dist/pic-tutor-v0.40.pdf (若本机存在 pandoc、xelatex 和可用 CJK 字体)

0. 写作说明

本书不是 WarpX 官方文档的翻译, 也不是只讲公式的 PIC 理论笔记。它的主线是: 先从 Vlasov-Maxwell / Vlasov-Poisson 这类连续模型出发, 说明为什么需要宏粒子; 再把宏粒子、网格、形函数、沉积和场求解拼成 PIC 算法; 最后回到本机 `../warpx` 的真实源码, 解释一个现代高性能 PIC 程序如何把这些步骤组织成可运行、可扩展、可验证的模拟软件。

当前书稿绑定的源码状态是:

- WarpX 分支: pkuHEDPbranch
- WarpX commit: 063f8b586f04321e13150ae3e730e0794ca75cb1
- 主要源码入口: `../warpx/Source/`
- 官方文档入口: `../warpx/Docs/source/`
- 示例入口: `../warpx/Examples/`
- regression 入口: `../warpx/Regression/`

本书的每个技术判断都应尽量落到六类证据: 物理方程、离散公式、WarpX 源码路径、输入参数、示例或测试、文献。DeepWiki、Zread 等 AI 解读页面可以用来快速找到模块名, 但不能作为最终依据。

本版先采用 Markdown-first 写法。这样做的原因是正文、源码路径、公式、后续 MinerU 论文笔记和本地运行记录都可以在同一个目录中增量维护。等样章和主线稳定后, 再迁移到 Quarto 或 LaTeX book。

本书默认使用以下记号: 粒子位置为

$$\mathbf{x}_p$$

, 粒子动量为

$$\mathbf{u}_p = \gamma \mathbf{v}_p$$

, 电磁场为

$$\mathbf{E}, \mathbf{B}$$

, 电荷和电流密度为

$$\rho, \mathbf{J}$$

, 粒子权重为

$$w_p$$

, 形函数为

$$S$$

。网格量的上标表示时间层, 例如

$$\mathbf{B}^{n+1/2}$$

; 粒子量一般按 leapfrog 交错在位置和动量时间层上。

阅读建议: 先读第 1-3 章建立“物理-算法-代码调用链”的整体图, 再读第 4-7 章理解各个核心模块, 最后用第 8 章的 Langmuir wave 和 uniform plasma 案例检查自己是否真正能把输入参数、源码和输出诊断连起来。

1. 动理学模型与 PIC 的基本思想

PIC 代码不是先有“粒子数组”和“场数组”，再去给它们找物理意义。它的上游模型是动理学方程：对每个物种，真实对象首先是相空间分布函数

$$f_s(\mathbf{x}, \mathbf{p}, t),$$

而不是单个粒子轨道。本章的任务是把这条连续模型主线压实到后面源码会反复用到的几个边界：

1. Vlasov 方程本质上是相空间守恒定律，而不只是“一个偏微分方程”。
2. Vlasov-Maxwell 与 Vlasov-Poisson 不是两套无关模型，而是同一条自洽场闭合在不同物理极限下的分支。
3. 宏粒子、权重和 shape factor 不是数值技巧附会到物理上，而是 coarse-grained kinetic model 的一部分。
4. PIC 的主要误差不是单一来源，而是采样噪声、有限粒子权重、有限网格和离散时间层共同作用的结果。

本章当前主要回链到：

- Birdsall 1985
- Dawson 1983
- 本地已读的 WarpX 主循环与沉积/场求解章节

其中 Hockney-Eastwood 与 Yee 1966 仍在 acquisition 阶段，当前不拿它们的未核实细节当正文证据。

1.1 Vlasov 方程首先是相空间守恒律

对物种 s ，若忽略碰撞、衰变、电离和其他 source/sink，分布函数满足无碰撞相对论 Vlasov 方程

$$\frac{\partial f_s}{\partial t} + \dot{\mathbf{x}} \cdot \nabla_{\mathbf{x}} f_s + \dot{\mathbf{p}} \cdot \nabla_{\mathbf{p}} f_s = 0.$$

对带电粒子，

$$\dot{\mathbf{x}} = \mathbf{v}, \quad \dot{\mathbf{p}} = q_s (\mathbf{E} + \mathbf{v} \times \mathbf{B}),$$

因此可写成更常见的形式

$$\frac{\partial f_s}{\partial t} + \mathbf{v} \cdot \nabla_{\mathbf{x}} f_s + q_s (\mathbf{E} + \mathbf{v} \times \mathbf{B}) \cdot \nabla_{\mathbf{p}} f_s = 0.$$

这里真正需要记住的不是式子本身，而是它的守恒含义。把相空间速度记成

$$\mathbf{Z} = (\mathbf{x}, \mathbf{p}), \quad \dot{\mathbf{Z}} = (\dot{\mathbf{x}}, \dot{\mathbf{p}}),$$

则 Vlasov 方程也可写成

$$\frac{\partial f_s}{\partial t} + \nabla_{\mathbf{Z}} \cdot (f_s \dot{\mathbf{Z}}) = 0.$$

若相空间流满足

$$\nabla_{\mathbf{Z}} \cdot \dot{\mathbf{Z}} = 0,$$

就得到 Liouville 图像：沿特征线传播时，分布函数值保持不变，相空间体积也不被压缩或膨胀。对后面的 PIC 来说，这一点比公式本身更基础，因为它解释了为什么：

- 粒子推进器不能随意破坏轨道拓扑；
- 离散时间推进若不尊重相空间结构，就会把数值伪耗散或伪加热写进分布函数；
- Boris / Vay / Higuera-Cary 这类 pusher 的比较，不只是在比轨道误差，而是在比它们怎样离散化相空间流。

1.2 碰撞项不是 Vlasov 的一部分，但必须保留边界

一旦考虑碰撞、衰变、外部源项或粒子数变化，更一般的 kinetic equation 应写成

$$\frac{\partial f_s}{\partial t} + \mathbf{v} \cdot \nabla_{\mathbf{x}} f_s + q_s (\mathbf{E} + \mathbf{v} \times \mathbf{B}) \cdot \nabla_{\mathbf{p}} f_s = C_s[f] + S_s - L_s.$$

这里：

- $C_s[f]$ 表示碰撞算子，可以是 Boltzmann、Landau、Fokker-Planck 或 Monte-Carlo 近似；
- S_s 表示外加源项，如电离生成、注入、衰变产物；
- L_s 表示损失项，如吸收、衰变消失、边界流出。

这条边界在 WarpX 里很重要，因为：

- 主 PIC loop 默认走的是无碰撞 Vlasov-Maxwell 主线；
- CollisionHandler、field ionization、QED、ContinuousFluxInjection、粒子边界吸收/反射，都是往这条无碰撞主线上附加 C/S/L；
- 它们不该被写成“另一个独立程序”，而应被理解成对同一 kinetic balance 的修改。

因此本书后面凡是讲 collisions、ionization、QED、scraping，都应问两个问题：

1. 它改的是分布函数右端的哪一项？
2. 它是在一个时间步的哪个时间层插入进去？

否则很容易把“物理过程存在”和“离散时间组织正确”混成一件事。

1.3 Vlasov-Maxwell：源项、约束与闭合

相对论动量与速度满足

$$\mathbf{p} = \gamma m_s \mathbf{v}, \quad \gamma = \sqrt{1 + \frac{|\mathbf{p}|^2}{m_s^2 c^2}}, \quad \mathbf{v} = \frac{\mathbf{p}}{\gamma m_s}.$$

分布函数的低阶矩给出 Maxwell 方程右端的源项：

$$\rho(\mathbf{x}, t) = \sum_s q_s \int f_s(\mathbf{x}, \mathbf{p}, t) d\mathbf{p},$$

$$\mathbf{J}(\mathbf{x}, t) = \sum_s q_s \int \mathbf{v}(\mathbf{p}) f_s(\mathbf{x}, \mathbf{p}, t) d\mathbf{p}.$$

然后场满足

$$\nabla \cdot \mathbf{E} = \frac{\rho}{\epsilon_0}, \quad \nabla \cdot \mathbf{B} = 0,$$

$$\nabla \times \mathbf{E} = -\frac{\partial \mathbf{B}}{\partial t}, \quad \nabla \times \mathbf{B} = \mu_0 \mathbf{J} + \frac{1}{c^2} \frac{\partial \mathbf{E}}{\partial t}.$$

这四式里最容易被误写的是：Gauss 定律和 $\nabla \cdot \mathbf{B} = 0$ 不是“额外条件”，而是系统闭合的一部分。只要连续性方程

$$\frac{\partial \rho}{\partial t} + \nabla \cdot \mathbf{J} = 0$$

成立，并且初值满足约束方程，Maxwell 演化就会传播这些约束。反过来，如果离散沉积和离散场推进不一致，那么程序里最先坏掉的往往不是 curl 更新，而是：

- `divE-rho/epsilon0`
- `divB`

- 边界附近的伪电荷
- 以及随之而来的非物理电场和数值加热

这正是后面第 5 章和第 6 章为什么要反复围着 source synchronization、current correction、Gauss-law regression 打转。

1.4 Vlasov-Maxwell 的能量与动量守恒边界

在闭域、无外源、忽略边界通量时，连续系统满足总能量守恒：

$$\frac{d}{dt} \left[\sum_s \int \gamma m_s c^2 f_s d\mathbf{x} d\mathbf{p} + \int \left(\frac{\epsilon_0}{2} |\mathbf{E}|^2 + \frac{1}{2\mu_0} |\mathbf{B}|^2 \right) d\mathbf{x} \right] = 0.$$

它说明两件事：

1. 粒子动能和场能不是分开各自守恒，而是可以相互交换。
2. 程序里若只监控粒子能量或只监控场能量，都不足以判断离散系统是否健康。

同理，总动量守恒应理解为“粒子动量 + 电磁场动量 + 边界 Maxwell stress”一起守恒，而不是单看粒子束团动量曲线。对后面的 implicit、hybrid、electrostatic sphere、planar pinch 和 FEL 例子，这个边界都非常关键：很多 regression 真正检查的是完整能量账本，而不是某个单独变量“看起来没漂”。

1.5 Vlasov-Poisson / electrostatic 极限不是另一套世界

当系统关注的是电荷分离和纵向静电响应，而电磁波传播、辐射和横向磁反馈不是主导效应时，可以把自治场闭合约化到 Vlasov-Poisson：

$$\frac{\partial f_s}{\partial t} + \mathbf{v} \cdot \nabla_{\mathbf{x}} f_s + q_s \mathbf{E} \cdot \nabla_{\mathbf{p}} f_s = 0,$$

$$\mathbf{E} = -\nabla \phi, \quad -\nabla^2 \phi = \frac{\rho}{\epsilon_0}.$$

它不是凭空把 Maxwell 方程换掉，而是对应这样一组物理假设：

- transverse electromagnetic radiation 不是主要自由度；
- 场主要由电荷分离决定；
- 电磁传播时间尺度不是当前主导尺度；
- 所关心的现象更接近 Langmuir、space-charge、electrostatic expansion、Poisson boundary-value problem。

因此：

- electrostatic PIC 仍然是 kinetic PIC；
- 只是“粒子如何给场提供源项、场如何回馈粒子”这一闭合从 Maxwell 变成了 Poisson。

这也是为什么：

- 第 6 章不能把 electrostatic solver 当作“Maxwell solver 的低配版”；
- electrostatic sphere、Pierce diode、effective potential 这些例子要和 Poisson 边界条件、势能账本一起讲；
- WarpX::OneStep() 里 electrostatic / hybrid 路线的场解位置会和标准 electromagnetic loop 不同。

1.6 宏粒子不是假粒子，而是 coarse-grained 分布函数载体

PIC 的核心近似不是把等离子体变成少数真实粒子，而是用有限数量的宏粒子采样分布函数。形式上可写成

$$f_s(\mathbf{x}, \mathbf{p}, t) \approx \sum_{p \in s} w_p S_x(\mathbf{x} - \mathbf{x}_p(t)) S_p(\mathbf{p} - \mathbf{p}_p(t)).$$

这里：

- w_p 是宏粒子权重；
- S_x 是空间形函数；
- S_p 常在实际 PIC 中退化成粒子自身在动量空间的离散采样。

从 Dawson 1983 的角度，更准确的说法是：宏粒子不是“把许多真实粒子团成一个球”的形象化故事，而是 coarse-grained kinetic model 的载体。它的目标是：

1. 用有限自由度代表连续分布；
2. 保住真正重要的低阶矩和 collective behavior；
3. 接受某些细粒度 phase-space 结构会被采样误差和 coarse graining 吞掉。

1.7 权重可以不同，但不是没有代价

宏粒子并不必然等权。Dawson 1983 讨论了

$$q_i = -\alpha_i e, \quad m_i = \alpha_i m, \quad \frac{q_i}{m_i} = -\frac{e}{m}$$

这一类不同电荷和质量、但相同荷质比的电子群，并说明由它们组成的加权分布函数仍满足通常的 Vlasov 方程。

这条结论的意义是：

- weighted macroparticles 从一开始就是合法的 kinetic coarse graining；
- 它允许把 phase-space 分辨率集中到真正需要的区域；
- 但它也会引入新的统计与 collisional side effects。

所以“加权宏粒子”更准确的理解不是“自适应采样免费升级”，而是：

- 你获得了 phase-space resolution redistribution；
- 但要付出更复杂的噪声、散射和统计解释代价。

这条边界对后面理解 WarpX 中：

- species 权重
- Gaussian beam / flux injection
- collision/QED product creation
- reduced-dimension weighting compensation

都很重要。

1.8 shape factor 不是插值细节，而是粒子-网格合同

若把粒子源项沉积到网格单元或网格点 i ，最基本的电荷密度形式是

$$\rho_i^n = \frac{1}{\Delta V_i} \sum_p q_p w_p S_i(\mathbf{x}_p^n).$$

场 gather 则用同一类 shape family 从网格插值回粒子位置：

$$\mathbf{E}_p^n = \sum_i S_i(\mathbf{x}_p^n) \mathbf{E}_i^n, \quad \mathbf{B}_p^n = \sum_i S_i(\mathbf{x}_p^n) \mathbf{B}_i^n.$$

但 shape factor 的意义远不止“双向插值”：

1. 它定义了宏粒子在空间上的 coarse-grained 电荷云。
2. 它决定了粒子-网格耦合的 stencil 宽度。
3. 它会系统改写短波 aliasing、self-force 和统计噪声。
4. 它直接影响 guard-cell 需求、通信宽度和算子局域性。

Birdsall 1985 与 Dawson 1983 的共同结论都指向这一点：finite-size particles 不是为了把图画得更平滑，而是为了软化 point-charge 的短程奇异作用、压低非物理 collisionality，并把系统真正保留成“长程 collective physics + 可控短程误差”。

这也是为什么后面的第 5 章必须把：

- shape factor
- charge/current deposition

- sampled density
- finite-grid effects
- aliasing

放在同一章里讲，而不是把 shape factor 单独缩成一个插值小节。

1.9 PIC 的噪声不是 bug，而是模型代价

把连续分布函数换成有限宏粒子之后，最基本的代价就是采样噪声。它不是代码写坏了才出现，而是：

- 有限粒子数
- 有限权重
- 有限网格
- 有限时间平均

共同带来的统计涨落。

从 Birdsall 1985 的 thermal-plasma 讨论看，这种噪声不能只被理解成“粒子数不够大”。更准确的图像是：

1. sampled density 会生成 alias branches;
2. shape factor 会修改 fluctuation spectrum;
3. finite Δx 和 finite Δt 会把 continuum 改写成带离散谱结构和 effective transport 的系统;
4. 若离散合同处理不好，噪声会演化成 numerical heating、drag、diffusion，甚至弱不稳定增长率的误判。

因此，本书后面凡是说“噪声更小”“结果更平滑”，都不应只停在图像层，而应继续问：

- 是 modal fluctuation level 变了？
- 是 alias branch 被压了？
- 还是只把可见图像平滑了，但守恒与统计量并没有更好？

1.10 Debye 长度、粒子数与统计时间尺度

Birdsall 1985 对 sheet model 的讨论给了一个比教科书定义更适合写进程序书的视角。

首先，Debye 长度 λ_D 和 Debye 球内粒子数 N_D 不是孤立的公式，而是“这个 plasma 是否能被当作 collective medium”与“统计噪声会以什么尺度渗入观测量”的共同边界。

其次，在 reduced model 下，

$$\tau \sim \frac{2N_D}{\omega_p}$$

更适合被理解成：

- randomization time
- correlation time
- 统计独立采样间隔

而不是整个分布完全热化成 Maxwellian 的总弛豫时间。后者通常更慢，量级更接近 N_D^2 。

对 PIC 用户来说，这比“记住 Debye 长度定义”更实用，因为它直接影响：

- uniform-plasma 噪声底怎么看；
- reduced diagnostics 应平均多久；
- 弱效应、弱不稳定和 Landau damping 的 measurement window 多大才可信。

1.11 从连续模型到 PIC 离散变量

前面的方程还没有直接变成程序里的数组。PIC 的第一步不是把分布函数存成一个高维网格，而是用带权粒子样本代表它，再用网格上的有限差分或谱变量承载场。可以把这条映射写成下面的最小合同：

连续对象	PIC 离散载体	典型时间层/位置	在 WarpX 代码中应如何理解
$f_s(x, p, t)$	物种粒子的位置、动量、权重集合	$x_p^n, p_p^{n-1/2}, w_p$	ParticleContainer 中的粒子样本，不是一个逐点存储的分布函数
$\rho(x, t)$	shape-weighted charge density	ρ^n 或 $\rho^{n+1/2}$	由粒子沉积得到；rho_fp 与 rho_buf 还可能分别属于 fine 与 coarse-buffer 路径
$J(x, t)$	trajectory-based current density	通常跨越 $n- > n+1$	Esirkepov/Villasenor 等 current deposition 需要 old/new 轨迹或等价的 crossing 信息
E, B	staggered/collocated grid fields	由 solver 规定	Efield_*, Bfield_* 的后缀表示时间层、网格位置或辅助副本，不能只按变量名猜物理时刻
连续性方程	离散 source synchronization	每个粒子推进步或 solver stage	SyncCurrentAndRho()、guard exchange、AMR average-down 等共同完成可供场求解器使用的 source

最容易被忽略的是 ρ 和 J 的时间语义不同。单时间层的 charge deposition 可以直接对粒子位置取样：

$$\rho_i^n = (1/\Delta V_i) \sum_p q_p w_p S_i(x_p^n),$$

而守恒的 current deposition 必须表达粒子从 x_p^n 到 x_p^{n+1} 的输运：

$$\Delta t (\nabla_h \cdot J)_i = \rho_i^n - \rho_i^{n+1}.$$

这里的第二式不是说所有 current deposition 都在程序中显式先算出左右两边，而是说明它们必须共享同一条离散连续性合同。也因此，

- DepositCharge() 负责单时间层的 ρ 采样及其时间层、几何和 AMR 桥接；
- DepositCurrent() 及其 Esirkepov/Villasenor 等路径负责轨迹输运产生的 J ；
- SyncCurrentAndRho() 负责把不同 level、边界和 source buffer 中的结果整理成 solver 可消费的源项。

这三层不能合并成“粒子把电荷写到网格”一句话。后续第 5 章会从 kernel 角度展开，第 6 章则会继续说明不同 field solver 如何消费这些 source。附录 A 给出 rho_fp、rho_buf、current_fp、current_buf 和 lev 等项目内变量的速查定义。## 1.12 这一章对后面源码章节的真正约束

到这里，后续读 WarpX 代码时至少要带着下面这些硬问题，而不是只盯函数名：

1. 粒子推进器是否在离散时间层上合理近似了 Liouville 流？
2. 沉积算法是否把连续性方程离散闭合到了 rho/J？
3. field solver 处理的是 Maxwell 还是 Poisson，约束方程怎样传播？
4. shape factor 和 finite-size particles 是如何改写噪声、aliasing 和 self-force 的？
5. diagnostics 到底在测真实物理量，还是只看离散噪声底的一个投影？

如果没有这几层边界，后面源码里的：

- OneStep_nosub
- PushParticlesandDeposit
- SyncCurrentAndRho
- PushPSATD
- ElectrostaticSolver
- ImplicitSolver

都会被读成“工程控制流”，而不是“连续模型的离散化实现”。

1.13 本章当前文献边界

本章当前正文已真正依托的文献边界是：

- Birdsall 1985
 - 已精读并回填：
 - * sheet model 的 randomization / correlation / thermalization 时间尺度
 - * finite-grid / aliasing / fluctuation / heating 主线
- Dawson 1983
 - 已精读并回填：
 - * numerical experiment 视角
 - * superparticle / weighted particles 的 kinetic 边界
 - * finite-size particles + grid + FFT-Poisson 的标准 electrostatic contract

仍未在本章直接依赖其正文细节的文献是：

- Hockney-Eastwood
 - acquisition 尚未完成
- Yee 1966
 - acquisition 尚未完成

基础章节当前允许直接作为正文证据、以及哪些条目仍只能写成 acquisition / metadata 边界，现统一收口到：

- 基础章节文献清单

因此本章当前版本已经足够支撑后续源码阅读，但基础文献层仍不是最终完成态。后续还需要继续：

1. 补 Hockney-Eastwood 或其 article-level fallback 对 weighted particles / heating estimates / optimum path 的原始证据；
2. 补 Yee 1966 对 staggered FDTD 与离散约束传播的原始文献入口；
3. 再把本章和第 2 章之间关于 leapfrog、CFL、Debye 长度、数值色散的边界继续压紧。

1.14 练习与源码定位

1. 变量桥接题：根据 1.11 的映射表，说明为什么 ρ_{fp}/ρ_{buf} 不能直接当作两个不同物理量，并指出它们分别在哪个 AMR/source-synchronization 场景出现。
2. 尺度判断题：给定 $\lambda_D/\Delta x = 2$ 和 $v_t \Delta t/\Delta x = 0.4$ ，列出至少两个可能的数值风险，并说明它们分别属于空间分辨率还是时间推进约束。
3. 源码定位题：在当前 WarpX checkout 中定位 `PushParticlesandDeposit()`、`SyncCurrentAndRho()` 和一个 field-solver 入口，分别写出它们连接连续模型中哪一个对象：粒子输运、源项连续性还是 Maxwell/Poisson 闭合。

2. PIC 总循环：从 Vlasov-Maxwell 到离散时间推进

本章先不急着重进入某一个 WarpX 函数。生产级 PIC 代码的困难不在于“有粒子、有网格、有 Maxwell 方程”这几个名词，而在于这些对象必须在离散时间层、离散空间布局、并行 guard cells、边界条件和守恒约束之间保持一致。后续逐行读 WarpX 时，本章给出判断代码是否“物理上在做正确事情”的基准。

本章对应的第一批源码阅读笔记保存在 `notes/code-reading/evolve/01-pic-time-layers.md` 和 `notes/code-reading/evolve/02-evolve.md`。

本章当前依据的 WarpX 源码版本是：

- `../warpX`
- 分支：pkuHEDPbranch
- commit: 8c488b1a9

v0.2 校准说明：本章已把主循环相关源码路径同步到当前 WarpX 目录结构 `Source/Evolve/`，并重核 `Evolve()`、`OneStep_nosub()` 与 FDTD/PSATD 分支的关键行号。章节仍保留基础文献缺口；Hockney-Eastwood、Yee 1966 等一手材料的 MinerU 笔记尚未完成，暂不把这些缺口伪装成已闭环引用。

2.1 连续模型：Vlasov-Maxwell 系统

对物种 s ，相空间分布函数 $f_s(\mathbf{x}, \mathbf{p}, t)$ 满足 Vlasov 方程：

$$\frac{\partial f_s}{\partial t} + \mathbf{v} \cdot \nabla_{\mathbf{x}} f_s + q_s (\mathbf{E} + \mathbf{v} \times \mathbf{B}) \cdot \nabla_{\mathbf{p}} f_s = 0.$$

相对论动量与速度满足

$$\mathbf{p} = \gamma m_s \mathbf{v}, \quad \gamma = \sqrt{1 + \frac{|\mathbf{p}|^2}{m_s^2 c^2}}, \quad \mathbf{v} = \frac{\mathbf{p}}{\gamma m_s}.$$

电磁场满足 Maxwell 方程：

$$\frac{\partial \mathbf{B}}{\partial t} = -\nabla \times \mathbf{E},$$

$$\frac{\partial \mathbf{E}}{\partial t} = c^2 \nabla \times \mathbf{B} - \frac{\mathbf{J}}{\epsilon_0},$$

以及约束方程

$$\nabla \cdot \mathbf{E} = \frac{\rho}{\epsilon_0}, \quad \nabla \cdot \mathbf{B} = 0.$$

源项来自分布函数的矩：

$$\rho(\mathbf{x}, t) = \sum_s q_s \int f_s(\mathbf{x}, \mathbf{p}, t) d\mathbf{p},$$

$$\mathbf{J}(\mathbf{x}, t) = \sum_s q_s \int \mathbf{v}(\mathbf{p}) f_s(\mathbf{x}, \mathbf{p}, t) d\mathbf{p}.$$

PIC 的核心任务就是把这套连续耦合系统离散成两个相互交换信息的对象：宏粒子和网格场。

2.2 宏粒子表示与形函数

宏粒子近似把 f_s 写成有限个带权粒子的和：

$$f_s(\mathbf{x}, \mathbf{p}, t) \approx \sum_{p \in s} w_p S_x(\mathbf{x} - \mathbf{x}_p(t)) S_p(\mathbf{p} - \mathbf{p}_p(t)).$$

这里 w_p 是宏粒子权重， S_x 是空间形函数。把粒子源项沉积到网格单元或网格点 i ，常见电荷密度形式是

$$\rho_i^n = \frac{1}{\Delta V_i} \sum_p q_p w_p S_i(\mathbf{x}_p^n).$$

场 gather 则使用同一类形函数从网格插值回粒子位置：

$$\mathbf{E}_p^n = \sum_i S_i(\mathbf{x}_p^n) \mathbf{E}_i^n, \quad \mathbf{B}_p^n = \sum_i S_i(\mathbf{x}_p^n) \mathbf{B}_i^n.$$

如果只看这两个公式，很容易以为 PIC 的网格-粒子耦合只是“双向插值”。这个理解不够。电流 $\mathbf{J}$ 的沉积必须表达粒子在一个时间步内的轨迹，否则离散电荷守恒会被破坏。

连续系统满足连续性方程：

$$\frac{\partial \rho}{\partial t} + \nabla \cdot \mathbf{J} = 0.$$

在网格上，电流沉积应尽量满足对应的离散形式：

$$\frac{\rho_i^{n+1} - \rho_i^n}{\Delta t} + (\nabla_h \cdot \mathbf{J}^{n+1/2})_i = 0.$$

这解释了为什么 WarpX 这样的代码会提供 Esirkepov、Villasenor-Buneman、Vay 等沉积分支。它们的差异不是表面上的“把电流放到哪里”，而是如何在离散网格上把粒子轨迹、电荷守恒、网格布局和并行边界结合起来。

2.3 leapfrog 时间层

显式电磁 PIC 的标准时间层是 leapfrog：

- 粒子位置在整数步： $\mathbf{x}^n, \mathbf{x}^{n+1}$ 。
- 粒子动量在半整数步： $\mathbf{p}^{n-1/2}, \mathbf{p}^{n+1/2}$ 。
- 电流自然在半整数步： $\mathbf{J}^{n+1/2}$ 。
- 电磁场在 Yee/FDTD 路径中按半步磁场和整步电场交错推进。

粒子位置推进可写成

$$\frac{\mathbf{x}^{n+1} - \mathbf{x}^n}{\Delta t} = \mathbf{v}^{n+1/2}.$$

动量推进写成抽象形式：

$$\frac{\mathbf{p}^{n+1/2} - \mathbf{p}^{n-1/2}}{\Delta t} = q (\mathbf{E}_p^n + \bar{\mathbf{v}} \times \mathbf{B}_p^n).$$

其中 $\bar{\mathbf{v}}$ 的具体定义取决于 pusher。Boris、Vay、Higuera-Cary 等 pusher 的差别留到粒子推进章节逐行讲解。本章只强调主循环必须给 pusher 提供正确时间层的 $\mathbf{x}$ 、 $\mathbf{p}$ 、 $\mathbf{E}$ 、 $\mathbf{B}$ 。

WarpX 的显式无 subcycling 路径在 `../warpX/Source/Evolve/WarpXEvolve.cpp:515-518` 直接把这个时间层写进注释：

```
Push particle from  $\mathbf{x}^{\{n\}}$  to  $\mathbf{x}^{\{n+1\}}$ 
    from  $\mathbf{p}^{\{n-1/2\}}$  to  $\mathbf{p}^{\{n+1/2\}}$ 
Deposit current  $\mathbf{j}^{\{n+1/2\}}$ 
Deposit charge density  $\rho^{\{n\}}$ 
```

这四行是读 WarpX 主循环的锚点。任何 field gather、collision、ionization、deposition、sync、field solve 的位置都应围绕这些时间层理解。

2.3.1 ω_p 不是背景常数，而是时间离散必须尊重的最快等离子体尺度

对电子等离子体，最基本的本征时间尺度是 plasma frequency：

$$\omega_p = \sqrt{\frac{n_e e^2}{m_e \epsilon_0}}.$$

它决定了最简单的 Langmuir 振荡周期

$$T_p = \frac{2\pi}{\omega_p}.$$

对显式 leapfrog PIC，稳定和分辨不是同一件事。只要时间层排列正确、场更新满足 CFL，代码也许不会立刻炸掉；但如果

$$\omega_p \Delta t$$

已经接近或超过 1 的量级，那么单步内粒子和场已经跨过了 plasma oscillation 的核心相位结构，后面看到的高噪声、相位误差、非物理 heating，往往不是“分析脚本太苛刻”，而是主循环本身没有分辨这个最快内禀尺度。

这也是为什么 Birdsall 1985 后面会把

$$\omega_p \Delta t, \quad v_t \Delta t / \Delta x$$

都写成数值健康度的第一层控制量。对 WarpX 来说, 这条边界不会自动由 ComputeDt() 替你保证。ComputeDt() 只根据 solver/CFL、const_dt、max_dt、maxParticleVelocity() 和 AMR refinement 给出一个可运行步长; 它并不知道你要不要精确分辨 Langmuir 振荡、electrostatic shielding 或弱不稳定增长率。

所以本章这里先压实一个最重要的判断:

- ComputeDt() 保证的是一层离散稳定性和时间步组织契约;
- \omega_p \Delta t 是否足够小, 仍然是物理建模和分辨率设计问题。

2.3.2 \lambda_D 不只是一条长度定义, 它直接约束 \Delta x

和 \omega_p 对偶的空间尺度是 Debye length。对非相对论热电子,

$$\lambda_D = \sqrt{\frac{\epsilon_0 k_B T_e}{n_e e^2}} = \frac{v_{th,e}}{\omega_p}.$$

这条式子把热速度、plasma frequency 和 shielding length 绑在了一起。对 PIC 而言, 能否把 plasma 当作 collective medium 与 网格是否真的分辨了 shielding 不是分开的两个问题。

如果

$$\Delta x \gg \lambda_D,$$

那么 cell 内已经把最基本的 shielding 结构粗化掉了。接下来即使宏观波形看起来还能跑, field fluctuation、aliasing、self-force 和 nonphysical collisionality 也会被系统性放大。这就是为什么第 1 章已经把 \lambda_D、N_D 和统计时间尺度单独拎出来; 在第 2 章里, 它进一步变成主循环的硬分辨率边界:

- \Delta t 决定是否分辨 \omega_p;
- \Delta x 决定是否分辨 \lambda_D;
- 两者一起决定 leapfrog + grid PIC 到底是在近似同一个 plasma, 还是已经换成了另一个更噪、更热、更强 alias 的离散模型。

2.4 FDTD 场更新的数学骨架

忽略 PML、divergence cleaning、宏观介质和边界时, Yee/FDTD 的主更新可以写成三段:

$$\begin{aligned} \mathbf{B}^{n+1/2} &= \mathbf{B}^n - \frac{\Delta t}{2} \nabla_h \times \mathbf{E}^n, \\ \mathbf{E}^{n+1} &= \mathbf{E}^n + c^2 \Delta t \nabla_h \times \mathbf{B}^{n+1/2} - \frac{\Delta t}{\epsilon_0} \mathbf{J}^{n+1/2}, \\ \mathbf{B}^{n+1} &= \mathbf{B}^{n+1/2} - \frac{\Delta t}{2} \nabla_h \times \mathbf{E}^{n+1}. \end{aligned}$$

这说明一个电磁 PIC step 的顺序不能随意交换。电场更新需要本步沉积出来的 $\mathbf{J}^{n+1/2}$, 所以粒子推进与电流沉积必须在 EvolveE(dt) 之前完成。WarpX 的 FDTD 路径正是这样组织的:

对应的核心源码节选来自 ../warpX/Source/Evolve/WarpXEvolve.cpp:559-628, 这里保留源项同步和 FDTD 场推进部分; PSATD 分支和 PML 后处理在第 3 章继续展开:

```

// Synchronize J and rho:
// filter (if used), exchange guard cells, interpolate across MR levels
// and apply boundary conditions
SyncCurrentAndRho();

// For extended PML: copy J from regular grid to PML, and damp J in PML
if (do_pml && pml_has_particles) { CopyJPML(); }
if (do_pml && do_pml_j_damping) { DampJPML(); }

ExecutePythonCallback("beforeEsolve");

EvolveF(0.5_rt * dt[0], /*rho_comp=*/0);
EvolveG(0.5_rt * dt[0]);
FillBoundaryF(guard_cells.ng_FieldSolverF);
FillBoundaryG(guard_cells.ng_FieldSolverG);

EvolveB(0.5_rt * dt[0], SubcyclingHalf::FirstHalf, a_cur_time); // We now have  $B^{n+1/2}$ 
FillBoundaryB(guard_cells.ng_FieldSolver, WarpX::sync_nodal_points);

if (m_em_solver_medium == MediumForEM::Vacuum) {
    // vacuum medium
    EvolveE(dt[0], a_cur_time); // We now have  $E^{n+1}$ 
} else if (m_em_solver_medium == MediumForEM::Macroscopic) {
    // macroscopic medium
    MacroscopicEvolveE(dt[0], a_cur_time); // We now have  $E^{n+1}$ 
} else {
    WARPX_ABORT_WITH_MESSAGE("Medium for EM is unknown");
}
FillBoundaryE(guard_cells.ng_FieldSolver, WarpX::sync_nodal_points);

EvolveF(0.5_rt * dt[0], /*rho_comp=*/1);
EvolveG(0.5_rt * dt[0]);
EvolveB(0.5_rt * dt[0], SubcyclingHalf::SecondHalf, a_cur_time + 0.5_rt * dt[0]); // We now have  $B^{n+1}$ 

```

数学动作	WarpX 源码位置
粒子从 $\mathbf{x}^n, \mathbf{p}^{n-1/2}$ 推到 $\mathbf{x}^{n+1}, \mathbf{p}^{n+1/2}$, 并沉积源项	../warpX/Source/Evolve/WarpXEvolve.cpp:520-557
同步 $\rho, \mathbf{J}$: 滤波、guard cells、AMR、边界	../warpX/Source/Evolve/WarpXEvolve.cpp:559-564 与 SyncCurrentAndRho()
F, G 半步、 $\mathbf{B}$ 半步	../warpX/Source/Evolve/WarpXEvolve.cpp:607-613
$\mathbf{E}$ 整步	../warpX/Source/Evolve/WarpXEvolve.cpp:615-623
F, G 半步、 $\mathbf{B}$ 半步	../warpX/Source/Evolve/WarpXEvolve.cpp:626-628
PML 与 guard cell 处理	../warpX/Source/Evolve/WarpXEvolve.cpp:630-642

这里的 F, G 是 divergence cleaning 相关辅助场, 不是最小 Maxwell 更新必需项。WarpX 把它们插在场推进两侧, 是为了在实际模拟中控制 $\nabla \cdot \mathbf{E}$ 或 $\nabla \cdot \mathbf{B}$ 误差及 PML 相关处理。

2.4.1 CFL 不是经验参数, 而是离散 Maxwell 更新的因果上界

WarpX 的 ComputeDt() 不会把 FDTD 时间步写死成 $\min(\Delta x_i)/c$, 而是把这件事委托给具体差分算法。../warpX/Source/Evolve/WarpXCFL 直接把 solver 分叉写死在主类层:

```

} else if (electromagnetic_solver_id == ElectromagneticSolverAlgo::PSATD) {
    deltata = cfl * minDim(dx) / PhysConst::c;
} else {
    #if defined(WARPX_DIM_RZ) || defined(WARPX_DIM_RCYLINDER)
    if (electromagnetic_solver_id == ElectromagneticSolverAlgo::Yee) {

```

```

        deltat = cfl * CylindricalYeeAlgorithm::ComputeMaxDt(dx, n_rz_azimuthal_modes);
#elif defined(WARPX_DIM_RSPHERE)
    if (electromagnetic_solver_id == ElectromagneticSolverAlgo::Yee) {
        deltat = cfl * SphericalYeeAlgorithm::ComputeMaxDt(dx);
    }
#else
    if (grid_type == GridType::Collocated) {
        deltat = cfl * CartesianNodalAlgorithm::ComputeMaxDt(dx);
    } else if (electromagnetic_solver_id == ElectromagneticSolverAlgo::Yee
        || electromagnetic_solver_id == ElectromagneticSolverAlgo::ECT) {
        deltat = cfl * CartesianYeeAlgorithm::ComputeMaxDt(dx);
    } else if (electromagnetic_solver_id == ElectromagneticSolverAlgo::CKC) {
        deltat = cfl * CartesianCKCAlgorithm::ComputeMaxDt(dx);
    }

```

对标准 Cartesian Yee,真正的 CFL 上界在 ../warpx/Source/FieldSolver/FiniteDifferenceSolver/FiniteDifferenceAlgorithms

```

static amrex::Real ComputeMaxDt ( amrex::Real const * const dx ) {
    using namespace amrex::literals;
    amrex::Real const delta_t = 1._rt / ( std::sqrt( AMREX_D_TERM(
        1._rt / (dx[0]*dx[0]),
        + 1._rt / (dx[1]*dx[1]),
        + 1._rt / (dx[2]*dx[2])
    ) ) * PhysConst::c );

    return delta_t;
}

```

也就是

$$\Delta t_{\max}^{\text{Yee}} = \frac{1}{c\sqrt{\Delta x^{-2} + \Delta y^{-2} + \Delta z^{-2}}}.$$

它表达的是离散光锥约束：单步内，Yee curl stencil 不能让信息传播得比离散网格所允许的因果速度更快。warpx.cfl 只是把这个严格上界再乘一个安全系数，不是“拍脑袋调参”。

2.4.2 数值色散从这里开始：连续光锥被离散 stencil 改写

一旦用有限差分近似 curl，连续真空色散关系

$$\omega^2 = c^2 |\mathbf{k}|^2$$

就不再原样保留。以 Yee 为例，差分导数对应的是

$$k_d \rightarrow \frac{2}{\Delta d} \sin \frac{k_d \Delta d}{2},$$

于是离散色散关系变成近似的

$$\sin^2 \frac{\omega \Delta t}{2} = c^2 \Delta t^2 \sum_d \frac{\sin^2(k_d \Delta d / 2)}{\Delta d^2}.$$

这意味着 phase velocity 和 group velocity 都会偏离连续值，且偏离大小依赖传播方向、波数和网格各向异性。数值色散不是后处理图里才会出现的现象，它在 UpwardDx() / DownwardDx() 这种最基本的差分定义处就已经写进去了。

Yee 的 x 向导数在 ../warpx/Source/FieldSolver/FiniteDifferenceSolver/FiniteDifferenceAlgorithms/CartesianYeeAlgo

```

static amrex::Real UpwardDx (
    amrex::Array4<amrex::Real const> const& F,
    amrex::Real const * const coefs_x, int const /*n_coefs_x*/,
    int const i, int const j, int const k, int const ncomp=0 ) {

```

```

...
amrex::Real const inv_dx = coefs_x[0];
return inv_dx*( F(i+1,j,k,ncomp) - F(i,j,k,ncomp) );
}

template< typename T_Field>
static amrex::Real DownwardDx (
    T_Field const& F,
    amrex::Real const * const coefs_x, int const /*n_coefs_x*/,
    int const i, int const j, int const k, int const ncomp=0 ) {
    ...
    amrex::Real const inv_dx = coefs_x[0];
    return inv_dx*( F(i,j,k,ncomp) - F(i-1,j,k,ncomp) );
}

```

也就是 staggered grid 上的前/后向一阶差分：

$$D_x^+ F_i = \frac{F_{i+1} - F_i}{\Delta x}, \quad D_x^- F_i = \frac{F_i - F_{i-1}}{\Delta x}.$$

这里的 Upward/Downward 不只是“正向/反向”，而是在 nodal 与 cell-centered 位置之间搬运离散导数。正是这种 staggered 几何，让 Yee 在保持二阶精度的同时把 E/B 交错布置起来。

2.4.3 Yee / Nodal / CKC 的差别本质上是离散色散不同

对 collocated/nodal solver, WarpX 的 CartesianNodalAlgorithm 不再用 staggered 前后差分，而是直接用中心差分。

../warpx/Source/FieldSolver/FiniteDifferenceSolver/FiniteDifferenceAlgorithms/CartesianNodalAlgorithm.H:71

```

static amrex::Real UpwardDx (
    amrex::Array4<amrex::Real const> const& F,
    amrex::Real const * const coefs_x, int const /*n_coefs_x*/,
    int const i, int const j, int const k, int const ncomp=0 ) {
    ...
    Real const inv_dx = coefs_x[0];
    return 0.5_rt*inv_dx*( F(i+1,j,k,ncomp) - F(i-1,j,k,ncomp) );
}

static amrex::Real DownwardDx (
    amrex::Array4<amrex::Real const> const& F,
    amrex::Real const * const coefs_x, int const n_coefs_x,
    int const i, int const j, int const k, int const ncomp=0 ) {
    ...
    return UpwardDx( F, coefs_x, n_coefs_x, i, j, k, ncomp);
}

```

所以 nodal grid 上 UpwardDx 和 DownwardDx 等价，说明这里已经没有 Yee 那种 staggered 位置语义，而是一个 collocated 中心差分系统。它的 CFL 上界虽然在当前实现里和 Yee 一样都是

$$\Delta t_{\max} \sim \frac{1}{c \sqrt{\sum_d \Delta d^{-2}}},$$

但数值色散与奇偶模耦合特征已经不同。

CKC 则更进一步，不再满足“一个方向只看一对最近邻”的局部导数定义。../warpx/Source/FieldSolver/FiniteDifferenceSolver/FiniteDiffe

```

static amrex::Real ComputeMaxDt ( amrex::Real const * const dx ) {
    #if (defined WARPX_DIM_1D_Z)
        amrex::Real const delta_t = dx[0]/PhysConst::c;
    #elif (defined WARPX_DIM_XZ)

```

```

        amrex::Real const delta_t = std::min( dx[0], dx[1] )/PhysConst::c;
#else
        amrex::Real const delta_t = std::min( dx[0], std::min( dx[1], dx[2] ) ) / PhysConst::c;
#endif
    return delta_t;
}

...
return alphax * (F(i+1,j ,k ,ncomp) - F(i , j, k ,ncomp))
+ betaxy * (F(i+1,j+1,k ,ncomp) - F(i , j+1,k ,ncomp)
+ F(i+1,j-1,k ,ncomp) - F(i , j-1,k ,ncomp))
+ betaxz * (F(i+1,j ,k+1,ncomp) - F(i , j ,k+1,ncomp)
+ F(i+1,j ,k-1,ncomp) - F(i , j ,k-1,ncomp))

```

这说明 CKC 的真实目标不是“换一套写法”，而是通过更宽的横向耦合 stencil 改写离散色散关系，从而改善高方向性传播下的 phase error。代价则是：

- stencil 更宽；
- 算法/geometry 适用范围更窄；
- guard-cell 和边界处理的组合空间更复杂。

2.5 PSATD 与 FDTD 的主循环差异

FDTD 在实空间用局部 stencil 近似 curl。PSATD 则在谱空间解析积分线性 Maxwell 方程的一部分。物理方程相同，但离散算法不同：

- FDTD 的优势是局部、显式、边界和 AMR 工程路径相对直接。
- PSATD 能显著降低数值色散，适合相对论束流、激光等问题，但对 FFT、并行 domain decomposition、边界、current correction 和时间平均场有更复杂要求。

这条分界线和前面几节正好连起来：

- leapfrog 规定了粒子、场和源项的时间层合同；
- ω_p 与 λ_D 规定了 plasma 自身是否被当前 $\Delta t / \Delta x$ 分辨；
- CFL 规定了 Maxwell 更新是否还能保持离散因果；
- Yee/Nodal/CKC/PSATD 则进一步决定同一组 $\Delta t, \Delta x$ 会把波动相速度、群速度和 aliasing 改写成什么样。

所以“主循环能跑”不等于“主循环近似的是对的物理系统”。真正的 PIC loop 要同时满足：

time-layer consistency + charge/source consistency + physical scale resolution + solver-dependent stability/dispersion control.

WarpX 在 OneStep_nosub() 内部把两者清楚分开：

- PSATD 分支在 ../warpX/Source/Evolve/WarpXEvolve.cpp:576-605，核心是 PushPSATD(a_cur_time)。
- FDTD 分支在 ../warpX/Source/Evolve/WarpXEvolve.cpp:606-642，核心是 EvolveB/EvolveE/EvolveB。

这意味着本书后续讲 field solver 时不能把“Maxwell solver”写成单一算法。algo.maxwell_solver 的选择会改变主循环内的场推进、同步、边界和可用功能。

2.6 一个真实 PIC step 的工程层次

把物理动作映射到生产代码，一个时间步至少包含这些层次：

1. 用户 callback、信号、诊断、负载均衡和步长更新。
2. 场 gather 前的 guard cell 与 auxiliary field 准备。
3. 电离、QED、粒子注入等改变粒子集合的多物理模块。
4. 粒子推进、碰撞、沉积电流和电荷。
5. 源项同步：滤波、guard cells、AMR fine/coarse 交换、边界条件。
6. 场推进：FDTD、PSATD、implicit、electrostatic 或 hybrid。
7. PML、moving window、粒子边界、重分布、排序。
8. 诊断写出和终止条件检查。

WarpX 的 ../warpX/Source/Evolve/WarpXEvolve.cpp:147-390 正是围绕这些层次组织外层 Evolve() 循环。真正的主循环不是教科书五行伪代码，而是把守恒离散化、时间层一致性和大规模并行工程组合起来的控制流。

2.6.1 AMR subcycling: 两个时间步不是同一个时间步的重复调用

无 subcycling 时, 第 0 层和更细层使用同一个外层时间步, `OneStep_nosub()` 可以把粒子推进、source synchronization 和场推进看成一条统一的 $n- > n+1$ 链。打开 subcycling 后, 这个图像不再成立。当前 WarpX 的 `OneStep_sub1()` 在 `./warpX/Source/Evolve/WarpXEvolve.cpp:1040` 附近明确限定: 只支持两级 mesh refinement, 且每个方向的 refinement ratio 必须为 2。

令粗层时间步为 Δt_c , 细层时间步为

$$\Delta t_f = \frac{\Delta t_c}{2}.$$

一个粗层周期内的基本推进结构是:

阶段	细层	粗层/母网格	源项职责
第一个半周期	推进一次粒子和场, 步长 Δt_f	暂不完成整步	<code>current_fp</code> 、 <code>rho_fp</code> 先 restrict 到 coarse patch
中间同步	细层已到 $t + \Delta t_f$	粗层推进到相应中间时间	<code>AddCurrentFromFineLevelandSumBoundary</code> 与 <code>AddRhoFromFineLevelandSumBoundary</code>
第二个半周期	再推进一次粒子和场, 步长 Δt_f	继续完成粗层剩余半步	合并细层、粗层和 buffer 源 第二次 fine source 经 restrict/add 后参与粗层后半步场更新
粗层周期末	到 $t + \Delta t_c$	到 $t + \Delta t_c$	粗细层场、源项和 guard cells 重新达到可交换状态

因此, subcycling 不是简单地把 `OneStep_nosub()` 调两次。粗层粒子只推进一次, 而细层粒子推进两次; 粗层场的更新还要消费两个细层时间片上累积并平均/合成后的电流。源码中的调用顺序可以压缩成:

```
fine push/deposit at  $t \rightarrow$  restrict source  $\rightarrow$  fine field half/full/half,
coarse push/deposit at  $t \rightarrow$  combine coarse+fine source  $\rightarrow$  coarse field advance,
fine push/deposit at  $t + \Delta t_f \rightarrow$  restrict source  $\rightarrow$  fine field half/full/half,
combine second fine source  $\rightarrow$  coarse field completion.
```

这里的 `restrict` 和 `add` 不能与普通 guard-cell exchange 混为一谈: 前者改变的是 coarse/fine source 的层级表示, 后者只是同一层相邻 patch 间的数据可见性。对电荷来说, `rho_buf` 还可能来自 transition-zone 粒子在 coarse 几何上的直接沉积; 因此 subcycling 中的 source 合成既不是“把 fine rho 全部平均下来”这么简单, 也不是由场求解器自动补齐。

当前实现还显式禁止 electrostatic solver 与 subcycling 组合。这个限制写在 `OneStep_sub1()` 的入口断言中, 原因不是 electrostatic 不能使用 AMR, 而是这条 subcycling 例程的时间组织只为显式 electromagnetic field advance 编写, 不能把 Poisson/electrostatic 路径的源项和场解时序悄悄套进来。

所以读 AMR PIC loop 时要同时检查四个不变量:

1. 细层是否确实使用 $\Delta t_c/2$, 且在一个粗层周期内推进两次;
2. 粗层粒子是否只推进一次, 避免重复计数粒子输运;
3. 两个细层时间片的 `current/rho` 是否分别完成 restrict、边界合并和 coarse source add;
4. 粗细层字段与 `auxiliary/guard data` 是否在下次 gather 前重新可见。

这四项共同构成 AMR subcycling 的时间合同。缺少其中任何一项, 都可能得到“每个 patch 都成功更新”但跨层电荷守恒、场相位或粒子 gather 已经不一致的结果。

2.6.2 JRhom 与 implicit: 同一个外层 step 内部也可能有不同的时间合同

标准 `OneStep_nosub()`、PSATD-JRhom 和 implicit solver 都可能被外层 `WarpX::OneStep()` 视为一次迭代, 但它们内部对“源项在什么时候被求值”的定义不同。当前源码的分派关系是:

路径	外层入口	源项/粒子时间组织	场推进特点	当前组合边界
标准显式 electromagnetic PIC	OneStep_nosub()	一次粒子推进, 得到 J 与 rho, 随后统一 SyncCurrentAndRho()	FDTD 的 B-E-B 或一次 PSATD 推进	可与普通显式 collision placement 组合
PSATD-JRhom	OneStep_JRhom()	先推进粒子但跳过普通沉积, 再按 rho/J 时间依赖在 Δt 内做多次相对时间沉积	每个 deposit interval 都执行一次谱空间场推进; 可选跨 $2\Delta t$ 时间平均	只支持 PSATD; current_correction 不支持; split momentum collision push 不支持
implicit electromagnetic PIC	ImplicitSolver::OneStep()	在非线性/线性 RHS 评估中反复推进粒子并构造 $J^{n+1/2}$	通过 nonlinear solver 求自洽中间电场, 再完成粒子和场的后半步	不能把一次 RHS 评估误当成一次物理时间步; mass-matrix/JFNK 还会改变 J 的构造路径

JRhom 的关键不是“PSATD 多调用几次”, 而是把时间依赖的源项显式建模成一组谱空间可消费的历史量。OneStep_JRhom() 的顺序可以压缩为:

1. 以 skip_deposition=true 把粒子从 $x^n, p^{n-1/2}$ 推到 $x^{n+1}, p^{n+1/2}$;
2. 把 E/B 以及可选的 divergence-cleaning 场变换到谱空间;
3. 按 time_dependency_rho 和 time_dependency_J 在相对时间上沉积 rho_new 与 J, 每次执行 filter、guard/AMR 同步和 Fourier transform;
4. 对每个子区间执行 PSATDPushSpectralFields(), 最后把场变回实空间并处理 PML、边界和 guard cells。

其中 m_JRhom_subintervals 把一个外层步拆成 sub_dt = $\Delta t / n_deposit$ 的多个沉积区间; 打开 time averaging 时, 内部循环还会覆盖两个完整时间步的源/场积分。因而 JRhom 中的 rho 和 J 不是标准显式路径里“本步各沉积一次”的同义词, 不能直接用 OneStep_nosub() 的单点时间图解释它。

implicit 路径的差异更加根本。以 SemiImplicitEM::OneStep() 为例, 程序先保存粒子和旧场, 将磁场推进到半步, 然后由 nonlinear solver 反复调用 ComputeRHS(); ImplicitSolver::PreRHSOp() 在每次 RHS 构造里:

- 用当前猜测的中间电场推进粒子;
- 形成半步电流;
- 在需要时把 current_fp_non_suborbit、mass matrices 或 JFNK 线性阶段的贡献合并进 J;
- 最后调用 SyncCurrentAndRho(), 再把同步后的源项交给隐式残差/线性系统。

所以 implicit 中必须区分三个层次:

1. 物理时间步 $t^n \rightarrow t^{n+1}$;
2. 非线性迭代中的中间场猜测 $E^{n+\theta}$;
3. 每次 RHS 或 Jacobian 评估中的粒子/source 重算。

如果把第 3 层误写成“程序又推进了一个物理时间步”, 就会错误理解粒子数、能量账本和 SyncCurrentAndRho() 的调用次数。相反, 如果把 JRhom 的多个 deposit interval 当成 nonlinear iteration, 也会把时间积分和求解器迭代混为一谈。

本书后续章节的阅读规则因此固定为: 先识别外层物理时间步, 再识别内部的 source subinterval 或 nonlinear iteration, 最后才判断某次 PushParticlesandDeposit() 是物理推进、试探性 RHS 构造, 还是历史源项重建。

读者可以用下面这张决策图快速定位一个输入卡实际采用的时间合同:

```

flowchart TD
    A["WarpX::OneStep(t, dt)"] --> B{"m_implicit_solver?"}
    B -->|"yes"| C["ImplicitSolver::OneStep"]
    C --> C1["nonlinear iteration / RHS source rebuild"]
    C1 --> C2["one physical step committed"]
    B -->|"no"| D{"psatd.JRhom?"}
    D -->|"yes"| E["OneStep_JRhom"]
    E --> E1["multiple relative-time rho/J deposits"]
    E1 --> E2["PSATD spectral advances"]
    D -->|"no"| F{"AMR subcycling?"}
    F -->|"yes"| G["OneStep_sub1"]
    G --> G1["fine: two dt/2 advances"]

```

```

G1 --> G2["restrict/add fine source to coarse"]
G2 --> G3["coarse: one dt advance"]
F --> |"no" | H["OneStep_nosub"]
H --> H1["push/deposit -> SyncCurrentAndRho"]
H1 --> H2["FDTD or PSATD field advance"]

```

图中三种“内部多次执行”含义不同: implicit 的重复是求解器试探, JRhom 的重复是同一物理时间步内的源项时间积分, subcycling 的重复则是真实的细层物理推进。只有最后完成的外层调用才代表一次可提交的物理时间步。

2.7 本章后的源码阅读入口

读者现在可以从三个源码入口继续:

目标	入口
看外层时间步如何组织	../warpx/Source/Evolve/WarpXEvolve.cpp:147-390
看显式电磁无 subcycling 的标准 step	../warpx/Source/Evolve/WarpXEvolve.cpp:507-646
看主循环如何进入粒子容器	../warpx/Source/Evolve/WarpXEvolve.cpp:1311-1415
看两级 AMR subcycling 的细/粗层时间组织	../warpx/Source/Evolve/WarpXEvolve.cpp:1040-1265
看 JRhom 的多次相对时间沉积与谱推进	../warpx/Source/Evolve/WarpXEvolve.cpp:843-1042
看 implicit RHS 中的粒子推进与 source synchronization	../warpx/Source/FieldSolver/ImplicitSolvers/ImplicitSolver

2.8 参数示例与最小运行案例

如果把本章压回一个最小、可运行、可验证的输入骨架, 当前最合适的入口还是:

- ../warpx/Examples/Tests/langmuir/inputs_test_1d_langmuir_multi

它把本章真正讨论的五类量都放在同一个最小问题上:

- geometry.dims = 1
- algo.maxwell_solver = yee
- algo.current_deposition = esirkepov
- algo.field_gathering = energy-conserving
- warpx.cfl = 0.8
- max_step = 80
- 周期场边界

也就是说, 本章的抽象讨论并不是悬空的。这里的:

- leapfrog 时间层
- \omega_p
- \lambda_D
- FDTD curl 更新
- rho/J 连续性合同

都能在这条最小 Langmuir 主线上落到真实输入。

当前本地最小运行记录已经建立在:

- /Volumes/PHILIPS/programs/PIC/PIC-tutor/runs/stage-c-validation/langmuir_1d

对应命令:

```

env OMP_NUM_THREADS=1 FI_PROVIDER=tcp \
/Volumes/PHILIPS/programs/PIC/warpx/build_full/bin/warpx.1d.MPI.OMP.DP.PDP.OPMD.FFT.EB.QED.GENQEDTABLES
/Volumes/PHILIPS/programs/PIC/warpx/Examples/Tests/langmuir/inputs_test_1d_langmuir_multi

```

它对本章最重要的不是“程序成功退出”, 而是:

- 解析场相对误差 $1.7027848999745115e-3 < 5e-2$
- $\text{divE-rho}/\epsilon_0$ 相对误差 $8.34503170903001e-12 < 1e-11$

所以本章已经具备:

- 参数示例
- 最小运行案例
- 物理检查量

而不是只停在连续模型和离散方程层。

2.9 本章当前基础文献清单

本章当前已经直接依托的基础来源是：

- Birdsall 1985
 - leapfrog 最小教学骨架
 - $\omega_p \Delta t$
 - $v_t \Delta t / \Delta x$
 - finite-grid / aliasing / heating 主线
- Dawson 1983
 - electrostatic / full EM 的数值模型边界
 - full EM 时间步与 light mode / CFL 的关系
 - Darwin 作为 radiation-free low-frequency route

本章当前还没有直接依托其正文细节的一手来源是：

- Yee 1966
 - 当前只到 metadata/acquisition 边界
- Hockney-Eastwood
 - 当前原书与 article-level fallback 仍未 materialize 为项目内 full-text + MinerU 资产

更完整的基础章节文献状态、local-materialization 状态和“可直接作为正文证据/只可作待补边界”的分工，统一见：

- 基础章节文献清单

2.10 进一步阅读与练习

进一步阅读：

- 03-warpx-evolve.md：把本章的 PIC loop 抽象结构接到 `main.cpp -> WarpX::Evolve()` 的真实调用链。
- Birdsall 1985 中文讲解：继续看 $\omega_p \Delta t$ 、 $\lambda_D / \Delta x$ 、finite-grid aliasing 和 numerical heating。
- Dawson 1983 中文讲解：继续看 full EM、Darwin、quiet start 和 statistical measurements 如何改变“PIC 总循环”的解释方式。

练习题：

- 解释为什么 `ComputeDt()` 给出的可运行时间步，不自动保证 $\omega_p \Delta t \ll 1$ 。
- 用本章的 λ_D 讨论说明：为什么 $\Delta x \gg \lambda_D$ 时，即使主循环稳定，也可能已经不是同一个物理 plasma。
- 对照 00-langmuir-wave.md，指出 `analysis_1d.py` 的两条核心断言分别对应本章哪两类理论边界。

下一章将逐段解释这些源码，并把 `main.cpp`、`WarpX` 单例、`ReadParameters()`、`InitData()`、`ComputeDt()` 和 `Evolve()` 接成完整调用链。

3. WarpX 主演化路径：生命周期、初始化与 Evolve()

本章开始进入 WarpX 源码。目标不是概括“WarpX 有一个 Evolve 函数”，而是建立一个可复查的调用图：程序从 `main.cpp` 进入，如何构造 WarpX 对象，如何读取参数和初始化数据，如何计算步长，最后如何在 `WarpXEvolve.cpp` 中把一个个 PIC step 推进下去。

本章基于本地只读源码 `../warpx`，当前已读证据整理在 `notes/code-reading/evolve/00-lifecycle-and-callgraph.md` 和 `notes/code-reading/evolve/02-evolve-source-evidence.md`。

本章当前依据的 WarpX 源码版本是：

- `../warpx`
- 分支：pkuHEDPbranch
- commit：8c488b1a9

v0.2 校准说明: 本章已按当前 checkout 复核 main.cpp、WarpX.H、WarpX.cpp、Source/Evolve/WarpXEvolve.cpp 与 Source/Initialization/WarpXInitData.cpp 的主链行号。本章现已补入 OneStep_sub1()、PSATD-JRhom 和 implicit solver 的主入口与时间组织边界; 更细的场算法公式、粒子 nonlinear solve 参数和 mass-matrix kernel 仍分别留在第 5/6 章及配套源码笔记中。

3.1 顶层入口: main.cpp

WarpX 的可执行入口在 ../warpx/Source/main.cpp。主函数的控制流非常短:

```
initialize_external_libraries(argc, argv)
warpx = WarpX::GetInstance()
warpx.InitData()
warpx.Evolve()
is_warpx_verbose = warpx.Verbose()
WarpX::Finalize()
print timer if verbose
finalize_external_libraries()
```

对应行号:

行号	代码含义
Source/main.cpp:20	初始化外部库。这里包括 AMReX/MPI/GPU runtime 等 WarpX 依赖的底层运行环境。
Source/main.cpp:27	auto& warpx = WarpX::GetInstance();, 取得全局 WarpX 单例。
Source/main.cpp:28	warpx.InitData();, 把参数、网格、场、粒子、诊断和边界准备好。
Source/main.cpp:29	warpx.Evolve();, 进入时间推进主循环。
Source/main.cpp:31	WarpX::Finalize();, 释放 WarpX 单例。
Source/main.cpp:41	结束外部库。

本段源码原文如下, 位置为 ../warpx/Source/main.cpp:20-41:

```
warpx::initialization::initialize_external_libraries(argc, argv);

auto const strt_total = amrex::second();
auto strt = strt_total;
BL_PROFILE_INITIALIZE();

auto& warpx = WarpX::GetInstance();
warpx.InitData();
warpx.Evolve();
const auto is_warpx_verbose = warpx.Verbose();
WarpX::Finalize();

auto const end = amrex::second() - strt;
if (is_warpx_verbose) {
    amrex::Print() << "Total Time" << " " << end << '\n';
}
BL_PROFILE_FINALIZE();

warpx::initialization::finalize_external_libraries();
```

逐块看, 这段代码只有一个物理模拟对象 warpx。InitData() 之前还没有进入时间推进; Evolve() 返回之后模拟已经结束, 后续只做计时、profiling 和库资源释放。这里没有任何粒子或场循环, 因为 WarpX 把模拟状态封装在 WarpX 单例内部。

这个入口说明: WarpX 的主程序不直接管理粒子数组或场数组; 它只管理生命周期。真正的模拟状态集中在 WarpX 对象及其持有的 MultiParticleContainer、field register、diagnostics、solver、PML 等成员中。

3.2 WarpX 类与单例构造

WarpX 类定义在 `../warpx/Source/WarpX.H`。关键声明包括：

行号	声明	作用
Source/WarpX.H:85	<code>class WarpX : public amrex::AmrCore</code>	WarpX 以 AMReX 的 AMR 基类为基础。
Source/WarpX.H:89	<code>static WarpX& GetInstance();</code>	全局单例入口。
Source/WarpX.H:97	<code>static void Finalize();</code>	删除单例。
Source/WarpX.H:130	<code>void InitData();</code>	初始化模拟数据。
Source/WarpX.H:132	<code>void Evolve(int numsteps=-1);</code>	外层时间推进。
Source/WarpX.H:1041-1062	<code>OneStep、OneStep_nosub、OneStep_sub1、OneStep_JRhom</code>	主循环内部的单步推进路径。

单例实现位于 `../warpx/Source/WarpX.cpp`：

- Source/WarpX.cpp:298-305: `GetInstance()` 检查 `m_instance`，为空则调用 `MakeWarpX()`。
- Source/WarpX.cpp:317-320: `Finalize()` 调 `ResetInstance()` 删除对象。
- Source/WarpX.cpp:322-350: 构造函数设置 `m_instance=this`，初始化 `warning manager`，调用 `ReadParameters()`，做向后兼容处理，初始化 `EB`，建立 `istep/nsubsteps/t_new/t_old/dt` 数组，并创建 `MultiParticleContainer`。

构造函数里最重要的一行是 Source/WarpX.cpp:329 的 `ReadParameters()`。这意味着 `solver` 类型、边界、步长策略、滤波、静电/电磁模式等会在 `InitData()` 前决定。

3.3 ReadParameters(): 主循环分支的来源

`WarpX::ReadParameters()` 从 `../warpx/Source/WarpX.cpp:547` 开始。完整参数系统很大，本章只列出直接影响主循环的部分。

源码位置	参数或逻辑	对主循环的影响
Source/WarpX.cpp:550-552	<code>max_step、stop_time</code>	<code>Evolve()</code> 用它们限制循环终点。
Source/WarpX.cpp:563-565	<code>algo.maxwell_solver</code>	选择 PSATD、Yee、CKC、ECT、HybridPIC、None 等路径。
Source/WarpX.cpp:581-595	PSATD 不支持 PEC/PMC 的断言	<code>solver</code> 选择会反过来限制边界条件。
Source/WarpX.cpp:598	<code>algo.evolve_scheme</code>	决定 <code>explicit</code> 、 <code>theta implicit</code> 等演化框架。
Source/WarpX.cpp:679-684	<code>warpx.cfl、verbose、regrid_int、do_subcycling</code>	控制步长、输出、重网格和 AMR 子循环。
Source/WarpX.cpp:729-733	<code>warpx.do_electrostatic</code>	静电 <code>solver</code> 非空时把 <code>electromagnetic solver</code> 设为 <code>None</code> 。
Source/WarpX.cpp:796-812	<code>const_dt、max_dt、dt_update_interval</code>	控制固定步长和运行中步长更新。
Source/WarpX.cpp:814-828	<code>filter</code> 默认开关	显式 <code>scheme</code> 默认滤波，隐式 <code>scheme</code> 默认关闭滤波。

这里有一个阅读原则：输入文件里的参数名只是表层。要理解参数的真实含义，必须追到 `ReadParameters()` 中它如何被读入、被断言约束、被改写，并进一步影响 `Evolve()`、`ComputeDt()` 或 `solver` 对象。

3.3.1 构造函数只建“跨 level 外壳”，不直接建完整网格数据

仅从 `ReadParameters()` 还看不出一个常见实现边界：`WarpX::WarpX()` 构造函数里虽然已经决定了 `solver` 路线，但此时还没有有效的 `BoxArray` 和 `DistributionMapping`。源码在 `../warpx/Source/WarpX.cpp:337-341` 明说：

```
// Geometry on all levels has been defined already.  
// No valid BoxArray and DistributionMapping have been defined.  
// But the arrays for them have been resized.
```

所以构造函数能做的是：

- 创建不依赖具体网格盒划分的跨 level 对象：
 - MultiParticleContainer
 - m_electrostatic_solver
 - m_hybrid_pic_model
 - solver 指针数组外壳
- 按 maxLevel()+1 先 resize 各种 level 容器：
 - istep
 - nsubsteps
 - t_old/t_new
 - dt

但它不能在这里直接分配真正的 Efield_fp/Bfield_fp/current_fp/rho_fp。这些字段必须等到 MakeNewLevelFromScratch() -> AllocLevelData() -> AllocLevelMFs(), 拿到真实的 ba/dm、index type 和 guard cell 之后才创建。

这也是为什么：

- effective potential 在构造期只创建 EffectivePotentialES 对象；
- hybrid PIC 在构造期只创建 HybridPICModel 对象；
- implicit solver 即使已经存在，也还没分配 mass matrices。

真正的 level 级附加字段要到后面才各自落地。

3.3.2 AllocLevelData() / AllocLevelMFs() 里，四条 solver 分支的落点不同

AllocLevelData() 位于 ../warpx/Source/WarpX.cpp:2271 之后。它先调用 guard_cells.Init(...), 再进入 AllocLevelMFs(...)。这一层真正决定的是：哪些 MultiFab 需要随着 level 一起出生。

先看隐式分支。AllocLevelMFs() 并不会一次性把全部隐式字段都分好，它只先放两类最基础的 level 数据：

```
if (m_implicit_solver) {
    m_fields.alloc_init(FieldType::current_fp_non_suborbit, Direction{0}, lev,
                        amrex::convert(ba, jx_nodal_flag), dm, ncomps, ngJ, 0.0_rt);
    ...
    m_fields.alloc_init(FieldType::E_old, Direction{2}, lev,
                        amrex::convert(ba, Ez_nodal_flag), dm, ncomps, ngEB, 0.0_rt);
}
```

也就是说，隐式路线在 level 分配期先保存：

- 非 suborbit 粒子的独立电流容器 current_fp_non_suborbit
- 旧电场时间层 E_old

而更重的 MassMatrices_X/Y/Z/MassMatrices_PC 不是在这里分配，而是在后续 ImplicitSolver::InitializeMassMatrices() 里，等标准 current_fp/Efield_fp 的 index type、guard cells 和 deposition 算法都稳定后再决定组件数。

再看 hybrid PIC。它在构造期只有一个模型对象，真正的 level 字段由 HybridPICModel::AllocateLevelMFs(...) 填入共享的 m_fields register：

```
if (WarpX::electromagnetic_solver_id == ElectromagneticSolverAlgo::HybridPIC)
{
    m_hybrid_pic_model->AllocateLevelMFs(
        m_fields,
        lev, ba, dm, ncomps, ngJ, ngRho, ngEB, jx_nodal_flag, jy_nodal_flag,
        jz_nodal_flag, rho_nodal_flag, Ex_nodal_flag, Ey_nodal_flag, Ez_nodal_flag,
        Bx_nodal_flag, By_nodal_flag, Bz_nodal_flag
    );
}
```

这一调用会分配：

- hybrid_electron_pressure_fp
- hybrid_rho_fp_temp
- hybrid_current_fp_temp

- hybrid_current_fp_plasma
- 可选的 hybrid_current_fp_external

若启用 add_external_fields, 还会进一步分配 external vector potential 相关字段。也就是说, hybrid 不是在 WarpX 根层自己持有一套独立网格, 而是把专用状态嵌进统一的 field register。

再看 effective potential electrostatic solver。这条线在构造期创建了 EffectivePotentialES 对象, 但 level 分配期没有额外的 effective_potential_* 专用字段。它复用的是静电共享合同:

```
if( (electrostatic_solver_id == ElectrostaticSolverAlgo::LabFrame) ||
    (electrostatic_solver_id == ElectrostaticSolverAlgo::LabFrameElectroMagnetostatic) ||
    (electrostatic_solver_id == ElectrostaticSolverAlgo::LabFrameEffectivePotential) ||
    (electromagnetic_solver_id == ElectromagneticSolverAlgo::HybridPIC) ) {
    rho_ncomps = ncomps;
}

if (electrostatic_solver_id == ElectrostaticSolverAlgo::LabFrame ||
    electrostatic_solver_id == ElectrostaticSolverAlgo::LabFrameElectroMagnetostatic ||
    electrostatic_solver_id == ElectrostaticSolverAlgo::LabFrameEffectivePotential )
{
    m_fields.alloc_init(FieldType::phi_fp, lev, amrex::convert(ba, phi_nodal_flag), dm,
                        ncomps, ngPhi, 0.0_rt );
}
```

所以 effective potential 在根层最关键的差异不是“多了什么字段”, 而是后续 solver 如何解释和消费 rho_fp/phi_fp。

最后是 PML。它同样不是在 AllocLevelMFs() 里出生。真正创建 PML 对象是在 InitFromScratch() 的最后一步:

```
AmrCore::InitFromScratch(time); // This will call MakeNewLevelFromScratch
...
mypc->AllocData();
mypc->InitData();

InitPML();
```

这条顺序说明:

- 先有普通 level 的 fields / particles
- 后有作为边界子系统的 PML

因此 PML 是初始化末端附加上的 patch 级边界对象, 而不是和 Efield_fp/Bfield_fp 同时由 AllocLevelMFs() 常规分配的主网格字段。

把这四条线压成同一张状态图, 会更不容易误读:

```
flowchart TD
    A["MakeWarpX()"] --> B["WarpX::WarpX()"]
    B --> B1["ReadParameters()"]
    B --> B2[" 创建跨-level 外壳:  
mypc  
electrostatic solver object  
hybrid model object  
solver pointer arrays"]
    B --> B3[" 只 resize per-level containers:  
istep/nsubsteps  
t_old/t_new  
dt"]
    B --> C["AmrCore::InitFromScratch()"]
    C --> D["MakeNewLevelFromScratch()"]
    D --> E["AllocLevelData()"]
    E --> E1["guard_cells.Init()"]
    E --> F["AllocLevelMFs()"]
```

```

F --> F1["standard fields:
Efield_fp/Bfield_fp/current_fp/rho_fp/phi_fp"]
F --> F2["implicit first-layer fields:
current_fp_non_suborbit
E_old"]
F --> F3["hybrid first-layer fields:
electron pressure
temp rho/current
plasma/external current"]
F --> F4["effective potential:
no extra level-only field family;
reuse rho_fp/phi_fp"]

C --> G["m_implicit_solver->Define()
CreateParticleAttributes()"]
C --> H["mypc->AllocData()
mypc->InitData()"]
C --> I["InitPML()"]
I --> I1[" 附加 PML patch objects;
not born inside AllocLevelMFs()"]

G --> J["later implicit linear-stage prep"]
J --> J1["InitializeMassMatrices():
MassMatrices_X/Y/Z
MassMatrices_PC"]

```

同样也可以压成一张对象落点表：

路线	构造期 WarpX::WarpX()	level 分配期 AllocLevelMFs()	初始化末段 / 后续
标准场	只准备容器外壳	Efield_fp/Bfield_fp/current_fp/rho_fp/phi_fp	创建 InitLevelData() 填充物理值
effective potential	创建 EffectivePotentialES 对象	不额外分专用字段，复用 rho_fp/phi_fp	由静电求解器后续消费
hybrid PIC	创建 HybridPICModel 对象	hybrid_* 字段写入共享 m_fields	HybridPICModel::InitData() 编译 parser、准备外加电流/矢势
implicit	solver 对象已存在	只分 current_fp_non_suborbit、E_old	Define()、CreateParticleAttributes()，再到 InitializeMassMatrices()
PML	仅参数/开关已知	不在此时创建 PML patch	InitPML() 用真实 boxArray/dm/dt/m_fields 延后创建

3.4 InitData(): 把状态准备到可推进

WarpX::InitData() 位于 ../warpx/Source/Initialization/WarpXInitData.cpp:793-949。它不是简单分配内存，而是把一个模拟从“参数已读”变成“可以推进第一步”。

核心顺序如下：

行号	操作	解释
793-800	进入 InitData(), 检查 MPI thread level	并行运行前的运行环境检查。

行号	操作	解释
810-814	创建 MultiDiagnostics 和 MultiReducedDiags	诊断系统在初始化早期建立。
824-830	非 restart: ComputeDt()、打印步长网格、InitFromScratch()、InitDiagnostics()	从头运行时先确定步长，再建立网格/粒子/诊断。
831-837	restart: InitFromCheckpoint()、打印步长网格、PostRestart()	checkpoint 恢复不走完全相同的初始化路径。
839-847	ComputeMaxStep()、PML factors、NCI corrector、buffer masks	准备停止步数、吸收边界和数值不稳定修正。
849-863	宏观介质、静电 solver、HybridPIC 初始化	solver 相关对象拿到场布局 and 参数。
865-878	网格摘要、guard cell 检查、打印 PIC 参数、写 used inputs	把运行状态和输入记录下来。
880-913	初始 div cleaning、自洽静电/磁静场、外场叠加	从头运行时在第一个 step 前建立初始场。
918-928	初始 full/reduced diagnostics	允许输出第 0 步或 restart 后诊断。
930-948	性能提示和 solver issue 检查	给出已知风险提示。

InitFromScratch() 在 Source/Initialization/WarpXInitData.cpp:993-1009。它调用 AmrCore::InitFromScratch(time) 建立 AMR level, 然后让 mypc->AllocData() 和 mypc->InitData() 初始化粒子, 最后初始化 PML。

3.5 ComputeDt(): 步长不是一个固定常数

WarpX::ComputeDt() 在 ../warpX/Source/Evolve/WarpXComputeDt.cpp:45-108。

逻辑可以分成四类:

1. HybridPIC 必须显式给出 warpx.const_dt, 见 :49-50。
2. 纯静电或无 Maxwell solver 时, 必须给出 const_dt 或激活 dt_update_interval, 见 :51-55。
3. 若用户给了 const_dt, 直接使用, 见 :62-63。
4. 否则按 solver 计算 CFL 限制: 静电/PSATD 用最小 cell size 与 c , FDTD 调用具体几何和算法的 ComputeMaxDt(), 见 :64-97。

最终 dt 被 resize 到 max_level+1, 见 :100-101。若启用 subcycling, 粗层步长由细层步长乘 refinement ratio 得到, 见 :103-107。

这四类其实可以再压成一张更明确的决策表, 而不只是“按 CFL 算”:

条件	dt 来源	说明
warpx.const_dt 已设置	const_dt	直接覆盖所有 CFL 估计; 稳定性由用户自己保证。
HybridPIC	必须是 const_dt	Hybrid 路线不接受“缺省光速 CFL”。
electrostatic 且设置 max_dt	max_dt	静电路径可直接把 max_dt 当作初值。
electrostatic 且未设置 max_dt	cfl*min(dx)/c	这里只是 fallback 尺度, 后续仍可由粒子速度更新。
PSATD	cfl*min(dx)/c	谱 solver 这里用最小网格尺度给出初始步长。
Cartesian Yee/ECT	cfl*CartesianYeeAlgorithm::ComputeMaxDt(dx, dz, dt)	Cartesian FDTD 由具体差分算法给出稳定上限。
Cartesian CKC	cfl*CartesianCKCAlgorithm::ComputeMaxDt(dx, dz, dt)	CKC 是 ECT 的复用 Yee CFL。
collocated/nodal	cfl*CartesianNodalAlgorithm::ComputeMaxDt(dx, dz, dt)	collocated 的的稳定上限单独计算。
RZ/RCYLINDER Yee	cfl*CylindricalYeeAlgorithm::ComputeMaxDt(dx, dz, dt, azimuthal_modes)	RZ 是 ECT 的复用 Yee CFL。
RSPHERE Yee	cfl*SphericalYeeAlgorithm::ComputeMaxDt(dx, dz, dt)	RSPHERE 的的稳定上限单独给稳定上限。

因此, ComputeDt() 不只是“取最小网格长度除以光速”, 而是在参数层先决定有没有用户强制时间步, 再按 solver 家族和几何去选真正的稳定上限公式。

运行中自适应步长在 WarpX::UpdateDtFromParticleSpeeds(), 位于 Source/Evolve/WarpXComputeDt.cpp:115-142。它从 mypc->maxParticleVelocity() 得到最大粒子速度, 用

$$\Delta t_{\text{new}} = \text{CFL} \frac{\Delta x_{\min}}{v_{\max}}$$

更新 finest level 的 dt, 再向粗层回推。

这里还要补一条参数层边界: warpx.const_dt 与 warpx.dt_update_interval 在 ReadParameters() 中就是互斥的。也就是说, 运行时 adaptive timestep 不是“在固定步长上再做微调”, 而是一条和 const_dt 完全不同的时间组织路线。Evolve() 里只有当 m_dt_update_interval.contains(step+1) 为真时, 才会在步首调用 UpdateDtFromParticleSpeeds()。

3.6 Evolve() 外层时间步

WarpX::Evolve() 位于 ../warpx/Source/Evolve/WarpXEvolve.cpp:147-390。它不是单纯调用 OneStep(), 而是在每个 step 前后管理大量状态。

外层结构是:

```
cur_time = t_new[0]
numsteps_max = max_step or istep[0] + numsteps
for step in range while cur_time < stop_time:
    check signals
    multi_diags->NewIteration()
    run beforestep callback
    CheckLoadBalance(step)
    maybe update dt from particle speeds
    ExplicitFillBoundaryEBUpdateAux()
    hybrid initialization if first step
    field ionization / QED / particle injection
    OneStep(cur_time, dt[0], step)
    resampling / mirrors
    increment istep and time
    diagnostics prepack, moving window, particle boundary handling
    electrostatic or hybrid field solve if selected
    optional velocity synchronization for diagnostics
    afterstep callback, diagnostics, warnings, signals, stop check
```

关键行号:

行号	操作	读法
155-166	初始化 cur_time 和循环边界	numsteps=-1 时使用全局 max_step。
171-175	信号检查、诊断新迭代	支持运行中断、checkpoint 和诊断状态更新。
192-205	callback、负载均衡、可选步长更新	更新步长前需要同步粒子速度时间层。
208-212	ExplicitFillBoundaryEBUpdateAux()	显式 scheme 为后续 field gather 准备场。
222-232	field ionization、QED、particle injection	多物理事件在 OneStep() 前改变粒子集合。
234-235	OneStep(cur_time, dt[0], step)	进入单步推进分派。
248-259	更新 istep 和 t_old/t_new	单步推进后更新时间状态。
261-276	诊断预处理、moving window、粒子边界	OneStep() 后的工程处理同样影响物理结果。
289-323	静电或 HybridPIC 的场解	非标准电磁路径的场更新位置不同。
326-333	诊断需要时同步粒子速度	为输出把 p 与 x 放到同步时间层。
336-350	reduced/full diagnostics 和 callback	诊断写出发生在本步状态更新之后。
352-372	未使用输入检查、计时、信号、停止	第一步后检查输入 typo, 最后判断是否退出。

注意 Evolve() 中多物理和诊断并不都在 OneStep() 内部。比如 field ionization、QED 和 particle injection 在 OneStep() 之前, resampling、moving window、粒子边界和某些 electrostatic/hybrid 场解在 OneStep() 之后。

3.6.1 步末 moving window: 连续坐标与整数网格平移

moving window 的完整精读见 notes/code-reading/evolve/05-moving-window.md。这里先把它放回 Evolve() 主循环中: MoveWindow(step+1, move_j) 发生在 OneStep() 完成、cur_time 和 t_new 更新之后, 粒子边界处理之前。对应源码为 ../warpx/Source/Evolve/WarpXEvolve.cpp:252-276:

```

cur_time += dt[0];

ShiftGalileanBoundary();

// sync up time
for (int i = 0; i <= max_level; ++i) {
    t_old[i] = t_new[i];
    t_new[i] = cur_time;
}
multi_diags->FilterComputePackFlush( step, false, true );

const bool move_j = m_is_synchronized;
// If m_is_synchronized we need to shift j too so that next step we can evolve E by dt/2.
// We might need to move j because we are going to make a plotfile.
const int num_moved = MoveWindow(step+1, move_j);

MoveWindow() 的第一层逻辑是维护一个连续窗口位置 moving_window_x, 但只有当它相对当前几何左边界跨过整数个 cell 时才真正平移网格数据。源码为 ../warpx/Source/Utils/WarpXMovingWindow.cpp:372-397:

if (!moving_window_active(step)) { return 0; }

// Update the continuous position of the moving window,
// and of the plasma injection
moving_window_x += (moving_window_v - WarpX::beta_boost * PhysConst::c)/(1 - moving_window_v * WarpX::beta_boost);
const int dir = moving_window_dir;

// Update current injection position for all containers
::UpdateInjectionPosition(*mypc, gamma_boost, beta_boost, boost_direction, moving_window_dir, dt[0]);

// Update antenna position for all lasers
// TODO Make this specific to lasers only
mypc->UpdateAntennaPosition(dt[0]);

// compute the number of cells to shift on the base level
amrex::Real new_lo[AMREX_SPACEDIM];
amrex::Real new_hi[AMREX_SPACEDIM];
const amrex::Real* current_lo = geom[0].ProbLo();
const amrex::Real* current_hi = geom[0].ProbHi();
const amrex::Real* cdx = geom[0].CellSize();
const int num_shift_base = static_cast<int>((moving_window_x - current_lo[dir]) / cdx[dir]);

if (num_shift_base == 0) { return 0; }

```

这段代码中的速度变换是相对论速度合成公式：

$$v'_w = \frac{v_w - \beta_b c}{1 - v_w \beta_b / c}.$$

因此 boosted-frame 模拟中窗口速度不是简单使用输入的 moving_window_v。输入参数在 read_moving_window_parameters() 中先由以 c 为单位的无量纲数转成 SI 速度；运行时再按 boost 速度变换到模拟坐标系。

active 判定本身也不是模糊的“某个阶段窗口有效”，而是源码里一个明确的闭开区间：

$$\text{start_moving_window_step} \leq n < \text{end_moving_window_step},$$

若 end_moving_window_step < 0 则表示没有终止步。这也是为什么 Evolve() 调的是 MoveWindow(step+1, ...): 窗口平移是这一步结束后、下一步开始前的状态更新。

当 num_shift_base != 0 时, MoveWindow() 调用 ResetProbDomain() 更新几何域,并用 shiftMF() 平移 E/B/current/PML/F/G/rho/flux 等 MultiFab。shiftMF() 的核心赋值为 ../warpx/Source/Utils/WarpXMovingWindow.cpp:180-190:

```
amrex::Box dstBox = mf[mfi].box();
if (num_shift > 0) {
    dstBox.growHi(dir, -num_shift);
} else {
    dstBox.growLo(dir, num_shift);
}
AMREX_PARALLEL_FOR_4D ( dstBox, nc, i, j, k, n,
{
    dstfab(i,j,k,n) = srcfab(i+shift.x,j+shift.y,k+shift.z,n);
})
即
```

$$F_{\text{new}}(\mathbf{i}) = F_{\text{old}}(\mathbf{i} + N_{\text{shift}} \hat{e}_{\text{dir}}).$$

新露出的边界层不是随便置零。对外部场, shiftMF() 可以用常量外场或 parser 外场重新初始化;对背景粒子, MoveWindow() 构造整数 cell 宽度的 particleBox 并调用 pc.ContinuousInjection(particleBox)。这解释了 moving window 的数值设计: 连续窗口位置负责物理速度, 整数 cell 平移负责保持网格离散结构和宏粒子 spacing。

这里还要和步末的 ContinuousFluxInjection(cur_time, dt[0]) 分开。二者不是同一件事:

- moving-window continuous injection: 在新露出的体网格区域里补背景体分布;
- continuous flux injection: 从定义好的注入面持续打入粒子通量。

前者是“窗口移动后补齐新计算域”, 后者是“边界源项继续往域内送粒子”。

3.7 OneStep(): 求解器分派器

WarpX::OneStep() 位于 ../warpx/Source/Evolve/WarpXEvolve.cpp:392-495。它按 solver 和 AMR 情况分派:

```
if m_implicit_solver:
    m_implicit_solver->OneStep(...)
else:
    if electromagnetic_solver_id is None or HybridPIC:
        push particles, optionally split collisions, skip deposition
    else:
        if finest_level == 0:
            OneStep_nosub or OneStep_JRhom
        else:
            OneStep_nosub or OneStep_sub1
```

这段代码体现了 WarpX 的设计: OneStep() 不直接写某一种 PIC 算法的全部细节, 而是先把路径分开。

分支	源码位置	含义
implicit solver electrostatic / HybridPIC	Source/Evolve/WarpXEvolve.cpp:398-409 :405-445	交杂式 solver 自己推进一整步。 粒子推进但跳过标准电磁沉积路径, 场解在外层后处理。
标准电磁无 MR	:448-467	进入 OneStep_nosub() 或 PSATD-JRhom。
有 MR 无 subcycling	:469-474	仍进入 OneStep_nosub(), 所有 level 使用同一步长推进。
有 MR 且 subcycling	:475-492	进入 OneStep_sub1(), 当前限制最多两个 level。

几个断言值得后续单独讲:

- JRhom 与 split momentum collision 当前不能组合, 见 Source/Evolve/WarpXEvolve.cpp:456-459。
- subcycling 当前要求 `finest_level == 1`, 见 :477-480。
- subcycling 与 split momentum collision 当前也不能组合, 见 :481-484。

这些不是文档层面的“建议”, 而是源码级功能边界。

还应补一条输入层边界: `psatd.JRhom` 不是布尔开关, 而是一个编码了源项时间模型的字符串。`ReadParameters()` 里它会同时决定:

- J 的时间依赖是 constant / linear / quadratic;
- rho 的时间依赖是 constant / linear / quadratic;
- 一个 PIC step 里切成多少个 JRhom subinterval。

因此 JRhom 开启后, 后续变的不是“另一个小优化开关”, 而是 `OneStep()` 内部的时间组织、rho_fp 的组件语义和谱空间源项更新公式。

3.8 OneStep_nosub(): 显式电磁标准路径

`WarpX::OneStep_nosub()` 位于 `../warpx/Source/Evolve/WarpXEvolve.cpp:507-646`。这是本书第一个需要逐行读懂的核心函数。

它的结构分为四段。

第一段: 粒子推进、碰撞与沉积, 见 :515-557。

源码原文:

```
// Push particle from  $x^{\{n\}}$  to  $x^{\{n+1\}}$ 
//                               from  $p^{\{n-1/2\}}$  to  $p^{\{n+1/2\}}$ 
// Deposit current  $j^{\{n+1/2\}}$ 
// Deposit charge density  $\rho^{\{n\}}$ 

ExecutePythonCallback("beforedeposition");

// with collisions placed in the middle of the momentum push
if (m_collisions_split_momentum_push) {
    // push particles (half momentum)
    PushParticlesandDeposit(
        a_cur_time,
        /*skip_deposition=*/true,
        PositionPushType::None,
        MomentumPushType::FirstHalf
    );
    // perform particle collisions
    ExecutePythonCallback("beforecollisions");
    mypc->doCollisions(a_step, a_cur_time, a_dt);
    ExecutePythonCallback("aftercollisions");

    // push particles (full position and half momentum)
    PushParticlesandDeposit(
        a_cur_time,
        /*skip_deposition=*/false,
        PositionPushType::Full,
        MomentumPushType::SecondHalf
    );
}
else {
    ExecutePythonCallback("beforecollisions");
    mypc->doCollisions(a_step, a_cur_time, a_dt);
    ExecutePythonCallback("aftercollisions");

    PushParticlesandDeposit(
        a_cur_time,
```

```

        /*skip_deposition=*/false,
        PositionPushType::Full,
        MomentumPushType::Full
    );
}

```

ExecutePythonCallback("afterdeposition");

- 注释明确时间层: $\mathbf{x}^n \rightarrow \mathbf{x}^{n+1}$, $\mathbf{p}^{n-1/2} \rightarrow \mathbf{p}^{n+1/2}$, 沉积 $\mathbf{J}^{n+1/2}$ 和 ρ^n 。
- 如果 m_collisions_split_momentum_push 为真, 先做半个动量 push, 再碰撞, 再做位置 push 和后半动量 push。
- 否则先做碰撞, 再调用 PushParticlesandDeposit() 完整推进粒子并沉积。

第二段: 源项同步, 见 :559-572。

源码原文:

```

// Synchronize J and rho:
// filter (if used), exchange guard cells, interpolate across MR levels
// and apply boundary conditions
SyncCurrentAndRho();

```

```

// At this point, J is up-to-date inside the domain, and E and B are
// up-to-date including enough guard cells for first step of the field
// solve.

```

```

// For extended PML: copy J from regular grid to PML, and damp J in PML
if (do_pml && pml_has_particles) { CopyJPML(); }
if (do_pml && do_pml_j_damping) { DampJPML(); }

```

- SyncCurrentAndRho() 会处理滤波、guard cells、AMR 跨层插值/加和和边界。
- PML 若含粒子或需要电流阻尼, 会复制和阻尼 PML 中的电流。

第三段: PSATD 或 FDTD 场推进, 见 :574-642。

FDTD 分支的核心源码原文如下, 位置为 ../warpx/Source/Evolve/WarpXEvolve.cpp:606-628:

```

} else {
    EvolveF(0.5_rt * dt[0], /*rho_comp=*/0);
    EvolveG(0.5_rt * dt[0]);
    FillBoundaryF(guard_cells.ng_FieldSolverF);
    FillBoundaryG(guard_cells.ng_FieldSolverG);

    EvolveB(0.5_rt * dt[0], SubcyclingHalf::FirstHalf, a_cur_time); // We now have B~{n+1/2}
    FillBoundaryB(guard_cells.ng_FieldSolver, WarpX::sync_nodal_points);

    if (m_em_solver_medium == MediumForEM::Vacuum) {
        EvolveE(dt[0], a_cur_time); // We now have E~{n+1}
    } else if (m_em_solver_medium == MediumForEM::Macroscopic) {
        MacroscopicEvolveE(dt[0], a_cur_time); // We now have E~{n+1}
    } else {
        WARPX_ABORT_WITH_MESSAGE("Medium for EM is unknown");
    }
    FillBoundaryE(guard_cells.ng_FieldSolver, WarpX::sync_nodal_points);

    EvolveF(0.5_rt * dt[0], /*rho_comp=*/1);
    EvolveG(0.5_rt * dt[0]);
    EvolveB(0.5_rt * dt[0], SubcyclingHalf::SecondHalf, a_cur_time + 0.5_rt * dt[0]); // We now have B~{n+

```

- PSATD 走 PushPSATD(a_cur_time), 并处理 hybrid QED、PML、平均场、F/G guard cells。
- FDTD 走 EvolveF/G 半步、EvolveB(dt/2)、EvolveE(dt)、EvolveF/G 半步、EvolveB(dt/2)。

第四段：回调，见 :642。

- `afterEsolve` callback 在场更新后执行。

从物理角度看，`OneStep_nosub()` 做的事情是：用旧场 `gather` 推粒子，得到新位置和半步电流；把源项修整到网格和边界一致；再用这些源项推进电磁场。

3.9 SyncCurrentAndRho(): 源项不是沉积完就可用

`SyncCurrentAndRho()` 位于 `../warpX/Source/Evolve/WarpXEvolve.cpp:768-837`。

它的分支很重要：

- PSATD 且 `periodic single box` 时，会立即同步 J 和 ρ ，见 :773-785。
- PSATD 非 `periodic single box` 时，若没有 `current correction` 且不是 `Vay deposition`，才在这里同步，见 :787-797。
- `Vay deposition` 在特定情况下先只做 `filter`，见 :799-806。
- FDTD 路径总是 `SyncCurrent("current_fp")` 和 `SyncRho()`，见 :809-813。
- 最后对 ρ 和 J 施加 PEC 等边界处理，见 :815-836。

这说明“沉积”与“可用于场解”之间有一段不可忽略的工程层：滤波、guard cell、AMR 和边界会改变源项数组的可用状态。

3.10 PushParticlesandDeposit(): 进入粒子容器

`PushParticlesandDeposit()` 的两个重载位于 `../warpX/Source/Evolve/WarpXEvolve.cpp:1311-1415`。

第一层重载 :1311-1333 遍历所有 AMR level。第二层重载 :1335-1415 做三件事：

1. 根据 `do_current_centering` 和 `current_deposition_algo == Vay` 选择当前沉积字段名，见 :1349-1362。
2. 调用 `mypc->Evolve(...)`，把 `field register`、`level`、`字段名`、`时间`、`dt[lev]`、`subcycling half`、是否跳过沉积、位置/动量 `push` 类型传入粒子容器，见 :1364-1375。
3. 对 `RZ/柱/球几何` 做逆体积缩放，并在有流体物种时调用流体容器演化，见 :1377-1413。

因此，下一阶段逐行阅读必须从 `mypc->Evolve()` 继续进入 `Source/Particles`。`PushParticlesandDeposit()` 是主循环到粒子模块的接口，不是粒子 `pusher` 本身。

3.11 OneStep_sub1() 与 JRhom 的位置

完整精读见 `notes/code-reading/evolve/03-subcycling-and-jrhom.md`。这里先把两个特殊分支放回主循环时间层。

`OneStep_sub1()` 位于 `../warpX/Source/Evolve/WarpXEvolve.cpp:1043-1269`。当前 `subcycling` 只支持两个 `level` 和 `refinement ratio 2`：fine patch 用小步长推两次，coarse patch 和 mother grid 推一次，coarse 场使用两次 fine current 的平均效果。

源码注释直接说明这一点：

```
* This version of subcycling only works for 2 levels and with a refinement
* ratio of 2.
* The particles and fields of the fine patch are pushed twice
* (with dt[coarse]/2) in this routine.
* The particles of the coarse patch and mother grid are pushed only once
* (with dt[coarse]). The fields on the coarse patch and mother grid
* are pushed in a way which is equivalent to pushing once only, with
* a current which is the average of the coarse + fine current at the 2
* steps of the fine grid.
```

第一段 fine step 的核心源码如下：

```
PushParticlesandDeposit(fine_lev, cur_time, SubcyclingHalf::FirstHalf);
RestrictCurrentFromFineToCoarsePatch(
    m_fields.get_mr_levels_alldirs(FieldType::current_fp, finest_level),
    m_fields.get_mr_levels_alldirs(FieldType::current_cp, finest_level, skip_lev0_coarse_patch), fine_lev)
RestrictRhoFromFineToCoarsePatch(fine_lev);

EvolveB(fine_lev, PatchType::fine, 0.5_rt*dt[fine_lev], SubcyclingHalf::FirstHalf, cur_time);
EvolveF(fine_lev, PatchType::fine, 0.5_rt*dt[fine_lev], /*rho_comp=*/0);
```

```

EvolveE(fine_lev, PatchType::fine, dt[fine_lev], cur_time);
EvolveB(fine_lev, PatchType::fine, 0.5_rt*dt[fine_lev], SubcyclingHalf::SecondHalf, cur_time + 0.5_rt * dt);
EvolveF(fine_lev, PatchType::fine, 0.5_rt*dt[fine_lev], /*rho_comp=*/1);

```

因此 subcycling 的物理时间层是

$$\Delta t_0 = 2\Delta t_1.$$

fine level 在一个 coarse step 内走两个完整 leapfrog 小步。RestrictCurrentFromFineToCoarsePatch() 和 RestrictRhoFromFineToCoarsePatch() 把 fine 层沉积源项平均到 coarse patch; StoreCurrent()/RestoreCurrent() 保证 coarse 粒子自身 current 能在两个 half coarse step 中分别叠加对应的 fine contribution。

这里 StoreCurrent()/RestoreCurrent() 的角色需要说得更硬一点: subcycling 不是简单把 fine current 直接覆写 coarse current, 而是要先保留 coarse 粒子本身在大步时间层上的电流, 再把两次 fine-step 的 restriction 结果分别叠回 coarse half-step。否则 coarse mother grid 看到的就不是“一个 coarse 大步上等效的平均源项”, 而会把 coarse 自身电流和 fine 补偿混在一起。

OneStep_JRhom() 位于 ../warpx/Source/Evolve/WarpxEvolve.cpp:839-1041。它是 PSATD-JRhom 专用路径, 会多次沉积 J 和 ρ , 在谱空间推进字段, 并支持时间平均场。入口先断言 solver 必须是 PSATD, 并且粒子 push 时跳过标准沉积:

```

WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
    WarpX::electromagnetic_solver_id == ElectromagneticSolverAlgo::PSATD,
    "JRhom algorithm not implemented with the FDTD solver"
);

// Push particle from x~{n} to x~{n+1}
//           from p~{n-1/2} to p~{n+1/2}
const bool skip_deposition = true;
PushParticlesandDeposit(cur_time, skip_deposition);

```

// Initialize PSATD-JRhom loop:

```

// 1) Prepare E,B,F,G fields in spectral space
PSATDForwardTransformEB();
if (WarpX::do_dive_cleaning) { PSATDForwardTransformF(); }
if (WarpX::do_divb_cleaning) { PSATDForwardTransformG(); }

```

随后 JRhom 以

$$\delta t = \frac{\Delta t}{m}$$

为子区间, 多次在相对时间 t_deposit_current 和 t_deposit_charge 沉积源项:

```

const int n_deposit = WarpX::m_JRhom_subintervals;
const amrex::Real sub_dt = dt[0] / static_cast<amrex::Real>(n_deposit);
const int n_loop = (WarpX::fft_do_time_averaging) ? 2*n_deposit : n_deposit;

for (int i_deposit = 0; i_deposit < n_loop; i_deposit++)
{
    if (time_dependency_J != TimeDependencyJ::Constant) { PSATDMoveJNewToJOld(); }

    const amrex::Real t_deposit_current = (time_dependency_J == TimeDependencyJ::Linear) ?
        (i_deposit-n_deposit+1)*sub_dt : (i_deposit-n_deposit+0.5_rt)*sub_dt;

    const amrex::Real t_deposit_charge = (time_dependency_rho == TimeDependencyRho::Linear) ?
        (i_deposit-n_deposit+1)*sub_dt : (i_deposit-n_deposit+0.5_rt)*sub_dt;

    mypc->DepositCurrent( m_fields.get_mr_levels_alldirs(current_string, finest_level), dt[0], t_deposit_c

```



```
SyncCurrent("current_fp");
PSATDForwardTransformJ("current_fp", "current_cp");
```

所以 JRhom 的核心不是多次 gather, 也不是多次粒子 push, 而是用同一次粒子轨道在多个相对时间沉积源项, 让 PSATD 在一个 step 内看到更高阶的 $\tilde{\mathbf{J}}(t)$ 和 $\tilde{\rho}(t)$ 。

这条线路还有两条必须写清的功能限制:

1. JRhom 当前不支持 FDTD, 只能走 PSATD。
2. JRhom 当前和 current_correction、Vay deposition 都不兼容; 源码层会把 current_correction 关掉, 并显式禁止 Vay deposition 和 JRhom 组合。

所以这一支的真实定位是: 它不是“PSATD 上再附加一个任意可叠加的小修正”, 而是 PSATD 自身的一种替代性时间积分组织方式。

3.11.1 implicit 分支: 一次物理步包含多次试探性 source 重建

在 WarpX::OneStep() 中, 只要 m_implicit_solver 非空, 程序就不会进入 OneStep_nosub()、OneStep_sub1() 或 OneStep_JRhom(), 而是把整步交给 m_implicit_solver->OneStep(...)。当前入口位于 ../warpx/Source/Evolve/WarpXEvolve.cpp:30。

以 SemiImplicitEM::OneStep() 为代表, 隐式电磁步的控制流是:

顺序	源码动作	时间层/物理含义
1	SaveParticlesAtImplicitStepStart	保存 x^n, p^n , 供非线性迭代和最终提交使用
2	初始化 $E^{n+\theta}$ 猜测、保存 E_old	构造 solver 的中间场未知量, 而不是直接写最终 E^{n+1}
3	EvolveB($\Delta t/2$)	先把 WarpX 所有的磁场推进到半步
4	m_nlsolver->Solve(...)	反复调用 ComputeRHS(), 求粒子和中间电场自治的离散方程
5	SetElectricFieldAndApplyBCs()、 FinishImplicitParticleUpdate()	将收敛的中间场写回, 并把粒子从半步状态完成到 t^{n+1}
6	第二个 EvolveB($\Delta t/2$)	完成磁场后半步, 物理时间步才真正结束

因此 m_nlsolver->Solve() 不是一个普通的函数调用包装, 而是这条路径的核心时间组织。SemiImplicitEM::ComputeRHS() 会先用当前猜测的 $E^{n+1/2}$ 更新 WarpX 持有的电场, 然后调用 PreRHSOp(); 后者在 ../warpx/Source/FieldSolver/ImplicitSolvers/ImplicitSolver中完成:

1. 以当前中间场推进粒子位置和速度;
2. 形成 $J^{n+1/2}$;
3. 按需要合并 current_fp_non_suborbit、mass-matrix 或 JFNK 线性阶段贡献;
4. 做 RZ/柱/球几何的逆体积缩放;
5. 调用 SyncCurrentAndRho(), 把源项整理后交给隐式 RHS 或 Jacobian。

这条链中的 PushParticlesandDeposit() 可能在多个 nonlinear iteration、Jacobian evaluation 和 particle Picard iteration 中重复出现, 但这些重复并不代表多个物理时间步。它们是在同一个 $t^n \rightarrow t^{n+1}$ 问题上, 对不同中间场猜测重新计算离散残差。

implicit 还有两个容易误读的实现边界:

- CumulateJ() 必须在 SyncCurrentAndRho() 之前, 把 mass-matrix 路径之外的 J 贡献合入 current_fp; 否则同步的是不完整源项。
- m_use_mass_matrices_jacobian 和 m_particle_suborbits 会让 Jacobian 阶段只推进 suborbit 粒子或直接用 ComputeJfromMassMatrices() 构造电流, 因而不能假定每次 RHS 都走同一个粒子 kernel。

这也解释了为什么 implicit 的验证不能只复制显式 Langmuir 的“单步场误差”判据。至少要分别检查: 非线性求解是否收敛、粒子最终状态是否只提交一次、RHS 期间的 source synchronization 是否完整, 以及最终 E/B 时间层是否与 FinishImplicitParticleUpdate() 一致。

3.11.2 nonlinear solver、JFNK 与 mass-matrix: J 的三种构造层

ImplicitSolver::parseNonlinearSolverParams() 位于 ../warpx/Source/FieldSolver/ImplicitSolvers/ImplicitSolver。它首先读取 nonlinear_solver, 再决定后续 J 是通过完整粒子响应、Newton/JFNK 近似, 还是 PETSc SNES 的线性阶段构造:

配置	solver 对象	粒子/电流路径	关键边界
picard	PicardSolver	每次 RHS 直接用当前场推进粒子并沉积 J	当前实现把 <code>max_particle_iterations=1</code> 、 <code>particle_tolerance=0</code> 固定为最小 Picard 路径
newton	NewtonSolver	非线性外层配合 JFNK；可选 particle suborbits 与 mass-matrix Jacobian 由 PETSc 管理	<code>use_mass_matrices_jacobian</code> 和 <code>use_mass_matrices_pc</code> 只能在该类 solver 中启用
petsc_snes	PETScSNES	nonlinear/linear solve, 仍复用 <code>PreRHSOp()</code> 构造源项	必须以 <code>AMREX_USE_PETSC</code> 编译, 否则源码直接 abort

在普通 nonlinear RHS 阶段, `PreRHSOp()` 的电流来源可以概括为完整粒子响应:

$$J = J_{\text{particle}} + J_{\text{non-suborbit}}.$$

但在使用 mass matrices 的 Jacobian 阶段, 源码采用的是线性化响应:

$$J(E_0 + \delta E) = J_{\text{suborbit}} + J_0 + M \delta E,$$

其中 (E_0) 是 Newton step 开始时由 `SaveE()` 保存的电场, (J_0) 是以 (E_0) 推进的非 mass-matrix 粒子电流, (M) 是 `MassMatrices_X/Y/Z` 表示的离散响应算子。这个式子正是 `CumulateJ()` 和 `ComputeJfromMassMatrices()` 之间的职责分界:

- `CumulateJ()` 把 `current_fp_non_suborbit` 加回当前 `current_fp`, 补上不在 mass matrices 中的粒子贡献;
- `ComputeJfromMassMatrices()` 根据当前 $E-E_0$ 、 J_0 和各方面交叉响应, 把 $M \delta E$ 写入 `current_fp`;
- `SyncCurrentAndRho()` 只负责之后的滤波、边界、guard/level 通信, 不负责判断 J 应该由完整粒子还是线性响应产生。

`ComputeJfromMassMatrices()` 还必须处理 Yee/nodal staggering。源码先根据 $J_x/J_y/J_z$ 的 `ixType()` 计算 `offset_xx ... offset_zz`, 再用 $S_{xx}/S_{xy}/.../S_{zz}$ 的多分量 stencil 访问邻近电场。因此 mass matrix 不是一个可以在任意 centering 上直接相乘的标量系数; 它同时编码了方向耦合、空间 support 和网格位置偏移。把它简写成“($M=dJ/dE$)”只足以说明物理意图, 不足以替代对 index type 和 component offset 的源码核对。

配置层也有明确的几何限制: 当前源码禁止 3D 使用 `use_mass_matrices_jacobian`, 禁止 RSPHERE 使用 mass matrices; `mass_matrices_pc_width` 只在非 3D 情况下读取。因而这条路径不能被描述成所有 implicit geometry 的通用加速开关。

最后, `particle_suborbits` 改变的是粒子响应如何被拆分, 而不是外层物理时间步。在线性 Jacobian 阶段, 若启用 suborbit, `PreRHSOp()` 可以只推进 suborbit 粒子并用 `ComputeJfromMassMatrices(J_from_MM_only)` 补齐响应; 若未启用, 则由 mass matrices 直接构造线性阶段的 J 。这正是 implicit 验证必须同时记录 solver 类型、particle suborbit、mass-matrix 开关和最终 source gate 的原因。

3.12 参数示例与最小运行闭环

如果把本章压成一个最小、可执行、可回查的 runtime entry, 当前最合适的样章输入仍然是:

- `../warpx/Examples/Tests/langmuir/inputs_test_1d_langmuir_multi`

它直接消费了本章讲到的顶层参数和控制流:

- `max_step = 80`
- `algo.maxwell_solver = yee`
- `algo.current_deposition = esirkepov`
- `algo.field_gathering = energy-conserving`
- `warpx.cfl = 0.8`
- 周期场边界

对本章来说, 这个输入最重要的意义不是“Langmuir 物理本身”, 而是它确实会走:

```
main.cpp
-> WarpX::GetInstance()
-> InitData()
```

```

-> ComputeDt()
-> Evolve()
-> OneStep()
-> PushParticlesandDeposit()
-> SyncCurrentAndRho()
-> EvolveB/EvolveE/EvolveB
-> diagnostics

```

当前本地运行记录已经在:

- /Volumes/PHILIPS/programs/PIC/PIC-tutor/runs/stage-c-validation/langmuir_1d

实际命令:

```

env OMP_NUM_THREADS=1 FI_PROVIDER=tcp \
/Volumes/PHILIPS/programs/PIC/warpx/build_full/bin/warpx.1d.MPI.OMP.DP.PDP.OPMD.FFT.EB.QED.GENQEDTABLES
/Volumes/PHILIPS/programs/PIC/warpx/Examples/Tests/langmuir/inputs_test_1d_langmuir_multi

```

它现在已经把本章的“主循环入口”压成了真实闭环:

- 有源码路径
- 有参数入口
- 有可执行命令
- 有输出目录 diags/diag1000080
- 有物理检查量

这说明本章讲的 WarpX 主类生命周期和 Evolve() 主链, 不再只是静态调用图, 而是已经有一条本地运行证据能够落回这些控制流节点。

3.13 进一步阅读与练习

进一步阅读:

1. 04-particle-pushers.md: 继续从 PushParticlesandDeposit() 进入 mypc->Evolve() 和粒子推进器。
2. 05-deposition-shapes.md: 继续展开 SyncCurrentAndRho()、沉积、guard/source synchronization。
3. 00-lifecycle-and-callgraph.md、03-subcycling-and-jrhon.md、05-moving-window.md: 继续下钻本章只做第一轮压缩的分支。

练习题:

1. 说明为什么 WarpX::WarpX() 里只能创建跨-level 外壳, 而不能直接分配完整 MultiFab 主字段。
2. 用本章的 StoreCurrent()/RestoreCurrent() 解释: 为什么 subcycling 不能简单拿 fine current 覆盖 coarse current。
3. 结合 00-langmuir-wave.md, 指出 inputs_test_1d_langmuir_multi 中哪些参数分别进入 ReadParameters()、ComputeDt() 和 OneStep_nosub() 的不同层次。

3.14 本章小结

本章建立了 WarpX 主演化路径的第一张精确地图:

flowchart TD

```

A["main.cpp"] --> B["WarpX::GetInstance"]
B --> C["WarpX::WarpX"]
C --> D["ReadParameters"]
C --> E["MultiParticleContainer"]
B --> F["InitData"]
F --> G["ComputeDt"]
F --> H["InitFromScratch / InitFromCheckpoint"]
H --> I["mypc->AllocData / InitData"]
F --> J["PML / diagnostics / initial fields"]
F --> K["Evolve"]
K --> L["per-step callbacks, load balance, ionization, injection"]
L --> M["OneStep"]
M --> N["implicit solver"]

```

```

M --> O["electrostatic / HybridPIC particle path"]
M --> P["OneStep_nosub"]
M --> Q["OneStep_sub1"]
M --> R["OneStep_JRhom"]
P --> S["PushParticlesandDeposit"]
S --> T["mypc->Evolve"]
P --> U["SyncCurrentAndRho"]
P --> V["PushPSATD or EvolveB/EvolveE/EvolveB"]
K --> W["moving window, boundaries, diagnostics, stop"]

```

后续章节将从 mypc->Evolve() 进入粒子推进、field gather 和沉积内核，再从 EvolveE/B 进入 FDTD/PSATD 场求解器。

3A. WarpX 初始化链：从 InitData() 到初始粒子和外部场

本章把 WarpX::InitData() 展开成一条完整的初始化链。它补足第 3 章中“初始化”只作为主循环前置步骤的不足：这里开始逐块解释 fresh run / restart、AMR level 初始化、外部场、species 注入器、粒子创建 kernel、Gaussian beam、openPMD 文件注入和 projection divergence cleaning。

本章绑定本地源码 ../warpX, 分支 pkuHEDPbranch, commit 8c488b1a9.v0.2 已复核 Source/Initialization/WarpXInitData.cpp 中 InitData()、InitFromScratch()、InitDiagnostics()、AddExternalFields() 的主链行号，并把本章定位从“材料堆叠”调整为“可审校长草稿”。详细源码笔记见：

- notes/code-reading/initialization/08-initialization-bootstrap.md
- notes/code-reading/initialization/09-preconstruct-parameter-locking.md
- notes/code-reading/initialization/10-readparameters-runtime-landing.md
- notes/code-reading/initialization/11-readparameters-combination-constraints.md
- notes/code-reading/initialization/12-alloclevelmfs-specialized-branches.md
- notes/code-reading/initialization/13-initdata-postallocation-consumption.md
- notes/code-reading/initialization/14-initialization-validation-map.md
- notes/code-reading/initialization/15-initialization-validation-map-external-relativistic-openbc.md
- notes/code-reading/initialization/16-initialization-validation-map-density-magnetostatic-nodal.md
- notes/code-reading/initialization/00-init-callgraph.md
- notes/code-reading/initialization/01-external-fields.md
- notes/code-reading/initialization/02-plasma-injector.md
- notes/code-reading/initialization/03-density-momentum-dispatch.md
- notes/code-reading/initialization/04-particle-creation-kernels.md
- notes/code-reading/initialization/05-projection-div-cleaner.md
- notes/code-reading/initialization/06-gaussian-beam-openpmd-injection.md

3A.1 初始化链为什么值得单独成章

v0.2 的本章目标不是继续扩张所有初始化分支，而是把读者最需要的三层边界钉牢：

1. WarpX::WarpX() 构造期只完成参数读取和跨 level 外壳创建。
2. InitData() 才在 fresh run / restart 之间分叉，并把 AMR level、粒子、诊断、PML、外场和初始静电/磁静场组织到第一步之前。
3. species 注入、外部场、projection cleaning、openPMD/Gaussian beam 等细节已有材料，但出版前还需要拆成更短的小节和图表。

PIC 程序的时间推进方程通常写成：

$$f^n, \mathbf{E}^n, \mathbf{B}^n \longrightarrow f^{n+1}, \mathbf{E}^{n+1}, \mathbf{B}^{n+1}.$$

但第一个时间步之前必须先构造一个满足几何、边界、粒子权重、外部场、离散约束和并行布局的初态。WarpX 的初始化不是“读入参数然后开始跑”，而是完成以下任务：

1. 选择 fresh run 或 checkpoint restart。
2. 建立 AMReX level、field MultiFab、PML、EB 和 solver 数据结构。
3. 初始化 diagnostics 和 reduced diagnostics。
4. 创建 species、解析密度/动量/位置分布，并生成宏粒子。
5. 读入或解析外部场，并把外部场叠加到 self field 或 particle external field。

6. 对外部 A/B 场做 projection divergence cleaning.
7. 输出第 0 步 diagnostics, 并检查 guard cell、solver 配置和 known issue.

这一章先给出正式书稿版的主干, 后续章节会继续把每个子函数扩成更细的逐行讲解。

3A.2 启动层先于 InitData(): MPI、AMReX、FFT、PETSc 与运行时契约

前面的初始化笔记大多从 WarpX::InitData() 往后讲, 但 WarpX 真正的初始化链更早就开始了。main.cpp 最外层先做的是:

```
int
main (int argc, char* argv[]) {
    warpx::initialization::initialize_external_libraries(argc, argv);
    {
        auto& warpx = WarpX::GetInstance();
        warpx.InitData();
        warpx.Evolve();
        WarpX::Finalize();
    }
    warpx::initialization::finalize_external_libraries();
}
```

这说明 InitData() 之前还有一层独立的 bootstrap 逻辑, 负责:

1. 初始化 MPI、AMReX、FFT, 以及可选 PETSc.
2. 覆盖一批 AMReX 默认 parser 行为。
3. 解析 geometry、periodicity 和整数数组表达式。
4. 锁定 geometry.dims、moving window 和 warning policy 这类全局运行时契约。

这一层主要分布在:

- Initialization/WarpXAMReXInit.*
- Initialization/WarpXInit.*
- WarpX::MakeWarpX()

3A.2.1 参数系统先分命名空间, 再分读取语义

WarpX 的参数不是“一个大表一次性读完”。它先按 ParmParse 前缀拆成局部命名空间, 然后再决定是:

- 立即求值成数值;
- 保留 parser 字符串, 稍后编译;
- 解释成 interval 切片;
- 还是先解析成枚举型算法选择。

最上游的运行控制参数就是无前缀直接读取:

```
const ParmParse pp; // Traditionally, max_step and stop_time do not have prefix.
utils::parser::queryWithParser(pp, "max_step", max_step);
utils::parser::queryWithParser(pp, "stop_time", stop_time);
```

而大部分初始化参数则先按前缀取视图:

```
const ParmParse pp_amr("amr");
const ParmParse pp_algo("algo");
ParmParse const pp_warpx("warpx");
const ParmParse pp_geometry("geometry");
const ParmParse pp_boundary("boundary");
const ParmParse pp_psatd("psatd");
```

接下来又会分成几种读取壳:

1. queryWithParser/queryArrWithParser
 - 适合 warpx.cfl、const_dt、amr.n_cell 这类“读入就该变成数值”的参数;
2. Store_parserString + makeParser

- 适合 `ref_patch_function(x,y,z)`、外场 `parser`、`diagnostics` 过滤函数这类“先保留表达式，再在别处编译执行”的参数；
3. `IntervalsParser`
- 适合 `sort_intervals`、`dt_update_interval`、`diagnostics intervals` 这类切片语法，不应误看成普通整数；
4. `query_enum_sloppy + AMREX_ENUM`
- 适合 `algo.evolve_scheme`、`warpX.do_electrostatic`、`algo.current_deposition` 这类算法枚举入口。

这层结构解释了为什么同样都出现在 `parameters.rst` 里，`max_step`、`ref_patch_function(x,y,z)` 和 `algo.evolve_scheme` 在源码中会走三种完全不同的消费链。后续参数索引如果想真正有用，就必须至少区分：

- 参数前缀；
- 第一次被哪个 `ParmParse` 命名空间读取；
- 它属于数值参数、`parser` 字符串、`interval` 还是枚举型算法选择。

例如 `WarpXAMReXInit.cpp` 并不是简单调用 `amrex::Initialize()`，而是把一个缺省值覆盖回调一起传进去：

```
amrex::Initialize(
    argc,
    argv,
    build_parm_parse,
    MPI_COMM_WORLD,
    ::overwrite_amrex_parser_defaults
);
```

这个回调会统一注入常量、改写 `amrex.abort_on_out_of_gpu_memory`、`amrex.the_arena_is_managed`、`amrex.omp_threads`、粒子 `tiling` 缺省值，以及由 `boundary.field_* / boundary.particle_*` 反推 `geometry.is_periodic`。换句话说，`WarpX` 从程序一启动就已经不是“AMReX 原生默认设置”。

同一个文件还会在 AMReX 初始化后立即把 `geometry.prob_lo/prob_hi`、`amr.n_cell`、`max_grid_size*blocking_factor*` 这类可能带表达式的输入预解析并写回 `parser`，保证后面的 `Geometry`、`warpX_job_info` 和 `yt` 读到的是数值结果，而不是未展开的表达式字符串。

3A.2.2 文档 alias、AMReX-owned 参数与 WarpX 本地 parser 不是一回事

参数索引继续往下清理后，会发现还有一类参数不能简单用“有没有 `grep` 到同名字符串”来理解。典型例子是：

- `geometry.prob_lo/hi`
- `boundary.field_lo/hi`
- `boundary.potential_lo/hi_x/y/z`
- `psatd.nox/noy/noz`
- `qed_schwinger.xmin/ymin/zmin/xmax/ymax/zmax`

这些名字在文档里是 `grouped alias`，但源码里真实读取的仍然是拆开的 `key`。以 `geometry.prob_lo/hi` 为例，`WarpX` 真正做的是：

```
utils::parser::getArrWithParser(
    pp_geometry, "prob_lo", prob_lo, 0, AMREX_SPACEDIM);
utils::parser::getArrWithParser(
    pp_geometry, "prob_hi", prob_hi, 0, AMREX_SPACEDIM);

pp_geometry.addarr("prob_lo", prob_lo);
pp_geometry.addarr("prob_hi", prob_hi);
```

所以 `geometry.prob_lo/hi` 只是文档层“成对参数”的写法，真正 `parser key` 仍然是 `prob_lo` 和 `prob_hi`。`boundary.field_lo/hi`、`boundary.particle_lo/hi`、`boundary.potential_lo/hi_x/y/z`、`psatd.nox/noy/noz` 和 `Schwinger` 区域边界框也都属于这一类。

还要再区分另一类：参数确实出现在 `WarpX` 手册里，但本地 `WarpX` 并不直接读取，而是由 AMReX 自己消费，`WarpX` 只消费结果。例如 `amr.ref_ratio / amr.ref_ratio_vect` 和 `amrex.async_out / amrex.async_out_nfiles`。当前本地 `WarpX` 源码没有独立的：

```
ParmParse("amr").query("ref_ratio", ...)
ParmParse("amrex").query("async_out", ...)
```

但后续代码会直接消费已经构造好的 `ref_ratios`, 或者在 `plotfile` I/O 邻近层继续设置 `WarpX` 自己的 `field_io_nfiles` / `particle_io_nfiles`。因此, 参数索引如果要稳定, 至少要区分三类:

1. `WarpX` 本地直接 parse;
2. `WarpX` 子对象 parse;
3. AMReX-owned 输入, `WarpX` 只消费 `materialized` 结果。

另一条关键链在 `WarpX::MakeWarpX()`:

```
warpX::initialization::check_dims();
warpX::initialization::read_moving_window_parameters(...);
ConvertLabParamsToBoost();
parse_field_boundaries();
parse_particle_boundaries(...);
CheckGriddingForRZSpectral();
m_instance = new WarpX();
```

因此单例构造前就已经锁定了四类全局事实:

- 当前可执行文件与 `geometry.dims` 是否匹配;
- `moving window` 是否开启、方向和速度是什么;
- `field / particle boundary` 类型是什么;
- RZ spectral 的 `gridding` 约束是否满足。

例如 `check_dims()` 直接把编译维度和 `geometry.dims` 做强一致断言, 而 `read_moving_window_parameters()` 会把输入文件里的 `moving_window_v` 从“以光速为单位的无量纲参数”转换成真正的物理速度 $v = (\cdots)c$, 再写进 `WarpX` 的全局静态状态。

这里还要补一层工程来源。`check_dims()` 能做这种强断言, 前提不是“`WarpX` 在运行时随意切换几何”, 而是构建系统已经先把几何变体锁死了。当前主构建链在顶层 `CMakeLists.txt` 中通过 `WarpX_DIMS` 生成一组按维度后缀分裂的 `lib_${SD}` / `pyWarpX_${SD}` target; 旧 GNUmake 链则通过 `DIM`、`USE_RZ` 以及 `-DWARPX_DIM_3D`、`-DWARPX_DIM_XZ`、`-DWARPX_DIM_RZ` 这组宏编码几何变体。因此初始化阶段读到的 `geometry.dims` 实际是在和“当前可执行文件已经按哪一种几何编译出来”做一致性检查, 而不是在决定后面要不要临时切换到另一种维度。

再往下, `WarpX` 构造函数开头还会调用:

```
warpX::initialization::initialize_warning_manager();
```

也就是在任何 `ReadParameters()` 和 `InitData()` 之前, 先读入:

- `warpX.always_warn_immediately`
- `warpX.abort_on_warning_threshold`

这说明 `warning manager` 在 `WarpX` 里也是初始化启动层的一部分, 而不是后面运行时再临时决定的日志选项。

这一层再往下细分, 还要注意 `MakeWarpX()` 和构造期 `ReadParameters()` 之间并不是简单前后顺序, 而是“前者已经开始改写后者将要读取的参数”。例如 `ConvertLabParamsToBoost()` 不只是保存 `gamma_boost`, 而是会先把:

- `geometry.prob_lo/prob_hi`
- `warpX.fine_tag_lo/fine_tag_hi`
- `slice.dom_lo/dom_hi`

按 `boosted-frame` 规则直接回写进 `parser`。若同时启用 `moving window`, 它计算的转换系数还会显式依赖 `moving_window_v/c`。因此 `boosted-frame` 与 `moving-window` 的第一次耦合, 其实发生在 `ReadParameters()` 之前, 而不是发生在后面的 `Evolve()`。

同样, `CheckGriddingForRZSpectral()` 也不是单纯做断言。对 `WARPX_DIM_RZ + algo.maxwell_solver = psatd` 这条组合, 它会在构造 `WarpX` 对象之前直接改写:

- `amr.blocking_factor_x/y`
- `amr.max_grid_size_x/y`

并要求 `longitudinal` 方向至少满足“每个 MPI rank 至少有一个 `block`, 且每个 rank 至少有 8 个 `z cells`”的约束。也就是说, 构造函数里的 `ReadParameters()` 读到的并不总是用户输入文件里的原始 AMR 分块值, 而往往已经是启动层重写过的 `spectral-compatible` 版本。

把这一点看清之后, 初始化启动层就能更准确地拆成两段:

1. 预初始化与 `parser` 改写段:
 - `initialize_external_libraries()`

- amrex_init()
- parse_geometry_input()
- read_moving_window_parameters()
- ConvertLabParamsToBoost()
- CheckGriddingForRZSpectral()

2. 构造与运行态落地段:

- initialize_warning_manager()
- ReadParameters()
- BackwardCompatibility()
- InitEB()
- MultiParticleContainer / ParticleBoundaryBuffer / solver objects‘ 的真正创建

这个分层很有用, 因为后面再读 boosted-frame 例子、RZ spectral gridding、moving window 连续注入或 warning manager, 就不会再把“参数在哪一步被锁定”与“对象在哪一步消费这些参数”混在一起。

再往前走一步, ReadParameters() 还不只是“把输入存进成员”。它已经在替后面的对象图做分叉。例如:

- do_fluid_species 先通过 fluids.species_names 决定, 随后直接控制是否创建 MultiFluidContainer;
- electrostatic_solver_id 与 electromagnetic_solver_id 先在 ReadParameters() 中互相覆盖和约束, 随后决定静电 solver 具体实例化成 LabFrameExplicitES、EffectivePotentialES 还是 RelativisticExplicitES;
- electromagnetic_solver_id == HybridPIC 时, 构造函数后半段才真正创建 HybridPICModel;
- grid_type、do_current_centering、n_rz_azimuthal_modes 等参数先在 ReadParameters() 中完成默认值推导和合法组合检查, 后面再影响 nodal flags、centering coefficients、fluid 兼容性与容器初始化。

同样, ReadParameters() 里还有一类“源码先推导默认, 再由用户输入覆盖”的派生状态。最典型的是:

```
if (!do_divb_cleaning
    && m_p_ext_field_params->B_ext_grid_type != ExternalFieldType::default_zero
    && m_p_ext_field_params->B_ext_grid_type != ExternalFieldType::constant
    ...
    && (electromagnetic_solver_id == Yee
        || electromagnetic_solver_id == HybridPIC
        || ...))
{
    m_do_initial_div_cleaning = true;
}
pp_warpx.query("do_initial_div_cleaning", m_do_initial_div_cleaning);
```

这里 do_initial_div_cleaning 不是单纯从输入文件机械读取, 而是先根据外部场类型、grid type 和 solver 组合推导一个默认值, 再允许用户显式覆盖。也就是说, 后面 ProjectionDivCleaner 是否参与初态构造, 不是孤立地由一个布尔开关决定, 而是由初始化启动层和 ReadParameters() 主体共同塑造的。

到这一层为止, 初始化阶段已经可以更清楚地压成一条完整链:

1. 启动层先改 parser、锁定全局静态状态。
2. ReadParameters() 再把这些值落到 WarpX 成员, 并决定对象图分叉。
3. 构造函数后半段据此创建 MultiParticleContainer、ParticleBoundaryBuffer、MultiFluidContainer 和 solver objects。
4. 最后 InitData() 才真正开始分配 level data、生成粒子和构造初态。

但这里还差最后一层经常被误读的东西: ReadParameters() 里那些看上去分散的算法检查, 其实会继续决定后面到底分配哪类 MultiFab、是否创建 implicit solver 额外状态, 以及 ProjectionDivCleaner 是否在初态构造中插入。

最紧的一组约束是:

- grid_type=hybrid 会把 do_current_centering 推成默认真值, 并要求 algo.field_gathering=momentum-conserving;
- 这又会让 AllocLevelData() 中的 aux_is_nodal=true, 从而把 Efield_aux/Bfield_aux、后续 coarse-aux cax 和 gather buffer 路径切到 nodal 版本;
- 如果同时显式要求 do_current_centering=1, 源码只允许 grid_type=hybrid, 并且在 level 分配时额外分配 current_fp_nodal。

换句话说, `grid_type`、`field_gathering_algo`、`do_current_centering` 在这里并不是三个平行开关, 而是一组会继续改写场 `index type` 和附加存储布局的上游选择器。

同样, `current_deposition_algo` 也不是只影响某个沉积 kernel。源码先把 PSATD、HybridPIC 和 electrostatic 的默认 deposition 拉回 Direct, 再对 Vay 加上三重限制:

- 只能配 PSATD
- 不能配 mesh refinement
- 不能配 `do_current_centering`

这条限制后面会直接落成额外字段分配: 如果真的选了 Vay, `AllocLevelMFs()` 会专门分配 `current_fp_vay`。所以它已经是初始化对象图的一部分, 而不只是运行时算法分派。

implicit 系列更明显。`ReadParameters()` 一旦看到

- `semi_implicit_em`
- `theta_implicit_em`
- `strang_implicit_spectral_em`

就会立即构造 `m_implicit_solver`, 并同时要求:

- current deposition 只能是 Esirkepov / Villasenor / Direct
- EM solver 只能是 Yee / CKC / PSATD
- particle pusher 只能是 Boris / HigueraCary
- field gather 不能是 momentum-conserving

然后这条链会继续穿到 `InitFromScratch()` 和 `AllocLevelMFs()`:

- `InitFromScratch()` 在 `mypc->InitData()` 之前调用 `m_implicit_solver->Define()` 和 `CreateParticleAttributes()`;
- `AllocLevelMFs()` 在 fine patch 上额外分配 `current_fp_non_suborbit` 与 `E_old`。

因此 implicit 不是“场求解器章节内部的一个局部话题”, 而是在初始化阶段就已经改变粒子属性表和字段分配合同。

`particle_shape.filter` 和 `projection div cleaning` 也属于同一种“前置组合约束”。只要存在 `particles` 或 `lasers`, `algo.particle_shape` 就变成强制参数, 并直接设定 `nox=noy=noz`; 这反过来继续影响 guard-cell 配额、沉积 stencil 和排序默认值。`use_filter` 也不是后面可有可无的一步卷积, 因为 `AllocLevelData()` 会先 `InitFilter()`, 再把 filter stencil 长度交给 `guard_cells.Init(...)`, 于是它会继续改写 guard-cell 和 buffer 需求。

最后, `m_do_initial_div_cleaning` 也不是孤立的布尔开关。源码先根据外部 B 场类型、EM solver 组合和是否已经启用 `do_divb_cleaning` 推导默认值, 再允许用户覆盖; 而它的下游消费点就在初始化主链里, 直接决定 `ProjectionDivCleaner` 是否参与初始场构造。所以到这一步为止, `ReadParameters()` 的真实地位可以概括成一句话:

它不是单纯读参数, 而是在 `AllocLevelData()` 和 `InitFromScratch()` 之前, 先把“允许哪种物理-算法-存储组合存在”这件事裁成一个可执行的对象图。

接下来再往下走一步, 就会看到 `AllocLevelMFs()` 真正把这张对象图摊成一组具体字段。这里最容易误判的是 `rho`、`aux`、外场、Hybrid-PIC/fluid/macrosopic/EB 这些分支。

先看 `rho_fp`。它并不是总存在的基础字段, 而是只在几类路径下显式分配:

- electrostatic / electromagnetostatic / effective-potential
- HybridPIC
- `do_dive_cleaning`
- PSATD 且启用了 `update_with_rho` 或 `current_correction`

而且它的分量数也不是固定值: 普通 electrostatic 或 HybridPIC 只需要 `ncomps`, `do_dive_cleaning` 一般把它升到 `2*ncomps`, PSATD 又会根据 `JRhom` 是否开启在 `ncomps` 和 `2*ncomps` 之间切换。也就是说, WarpX 初始化阶段持有一份“可演化的 `rho` 状态”, 本身就是 solver contract 的一部分。

与此平行的还有三类不同的辅助标量/约束场:

- `phi_fp`: 只属于 electrostatic 系列;
- `F_fp`: 只属于 `div(E) cleaning`;
- `G_fp`: 只属于 `div(B) cleaning`。

不要把它们混成“又分配了几个标量场”。它们分别服务于 Poisson 势解、电场散度清理和磁场散度清理，而且 G 的 coarse-patch index type 还会继续受 grid_type 控制。

再看外场分支。源码实际上维护了两套完全不同的合同：

1. grid external fields 它们分配成 Efield_fp_external/Bfield_fp_external, index type 必须匹配 fp, 后面由 AddExternalFields() 加到主网格场上。
2. particle external fields 它们分配成 E_external_particle_field/B_external_particle_field, index type 必须匹配 aux, 而且分量数直接来自外部粒子场元数据。

所以 particle external field 不是 grid external field 的别名；它跟粒子 gather 所看的 aux 路径绑定得更紧。这也解释了为什么只要 mypc->m_E_ext_particle_s 或 m_B_ext_particle_s 是 read_from_file, 最常见的 aux -> fp alias 优化就会被打断, Efield_aux/Bfield_aux 必须改成单独分配。

剩下几条看似“模块化”的分支，其实也都在 AllocLevelMFs() 里继续扩展 field registry：

- electromagnetic_solver_id == HybridPIC 时, m_hybrid_pic_model->AllocateLevelMFs(...) 会追加 hybrid 自己的 level 字段；
- do_fluid_species 时, myfl->AllocateLevelMFs(...) 之后立刻 InitData(...), 说明 fluid 已经进入初始化主链，而不是仅仅挂了个容器；
- m_em_solver_medium == Macroscopic 时, m_macroscopic_properties->AllocateLevelMFs(...) 只允许 lev==0, 直接把 mesh refinement 排除在外；
- EB::enabled() 时, 所有 level 至少都会有 distance_to_eb 与 m_eb_reduce_particle_shape, 而 finest level 上又会继续长出 m_eb_update_E/B; 如果 solver 还是 ECT, 则再额外长出 edge_lengths、face_areas、area_mod、Venl、ECTRhofield 和 FaceInfoBox 借用关系。

因此 AllocLevelMFs() 的真实角色不是“机械把 MultiFab 开出来”，而是把前面 ReadParameters() 裁出来的对象图真正展开成可执行状态。

更关键的是，这些特例字段不会等到 Evolve() 才第一次使用。在 InitData() 后半段，源码会立刻：

- BuildBufferMasks()
- m_macroscopic_properties->InitData(...)
- m_electrostatic_solver->InitData()
- m_hybrid_pic_model->InitData(m_fields)
- ProjectionCleanDivB()

这就说明初始化链的最后三步其实是：

1. ReadParameters() 决定哪些组合允许存在；
2. AllocLevelMFs() 把这些组合摊成真实字段和对象；
3. InitData() 后半段立刻消费这些字段，把它们变成一份可跑的初态。

再往后一步，InitData() 后半真正把这条合同闭合掉。它的顺序不是“初始化完就直接写 diagnostics”，而是先做一轮收尾和首次消费：

- ComputeMaxStep()
- ComputePMLFactors()
- 可选 InitNCICorrector()
- BuildBufferMasks()
- m_macroscopic_properties->InitData(...)
- m_electrostatic_solver->InitData()
- m_hybrid_pic_model->InitData(m_fields)
- CheckGuardCells()
- ProjectionCleanDivB()

这一步说明前面 AllocLevelMFs() 分配出来的对象并不是先放着，很多都会在这里立刻进入第一次初始化消费。

尤其要区分两步经常被混在一起的电场初始化动作：

1. m_electrostatic_solver->InitData() 这是 solver 对象自身的初始化，fresh run 和 restart 都会走。
2. ComputeSpaceChargeField(reset_fields=false) 这是 fresh-run 下的初始 self-field / electrostatic solve, 只在需要时触发，而且还明确保留网格上已有的用户指定值，不会先把字段清空。

它的触发条件也不是“只有 electrostatic solver 才会跑”，而是三选一：

- 开启 electrostatic solver
- 任意 species 打开 initialize_self_fields
- 指定了 boundary potential

但如果当前走的是 HybridPIC，源码又会显式跳过这条初始 field-solve 路径。

在这之前，若 m_do_initial_div_cleaning 成立，还会先执行 ProjectionCleanDivB()。因此初始 B 场的散度修正顺序其实非常明确：先完成外场读入与对象初始化，再做 div B cleaning，随后才进入初始 self-field / electrostatic solve。

Python callback 也不是一个统一“初始化结束后调用”的事件，而是被拆成了三类窗口：

- beforeInitEsolve: 只在 fresh run，发生在初始场求解前；
- afterInitEsolve: 只在 fresh run，发生在初始场求解后、外加场提交前；
- afterInitatRestart: 只在 restart，表示恢复态后处理，而不是 fresh-run 初始求解窗口的一部分。

外加场这里也有两步，不能混读：

1. LoadExternalFields() 先把 external grid fields 装到 Efield_fp_external/Bfield_fp_external，把 particle-only external fields 装到 E_external_particle_field/B_external_particle_field，并在 finest level 给 Python loadExternalFields callback 一个写这些 buffer 的机会。
2. AddExternalFields() 再把 grid external fields 统一加回 Efield_fp/Bfield_fp 主场。

所以第 0 步最终主场不是“纯 self-field 结果”，而是“初始 self-field / electrostatic solve 结果，再叠加 grid external fields”。

这之后才进入第 0 步 diagnostics：

- multi_diags->FilterComputePackFlush(istep[0]-1)
- reduced_diags->ComputeDiags(...)
- reduced_diags->WriteToFile(...)

因此第 0 步输出的并不是“原始输入快照”，而是“初始化主链已经全部完成后的第一份可运行状态快照”。对于 restart，这一步则取决于 write_diagnostics_on_restart，表示是否要把恢复态也立刻写成一份 diagnostics。

3A.3 顶层入口：fresh run 与 restart 分叉

WarpX::InitData() 位于 ../warpx/Source/Initialization/WarpXInitData.cpp:794-951。第 3 章已经给过总表，这里看核心源码原文。

源码位置：../warpx/Source/Initialization/WarpXInitData.cpp:826-839。

```
if (!restart_chkfile.empty())
{
    InitFromCheckpoint(restart_chkfile);
    PrintDtDxDyDz();
    PostRestart();
}
else
{
    ComputeDt();
    PrintDtDxDyDz();
    InitFromScratch();
    InitDiagnostics();
}
```

这段分支决定初始化数据来源：

- restart: 从 checkpoint 恢复 mesh、field、particles 和时间层，再做 restart 后处理；
- fresh run: 先由 CFL、网格和 solver 计算 dt，再从零建立 AMR level、field、particles 和 diagnostics。

物理上，restart 应该恢复一个已经离散一致的状态；fresh run 则必须从输入参数构造这种一致状态。后面讲的外部场、初始粒子、Gaussian beam、openPMD 注入和 projection cleaning，主要属于 fresh run 路径。

可以把 fresh run 与 restart 的责任边界压缩成下面的读者侧流程图：

```

flowchart TD
    A["WarpX::InitData"] --> B{"restart_chkfile empty?"}
    B -->|"no: restart"| C["InitFromCheckpoint"]
    C --> C1["restore AMR levels, fields, particles, time"]
    C1 --> C2["PostRestart"]
    C2 --> Z["initial state ready for Evolve"]
    B -->|"yes: fresh run"| D["ComputeDt"]
    D --> E["InitFromScratch"]
    E --> E1["AmrCore::InitFromScratch"]
    E1 --> E2["AllocLevelData / solver state"]
    E2 --> E3["mypc->AllocData / InitData"]
    E3 --> E4["external fields and PML"]
    E4 --> F["initial diagnostics"]
    F --> Z

```

这张图的重点不是列出每一个初始化 helper, 而是固定数据来源的不可互换性: restart 路径恢复已经离散化的状态, fresh run 路径才负责从参数重新物化 AMR、solver、粒子、外部场和初始 diagnostics。后面的 PlasmaInjector、Gaussian beam、openPMD 文件注入和 projection cleaning 都必须放在 fresh-run 分支内理解, 不能被误写成 restart 的重复初始化。

3A.4 InitFromScratch(): AMReX level 与粒子初始化

源码位置: ../warpX/Source/Initialization/WarpXInitData.cpp:999-1016。

```

void
WarpX::InitFromScratch ()
{
    BL_PROFILE("WarpX::InitFromScratch()");

    const amrex::Real time = 0.0;
    amrex::AmrCore::InitFromScratch(time);

    AllocLevelData();

    mypc->AllocData();
    mypc->InitData();

    InitPML();
}

```

这里的顺序很重要:

1. AmrCore::InitFromScratch(time) 创建 AMR level。
2. AllocLevelData() 分配 WarpX 自己管理的场、solver、buffer 和 level 数据。
3. mypc->AllocData() 为粒子容器准备数据结构。
4. mypc->InitData() 创建初始粒子。
5. InitPML() 初始化吸收边界数据结构。

因此, species 初始化发生在 field/level 数据结构已经存在之后, 但在正式时间推进之前。

粒子容器入口源码位置: ../warpX/Source/Particles/PhysicalParticleContainer.cpp:429-433。

```

void PhysicalParticleContainer::InitData ()
{
    AddParticles(0); // Note - add on level 0
    Redistribute(); // We then redistribute
}

```

这两行是后续所有粒子创建逻辑的入口。AddParticles(0) 负责生成初始粒子, Redistribute() 负责把粒子分配到正确 tile, 并清理 invalid 粒子。

3A.5 外部场初始化: grid field 与 particle external field

WarpX 支持两类外部场:

- grid external field: 外部场先放在网格 MultiFab 上, 可参与 field solve 或作为背景场;
- particle external field: 外部场在 particle gather 时参与粒子受力。

外部场初始化的关键是: 外部场可以是常量、parser 函数、openPMD 文件或 Python 回调。读者应避免把“外部场”理解成单一数组。

以 projection cleaner 前的 external grid field 判断为例, 源码位置: `../warpx/Source/Initialization/WarpXInitData.cpp:1658-1664`。

```
if ( (m_p_ext_field_params->B_ext_grid_type == ExternalFieldType::read_from_file) ||
     (m_p_ext_field_params->E_ext_grid_type == ExternalFieldType::read_from_file) ||
     (mypc->m_B_ext_particle_s == "read_from_file") ||
     (mypc->m_E_ext_particle_s == "read_from_file") ) {
    ReadExternalFieldFromFile();
}
```

这段代码把 grid external field 与 particle external field 放在同一个文件读取入口下处理。真正写入哪一类 MultiFab, 取决于前面解析出的 B_ext_grid_type/E_ext_grid_type 和 m_B_ext_particle_s/m_E_ext_particle_s。

外部场的物理约束是: 如果读入的是 B 或矢量 A, 数值上还需要检查离散散度误差; 这就是后面 projection divergence cleaner 的用途。

3A.6 species 初始化: PlasmaInjector 是参数总容器

PlasmaInjector 的职责不是推进粒子, 而是把输入文件中一个 species 的初始化规则收集成一组可供 kernel 调用的对象。

源码位置: `../warpx/Source/Initialization/PlasmaInjector.cpp:126-153`。

```
std::string injection_style = "none";
utils::parser::query(pp_species, source_name, "injection_style", injection_style);
std::transform(injection_style.begin(),
               injection_style.end(),
               injection_style.begin(),
               ::tolower);

num_particles_per_cell_each_dim.assign(3, 0);

if (injection_style == "singleparticle") {
    setupSingleParticle(pp_species);
    return;
} else if (injection_style == "multipleparticles") {
    setupMultipleParticles(pp_species);
    return;
} else if (injection_style == "gaussian_beam") {
    setupGaussianBeam(pp_species);
} else if (injection_style == "nrandompercell") {
    setupNRandomPerCell(pp_species);
} else if (injection_style == "nfluxpercell") {
    setupNFluxPerCell(pp_species);
} else if (injection_style == "nuniformpercell") {
    setupNuniformPerCell(pp_species);
} else if (injection_style == "external_file") {
    setupExternalFile(pp_species);
} else if (injection_style != "none") {
    SpeciesUtils::StringParseAbortMessage("Injection style", injection_style);
}
```

这个分支定义了初始粒子创建的第一层分类:

injection_style	含义	后续创建路径
singleparticle	单个手工粒子	AddNParticles()
multipleparticles	手工粒子列表	AddNParticles()
gaussian_beam	空间高斯束流	AddGaussianBeam()
external_file	openPMD 粒子文件	AddPlasmaFromFile()
nrandompercell / nuniformpercell	体密度注入	AddPlasma()
nfluxpercell	面通量注入	AddPlasmaFlux()

在这一步之后，PlasmaInjector 还会把 host 侧 functor 拷贝到 device 侧，保证后续 GPU kernel 可以调用。

3A.7 密度和动量分布：从文本参数到 functor

密度解析由 SpeciesUtils::parseDensity() 完成。源码位置：../warpx/Source/Utils/SpeciesUtils.cpp:80-114。

```
if ( profile == "constant" ){
    pp_species_name.query("density", plasma_injector.density);
    plasma_injector.m_inj_rho =
        std::make_unique<InjectorDensity>(InjectorDensityConstant{
            plasma_injector.density});
} else if (profile == "parse_density_function") {
    std::string str_density_function;
    utils::parser::queryWithParser(pp_species_name, "density_function(x,y,z)", str_density_function);
    auto density_parser =
        std::make_unique<amrex::Parser>(
            utils::parser::makeParser(str_density_function,{"x","y","z"}));
    plasma_injector.m_inj_rho =
        std::make_unique<InjectorDensity>(InjectorDensityParser{
            std::move(density_parser)});
} else if (profile == "read_from_file") {
    std::string read_density_from_path;
    pp_species_name.query("read_density_from_path", read_density_from_path);
    bool read_density_distributed = false;
    pp_species_name.query("read_density_distributed", read_density_distributed);
    plasma_injector.m_inj_rho =
        std::make_unique<InjectorDensity>(InjectorDensityFromFile{
            read_density_from_path, read_density_distributed});
}
```

体注入中的宏粒子权重由密度决定：

$$w_p \approx n(\mathbf{x}) \frac{\Delta V}{N_{ppc}}.$$

动量解析由 SpeciesUtils::parseMomentum() 完成。常见分支包括：

- at_rest: u=0;
- constant: 固定 ux/uy/uz;
- gaussian: 每个方向正态采样;
- gaussianflux: 通量注入专用，法向速度按 $v_n f(v)$ 加权;
- uniform: 每个方向均匀采样;
- maxwell_boltzmann: 非相对论 Maxwellian;
- maxwell_juttner: 相对论热平衡分布;
- parse_momentum_function: 空间解析函数。

InjectorMomentum 的关键工程实现是手写 tagged union。源码位置：../warpx/Source/Initialization/InjectorMomentum.H:459-719。

```
struct InjectorMomentum
{
```

```

enum struct Type {
    constant,
    gaussian,
    gaussianflux,
    uniform,
    boltzmann,
    juttner,
    parser,
    gaussianparser
};

Type type;

union {
    InjectorMomentumConstant constant;
    InjectorMomentumGaussian gaussian;
    InjectorMomentumGaussianFlux gaussianflux;
    InjectorMomentumUniform uniform;
    InjectorMomentumBoltzmann boltzmann;
    InjectorMomentumJuttner juttner;
    InjectorMomentumParser parser;
    InjectorMomentumGaussianParser gaussianparser;
};

```

这不是普通虚函数多态，而是 GPU kernel 友好的平铺对象。后续 AddPlasma() 可以在 device 上用 getMomentum() 采样单粒子动量，用 getBulkMomentum() 得到平均漂移速度。

3A.7 AddParticles(): 按注入类型进入创建函数

源码位置: ../warpx/Source/Particles/ParticleCreation/AddParticles.cpp:194-260.

```

void
PhysicalParticleContainer::AddParticles (int lev)
{
    ABLASTR_PROFILE("PhysicalParticleContainer::AddParticles()");

    for (auto const& plasma_injector : plasma_injectors) {
        if (plasma_injector->add_single_particle) {
            if (WarpX::gamma_boost > 1.) {
                MapParticleToBoostedFrame(plasma_injector->single_particle_pos[0],
                                           plasma_injector->single_particle_pos[1],
                                           plasma_injector->single_particle_pos[2],
                                           plasma_injector->single_particle_u[0],
                                           plasma_injector->single_particle_u[1],
                                           plasma_injector->single_particle_u[2]);
            }
            const amrex::Vector<ParticleReal> xp = {plasma_injector->single_particle_pos[0]};
            const amrex::Vector<ParticleReal> yp = {plasma_injector->single_particle_pos[1]};
            const amrex::Vector<ParticleReal> zp = {plasma_injector->single_particle_pos[2]};
            const amrex::Vector<ParticleReal> xup = {plasma_injector->single_particle_u[0]};
            const amrex::Vector<ParticleReal> yup = {plasma_injector->single_particle_u[1]};
            const amrex::Vector<ParticleReal> zup = {plasma_injector->single_particle_u[2]};
            const amrex::Vector<amrex::Vector<ParticleReal>> attr = {{plasma_injector->single_particle_weight}};
            const amrex::Vector<amrex::Vector<int>> attr_int;
            AddNParticles(lev, 1, xp, yp, zp, xup, yup, zup,
                        1, attr, 0, attr_int, 0);
        }
    }
    return;
}

```

```
}
```

后半段分派如下:

```
if (plasma_injector->gaussian_beam) {
    AddGaussianBeam(*plasma_injector);
}

if (plasma_injector->external_file) {
    AddPlasmaFromFile(*plasma_injector,
                      plasma_injector->q_tot,
                      plasma_injector->z_shift);
}

if ( plasma_injector->doInjection() ) {
    AddPlasma(*plasma_injector, lev);
}
}
```

这个函数说明: PlasmaInjector 中可能同时保存多种初始化信息, 但最终创建时按 flag 调用不同路径。

3A.8 体注入 AddPlasma(): 候选粒子、密度、动量和权重

体注入的核心思想是: 先按 cell 和 num_particles_per_cell 创建候选粒子, 然后用真实 density/bounds 筛掉无效粒子, 并把有效粒子写入 SoA。

源码位置: ../warpx/Source/Particles/ParticleCreation/AddParticles.cpp:854-912.

```
// count the number of particles that each cell in overlap_box could add
amrex::Gpu::DeviceVector<amrex::Long> counts(overlap_box.numPts(), 0);
amrex::Gpu::DeviceVector<amrex::Long> offset(overlap_box.numPts());
auto *pcounts = counts.data();
amrex::Box fine_overlap_box; // default Box is NOT ok().
if (refine_injection) {
    fine_overlap_box = overlap_box & amrex::shift(fine_injection_box, -shifted);
}
amrex::ParallelFor(overlap_box, [=] AMREX_GPU_DEVICE (int i, int j, int k) noexcept
{
    const amrex::IntVect iv(AMREX_D_DECL(i, j, k));
    auto lo = getCellCoords(overlap_corner, dx, {0._rt, 0._rt, 0._rt}, iv);
    auto hi = getCellCoords(overlap_corner, dx, {1._rt, 1._rt, 1._rt}, iv);

    lo.z = applyBallisticCorrection(lo, inj_mom, gamma_boost, beta_boost, t);
    hi.z = applyBallisticCorrection(hi, inj_mom, gamma_boost, beta_boost, t);

    if (inj_pos->overlapsWith(lo, hi))
    {
        auto index = overlap_box.index(iv);
        const amrex::Long r = (fine_overlap_box.ok() && fine_overlap_box.contains(iv)) ?
            (AMREX_D_TERM(rrfac[0], *rrfac[1], *rrfac[2])) : (1);
        pcounts[index] = num_ppc*r;
    }
}
```

applyBallisticCorrection() 用 bulk velocity 把 boosted-frame 位置反推到 lab-frame 初始面。其公式是:

$$z_{0,lab} = \gamma_b [z_{boost}(1 - \beta_b \beta_z) - ct_{boost}(\beta_z - \beta_b)].$$

随后用 prefix scan 得到写入 offset:


```

const amrex::Long max_new_particles = amrex::Scan::ExclusiveSum(counts.size(), counts.data(), offset.data(
amrex::Long pid;
{
    pid = ParticleType::NextID();
    ParticleType::NextID(pid+max_new_particles);
}

```

这种“两遍法”避免在 GPU kernel 中动态 push 粒子。

真正填充粒子时，体注入权重系数为：

```

AMREX_GPU_HOST_DEVICE AMREX_FORCE_INLINE
amrex::Real compute_scale_fac_volume (const amrex::GpuArray<amrex::Real, AMREX_SPACEDIM>& dx,
                                     const amrex::Long pcount) {
    using namespace amrex::literals;
    return (pcount != 0) ? AMREX_D_TERM(dx[0],*dx[1],*dx[2])/pcount : 0.0_rt;
}

```

即

$$w_p = n(\mathbf{x}) \frac{\Delta V}{N_{ppc}}.$$

在 boosted-frame 分支中，密度和纵向动量做 Lorentz 变换：

```

dens = gamma_boost * dens * ( 1.0_rt - beta_boost*betaz_lab );
u.z = gamma_boost * ( u.z -beta_boost*gamma_lab );

```

对应：

$$n' = \gamma_b n (1 - \beta_b \beta_z), \quad u'_z = \gamma_b (u_z - \beta_b \gamma).$$

最后写入粒子 SoA：

```

u.x *= PhysConst::c;
u.y *= PhysConst::c;
u.z *= PhysConst::c;

amrex::Real weight = dens;
weight *= scale_fac;

pa[PIdx::w ][ip] = weight;
pa[PIdx::ux][ip] = u.x;
pa[PIdx::uy][ip] = u.y;
pa[PIdx::uz][ip] = u.z;

```

因此 InjectorMomentum 返回的 u 是无量纲 \gamma\beta，粒子数组中存的是 \gamma v。

3A.9 Gaussian beam: 显式束流列表注入

Gaussian beam 不使用 InjectorDensity，而是在 IO rank 上显式随机生成粒子列表。

源码位置：../warpx/Source/Particles/ParticleCreation/AddParticles.cpp:396-407。

```

if (ParallelDescriptor::IOProcessor()) {
    // If do_symmetrize, create either 4x or 8x fewer particles, and
    // Replicate each particle either 4 times (x,y) (-x,y) (x,-y) (-x,-y)
    // or 8 times, additionally (y,x), (-y,x), (y,-x), (-y,-x)
    if (do_symmetrize){
        npart /= symmetrization_order;
    }
}

```

```

}
// compute the weight from N_tot if the user specified npart_real = N_tot
// compute the weight from q_tot if the user specified q_tot
// note that npart is the number of macroparticles
const amrex::Real weight_3d = (N_tot > 0._rt) ? (N_tot / npart) : (q_tot / (npart*charge));

```

权重来自总真实粒子数或总电荷:

$$w_{3d} = \begin{cases} N_{\text{tot}}/N_p, \\ Q_{\text{tot}}/(N_p q). \end{cases}$$

随后按高斯分布采样空间位置:

```

#if defined(WARPX_DIM_3D) || defined(WARPX_DIM_RZ)
const amrex::Real weight = weight_3d;
amrex::Real x = amrex::RandomNormal(x_m, x_rms);
amrex::Real y = amrex::RandomNormal(y_m, y_rms);
amrex::Real z = amrex::RandomNormal(z_m, z_rms);
#elif defined(WARPX_DIM_XZ)
const amrex::Real weight = weight_3d/y_rms;
amrex::Real x = amrex::RandomNormal(x_m, x_rms);
constexpr amrex::Real y = 0._prt;
amrex::Real z = amrex::RandomNormal(z_m, z_rms);

```

如果设置 focal_distance, 代码用弹道近似从焦平面束斑反推出初始位置。核心公式是:

$$t = \frac{(\mathbf{x}_f - \mathbf{x}) \cdot \mathbf{n}}{\mathbf{v} \cdot \mathbf{n}}, \quad \mathbf{x} \leftarrow \mathbf{x} - \mathbf{v}_\perp t.$$

源码位置: ../warpx/Source/Particles/ParticleCreation/AddParticles.cpp:453-462.

```

// Compute the time at which the particle will cross the focal plane
const amrex::Real v_dot_n = v_x * n_x + v_y * n_y + v_z * n_z;
const amrex::Real t = ((x_f-x)*n_x + (y_f-y)*n_y + (z_f-z)*n_z) / v_dot_n;

// Displace particles in the direction orthogonal to the beam bulk momentum
// i.e. orthogonal to (n_x, n_y, n_z)
#if defined(WARPX_DIM_3D) || defined(WARPX_DIM_RZ)
x = x - (v_x - v_dot_n*n_x) * t;
y = y - (v_y - v_dot_n*n_y) * t;
z = z - (v_z - v_dot_n*n_z) * t;

```

如果设置 symmetrization, 代码为每个样本生成 4 或 8 个镜像粒子, 并把权重除以阶数。这降低横向低阶统计噪声。

3A.10 openPMD 粒子文件: 文件粒子列表注入

external_file 路径在构造期先打开 openPMD 文件, 读取可选 charge/mass。源码位置: ../warpx/Source/Initialization/PlasmaInjector

```

void PlasmaInjector::setupExternalFile (amrex::ParmParse const& pp_species)
{
#ifdef WARPX_USE_OPENPMD
WARPX_ABORT_WITH_MESSAGE(
    "WarpX has to be compiled with USE_OPENPMD=TRUE to be able"
    " to read the external openPMD file with species data");
#endif
external_file = true;
std::string str_injection_file;
utils::parser::get(pp_species, source_name, "injection_file", str_injection_file);
// optional parameters

```

```
utils::parser::queryWithParser(pp_species, source_name, "q_tot", q_tot);
utils::parser::queryWithParser(pp_species, source_name, "z_shift", z_shift);
```

质量和电荷优先级是:

```
input charge/mass > input species_type > openPMD charge/mass record
```

真正读入粒子时, 源码位置: ../warpX/Source/Particles/ParticleCreation/AddParticles.cpp:680-715.

```
for (auto i = decltype(npart){0}; i<npart; ++i){

    amrex::ParticleReal const weight = ptr_w.get()[i]*w_unit;

    #if !defined(WARPX_DIM_1D_Z)
        amrex::ParticleReal const x = ptr_x.get()[i]*position_unit_x + ptr_offset_x.get()[i]*position_offset_u
    #else
        amrex::ParticleReal const x = 0.0_prt;
    #endif
    #if defined(WARPX_DIM_3D) || defined(WARPX_DIM_RZ) || defined(WARPX_DIM_RCYLINDER) || defined(WARPX_DIM_RS
        amrex::ParticleReal const y = ptr_y.get()[i]*position_unit_y + ptr_offset_y.get()[i]*position_offset_u
    #else
        amrex::ParticleReal const y = 0.0_prt;
    #endif
    #if !defined(WARPX_DIM_RCYLINDER)
        amrex::ParticleReal const z = ptr_z.get()[i]*position_unit_z + ptr_offset_z.get()[i]*position_offset_u
    #else
        amrex::ParticleReal const z = 0.0_prt;
    #endif
```

openPMD 的 position 和 positionOffset 都乘以各自 unitSI, z_shift 是 WarpX 额外偏移。

动量换算:

```
if (plasma_injector.insideBounds(x, y, z)) {

    // The normalized momentum is u = p / m = gamma beta c
    // with m = m_e for photons, m the particle mass otherwise.
    amrex::ParticleReal const mass_eff = (m_mass > 0.0_prt) ? m_mass : PhysConst::m_e;
    amrex::ParticleReal const ux = ptr_ux.get()[i]*momentum_unit_x/mass_eff;
    amrex::ParticleReal const uz = ptr_uz.get()[i]*momentum_unit_z/mass_eff;
    amrex::ParticleReal uy = 0.0_prt;
    if (ps["momentum"].contains("y")) {
        uy = ptr_uy.get()[i]*momentum_unit_y/mass_eff;
    }
}
```

openPMD 文件中的 momentum 是物理动量 p。除以质量得到 $p/m=\gamma v$, 这正是 WarpX 粒子数组的 ux/uy/uz 量纲。文件中的 weighting 直接成为宏粒子权重; q_tot 只产生 warning, 不会重标定权重。

3A.11 Projection divergence cleaning: 外部 A/B 场的初始约束修正

如果外部加载的 B 或矢量 A 在离散网格上不满足散度约束, WarpX 可用 projection 方法清理。

数学上, 给定向量场 F, 构造:

$$\mathbf{F}' = \mathbf{F} + \nabla_h \phi, \quad \nabla_h \cdot \mathbf{F}' = 0.$$

于是

$$\nabla_h^2 \phi = -\nabla_h \cdot \mathbf{F}.$$

源码位置: ../warpX/Source/Initialization/DivCleaner/ProjectionDivCleaner.cpp:256-264.

```
WarpX::ComputeDivB(
    *m_source[ilev],
    0,
    {Bx, By, Bz},
    WarpX::CellSize(0)
);
```

```
m_source[ilev]->mult(-1._rt);
```

然后用 AMReX MLMG 解 Poisson 方程。源码位置:../warpX/Source/Initialization/DivCleaner/ProjectionDivCleaner.H:93-10

```
amrex::MLMG mlmg(linop);
mlmg.setMaxIter(m_max_iter);
mlmg.setMaxFmgIter(m_max_fmg_iter);
mlmg.setBottomSolver(m_bottom_solver);
mlmg.setVerbose(m_verbose);
mlmg.setBottomVerbose(m_bottom_verbose);
mlmg.setConvergenceNormType(amrex::MLMGNormType::greater);
mlmg.solve({m_solution[lev].get()}, {m_source[lev].get()}, m_rtol, m_atol);
```

最后修正场:

```
Bx_arr(i,j,k) += T::DownwardDx(sol_arr, coefs_x, n_coefs_x, i, j, k);
By_arr(i,j,k) += T::DownwardDy(sol_arr, coefs_y, n_coefs_y, i, j, k);
Bz_arr(i,j,k) += T::DownwardDz(sol_arr, coefs_z, n_coefs_z, i, j, k);
```

这不是演化阶段的 `warpX.do_dive_cleaning/do_divb_cleaning`。后者是 Maxwell solver 时间推进中的清理变量或修正方程；本节讲的是初始化或外部场加载后的 Poisson projection。

Laser antenna 与 profile 分派: laser 初始化并不走 PlasmaInjector

species 初始化走的是 PlasmaInjector -> AddPlasma/AddGaussianBeam/AddPlasmaFromFile 这一条链；laser 初始化则完全不同。它的入口是:

- lasers.names
- LaserParticleContainer
- Laser/LaserProfiles.*

LaserParticleContainer 构造函数先统一读取天线几何和公共物理参数:

```
utils::parser::getArrWithParser(pp_laser_name, "position", m_position);
utils::parser::getArrWithParser(pp_laser_name, "direction", m_nvec);
utils::parser::getArrWithParser(pp_laser_name, "polarization", m_p_X);
utils::parser::getWithParser(pp_laser_name, "wavelength", m_wavelength);
```

然后要求 `e_max` 和 `a0` 二选一:

```
const bool e_max_is_specified =
    utils::parser::queryWithParser(pp_laser_name, "e_max", m_e_max);
Real a0;
const bool a0_is_specified =
    utils::parser::queryWithParser(pp_laser_name, "a0", a0);
...
AMREX_ALWAYS_ASSERT_WITH_MESSAGE(
    e_max_is_specified ^ a0_is_specified,
    "Exactly one of e_max or a0 must be specified for the laser.\n");
```

如果给的是 `a0`, WarpX 会立即按

$$E_{\max} = \frac{m_e \omega c}{q_e} a_0$$

换算成真实场强 `e_max`。这说明在 `profile` 实现层, `a0` 已经不再存在, 剩下的只是规范化后的公共参数。

`profile` 类型分派也不是一串手写 `if-else`, 而是 `LaserProfiles.H` 中的工厂字典:

```
laser_profiles_dictionary =
{
    {"gaussian",
     [] () {return std::make_unique<GaussianLaserProfile>();} },
    {"parse_field_function",
     [] () {return std::make_unique<FieldFunctionLaserProfile>();} },
    {"from_file",
     [] () {return std::make_unique<FromFileLaserProfile>();} }
};
```

构造函数把 `profile` 字符串转成小写后直接做字典查找, 再调用统一接口:

```
m_up_laser_profile = laser_profiles_dictionary.at(laser_type_s());
...
m_up_laser_profile->init(pp_laser_name, common_params);
```

所以:

1. `gaussian` 走解析包络与聚焦/STC/chirp 公式;
2. `parse_field_function` 直接把 `field_function(X,Y,t)` 编译成 parser;
3. `from_file` 则走 `lasy_file_name` 或 `binary_file_name`, 并用 `time_chunk_size / delay` 做时间分块读入。

更重要的是, `laser pulse` 不是直接“写入一个初始场数组”, 而是通过人工天线粒子实现。`LaserParticleContainer.H` 的类注释写得很明确: 这些粒子均匀分布在一个平面上, 按预设位移沉积电流 `J`, 再由 `Maxwell solver` 在网格上生成真正的激光场。因此 `LaserParticleContainer` 需要 `current deposition`, 但不走普通 `FieldGather`, 它也正因此直接继承 `WarpXParticleContainer`, 而不是 `PhysicalParticleContainer`。

`continuous injection` 对 `laser` 还有额外几何约束:

```
AMREX_ALWAYS_ASSERT_WITH_MESSAGE(
    ...,
    "do_continuous_injection for laser particle only works"
    " if moving window direction and laser propagation direction are the same");
```

如果叠加 `boosted frame`, 目前还进一步要求 `boost` 方向是 `z`。因此 `laser` 的 `do_continuous_injection` 虽然和普通 `species` 同名, 但它的可用组合更窄, 实际上是“moving-window / boosted-frame 下天线何时进入域内”的控制开关。

这还只是 `laser` 初始化链的入口。真正运行起来后, `WarpX` 并不会把解析式或文件 `profile` 直接写进 `E/Bfield_fp`。它先把 `profile` 解释成“天线平面上每个人工粒子的目标发射振幅”, 再通过标准 `J/rho` 沉积把激光交给 `Maxwell solver`。

`GaussianLaserProfile::init()` 继续在公共参数外读取:

```
utils::parser::getWithParser(pp1, "profile_waist", m_params.waist);
utils::parser::getWithParser(pp1, "profile_duration", m_params.duration);
utils::parser::getWithParser(pp1, "profile_t_peak", m_params.t_peak);
utils::parser::getWithParser(pp1, "profile_focal_distance", m_params.focal_distance);
utils::parser::queryWithParser(pp1, "zeta", m_params.zeta);
utils::parser::queryWithParser(pp1, "beta", m_params.beta);
utils::parser::queryWithParser(pp1, "phi2", m_params.phi2);
utils::parser::queryWithParser(pp1, "phi0", m_params.phi0);
```

因此当前 `gaussian profile` 并不是“只有纵向和横向高斯包络”的最简形式, 而是已经直接包含:

- 聚焦距离 `profile_focal_distance`
- carrier-envelope phase `phi0`
- spatial chirp `zeta`
- angular dispersion `beta`
- temporal chirp `phi2`

`WarpX` 随后还会把 `stc_direction` 归一化, 并强制要求它与天线法向 `nvec` 正交:

```
WARPX_ALWAYS_ASSERT_WITH_MESSAGE(std::abs(dp2) < 1.0e-14,
    "stc_direction is not perpendicular to the laser plane vector");
```

这说明 `stc_direction` 的真实语义是“STC 在激光平面内的作用方向”，不是随手附带的一个参考向量。到了 `fill_amplitude()`，WarpX 再显式构造：

- `diffract_factor`
- `inv_complex_waist_2`
- `stretch_factor`

并按 3D/RZ、XZ、1D 分别选不同 `prefactor`。也就是说，当前 Gaussian 实现已经把 diffraction、Gouy phase、wavefront curvature、spatial chirp、angular dispersion 和 temporal chirp 全都折叠进同一个复 `envelope` 公式里，而不是后面再给场求解器额外修正。

`from_file` `profile` 的合同也比“读一个文件”更具体。源码强制要求：

```
WARPX_ALWAYS_ASSERT_WITH_MESSAGE (
    lasy_file_name.empty() != binary_file_name.empty(),
    "Exactly one of 'binary_file_name' and 'lasy_file_name' has to be specified");
```

因此 `from_file` 不是同时混用两种后端，而是在：

- `lasy_file_name`
- `binary_file_name`

之间二选一。并且它不是一次性把整份文件全读进来，而是先把 `time_chunk_size` 设成默认全时域，再允许用户把它改小，随后只预读第一块；真正运行时由 `m_up_laser_profile->update(t_lab)` 按需换块。`delay` 也不是写文件时的预处理，而是在 `update()` / `fill_amplitude()` 内统一平移 `profile` 时间轴。

更关键的是，`LaserParticleContainer::InitData()` 的产物不是“初始化场”，而是人工天线粒子。它先按 `finest cell` 和天线平面方向计算平面 `spacing S_X/S_Y`，再建立实验室坐标与激光平面坐标之间的 `Transform/InverseTransform`，最后只在注入盒覆盖的区域生成粒子：

- Cartesian / XZ / 1D：每个平面位置生成一对 `+w/-w` 粒子
- RZ：围绕轴向展开成 `spokes`，并用 `2*pi*r/n_spokes` 修正权重

到了 `LaserParticleContainer::Evolve()`，运行时主链是：

1. 若在 `boosted frame`，先把数值时间 `t` 转回实验室系 `t_lab`
2. `m_up_laser_profile->update(t_lab)`，必要时推进 `from_file` 的时间块缓存
3. `calculate_laser_plane_coordinates(...)`，把真实粒子位置映回激光平面坐标
4. `fill_amplitude(...)`，为每个天线粒子求目标 `E` 振幅
5. `update_laser_particle(...)`，把振幅变成粒子动量和位置
6. `DepositCurrent()` / `DepositCharge()`，把人工天线粒子写入 `current_fp/rho_fp`，必要时也写入 `coarse-fine current_buf/rho_buf`

所以 WarpX 的 `laser antenna` 不是“边界直接给定场”，而是“人为构造一层发射粒子，通过普通沉积链把 `profile` 写成 `J`，再让 Maxwell solver 自己生成传播场”。这也解释了为什么 `laser` 在 `mesh refinement` 下照样要走 `fp / buf` 分流，以及为什么 `lasers.deposit_on_main_grid` 其实属于 AMR/沉积合同，而不只是一个 `laser` 小选项。

最后，`laser` 的 `continuous injection` 也应理解得更准确一点。`ContinuousInjection()` 检查的是更新后的 `m_updated_position` 是否第一次进入当前 `injection_box`；一旦进入，就调用一次 `InitData()` 生成那片天线粒子，之后再由普通 `Evolve()` 周期性更新它们的发射振幅。它不是每步“重新生成整束激光”，而是在 `moving-window / boosted-frame` 下决定“天线什么时候进入域内并开始工作”。

`parse_field_function` 这条分支也需要单独说明，因为它和前两类 `profile` 的数学合同并不一样。官方参数文档把它定义成：

```
<laser_name>.field_function(X,Y,t)
```

这里给出的不是包络，而是完整电场本身；`X/Y` 是垂直于激光传播方向的平面坐标，而不是固定的仿真坐标轴。`FieldFunctionLaserProfile::init()` 在源码里只做两件事：

```
utils::parser::Store_parserString(
    ppl, "field_function(X,Y,t)", m_params.field_function);
m_parser = utils::parser::makeParser(m_params.field_function,{"X","Y","t"});
```

随后 `fill_amplitude()` 也没有再加任何 `envelope`、`phase` 或 `geometry` 修正，而是直接逐粒子求值：

```

auto parser = m_parser.compile<3>();
amrex::ParallelFor(np, [=] AMREX_GPU_DEVICE (int i) noexcept
{
    amplitude[i] = parser(Xp[i], Yp[i], t);
});

```

这说明 `parse_field_function` 的 `profile` 层是三类实现里最薄的一层：用户写什么场函数，天线平面上就得到什么目标场值；后续所有“把目标场值变成可沉积粒子运动”的责任，都落在 `LaserParticleContainer` 的更新 `kernel` 上。

这几个 `kernel` 里，`calculate_laser_plane_coordinates(...)` 负责把真实粒子位置减去天线参考点后，投影回 `m_u_X/m_u_Y` 基底，因此 `profile` 接口始终统一在激光平面坐标上。`ComputeWeightMobility()` 则先用固定的峰值速度上限 `eps = 0.05` 设定：

```

m_mobility = eps/m_e_max;
m_weight = PhysConst::epsilon_0 / m_mobility;
m_weight *= AMREX_D_TERM(1._rt, * Sx, * Sy);

```

也就是说，`WarpX` 不是先任意给天线粒子权重，再让速度自己长出来；而是先要求峰值场下粒子速度不超过 $0.05c$ ，再反推单粒子权重。到了 `update_laser_particle()`，`WarpX` 再把 `amplitude[i]` 变成：

1. 沿主偏振方向 `p_X` 的速度
2. `boosted-frame` 下额外减去沿传播方向 `nvec` 的平移速度
3. 相应的 relativistic momentum `ux/uy/uz`
4. 显式路径的整步位置推进，或 `implicit` 路径基于 `x_n/y_n/z_n` 的半步 `time-centered` 位置推进

因此 `laser` 的人工天线粒子并不是一个只服务显式 `solver` 的简单边界 `hack`。它同样要遵守 `implicit particle-centering` 合同，并继续进入普通 `DepositCurrent()/DepositCharge()` 主链。

当前本地 `checkout` 里，`parse_field_function` 的最明确真实用例是 `Examples/Tests/particle_absorbing_boundary/inputs_test_1d` 这个输入把：

- `laser1.profile = parse_field_function`
- `laser1.field_function(X,Y,t) = ...`

嵌进了吸收边界测试里。但对应 `analysis.py` 检查的是边界附近的负向高速电子是否被抑制，而不是直接检查激光场本身。这意味着 `parse_field_function` 目前是“有真实 `regression` 入口，但没有独立 `field-level` 解析断言”的状态，书稿里应把这个验证边界明确写出来。

把 `laser` 模块整体放回 `regression` 版图后，还能看到另一个重要事实：不同 `laser tests` 的证据强度差别很大。`Examples/Tests/laser_injection/` 的 `1D/2D analysis` 会直接比较 `Gaussian` 注入场的包络和主频；`implicit 1D/2D` 变体也继续复用同一组 `analysis`，因此并不只是“`implicit` 能跑通”的 `checksum test`。`Examples/Tests/laser_injection_from_file/` 则继续给 `lasy`、`legacy binary`、`boosted-frame` 和 `RZ thetaMode` 文件提供 `envelope/frequency` 双断言。

但这一组还必须再分出一层 `helper / prepare` 边界。两个目录里的 `analysis_default_regression.py` 都只是本地 `checksum helper` 副本：职责是自动识别 `plotfile/openPMD` 并按测试目录名调用 `evaluate_checksum(...)`，给 `active tests` 提供历史输出基线，而不是新增 `laser` 物理断言。更重要的是，`laser_injection_from_file/` 里那批 `inputs_test*_prepare.py` 并不是“待分析输入”，而是被 `CMakeLists.txt` 先行注册成 `dependency` 的外部文件生成阶段：

- 普通 `1D/2D/3D/RZ lasy` 变体统一先写 `gaussian_laser_3d`
- `legacy binary` 变体先手工写 `gauss_2d`
- `RZ thetaMode` 变体先写 `laguerre_laser_RZ`

因此这组 `regression` 的正确结构不是单段输入，而是：

1. `prepare`
 - 生成外部 `laser` 文件
2. `inject`
 - `WarpX` 按 `from_file / binary_file_name / RZ` 路径消费这些文件
3. `analysis`
 - 再对最终包络和主频做强断言

这条边界对后面精读 `Laser/` 很关键，因为它说明“外部 `laser` 文件格式合同”本身已经是 `active regression` 的一部分，而不只是示例配套脚本。

但到了 `Examples/Physics_applications/laser_acceleration/`，情况就不一样了。这个目录本质上不是一组 `laser-injection` 单元测试，而是一套 `LWFA runtime matrix`。`README.rst` 自己都把 `Analyze` 章节留成了 `TODO`，而当前大多数 `active tests` 在 `CMakeLists.txt` 中也都配置成 `analysis = OFF`，只保留 `checksum`；只有少数变体有明确 `analysis`：

- `analysis_1d_fluid_boosted.py`: 把 laser 驱动的 1D boosted fluid WFA 结果与理论 ODE 解对照, 检查 $E_z/J_z/\rho/V_z$
- `analysis_refined_injection.py`: 检查 `warpx.refine_plasma = 1` 场景下的总粒子数和 refinement edge 前方 ρ 切片均匀性
- `analysis_openpmd_rz.py`: 检查 RZ openPMD diagnostics 的 mesh shape、species ordering 和 $\rho_{<species>}$ 物理中心位置

更进一步, `inputs_base_1d/2d/3d/rz` 四个基础输入也说明了这组 family 先定义的是不同维度下的运行骨架:

- 1D: moving window + 连续电子注入 + Gaussian laser antenna + FieldProbe
- 2D: PML + moving window + refined patch + 连续背景电子 + Gaussian beam
- 3D: moving window + openPMD Full diagnostics + 自定义粒子属性
- RZ: `n_rz_azimuthal_modes = 2` + beam/plasma 共存 + species 变量输出

因此 `laser_acceleration` 目录里的大多数条目当前更准确的定位应该是:

- LWFA application/runtime checksum baseline
- 以及 boosted / MR / PICMI / Python callback / RZ / openPMD 的路径覆盖

而不是统一的 wake amplitude 或 laser envelope 解析 benchmark。

此外, `Examples/Tests/boosted_diags/analysis.py` 对 `test_3d_laser_acceleration_btd` 的验证重点也不是 laser 包络本身, 而是:

1. BTD plotfile 与 BTD openPMD 的 E_z 是否逐点一致
2. `random_fraction` 粒子子采样是否真的生效

所以当前本地 WarpX checkout 对 laser 的回归支持应这样理解:

- 注入本体: 1D/2D 强, 3D 较弱
- `from_file`: 强
- `parse_field_function`: 有真实入口, 但主要是间接覆盖
- `laser_acceleration`: 多数是下游 LWFA/LPI 工作流回归, 不应误写成 laser 注入公式的直接解析验证
- BTD / openPMD / Python callback: 更偏 diagnostics 和 workflow 合同

这组边界在书稿中必须显式写出, 否则很容易把“有 `analysis.py`”误判成“已经有强物理断言”, 或者把 `laser_acceleration` 目录整体误判成 laser injection 的单元测试集合。

再往运行态交界看一层, laser 初始化还必须和 moving window、boosted frame、continuous injection、external fields 的更新合同一起理解。WarpX::MoveWindow() 在真正平移网格前, 会先做三件互不等价的更新:

```
moving_window_x += ...;
::UpdateInjectionPosition(*mypc, gamma_boost, beta_boost, boost_direction, moving_window_dir, dt[0]);
mypc->UpdateAntennaPosition(dt[0]);
```

这里:

1. `moving_window_x` 是窗口几何本身的位置;
2. `UpdateInjectionPosition(...)` 更新普通 species 的 `m_current_injection_position`;
3. `UpdateAntennaPosition(dt)` 更新 laser antenna 的 `m_updated_position`。

普通 species 的连续注入位置来自 `PlasmaInjector` 的 bulk momentum, 再换成速度并在 boosted frame 下做洛伦兹变换; laser 则完全不走 `PlasmaInjector`, 只在 `do_continuous_injection=1` 且 `gamma_boost>1` 时按 boost velocity 平移天线平面。因此两者虽然都叫 continuous injection, 但运行态位置更新机制不同。

两者的 `ContinuousInjection()` 语义也不同。普通物理粒子是在 moving window 新扫进来的 level-0 `particleBox` 中反复调用 `AddPlasma(...)`; laser 则只在 `m_updated_position` 第一次进入当前 `injection_box` 时调用一次 `InitData()`, 之后靠已有入天线粒子在 `Evolve()` 中持续更新并沉积 J/ρ 。AMR 下这两个尺度也不同: species 的 runtime 注入盒按 level-0 cell 对齐, 而 `LaserParticleContainer::InitData()` 仍然按 `maxLevel()` 的 finest spacing 建立天线粒子。

external field 的 moving-window 合同也在这里锁死。`LoadExternalFields()` 对 `B/E_ext_grid_type == read_from_file` 以及 particle external field 的 `read_from_file` 都直接断言:

```
WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
    WarpX::do_moving_window == 0,
    "External fields from file are not compatible with the moving window." );
```


原因不是 openPMD 不能读, 而是 MoveWindow() 在新进入的 cells 上只能通过 shiftMF(...) 用 constant 或 parser 两种方式重建主场背景:

```
srcfab(i,j,k,n) = external_field;
srcfab(i,j,k,n) = field_parser(x,y,z);
```

因此 constant/parser 外场可以跟着 moving window 继续生成, 而 read_from_file 缺少“窗口每推进一次就按新的 physical coordinates 增量重读”的实现, 所以被源码显式禁止。这也解释了为什么当前 load_external_field* regressions 都天然的是静态窗口场景, 而 laser_acceleration_boosted、refined_injection、subcycling_mr 这些例子才更贴近 laser 与 moving-window 交界的真实运行态合同。

再往应用层走, laser 在本地 WarpX 里已经分叉成三种不同角色。laser_ion 是最典型的“laser 作为驱动器”的场景: 输入里同时绑了 Gaussian laser、solid-density target、full diagnostics、time-averaged diagnostics、ParticleHistogram、FieldProbe 和 ParticleHistogram2D。但它最硬的 regression 断言并不是离子能量标度, 而是 analysis_test_laser_ion.py 对 diagInst 最后 5 个 snapshot 的瞬时 Ez 平均值与 diagTimeAvg 原位 time-averaged Ez 的逐点比较。因此它在书稿里最适合承担“laser 主链怎样进入复杂 diagnostics 组合场景”的角色。

free_electron_laser 则正好是反例: 它没有 lasers.names = ..., 而是通过刚性注入电子/正电子束、boosted frame、moving window 和外加 undulator B_y(z) 让辐射在束流中自发增长。analysis_fel.py 在 lab-frame 与 boosted-frame diagnostics 上分别拟合 gain length, 并通过 FFT 反推出 radiation wavelength。这说明这里的“laser/辐射”不是天线输入, 而是束流和 external particle field 共同产生的结果量, 所以它更像 laser 相关应用, 而不是 Laser 模块本身的 injection regression。

再往实现层拆, free_electron_laser 真正依赖的三块基础设施是:

1. RigidInjectedParticleContainer
2. particles.B_ext_particle_init_style = parse_B_ext_particle_function
3. BackTransformed diagnostics

它的 species 不会实例化普通 PhysicalParticleContainer, 而是被 MultiParticleContainer 切到 RigidInjectedParticleContainer。zinject_plane 和 rigid_advance 决定束团在注入面之前是按各自 v_z 还是按平均束流速度作刚体传播; boosted-frame 下 zinject_plane_levels 还会继续按 beta_boost c 平移。与此同时, undulator 场也不是写入主场 Bfield_fp, 而是通过 particle external field parser 直接在 gather 侧提供 B_y(z)。因此这里的主链其实是“刚性束流 + 粒子背景场 + BTD 恢复 lab-frame 物理”, 而不是 laser antenna 本体。

rigid_injection 和 boosted_diags 两组 tests 则给这条链提供了更基础的硬断言。前者分别在 lab frame 和 BTD 下检查: 刚性传播是否真的把束宽保持到 zinject_plane、以及 plotfile/openPMD 回写的束团位置与动量是否一致; 后者额外验证 BTD 两种 writer 的场数据一致性与 random_fraction 粒子采样合同。也就是说, analysis_fel.py 负责最终 FEL 标度, rigid_injection* 负责 rigid propagation 本身, boosted_diags 负责 BTD 基础设施, 而这三层不应再被混写成一个笼统的“laser regression”。

laser_on_fine 则又是第三类。它确实使用真正的 Gaussian laser antenna, 但 CMakeLists.txt 里没有独立 analysis, 主要依赖 checksum; 输入重点在 max_level = 1、fine_tag_lo/hi、laser1.prob_lo/prob_hi 和 PML。也就是说它更像一个 AMR placement/solver 稳定性测试, 而不是下游应用 physics 场景。

因此, 后续书稿中的 laser 应用层不应只按 profile 分类, 而应按三种角色拆分:

1. laser 作为驱动器并配套 diagnostics 组合: laser_ion
2. 辐射/laser 作为输出结果量: free_electron_laser
3. laser 作为 AMR/placement 测试对象: laser_on_fine

还需要再补一个当前证据层更弱、但应用语义很典型的角色: plasma_mirror。它的输入已经把 Gaussian laser、solid-density target、前后指数梯度、PML、field filter 和双 species 固体靶骨架接在一起, 因此在应用语义上非常像“laser-solid surface-plasma 最小样板”; 但当前 active regression 只有 checksum helper, 没有独立 analysis, 也没有 PICMI 版输入。所以它更适合在书稿里承担“过密靶/表面等离子体应用骨架已经存在, 但强物理断言仍未单独压实”的角色, 而不应被写成 plasma-mirror 反射率或高次谐波 benchmark。

还需要再补一句边界: laser_ion 当前并不是“多物理全开”的综合 benchmark。它的输入确实给了三条可切换分叉:

- hydrogen.do_field_ionization = 1
- collisions.collision_names = ...
- 将来再接 do_qed_*

但在当前 regression 版本里, 这些开关都没有同时启用。它真正激活的是“Gaussian laser + 预电离 target + full/time-averaged/reduced diagnostics”。因此更准确的写法应该是: laser_ion 提供了一个 laser-target 骨架, field ionization、collisions 和 QED 都可以从这个骨架分叉出去, 但它们各自的物理正确性仍然主要由 field_ionization/、collision/、qed/ 这些独立 regression 目录兜底, 而不是由 analysis_test_laser_ion.py 一次性证明。

从源码链看，这三条分叉接入的位置也不同。field ionization 在 species 构造期只先记住 do_field_ionization，真正的 InitIonizationModule().mapSpeciesProduct() 和 doFieldIonization() 要到 MultiParticleContainer::InitMultiPhysics 与推进循环里才发生；collisions 则走 CollisionHandler 和 collision_names，并额外受 collisions.split_momentum_push 的 operator ordering 影响；QED 又只在 #ifdef WARPX_QED 编译路径下才会继续增加 opticalDepthQSR/BW、product species 映射和 InitQED()。所以，laser_ion 更适合承担“应用输入如何把这些模块挂到同一目标骨架上”的说明，而不应把不同层级的验证合同混写成一条单一主链。

3A.12 初始化验证入口：哪些 regressions 真正在兜底

前面的 3A.1-3A.11 讲的是“源码如何初始化”；但如果没有本地 regressions 对照，这些讲解很容易停留在静态阅读层。当前 WarpX 对 Initialization 的验证并没有集中在一个目录里，而是分散在几组物理 test 中。

第一组是 Langmuir。它通常被当成 evolve 基准，但对初始化同样关键，因为它直接覆盖：

- NUniformPerCell
- profile=constant
- parse_momentum_function
- 周期边界下的初始粒子/场一致性

例如 analysis_1d.py 的主断言不是 checksum，而是把输出 Ez 与理论 Langmuir 波逐点比较：

```
E_sim = data[("mesh", field)].to_ndarray()[:, 0, 0]
E_th = get_theoretical_field(field, t0)
max_error = abs(E_sim - E_th).max() / abs(E_th).max()
assert error_rel < tolerance_rel
check_charge_conservation(data)
```

这意味着 Langmuir 不只是“时间推进跑通”，而是在验证 parser 形式的初始扰动确实经过 PlasmaInjector、SpeciesUtils 和 AddPlasma() 正确落到了粒子和场上。

第二组是 space_charge_initialization。它最直接对应 species.initialize_self_fields = 1 这条初始化支线。输入文件显式打开：

```
beam.injection_style = "gaussian_beam"
beam.initialize_self_fields = 1
beam.momentum_distribution_type = "at_rest"
```

而 analysis 脚本把输出 Ex/Ey/Ez 与高斯电荷团理论场直接比较。因此这组 test 实际上在硬验证：

1. gaussian_beam 初始粒子云生成正确；
2. InitData() 检测到 has_initialize_self_fields；
3. ComputeSpaceChargeField(reset_fields=false) 在第一个时间步前给出了正确的 Coulomb 场。

第三组是 dive_cleaning。它验证的不是 projection cleaner，而是：

- 初始 Gaussian beam 状态进入演化后，
- warpx.do_dive_cleaning = 1 与 PML 能否把 $\text{div}(\mathbf{E}) - \rho / \epsilon_0$ 误差传播并吸收掉，
- 最终场是否回到理论 Gaussian beam 电场。

第四组是 gaussian_beam / external_file。这里要分两半看：

1. analysis_focusing_beam.py 和 analysis_rotated_beam.py 分别验证 focal_distance、束斑统计、旋转位置和旋转动量；
2. inputs_test_3d_focusing_gaussian_beam_from_openpmd_prepare.py + inputs_test_3d_focusing_gaussian_beam_ 则覆盖 external_file openPMD 粒子注入合同，包括 weighting、mass、charge、positionOffset 和 momentum.unit_SI = m_e c。

这一组现在还能再细一层：

- inputs_test_3d_focusing_gaussian_beam_photons 不是新物理 benchmark，而是把同一聚焦束斑统计合同重复到 species_type = photon 路径；
- inputs_test_3d_gaussian_beam_picmi.py 则主要覆盖 PICMI GaussianBunchDistribution 前端到 runtime attributes 的接线，当前主要依赖 checksum，而不是独立理论断言。

这里还要诚实记录一个当前本地 checkout 的边界:gaussian_beam/CMakeLists.txt 给 test_3d_focusing_gaussian_beam_from_openpmd 指定了 analysis.py,但 Examples/Tests/gaussian_beam/ 目录下当前没有这个文件。因此项目没有把缺失的官方文件伪装成已恢复,而是对同一个 native producer 输出执行了项目独立分析。l rank 运行结果位于 runs/stage-c-validation/gaussian_beam_native_openpmd/run/: openPMD iteration 0 读出 1,999,966 个宏粒子,总权重为 1.999966e10, 81 个有效 z slice 上的最大相对束斑误差为:

$$\epsilon_{\sigma_x} = 3.0515 \times 10^{-2} < 0.051, \quad \epsilon_{\sigma_y} = 3.6214 \times 10^{-2} < 0.038.$$

官方 analysis_focusing_beam.py 与项目脚本 scripts/analyze_gaussian_beam_focus_contract.py 均对该 producer 输出通过。因而这条线现在可以写成:

- openPMD prepare -> native external_file inject -> plotfile/openPMD producer 链已真实运行;
- native 变体已有独立束斑物理分析,但该脚本不是 WarpX 官方 CMake analysis,证据等级是项目级补强而非 upstream CI 已修复;
- PICMI sibling 仍复用官方 analysis_focusing_beam.py, 两条输入路径的物理合同已经可以直接对照。

第五组是 electrostatic / EB 初始化:

- effective_potential_electrostatic 用电子径向密度和解析 adiabatic expansion 基准比较,验证 effective-potential electrostatic solver;
- electrostatic_sphere_eb 则用 ChargeOnEB reduced diag 和 eb_covered 场,验证 InitEB()、Poisson 边界条件和带导体球的初始势问题。

最后一组是 projection_div_cleaner, 它对应 3A.11 的 Poisson projection, 而不是演化阶段的 do_dive_cleaning。当前本地 tests 已覆盖:

1. RZ openPMD 文件外场版本;
2. 3D PICMI 文件外场版本;
3. Python callback 版本;
4. 2D 解析外场版本。

这些脚本的共同断言都是:初始化完成后,从 raw staggered Bx_aux/By_aux/Bz_aux 重建的离散 divB 必须足够接近零。

这里也要区分强断言的位置:

- test_rz_projection_div_cleaner 的强断言在独立 analysis.py 里;
- test_3d_projection_div_cleaner_picmi、test_3d_projection_div_cleaner_callback_picmi 和 test_2d_projection_div_cleaner_initial_analytical_field_picmi 则都把 divB 断言直接写在输入脚本尾部,所以 CMakeLists.txt 里虽然 analysis=OFF,但并不等于这些条目只是 checksum-only。

再往 species 入口侧补一组,本地还有一个直接锚定 setupNFluxPerCell() 的 regression 家族:Examples/Tests/flux_injection/。这组 tests 分三条:

1. analysis_flux_injection_3d.py
 - 对 3D NFluxPerCell 场景同时检查总发射量、法向 Gaussian-flux 分布和切向 Gaussian 分布;
2. analysis_flux_injection_rz.py
 - 对 flux_normal_axis = t 的 RZ 连续注入检查粒子始终停留在预期 Larmor 半径带,并保持正确总通量;
3. analysis_flux_injection_from_eb.py
 - 对 inject_from_embedded_boundary = 1 的 2D/3D/RZ 变体检查发射总数、法向/切向速度统计,以及粒子不会落入 EB 内部。

因此 flux_injection 的意义不是普通 emitter 示例,而是 NFluxPerCell、Gaussian-flux rejection sampling 和 embedded-boundary surface emission 这三条运行态合同的直接验证入口。

因此,把本章和 regression 对上之后,Initialization 目前可以压成这样一张验证图:

- parser 初始化与常规粒子装填: Langmuir
- gaussian_beam 与束流几何: focusing_gaussian_beam、rotated_gaussian_beam
- openPMD 粒子文件注入: focusing_gaussian_beam_from_openpmd*
- 初始 self-field: space_charge_initialization
- electrostatic / effective potential / EB: effective_potential_electrostatic、electrostatic_sphere_eb*
- projection cleaner: projection_div_cleaner*
- NFluxPerCell / flux injection: flux_injection*
- 演化态 div(E) cleaning: dive_cleaning

这张图的意义不在于宣称“初始化层已经被完全证明”，而在于把三类情况分清：

1. 已有显式物理量 hard assert 的路径；
2. 当前主要靠 checksum regression 的路径；
3. native gaussian_beam openPMD variant 仍保留官方 CMake analysis 缺失这一证据边界。

这张验证图不再只覆盖“场和束流自场”，也覆盖了初始化分布 API 本身。

第一组是 initial_distribution。它不是普通 smoke test，而是一组多 species、多分布的综合强基准：同一输入里同时覆盖

- gaussian
- maxwell_boltzmann
- maxwell_juttner
- gaussian_beam
- parser 温度
- parser bulk velocity
- uniform
- parser-Gaussian 动量统计

analysis 脚本把 reduced histogram、束斑统计和解析分布逐条对照。这意味着它真正验证的是 PlasmaInjector、SpeciesUtils 和 momentum-dispatch 层的 built-in / parser 初始化合同，而不只是“粒子能被建出来”。

当前 checkout 的重建 binary 已成功运行官方完整输入，官方 analysis.py 的最大相对差为 $1.8931e-2 < 0.02$ 。该结果修复了此前由预编译 binary 与源码 checkout 不一致造成的假阻塞；运行命令和逐项结果见 runs/stage-c-validation/initial_distribution_full_current/conda。由于初始化使用随机采样，仓库 checksum 的默认 $1e-9$ 不应被当作确定性合同；本次观测最大相对差为 $3.18e-3$ ，仅在显式记录的 $5e-3$ sampling tolerance 下通过。

第二组是 initial_plasma_profile。这组当前没有独立 analysis.py，只有 checksum helper，但输入本身非常明确：

- injection_style = NUniformPerCell
- profile = parse_density_function

并把横向 parabolic channel 与纵向 ramp / plateau / ramp 组合成二维电子密度。所以它更准确地是：

- parse_density_function 抛物型通道初始化的 checksum-only 基线

而不是应继续留在 general / to classify 的未知条目。

再往 initialize_self_fields 这一支补一组，本地还有一个更小但更干净的两体基准：repelling_particles。它只放两个同号 SingleParticle species，却同时打开：

- electron1.initialize_self_fields = 1
- electron2.initialize_self_fields = 1

analysis 随后从连续 plotfiles 读取两粒子的间距和速度，并用两体排斥的非相对论能量守恒关系构造理论 $\beta(d)$ 。因此这组 regression 的真实意义不是“一对粒子大概率会分开”，而是：

1. 初始 electrostatic self-field 确实被建立起来；
2. 后续 pusher 对这份初始场的消费是对的；
3. 两者联立后能回到解析两体减速关系。

接着把另外三组容易落单的入口也补上之后，初始化验证图还能再细一层。

第一组是 load_external_field。它不是普通粒子轨道测试，而是在验证两套初始化合同：

1. grid external field:
 - LoadExternalFields()
 - ReadExternalFieldFromFile()
 - Bfield_fp_external/Efield_fp_external
 - AddExternalFields()
2. particle external field:
 - Particles/ExternalParticleFields.cpp
 - m_B_ext_particle_s / m_E_ext_particle_s
 - B_external_particle_field/E_external_particle_field
 - GetExternalFields.cpp

analysis_3d.py / analysis_rz.py 通过磁镜中的单粒子最终位置做硬断言, 说明这组 regression 同时验证了“初始化写场”和“后续 gather 消费场”的接口契合。时间依赖变体 analysis_time_scaling.py 则不看粒子, 而是直接比较两个时刻 plotfile 上 B 分量的缩放比, 从而验证 read_fields_*_dependency(t) parser 和多 field map 的时间缩放合同。

这组 family 里还要再区分一层 restart 保真。当前活跃的三条最小入口是:

- test_3d_load_external_field_particle_time_restart
- test_rz_load_external_field_grid_restart
- test_rz_load_external_field_particles_restart

它们都只是:

- 先继承对应非 restart 输入
- 再通过 amr.restart = ../../chk000150 从中间 checkpoint 恢复
- 然后复用 analysis_default_restart.py 逐字段比较 restart 与非 restart 输出

因此这三条 regression 验证的不是新的外场 physics, 而是:

1. read_from_file 的 grid external field 状态能否在 restart 后保持一致;
2. read_from_file / dependency parser 的 particle external field 状态能否在 restart 后保持一致;
3. 初始化阶段构造出的 external-field 寄存器, 不会在 checkpoint/restart 边界上丢失或漂移。

第二组是 relativistic_space_charge_initialization。它的输入和普通 space_charge_initialization 一样也打开了:

```
beam.initialize_self_fields = 1
beam.injection_style = "gaussian_beam"
```

但束流动量改成了 relativistic:

```
beam.uz = 100.0
```

因此这组 regression 实际验证的是 RelativisticExplicitES::ComputeSpaceChargeField(), 而不再只是静止高斯电荷团的 Coulomb 场。analysis 脚本把 Ex 与理论值比较, 并检查 By 是否满足相对论束流自场的 $B_y \approx E_x/c$ 结构。这说明 initialize_self_fields 在 relativistic solver 分支下对应的是另一份初始化合同。

第三组是 open_bc_poisson_solver。它把四个条件绑在一起:

```
boundary.field_lo = open open open
boundary.field_hi = open open open
warpx.do_electrostatic = relativistic
warpx.poisson_solver = fft
electron.initialize_self_fields = 1
```

同时粒子不是 gaussian_beam, 而是 parse_density_function + NUniformPerCell。analysis 脚本用 Bassetti-Erskine 公式逐个 z 截面比较 Ex/Ey, 因此它验证的不是一般的 electrostatic 初始化, 而是:

- open boundary
- relativistic bunch
- FFT Poisson
- 以及可选 warpx.use_2d_slices_fft_solver = 1

共同定义的初始 Poisson 解是否正确。

把这些补进去以后, 初始化章节的本地回归证据就可以更完整地压成:

1. parser 初始化与常规宏粒子装填: Langmuir
2. gaussian_beam 注入几何: focusing_gaussian_beam, rotated_gaussian_beam
3. openPMD 粒子文件注入: focusing_gaussian_beam_from_openpmd*
4. lab-frame 初始 self-field: space_charge_initialization
5. relativistic 初始 self-field: relativistic_space_charge_initialization
6. electrostatic / effective potential / EB: effective_potential_electrostatic, electrostatic_sphere_eb*
7. 外部 grid / particle fields: load_external_field*
8. projection cleaner: projection_div_cleaner*
9. 开放边界 relativistic Poisson 初始化: open_bc_poisson_solver*
10. 演化态 div(E) cleaning: dive_cleaning

这样第 3A 章就不再只是“源码怎么走”，而是已经能回答“这些初始化合同在本地 WarpX 里分别由哪组 regression 兜底”。

继续补入 `load_density`、`magnetostatic_eb` 和 `nodal_electrostatic` 之后，这张验证地图还要再加三层。

第一层是 `load_density`。这组 regression 的输入明确使用：

```
electrons.profile = "read_from_file"
electrons.read_density_from_path = "../test_*_load_density_prepare/example-density.h5"
electrons.do_continuous_injection = 1
warpx.do_moving_window = 1
```

源码上它直接对应 `SpeciesUtils::parseDensity()` 里 `profile = read_from_file` 分支，把 openPMD density mesh 装进 `InjectorDensityFromFile`，然后交给 `PlasmaInjector` 和连续注入主链消费。`analysis` 脚本则逐个 iteration 读取 diagnostics 中的 `rho`，与 `prepare` 脚本定义的 ramp / parabolic channel profile 比较。因此 `load_density` 验证的不是一般 I/O，而是“file-driven density profile + moving-window 连续注入”这条初始化合同。

第二层是 `magnetostatic_eb`。原生 inputs 文件把：

```
warpx.do_electrostatic = labframe-electromagnetostatic
beam.initialize_self_fields = 1
warpx.eb_implicit_function = "(x**2+y**2-radius**2)"
warpx.eb_potential(x,y,z,t) = "1."
```

绑在一起，而 `WarpXInitData.cpp` 的 `fresh-run` 分支会在 `ComputeSpaceChargeField(reset_fields)` 之后继续调用 `ComputeMagnetostaticField()`。所以这组 test 验证的是 embedded boundary、边界 potential、初始 self-field 和 magnetostatic solve 在初始化阶段的联动。这里还必须区分两层证据：原生 `inputs_test_3d_magnetostatic_eb` 目前主要由 checksum 兜底；两个 PICMI 输入文件则在 `sim.step()` 之后内嵌了解析 `E_r/B_\theta` 误差断言，因此它们不是简单的 checksum-only regression。

第三层是 `nodal_electrostatic`。这组输入把：

```
warpx.do_electrostatic = relativistic
warpx.grid_type = collocated
beam_p.initialize_self_fields = 1
beam_p.do_qed_quantum_sync = 1
```

放在同一条链上。它的 `analysis` 不直接比较 `E/B`，而是用 reduced diagnostics 断言 `ParticleExtrema_beam_p` 给出的最大 `chi` 极小，且 `ParticleNumber` 中 `photon` 数始终为零。也就是说，这组 regression 验证的是 collocated relativistic electrostatic 初始 self-field 没有制造出会假触发 QED 的非物理场，它更准确地是一个“零触发基准”。

把这三组再并进来以后，初始化章节的本地回归证据可以进一步压成：

1. parser 初始化与常规宏粒子装填: `Langmuir`
2. gaussian_beam 注入几何: `focusing_gaussian_beam`, `rotated_gaussian_beam`
3. openPMD 粒子文件注入: `focusing_gaussian_beam_from_openpmd*`
4. file-driven density profile 与连续注入: `load_density*`
5. lab-frame 初始 self-field: `space_charge_initialization`
6. relativistic 初始 self-field: `relativistic_space_charge_initialization`
7. effective-potential electrostatic: `effective_potential_electrostatic`
8. electrostatic / magnetostatic / EB 联合初始化: `magnetostatic_eb*`
9. electrostatic / EB Poisson: `electrostatic_sphere_eb*`
10. 外部 grid / particle fields: `load_external_field*`
11. projection cleaner: `projection_div_cleaner*`
12. collocated relativistic electrostatic 零触发基准: `nodal_electrostatic`
13. 开放边界 relativistic FFT Poisson 初始化: `open_bc_poisson_solver*`
14. 演化态 `div(E)` cleaning: `dive_cleaning`

这样 `nodal_electrostatic`、`open_bc_poisson_solver` 和 `relativistic_space_charge_initialization` 就不再需要继续共用一个过粗的 `electrostatic / Poisson` 桶。它们分别对应的是：

- collocated relativistic electrostatic 零触发基准
- open boundary + FFT/sliced FFT 的 relativistic Poisson 初始化
- relativistic Gaussian beam 的初始 self-field

不过还有一组此前仍容易被写得过粗: `electrostatic_sphere`。它不该和一般 Poisson 条目混成一桶, 因为它真正验证的是“一个静止均匀电子球在自身 Coulomb 场下膨胀”这条自场初始化主链。`analysis_electrostatic_sphere.py` 的第一层断言是解析电场对照: 脚本用最终输出时间 `t_max` 反解球半径 `r_end`, 再构造内外球解析 $E(r)$, 沿坐标轴比较 $E_x/E_y/E_z$ 或 RZ 下的 E_r/E_z 的相对 $L2$ 误差。这意味着它验证的不仅是 solver 跑通, 而是:

1. 初始电子球几何是否被正确装填;
2. 初始自场是否正确建立;
3. 后续 `electrostatic` 演化是否仍与解析膨胀解一致。

这组 test 的第二层断言只在 lab-frame 变体上才打开。只有当输入显式要求:

```
warpx.do_electrostatic = labframe
diag2.electron.variables = x y z ux uy uz w phi
```

analysis 才会利用粒子 `phi` 重建:

- 动能
- 自场势能 $0.5 \sum w q \phi$

并检查初末总能量。也就是说:

- 所有 `electrostatic_sphere` 变体都做解析电场对照;
- 只有写出 `phi` 的 lab-frame 版本额外验证能量账本。

各输入变体的角色也不同:

- `inputs_test_3d_electrostatic_sphere`
 - 3D relativistic electrostatic 自场膨胀基线
- `inputs_test_3d_electrostatic_sphere_lab_frame`
 - lab-frame 自场膨胀 + 能量守恒
- `inputs_test_3d_electrostatic_sphere_lab_frame_mr_emass_10`
 - lab-frame + MR; `electron.mass = 10` 主要是减慢膨胀, 便于短步数比较
- `inputs_test_3d_electrostatic_sphere_rel_nodal`
 - `warpx.grid_type = collocated`, 验证 collocated electrostatic 布局
- `inputs_test_3d_electrostatic_sphere_adaptive`
 - adaptive-dt 变体; analysis 仍用实际 `t_max` 做强对照
- `inputs_test_rz_electrostatic_sphere`
 - RZ 自场膨胀 + 能量守恒
- `inputs_test_rz_electrostatic_sphere_uniform_weighting`
 - 再额外覆盖 `radial_numpercell_power = 1.` 的 uniform-weighting 粒子装填

另外, 这组目录下还有两个容易误读的文件:

- `analysis_default_regression.py` 只是通用 checksum helper;
- `catalyst_pipeline.py` 只是 ParaView Catalyst 的可视化脚本。

因此, `electrostatic_sphere` 在初始化验证图里更准确的位置应单独写成:

14. electrostatic self-field expansion: `electrostatic_sphere*`

还剩另外两组此前也容易被写得过粗, 但它们和 `electrostatic_sphere` 不是一类问题。

第一组是 `electrostatic_dirichlet_bc`。这组不是带电粒子自场 benchmark, 而是纯粹在验证时变边界势是否真正进入 electrostatic 解。输入直接设置:

```
warpx.do_electrostatic = labframe
boundary.potential_lo_x = 150.0*sin(2*pi*6.78e+06*t)
boundary.potential_hi_x = 450.0*sin(2*pi*13.56e+06*t)
diag1.fields_to_plot = phi
```

analysis 并不读取内部粒子量, 而是逐个输出时刻取两侧边界上的平均 `phi`, 然后与理论正弦函数比较。因此它测的不是一般 Poisson 正确性, 而是:

- `boundary.potential_lo_x / potential_hi_x`
- time-dependent parser
- electrostatic Dirichlet boundary condition

- diagnostics 中 phi 的边界保真

PICMI 变体只是在 `Cartesian2DGrid(..., lower_boundary_conditions=["dirichlet", ...])`、`warpx_potential_lo_x`、`warpx_potential_hi_x` 这一层重复同一件事，所以它更准确的归类是：

15. time-dependent electrostatic boundary driving: `electrostatic_dirichlet_bc*`

第二组是 `effective_potential_electrostatic`。这组当前只有一个 PICMI test，而且它验证的也不是一般意义上的 electrostatic phi 场，而是 `warpx_effective_potential=True` 这条 solver 分支是否能在导体球约束下复现绝热膨胀 benchmark。PICMI 输入脚本同时做了三件关键事：

1. 用 `GaussianBunchDistribution` 初始化电子和离子球团；
2. 加入导体球 `embedded boundary`；
3. 选择：

```
picmi.ElectrostaticSolver(
    method="Multigrid",
    warpx_effective_potential=True,
    warpx_effective_potential_factor=C_EP,
)
```

analysis 则从 `sim_parameters.dpk1` 读回参数，构造 Connor et al. 风格的绝热膨胀近似电子密度，再把 openPMD 中的 `rho_electrons` 变成径向密度曲线，与理论值逐个输出时刻比较 RMS 误差。因此它的真实验证对象是：

- effective-potential electrostatic solver
- PICMI front-end
- 导体球约束下的电子密度膨胀近似

更准确的归类应是：

16. effective-potential electrostatic: `effective_potential_electrostatic`

3A.13 本章小结：初始化状态怎样进入第一步推进

到 `InitData()` 结束时，WarpX 已经完成：

```
inputs
-> WarpX::ReadParameters()
-> WarpX::InitData()
-> InitFromScratch or InitFromCheckpoint
-> AMReX levels and WarpX field data
-> external fields
-> species PlasmaInjector
-> AddParticles / AddPlasma / AddGaussianBeam / AddPlasmaFromFile
-> projection div cleaner if needed
-> initial diagnostics
-> Evolve()
```

这个状态才是 PIC 时间推进的初始条件。后续 `Evolve()`、`OneStep()`、`PushParticlesandDeposit()`、field solver 和 diagnostics 都在这个已经离散化、分布式、带权重和边界条件的状态上工作。

后续扩写方向：

- 把 `InitLevelData()` 中每一类 field allocation 展开到 `root/fieldsolver` 章节；
- 把 Gaussian beam 的 emittance/focal distance 公式结合 accelerator beam optics 文献继续推导；
- 把 openPMD 文件格式与 WarpX 单位约定加入诊断/I/O 章节；
- 判断 initialization 验证层是否已经阶段性收口，并切回下一未完成模块。

3A.14 练习与最小复现

1. **fresh/restart 定位题**：沿 `WarpXInitData.cpp` 追踪 `InitData()`，说明 `ComputeDt()` 为什么只出现在 fresh-run 分支，以及 restart 为什么必须进入 `PostRestart()`。

2. 初始化顺序题: 解释 `AmrCore::InitFromScratch()`、`AllocLevelData()`、`mypc->AllocData()`、`mypc->InitData()` 和 `InitPML()` 的先后关系; 指出把粒子初始化提前到 AMR level 创建之前会破坏哪类对象合同。
3. 复现实验题: 选取 `Examples/Tests/initial_distribution/` 中一个当前 binary 能运行的输入, 记录 `warpx_used_inputs`、首个 diagnostics 和粒子 species 数量; 若遇到 binary/input checkout 不匹配, 保留 abort 位置并说明它为什么不能被当作通过证据。

4. 粒子推进器: 从 Lorentz 方程到 PushPX()

粒子推进器负责把单个宏粒子从一个时间层推进到下一个时间层。它表面上是局部单粒子算法, 实际上依赖上一章建立的主循环条件: 场必须已经填好 gather guard cells, 粒子位置和动量必须处在正确 leapfrog 时间层, 外场、电离电荷态、辐射反作用和 QED 选项也必须在进入 pusher 前准备好。

本章对应源码笔记见 `notes/code-reading/particles/00-particle-evolve-callchain.md`、`notes/code-reading/particles/01-particle-evolve-calls.md`、`notes/code-reading/particles/02-gather-shape-deposition-kernels.md`、`notes/code-reading/particles/03-vay-high-order-derivatives.md`、`notes/code-reading/particles/24-thermalizer-validation-and-checksum-boundaries.md`、`notes/code-reading/particles/26-pusher-single-particle-and-photon-validation-map.md`、`notes/code-reading/particles/28-particle-diagnostics-python-interface-validation-map.md`、`notes/code-reading/particles/29-particle-diagnostics-validation-map.md` 和 `notes/code-reading/particles/30-pml-eb-contact-and-mr-validation-map.md`。

本章当前依据的 WarpX 源码版本是:

- `../warpx`
- 分支: `pkuHEDPbranch`
- commit: `8c488b1a9`

v0.3 校准说明: 本章已按当前 checkout 复核 `UpdateMomentumBoris.H`、`PushSelector.H`、`UpdateMomentumVay.H`、`UpdateMomentumHigueraCary.H`、`WarpXEvolve.cpp`、`MultiParticleContainer.cpp` 与 `PhysicalParticleContainer.cpp` 的主链行号。尤其需要注意: 当前 Boris half push 不再简单把磁旋转系数减半, 而是按 Birdsall-Langdon 半角关系重标定 t , 因此本章已同步修正旧草稿中的 `bconst` 写法。 `Examples/Tests/particle_pusher` 仍是本章当前最直接的 Higuera-Cary force-free 强验证入口。

4.1 连续 Lorentz 方程

WarpX 对带质量粒子推进的基本方程是相对论 Lorentz 方程。令

$$\mathbf{u} = \gamma \mathbf{v}, \quad \gamma = \sqrt{1 + \frac{|\mathbf{u}|^2}{c^2}}, \quad \mathbf{v} = \frac{\mathbf{u}}{\gamma}.$$

这里 WarpX 源码中的 `ux`, `uy`, `uz` 是 $\mathbf{u}$ 的三个分量, 而不是普通速度 $\mathbf{v}$ 。对质量 m 、电荷 q 的粒子,

$$\frac{d\mathbf{u}}{dt} = \frac{q}{m} (\mathbf{E} + \mathbf{v} \times \mathbf{B}),$$

$$\frac{d\mathbf{x}}{dt} = \mathbf{v} = \frac{\mathbf{u}}{\gamma}.$$

显式 leapfrog PIC 中, 常用时间层是

$$\mathbf{x}^n \rightarrow \mathbf{x}^{n+1}, \quad \mathbf{u}^{n-1/2} \rightarrow \mathbf{u}^{n+1/2}.$$

因此位置推进使用更新后的半步动量:

$$\mathbf{x}^{n+1} = \mathbf{x}^n + \frac{\mathbf{u}^{n+1/2}}{\gamma^{n+1/2}} \Delta t.$$

WarpX 的显式位置更新正是这个公式, 见 `../warpx/Source/Particles/Pusher/UpdatePosition.H:19-70`。

这里可以补一个 Dawson 1983 的历史边界。那篇综述在 `electromagnetic particle models and fractional dimensional models` 一节明确指出: 只要问题涉及自洽磁场或 `electromagnetic radiation`, particle model 就不能再被理解成“electrostatic 外面多加几个场分量”, 而必须回到 full Maxwell equations 与 self-consistent current/charge representation。与此同时, 低空间维度并不意味着

低速度维度；为了在一维或二维空间里承载 electromagnetic wave，仍必须保留 transverse $E/B/j$ 与 transverse velocity，于是才会自然出现 1 1/2-D、1 2/2-D 和 2 1/2-D 这些 fully electromagnetic reduced-dimension models。这个边界正好解释了为什么本章后面很多看似“低维”的 laser、beam 和 FEL benchmark，仍然必须认真讨论 magnetic rotation、relativistic momentum 与 transverse field coupling，而不能把它们误降成纯 electrostatic pusher。

4.2 Boris 推进的结构

Boris pusher 把动量更新拆成三个操作：

1. 电场半步；
2. 磁场旋转；
3. 电场半步。

设旧动量为 $\mathbf{u}^{n-1/2}$ ，先做电半步：

$$\mathbf{u}^- = \mathbf{u}^{n-1/2} + \frac{q\Delta t}{2m} \mathbf{E}^n.$$

再用 $\mathbf{u}^-$ 计算

$$\gamma^- = \sqrt{1 + \frac{|\mathbf{u}^-|^2}{c^2}}, \quad \mathbf{t} = \frac{q\Delta t}{2m\gamma^-} \mathbf{B}^n, \quad \mathbf{s} = \frac{2\mathbf{t}}{1 + |\mathbf{t}|^2}.$$

磁场旋转写成

$$\mathbf{u}' = \mathbf{u}^- + \mathbf{u}^- \times \mathbf{t},$$

$$\mathbf{u}^+ = \mathbf{u}^- + \mathbf{u}' \times \mathbf{s}.$$

最后做第二个电半步：

$$\mathbf{u}^{n+1/2} = \mathbf{u}^+ + \frac{q\Delta t}{2m} \mathbf{E}^n.$$

WarpX 的实现见 ../warpX/Source/Particles/Pusher/UpdateMomentumBoris.H:20-75:

源码原文如下：

```
const amrex::ParticleReal econst = 0.5_prt*q*dt/m;

if (momentum_push_type == MomentumPushType::FirstHalf || momentum_push_type == MomentumPushType::Full) {
    // First half-push for E
    ux += econst*Ex;
    uy += econst*Ey;
    uz += econst*Ez;
}
// Compute temporary gamma factor
constexpr auto inv_c2 = PhysConst::inv_c2_v< amrex::ParticleReal>;
const amrex::ParticleReal inv_gamma = 1._prt/std::sqrt(1._prt + (ux*ux + uy*uy + uz*uz)*inv_c2);
// Magnetic rotation
amrex::ParticleReal tx = econst*inv_gamma*Bx;
amrex::ParticleReal ty = econst*inv_gamma*By;
amrex::ParticleReal tz = econst*inv_gamma*Bz;
if (momentum_push_type == MomentumPushType::FirstHalf || momentum_push_type == MomentumPushType::SecondHalf) {
    const amrex::ParticleReal tsq = tx*tx + ty*ty + tz*tz;
    const amrex::ParticleReal factor = (tsq > 0._prt) ? (std::sqrt(1._prt + tsq) - 1._prt) / tsq : 0.5_prt;
    tx *= factor;
```

```

    ty *= factor;
    tz *= factor;
}
const amrex::ParticleReal tsqi = 2._prt/(1._prt + tx*tx + ty*ty + tz*tz);
const amrex::ParticleReal sx = tx*tsqi;
const amrex::ParticleReal sy = ty*tsqi;
const amrex::ParticleReal sz = tz*tsqi;
const amrex::ParticleReal ux_p = ux + uy*tz - uz*ty;
const amrex::ParticleReal uy_p = uy + uz*tx - ux*tz;
const amrex::ParticleReal uz_p = uz + ux*ty - uy*tx;
// - Update momentum
ux += uy_p*sz - uz_p*sy;
uy += uz_p*sx - ux_p*sz;
uz += ux_p*sy - uy_p*sx;
if (momentum_push_type == MomentumPushType::SecondHalf || momentum_push_type == MomentumPushType::Full){
// Second half-push for E
    ux += econst*Ex;
    uy += econst*Ey;
    uz += econst*Ez;
}

```

行号	代码动作	公式对应
:28	econst = 0.5*q*dt/m	$\frac{q\Delta t}{2m}$
:30-35	FirstHalf 或 Full 时做电半步	$\mathbf{u}^{n-1/2} \rightarrow \mathbf{u}^-$
:37-43	计算 inv_gamma 和 full-step t	γ^-, t
:44-57	FirstHalf/SecondHalf 时按半角关系重标定 t	使两次 half push 合成一次 Full 的磁旋转
:58-68	磁旋转	$\mathbf{u}^- \rightarrow \mathbf{u}^+$
:69-74	SecondHalf 或 Full 时做第二个电半步	$\mathbf{u}^+ \rightarrow \mathbf{u}^{n+1/2}$

文件注释 :13-18 说明 FirstHalf 和 SecondHalf 可以分裂执行, 并且连续执行应与一次 Full 更新数学等价。这正好服务于 WarpXEvolve.cpp 中把碰撞放在 momentum push 中间的路径。

这里有一个 v0.3 必须补上的实现细节: 当前 WarpX 的 half momentum push 不是简单把 $q \, dt \, B/(2m\gamma)$ 再乘 1/2。代码在 UpdateMomentumBoris.H:44-57 使用

$$\frac{|t_{\text{half}}|}{|t_{\text{full}}|} = \frac{\sqrt{1 + |t_{\text{full}}|^2} - 1}{|t_{\text{full}}|^2}$$

重标定 tx,ty,tz。这来自 $\tan(\alpha/2)$ 与 $\tan(\alpha/4)$ 的半角关系, 目的是让 FirstHalf 后接 SecondHalf 的组合仍对应 full Boris rotation, 而不是两个朴素半磁旋转的近似拼接。

4.3 WarpX 如何选择 pusher

WarpX 的单粒子动量推进分派在 ../warpx/Source/Particles/Pusher/PushSelector.H:39-104。函数名是 doParticleMomentumPush()

行号	选择
:61-62	species 电荷乘 ion_lev, 得到当前粒子的有效电荷。
:64-88	若启用 classical radiation reaction, 进入 UpdateMomentumBorisWithRadiationReaction()。
:89-92	ParticlePusherAlgo::Boris 进入 UpdateMomentumBoris()。
:93-96	ParticlePusherAlgo::Vay 进入 UpdateMomentumVay()。
:97-100	ParticlePusherAlgo::HigueraCary 进入 UpdateMomentumHigueraCary()。

这说明输入参数选择的不是一个高层“粒子模块”，而是每个粒子在 `PushPX()` device loop 内调用的单粒子 momentum update。下面继续展开 Vay 与 Higuera-Cary 的源码。

这一节背后的经典来源可以直接回到 Birdsall-Langdon 1985 第一分卷 4-3 到 4-5。那里把磁推进的核心先写成几何分裂：电场部分是半步 impulse，磁场部分是速度空间旋转；随后再把旋转压成 $t = \tan(\theta/2)$ 、 $s = 2t/(1+t^2)$ 、 $c = (1-t^2)/(1+t^2)$ 这组半角变量，并进一步给出向量 Boris 形式。对本章来说，这个来源有两个价值。第一，WarpX 的 Boris 更新不是孤立经验公式，而是这条“half-accel + rotation + half-accel”离散合同的现代实现。第二，Birdsall 在 4-5 里明确区分了 1d2v/1d3v 和真正的一维动力学，这正好解释了为什么即使空间维数较低，本章后面讨论的 mover 仍必须保留多速度分量与磁旋转结构。

物理上可以先记住：

- Boris：经典、鲁棒、磁场部分近似旋转，长期性质好。
- Vay：针对相对论漂移和 Lorentz 变换一致性问题设计，在 boosted frame 场景常用。
- Higuera-Cary：相对论粒子推进的结构保持改进，常用于减少高相对论问题中的系统误差。

正式误差比较必须回到对应论文和 benchmark；本章先把 WarpX 的实际实现讲清楚。

4.4 Vay pusher：相对论速度变换一致性的更新

Vay pusher 的 WarpX 实现在 `../warpX/Source/Particles/Pusher/UpdateMomentumVay.H:17-77`。源码注释明确引用 Vay 2008 的公式 (9)-(13)，并说明 FirstHalf 与 SecondHalf 连续执行应等价于 Full：

```
/** \brief Push the particle's positions over one timestep,
 *   given the value of its momenta `ux`, `uy`, `uz`
 *   Note that UpdateMomentumVay algorithm can be splitted into FirstHalf and SecondHalf
 *   momentum updates. FirstHalf and SecondHalf updates are constructed so that
 *   performing FirstHalf followed by SecondHalf is mathematically
 *   equivalent to a single Full update.
 *   For more details, see formulas (9)-(13) in J.L.Vay, "Simulation of beams or plasmas crossing at
 *   relativistic velocity", Phys. Plasmas 15, 056701 (2008).
 */
AMREX_GPU_HOST_DEVICE AMREX_INLINE
void UpdateMomentumVay(
    amrex::ParticleReal& ux, amrex::ParticleReal& uy, amrex::ParticleReal& uz,
    const amrex::ParticleReal Ex, const amrex::ParticleReal Ey, const amrex::ParticleReal Ez,
    const amrex::ParticleReal Bx, const amrex::ParticleReal By, const amrex::ParticleReal Bz,
    const amrex::ParticleReal q, const amrex::ParticleReal m, const amrex::Real dt,
    MomentumPushType momentum_push_type)
{
    using namespace amrex::literals;

    // Constants
    const amrex::ParticleReal econst = q*dt/m * ((momentum_push_type == MomentumPushType::Full) ? 1.0_prt
    const amrex::ParticleReal bconst = 0.5_prt*q*dt/m;
    // Compute initial gamma
    const amrex::ParticleReal inv_gamma = 1._prtlstd::sqrt(1._prt + (ux*ux + uy*uy + uz*uz)*PhysConst::inv
    // Get tau
    const amrex::ParticleReal taux = bconst*Bx;
    const amrex::ParticleReal tauy = bconst*By;
    const amrex::ParticleReal tauz = bconst*Bz;
    const amrex::ParticleReal tausq = taux*taux+tauy*tauy+tauz*tauz;
    // Get U', gamma'^2
    const amrex::ParticleReal uxpr = ux + econst*Ex + ((momentum_push_type == MomentumPushType::SecondHalf
    const amrex::ParticleReal uypr = uy + econst*Ey + ((momentum_push_type == MomentumPushType::SecondHalf
    const amrex::ParticleReal uzpr = uz + econst*Ez + ((momentum_push_type == MomentumPushType::SecondHalf

    if (momentum_push_type != MomentumPushType::FirstHalf) {
        // Get gamma'^2
        const amrex::ParticleReal gprsq = (1._prt + (uxpr*uxpr + uypr*uypr + uzpr*uzpr)*PhysConst::inv_c2_
```

```

// Get u*
const amrex::ParticleReal ust = (uxpr*taux + uypr*tauy + uzpr*tauz)*PhysConst::inv_c_v<amrex::Part
// Get new gamma
const amrex::ParticleReal sigma = gprsqr-tausq;
const amrex::ParticleReal gisq = 2._prt/(sigma + std::sqrt(sigma*sigma + 4._prt*(tausq + ust*ust))
// Get t, s
const amrex::ParticleReal bg = bconst*std::sqrt(gisq);
const amrex::ParticleReal tx = bg*Bx;
const amrex::ParticleReal ty = bg*By;
const amrex::ParticleReal tz = bg*Bz;
const amrex::ParticleReal s = 1._prt/(1._prt+tausq*gisq);
// Get t.u'
const amrex::ParticleReal tu = tx*uxpr + ty*uypr + tz*uzpr;
// Get new U
ux = s*(uxpr+tx*tu+uypr*tz-uzpr*ty);
uy = s*(uypr+ty*tu+uzpr*tx-uxpr*tz);
uz = s*(uzpr+tz*tu+uxpr*ty-uypr*tx);
}
else {
ux = uxpr;
uy = uypr;
uz = uzpr;
}
}

```

Vay pusher 和 Boris 的主要差别在 :51-70。Boris 使用旧的 γ^- 构造磁旋转；Vay 先构造 $\mathbf{u}'$ ，再通过

$$\sigma = \gamma'^2 - \tau^2, \quad \gamma_{\text{new}}^{-2} = \frac{2}{\sigma + \sqrt{\sigma^2 + 4(\tau^2 + u_*^2)}}$$

解析得到新的 γ 相关量。源码中的 `gisq` 就是这里的 γ_{new}^{-2} ，`bg=bconst*sqrt(gisq)` 对应 $(q\Delta t/2m)/\gamma_{\text{new}}$ 。这使磁旋转使用与更新后状态一致的相对论因子，适合 boosted-frame 或高速束流穿越问题。

FirstHalf 路径只返回 $\mathbf{u}'$ ，SecondHalf 路径不会再加入旧速度磁项。这和 Boris 文件中的分裂设计一致，服务于碰撞、外力或隐式/显式混合路径。

把这段源码重新放回 Vay 2008 原文，边界会更清楚。Vay 论文的第一性问题不是“再发明一个 relativistic mover”，而是对 relativistic crossing / boosted-frame 场景，离散 Lorentz force 必须尽量保住 electric 和 magnetic contributions 的 cancellation property。作者先用

$$\mathbf{E} + \mathbf{v} \times \mathbf{B} = 0$$

这个试金石说明 Boris 在一般非零 $\mathbf{E}$ 、 $\mathbf{B}$ 情况下会出现 spurious force，然后才提出新的 leapfrog velocity average。于是 WarpX 的 UpdateMomentumVay.H 最值得保留的历史定位不是“Boris 的另一个变体”，而是：它实现的是一条以 frame-change consistency 为目标的专门修正路线。

Vay 2008 后半段把这条历史论证链又压实了两次。第一，II.C 的两个单粒子测试并不是普通轨道展示，而是同一物理系统在 laboratory frame 和 moving frame 下都要和解析解一致的 frame-consistency test。常量 B_z 例子中，粒子以 $v_x=10^{-2}c$ 起步、 $\Delta t=10^{-2}\times 2\pi/\omega_c$ ，新 pusher 在实验室系和沿 $\hat{y}$ 方向 $\gamma_f=2$ 的 moving frame 里都贴住解析轨道；Boris 即使加上 $\tan(\omega_c\Delta t)/(\omega_c\Delta t)$ 修正，也会在 moving frame 中明显偏离，而且误差在 $\gamma_f=3$ 后迅速放大。常量 $E_x=1\text{ kV/m}$ 、电子在实验室系初始静止、100 步 1 ns 更新的例子更直接：三种 mover 在实验室系里都正确，只有新 pusher 在 $\gamma_f=100$ 的 moving frame 中仍保持解析一致。于是这篇文献给本章的最硬证据不是“Vay 在某些 case 里更稳”，而是 Boris 的误差会在 frame change 后由次要项变成主导项。

第二，这篇论文并没有顺手给出一个通用 Maxwell solver。它在 III 节明确把场求解边界限定在 waves 和 retardation 可忽略、并且对每个 species 可以在共动系里近似取

$$v_z \gg v_x, v_y, \quad \frac{\partial}{\partial t} \approx v_z \frac{\partial}{\partial z}$$

的场景。于是 field side 被压成带 γ z 拉伸的 Poisson 型求解，并近似保留 electrostatic、magnetostatic 以及沿主流向的 inductive effect。对 N 个 species，代价就是 N 次这类 Poisson solve。这条 bounded Darwin-lite explicit approximation 解释了为什么 IV 节的 LHC-like ultrarelativistic beam / electron-cloud 应用会特意选在 $\gamma \approx 16.5$ 的 moving frame 中做 first-principles PIC：在那里 beam 与 electron cloud 的 self-electric / self-magnetic cancellation 最强，最能放大 mover 的 frame-consistency 缺陷。文中报告 Boris 无论是否带 $\tan$ 修正，都会让 beam 和 electron 宏粒子以非物理速度丢失；只有新 pusher 才能恢复预期的 hose-like instability，并且给出和实验室系 quasistatic WARP calculation 一致的 vertical emittance growth rate 与 saturation level。因此，对 WarpX 而言，UpdateMomentumVay.H 的历史角色应理解成 relativistic beam-crossing / boosted-frame consistency repair，而不是一个与一般场求解器或一般 relativistic mover 等价并列的“备选算法”。

4.5 Higuera-Cary pusher: Boris-like 结构的相对论修正

Higuera-Cary pusher 的 WarpX 实现在 ../warpx/Source/Particles/Pusher/UpdateMomentumHigueraCary.H:16-65:

```
template <typename T>
AMREX_GPU_HOST_DEVICE AMREX_INLINE
void UpdateMomentumHigueraCary(
    T& ux, T& uy, T& uz,
    const T Ex, const T Ey, const T Ez,
    const T Bx, const T By, const T Bz,
    const T q, const T m, const amrex::Real dt )
{
    using namespace amrex::literals;

    // Constants
    const T qmt = 0.5_prt*q*dt/m;
    // Compute u_minus
    const T umx = ux + qmt*Ex;
    const T umy = uy + qmt*Ey;
    const T umz = uz + qmt*Ez;
    // Compute gamma squared of u_minus
    T gamma = 1._prt + (umx*umx + umy*umy + umz*umz)*PhysConst::inv_c2_v<T>;
    // Compute beta and betam squared
    const T betax = qmt*Bx;
    const T betay = qmt*By;
    const T betaz = qmt*Bz;
    const T betam = betax*betax + betay*betay + betaz*betaz;
    // Compute sigma
    const T sigma = gamma - betam;
    // Get u*
    const T ust = (umx*betax + umy*betay + umz*betaz)*PhysConst::inv_c_v<T>;
    // Get new gamma inverse
    gamma = 1._prt/std::sqrt(0.5_prt*(sigma + std::sqrt(sigma*sigma + 4._prt*(betam + ust*ust))) );
    // Compute t
    const T tx = gamma*betax;
    const T ty = gamma*betay;
    const T tz = gamma*betaz;
    // Compute s
    const T s = 1._prt/(1._prt+(tx*tx + ty*ty + tz*tz));
    // Compute um dot t
    const T umt = umx*tx + umy*ty + umz*tz;
    // Compute u_plus
    const T upx = s*( umx + umt*tx + umy*tz - umz*ty );
    const T upy = s*( umy + umt*ty + umz*tx - umx*tz );
    const T upz = s*( umz + umt*tz + umx*ty - umy*tx );
    // Get new u
    ux = upx + qmt*Ex + upy*tz - upz*ty;
    uy = upy + qmt*Ey + upz*tx - upx*tz;
```

```
uz = upz + qmt*Ez + upx*ty - upy*tx;
}
```

前半段 :31-35 仍是电场半步:

$$\mathbf{u}^- = \mathbf{u}^{n-1/2} + \frac{q\Delta t}{2m} \mathbf{E}.$$

随后源码用 `beta=qmt*B` 和 `u_minus` 计算

$$\sigma = \gamma_-^2 - \beta^2, \quad u_* = \frac{\mathbf{u}^- \cdot \beta}{c},$$

并通过 :48 得到新的 `gamma`, 这里变量名 `gamma` 在这一行之后实际保存的是 γ^{-1} 。`tx,ty,tz` 是 β/γ , $s=1/(1+t^2)$ 。`u_plus` 的形式像 Boris 的磁旋转, 但最后 :62-64 不是单纯加第二个电半步, 而是

$$\mathbf{u}^{n+1/2} = \mathbf{u}^+ + \frac{q\Delta t}{2m} \mathbf{E} \mathbf{u}^+ \times \mathbf{t}.$$

这个额外叉乘项是 Higuera-Cary 结构和 Boris 结构最容易混淆的地方。源码中 `upy*tz-upz*ty`、`upz*tx-upx*tz`、`upx*ty-upy*tx` 正是 $\mathbf{u}^+ \times \mathbf{t}$ 的三个分量。

把这段 kernel 放回 Higuera-Cary 2017 原文, WarpX 里这条算法线的真实边界会更清楚。那篇论文并不是在 Vay 2008 的 boosted-frame cancellation 问题上继续竞争, 而是把 Boris、Vay 和新方法放到三个并列判据下比较: $\mathbf{E}=0$ 时的能量守恒、crossed $\mathbf{E}/\mathbf{B}$ 场下正确的 $\mathbf{E} \times \mathbf{B}$ drift、以及 phase-space volume preservation。作者的核心判断是: Boris 保住 volume 但 drift 不对; Vay 保住 drift 但不 volume-preserving; Higuera-Cary 则是三者中唯一同时保住 volume 与 $\mathbf{E} \times \mathbf{B}$ drift 的二阶 relativistic momentum integrator。因此, `UpdateMomentumHigueraCary.H` 最准确的历史定位不是“又一个 Boris-like 变体”, 而是: 在 Boris 的 rotation skeleton 上, 把 centered average 和 γ 的 prescription 改写成一条双结构保持路线。

Higuera-Cary 2017 的 III-VI 节还把这个判断压成了 WarpX 读者真正需要的两层证据。第一层是实现证据: 新方法表面上仍是 implicit centered scheme, 但最终可以像 Boris/Vay 一样显式实现, 真正的实现分叉点几乎全部浓缩在 γ_{new} 的求法上。这正好解释了为什么 WarpX 源码里 `UpdateMomentumHigueraCary.H` 的外形与 Boris 非常接近, 却在 `sigma`、`ust` 和新的 relativistic factor 路径上分叉。第二层是数值/几何证据: 作者用 Poincare surface of section 而不是普通能量曲线来比较 practical timestep 下的轨道拓扑。小时间步时三种方法都能给出嵌套曲线; 但在更接近实际模拟的 $\Delta t=1/10$ 下, Vay 会出现 resonance island 和不同轨道 section 交叉, 而 Boris 与 Higuera-Cary 仍保持正确的 phase-space topology。对本章来说, 这意味着 Higuera-Cary 的价值判断标准不是 Vay 2008 那种 frame-change consistency, 而是 geometric/topological preservation at practical timestep。

如果进一步把论文公式和 WarpX kernel 逐式对位, 这条关系会更直观。Higuera-Cary 论文先定义

$$\vec{\epsilon} = \frac{q\Delta t}{2m} \vec{E}, \quad \vec{\beta} = \frac{q\Delta t}{2m} \vec{B}, \quad \vec{u}_- = \vec{u}_i + \vec{\epsilon},$$

而源码里的 `qmt`、`umx/umy/umz`、`betax/betay/betaz` 正是这些量。随后论文显式求解

$$\gamma_{new}^2 = \frac{1}{2} \left(\gamma_-^2 - \beta^2 + \sqrt{(\gamma_-^2 - \beta^2)^2 + 4(\beta^2 + |\vec{\beta} \cdot \vec{u}_-|^2)} \right),$$

WarpX 则不先存 γ_{new}^2 , 而是直接用 `sigma = gamma - betam`、`ust = (u_- · beta)/c` 和下一行的平方根公式, 把变量 `gamma` 改写成 γ_{new}^{-1} 。这一点很关键: 代码里的 `gamma` 在函数中段被重载了, 前半段是 γ_-^2 , 后半段则变成 rotation 要消费的 inverse relativistic factor。之后 `tx/ty/tz`、`s`、`umt`、`upx/upy/upz` 这条链, 就是论文 Boris-like rotation equation

$$\vec{u}_+ - \vec{u}_- = (\vec{u}_+ + \vec{u}_-) \times \frac{\vec{\beta}}{\gamma_{new}}$$

的直接实现。因此, `UpdateMomentumHigueraCary.H` 最本质的实现差异可以压成一句话: 它在 Boris 的 rotation skeleton 上, 仅通过改写 γ prescription, 就把 volume-preserving 与 $\mathbf{E} \times \mathbf{B}$ drift preservation 这两条性质同时保住了。

论文第 V 节的 Jacobian 证明也让这一点不再停留在口头层。作者证明新 integrator 的前后半步 Jacobian determinant 互为倒数, 所以一步更新的总体 Jacobian 恰好等于 1; 而 Vay 的 Jacobian 一般写成 $J(\mathbf{x}_i, \mathbf{u}_i)/J(\mathbf{x}_i, \mathbf{u}_f)$ 这样的比值, 通常不会化成 1。这正是后面数

值例子里 Vay 在 practical timestep 下出现 resonance island 和轨道交叉, 而 Higuera-Cary 仍保持 phase-space topology 的几何根源。

如果把这段证明再往下压一层, 最关键的中间对象其实是 $I - \Omega$ 。Higuera-Cary 论文先把后半步对 $\bar{u}_{new}$ 的 Jacobian 写成“Boris-like rotation 主干 $I - \Omega$ + 一个 rank-one correction”, 其中 $\Omega \cdot V = (\beta \times V) / \gamma_{new}$ 。这一步意味着作者不是直接对整个复杂映射硬算 determinant, 而是先把旋转骨架和 relativistic correction 拆开。随后利用 determinant lemma, 把后半步体积变化写成

$$J_{f,new} = \det(I - \Omega) \times (\text{scalar correction}),$$

再经过代数整理压成

$$J_{f,new} = 1 + \frac{\beta^2 + (\vec{\beta} \cdot \bar{u}_{new})^2}{\gamma_{new}^4}.$$

前半步可得同一形式的 determinant, 因此真正的 volume-preserving 不是“每个子步都单独等于 1”, 而是前后半步 Jacobian determinant 互为逆, 整步更新的 Jacobian 才严格等于 1。这一点很重要, 因为它把 UpdateMomentumHigueraCary.H 的稳定性来源从经验判断提升成了明确的 Jacobian 结构断言。

这里还要额外提醒一个记号陷阱: 论文里 $J_{\{f,new\}}$ 和 $J_{\{i,new\}}$ 最后都被压成相同的显式标量函数, 但这不代表它们是“同一个 Jacobian”。它们对应的是后半步和前半步那两条相反方向映射上的 determinant, 因此恰恰是因为它们处在 reciprocal 位置, 整步 Jacobian 才会严格回到 1。同样, 论文对 Vay 的结论也不是“任何情况下都会立刻出现 attractor/repeller”。作者保留了一个例外边界: 若磁场在时空上恒定, $J(x_i, u_i) / J(x_i, u_f)$ 这串比值会 telescoping, 再结合 $J(x, u)$ 在有界区域里的有界性, 不能直接推出灾难性体积失真。真正的问题是它缺少一般性的 volume-preservation, 因此在 practical timestep 和更复杂轨道拓扑下更容易暴露出 resonance-island 与 trajectory-crossing 这类非物理后果。

WarpX 参数文档把 algo.particle_pusher = vay 和 higuera 分别列为可选 pusher, 并引用 param-Vaypop2008 与 param-HigueraPOP2017。因此书中后续做 benchmark 时应至少比较 Boris、Vay、Higuera-Cary 在强磁场、相对论漂移和 boosted-frame 场景下的轨道误差。

当前本地 regression 和这篇文献的配对也要写得保守。最直接能接 Higuera-Cary 文献主线的是 Examples/Tests/particle_pusher: 它在 algo.particle_pusher = higuera、force-free 常量外部 E/B 构型下, 用 analysis.py 强检查长时间推进后 $x \approx 0$, 因此可以当作 relativistic force-free / drift-preservation 的本地强断言。相反, Examples/Tests/larmor 目前仍只有 checksum, 没有按 Poincare section 或 invariant drift 去复现论文第 VI 节 practical-timestep topology / resonance-island 的专门 analysis, 所以不能把这组本地 test 夸写成 Higuera-Cary 论文数值部分的完整本地再现。

4.6 从 MultiParticleContainer 到 PhysicalParticleContainer

主循环的入口是 ../warpx/Source/Evolve/WarpXEvolve.cpp:1324-1428 的 WarpX::PushParticlesandDeposit()。它选择 current 字段名后调用 mypc->Evolve(...)。

mypc 是 MultiParticleContainer。其 Evolve() 位于 ../warpx/Source/Particles/MultiParticleContainer.cpp:478-522:

行号	操作
:486-496	不跳过沉积时清零 current_fp/current_buf/rho_fp/rho_buf。
:497-518	隐式 solver 相关源项和 mass matrix 的额外清零逻辑。
:520-522	遍历所有 species, 调用每个 pc->Evolve(...)。

这层只负责多物种调度。真正的单 species 粒子推进在 PhysicalParticleContainer::Evolve()。

但在继续进入 PushPX() 之前, 还要先看清容器层次和属性系统, 否则后面很多变量名会被误读。

../warpx/Source/Particles/WarpXParticleContainer.H:55-88 先定义了编译期属性表 PIdx。它规定每个粒子天生就有:

- 位置分量 x/y/z 或非笛卡尔等价值;
- 权重 w;
- proper velocity ux/uy/uz;
- 在 RZ/球坐标几何下额外的 theta/phi。

而 `IntIdx::nattrs` 在 `../warpx/Source/Particles/WarpXParticleContainer.H:92-99` 里默认是 0, 这意味着 WarpX 的整数粒子属性默认都不是编译期内建, 而是后续按需动态添加。

顶层类层次则由 `../warpx/Source/Particles/MultiParticleContainer.cpp:96-125` 与各头文件共同决定:

```
MultiParticleContainer
-> WarpXParticleContainer
    -> PhysicalParticleContainer
        -> PhotonParticleContainer
        -> RigidInjectedParticleContainer
    -> LaserParticleContainer
```

这里几类容器的物理职责不同:

- `WarpXParticleContainer` 是统一基类, 提供粒子 SoA 骨架、gather/deposition/push 的共用接口;
- `PhysicalParticleContainer` 承担普通带质量 species 的主要物理路径;
- `PhotonParticleContainer` 继承 `PhysicalParticleContainer`, 但 `DepositCharge()` 和 `DepositCurrent()` 直接空实现, 因此它保留很多粒子基础设施, 却不承担带电沉积; 见 `../warpx/Source/Particles/PhotonParticleContainer.H:25-115`;
- `LaserParticleContainer` 则直接从 `WarpXParticleContainer` 继承, 因为激光天线粒子只需要 prescribed motion 和 current deposition, 不需要普通 `FieldGather`; 见 `../warpx/Source/Particles/LaserParticleContainer.H:30-61`。

这层分工直接决定了粒子属性的来源。`PhysicalParticleContainer` 构造函数在 `../warpx/Source/Particles/PhysicalParticleContainer.cpp` 会按模块开关注册第一批 runtime attributes:

- QED quantum synchrotron 时加 `opticalDepthQSR`;
- Breit-Wheeler 时加 `opticalDepthBW`;
- `addRealAttributes` / `addIntegerAttributes` 时加入用户 parser 驱动属性;
- `save_previous_position` 时加 `prev_x/prev_y/prev_z`。

电离模块稍后又会在 `../warpx/Source/Particles/PhysicalParticleContainer.cpp:1592-1595` 动态补上 `ionizationLevel`。因此 WarpX 的粒子属性系统不是一个静态结构体, 而是:

1. 编译期 builtin real: `x/y/z/w/ux/uy/uz/...`;
2. 构造期或模块初始化期动态加入的持久物理状态: 如 `opticalDepthQSR`、`opticalDepthBW`、`prev_*`、`ionizationLevel`;
3. 更晚才按算法路径加入的临时缓存属性。

最典型的临时属性来自 implicit solver。`../warpx/Source/FieldSolver/ImplicitSolvers/ImplicitSolver.cpp:10-34` 会统一给粒子容器补加:

- `x_n/y_n/z_n`
- `ux_n/uy_n/uz_n`
- 如果启用 particle suborbits, 再加 `nsuborbits`

而且都用 `comm = 0` 注册, 所以这些量既不参与通信, 也不写入 checkpoint。随后 `../warpx/Source/FieldSolver/ImplicitSolvers/WarpXImplicitSolver.cpp` 会在每个 implicit step 开始时, 把当前位置和动量快照写入 `x_n` 和 `ux_n` 这组缓存, 并把 `nsuborbits` 先置成 1。它们的角色不是长期物理属性, 而是 implicit 时间推进器本步用的局部状态。

`WarpXParticleContainer::AddNParticles()` 进一步说明了这套系统怎样落地。`../warpx/Source/Particles/WarpXParticleContainer.cpp` 先写 builtin `x/y/z/w/ux/uy/uz`, 再写调用者显式提供的 runtime real/int, 最后调用 `DefaultInitializeRuntimeAttributes()` 给剩余 runtime attrs 自动补默认值。于是:

- `opticalDepthQSR/BW` 可以由 QED engine 随机初始化;
- `ionizationLevel` 可以统一设成 `ionization_initial_level`;
- 用户 parser 属性可以按 `attribute.<name>(x,y,z,ux,uy,uz,t)` 自动求值。

也就是说, 进入 `PushPX()` 之前, WarpX 已经把“粒子是什么物种、当前带哪些附加物理状态、哪些只是临时算法缓存”这三件事分清了。后面看到 `ux`、`ux_n`、`ionizationLevel`、`opticalDepthQSR` 这些名字时, 必须先回到这里判断它们属于哪一层状态。

4.7 PhysicalParticleContainer::Evolve() 的 tile loop

`../warpx/Source/Particles/PhysicalParticleContainer.cpp:457-831` 是本章最重要的函数。它把一个 species 的粒子按 tile 遍历, 并把沉积、gather、push、buffer、隐式路径和 load-balance cost 放在一个局部循环里。

核心顺序是:

行号	操作	含义
:486-491	取得 Efield_aux 和 Bfield_aux	粒子 gather 使用 auxiliary fields。
:493-508	判断是否沉积 charge/current、是否 split particles	skip_deposition 和 do_not_deposit 会关掉沉积。
:523-580	遍历 tile, 必要时按 AMR buffer 分区粒子	fine/coarse gather 和 deposit 的粒子集合可能不同。
:585-598	push 前沉积 rho component 0	旧时间层电荷, 通常对应 ρ^n 。
:619-623	fine patch 粒子调用 PushPX()	gather fine fields 并推进粒子。
:675-682	buffer/coarse 粒子调用 PushPX()	AMR 边界附近可从 coarse auxiliary fields gather。
:703-738	沉积 current	显式路径 relative_time=-0.5*dt, 对应 $\mathbf{J}^{n+1/2}$ 。
:791-808	push 后沉积 rho component 1	新时间层电荷, 通常对应 ρ^{n+1} 。
:822-830	可选 particle splitting	subcycling 时避免 coarse level 重复沉积。

这段源码说明, 真实粒子推进不是“先推所有粒子, 再单独沉积”。WarpX 为了 AMR、缓存局部性、GPU/CPU 并行和时间层一致性, 在 tile 内完成 gather/push/deposit 的组合。

4.8 PushPX(): gather 和 push 的融合 kernel

PhysicalParticleContainer::PushPX() 位于 ../warpX/Source/Particles/PhysicalParticleContainer.cpp:1330-1575。它是真正进入单粒子并行循环的地方。

核心源码原文如下, 省略了 QED 宏分支和部分属性准备:

```
amrex::ParallelFor(
    TypeList<CompileTimeOptions<no_exteb,has_exteb>, CompileTimeOptions<no_qed ,has_qed>>{},
    {exteb_runtime_flag, qed_runtime_flag},
    np_to_push,
    [=] AMREX_GPU_DEVICE (long ip, auto exteb_control, auto qed_control)
{
    amrex::ParticleReal xp, yp, zp;
    getPosition(ip, xp, yp, zp);

    amrex::ParticleReal Exp = Ex_external_particle;
    amrex::ParticleReal Eyp = Ey_external_particle;
    amrex::ParticleReal Ezp = Ez_external_particle;
    amrex::ParticleReal Bxp = Bx_external_particle;
    amrex::ParticleReal Byp = By_external_particle;
    amrex::ParticleReal Bzp = Bz_external_particle;

    if (gather_fields) {
        // first gather E and B to the particle positions
        doGatherShapeN(xp, yp, zp, Exp, Eyp, Ezp, Bxp, Byp, Bzp,
            ex_arr, ey_arr, ez_arr, bx_arr, by_arr, bz_arr,
            ex_type, ey_type, ez_type, bx_type, by_type, bz_type,
            dinv, xyzmin, lo, n_rz_azimuthal_modes,
            nox, galerkin_interpolation);
    }

    scaleFields(xp, yp, zp, Exp, Eyp, Ezp, Bxp, Byp, Bzp);

    doParticleMomentumPush<0>(ux[ip], uy[ip], uz[ip],
        Exp, Eyp, Ezp, Bxp, Byp, Bzp,
        ion_lev ? ion_lev[ip] : 1,
        mass, q, pusher_algo, do_crr,
```

```

        dt, momentum_push_type);

    if (position_push_type == PositionPushType::Full) {
        UpdatePosition(xp, yp, zp, ux[ip], uy[ip], uz[ip], dt, mass);
        setPosition(ip, xp, yp, zp);
    }
});

```

关键步骤:

行号	操作
:1346-1350	检查 gather level, 并对空粒子直接返回。
:1352-1367	构造 gather box, 并按 ngEB 扩展 guard cells。
:1379-1444	准备粒子位置访问器、外场、动量数组、ionization level、旧位置缓存等。
:1446-1477	取 species 电荷/质量、pusher 算法、radiation/QED flags。
:1478-1486	启动带 compile-time option 的 amrex::ParallelFor。
:1488-1517	读取粒子位置, 调用 doGatherShapeN() 把网格场 gather 到粒子。
:1519-1524	叠加外部粒子场和缩放。
:1531-1555	调用 doParticleMomentumPush() 更新动量。
:1557-1560	若 PositionPushType::Full, 调用 UpdatePosition() 更新位置。

这给出了 WarpX 显式粒子推进的真实顺序:

```

for particle in tile:
    read x^n and u^(n-1/2)
    gather E/B at x^n
    add external particle fields
    push momentum to u^(n+1/2)
    push position to x^(n+1)

```

随后 PhysicalParticleContainer::Evolve() 在 :697-733 沉积半步电流, 在 :785-803 沉积新电荷。

4.9 doGatherShapeN(): 从网格场到粒子场

PushPX() 在调用 pusher 前先调用 doGatherShapeN()。这一步的物理含义是把网格上的 Yee/PSATD 场插值到粒子位置:

$$\mathbf{E}_p = \sum_i \mathbf{E}_i S_i(\mathbf{x}_p), \quad \mathbf{B}_p = \sum_i \mathbf{B}_i S_i(\mathbf{x}_p).$$

但 WarpX 不能只用一个标量形函数, 因为 $E_x, E_y, E_z, B_x, B_y, B_z$ 在交错网格上的中心位置不同; 同时 Galerkin 插值会让某些分量使用低一阶形函数。运行时入口在 ../warpx/Source/Particles/Gather/FieldGather.H:2119-2192:

```

void doGatherShapeN (const amrex::ParticleReal xp,
                    const amrex::ParticleReal yp,
                    const amrex::ParticleReal zp,
                    amrex::ParticleReal& Exp,
                    amrex::ParticleReal& Eyp,
                    amrex::ParticleReal& Ezp,
                    amrex::ParticleReal& Bxp,
                    amrex::ParticleReal& Byp,
                    amrex::ParticleReal& Bzp,
                    amrex::Array4<amrex::Real const> const& ex_arr,
                    amrex::Array4<amrex::Real const> const& ey_arr,
                    amrex::Array4<amrex::Real const> const& ez_arr,
                    amrex::Array4<amrex::Real const> const& bx_arr,
                    amrex::Array4<amrex::Real const> const& by_arr,

```

```

        amrex::Array4<amrex::Real const> const& bz_arr,
        const amrex::IndexType ex_type,
        const amrex::IndexType ey_type,
        const amrex::IndexType ez_type,
        const amrex::IndexType bx_type,
        const amrex::IndexType by_type,
        const amrex::IndexType bz_type,
        const amrex::XDim3 & dinv,
        const amrex::XDim3 & xyzmin,
        const amrex::Dim3& lo,
        const int n_rz_azimuthal_modes,
        const int nox,
        const bool galerkin_interpolation)
{
    if (galerkin_interpolation) {
        if (nox == 1) {
            doGatherShapeN<1,1>(xp, yp, zp, Exp, Eyp, Ezp, Bxp, Byp, Bzp,
                                ex_arr, ey_arr, ez_arr, bx_arr, by_arr, bz_arr,
                                ex_type, ey_type, ez_type, bx_type, by_type, bz_type,
                                dinv, xyzmin, lo, n_rz_azimuthal_modes);
        } else if (nox == 2) {
            doGatherShapeN<2,1>(xp, yp, zp, Exp, Eyp, Ezp, Bxp, Byp, Bzp,
                                ex_arr, ey_arr, ez_arr, bx_arr, by_arr, bz_arr,
                                ex_type, ey_type, ez_type, bx_type, by_type, bz_type,
                                dinv, xyzmin, lo, n_rz_azimuthal_modes);
        } else if (nox == 3) {
            doGatherShapeN<3,1>(xp, yp, zp, Exp, Eyp, Ezp, Bxp, Byp, Bzp,
                                ex_arr, ey_arr, ez_arr, bx_arr, by_arr, bz_arr,
                                ex_type, ey_type, ez_type, bx_type, by_type, bz_type,
                                dinv, xyzmin, lo, n_rz_azimuthal_modes);
        } else if (nox == 4) {
            doGatherShapeN<4,1>(xp, yp, zp, Exp, Eyp, Ezp, Bxp, Byp, Bzp,
                                ex_arr, ey_arr, ez_arr, bx_arr, by_arr, bz_arr,
                                ex_type, ey_type, ez_type, bx_type, by_type, bz_type,
                                dinv, xyzmin, lo, n_rz_azimuthal_modes);
        }
    } else {
        if (nox == 1) {
            doGatherShapeN<1,0>(xp, yp, zp, Exp, Eyp, Ezp, Bxp, Byp, Bzp,
                                ex_arr, ey_arr, ez_arr, bx_arr, by_arr, bz_arr,
                                ex_type, ey_type, ez_type, bx_type, by_type, bz_type,
                                dinv, xyzmin, lo, n_rz_azimuthal_modes);
        } else if (nox == 2) {
            doGatherShapeN<2,0>(xp, yp, zp, Exp, Eyp, Ezp, Bxp, Byp, Bzp,
                                ex_arr, ey_arr, ez_arr, bx_arr, by_arr, bz_arr,
                                ex_type, ey_type, ez_type, bx_type, by_type, bz_type,
                                dinv, xyzmin, lo, n_rz_azimuthal_modes);
        } else if (nox == 3) {
            doGatherShapeN<3,0>(xp, yp, zp, Exp, Eyp, Ezp, Bxp, Byp, Bzp,
                                ex_arr, ey_arr, ez_arr, bx_arr, by_arr, bz_arr,
                                ex_type, ey_type, ez_type, bx_type, by_type, bz_type,
                                dinv, xyzmin, lo, n_rz_azimuthal_modes);
        } else if (nox == 4) {
            doGatherShapeN<4,0>(xp, yp, zp, Exp, Eyp, Ezp, Bxp, Byp, Bzp,
                                ex_arr, ey_arr, ez_arr, bx_arr, by_arr, bz_arr,
                                ex_type, ey_type, ez_type, bx_type, by_type, bz_type,

```

```

        dinv, xyzmin, lo, n_rz_azimuthal_modes);
    }
}

```

这里的 `nox` 不是运行时循环里的 `shape` 阶数变量，而是被转成模板参数 `depos_order`。这样 GPU kernel 内部可以用 `if constexpr` 展开阶数，避免每个粒子再做阶数分支。`galerkin_interpolation` 同理变成第二个模板参数，后面直接影响数组长度 `depos_order + 1 - galerkin_interpolation`。

模板主体开头在 `../warpX/Source/Particles/Gather/FieldGather.H:348-439`。下面只列 `x` 方向；`y/z` 方向同构，但按各自场分量的 `staggering` 选择 `node` 或 `cell`：

```

template <int depos_order, int galerkin_interpolation>
AMREX_GPU_HOST_DEVICE AMREX_FORCE_INLINE
void doGatherShapeN ([[maybe_unused]] const amrex::ParticleReal xp,
                    [[maybe_unused]] const amrex::ParticleReal yp,
                    [[maybe_unused]] const amrex::ParticleReal zp,
                    amrex::ParticleReal& Exp,
                    amrex::ParticleReal& Eyp,
                    amrex::ParticleReal& Ezp,
                    amrex::ParticleReal& Bxp,
                    amrex::ParticleReal& Byp,
                    amrex::ParticleReal& Bzp,
                    amrex::Array4<amrex::Real const> const& ex_arr,
                    amrex::Array4<amrex::Real const> const& ey_arr,
                    amrex::Array4<amrex::Real const> const& ez_arr,
                    amrex::Array4<amrex::Real const> const& bx_arr,
                    amrex::Array4<amrex::Real const> const& by_arr,
                    amrex::Array4<amrex::Real const> const& bz_arr,
                    const amrex::IndexType ex_type,
                    const amrex::IndexType ey_type,
                    const amrex::IndexType ez_type,
                    const amrex::IndexType bx_type,
                    const amrex::IndexType by_type,
                    const amrex::IndexType bz_type,
                    const amrex::XDim3 & dinv,
                    const amrex::XDim3 & xyzmin,
                    const amrex::Dim3& lo,
                    [[maybe_unused]] const int n_rz_azimuthal_modes)
{
    using namespace amrex;

    constexpr int NODE = amrex::IndexType::NODE;
    constexpr int CELL = amrex::IndexType::CELL;

    Compute_shape_factor< depos_order > const compute_shape_factor;
    Compute_shape_factor<depos_order - galerkin_interpolation > const compute_shape_factor_galerkin;

    #if !defined(WARPX_DIM_1D_Z)
        const amrex::Real x = (xp - xyzmin.x)*dinv.x;

        amrex::Real sx_node[depos_order + 1] = {0._rt};
        amrex::Real sx_cell[depos_order + 1] = {0._rt};
        amrex::Real sx_node_galerkin[depos_order + 1 - galerkin_interpolation] = {0._rt};
        amrex::Real sx_cell_galerkin[depos_order + 1 - galerkin_interpolation] = {0._rt};

        int j_node = 0;
        int j_cell = 0;
    #endif
}

```

```

int j_node_v = 0;
int j_cell_v = 0;
if ((ey_type[0] == NODE) || (ez_type[0] == NODE) || (bx_type[0] == NODE)) {
    j_node = compute_shape_factor(sx_node, x);
}
if ((ey_type[0] == CELL) || (ez_type[0] == CELL) || (bx_type[0] == CELL)) {
    j_cell = compute_shape_factor(sx_cell, x - 0.5_rt);
}
if ((ex_type[0] == NODE) || (by_type[0] == NODE) || (bz_type[0] == NODE)) {
    j_node_v = compute_shape_factor_galerkin(sx_node_galerkin, x);
}
if ((ex_type[0] == CELL) || (by_type[0] == CELL) || (bz_type[0] == CELL)) {
    j_cell_v = compute_shape_factor_galerkin(sx_cell_galerkin, x - 0.5_rt);
}
const amrex::Real (&sx_ex)[depos_order + 1 - galerkin_interpolation] = ((ex_type[0] == NODE) ? sx_node : sx_cell);
const amrex::Real (&sx_ey)[depos_order + 1] = ((ey_type[0] == NODE) ? sx_node : sx_cell);
const amrex::Real (&sx_ez)[depos_order + 1] = ((ez_type[0] == NODE) ? sx_node : sx_cell);
const amrex::Real (&sx_bx)[depos_order + 1] = ((bx_type[0] == NODE) ? sx_node : sx_cell);
const amrex::Real (&sx_by)[depos_order + 1 - galerkin_interpolation] = ((by_type[0] == NODE) ? sx_node : sx_cell);
const amrex::Real (&sx_bz)[depos_order + 1 - galerkin_interpolation] = ((bz_type[0] == NODE) ? sx_node : sx_cell);
int const j_ex = ((ex_type[0] == NODE) ? j_node_v : j_cell_v);
int const j_ey = ((ey_type[0] == NODE) ? j_node : j_cell);
int const j_ez = ((ez_type[0] == NODE) ? j_node : j_cell);
int const j_bx = ((bx_type[0] == NODE) ? j_node : j_cell);
int const j_by = ((by_type[0] == NODE) ? j_node_v : j_cell_v);
int const j_bz = ((bz_type[0] == NODE) ? j_node_v : j_cell_v);
#endif

```

这段源码有三个关键点。

1. $x = (x_p - xyzmin.x) * d_{inv}.x$ 把物理坐标变成网格坐标。后面形函数都在无量纲网格坐标上计算。
2. x 和 $x - 0.5_{rt}$ 分别对应 node-centered 和 cell-centered 自由度。也就是说，场分量的 staggered center 不是后处理标签，而是直接改变粒子看到的插值权重。
3. Galerkin 路径给 ex/by/bz 使用 compute_shape_factor_galerkin，阶数是 depos_order - 1；非 Galerkin 时第二个模板参数为 0，因此阶数不变。

以 2D XZ 编译为例，真正累加网格场的源码在 ../warpx/Source/Particles/Gather/FieldGather.H:547-581:

```

#elif defined(WARPX_DIM_XZ)
    // Gather field on particle Eyp from field on grid ey_arr
    for (int iz=0; iz<=depos_order; iz++){
        for (int ix=0; ix<=depos_order; ix++){
            Eyp += sx_ey[ix]*sz_ey[iz]*
                ey_arr(lo.x+j_ey+ix, lo.y+l_ey+iz, 0, 0);
        }
    }
    // Gather field on particle Exp from field on grid ex_arr
    // Gather field on particle Bzp from field on grid bz_arr
    for (int iz=0; iz<=depos_order; iz++){
        for (int ix=0; ix<=depos_order-galerkin_interpolation; ix++){
            Exp += sx_ex[ix]*sz_ex[iz]*
                ex_arr(lo.x+j_ex+ix, lo.y+l_ex+iz, 0, 0);
            Bzp += sx_bz[ix]*sz_bz[iz]*
                bz_arr(lo.x+j_bz+ix, lo.y+l_bz+iz, 0, 0);
        }
    }
    // Gather field on particle Ezp from field on grid ez_arr
    // Gather field on particle Bxp from field on grid bx_arr

```

```

for (int iz=0; iz<=depos_order-galerkin_interpolation; iz++){
  for (int ix=0; ix<=depos_order; ix++){
    Ezp += sx_ez[ix]*sz_ez[iz]*
    ez_arr(lo.x+j_ez+ix, lo.y+l_ez+iz, 0, 0);
    Bxp += sx_bx[ix]*sz_bx[iz]*
    bx_arr(lo.x+j_bx+ix, lo.y+l_bx+iz, 0, 0);
  }
}
// Gather field on particle Byp from field on grid by_arr
for (int iz=0; iz<=depos_order-galerkin_interpolation; iz++){
  for (int ix=0; ix<=depos_order-galerkin_interpolation; ix++){
    Byp += sx_by[ix]*sz_by[iz]*
    by_arr(lo.x+j_by+ix, lo.y+l_by+iz, 0, 0);
  }
}

```

公式上, 第一段就是

$$E_{y,p} = \sum_{i=0}^p \sum_{k=0}^p S_{E_y,i}^{(x)} S_{E_y,k}^{(z)} E_y(j_{E_y} + i, l_{E_y} + k).$$

Exp/Bzp、Ezp/Bxp、Byp 的循环上限不同, 是因为 Galerkin 插值会沿某些方向把 shape order 从 p 降到 $p-1$ 。这不是任意优化, 而是和离散 Maxwell operator、field staggering 与能量/电荷性质匹配的插值选择。

RZ 编译下, gather 先得到柱坐标分量, 再转回笛卡尔粒子 pusher 需要的 E_x, E_y, B_x, B_y 。关键转换在 `../warpx/Source/Particles/Gather/FieldGa`

```

amrex::Real costheta;
amrex::Real sintheta;
if (rp > 0.) {
  costheta = xp/rp;
  sintheta = yp/rp;
} else {
  costheta = 1.;
  sintheta = 0.;
}
const Complex xy0 = Complex{costheta, -sintheta};
Complex xy = xy0;

for (int imode=1 ; imode < n_rz_azimuthal_modes ; imode++) {

  // Gather field on particle Ethetap from field on grid ey_arr
  for (int iz=0; iz<=depos_order; iz++){
    for (int ix=0; ix<=depos_order; ix++){
      const amrex::Real dEy = (+ ey_arr(lo.x+j_ey+ix, lo.y+l_ey+iz, 0, 2*imode-1)*xy.real()
        - ey_arr(lo.x+j_ey+ix, lo.y+l_ey+iz, 0, 2*imode)*xy.imag());
      Ethetap += sx_ey[ix]*sz_ey[iz]*dEy;
    }
  }

  // Gather field on particle Erp from field on grid ex_arr
  // Gather field on particle Bzp from field on grid bz_arr
  for (int iz=0; iz<=depos_order; iz++){
    for (int ix=0; ix<=depos_order-galerkin_interpolation; ix++){
      const amrex::Real dEx = (+ ex_arr(lo.x+j_ex+ix, lo.y+l_ex+iz, 0, 2*imode-1)*xy.real()
        - ex_arr(lo.x+j_ex+ix, lo.y+l_ex+iz, 0, 2*imode)*xy.imag());
      Erp += sx_ex[ix]*sz_ex[iz]*dEx;
      const amrex::Real dBz = (+ bz_arr(lo.x+j_bz+ix, lo.y+l_bz+iz, 0, 2*imode-1)*xy.real()
        - bz_arr(lo.x+j_bz+ix, lo.y+l_bz+iz, 0, 2*imode)*xy.imag());
    }
  }
}

```

```

        Bzp += sx_bz[ix]*sz_bz[iz]*dBz;
    }
}
// Gather field on particle Ezp from field on grid ez_arr
// Gather field on particle Brp from field on grid bx_arr
for (int iz=0; iz<=depos_order-galerkin_interpolation; iz++){
    for (int ix=0; ix<=depos_order; ix++){
        const amrex::Real dEz = (+ ez_arr(lo.x+j_ez+ix, lo.y+l_ez+iz, 0, 2*imode-1)*xy.real()
                                - ez_arr(lo.x+j_ez+ix, lo.y+l_ez+iz, 0, 2*imode)*xy.imag());
        Ezp += sx_ez[ix]*sz_ez[iz]*dEz;
        const amrex::Real dBx = (+ bx_arr(lo.x+j_bx+ix, lo.y+l_bx+iz, 0, 2*imode-1)*xy.real()
                                - bx_arr(lo.x+j_bx+ix, lo.y+l_bx+iz, 0, 2*imode)*xy.imag());
        Brp += sx_bx[ix]*sz_bx[iz]*dBx;
    }
}
// Gather field on particle Bthetap from field on grid by_arr
for (int iz=0; iz<=depos_order-galerkin_interpolation; iz++){
    for (int ix=0; ix<=depos_order-galerkin_interpolation; ix++){
        const amrex::Real dBy = (+ by_arr(lo.x+j_by+ix, lo.y+l_by+iz, 0, 2*imode-1)*xy.real()
                                - by_arr(lo.x+j_by+ix, lo.y+l_by+iz, 0, 2*imode)*xy.imag());
        Bthetap += sx_by[ix]*sz_by[iz]*dBy;
    }
}
xy = xy*xy0;
}

// Convert Erp and Ethetap to Ex and Ey
Exp += costheta*Erp - sintheta*Ethetap;
Eyp += costheta*Ethetap + sintheta*Erp;
Bxp += costheta*Brp - sintheta*Bthetap;
Byp += costheta*Bthetap + sintheta*Brp;

```

所以 PushPX() 里的 pusher 始终看见 Cartesian-like 的 Exp,Eyp,Ezp,Bxp,Byp,Bzp. RZ 的 Fourier mode 和柱坐标细节被 gather 层封装, 但不是消失: 它们决定粒子场的实际插值。

把这层再向下看, WarpX 的 gather 还不是“只剩 3D/XZ/RZ 三个分支”。FieldGather.H 后面还继续给出了:

- 1D_Z: 只沿 z 一个方向做 shape 累加, 但仍保留 Ez/Bx/By 这组可能走 p-1 的 Galerkin 降阶路径;
- RCYLINDER: 先 gather Fr/Ftheta/Fz, 再按粒子极角转回 Fx/Fy/Fz;
- RSPHERE: 先 gather Er/Etheta/Ephi, 再按球坐标角转回 Ex/Ey/Ez;
- 3D: 完整保留 Ex/Ey/Ez/Bx/By/Bz 六个分量各自不同的循环上限, 因此 Galerkin 不是一个全局“降一阶开关”, 而是对特定分量沿特定方向降阶。

这也能更准确地解释官方文档里 energy-conserving 与 momentum-conserving gather 的差别。doGatherShapeN() 本身并不读一个“gather family”枚举, 它真正消费的是传进来的 IndexType。因此两族 gather 的代码差异不在这个 wrapper 里, 而在更早的 field centering 上:

- energy-conserving: 直接从原来的 staggered 或 nodal 网格 gather;
- momentum-conserving: 先把场中心化到 node, 再把 nodal IndexType 送进 doGatherShapeN()。

这也解释了为什么 collocated grid 下两者等价, 而 hybrid grid 下必须强制 momentum-conserving。一旦所有分量都已经是 nodal/collocated, doGatherShapeN() 看到的就只是同一组 NODE/CELL 组合。

如果只停在这层实现差异, 这个问题还是讲浅了。Birdsall-Langdon 第一分卷 Chapter 10 给出的更硬边界是: energy-conserving 和 momentum-conserving 从来都不是同一套离散系统上“换一种 gather 插值”这么简单, 而是两套不同的守恒合同。momentum-conserving 路线保留的是更自然的零总力/粒子-粒子相互作用对称结构, 但它一般并不存在严格守恒的总能量; energy-conserving 路线则把

$$W_E = \frac{V_c}{2} \sum_j \rho_j \phi_j$$

当成第一性对象，再把粒子受理解成

$$\mathbf{F}_i = -\frac{\partial W_E}{\partial \mathbf{x}_i},$$

也就是不再先“在网格上差分 ϕ 得 E 、再把 E gather 给粒子”，而是要求粒子受力、离散 Poisson 解和场能量账本共享同一套 reciprocity 合同。对 WarpX 来说，这当然不意味着当前 `FieldGather.H` 里就直接复现了 Birdsall 的整个 energy-conserving electrostatic 变分算法；更准确的说法是，官方文档和 regression 里保留下来的 `energy-conserving gather / momentum-conserving gather` 命名，只有放回这条更老的理论分叉里才不会被误解成“两个 wrapper 里哪一个 stencil 更光滑”。

因此，本章后面凡是提到 gather family、field centering、collocated grid、Langmuir 守恒基线时，都应该记住自己实际上在比较三层东西：

1. `IndexType` 与 stagger/nodal centering 的实现差别；
2. sampled field 怎样被回插到粒子；
3. 这套回插究竟服务于哪一种离散守恒合同。

还有一层容易漏掉：implicit path 并不复用显式 gather。`FieldGather.H:2195-2328` 的 `doGatherShapeNImplicit(...)` 会先按沉积算法分派：

- `Esirkepov`: `doGatherShapeNEsirkepovStencilImplicit`
- `Villasenor`: `doGatherPicnicShapeN`
- `Direct`: 才退回普通半步位置 gather

所以 implicit 里不是“先统一 gather，再换 deposition”，而是 gather stencil 本身就要和后面的 charge-conserving 轨道几何解释匹配。

最后，external particle fields 也不止一种接入方式。`PushPX()` 里真实顺序是：

1. 先把常量 `m_E_external_particle/m_B_external_particle` 加到粒子局部 `Exp...Bzp`
2. 再对主网格场调用 `doGatherShapeN()`
3. 最后再执行 `GetExternalEBField` functor

其中：

- `parse*_ext_particle_function` 和 `repeated_plasma_lens` 是逐粒子直接相加；
- `read_from_file` 则不会在 gather kernel 内逐粒子加场，而是先把外场加到 `Efield_aux/Bfield_aux` 这类 `MultiFab`，再让粒子像 gather 主场一样去 gather 它们。

对应的 regression 也已经有了比较清楚的分层：

- `energy_conserving_thermal_plasma`: 在 electrostatic 周期热等离子体里检查总能量增长不超过 0.3%，这是当前对 energy-conserving gather 最直接的物理断言；
- `langmuir_multi_psatd_momentum_conserving`: 在 PSATD + momentum-conserving gather 组合下继续用 Langmuir 问题做守恒/稳定性基线；
- `load_external_field`: 分别用 3D/RZ 磁镜单粒子轨道和时间缩放脚本验证 external particle fields 的 read-from-file、time dependency 和 multi-field superposition。

4.10 位置推进与无质量粒子

显式位置推进在 `../warpX/Source/Particles/Pusher/UpdatePosition.H:19-70`。对有质量粒子，源码先计算

$$\gamma^{-1} = \frac{1}{\sqrt{1 + |\mathbf{u}|^2/c^2}},$$

再按编译维度更新坐标：

$$x \leftarrow x + u_x \gamma^{-1} \Delta t, \quad y \leftarrow y + u_y \gamma^{-1} \Delta t, \quad z \leftarrow z + u_z \gamma^{-1} \Delta t.$$

对无质量粒子，源码使用

$$\mathbf{v} = c \frac{\mathbf{u}}{|\mathbf{u}|}$$

更新位置, 见 `UpdatePosition.H:52-69`。因此 `photon container` 可以复用位置推进形式, 但动量和沉积行为不同; 光子容器的专门逻辑后续多物理章节再展开。

4.11 RR、implicit 与 photon path

前面 4.1 到 4.10 主要还是显式带电粒子的主线, 但 WarpX 的粒子推进并不只有这一条路径。至少还有三条不能忽略的分支:

1. classical radiation reaction;
2. implicit particle push;
3. photon container 的无质量推进。

先看 RR。../warpX/Source/Particles/Pusher/PushSelector.H:61-104 说明, RR 不是第四种独立 pusher, 而是优先级高于 ParticlePusherAlgo 的一个分支:

```
if (do_crr) {
    ...
    UpdateMomentumBorisWithRadiationReaction(...);
} else if (pusher_algo == ParticlePusherAlgo::Boris) {
    UpdateMomentumBoris(...);
} else if (pusher_algo == ParticlePusherAlgo::Vay) {
    UpdateMomentumVay(...);
} else if (pusher_algo == ParticlePusherAlgo::HigueraCary) {
    UpdateMomentumHigueraCary(...);
}
```

因此, 一旦打开 `do_crr`, 当前粒子不会再走 Vay 或 Higuera-Cary, 而是强制退回“Boris 加修正力”结构。`UpdateMomentumBorisWithRadiationReaction` 在 ../warpX/Source/Particles/Pusher/UpdateMomentumBorisWithRadiationReaction.H:21-90 的实现也证实了这一点: 它先调用普通 `UpdateMomentumBoris()`, 再用新旧动量平均构造中间时刻的 γ_n 、 $\mathbf{v}_n$ 和 Lorentz force, 最后再把辐射反作用力乘 `dt` 加回动量。代码结构上, 这是一种 Boris 后附加阻尼项, 而不是完全重写一套 relativistic mover。

再看 implicit path。它和显式 `PushPX()` 的根本区别, 不在于换了另一个 `UpdateMomentum*`, 而在于时间层和收敛逻辑都改了。../warpX/Source/Particles/Pusher/ImplicitPushPX.cpp:369-378 的注释直接说明了顺序:

1. 先 position push 半步;
2. 再 gather 场;
3. 再做 velocity push;
4. 再把 old/new velocity 平均成 time-centered 值;
5. 位置和速度彼此依赖, 因此做 Picard 固定点迭代, 直到 step norm 收敛。

而这里真正把上一篇属性图接进来的, 是 `x_n/y_n/z_n/ux_n/uy_n/uz_n` 和 `nsuborbits`。在 ../warpX/Source/Particles/Pusher/ImplicitPushPX.cpp:380-390 这些量被明确当成“the positions and velocities saved at the start of the step”取出; 随后粒子初值直接从 `x_n` 和 `ux_n` 开始, 而不是从当前位置盲目继续推进。也就是说, `x_n/ux_n` 在 implicit 路径里不是诊断缓存, 而是 nonlinear solve 的参考态。

`nsuborbits` 则是 implicit 不收敛时的 fallback 状态。`ImplicitPushXP()` 在 ../warpX/Source/Particles/Pusher/ImplicitPushPX.cpp:400-410 中, 如果粒子没收敛, 就把 `nsuborbits[ip] = 2`, 再通过 `SetupSuborbitParticles()` 把这些粒子的权重临时置零、单独收集索引。后续 `ImplicitPushXPSubOrbits()` 又会强制把沉积算法切到 Villaseñor, 见 `ImplicitPushPX.cpp:734-738`。所以 suborbit 不只是“多分几步时间步”, 还会连带改变当前粒子的沉积路径。

最后看 photon container。../warpX/Source/Particles/PhotonParticleContainer.cpp:242-255 的 `PhotonParticleContainer::Evolve()` 并没有重写 `species` 外层循环, 而是继续调用 `PhysicalParticleContainer::Evolve(...)`。也就是说, tile loop、AMR buffer 分区、gather 外壳这些基础设施仍然复用。但 photon 通过两层专门改写改变了物理语义:

- `PhotonParticleContainer.H:86-115` 中 `DepositCharge()` 和 `DepositCurrent()` 都是空实现;
- `PhotonParticleContainer::PushPX()` 在 ../warpX/Source/Particles/PhotonParticleContainer.cpp:83-239 里只做 gather、可选 Breit-Wheeler optical depth 演化、以及无质量 `UpdatePosition(...)`, 并不调用 Boris/Vay/Higuera-Cary 的带电动量更新。

因此 photon path 和普通 charged species 的差异不只是“不沉积电流”, 而是连 momentum update 的物理模型都变了。它消耗的 runtime attributes 主要是:

- builtin `ux/uy/uz`;
- `opticalDepthBW`;
- 可选 `*_btd`;

而不会消费普通 charged implicit 路径里的 `ionizationLevel`、`opticalDepthQSR`、`x_n/ux_n/nsuborbits`。

从这一层往后看，WarpX 粒子推进可以总结成三种结构：

- 显式主线：Boris / Vay / Higuera-Cary；
- 显式修正：Boris + classical RR；
- 非标准推进：implicit fixed-point / suborbit fallback / photon push。

它们共享的是 `PhysicalParticleContainer::Evolve()`、gather 外壳和粒子属性系统；真正分叉的是时间层、收敛控制、沉积算法约束和具体消费的 runtime attributes。

如果再往 implicit solver 深处走一步，还要继续把“suborbit 轨道本身”和“JFNK 线性化源项拼装”分开看。../warpX/Source/FieldSolver/ImplicitS 明确把 linear stage 的电流写成

$$J(E) = J_{\text{suborbit}} + J_0 + \text{MM}(E - E_0).$$

这里：

- `J_0` 对应 `current_fp_non_suborbit`；
- `MM` 对应 `MassMatrices_X/Y/Z`；
- `J_suborbit` 才是那些真正需要 suborbit fallback 的粒子继续显式推进后沉到 `current_fp` 的部分。

这正好解释了为什么 `MultiParticleContainer::Evolve()` 在 implicit 模式下还要额外处理 `current_fp_non_suborbit` 和 `MassMatrices_PC` 的清零时机，见 ../warpX/Source/Particles/MultiParticleContainer.cpp:491-505。它不是普通 bookkeeping，而是在维护 JFNK 的三项分拆。

这一节在本地 checkout 里也有一条非常直接的 regression 入口：`Examples/Tests/radiation_reaction/`。它不是应用级 checksum，而是强 analysis：

- 平行动量 case 要求 `gamma` 保持不变；
- 垂直动量 case 要求 `gamma(t)` 满足解析 Landau-Lifshitz 衰减公式；
- 容差统一为 5%

因此它正好锚定了上面这条“先 Boris，再加 RR 修正”的源码路径，而不是泛化地证明“高能粒子大概会辐射”。

`ImplicitPushXSubOrbits()` 里还有两条实现约束非常重要。第一，../warpX/Source/Particles/Pusher/ImplicitPushPX.cpp:734-738 强制把 suborbit 路径的 current deposition 切到 Villasenor：

```
const auto depos_type = CurrentDepositionAlgo::Villasenor;
```

所以一旦粒子进入 suborbit fallback，用户原来选择的沉积算法并不会继续沿用。第二，../warpX/Source/Particles/Pusher/ImplicitPushPX.cpp: 把 `deposit_mass_matrices` 限定成

```
use_mass_matrices_pc && !linear_stage_of_jfnk
```

这说明 suborbit 粒子的 mass-matrix deposit 当前只服务 preconditioner，不直接服务 Jacobian 的 linear stage。

于是 implicit 粒子推进实际上还要再分成四层：

1. `x_n/ux_n/nsuborbits` 这组 step-start 属性；
2. `PushXPSingleStep()` 的 fixed-point 单轨道求解；
3. 不收敛粒子的 suborbit fallback 与 Villasenor-only 重沉积；
4. `current_fp_non_suborbit + MM*(E-E_0) + J_suborbit` 这套线性化源项拼装。

这也是为什么 implicit 路径不能被简单概括成“把显式 pusher 改成 Picard 迭代”。它已经把粒子推进、沉积算法、质量矩阵和 Newton/JFNK 线性化绑成了一套更大的数值系统。

4.12 Field Ionization: ADK 不是附加后处理，而是粒子属性和沉积权重一起改写

前面已经出现过 `ionizationLevel`，但如果不单独拆开，很容易把 field ionization 理解成“额外生成几个电子”。源码里真实发生的是三件事一起改变：

1. 源离子 species 在运行时新增整数属性 `ionizationLevel`；
2. 电离事件通过 `filterCopyTransformParticles` 复制到 product electron species；
3. 后续 `rho/J` 沉积时，源离子的有效电荷按 `ionizationLevel` 动态乘上去。

PhysicalParticleContainer::InitIonizationModule() 会在拿到运行时 dt 后, 才真正初始化 ADK 所需的:

- ionization_energies
- adk_power
- adk_prefactor
- adk_exp_prefactor
- 可选 adk_correction_factors

同时还会在没有现成属性时补上:

```
if (!HasAttrib("ionizationLevel")) {  
    AddIntComp("ionizationLevel");  
}
```

这说明 ionizationLevel 属于“模块初始化阶段动态加入的持久物理状态”, 而不是 PIdx builtin。

这里还有两个容易被漏掉的源码边界。

第一, 当前工作树里的 field ionization 只有 ADK 主链, 没有独立 OTB 分支。官方参数文档对 do_field_ionization 的描述明确写的是 using the ADK theory, 源码里读到的也只有 do_adk_correction、adk_prefactor、adk_exp_prefactor 和 adk_power。因此如果后面讨论 OTB, 那只能作为“当前源码未实现的外部模型边界”, 不能写成 Ionization.* 已有的一条并行实现。

第二, physical_element 也不是“用户手工给一串电离能表”的接口。InitIonizationModule() 实际是通过 ion_map_ids、ion_atomic_numbers 和 ion_energy_offsets 从 WarpX 内建的 table_ionization_energies 里切出整段 successive ionization energies, 再按当前运行时 dt 现算 ADK prefactor。因此 species 构造期不能完成这一步, 真正原因不是代码组织习惯, 而是 ADK 率已经把本步 dt 吸进了系数里。

真正的单粒子电离判定在 Particles/ElementaryProcess/Ionization.H 的 IonizationFilterFunc::operator()。它不会只看实验室系 $|E|$, 而是先 gather 主场和 external particle field, 再结合当前 $ux/uy/uz$ 计算粒子系场幅值, 然后才用 ADK 公式给出概率:

```
Real w_dtau = (E <= 0._rt) ? 0._rt : 1._rt/ ga * m_adk_prefactor[ion_lev] *  
    std::pow(E, m_adk_power[ion_lev]) *  
    std::exp( m_adk_exp_prefactor[ion_lev]/E );  
const Real p = 1._rt - std::exp( - w_dtau );
```

因此 boosted-frame field_ionization regression 不是“换个坐标跑一遍”而已, 它实际上在验证这层相对论一致性。

更容易被忽略的是 transform 本身。IonizationTransformFunc 看起来只有一句:

```
src.m_runtime_idata[0][i_src] += 1;
```

但这并不意味着 product electron 没被创建。真正的 product species 写入, 是由 MultiParticleContainer::doFieldIonization() 里的:

```
filterCopyTransformParticles<1>(...)
```

统一完成的。也就是说, 一次电离事件被拆成:

- 源离子 ionizationLevel += 1
- 新电子复制到 ionization_product_species

两部分。

最后, 这个离化态不会只停留在 diagnostics。WarpXParticleContainer::DepositCurrent() 和 DepositCharge() 都会把 ionizationLevel 传下去, 而底层沉积 kernel 会做:

```
if (do_ionization) { wq *= ion_lev[ip]; }
```

因此 field ionization 的真正闭环是:

ADK event $\rightarrow$ ionizationLevel $\rightarrow$ effective source charge $\rightarrow \rho, J \rightarrow$ fields and diagnostics.

从粒子推进的视角看, 这说明 WarpX 的多物理不总是“在主循环旁边加一个模块”。像 field ionization 这样的过程, 会直接改写粒子属性系统和沉积权重, 所以它应该被视为 particle algorithm 本体的一部分。

更细一点说, 这里的“改写沉积权重”并不是去改宏观粒子统计权重 w , 而是保持 w 不变、只把有效物理电荷改成

$$wq_e \times \text{ionizationLevel}.$$

这也是 `InitIonizationModule()` 一开始就把 `ionizable species` 的 `charge` 强制改回 `q_e` 的原因：源码要把“多少个 `physical particles`”与“每个 `physical particle` 当前带几个基本电荷”这两层语义拆开，前者留在 `w`，后者留在 `ionizationLevel`。

4.13 Collisions: 不是旁路模块，而是和 `momentum push` 强耦合的时间调度层

`field ionization` 还是“每个源 `species` 自己带 `product species` 逻辑”的模块，但 `collision` 从入口层开始就不是这样。`MultiParticleContainer` 构造时直接建：

```
collisionhandler = std::make_unique<CollisionHandler>(this);
```

运行时则统一走：

```
collisionhandler->doCollisions(step, cur_time, dt, this);
```

这说明 `collision` 在 `WarpX` 里的第一抽象层不是某个 `species` 的附加属性，而是一个面向整个 `MultiParticleContainer` 的统一调度器。

`CollisionHandler` 的第一职责也不是实现物理公式，而是把：

```
collisions.collison_names
<collison_name>.type
```

映射成具体对象。当前入口层已经能分出：

- `pairwisecoulomb`
- `background_mcc`
- `pulsed_decay`
- `background_stopping`
- `dsmc`
- `nuclearfusion`
- `bremsstrahlung`
- `linear_breit_wheeler`
- `linear_compton`

其中有些是独立的 `CollisionBase` 派生类，有些则走模板化的 `BinaryCollision<CollisionFunc, CreationFunc>`。因此“`collision family`”在 `WarpX` 里从一开始就包含了既可能只改动动量、也可能会生成新粒子的过程。

真正的公共合同在 `CollisionBase`。它提供的不是一条统一散射公式，而是：

- `species`
- `ndt`
- `CollisionSteppingMode::`{`Supercycle`, `Subcycle`}

两种 `stepping mode` 的语义不同：

- `Subcycle`: 一个 `PIC step` 内跑 `ndt` 次，每次用 `dt/ndt`
- `Supercycle`: 每 `ndt` 个 `PIC steps` 才跑一次，但单次碰撞用 `dt*ndt`

这说明 `collision` 入口首先定义的是“碰撞算子相对于 `PIC` 主步怎样调度”，而不是“碰撞算子本体长什么样”。

更关键的是，`collision` 从全局参数读取阶段就已经和 `momentum pusher` 强耦合。`WarpX.cpp` 只要看到：

```
collisions.collison_names = ...
```

默认就会打开：

```
m_collisions_split_momentum_push = true;
```

然后再允许用户用：

```
collisions.split_momentum_push
```

覆盖。这说明 `collision` 默认并不是“在完整粒子推进前或后统一做一次”，而是插在 `momentum push` 的中间。源码还明确给了当前边界：

- `implicit evolve scheme` 下不真正支持 `split momentum push`
- `Higuera-Cary` 目前也不支持

因此, collision 入口从一开始就已经和:

- pusher 选择
- electrostatic / electromagnetic solver
- 主循环时间组织

绑在一起。

这层耦合并不是只靠源码阅读猜出来的, analysis_test_2d_collisions_split_momentum_push.py 直接拿 reduced diagnostics 的:

- field_energy
- particle_energy

做两类断言:

1. 总能量守恒误差必须足够小
2. 场能量涨落长期平均必须接近 equipartition 参考值

所以这组 regression 真正在验证的是 collision insertion ordering 的数值合同, 而不是某个散射截面是否长得对。

从粒子推进章节的视角看, 这意味着 WarpX 的多物理插入至少分成三种类型:

1. 改写粒子属性和沉积权重: field ionization
2. 改写 momentum push 的时间组织: collisions
3. 改写单粒子推进与产品粒子统计: QED / photon emission / pair creation

它们都不是“主循环旁边挂一个功能块”这么简单, 而是直接进入粒子算法本体。

4.13.1 BinaryCollision 先产出事件表, ParticleCreationFunc 再真正创建 products

如果继续沿 collisions 这一支往下读, 就会发现 BinaryCollision<CollisionFunc, CopyTransformFunc> 的 cell-level functor 并不直接往 product species 里塞粒子。它先输出的是几张事件表:

- p_mask
- p_pair_indices_1
- p_pair_indices_2
- p_pair_reaction_weight

也就是:

- 哪一对 parent 真发生了反应
- 这次反应对应哪两个 parent 粒子
- 这次反应要抽走多少权重交给 products

真正把这些事件落到 product species tile 里的, 是后面的 ParticleCreationFunc。

这层分工很重要, 因为它说明 collision 模块内部也不是“一次 kernel 既判断又创建”, 而是:

```
CollisionFunc
-> event tables
-> ParticleCreationFunc
-> type-specific momentum initialization
```

ParticleCreationFunc 里最关键的分支是:

```
const int products_per_reactant_factor =
    (m_collision_type == CollisionType::LinearCompton) ? 1 : 2;
```

它把 binary creation 分成两类:

1. LinearCompton
 - 每个 product species 每个事件只创建 1 个宏粒子
 - 散射 photon 继承入射 photon 位置
 - 散射 electron 继承入射 electron 位置
2. 几乎所有其他 binary creation
 - 包括 LinearBreitWheeler

- 都会在两个 reactant 的位置各复制一套 products

这就是为什么 LinearBreitWheeler 虽然物理上只产生一对 electron + positron, 实现上却会生成:

- 2 个电子宏粒子
- 2 个正电子宏粒子

并且每个宏粒子只拿一半反应权重。WarpX 用这种“复制到两个 parent 位置”的实现换取了局域电荷守恒。

LinearCompton 则反过来是当前最明显的特例。它不会做双份复制, 而是:

- 1 个散射 photon
- 1 个散射 electron

然后用 LinearComptonInitializeMomentum() 在电子静止系里按 Klein-Nishina 分布抽样散射角, 再 Lorentz 变换回 lab frame, 并用总动量守恒补出散射后电子动量。

所以如果把 collisions 再往 product 创建层推进一步, 当前最值得记的不是某个碰撞截面公式, 而是:

- BinaryCollision 先做事件压缩
- ParticleCreationFunc 再做统一 product 落地
- LinearBreitWheeler 和 LinearCompton 在“一个事件到底创建几个宏粒子”这件事上故意采用了两种不同合同

这也解释了为什么 linear_breit_wheeler 与 linear_compton 的 regression 都把:

- 守恒量
- product species 数目
- 最终产额

看得非常重, 因为真正容易出错的地方就在 product 布局和权重分配, 不只是事件概率。

4.13.2 BackgroundMCC、PulsedDecay 与 DSMC: collision 还有三条完全不同的后半段

前一小节已经把 BinaryCollision -> ParticleCreationFunc 这条 product 创建主链打通了, 但 WarpX 的 collision 还至少有三条不能混写进去的实现分叉。

第一条是 BackgroundMCCCollision。它不是“两个显式 species 配对后再散射”, 而是:

- 一个显式运动 species
- 一个由 background_density(x,y,z,t)、background_temperature(x,y,z,t) 或常数描述的背景气体
- 一个用 m_max_background_density 和截面表扫出来的 nu_max

因此它的核心上游稳定性门是

$$\nu_{\max}\Delta t,$$

而不是前面 BinaryCollision 那种 pair enumeration。更关键的是, 若打开 impact ionization, 它也不是走 ParticleCreationFunc, 而是走:

```
ImpactIonizationFilterFunc
-> filterCopyTransformParticles
-> 原电子动量更新 + 新电子/新离子创建
```

也就是说, BackgroundMCC 的 ionization 是 filter/copy/transform 分支, 不是 pair-event-table 分支。

第二条是 PulsedDecay。它甚至不再是 pairwise 碰撞, 而是 cell 内总权重衰变问题。源码先在每个 cell 统计 parent species 的总权重, 再按

$$W_{\text{prod}} = W_1 (1 - e^{-\nu(x,y,z,t)\Delta t})$$

算出本步应衰变掉多少权重, 然后再用 fixed_product_weight 把这份总权重离散成 product 宏粒子数。新粒子的位置和基底速度来自该 cell 内随机挑选的 parent 粒子, 再叠加方向相关 thermal speed。这里真正的物理合同是:

- parser 驱动的 decay_rate(x,y,z,t)
- fixed-weight 宏粒子离散化
- parent 权重逐步扣减

而不是“一对 parent 粒子 -> 一次散射事件”。

第三条是 DSMC 的 `SplitAndScatterFunc`。它虽然仍挂在 `BinaryCollision` 大框架下，但前半段 `DSMCFunc` 只负责：

- 组 pair
- 调 `CollisionPairFilter`
- 写 `p_mask/p_pair_indices/p_pair_reaction_weight`
- 先从 reactants 扣权重

真正的后半段 `product/child` 创建则交给 `SplitAndScatterFunc`，而不是前面那条 `ParticleCreationFunc`。这里最重要的 slot 语义是：

- slot 0: 第一 reactant 的 child copy
- slot 1: 第二 reactant 的 child copy
- slot 2/3: true products

于是：

- `elastic / back` 只在 slot 0/1 生成 weight-split child，并在 COM frame 随机旋转或反转速度；
- `charge_exchange / two_product_reaction` 在 slot 2/3 生成 products，并由 `TwoProductComputeProductMomenta(...)` 统一处理动量；
- `charge_exchange` 还会交换 products 的生成位置，以维持局域电荷守恒；
- DSMC ionization 则会同时保留 slot 0 的 incident child，并在 slot 2/3 生成 ejected electron 和 ion。

所以到这里，WarpX 的 collision 至少要分成四种后半段语义：

1. `BackgroundMCC` 的背景介质 + filter/transform
2. `PulsedDecay` 的 cell 内总权重衰变
3. `BinaryCollision` + `ParticleCreationFunc`
4. `BinaryCollision` + `SplitAndScatterFunc`

这也是为什么不能再用一套统一口径把 `BackgroundMCC`、`PulsedDecay`、`DSMC`、`LinearCompton` 和 `QED` 都简单写成“会产生新粒子的碰撞”。

在 regression 层，这三条分叉也看完全不同的量：

- `analysis_collision_3d_pulsed_decay.py` 比的是最终 ion 总权重和 0D 衰变模型；
- `analysis_ionization_dsmc_3d.py` 比的是 `n_e`、`n_n` 和 `n_eT_e` 的全局模型；
- `analysis_charge_exchange_dsmc_1d.py` 比的是通量指数衰减；
- `analysis_two_product_reaction_dsmc_1d.py` 比的是 products 的理论速度；
- `analysis_photoneutralization_dsmc_1d.py` 还把快电子的产物验证接到了 `BoundaryScraping` diagnostics。

4.13.3 `pairwisecoulomb`、`bremsstrahlung`、`background_stopping` 与 `nuclearfusion`

把上一节的 `BackgroundMCC` / `PulsedDecay` / `DSMC` 再往外扩，WarpX 当前 collision 里还剩四条不能并进同一套 product 语义的主分叉。

第一条是 `pairwisecoulomb`。它虽然也走 `BinaryCollision` 的 cell 内 pairing 外壳，但 `PairWiseCoulombCollisionFunc` 最后只做一件事：

`ElasticCollisionPerez(...)`

也就是说，它没有 products，也不依赖后面的 `ParticleCreationFunc` 或 `SplitAndScatterFunc`。构造期真正重要的只有：

- `CoulombLog`
- `use_global_debye_length`

当 `CoulombLog < 0` 且不使用 global Debye length 时，源码会进一步要求运行时 local temperature，从而按局域状态估算碰撞尺度。它的 regression 口径也因此不是“有没有新粒子”，而是：

- `analysis_collision_1d.py` 检查 relaxation benchmark
- `analysis_collision_1d_correct_conservation.py` 检查总动量和总动能守恒

第二条是 `bremsstrahlung`。它重新回到 pair-event 表，但后半段 product 创建也不是通用 `ParticleCreationFunc`，而是专门的 `PhotonCreationFunc`。前半段 `BremsstrahlungFunc` 只给出：

- 哪一对发生事件
- photon 权重
- photon 能量

后半段再由 PhotonCreationFunc:

1. 在第一 reactant 位置上创建 photon
2. 同时回写 parent electron/ion 动量
3. 用能量动量守恒补全 photon 动量

因此 bremsstrahlung 的真实语义是“先算失能和 photon energy,再做 parent+product 一起更新”。analysis_collision_1d_Bremsstrahlung. 也正是按这个口径验证:

- 总能量守恒
- 总动量守恒
- dE/dx 与解析估计
- 每步新 photon 数与解析截面估计

第三条是 background_stopping。这已经不再是二体散射,而是对单 species 应用解析 slowing-down law。源码在 background_type = electrons 与 ions 之间分成两条公式:

- 对电子背景:

$$\frac{d\mathbf{u}}{dt} = -\alpha\mathbf{u} \Rightarrow \mathbf{u}^{n+1} = \mathbf{u}^n e^{-\alpha\Delta t}$$

- 对离子背景:

$$\frac{dW}{dt} = -\frac{\alpha}{\sqrt{W}}$$

再按解析积分结果缩放速度

因此 background_stopping 的 regression 口径也完全不同: ion_stopping/analysis.py 直接用 Python 重写同一组 slowing-down 公式,对 constant / parsed 的电子与离子背景逐点对照最终粒子能量。

第四条是 nuclearfusion。它又回到 pair-event 框架,但事件概率控制比普通 BinaryCollision 更复杂。构造期最关键的不是单个截面文件,而是:

- event_multiplier
- probability_threshold
- probability_target_value
- fusion type

其中 event_multiplier 用来提高稀有 fusion 事件的统计量,而 probability_threshold / probability_target_value 用来在单 pair 概率过大时自动把 multiplier 压回安全范围,避免系统性低估 yield。源码上的截面模型至少分成:

- BoschHaleFusionCrossSection: D-D / D-T 之类经典两产物 fusion
- ProtonBoronFusionCrossSection: p-B11, 对应 Tentori-Belloni 2023 的拟合

它们的 regression 也相应分层:

- analysis_two_product_fusion.py 检查 reactant 消耗、product 数、能量动量守恒、各向同性和理论 yield
- analysis_proton_boron_fusion.py 进一步检查 p-B11 的三 alpha 产物、热率拟合,以及 probability_threshold 过大时的故意失真场景

所以如果把已经成文的 collision 分支一起看,WarpX 至少已经出现了下面这些互不等价的后半段:

1. 纯动量散射: pairwisecoulomb
2. filter/transform: BackgroundMCC ionization
3. 固定权重衰变: PulsedDecay
4. DSMC child/product 重建: SplitAndScatterFunc
5. 通用 products: ParticleCreationFunc
6. photon 专用 products: bremsstrahlung 的 PhotonCreationFunc
7. fusion event-probability + specialized kinematics: nuclearfusion

这说明到目前为止,“collision 模块”已经不能再被当作一个统一 product 创建器来讲,而必须按物理合同和后半段实现机制分开处理。

4.13.4 kernel 细节层: Perez 有效碰撞密度、Bremsstrahlung photon 写回、fusion products 复制语义

把上一节四条分叉再往下读, 最容易误判的不是类型分派, 而是它们在 kernel 末端到底怎样把统计事件变成具体粒子动量。

对 `pairwisecoulomb` 来说, `PairWiseCoulombCollisionFunc` 真正交给 `ElasticCollisionPerez(...)` 的并不是“原始 cell 密度”, 而是一套先闭合局域尺度、再构造有效配对密度的量:

- 若 `CoulombLog < 0`, 先按局域 n, T 算 Debye 长度
- 再取 `bmax = max(lambda_D, r_min)`, 保证 screening length 不小于原子间距
- 然后构造加权后的

$$n_{12}$$

来驱动 Perez 散射

因此 WarpX 当前的 weighted macroparticle Coulomb 碰撞, 真正进入单对散射 kernel 的强度不是简单的 cell 物理密度, 而是“局域 plasma 尺度 + 抽样配对统计”共同决定的有效量。这也解释了为什么 Coulomb regression 里一个重点看 relaxation, 另一个重点看守恒。

对 `bremsstrahlung` 来说, 上一节只讲到了前半段 `BremsstrahlungFunc` 如何给出 photon energy。再往后看 `PhotonCreationFunc`, 会发现它不是只把 energy 填进 product, 然后结束。它会先在离子静止系内解出:

- 更新后的 electron
- 更新后的 ion
- photon 三动量

再一起变回 lab frame。最后写到 photon species 的不是“真实质量意义下的速度”, 而是

$$u = \frac{p}{m_e}$$

型的统一接口变量:

```
upx = p3x_rel / PhysConst::m_e;  
upy = p3y_rel / PhysConst::m_e;  
upz = p3z_rel / PhysConst::m_e;
```

这样做不是修改 photon 物理, 而是为了让 collision-created photons 继续复用 WarpX 现有的粒子 SoA 与 diagnostics 链。

对 fusion 来说, 上一节已经区分了 D-D / D-T 两产物和 p-B11 三 alpha 两条物理路径; 这一层再补的是“为什么实现里看到的宏粒子数更多”。两产物 fusion 的 `TwoProductFusionInitializeMomentum(...)` 其实只算一次 products 动量, 然后把:

- 第一 product 的一组 (ux, uy, uz) 写两次
- 第二 product 的一组 (ux, uy, uz) 也写两次

所以实现里出现 4 个宏粒子, 并不表示一次 fusion 物理上有 4 个独立 products, 而是因为 WarpX 在两个 parent 位置各复制一套 child。

p-B11 的 `ProtonBoronFusionInitializeMomentum(...)` 进一步复杂一些。它先按

`p + B11 -> alpha1 + Be8*`

做一次两产物动量求解, 再在 `Be8*` 静止系里把剩余 3.12 MeV 衰变能随机分到:

`Be8* -> alpha2 + alpha3`

最后再变回 lab frame。于是实现里会看到 6 个 alpha 宏粒子:

- alpha1 两份
- alpha2 两份
- alpha3 两份

本质上仍然是“在两个 parent 位置各复制一套 products”的事件记账方式, 而不是 6 个不同物理 alpha 通道。

所以到这一层可以把 collision 剩余 kernel 细节压成三句:

1. Perez 路径最关键的是局域尺度闭合和加权 n_{12}
2. Bremsstrahlung 最关键的是 photon 动量写回并与 parent 更新同时完成
3. Fusion 最关键的是“先算真实 kinematics, 再做双 parent 位置复制”的宏粒子实现语义

4.13.5 更深一层的 event kernel: Perez 角分布、Bremsstrahlung 能谱反演与 fusion 概率控制

如果再往 BinaryCollision 的 event kernel 里读, collision 还有一层之前没写清的共同问题: 上一节讲的是“哪些量会被写回”, 这一层讲的是“这些量到底怎样被抽出来”。

对 pairwisecoulomb 而言, ElasticCollisionPerez(...) 上游已经准备好了:

- bmax
- sigma_max
- 加权后的 n12

但 UpdateMomentumPerezElastic(...) 并不是把这些量直接当成一次 Bernoulli 碰撞概率。它先在 COM frame 内构造归一化散射长度

$$s_{12},$$

然后按 s12 所处区间切到四段式角分布:

- $s_{12} \leq 0.1$: $\cos X_s = 1 + s_{12} * \log(r)$
- $0.1 < s_{12} \leq 3$: 用五次多项式近似的 Ainv
- $3 < s_{12} \leq 6$: 用 $A = 3 \exp(-s_{12})$ 的另一段反演
- $s_{12} > 6$: 直接退化成 $\cos X_s = 2r - 1$ 的各向同性散射

随后再抽一个独立方位角, 把 post-collision 动量从 COM frame 变回 lab frame。更关键的是, 最后并不是无条件同时更新两只 reactant, 而是分别按:

```
if (w2 > r * max(w1, w2)) update particle 1
if (w1 > r * max(w1, w2)) update particle 2
```

做 weighted rejection。因此 WarpX 当前的 weighted macroparticle Coulomb collision, 真正的语义是:

- n12 和 s12 决定散射强度
- reactant 是否真正写回, 再由宏粒子权重比决定

对 bremsstrahlung 而言, 前两节已经区分了 BremsstrahlungFunc 和 PhotonCreationFunc。更深层看 BremsstrahlungEvent(...), 会发现它的前半段先把电子变到离子静止系, 再结合电子密度构造 plasma-frequency cutoff:

$$E_{\text{cut}} = \hbar \omega_{pe}.$$

如果电子在离子静止系里的动能 KE_eV 低于这个阈值, 就直接返回 0, 不让软光子进入采样。若允许发生事件, 代码并不是从表里拿一个最近点, 而是先按电子能量方向插值, 再在 k/T1 网络上逐段积累 trapezoidal cross section, 最后在命中的区间内反演出连续 photon energy。

这一层还有一个必须记录的当前实现边界: event probability 用的核心量是

$$\arg = f_{\text{multi}} v_1 \sigma_{\text{total}} n_2 dt \frac{\gamma_1^{\text{rel}}}{\gamma_1 \gamma_2}.$$

当 $\arg > 1$ 时, 源码会先把 $\arg$ 饱和到 1, 再去改 fmulti。按当前实现, 这一分支直接生效的是 probability saturation, 而不是像 fusion 那样形成真正有效的 multiplier backoff。这一点应当被视为当前源码边界, 而不是已有功能。

nuclearfusion 的 SingleNuclearFusionEvent.H 则正好相反: 这里的 probability control 是真的做完了。它先算

$$P_{\text{est}} = \text{multiplier_ratio} f_{\text{mult}} \text{lab_to_COM_factor} w_{\text{max}} \sigma_f(E_{\text{coll}}) v_{\text{coll}} \frac{dt}{dV},$$

如果超过 probability_threshold, 就按 probability_target_value 回退到一个新的 fusion_multiplier_eff >= 1, 再把 product weight 缩成

$$w_{\text{product}} = \frac{w_{\text{min}}}{f_{\text{mult,eff}}}.$$

最终真正使用的事件率也不是简单线性截断, 而是

$$P = 1 - e^{-P_{\text{est}}},$$

在代码里写成 -std::expm1(-probability_estimate), 专门保证小概率时的数值稳定性。

这一层和 regression 的关系也更清楚了:

- 2D / 3D Coulomb tests 看的主要是指数 relaxation fit

- `inputs_test_3d_collision_iso{,_subcycle}` 看的是各向异性温度的 isotropization 解析解, 以及 `ndt_subcycle` 是否保持同一 collision timestep
- `inputs_test_rz_collision` 看的是 cylindrical-cell 配对与临时动量旋转假设下, 局域共线粒子几乎不发生有效散射
- `background_mcc` 的 `capacitive_discharge` 条目则不该再混成普通 collision 单元测试: 1D PICMI 入口实际拆成 `analysis_1d.py` 与 `analysis_dsmc.py` 两条 case-1 profile 对照, 前者验证 `background_mcc + external Poisson solver callback`, 后者验证把 DSMC 分支接入同一骨架后的离子密度 profile; 2D native / PICMI 入口当前仍主要是应用级 checksum 基线, 而 `test_2d_background_mcc_dp_psp` 在当前 `CMakeLists.txt` 中仍是注释掉的遗留变体

4.13.6 性能与数值后处理层: resampling、thermalizer 与 sorting

在 collision/QED 这些多物理分支之外, `Particles/` 里还有一层更偏数值控制与性能优化的对象图:

- Resampling
- ParticleThermalizer
- Sorting

它们不直接改变场求解器, 但会显著改变宏粒子数、粒子分布以及后续 kernel 的访存局部性。

Resampling 是 species 级开关。`PhysicalParticleContainer.cpp` 先读 `do_resampling`, 再按 species 构造一个 Resampling 对象。它内部又拆成:

- ResamplingTrigger
- ResamplingAlgorithm

触发条件不是只有固定步数, 而是

$$\text{intervals.contains(step)} \vee \left(\frac{N_{\text{global}}}{N_{\text{cells,global}}} > \text{resampling_trigger_max_avg_ppc} \right).$$

调用位置在主循环里对应的是 `istep[0]+1`, 所以匹配的是“下一步编号”。一旦触发, WarpX 会先 `Redistribute()`, 再在各 level/tile 上调用具体算法, 最后统一 `deleteInvalidParticles()`。因此 resampling 和 collision、absorbing boundary、EB scraping 一样, 仍沿用“kernel 内先标 invalid, 之后统一删粒子”的合同。

当前正式接入的算法有两条。LevelingThinning 是按 cell 独立工作的: 对每个 cell 先算平均权重

$$\bar{w}_{\text{cell}},$$

再定义

$$w_{\text{level}} = \bar{w}_{\text{cell}} \times \text{target_ratio}.$$

凡是 $w_i \leq w_{\text{level}}$ 的粒子, 就以

$$1 - \frac{w_i}{w_{\text{level}}}$$

的概率删除, 否则把权重直接抬到 w_{level} 。这条算法只会抬低权重粒子, 不会动大权重尾部。对应的 `test_2d_leveling_thinning analysis` 也比较强: 一类 species 检查最终粒子数和统一权重, 一类 species 检查 Gaussian 权重分布下的 level-weight 和 untouched heavy tail。

VelocityCoincidenceThinning 则不是简单抽稀, 而是 merge。它先在每个 cell 内按速度空间分 bin, 可以是 spherical grid, 也可以是 cartesian grid; 然后把同一个 velocity bin 内的 cluster 压成两个保留粒子。实现上会累计 cluster 的总权重、加权平均位置、加权平均动量和总动能, 再构造一对关于平均动量对称的新动量, 使 cluster 内的线动量和动能都守恒, 其余粒子全部标 invalid。这里 `resampling_algorithm_target_weight` 的实际语义也要记清: 内部会乘 2, 因为每个 cluster 最后固定压成两粒子。当前这条路径只有 checksum, 没有像 LevelingThinning 那样的独立 analysis。

ParticleThermalizer 则是单独的全局参数块 `particle_thermalizer.*`, 不放在 species 段里。它当前只支持 Cartesian 1D/2D/3D, 不支持 RZ/RCYLINDER/RSPHERE。在主循环中的插入位置非常具体:

1. moving window
2. particle boundary / EB scraping / invalid 删除 / sorting
3. `m_particle_thermalizer.applyThermalizer(*mypc)`
4. collisions

因此 thermalizer 的效果会先落到“已经通过边界筛选保留下来的粒子”上, 再进入碰撞模块。它也不是硬墙式 momentum reset, 而是在一个有法向、起止位置和渐进概率的 thermal region 内, 对超过 `momentum_threshold` 的动量分量抽样重置为方差由 `theta` 决定的 Gaussian。当前本地并非完全没有 thermalizer 验证: `particle_absorbing_boundary` 会显式打开 `particle_thermalizer.*`,

并用 PhaseSpaceElectrons 直方图断言吸收边界附近的反向高速电子权重显著降低；但这仍是 absorbing boundary、field-function laser、reduced diagnostic 和 thermalizer 的耦合 regression，不是 dedicated thermalizer-only 单测。

Sorting 则必须和前文 AMR coarse-fine 的 PartitionParticlesInBuffers() 区分开来。后者是物理分流，前者是全局性能阶段。WarpXEvolve.cpp 在 boundary 处理之后检查 sort_intervals.contains(step+1)，触发后调用：

```
mypc->SortParticlesByBin(
    sort_bin_size, m_sort_particles_for_deposition, m_sort_idx_type);
```

文档和 WarpX.cpp 一起看，当前默认值是平台相关的：

- GPU 默认 sort_intervals = 4
- CPU 默认 sort_intervals = -1

目的是提升 memory locality，不是物理修正。并且还有两种模式：

- sort_particles_for_deposition = true
 - 走 deposition-specialized sort
- sort_particles_for_deposition = false
 - 走 generic bin sort，粒度由 sort_bin_size 控制

所以到这里，Particles/ 的“非主物理 kernel 层”也形成了清晰分层：

- resampling: 调粒子数与权重/局域 phase-space 代表性
- thermalizer: 调局部高动量粒子的热化行为
- sorting: 调后续 deposition/gather 的访问局部性

4.13.8 particle_pusher、single_particle、larmor、photon_pusher 的真实验证边界

Particles 目录里还有一组很容易被简单统称为“单粒子 test”的 regression：

- particle_pusher
- single_particle
- larmor
- photon_pusher

但把 inputs、analysis 和源码入口对起来之后，它们其实测的是四种不同合同。

particle_pusher 是最直接的一条。输入里固定：

```
algo.particle_pusher = "higuera"
particles.B_ext_particle_init_style = "constant"
particles.B_external_particle = 0.0 0.0 1.0
particles.E_ext_particle_init_style = "constant"
particles.E_external_particle = -2.994174829214179e+08 0.0 0.0
```

并让单个 positron 在满足

$$E_x = -v_y B_z$$

的 force-free 场中跑 10000 步。analysis.py 只检查最终

$$x \approx 0.$$

因此它并不是一般轨道画图，而是直接给：

- PushSelector.H 的 ParticlePusherAlgo::HigueraCary
- UpdateMomentumHigueraCary(...)
- UpdatePosition(...)

这条 relativistic Higuera-Cary push 主链做强断言。

本项目在当前 checkout 上又完成了一次真实单进程复现。运行产物位于：

- /Volumes/PHILIPS/programs/PIC/PIC-tutor/runs/stage-c-validation/particle_pusher_higuera
- 末态 plotfile: diags/diag1010000
- 项目分析脚本: scripts/analyze_particle_pusher_contract.py

- 合同报告: `particle-pusher-contract.json`、`particle-pusher-contract.md`

实际末态为 `current_time = 100.00000000001425`, 单个 positron 的

$$\max |x| = 1.1430664323700516 \times 10^{-4}$$

小于官方 $1e-3$ 容差, 且官方 `analysis.py` 与项目脚本都通过。这个证据可以把“当前 binary 确实走 Higuera-Cary force-free 主链”从静态输入/源码判断推进到运行级验证, 但它仍然只覆盖单进程、单粒子、恒定外场和 `x approximately 0` 这一条合同, 不等价于对 Boris/Vay/Higuera-Cary 三者的完整轨道 benchmark。

在同一官方输入上只替换 `algo.particle_pusher` 后, 本项目还做了一个 local sibling 对照:

pusher	末态 $\max x $	$1e-3$ gate	解释
Boris	2.3213958529e3	FAIL	force-free cancellation 在这个高相对论设置下明显失真
Vay	1.0795497978e-4	PASS	保留较好的 relativistic frame/cancellation 行为
Higuera-Cary	1.1430664324e-4	PASS	在该合同下与 Vay 同量级

完整 sibling JSON/Markdown 对照报告位于 `/Volumes/PHILIPS/programs/PIC/PIC-tutor/runs/stage-c-validation/particle_pusher` 由 `scripts/compare_particle_pusher_siblings.py` 重建。必须保留其证据等级: 三组使用的是官方输入加 `pusher-only override` 的项目级对照, 不是 `CMakeLists.txt` 注册的三条独立官方 regression; 它支持的是 force-free cancellation 的差异提示, 不替代 boosted-frame、Poincare section 或长期能量/相空间 benchmark。

`single_particle` 则必须拆成两类。

第一类 `inputs_test_2d_bilinear_filter` 虽然也只有一个电子, 但 `analysis` 完全不看轨道, 而是手工构造未滤波 `Jx`、再用二维 bilinear kernel 卷积, 最后和 `plotfile` 里的 `jx` 比较。它真正验证的是:

- 单粒子沉积后的 current stencil
- `warpx.use_filter`
- `warpx.filter_npass_each_dir`
- 对称边界 bilinear filtering 的实现

所以它更接近 deposition/filter regression。

第二类 `inputs_test_1d_synchronize_velocity` 则在 `algo.maxwell_solver = none` 下给一个电子施加常量 `E_z`, 同时打开:

`warpx.synchronize_velocity_for_diagnostics = 1`

`analysis` 先在 Python 里手动做 half-backward、5 步 leapfrog、再加 half-forward 得到同步后的速度, 然后和 `diagnostics` 输出比较。这条 regression 直接对应:

- 参数 `warpx.synchronize_velocity_for_diagnostics`
- `WarpXEvolve.cpp` 里 `diagnostics` 前的 `SynchronizeVelocityWithPosition()`

也就是说, 它验证的不是 Boris 物理轨道误差, 而是“输出给 `diagnostics` 的速度是否和位置处在同一时间层”。

本项目对这条路径也完成了真实单进程复现。运行产物位于:

- `/Volumes/PHILIPS/programs/PIC/PIC-tutor/runs/stage-c-validation/single_particle_synchronize_velocity`
- 末态 `plotfile`: `diags/diag1000005`
- 项目分析脚本: `scripts/analyze_single_particle_synchronization.py`
- 合同报告: `single-particle-sync-contract.json`、`single-particle-sync-contract.md`

第 5 个诊断步的理论/模拟结果为:

- $z = 2.2985203786002786/2.2985203756075920$;
- $u_z = 879410.0053860814/879410.0053860815$;
- 相对速度误差 $1.3237889e-16 < 1e-15$ 。

这条运行证据支持的是 diagnostics time-level synchronization, 不应被误写成单粒子 pusher 的独立轨道精度 benchmark。

photon_pusher 又是另一类。输入里建了 16 个 photon species, 覆盖:

- 六个坐标轴正负方向
- 对角方向
- 动量大小 1 和 10

analysis 用理论式

$$\mathbf{x}(t) = \mathbf{x}_0 + ct \hat{\mathbf{u}}, \quad \mathbf{p}(t) = \mathbf{p}_0$$

逐 species 比较最终位置和动量。因此它真正打到的是:

- PhotonParticleContainer::PushPX()
- UpdatePosition(...) 里的 massless branch
- photon 不做 charge/current deposition 的合同

本项目已完成该官方 regression 的单进程复现。运行产物位于:

- /Volumes/PHILIPS/programs/PIC/PIC-tutor/runs/stage-c-validation/photon_pusher
- 末态 plotfile: diags/diag1000050
- 项目分析脚本: scripts/analyze_photon_pusher_contract.py
- 合同报告: photon-pusher-contract.json、photon-pusher-contract.md

16 个 photon species 的末态最大相对误差为:

- 位置直线传播: $6.0986372\text{e-}16 < 1\text{e-}14$;
- 动量保持: $1.7217530\text{e-}16 < 2.2204460\text{e-}16$ 。

这条证据支持的是无质量粒子的 c 速率传播、方向保持和不参与带电粒子 current deposition 的路径, 不应与 Boris/Vay/Higuera-Cary 的带电动量旋转合同混写。

最后 larmor 反而要最保守。它输入里组合了:

- electron 和 positron
- 常量外部粒子磁场 B_y
- amr.max_level = 1
- PML
- warpx.do_div_cleaning = 1
- full/raw diagnostics

但 CMakeLists.txt 里没有独立 analysis, 只有 checksum。因此当前更准确的说法是:

- 这是一个 charged-particle gyro-motion 与 external-particle-field、MR、PML、div-cleaning 组合稳定性的应用级 checksum 基线
- 不是已经有独立解析半径/回旋频率对照的强单粒子 analysis

本项目也运行了该 case 的单进程版本, 并新增 scripts/analyze_larmor_continuum_audit.py 做 uniform-(B_y) 连续轨道审计。末态位于 /Volumes/PHILIPS/programs/PIC/PIC-tutor/runs/stage-c-validation/larmor_single_process/diags/diag100001 电子/正电子的轨迹相对位移误差均为 $1.28285096\text{e-}2$, 动量相对误差均为 $3.44029897\text{e-}2$ 。这不是一个失败的官方 regression: checksum 仍是当前官方合同; 它说明在 MR/PML/div-cleaning 组合下, 直接把连续 uniform-(B) 解析轨道当成严格 gate 并不成立。因此本章继续把 larmor 标为 checksum-only, 并把该审计报告定位为“为什么暂不升级强物理 gate”的证据。

这组 regression 目前能明确支持的结论是:

- Higuera-Cary force-free relativistic push: 有强 analysis。
- diagnostics 半步速度同步: 有强 analysis。
- bilinear current filter: 有强 analysis。
- photon 直线传播与动量守恒: 有强 analysis。
- Larmor 半径/频率独立解析对照: 当前这组里没有看到, 不能夸大。
- Larmor 连续轨道审计: 已运行, 但当前只作为 diagnostic evidence, 未升级为强 gate。

4.13.9 particle_fields_diags 与 plasma_lens: 粒子 diagnostics 和粒子侧外场的两类强验证

在这组“单粒子/推进器”validation 之外, 还有两类更适合回填第 4 章正文的粒子相关 regression:

- particle_fields_diags
- plasma_lens

它们都直接消费粒子位置、动量或外部粒子场, 但验证对象并不是普通 pusher 轨道。

particle_fields_diags 的关键输入不是单粒子初值, 而是 diagnostics 配置:

```
diag1.particle_fields_to_plot = z uz uz_filt zuz jz
diag1.particle_fields_species = electrons protons photons
diag1.particle_fields.z(x,y,z,ux,uy,uz) = z
diag1.particle_fields.uz(x,y,z,ux,uy,uz) = uz
diag1.particle_fields.uz_filt(x,y,z,ux,uy,uz) = uz
diag1.particle_fields.uz_filt.filter(x,y,z,ux,uy,uz) = (uz < 0)
diag1.particle_fields.zuz(x,y,z,ux,uy,uz) = z * uz
diag1.particle_fields.jz(x,y,z,ux,uy,uz) = uz*q_e
diag1.particle_fields.jz.do_average = 0
```

analysis 会同时做三件事:

1. 用 yt 从粒子数据直接手工重建 cell-centered quantity
2. 读取 Full plotfile 中的 boxlib particle-field meshes
3. 再读取 openPMD 中同名 meshes

然后逐一比较:

- zavg
- uzavg
- zuzavg
- uzavg_filt
- jz

因此它真正验证的是:

- Diagnostics.cpp 对 particle_fields_to_plot、.filter(...).do_average 的参数解析
- ParticleReductionFunctor 如何把粒子归约成 cell-centered diagnostics field
- plotfile 与 openPMD writer 在这一层是否一致

这条 regression 说明: WarpX 不仅能把粒子作为散点写出去, 还能把 species 内的粒子数据按 parser 合同重新投影成网格诊断量。

plasma_lens 则落在粒子侧外场主链上。analysis 不是读主网格场, 而是直接比较两颗测试电子穿过 lens 序列后的:

- 最终横向位置
- 最终横向动量

与解析透镜串联模型是否一致。

这里要区分两条源码入口。

第一条是:

```
particles.E_ext_particle_init_style = repeated_plasma_lens
particles.B_ext_particle_init_style = repeated_plasma_lens
```

它对应:

- MultiParticleContainer.cpp 读取 lens 周期、起点、长度、强度
- GetExternalEBField 在粒子 gather 之后、push 之前按粒子位置实时叠加 lens focusing field

第二条是 hard-edged 版本:

```
lattice.elements = ...
plasmalens*.type = plasmalens
plasmalens*.dEdx = ...
```


它对应 accelerator lattice 分支里的 HardEdgedPlasmaLens。

两者共用同一个 analysis, 因此真正被验证的是:

- repeated-plasma-lens 粒子侧外场
- accelerator-lattice hard-edged lens
- boosted-frame plasma-lens 反变换后一致性
- short-lens residence correction

也就是说, 这一组 regression 对第 4 章的重要补充不是“又一个轨道图”, 而是:

- 粒子 diagnostics 可以把粒子属性重新压成 cell-centered field
- 粒子侧外场可以完全绕过主网格场寄存器, 直接通过 GetExternalEBField 进入 PushPX()

accelerator lattice 自身也已经有当前本地很直接的强基准: Examples/Tests/accelerator_lattice/hard_edged_quadrupoles*. 这组 tests 用单电子穿过 drift + quad + drift + quad 串联, analysis 直接从输入参数重建 lattice.elements、drift.ds、quad.ds、quad.dEdx, 再用解析 hard-edged quadrupole 透镜公式逐段积分, 要求最终 x 误差低于 1%、u_x 误差低于 0.2%。而 boosted-frame 与 moving-window 变体继续共用同一解析对照, 因此这里真正被验证的不是“lattice 参数能读入”, 而是 HardEdgedQuadrupole + LatticeElementFinder + PushPX() 的联合运行态合同。

这里还有一个需要在正文里说清的源码边界: drift 在 accelerator lattice 中只提供 ds -> zs/ze 的几何账本, 不直接返回外场; 运行期真正给粒子加 E/B 的只有 HardEdgedQuadrupole 和 HardEdgedPlasmaLens 两类 device element, 而 LatticeElementFinder 做的是按 tile 建 nearest-element lookup table、把 boosted-frame 下的粒子坐标和步末 $z+v_z dt$ 反变换回 lab frame、调用各元件 get_field(...), 最后再把累计场变回 boosted frame 后加进 PushPX() 的粒子外场。也就是说, drift + quad + drift + quad 里的 drift 只进入解析 beamline 几何和 residence 区间判定, 不进入 runtime field accumulation。

至于 pass_mpi_communicator, 当前更适合留在 regression 索引和工程状态里, 不需要占正文篇幅。它现在的真实状态是:

- PICMI / Python 层已经暴露 mpi_comm 参数
- 但 _libwarpx.py 仍对非空 mpi_comm 直接报“not yet supported”
- 因此 CMake 中 analysis/checksum 都是 OFF

它验证的是 Python/MPI 初始化接口, 而不是粒子算法本体。

4.13.10 particle_boundary_scrape、particle_data_python 与 single-precision diagnostics: 粒子 Python 接口的三类合同

如果把 Particles 的 validation 再往“接口层”收紧一层, 当前本地 checkout 里还有三组很关键但容易被混成杂项的条目:

- particle_boundary_scrape
- particle_data_python
- particle_fields_diags single-precision FIXME

它们共同验证的不是单粒子轨道, 而是:

- scraped-particle buffer
- Python runtime attribute / injection / deposition wrapper
- 单精度粒子 diagnostics 误差边界

particle_boundary_scrape 有两层证据。native 输入 inputs_test_3d_particle_scrape 配一个立方体 EB 和一束电子, analysis_scrape.py 只做最小但很硬的断言:

- 第 40 步还应有 612 个电子
- 第 60 步主 species 中电子数应变成 0

这说明 ScrapeParticlesAtEB() 确实把撞到 embedded boundary 的粒子删掉了。

但更强的是 PICMI 变体 inputs_test_3d_particle_scrape_picmi.py。它在 sim.step(...) 之后直接构造 ParticleBoundaryBufferWrapper(), 然后检查:

- EB buffer 中累计粒子数是 612
- stepScraped 全都大于 40
- 所有 rank 汇总后的 buffer 粒子总数仍是 612
- clear_buffer() 之后 buffer size 回到 0

所以这一组 regression 真正验证的是两层合同同时成立:

1. EB scraping 确实把粒子从主容器里删除
2. 删除掉的粒子确实进入了 Python 可访问、可清空的 boundary buffer

particle_data_python 则完全是另一类测试。它没有独立 analysis.py, 强断言直接写在 PICMI 输入脚本里。inputs_test_2d_particle_attr_access.py 会:

- sim.initialize_warpx()
- sim.particles.get("electrons")
- add_real_comp("newPid")
- 在 beforestep callback 里持续 add_particles(...)
- 再直接断言 get_real_comp_index(...), tile 里的 newPid 值, 以及 Python wrapper 暴露的 deposit_current(...) 确实能把电流沉到 current_fp

inputs_test_2d_prev_positions_picmi.py 则验证:

- warpx_save_previous_position=True
- prev_x/prev_z runtime attributes 确实被加进 species
- PushPX() 在推进前确实保存了旧位置

也就是说, particle_data_python 这组 regression 的本体不是“场或轨道物理”, 而是:

- Python 对 runtime attributes 的增删访问
- Python 注入粒子接口
- Python 手动沉积接口
- Python 到 C++ save_previous_position 运行时属性链

从当前 Python binding 源码再往下看, 这组接口其实正好对应三层薄桥: PICMI sim.particles 只是 convenience property, 真正返回的是 pybind 暴露的 WarpX::GetPartContainer(); species 级操作最终落在 WarpXParticleContainer.cpp 里诸如 add_n_particles(...), deposit_current(...), deposit_charge(...) 这些 binding; 而 beforestep / afterstep 一类注入回调则不是直接逐个注册到 C++, 而是先进入 Python CallbackFunctions 聚合表, 再以“每个 callback 名一个聚合 callable”的形式注册给 ExecutePythonCallback(name)。因此 particle_data_python 真正保护的是 PICMI convenience layer、pybind runtime object 和 callback bridge 这三层一起成立, 而不是单独某个 Python helper。

这里还有一个必须保守记下来的实现边界: test_2d_particle_attr_access_unique_picmi 名义上想验证 --unique 变体, 但当前输入脚本里 add_particles(...) 仍然硬编码 unique_particles=True, 并没有真正消费 args.unique。所以这条条目现在更像是“命名上存在的分支”, 不能写成已经覆盖了 unique/non-unique 两种注入语义。

同一条 Python 粒子接口线里, restart/inputs_test_2d_id_cpu_read_picmi.py 和 restart/inputs_test_2d_runtime_components.py 也值得单独记下。前者当前真正验证的是 idcpu 解包读取合同: 脚本直接遍历 pti["idcpu"], 再用 unpack_ids/unpack_cpus 断言累计和等于 5050/0。后者则把 add_real_comp("newPid"), callback add_particles(...), get_real_comp_index("newPid") 和 picmi.Checkpoint(...) 绑在一起, 证明动态 runtime component 与 checkpoint front-end 可以共存; 但它对应的 test_2d_runtime_components_picmi_restart 仍是 FIXME scaffold, 所以当前还不能写成“restart 后 runtime attrs 已被完整强回归验证”。

最后, particle_fields_diags 的 single-precision 变体也要单独说明。当前不是完全没有这条线, 而是:

- analysis_particle_diags_single.py 已经存在
- 它复用同一实现, 只把容差放宽到 5e-3
- 但 CMakeLists.txt 里整条 test_3d_particle_fields_diags_single_precision 仍被 # FIXME 注释掉

因此最准确的表述应当是:

- single-precision particle-field reductions 的 analysis 预案已经在本地源码树里
- 但它还不是活跃 regression

把这三组并到一起之后, 第 4 章里关于 Particles 的 validation 图景就更完整了: 不只是 pusher、collision、QED 和 diagnostics 场输出, 还有一整层 Python-side 粒子接口与 boundary-buffer / reduction 的工程合同正在被回归保护。

4.13.11 particle_boundary_interaction、particle_boundary_process、particle_thermal_boundary 与 plasma_lens_python

在 particle_boundary_scrape 和 particle_data_python 之外, 当前本地 checkout 里还有四组更靠近“边界行为 + Python front-end”的 regression:

- particle_boundary_interaction
- particle_boundary_process
- particle_thermal_boundary
- plasma_lens_python

它们共同的特点是：都不只是看粒子最终轨道，而是在测“粒子边界语义如何经由 Python callback、buffer、parser 或 reduced diagnostics 暴露出来”。

particle_boundary_interaction 的关键点首先不是 analysis，而是输入脚本本身的结构。它不是直接依赖 WarpX 内建的 EB 反射，而是：

1. 把电子打到一个球形 embedded boundary 上
2. 打开 warpx_save_particles_at_eb=1
3. 在 afterstep callback 里用 ParticleBoundaryBufferWrapper.get_particle_scraped_this_step(...) 取出：
 - deltaTimeScraped
 - r/theta/z
 - ux/uy/uz
 - nx/ny/nz
4. 在 Python 里手工做镜面反射
5. 再按 `dt - delta_t` 把粒子推进到这一步的末尾并重新 `add_particles(...)`

因此它真正验证的是：WarpX 的 scraped buffer 是否给出了足够的几何和时间信息，让用户能自己实现一个 boundary-interaction model。后面的 analysis.py 再用解析几何反射轨道比较最终位置，要求 x/z 相对误差都足够小。

这意味着 particle_boundary_interaction 的定位不是普通 boundary condition test，而是：

- scraped buffer + Python callback + custom reinjection model

在这些更复杂的边界 regression 之外，本地源码树里还有一个更“教科书式”的最小强基准：Examples/Tests/boundaries/。它不碰 embedded boundary、不碰 callback，也不依赖 reduced diagnostics，而是把三类 domain particle boundary 直接拆开测试：

- x 方向 reflecting
- y 方向 absorbing
- z 方向 periodic

输入里三类 species 都用 MultipleParticles 直接给定初始位置和动量；analysis 则显式检查：

- absorbing species 最终只剩 1 个粒子
- reflecting species 的速度严格翻号
- periodic species 的速度保持不变
- 两类保留粒子的位置都与解析反射/wrap-around 位置逐点一致

因此 particle_boundaries 验证的不是应用级“边界大致工作”，而是 ParticleBoundaries_K.H 最核心的三种 domain particle boundary 语义本体。

particle_boundary_process 又不同。它当前其实分成两条合同。

第一条是 test_2d_particle_reflection_picmi。虽然 analysis = OFF，但输入脚本本身做了直接自检：

- warpx_reflection_model_zhi = "0.5"
- 打开 save_particles_at_zhi/zlo
- 跑完后直接检查：
 - z_hi buffer 中粒子数是 63
 - z_lo buffer 中粒子数是 67
 - z_hi 的 stepScraped 全都等于 4
 - z_lo 的 stepScraped 全都等于 8

这组 test 真正测的是：

- absorbing boundary 上的随机反射模型 parser
- 上下边界 scraped buffer 的分流
- stepScraped 时间戳语义

第二条是 test_3d_particle_absorption。这里 analysis 很简单，只检查：

- 第 40 步仍有 612 个电子
- 第 60 步电子数变成 0

所以它更接近“EB 吸收后主 species 中粒子确实消失”的强吸收断言。

particle_thermal_boundary 也必须和前面已经整理过的 ParticleThermalizer 区分开。这里输入打开的是：

- boundary.particle_lo = thermal thermal
- boundary.particle_hi = thermal thermal
- boundary.<species>.u_th = ...

也就是 domain particle boundary 本身就是 thermal boundary。对应 ParticleBoundaries_K.H 的语义是：先像反射边界一样处理位置，再对法向/切向动量做热化抽样。analysis 并不看单粒子散射角，而是读取 reduced diagnostics 的：

- FieldEnergy
- ParticleEnergy

并要求：

- 场能不能无界增长
- 粒子总能量相对初值偏离不超过 2%

所以这组 regression 更准确的意义是：

- thermal particle boundary 在长时间粒子出入边界时的总量稳定性

最后，plasma_lens_python 复用和 native/PICMI plasma-lens 相同的 analysis.py，所以物理断言没有变化，仍然是两颗测试电子穿过 lens 序列后的：

- 最终横向位置
- 最终横向动量

与解析模型一致。

但它新增覆盖的不是物理，而是 front-end：输入不再走 native 文件或 PICMI，而是直接用 pywarpx 参数对象设置：

- particles.E_ext_particle_init_style = "repeated_plasma_lens"
- particles.B_ext_particle_init_style = "repeated_plasma_lens"
- particles.repeated_plasma_lens_* = ...
- 最后 warpx.init(); warpx.step(...)

因此它补上的验证边界是：

- 纯 Python 参数前端
- 到 MultiParticleContainer / GetExternalEBField repeated-plasma-lens 主链

这四组合起来之后，Particles 的边界与 Python 验证层就更清楚了：

- 一类是在测 scraped buffer 是否足够强，能支撑用户自己写 boundary physics
- 一类是在测 boundary kernel 自带的 absorbing / probabilistic reflection / thermalization 合同
- 一类是在测 pure Python front-end 是否能把参数正确接到既有粒子物理主链

所以它们都不该继续留在 general / to classify，也不该混成一个模糊的“边界条件测试”桶。

同一条 Python scraped-buffer 物理链还有一个更直接的边界物理 regression：secondary_ion_emission。它不是只拿 buffer 做统计，而是在 afterstep callback 里直接用

- r/theta/z
- ux/uy/uz
- nx/ny/nz
- deltaTimeScraped

为撞击球形 EB 的离子生成次级电子。analysis 再要求：

1. 固定随机种子下最终恰好产生 2 个电子；
2. 电子反向传播到撞击时刻后，应落在解析球面撞击点附近

所以这组 test 更准确地证明了: ParticleBoundaryBufferWrapper 提供的几何和时间元数据已经足够支撑真正的 callback 驱动边界二次发射物理, 而不只是后处理统计。

4.13.12 point_of_contact_eb、particles_in_pml、subcycling_mr 与 Langmuir multi_mr

再往外一层, 当前粒子相关 regression 里还有四类容易被混成“PML/EB/MR 杂项”的条目:

- point_of_contact_eb
- particles_in_pml
- subcycling_mr
- Langmuir multi_mr

但把 inputs、analysis 和 writer 路径对起来之后, 它们其实在测四种完全不同的合同。

point_of_contact_eb 的关键是它打开了两套 diagnostics:

- diag1 = Full
- diag2 = BoundaryScraping

analysis 根本不看主 species 的最终位置, 而是直接读:

diags/diag2/particles_at_eb/

也就是 BoundaryScraping 写出的 scraped-particle openPMD 输出。它比较的是:

- stepScraped
- deltaTimeScraped
- x/y/z
- nx/ny/nz

与解析接触点、接触时刻和表面法向是否一致。因此这组 regression 真正验证的是:

- EB 接触事件是否被正确记录到 particles_at_eb
- BoundaryScraping 输出里的几何量和时间量是否正确

这和前面的 particle_boundary_scrape 有本质区别。后者更侧重“粒子被删掉并进了 buffer”, 而 point_of_contact_eb 更侧重“记录下来的接触几何是否对”。

particles_in_pml 则不是看粒子轨道, 而是看粒子离场后留下的场。analysis 的核心只有一句:

```
assert max_Efield < tolerance_abs
```

它先取最后一步的 Ex/Ey/Ez, 再要求域内残余电场足够小。对应物理含义是:

- 如果 PML 只吸场不吸粒子, 离开的带电粒子会留下伪电荷和伪场
- 打开 warpx.pml_has_particles = 1 后, 这个 spurious field 必须被压到足够低

2D/3D 与 MR 版本共享同一 analysis, 只是 tolerance 按:

- 维度
- max_level

分开设置。因此 particles_in_pml 真正验证的是:

- particle-aware PML 的 residual-field cleanup

而不是单纯的 PML 场反射率。

subcycling_mr 又更弱一层。它当前没有独立 analysis, 只有 checksum。输入里同时打开了:

- warpx.do_subcycling = 1
- amr.max_level = 1
- moving window
- driver / beam / plasma continuous injection
- particles.deposit_on_main_grid = plasma_e plasma_p
- n_current_deposition_buffer = 0
- n_field_gather_buffer = 0

所以它当前最准确的定位只能是：

- AMR + subcycling + moving window + continuous injection + deposit_on_main_grid

这组粒子/网格组合的稳定性 checksum 基线。不能把它夸大成“已经有独立 refined-injection analysis”。

最后, Langmuir multi_mr 与 subcycling_mr 正好相反。它虽然也在测 MR, 但并不只是 checksum, 而是继续复用 analysis_2d.py 的强验证：

1. 取 E_x/E_z
2. 与解析 Langmuir-wave 场解比较
3. 在满足条件时, 再检查 $\text{div}E$ 与 ρ/ϵ_0 的相对误差

而不同 MR 变体测的是不同 refinement 组合：

- mr
 - amr.max_level = 1
 - amr.ref_ratio = 4
- mr_anisotropic
 - amr.ref_ratio_vect = 4 2
- mr_maxlevel2
 - amr.max_level = 2
- mr_momentum_conserving
 - gather scheme 改成 momentum-conserving
- mr_psatd
 - solver 改成 PSATD

因此这组条目的真正价值是：

- 在 mesh refinement 打开后, 继续用解析场解和 charge conservation 验证粒子-场链没有被粗网格破坏

把这四类并到一起后, 粒子验证层里一个此前容易混乱的区域就清楚了：

- point_of_contact_eb 测的是 scraped-contact geometry writer
- particles_in_pml 测的是 particle-aware PML 的残余场清理
- subcycling_mr 当前仍是组合稳定性的 checksum 基线
- Langmuir multi_mr 则是 MR 下仍保留强解析验证的真正基准

所以它们不该再笼统地记成 general / to classify、纯 PML, 或者过粗的 plasma oscillation 占位项。

4.13.13 余下 embedded_boundary、electrostatic_sphere_eb 与辅助绘图脚本的真实边界

还有一组条目此前虽然已经进入 example-regression-map.md, 但标签仍然过粗：

- particle_absorbing_boundary/plot_2d.py
- particle_absorbing_boundary/plot_phase.py
- embedded_boundary_cube
- embedded_boundary_rotated_cube
- embedded_boundary_diffraction
- embedded_boundary_em_particle_absorption
- embedded_boundary_python_api
- electrostatic_sphere_eb
- scraping

先说最简单的: plot_2d.py 和 plot_phase.py 都不是 regression analysis。

它们只是：

- 画 2D full diagnostics 的 E_z slice
- 画 PhaseSpaceElectrons reduced diagnostic 的相图

因此更准确的角色只是：

- visualization helper

而不是物理断言脚本。

真正带强 analysis 的 EB 条目可以再分四类。

第一类是 cavity 模态解析对照：

- embedded_boundary_cube
- embedded_boundary_rotated_cube

analysis_fields.py、analysis_fields_2d.py 和 analysis_fields_3d.py 都是显式构造 PEC cavity 的解析本征模，再比较 B_y 、 B_z 或 E_y/c 的相对 L2 误差。区别只是：

- cube: 轴对齐 cavity
- rotated_cube: 旋转后的 cavity, analysis 里要先把坐标和场分量反旋回解析坐标系
- cube_macroscopic: 同一模态，但频率按介质 epsilon_r 修正

所以这两组不该再笼统记成 boundary condition，而应理解成：

- embedded boundary / PEC cavity eigenmode
- embedded boundary / rotated PEC cavity eigenmode

第二类是几何散射与几何访问：

- embedded_boundary_diffraction
- embedded_boundary_python_api

embedded_boundary_diffraction/analysis_fields.py 读取 RZ 下的 E_x ，提取衍射图样第一极小值半径，再与 Airy pattern 的

$$\theta \sim 1.22\lambda/d$$

预测比较，因此它测的是：

- embedded boundary / diffraction / Airy first minimum

embedded_boundary_python_api 更特殊。CMake 里虽然 analysis = OFF，但 PICMI 输入脚本本身在运行时就会读取：

- edge_lengths
- face_areas

并在三个中间切片上重建 cavity 的 perimeter 和 area，再和解析几何值比较。所以它并不是单纯 checksum，而是：

- embedded boundary / PICMI wrapper / edge_lengths-face_areas

第三类是 EB 吸收与 scraping 合同：

- embedded_boundary_em_particle_absorption
- scraping

embedded_boundary_em_particle_absorption/analysis.py 做的不是看粒子是否消失，而是把 $\text{div}E$ 做时间平均，去掉沿 EB 传播的真实波动分量后，检查是否还残留静态伪电荷。因此它真正验证的是：

- embedded boundary / EM particle absorption / no spurious charge build-up

而 scraping/analysis_rz.py 与 analysis_rz_filter.py 测的是另一条 writer 合同：

- 最终剩余粒子数是否正确
- remaining + scraped = initial 是否逐步成立
- scraped buffer 中的 id 是否和初始全集闭合
- 打开 plot_filter_function 后，是否真的只记录 $z > 0$ 半域的 scraped particles

因此 scraping 更准确的分类是：

- embedded boundary / BoundaryScraping / particle accounting
- embedded boundary / BoundaryScraping / plot_filter_function

最后一类是 electrostatic_sphere_eb。这组其实至少分成三层：

1. 3D analysis.py
 - reduced diagnostic eb_charge.txt

- 理论球导体电荷

$$q = 4\pi\epsilon_0\phi_0 R$$

- eb_covered 场是否满足内 1 外 0

2. RZ analysis_rz.py

- 比较解析

$$\phi(r) = A + B \log r, \quad E_r(r) = -B/r$$

3. RZ analysis_rz_mr.py

- 把同一 ϕ/E_r 对照扩展到每个 refinement level

只有 inputs_test_3d_electrostatic_sphere_eb_mixed_bc 目前没有独立 analysis, 它更准确的角色只是:

- embedded boundary / electrostatic sphere / mixed-BC checksum baseline

因此这组条目现在至少可以从三种过粗标签里退出:

- boundary condition
- electrostatic / Poisson
- general / to classify

改写成更贴近真实断言对象的 EB、writer、wrapper 和解析场解分类。

4.14 QED: 不是一个统一开关, 而是三条不同的 product-species 事件链

从 Particles/ 入口再往下看, WarpX 当前的 QED 主链至少分成三种完全不同的事件类型:

1. Quantum Synchrotron: lepton source 产生 photon product
2. Breit-Wheeler: photon source 产生 electron/positron products
3. Schwinger: 不以 source species 为起点, 而是直接由场在网格上创建对

4.14.1 runtime attribute 和 product species 在构造期就锁定

PhysicalParticleContainer.cpp 在构造期先决定:

```
pp_species_name.query("do_qed_quantum_sync", m_do_qed_quantum_sync);
if (m_do_qed_quantum_sync) {
    AddRealComp("opticalDepthQSR");
}

pp_species_name.query("do_qed_breit_wheeler", m_do_qed_breit_wheeler);
if (m_do_qed_breit_wheeler) {
    AddRealComp("opticalDepthBW");
}
```

这不是小细节, 而是 QED 和前面 field ionization 一样, 都先把“事件统计状态”写进 species 的 runtime attribute 系统里。后面所有 QED 事件, 首先都依赖:

- opticalDepthQSR
- opticalDepthBW

这两个持久属性。

与之配套, PhotonParticleContainer.cpp 又明确禁止 photon species 再开 quantum synchrotron:

```
WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
    test_quantum_sync == 0,
    "ERROR: do_qed_quantum_sync can't be enabled for photon particles!");
```

也就是说, WarpX 在 species 层已经把“谁是 source、谁是 product”分得很清楚。

4.14.2 InitQED() 真正做的是 engine/table 装配, 不是事件执行

MultiParticleContainer::InitMultiPhysicsModules() 在 InitData()/PostRestart() 前先做:

```
mapSpeciesProduct();
CheckQEDProductSpecies();
InitQED();
```

这里三步分别对应:

1. 把 qed_quantum_sync_phot_product_species、qed_breit_wheeler_ele_product_species、qed_breit_wheeler_pos_pr 从字符串映射成容器索引;
2. 检查 product species 类型是否正确;
3. 创建 QuantumSynchrotronEngine / BreitWheelerEngine 并按 qed_qs.*、qed_bw.* 初始化 lookup tables。

因此 InitQED() 的语义不是“提前跑一遍 QED”, 而是把:

- 谁参与 quantum synchrotron
- 谁参与 Breit-Wheeler
- 表从 builtin/load/generate 哪条路径来

全部固定下来。

4.14.3 主循环里的插入顺序: field ionization 后, particle injection 前

顶层主循环 WarpXEvolve.cpp 里, 多物理顺序非常直接:

```
doFieldIonization();
```

```
#ifdef WARPX_QED
doQEEvents();
mypc->doQEDSchwinger();
#endif
```

```
ExecutePythonCallback("particleinjection");
OneStep(cur_time, dt[0], step);
```

这说明 QED 事件发生在:

- field ionization 之后
- 用户 particleinjection callback 之前
- 正常 OneStep() 之前

也就是说, QED 不是像 collisions 那样嵌在 split-momentum push 组织里, 而是更早地直接消费当下的 Efield_aux/Bfield_aux。

4.14.4 Quantum Synchrotron 与 Breit-Wheeler 都走 filterCopyTransformParticles

doQedQuantumSync() 的骨架是:

```
const auto Filter = phys_pc_ptr->getPhotonEmissionFilterFunc();
const auto CopyPhot = copy_factory_phot.getSmartCopy();

auto Transform = PhotonEmissionTransformFunc(
    m_shr_p_qs_engine->build_optical_depth_func(),
    pc_source->GetRealCompIndex("opticalDepthQSR") - pc_source->NArrayReal,
    m_shr_p_qs_engine->build_phot_em_func(),
    pti, lev, Ex.nGrowVect(), ...);

filterCopyTransformParticles<1>(<
    *pc_product_phot, dst_tile, src_tile, np_dst,
    Filter, CopyPhot, Transform);
```

doQedBreitWheeler() 则是双 product 版本:

```

const auto Filter = phys_pc_ptr->getPairGenerationFilterFunc();
const auto pair_gen_functor = m_shr_p_bw_engine->build_pair_functor();

auto Transform = PairGenerationTransformFunc(pair_gen_functor,
                                              pti, lev, Ex.nGrowVect(), ...);

filterCopyTransformParticles<1>(
    *pc_product_ele, *pc_product_pos,
    dst_ele_tile, dst_pos_tile, src_tile,
    np_dst_ele, np_dst_pos,
    Filter, CopyEle, CopyPos, Transform);

```

这两条链有一个很关键共同点：QED 不是只在 product species 上 `push_back` 一批新粒子，而是同时做三件事：

1. 用 source 粒子的 optical depth 判断是否触发事件
2. 在 Transform 里读取主网格场和 external particle fields
3. 同时更新 source 状态并创建 product particles

这和前面 field ionization 的 `filterCopyTransformParticles` 思路很接近，但它依赖的是 QED engine 表和 optical-depth 统计变量。

4.14.5 Schwinger 是第三条完全不同的创建路径

`doQEDSchwinger()` 不再从某个 source species 复制，而是直接在网格上用：

- `Efield_aux/Bfield_aux`
- Schwinger 激活区域
- `filterCreateTransformFromFAB<1>(...)`

生成 `ele_schwinger` 和 `pos_schwinger`。而且它目前硬性要求：

- collocated grid 或 momentum-conserving gather
- 无 mesh refinement
- 非 RZ
- 非 1D

所以这里不能把 Schwinger 和前两条 source->product 路径混写成同一类机制。

4.14.6 regression 的三组 strongest evidence

- `analysis_quantum_sync.py`:
 - 检查 photon 数、权重、发射方向、能谱、以及 source/product optical depth 分布；
 - 对应 Quantum Synchrotron 的主合同。
- `analysis_breit_wheeler_core.py`:
 - 检查 pairs 数、残余 photon 动量、单事件能量守恒、pair 能谱和 optical depth；
 - 对应 Breit-Wheeler 的 product-species 合同。
- `analysis_schwinger.py`:
 - 检查理论率对应的 pair 数窗口，以及电子/正电子权重数组一致性；
 - 对应 Schwinger 的强场真空对产生合同。

因此“QED regression”不是一组同质测试，而是三条不同物理路径的独立证据。

4.14.7 kernel 层真正的触发条件：先推进 optical depth，再跑 event pass

把入层再往下读到 `QEDPhotonEmission.H`、`QEDPairGeneration.H` 和两个 engine wrapper，会发现 QED 事件触发并不是“在 event pass 里直接抽一次随机数”。

`PhotonEmissionFilterFunc` 和 `PairGenerationFilterFunc` 都只做一件事：

```
return (opt_depth < 0.0_rt);
```

所以真正的统计演化发生在更早的 push 阶段：

- `PhysicalParticleContainer::PushPX()` 里先用 `QuantumSynchrotronEvolveOpticalDepth` 推进 `opticalDepthQSR`
- `PhotonParticleContainer::PushPX()` 里先用 `BreitWheelerEvolveOpticalDepth` 推进 `opticalDepthBW`

只有当 `optical depth` 在 `push` 中被推进到负值, 后面的:

- `doQedQuantumSync()`
- `doQedBreitWheeler()`

才会在 `event pass` 里通过 `filterCopyTransformParticles` 真正触发事件。

4.14.8 wrapper 的职责边界: WarpX 管场与属性, PICSAR-QED 管采样

`QuantumSyncEngineWrapper.H` 和 `BreitWheelerEngineWrapper.H` 不是又实现了一遍 QED 理论, 而是把 PICSAR-QED core 包成 WarpX 可在 GPU kernel 中调用的 functor:

- `QuantumSynchrotronEvolveOpticalDepth` 最终调用 `pxr_qs::evolve_optical_depth`
- `QuantumSynchrotronPhotonEmission` 最终调用 `pxr_qs::generate_photon_update_momentum`
- `BreitWheelerEvolveOpticalDepth` 最终调用 `pxr_bw::evolve_optical_depth`
- `BreitWheelerGeneratePairs` 最终调用 `pxr_bw::generate_breit_wheeler_pairs`

WarpX 在这一层真正负责的是:

1. gather 主网格与 external particle fields
2. 提供 source/product species 的 SoA 指针
3. 存取 `opticalDepthQSR/BW`
4. 组织 `filterCopyTransformParticles`

这条分工线非常重要, 因为它决定了后面再追 QED internals 时, 哪些属于 WarpX 容器/调度问题, 哪些其实已经落到 PICSAR-QED 的 table 和采样算法里了。

4.14.9 Quantum Synchrotron 与 Breit-Wheeler 的 source 语义不同

两条 event kernel 都会先 gather E/B, 但 source 处理方式并不一样:

- `PhotonEmissionTransformFunc` 会原地改写 source lepton 动量, 并把 source optical depth 重新抽样初始化;
- `PairGenerationTransformFunc` 会生成 electron/positron 两个 product, 并把 source photon 直接标记成 invalid。

这正好解释了为什么:

- `analysis_quantum_sync.py` 要重点检查 source/product optical-depth 分布能否继续保持指数;
- `analysis_breit_wheeler_core.py` 要重点检查 residual photons、丢失 photon 数和新 pairs 数之间的对应关系。

4.14.10 QED table 不是附属数据, 而是 kernel 可执行性的前提

如果继续往 `QEDInternals/QuantumSyncEngineWrapper.cpp` 和 `BreitWheelerEngineWrapper.cpp` 读, 会发现 WarpX 当前的 QED wrapper 内部都不是只持有一张表, 而是:

- 一张 `dndt` 表
- 一张真正用于事件采样的二维表
- 一个最小 `chi` 门槛

更具体地说:

- Quantum Synchrotron 持有 `m_dndt_table + m_phot_em_table + m_qs_minimum_chi_part`
- Breit-Wheeler 持有 `m_dndt_table + m_pair_prod_table + m_bw_minimum_chi_phot`

因此前面看到的:

- `build_optical_depth_functor()`
- `build_phot_em_functor()`
- `build_pair_functor()`

都会先要求 `m_lookup_tables_initialized == true`。这说明 WarpX 的 QED kernel 不是“有 wrapper 就能跑”, 而是必须先把表生命周期走完。

`InitQED()` 里 `qed_qs.lookup_table_mode` 和 `qed_bw.lookup_table_mode` 只允许三种模式:

- builtin
- load
- generate

其中：

- builtin 直接使用 wrapper .cpp 中硬编码的低分辨率测试表
- load 从外部二进制文件读 raw bytes，再反序列化化成两张子表
- generate 运行时现生成表，但最后仍然导出成同一种 raw binary 格式，并通过 MPI 广播给所有 rank

所以 generate 和 load 的真正共同点是：后续 kernel 消费的不是“生成态”或“加载态”的 C++ 对象，而是同一种序列化结果重新装配出来的 wrapper。

这一点在 QuantumSyncGenerateTable() / BreitWheelerGenerateTable() 里最清楚：

1. 只在 IOProcessor 上按 tab_dndt_*, tab_em_* 或 tab_pair_* 生成两张子表
2. 调 export_lookup_tables_data() 导出 raw bytes
3. 把 bytes 写入 save_table_in
4. 再把同一份 bytes 广播给所有 rank
5. 非 IOProcessor 用 init_lookup_tables_from_raw_data(...) 重建 wrapper

因此，完整主链应写成：

```
runtime attributes / product species
-> InitQED()
-> builtin/load/generate tables
-> build device functors
-> PushPX() 中演化 optical depth
-> event pass 中查表并生成 product particles
```

这也是为什么 examples 里的 lookup_table_mode = builtin / load / generate 注释块，不只是输入模板，而是直接切换 QED kernel 的上游可执行合同。

4.14.11 QedChiFunctions、virtual photons 与 linear_breit_wheeler 其实属于两棵不同的树

继续沿源码往下追，会碰到一组很容易被误写进同一章法的名字：

- QedChiFunctions
- do_qed_virtual_photons
- linear_breit_wheeler
- linear_compton

它们都带着 QED / photons 的标签，但并不共享同一套事件骨架。

QedChiFunctions.H 本身只有两个薄包装：

- QedUtils::chi_ele_pos(...)
- QedUtils::chi_photon(...)

它们只把 SI 单位下的动量与 E/B 场交给 PICSAR 的 chi 公式。当前源码里，这两个函数主要只服务三类地方：

1. PushSelector.H 里 RR 与 QED 联动时的 chi 阈值判断
2. QuantumSyncEngineWrapper 的 optical-depth 与 photon-emission 采样
3. BreitWheelerEngineWrapper 的 optical-depth 与 pair-generation 采样

所以 QedChiFunctions 属于前面已经解释过的强场 QED 树：

```
gather E/B
-> compute chi
-> evolve optical depth
-> lookup-table sampling
-> update source and create products
```

但 virtual photons 走的不是这条路。

PhysicalParticleContainer.cpp 允许 lepton species 打开：

- do_qed_virtual_photons
- qed_virtual_photon_species_name
- qed_virtual_photons_do_beam_size_effect

随后 `CollisionHandler::doCollisions()` 在真正做任何碰撞之前, 会统一先调用:

```
collision::binarycollision::virtualphotons::GenerateVirtualPhotons(mypc);
```

这说明 virtual photons 的运行时位置在 collision 调度层, 而不是:

- InitQED()
- doQedEvents()
- QEDPhotonEmission.H
- QEDPairGeneration.H

`GenerateVirtualPhotons()` 的语义是: 每个 coarse step 都从 lepton species 重新采样一批辅助 photon species; 旧 virtual photons 会被下一步覆盖。3D 下若打开 beam-size effect, 还会在垂直于动量的平面内给这些虚光子加上有限半径位移。

因此 virtual photons 不是强场 QED 事件生成出来、随后继续 push 的 product photons, 而是碰撞模块临时重建的一份辅助 photon 分布。

接下来的 linear_breit_wheeler 与 linear_compton 也继续留在碰撞树里。

`CollisionHandler.cpp` 对 `type = linear_breit_wheeler` 的分派是:

```
std::make_unique<
    BinaryCollision<LinearBreitWheelerCollisionFunc, ParticleCreationFunc>
>(...)
```

这说明它不走前面的:

- doQedBreitWheeler()
- PairGenerationFilterFunc
- BreitWheelerEngineWrapper
- opticalDepthBW

而是走另一套碰撞骨架:

```
optional GenerateVirtualPhotons()
-> BinaryCollision pairing in each cell
-> p_mask / reaction weight
-> ParticleCreationFunc
```

`LinearBreitWheelerCollisionFunc` 与 `LinearComptonCollisionFunc` 自己也写得很清楚: 它们实现的是 Higginson 风格的 binary-collision 算法, 控制参数是:

- event_multiplier
- probability_threshold
- probability_target_value

而不是强场 QED 那套:

- chi_min
- lookup_table_mode
- photon_creation_energy_threshold

因此, WarpX 当前至少有两棵名字都带 QED / photons 的树:

1. 强场 QED / ElementaryProcess
 - QedChiFunctions
 - optical depth
 - tables
 - filterCopyTransformParticles
2. 碰撞 QED / BinaryCollision
 - optional virtual photons
 - cell-local pairing
 - reaction masks

- ParticleCreationFunc

这也是为什么 regression 也分成两组完全不同的证据:

- analysis_quantum_sync.py、analysis_breit_wheeler_core.py、analysis_schwinger.py
– 验证强场 QED 主链
- analysis_virtual_photons.py、analysis_beamsize_effect.py、analysis_many_photons.py
– 验证 virtual-photon 采样与 linear Breit-Wheeler 碰撞分叉

如果不把这两棵树拆开, 后面读 BinaryCollision 或继续追 QedChiFunctions 时就会把不同层次的多物理路径混写。

4.15 本章结论

粒子推进器不等于孤立的 Boris 公式。WarpX 的实际路径是:

```
flowchart TD
    A["WarpX::PushParticlesandDeposit"] --> B["MultiParticleContainer::Evolve"]
    B --> C["for each species"]
    C --> D["PhysicalParticleContainer::Evolve"]
    D --> E["tile loop"]
    E --> F["rho component 0 before push"]
    E --> G["PushPX"]
    G --> H["doGatherShapeN"]
    G --> I["doParticleMomentumPush"]
    I --> J["Boris / Vay / Higuera-Cary / RR"]
    G --> K["UpdatePosition"]
    E --> L["current deposition"]
    E --> M["rho component 1 after push"]
```

后续继续深入时, 应分别追踪三个子问题: doGatherShapeN() 的形函数插值、doParticleMomentumPush() 的具体 pusher 公式、DepositCurrent/DepositCharge() 的守恒沉积算法。

4.16 练习与复现实验

1. **pusher 对照题:** 用 scripts/compare_particle_pusher_siblings.py 读取 Boris/Vay/Higuera-Cary sibling 报告, 解释为什么 Boris 的大位移结果不能被简单归结为“代码运行失败”, 而应联系 force-free relativistic pusher contract 判断。
2. **源码定位题:** 从 PhysicalParticleContainer::Evolve() 定位 doGatherShapeN()、momentum push、UpdatePosition 和 current/charge deposition, 画出一粒粒子 tile loop 的四个时间层节点。
3. **最小复现实验:** 运行官方 particle_pusher 或 photon_pusher analysis, 并同时记录官方 analysis 与项目独立合同脚本的输出; 说明 charged pusher 的位置误差和 massless photon 的位置/动量误差为何不能共用同一容差。

5. 电荷、电流沉积与形函数: 源项如何回到网格

上一章从粒子侧解释了 field gather 和 pusher。本章看反方向: 粒子推进后如何把电荷和电流交回网格。沉积不是输出或后处理, 而是 PIC 离散方程的一部分。它直接决定离散连续性方程、Gauss 定律误差、数值噪声、guard cell 需求和 AMR fine/coarse 同步方式。

本章对应源码笔记见 notes/code-reading/particles/00-particle-evolve-callchain.md、notes/code-reading/particles/01-particle-evolve-current-deposition-callchain.md 和 notes/code-reading/particles/02-gather-shape-deposition-kernels.md。

本章当前依据的 WarpX 源码版本是:

- ../warpx
- 分支: pkuHEDPbranch
- commit: 8c488b1a9

本章当前按同级 ../warpx checkout 逐段复核了 ShapeFactors.H、WarpXParticleContainer.cpp、PhysicalParticleContainer.cpp 与 CurrentDeposition.H 的沉积主链。组织上分成三层: ShapeFactors.H 定义 0-4 阶形函数, WarpXParticleContainer::DepositCurrent() / DepositCharge() 负责 tile 级分派与桥接合同, Particles/Deposition/CurrentDeposition.H 承载 Direct、Esirkepov、Villasenor 和 Vay 的实际 current kernel。验证证据以 Examples/Tests/langmuir/analysis_utils.py 和 Examples/Tests/vay_deposition/analysis.py 的 divE-rho/epsilon_0 检查为主。相应 primary deposition 文献目录与 access audit 已建立; 其中 Esirkepov 2001 已通过作者 arXiv 预印本完成第一轮 MinerU 与中文讲解, Villasenor-Buneman 1992

也已从本机现成 PDF/MinerU 资产 materialize 到项目目录并补出第一轮中文讲解。因此本章现在不再只是单侧的 Esirkepov paper-backed 论证，而是两条 charge-conserving 路径都已经开始拥有第一手论文支撑；剩余工作主要转成把这两篇论文系统地回写成更出版化的正文。

Birdsall-Langdon 在 Plasma Physics via Computer Simulation 第一分卷的 4-6 到 4-8 给了一个很硬的理论边界：只要粒子通过空间网格被观测和求场，它就不再表现成零厚度 point particle，而必须被理解成具有有效形状因子 $S(\mathbf{x})$ 、频域响应 $S(\mathbf{k})$ 的 finite-size cloud。这样一来，shape order 不是单纯“更光滑的插值公式”，而是同时改写三件事：

1. 粒子如何把 ρ/J 交回网格；
2. 网格场如何再被 gather 回粒子；
3. 粒子间短程相互作用怎样被平滑，以及 grid force / aliasing 从哪里进入离散系统。

因此本章不能把 shape、deposition 和 finite-grid effects 分开讲。对 WarpX 来说，ShapeFactors.H、charge/current deposition kernel、AMR coarse-fine buffer 和后续的 current correction 共同实现的，正是这条 Birdsall 已经提前写清的离散合同。

Birdsall-Langdon 在 Chapter 8 又把这条线再往前推了一步：aliasing 不是“把结果做 FFT 之后才看见的谱污染”，而是在

$$\rho(k) = q \sum_p S(k_p) n(k_p), \quad k_p = k - pk_g$$

这一步就已经发生。也就是说，particle continuum information 被 sample 到 grid 上时，不同 aliases 已经混进同一个 $\rho(\mathbf{k})$ 。这正是为什么本章既要讲 shape factor，也必须讲 finite-grid effects 和 sampled density 的合同；否则只讨论 ShapeFactors.H 的局部公式，会把真正的 alias source 讲窄。

Birdsall 到 Chapter 10 又把这条线往前压了一步：energy-conserving 和 momentum-conserving 不是同一套沉积/受力合同上“谁更准一点”的实现差别，而是两条不同的离散守恒路线。前者把离散场能量

$$W_E = \frac{V_c}{2} \sum_j \rho_j \phi_j$$

当成第一性对象，再从 $-\partial W_E / \partial \mathbf{x}_i$ 构造粒子受力；后者则保持更常见的 grid force / zero-total-force 结构。因此本章后面讨论 ShapeFactors.H、charge/current deposition 和 sampled density 时，必须把它们同时视作“守恒合同”的一部分，而不是孤立的插值技术细节。

Birdsall 在 Chapter 13 又把这条 shape-factor 主线往“长期数值健康度”推进了一步：对 thermal plasma，weighting order 与 short-wavelength smoothing 不只是决定瞬时噪声有多平滑，还会直接改写 self-heating time τ_H 。一维结果和 Hockney 的 2d2v 长时间实验都说明：

- 更高阶 particle shape 会更强地削弱 alias coupling；
- 更激进的高波数截断会进一步拉长 τ_H ；
- 但 collisional slowing-down time τ_s 未必同步等比例变化。

所以本章讨论 shape order 时，不能只写“更高阶更光滑、噪声更低”。更准确的说法是：shape order、cloud width 和 smoothing policy 一起决定了热等离子体多久会因为 finite-grid effects 累积出不可忽略的数值自热。

Dawson 1983 则把同一条线往前退回到更基础的动机层：finite-size particles 的第一性目的不是“插值更方便”，而是先把 point-charge 的近距离大冲量软化掉，从而压低不想要的 collisional effects，同时保住长程 Coulomb collective behavior。也正因为粒子已经被改写成有限尺寸 cloud，空间上比 cloud 更细的电荷起伏本来就不再分辨，grid 才成为一种自然的 coarse-grained source representation。这条综述级表述很适合放在本章开头，因为它比直接从 ShapeFactors.H 展开更清楚地交代了：shape、charge sharing 和 sampled density 本来就是同一个物理建模决定的三个侧面。

5.1 电荷沉积的基本形式

对宏粒子 p ，电荷、权重和位置分别为 $q_p, w_p, \mathbf{x}_p$ 。最基本的网格电荷沉积是

$$\rho_i = \frac{1}{\Delta V_i} \sum_p q_p w_p S_i(\mathbf{x}_p).$$

这里 S_i 是粒子形函数对网格自由度 i 的权重。形函数阶数越高，粒子影响的网格范围越大，噪声通常越低，但 stencil、guard cell 和通信成本也更高。

WarpX 中 shape 阶数通过 nox/noy/noz 等内部变量进入 gather 和 deposition 分派。当前源码里，0-4 阶权重的唯一基础定义在 `../warpX/Source/Particles/ShapeFactors.H:27-156`；current deposition 再在 `../warpX/Source/Particles/WarpXParticleC`

根据 WarpX::nox 与 CurrentDepositionAlgo 选择 doEsirkepovDepositionShapeN<N>()、doVillasenorDepositionShapeN*<N>()、doVayDepositionShapeN<N>() 或 direct doDepositionShapeN<N>()。因此读者应把 nox/noy/noz 看成“shape order 的全局分派键”，而不是某一个 kernel 的局部参数。

5.2 ShapeFactors.H: WarpX 实际使用的 0 到 4 阶形函数

形函数不是抽象参数。WarpX 在 ../warpX/Source/Particles/ShapeFactors.H:27-84 直接给出 0 到 4 阶的权重和最左网格点索引：

```
template <int depos_order>
struct Compute_shape_factor
{
    template< typename T >
    AMREX_GPU_HOST_DEVICE AMREX_FORCE_INLINE
    int operator()(
        T* const sx,
        T xmid) const
    {
        if constexpr (depos_order == 0){
            const auto j = static_cast<int>(xmid + T(0.5));
            sx[0] = T(1.0);
            return j;
        }
        else if constexpr (depos_order == 1){
            const auto j = static_cast<int>(xmid);
            const T xint = xmid - T(j);
            sx[0] = T(1.0) - xint;
            sx[1] = xint;
            return j;
        }
        else if constexpr (depos_order == 2){
            const auto j = static_cast<int>(xmid + T(0.5));
            const T xint = xmid - T(j);
            sx[0] = T(0.5)*(T(0.5) - xint)*(T(0.5) - xint);
            sx[1] = T(0.75) - xint*xint;
            sx[2] = T(0.5)*(T(0.5) + xint)*(T(0.5) + xint);
            // index of the leftmost cell where particle deposits
            return j-1;
        }
        else if constexpr (depos_order == 3){
            const auto j = static_cast<int>(xmid);
            const T xint = xmid - T(j);
            sx[0] = (T(1.0))/(T(6.0))*(T(1.0) - xint)*(T(1.0) - xint)*(T(1.0) - xint);
            sx[1] = (T(2.0))/(T(3.0)) - xint*xint*(T(1.0) - xint/(T(2.0)));
            sx[2] = (T(2.0))/(T(3.0)) - (T(1.0) - xint)*(T(1.0) - xint)*(T(1.0) - T(0.5)*(T(1.0) - xint));
            sx[3] = (T(1.0))/(T(6.0))*xint*xint*xint;
            // index of the leftmost cell where particle deposits
            return j-1;
        }
        else if constexpr (depos_order == 4){
            const auto j = static_cast<int>(xmid + T(0.5));
            const T xint = xmid - T(j);
            sx[0] = (T(1.0))/(T(24.0))*(T(0.5) - xint)*(T(0.5) - xint)*(T(0.5) - xint)*(T(0.5) - xint);
            sx[1] = (T(1.0))/(T(24.0))*(T(4.75) - T(11.0)*xint + T(4.0)*xint*xint*(T(1.5) + xint - xint*xint));
            sx[2] = (T(1.0))/(T(24.0))*(T(14.375) + T(6.0)*xint*xint*(xint*xint - T(2.5)));
            sx[3] = (T(1.0))/(T(24.0))*(T(4.75) + T(11.0)*xint + T(4.0)*xint*xint*(T(1.5) - xint - xint*xint));
            sx[4] = (T(1.0))/(T(24.0))*(T(0.5) + xint)*(T(0.5) + xint)*(T(0.5) + xint)*(T(0.5) + xint);
        }
    }
};
```



```

        // index of the leftmost cell where particle deposits
        return j-2;
    }
    else{
        WARPPX_ABORT_WITH_MESSAGE("Unknown particle shape selected in Compute_shape_factor");
        amrex::ignore_unused(sx, xmid);
    }
    return 0;
}
};

```

对一阶, 若 $x_{\text{mid}} = j + \xi$, $0 \leq \xi < 1$, 源码就是

$$S_0 = 1 - \xi, \quad S_1 = \xi.$$

对二阶, 源码先把最近节点/中心取成 $j = \text{int}(x_{\text{mid}} + 0.5)$, 再使用

$$S_0 = \frac{1}{2} \left(\frac{1}{2} - \xi \right)^2, \quad S_1 = \frac{3}{4} - \xi^2, \quad S_2 = \frac{1}{2} \left(\frac{1}{2} + \xi \right)^2,$$

并返回 $j-1$ 作为 stencil 左端。三阶和四阶同样是 B-spline 形函数的展开式。源码里 0/2/4 阶用 $x_{\text{mid}}+0.5$ 找中心, 1/3 阶用 x_{mid} 找左端, 这是因为偶数阶和奇数阶 shape 的自然支撑中心不同。

Esirkepov 沉积还需要把旧位置的 shape 写进与新位置对齐的数组。对应源码在 `../warpx/Source/Particles/ShapeFactors.H:93-156`:

```

template <int depos_order>
struct Compute_shifted_shape_factor
{
    template< typename T >
    AMREX_GPU_HOST_DEVICE AMREX_FORCE_INLINE
    int operator()(
        T* const sx,
        const T x_old,
        const int i_new) const
    {
        if constexpr (depos_order == 0){
            const auto i = static_cast<int>(std::floor(x_old + T(0.5)));
            const int i_shift = i - i_new;
            sx[1+i_shift] = T(1.0);
            return i;
        }
        else if constexpr (depos_order == 1){
            const auto i = static_cast<int>(std::floor(x_old));
            const int i_shift = i - i_new;
            const T xint = x_old - T(i);
            sx[1+i_shift] = T(1.0) - xint;
            sx[2+i_shift] = xint;
            return i;
        }
        else if constexpr (depos_order == 2){
            const auto i = static_cast<int>(x_old + T(0.5));
            const int i_shift = i - (i_new + 1);
            const T xint = x_old - T(i);
            sx[1+i_shift] = T(0.5)*(T(0.5) - xint)*(T(0.5) - xint);
            sx[2+i_shift] = T(0.75) - xint*xint;
            sx[3+i_shift] = T(0.5)*(T(0.5) + xint)*(T(0.5) + xint);
            // index of the leftmost cell where particle deposits

```

```

        return i - 1;
    }
    else if constexpr (depos_order == 3){
        const auto i = static_cast<int>(x_old);
        const int i_shift = i - (i_new + 1);
        const T xint = x_old - T(i);
        sx[1+i_shift] = (T(1.0))/(T(6.0))*(T(1.0) - xint)*(T(1.0) - xint)*(T(1.0) - xint);
        sx[2+i_shift] = (T(2.0))/(T(3.0)) - xint*xint*(T(1.0) - xint/(T(2.0)));
        sx[3+i_shift] = (T(2.0))/(T(3.0)) - (T(1.0) - xint)*(T(1.0) - xint)*(T(1.0) - T(0.5)*(T(1.0) -
        sx[4+i_shift] = (T(1.0))/(T(6.0))*xint*xint*xint;
        // index of the leftmost cell where particle deposits
        return i - 1;
    }
    else if constexpr (depos_order == 4){
        const auto i = static_cast<int>(x_old + T(0.5));
        const int i_shift = i - (i_new + 2);
        const T xint = x_old - T(i);
        sx[1+i_shift] = (T(1.0))/(T(24.0))*(T(0.5) - xint)*(T(0.5) - xint)*(T(0.5) - xint)*(T(0.5) - x
        sx[2+i_shift] = (T(1.0))/(T(24.0))*(T(4.75) - T(11.0)*xint + T(4.0)*xint*xint*(T(1.5) + xint -
        sx[3+i_shift] = (T(1.0))/(T(24.0))*(T(14.375) + T(6.0)*xint*xint*(xint*xint - T(2.5)));
        sx[4+i_shift] = (T(1.0))/(T(24.0))*(T(4.75) + T(11.0)*xint + T(4.0)*xint*xint*(T(1.5) - xint -
        sx[5+i_shift] = (T(1.0))/(T(24.0))*(T(0.5) + xint)*(T(0.5) + xint)*(T(0.5) + xint)*(T(0.5)+xin
        // index of the leftmost cell where particle deposits
        return i - 2;
    }
    else{
        WARPX_ABORT_WITH_MESSAGE("Unknown particle shape selected in Compute_shifted_shape_factor");
        amrex::ignore_unused(sx, x_old, i_new);
    }
    return 0;
}
};

```

这里的 `i_shift` 是 Esirkepov 的关键工程细节：旧位置和新位置可能跨过 cell 边界，不能把两个 shape 数组各自放在自己的左端后直接相减。WarpX 把旧 shape 平移到以 `i_new` 为参考的数组里，后面才能逐项计算 `sx_old[i] - sx_new[i]`。

再往下一层看，`ShapeFactors.H` 里这三个 functor 其实对应三种不同的离散合同，而不只是“都能算一组权重”。

1. `Compute_shape_factor`
 - 给出单时间层的 shape
 - 同时返回当前 stencil 的最左写入点；
2. `Compute_shifted_shape_factor`
 - 不把 old shape 独立排到自己的左端
 - 而是把它平移进“以 new shape 为参考”的数组框架；
3. `Compute_shape_factor_pair`
 - 给同一 Villasenor segment 的横向 old/new 权重提供共同 leftmost index。

对第 5 章来说，这条区分很重要，因为后面 Esirkepov 与 Villasenor 看起来都在“算 old/new shape”，但它们真正需要的不是同一类 helper：前者需要 old/new difference 的同框对齐，后者需要 segment-local 横向共同支撑。

这里还有一条容易忽略的数值实现边界：`ShapeFactors.H` 文件头已经说明，current deposition 中这些 functor 可以用 `double` 参数求值，以避免粒子一步只移动很短距离时，单精度把 old/new 差分结构磨掉。`CurrentDeposition.H` 的 implicit Esirkepov 里也确实把 `x_new/x_old` 与 `sx_new/sx_old` 明确声明成 `double`。因此这里的 `double` 不是泛泛的“更高精度更好”，而是离散守恒实现的一部分。

5.3 电流沉积的守恒要求

电荷沉积只看某一时间层的粒子位置。电流沉积必须看粒子在时间步内穿过网格的轨迹。离散电磁 PIC 希望满足

$$\frac{\rho_i^{n+1} - \rho_i^n}{\Delta t} + (\nabla_h \cdot \mathbf{J}^{n+1/2})_i = 0.$$

如果这个式子不成立，Maxwell solver 即使形式上正确，离散 Gauss 定律也会漂移：

$$\nabla_h \cdot \mathbf{E}^{n+1} - \frac{\rho^{n+1}}{\epsilon_0} \neq 0.$$

这解释了为什么 WarpX 需要 Esirkepov、Villasenor、Vay、Direct 等多种 current deposition。Direct deposition 直观但自动保证电荷守恒；Esirkepov 和 Villasenor 属于 charge-conserving 路径；Vay deposition 与 PSATD/current correction 等算法组合有关。

把这个合同再往前压一步，可以直接写成单粒子 shape difference：

$$\rho_i^n = \frac{1}{\Delta V_i} \sum_p q_p w_p S_i(x_p^n),$$

因此一步中的净电荷变化本质上就是

$$\rho_i^{n+1} - \rho_i^n = \frac{q_p w_p}{\Delta V_i} (S_i(x_p^{n+1}) - S_i(x_p^n)).$$

charge-conserving deposition 真正要做的，不是“再估一个差不多的 $\mathbf{J}$ ”，而是构造某个离散电流，使得

$$\frac{\rho_i^{n+1} - \rho_i^n}{\Delta t} = -(\nabla_h \cdot \mathbf{J}^{n+1/2})_i.$$

这给后面的算法分叉一个更稳定的读法：

- **Esirkepov**: 围绕 old/new shape difference 直接构造守恒电流；
- **Villasenor**: 把轨迹按 cell crossing 切 segment，再让每段局部输运共同满足同一离散守恒；
- **Direct**: 直接写 $\mathbf{q} \cdot \mathbf{w} / \Delta V$ ，所以不自动满足这个合同；
- **Vay**: 属于显式-only 的两阶段 D-field 重组算法，离散守恒不是通过 Esirkepov/Villasenor 这种单阶段 charge-conserving kernel 来实现。

5.3.1 五条路径的责任边界总览

为了避免把“沉积到网格”误读成一个单一 kernel，先把本章涉及的五类路径放在同一张表中。表中的“第一性对象”指该路径在局部循环里真正组织和累加的数学对象；“外层职责”则指它不能单独承担、必须由容器或同步层完成的工作。

路径	第一性对象	是否以离散连续性为设计目标	一步轨迹如何处理	WarpX 当前实现	主要边界
Direct current	$\mathbf{q} \cdot \mathbf{w}$ 与单点 shape 权重	否	不恢复守恒轨迹；按当前速度和 shape 直接写入 $\mathbf{J}$	doDepositionShape 和 CurrentDeposition 两个 kernel	不能保证 $\text{div}_h \mathbf{J}$ 与 $\rho^{(n+1)} - \rho^n$ 一致
Esirkepov current	old/new shape difference 的方向分解与 prefix accumulation	是	使用 old/new 端点；跨 cell 时先做 shifted-shape 对齐	doEsirkepovDeposition 和 sdx/sdy/sdzk	需要 <code>inShapeN<1..4>()</code> , shape、轨迹端点和足够 stencil；implicit 前端的 suborbit 不是原文的原样内容

路径	第一性对象	是否以离散连续性为设计目标	一步轨迹如何处理	WarpX 当前实现锚点	主要边界
Villasenor current	crossing-driven segment 的局部 boundary flux	是	按最早 cell crossing 反复切分 segment, 再逐段写回 <code>this_J*</code>	<code>doVillasenorDeposition</code> <code>cell_crossings</code> <code>/</code> <code>num_segments</code>	需要调用局部函数 <code>doVillasenorCurrent()</code> , crossing 几何; 不能把它简化成一次 old/new shape 差分
Vay current	两阶段 D-field 沉积与局部重组	由专用组合路径承担	显式路径中按 Vay 专用的 D-field 组织, 不走 Esirkepov/Villasenor 单阶段结构	<code>doVayDeposition</code> 与 <code>current_fp_vay</code>	<code>ShapeOnlyVayCurrent()</code> Cartesian-only、非 shared-memory; 最终 source consistency 还依赖 PSATD/current-correction 同步链
<code>DepositCharge()</code>	单时间层 <code>rho</code> 的 shape-weighted sample	不是 current mover	由外层 <code>relative_time</code> 取样和内层 <code>icomp/time_shift_delta</code> 选择时间层参考位置	<code>WarpXParticleContainer::DepositCharge()</code> -> ABLASTR / <code>ChargeDeposition</code> <code>diff</code>	不计算电荷: <code>DepositCharge()</code> old/new shape difference、不选择 current algorithm; guard、PEC、volume scaling、AMR 整理在外层完成

这张表的用法是：先问当前代码在累加哪一种“第一性对象”，再问守恒属性由哪一层承担。例如，Esirkepov 的 `sdxi/sdyj/sdzk` 属于 current kernel 内部的方向分解；`DepositCharge()` 的 `icomp` 却只是时间层和网格参考原点的桥接参数，不能据此推断它正在执行 implicit current deposition。类似地，Vay 的 `current_fp_vay` 不是 Esirkepov 电流的另一种 shape 写法，而是专门的两阶段目标字段。

从软件职责看，这五条路径还共享一个更高层的收口：局部 kernel 只负责把粒子信息写入某个 tile-local 或 field component；`SumBoundary`、fine/coarse source synchronization、PEC source-boundary、inverse-volume scaling、filter 和最终 field-solver handoff 不应被反向归因给局部 shape loop。后文的 5.8、5.9 和 5.13 分别展开 charge bridge、charge kernel 与沉积后同步，正是因为这三层合同不能压缩成同一个“沉积算法”名词。

5.4 WarpX 的旧电荷、新电荷和半步电流

`PhysicalParticleContainer::Evolve()` 在 `../warpx/Source/Particles/PhysicalParticleContainer.cpp:457-831` 中组织单 species 的沉积顺序。

关键源码节选如下，来自同一个 tile loop；这里省略了 buffer/coarse gather 和隐式 suborbit 的长分支，但保留 push 前电荷、粒子推进、半步电流和 push 后电荷的原始调用形态：

```

if (deposit_charge) {
    // Deposit charge before particle push, in component 0 of MultiFab rho.
    const int* const AMREX_RESTRICT ion_lev = (do_field_ionization)?
        pti.GetiAttribs("ionizationLevel").dataPtr(): nullptr;

    amrex::MultiFab* rho = fields.get(FieldType::rho_fp, lev);
    DepositCharge(pti, wp, ion_lev, rho, 0, 0,
        np_to_deposit, thread_num, lev, lev);
}

if (! do_not_push)
{
    if (push_type == PushType::Explicit) {
        PushPX(pti, exfab, eyfab, ezfab,

```

```

        bxfab, byfab, bzfab,
        Ex.nGrowVect(), e_is_nodal,
        0, np_to_push, lev, gather_lev, dt,
        ScaleFields(false), subcycling_half,
        position_push_type, momentum_push_type);
    }

    // Current Deposition
    if (deposit_current)
    {
        // Deposit at  $t_{n+1/2}$  with explicit push
        const amrex::Real relative_time = (push_type == PushType::Explicit ? -0.5_rt * dt : 0.0_rt);

        amrex::MultiFab * jx = fields.get(current_fp_string, Direction{0}, lev);
        amrex::MultiFab * jy = fields.get(current_fp_string, Direction{1}, lev);
        amrex::MultiFab * jz = fields.get(current_fp_string, Direction{2}, lev);
        DepositCurrent(pti, wp, uxp, uyp, uzp, ion_lev, jx, jy, jz,
            0, np_to_deposit, thread_num,
            lev, lev, dt, relative_time, push_type);
    }
}

if (deposit_charge) {
    // Deposit charge after particle push, in component 1 of MultiFab rho.
    // (Skipped for electrostatic solver, as this may lead to out-of-bounds)
    if (WarpX::electrostatic_solver_id == ElectrostaticSolverAlgo::None) {
        amrex::MultiFab* rho = fields.get(FieldType::rho_fp, lev);
        DepositCharge(pti, wp, ion_lev, rho, 1, 0,
            np_to_deposit, thread_num, lev, lev);
    }
}
}

```

行号	动作	时间层解释
:585-598	push 前沉积 rho component 0	旧电荷, 通常是 ρ^n 。
:619-623, :675-682	调用 PushPX() 推进粒子	$\mathbf{x}^n, \mathbf{u}^{n-1/2} \rightarrow \mathbf{x}^{n+1}, \mathbf{u}^{n+1/2}$ 。
:703-738	push 后沉积 current	显式路径 $\text{relative_time}=-0.5*dt$, 对应 $\mathbf{J}^{n+1/2}$ 。
:791-808	push 后沉积 rho component 1	新电荷, 通常是 ρ^{n+1} 。

为什么电流在 push 后沉积还要 $\text{relative_time}=-0.5*dt$? 因为粒子位置已经是 $\mathbf{x}^{n+1}$, 而电流应位于半步 $n + 1/2$ 。WarpX 在 DepositCurrent() 的注释中说明: relative_time 非零时会临时修改粒子位置以匹配沉积时间, 见 `../warpX/Source/Particles/WarpXParticle`

这也是读源码时必须区分“粒子当前数组中的位置”和“沉积物理时间层”的原因。

5.5 多物种层如何清零和汇总源项

`../warpX/Source/Particles/MultiParticleContainer.cpp:478-522` 是多物种粒子推进口。

若不跳过沉积, MultiParticleContainer::Evolve() 先把本 level 的当前步源项清零:

- current_fp 三个方向: :489-491;
- current_buf 三个方向: :492-494;
- rho_fp: :495;
- rho_buf: :496。

然后在 :520-522 遍历 allcontainers, 每个 species 各自沉积到同一组源项数组中。也就是说, 最终的 ρ 和 $\mathbf{J}$ 是所有物种贡献之和。

独立调用的 DepositCurrent() 和 DepositCharge() 也有类似结构:

- `MultiParticleContainer::DepositCurrent()` 位于 `../warpX/Source/Particles/MultiParticleContainer.cpp:586-604` 先清零多层 J , 再逐 species 调 `pc->DepositCurrent()`, RZ/RCYLINDER/RSPHERE 下随后做 inverse-volume scaling。
- `MultiParticleContainer::DepositCharge()` 位于 `../warpX/Source/Particles/MultiParticleContainer.cpp:614-644` 先清零 ρ , 若 `relative_time != 0` 则临时 `PushX(relative_time)`, 逐 species 沉积后再推回; 在 RZ / RCYLINDER / RSPHERE 下, 最后还会对整张 rho 做 `ApplyInverseVolumeScalingToChargeDensity(...)`。

这些函数主要服务于 PSATD-JRhom、多时间层 charge/current、静电场或诊断等场景。

这里还需要把两层时间语义拆开, 否则很容易把后面的 `charge deposition` 读错。`MultiParticleContainer::DepositCharge(relative_time)` 这层 `PushX(relative_time)` 做的是外层粒子位置平移: 它先把粒子整体挪到需要诊断或沉积的物理时刻, 沉积完成后再推回。后面 `WarpXParticleContainer::DepositCharge(..., icomp, ...)` 里的 `time_shift_delta` 做的则是内层 Galilean 网格参考框架校正: 即使粒子已经在正确时间层上, `xyzmin = LowerCorner(tilebox, depos_lev, time_shift_delta)` 仍可能因为 `icomp=0/1` 而对应不同的移动网格坐标原点。这两层都与“时间”有关, 但一个改的是粒子位置数组本身, 另一个改的是沉积 kernel 所看到的 tile 几何参考系。

5.6 AMR coarse-fine interface: 粒子怎样切到 aux/cax/buf 路径

前面几章已经讲过 AMR 的 substitution 公式

$$F(a) = F(r) + I[F(s) - F(c)]$$

以及 `UpdateAuxiliaryData*`() 在 `WarpX` 里的实现

$$\text{aux}(\ell) = \text{fp}(\ell) + I[\text{aux}(\ell - 1) - \text{cp}(\ell)].$$

但只知道这条公式, 还不知道粒子在 coarse-fine transition zone 到底怎样用它。真正把这层逻辑接到粒子 kernel 上的是 `PhysicalParticleContainer::Evolve()`。

它不会在每个 gather/deposition kernel 内实时查 coarse-fine mask, 而是先做一次粒子重排:

```
long nfine_deposit = np;
long nfine_gather = np;
if (has_buffer && !do_not_push) {
    PartitionParticlesInBuffers( nfine_deposit, nfine_gather, np,
                                pti, lev, WarpX::n_field_gather_buffer,
                                WarpX::n_current_deposition_buffer, current_masks, gather_masks );
}
```

源码位置: `../warpX/Source/Particles/PhysicalParticleContainer.cpp:568-580`。

这一步之后:

- 前 `nfine_gather` 个粒子继续从 fine patch gather;
- 后 `np-nfine_gather` 个粒子改从 lower refinement level gather;
- 前 `nfine_deposit` 个粒子继续沉积到 fine patch;
- 后 `np-nfine_deposit` 个粒子改沉积到 lower refinement level buffer。

接下来的 gather 分成两段。先看 fine interior 粒子:

```
const auto np_to_push = np_gather;
const auto gather_lev = lev;
PushPX(pti, exfab, eyfab, ezfab,
        bxfab, byfab, bzfab,
        Ex.nGrowVect(), e_is_nodal,
        0, np_to_push, lev, gather_lev, dt, ...);
```

源码位置: `../warpX/Source/Particles/PhysicalParticleContainer.cpp:617-623`。

这里的 `exfab/eyfab/...` 来自 `Efield_aux/Bfield_aux`, 也就是已经做完 substitution 的 full solution。

随后是 transition-zone 粒子:

```

amrex::MultiFab & cEx = *fields.get(FieldType::Efield_cax, Direction{0}, lev);
...
PushPX(pti, cexfab, ceyfab, cezfab,
       cbxfab, cbyfab, cbzfab,
       cEx.nGrowVect(), e_is_nodal,
       nfine_gather, np-nfine_gather,
       lev, lev-1, dt, ...);

```

源码位置: ../warpx/Source/Particles/PhysicalParticleContainer.cpp:643-682。

这里不再使用 fine-level aux, 而是使用 coarse-aux 副本 E/Bfield_cax, 并且把 gather_lev 显式设为 lev-1。

因此, transition-zone 的粒子不是“先从 fine patch gather 再做后处理修正”, 而是一开始就改用 lower-level full solution。

沉积也完全平行地拆成两段:

```

DepositCurrent(... jx, jy, jz,
               0, np_to_deposit, thread_num,
               lev, lev, dt, relative_time, push_type);
...
DepositCurrent(... cjx, cjy, cjz,
               np_to_deposit, np-np_to_deposit, thread_num,
               lev, lev-1, dt, relative_time, push_type);

```

以及

```

DepositCharge(... rho, 0, 0,
              np_to_deposit, thread_num, lev, lev);
...
DepositCharge(... crho, 0, np_to_deposit,
              np-np_to_deposit, thread_num, lev, lev-1);

```

源码位置: ../warpx/Source/Particles/PhysicalParticleContainer.cpp:591-598, 712-738, 802-808。

这里:

- current_fp/rho_fp 接收 fine interior 粒子;
- current_buf/rho_buf 接收 transition-zone 粒子;
- depos_lev = lev-1 时, 不是“先沉积在 fine 上再 restrict”, 而是一开始就在 coarse buffer patch 的几何上沉积。

这一点在 WarpXParticleContainer::DepositCurrent() 和 DepositCharge() 的 tilebox 处理里写得非常直接:

```

if (lev == depos_lev) {
    tilebox = pti.tilebox();
} else {
    const IntVect& ref_ratio = WarpX::RefRatio(depos_lev);
    tilebox = amrex::coarsen(pti.tilebox(), ref_ratio);
}

```

源码位置: ../warpx/Source/Particles/WarpXParticleContainer.cpp:465-471, 1763-1769。

也就是说, buffer deposition 真正变化的是:

- offset
- np_to_deposit
- depos_lev
- 以及由此确定的 tilebox、dinv、xyzmin

但电流/电荷沉积算法本身并没有为 AMR 单独再写一套。无论是 Esirkepov、Villasenor、Vay 还是 Direct, WarpX 都是在“同一个 deposition 数学 + 不同 level 的几何解释”上复用。

所以, AMR coarse-fine interface 在粒子层的闭环可以概括为:

1. UpdateAuxiliaryData*() 先构造 fine patch 的 aux;
2. BuildBufferMasks*() 给出 transition-zone 的 gather/current masks;

3. PartitionParticlesInBuffers() 先重排粒子;
4. fine interior 粒子走 aux + fp 路径;
5. transition-zone 粒子走 cax + buf 路径;
6. 最后再由 SyncCurrent() / SyncRho() 把 coarse-fine source 合并回通信链。

5.7 WarpXParticleContainer::DepositCurrent() 分派

tile 级 current deposition 在 ../warpx/Source/Particles/WarpXParticleContainer.cpp:392-900。

入口先做安全检查和局部数组准备:

行号	操作
:401-409	检查 deposition level, 只处理非空粒子且 do_not_deposit 为假。
:411-446	取得 ng_J, 检查粒子 shape 是否放得进 tile/guard cells。
:448-520	准备沉积 level 的 cell size、tilebox、field array 和边界 cropping。
:546-550	Esirkepov/Villasenor 不能用于 collocated grid。

随后按沉积算法分派。这里真正重要的不是把 ShapeN<1..4> 四套模板参数全部重复抄一遍, 而是看清分派的逻辑骨架: Esirkepov 在 explicit/implicit 下分别进入 doEsirkepovDepositionShapeN<N>() 与 doChargeConservingDepositionShapeNImplicit<N>(), Villasenor 在 explicit/implicit 下分别进入 doVillasenorDepositionShapeNExplicit<N>() 与 doVillasenorDepositionShapeNImplicit<N>(), Vay 只允许 explicit, 而 Direct 则保留 explicit/implicit 两条非守恒路径。源码位置统一在 ../warpx/Source/Particles/WarpXParticleContainer.cpp。

源码位置	算法
:556-650	shared-memory current deposition, 只支持 direct; Esirkepov、Villasenor、Vay 会 abort。
:654-695	explicit Esirkepov, 调用 doEsirkepovDepositionShapeN<N>()。
:696-751	implicit charge-conserving deposition。
:752-835	Villasenor explicit/implicit deposition。
:836-864	Vay deposition; 隐式路径直接 abort。
:865-900	Direct deposition explicit/implicit 分支开头。

这个分派地图只是入口。下面继续进入 Source/Particles/Deposition/ChargeDeposition.H 和 CurrentDeposition.H 的 kernel, 把 shape 权重、电荷归一化、direct current、Esirkepov 守恒电流, 以及 Villasenor/Vay/implicit 的时间层与几何边界逐块展开。正文保留主叙述, 更细的 runtime/geometry 合同留在配套 notes。

如果把 WarpXParticleContainer::DepositCurrent() 只理解成“按算法名切 kernel”, 会漏掉调用层真正更强的合同。源码在进入 normal-path 分派之前, 先固定了四层边界:

1. depos_lev 只能是 lev 或 lev-1, 也就是 current buffer 只允许 coarse 一层;
2. 当前维度下的 shape_extent 必须落在 tile/guard-cell 允许范围内, 否则直接触发 numParticlesOutOfRange(...) == 0 断言;
3. Esirkepov / Villasenor 只要配上 collocated grid 就在入口 abort, 而不会等到 kernel 内部再失败;
4. 若打开 do_shared_mem_current_deposition, 则整条路径先转成 DenseBins + shared_tilesize + max_tbox_size 的 tile-binned performance contract, 并且这条合同当前只接受 explicit direct deposition, implicit / Esirkepov / Villasenor / Vay 都在入口直接 abort。

因此 WarpX 当前的 dispatch 次序其实是:

- 先判断这批粒子在当前 tile/guard-cell 几何上是否允许沉积;
- 再决定是否走 shared-memory performance path;
- 只有在 normal path 里, 才继续分成 Direct / Esirkepov / Villasenor / Vay 四个 kernel 家族。

这里还有一个容易讲混的细节: domain_double 与 do_cropping 虽然在入口统一构造, 但并不会发给所有算法。它们只会继续传给 Esirkepov 和 Villasenor 这两条 charge-conserving 路径; Direct 与 Vay 即使运行在同一 tile 上, 也只拿到时间层、dinv 和 xyzmin 这组几何

缩放信息，而拿不到可裁剪轨迹合同。也就是说，DepositCurrent() 自己就已经把 near-boundary/charge-conserving 的接口能力划给了特定算法家族，而不是留给 kernel 临时自选。

如果只看 AMR coarse-fine buffer，这几种算法的差异可以压缩成“共享同一套 coarse patch 几何，但恢复粒子轨迹的方式不同”。

首先，进入 current_buf 的粒子和进入 current_fp 的粒子，在接口层共享同样的沉积壳：

```
if (lev == depos_lev) {
    tilebox = pti.tilebox();
} else {
    const IntVect& ref_ratio = WarpX::RefRatio(depos_lev);
    tilebox = amrex::coarsen(pti.tilebox(), ref_ratio);
}
tilebox.grow(ng_J);
const amrex::XDim3 dinv = WarpX::InvCellSize(std::max(depos_lev, 0));
const amrex::XDim3 xyzmin = WarpX::LowerCorner(tilebox, depos_lev, 0.5_rt*dt);
```

源码位置：../warpx/Source/Particles/WarpXParticleContainer.cpp:465-480,520。

因此，AMR buffer 本身只改变：

- depos_lev
- tilebox
- dinv
- xyzmin
- 以及后续 domain_double / do_cropping

并不会为 coarse-fine interface 再定义另一套 current deposition 数学。

真正的分界线是各算法如何恢复 old/new/mid 轨迹：

- **Vay**: 显式-only; 接口层直接禁止 implicit。
- **Villasenor explicit**: 使用当前粒子位置、relative_time 和 dt 回推 x_old/x_new。
- **Villasenor implicit**: 不再用 relative_time, 改为显式使用 x_n、u_n、u_{n+1/2} 恢复轨迹。
- **Esirkepov implicit**: 时间层输入与 implicit Villasenor 相似，但后续仍走 old/new shape 差分的守恒电流构造。

例如显式 Villasenor 会先写：

```
amrex::Real const xp_new = xp + (relative_time + 0.5_rt*dt)*uxp[ip]*gaminv;
amrex::Real const xp_old = xp_new - dt*uxp[ip]*gaminv;
```

源码位置：../warpx/Source/Particles/Deposition/CurrentDeposition.H:2236-2237。

而隐式 Villasenor / Esirkepov 则改为：

```
amrex::ParticleReal const xp_np1 = 2._prt*xp_nph - xp_n;
```

源码位置：

- Villasenor implicit: ../warpx/Source/Particles/Deposition/CurrentDeposition.H:2347
- Esirkepov implicit: ../warpx/Source/Particles/Deposition/CurrentDeposition.H:1171

所以，AMR coarse-fine buffer 下 current deposition 的最小结论是：

1. coarse-fine 只决定粒子在哪个 level 的几何上沉积；
2. 时间层恢复方式仍由 current deposition 算法自身决定；
3. 也正因为如此，current_buf 可以统一复用 Villasenor、Vay、Esirkepov、Direct 的现有 kernel，而不需要 AMR 专用变体。

这里还要补一条论文边界，否则很容易把 WarpX 今天的 implicit 路径误读成两篇经典论文的“原样实现”。无论是 Esirkepov 2001 还是 Villasenor-Buneman 1992，原始论文讨论的核心都还是显式 PIC 的守恒电流构造：前者围绕 old/new shape difference 的唯一线性分解，后者围绕 crossing-driven local flux mover。WarpX 当前 explicit 分支仍直接继承这两条主干；但 implicit 分支已经额外引入了 x_n、u_n、u_{n+1/2}、suborbit endpoint reconstruction 和 matching gather 兼容性这些现代工程语义。也就是说：

- **论文原始结构**: 守恒电流该怎样由一条 one-step orbit 构造出来；
- **WarpX implicit 扩展**: 当 orbit 本身来自隐式推进、suborbit fallback 和 near-boundary gather 约束时，怎样先把那条 orbit 恢复出来，再送回原来的守恒沉积骨架。

对 Villasenor 来说,这里还要再往下压一层:doVillasenorDepositionShapeNExplicit(...) 和 doVillasenorDepositionShapeNImplicit(...) 自己并不实现 segment loop。两条入口在完成各自的 endpoint reconstruction 之后,都会把

- old/new 轨迹端点
- wq
- 中间时间层动量 uxp_mid/uyy_mid/uzp_mid
- gaminv
- domain_double/do_cropping

统一交给同一个 VillasenorDepositionShapeNKernel(...)。也就是说,explicit/implicit 这条分界回答的是“端点怎么恢复”,而不是“segment 数学怎么改写”;真正的 crossing 统计、segment 推进、cell/node 权重构造和局部 this_J* 写回都属于共享后端。

如果继续往 kernel 本体再走一步,就会发现 Villasenor 和 Esirkepov 的 charge-conserving 结构并不只是“名字不同”,而是两种不同的守恒组织方式:

- **Esirkepov**: 先把整条轨迹压成同框的 old/new shape difference,再沿沉积方向做前缀累加;
- **Villasenor**: 先按真实 cell crossing 切出多个局部 segment,再对每个 segment 分别沉积。

这也是为什么源码注释会说 Villasenor “results in a tighter stencil”: 它的支持域围绕真实 crossing segment 局部组织,而不是像 Esirkepov 那样由整条 old/new difference support 一次性决定。更细的 3D 循环、cell/node 权重和 this_J* 写回细节,放到后面的 Esirkepov current deposition 小节再展开。

更关键的是,Villasenor 的“segment-by-segment”不只是数学风格差异,而是它的接口本身已经把边界裁剪写死了。VillasenorDepositionShapeNKernel 直接接收:

- xp_old/xp_new
- domain_double
- do_cropping

进入 kernel 后,第一批动作就是 ParticleUtils::crop_at_boundary(...),然后才统计 cell_crossings、得到 num_segments、逐段恢复 x0_old -> x0_new 并沉积。源码见 ../warpx/Source/Particles/Deposition/CurrentDeposition.H:1499-1775。因此第 5 章不应把 Villasenor 理解成“另一种 charge-conserving 公式”,而应把它看成:

1. 以完整轨迹端点为输入;
2. 允许在 PEC/PECInsulator 邻近几何上先裁剪轨迹;
3. 再把剩余轨迹按 crossing 切成局部 segment 的沉积路线。

这条合同与 direct 的“单个时间中心位置 + 速度加权沉积”完全不是同一层次。

这也是 ImplicitPushPX.cpp 在 suborbit fallback 里直接强制改成 Villasenor 的原因。源码注释写得很直白:为了 energy conservation, suborbit push 必须使用 matching gather,因此这里会覆盖 runtime-selected deposition type,强制改成 CurrentDepositionAlgo::Villasenor。见 ../warpx/Source/Particles/Pusher/ImplicitPushPX.cpp:735-738。换句话说,WarpX 在这里需要的不是“任意一个能沉 J 的 kernel”,而是:

- 与 implicit gather stencil 配套;
- 保留 boundary crop + segment decomposition;
- 在 near-boundary/suborbit 情形下仍维持局部守恒几何的那一条沉积合同。

因此在 implicit/suborbit 一侧,WarpX 真正需要的不是“任意一个能沉 J 的 kernel”,而是保留 boundary crop、segment decomposition 和 matching gather 兼容性的那条沉积合同。

对 Esirkepov 也应作同样辨析。doChargeConservingDepositionShapeNImplicit<N>() 当然仍保留了论文那条 old/new shape-difference 守恒主线,但它前面多出来的 x_n \to x_{n+1} 恢复、几何分支坐标改写、double 精度 shape functor 以及 implicit gather 配套语义,都属于 WarpX 在原始论文主干之外加上的工程前端。换句话说,读第 5 章时应把

1. “守恒电流如何由 old/new shape difference 或 segment flux 构造出来”,和
2. “隐式推进下怎样先把这条轨迹恢复成可沉积对象”

严格分成两层:前一层是论文主结果,后一层是现代代码为把论文主结果接进更复杂时间推进框架而增加的实现层。

还有一条算法要单独区分出来:Vay deposition。它和 Direct、Esirkepov、Villasenor 的差别,不只是“权重系数不同”,而是整个执行拓扑都不同。

CurrentDeposition.H 的注释写得很直接:

deposit D in real space and store the result in Dx_fab, Dy_fab, Dz_fab

源码位置: `../warpx/Source/Particles/Deposition/CurrentDeposition.H:2361-2363`。

这说明 Vay 路径的第一目标不是直接形成普通意义上的 $J_x/J_y/J_z$, 而是先沉积一组 D 量。对应地, 它一开始就会额外分配一个 temporary FAB:

```
#if defined(WARPX_DIM_3D)
amrex::FArrayBox temp_fab{Dx_fab.box(), 4};
#elif defined(WARPX_DIM_XZ)
amrex::FArrayBox temp_fab{Dx_fab.box(), 2};
#endif
temp_fab.setVal<amrex::RunOn::Device>(0._rt);
```

源码位置: `../warpx/Source/Particles/Deposition/CurrentDeposition.H:2432-2440`。

也就是说, Vay deposition 不是单阶段沉积, 而是两阶段:

1. 粒子 loop 先写 temp_arr 中间量;
2. 再由 box 级 ParallelFor 把它们重组为三个方向。

例如 3D 的第二阶段就是:

```
const amrex::Real t_a = temp_arr(i,j,k,0);
const amrex::Real t_b = temp_arr(i,j,k,1);
const amrex::Real t_c = temp_arr(i,j,k,2);
const amrex::Real t_d = temp_arr(i,j,k,3);
Dx_arr(i,j,k) += (1._rt/6._rt)*(2_rt*t_a      + t_b      + t_c - 2._rt*t_d);
Dy_arr(i,j,k) += (1._rt/6._rt)*(2_rt*t_a      + t_b - 2._rt*t_c      + t_d);
Dz_arr(i,j,k) += (1._rt/6._rt)*(2_rt*t_a - 2._rt*t_b      + t_c      + t_d);
```

源码位置: `../warpx/Source/Particles/Deposition/CurrentDeposition.H:2649-2657`。

这正是它和 Esirkepov / Villasenor 的根本区别:

- **Esirkepov / Villasenor**: 粒子 loop 内直接形成 charge-conserving J;
- **Vay**: 粒子 loop 只形成 D 的中间组合量, 真正的三个方向要靠第二阶段线性重组。

同时, Vay 还有一组清晰的实现边界:

- 不支持 implicit;
- 不支持 RZ;
- 不支持 1D / RCYLINDER / RSPHERE;
- 不支持 shared-memory current deposition。

这组边界不是上层文档约定, 而是 kernel 内部直接 abort:

```
#if defined(WARPX_DIM_RZ)
    WARPX_ABORT_WITH_MESSAGE("Vay deposition not implemented in RZ geometry");
#endif

#if defined(WARPX_DIM_1D_Z) || defined(WARPX_DIM_RCYLINDER) || defined(WARPX_DIM_RSPHERE)
    WARPX_ABORT_WITH_MESSAGE("Vay deposition not implemented in 1D geometry");
#endif
```

源码位置: `../warpx/Source/Particles/Deposition/CurrentDeposition.H:2406-2417`。

与此相对, implicit charge-conserving 和 Villasenor 路径反而把几何差异显式展开了。比如 implicit charge-conserving 里直接分成:

- RZ / RCYLINDER
 - 从 (x,y) 恢复半径 r
 - 再用 costheta/sintheta 重建分量
- RSPHERE
 - 再进一步恢复 r, \theta, \phi
- 1D_Z
 - 空间支撑只剩 z
 - 但横向速度分量仍可能进入 current 分量的几何解释

源码原文如下，位置为 ../warpx/Source/Particles/Deposition/CurrentDeposition.H:1191-1261:

```
#if defined(WARPX_DIM_RZ) || defined(WARPX_DIM_RCYLINDER)
...
const amrex::Real costheta_mid = (rp_mid > 0._rt ? xp_mid/rp_mid : 1._rt);
const amrex::Real sintheta_mid = (rp_mid > 0._rt ? yp_mid/rp_mid : 0._rt);
#elif defined(WARPX_DIM_RSPHERE)
...
const amrex::Real cosphi_mid = (rp_mid > 0. ? rpxy_mid/rp_mid : 1._rt);
const amrex::Real sinphi_mid = (rp_mid > 0. ? zp_mid/rp_mid : 0._rt);
#elif defined(WARPX_DIM_1D_Z)
amrex::Real const vx = uxp_nph[ip]*gaminv;
amrex::Real const vy = uyp_nph[ip]*gaminv;
#endif
```

但这段几何恢复还不能和后面的 shape factor 分开看。implicit Esirkepov 的真实链路是：

1. 从 x_n 与 $x_{n+1/2}$ 恢复 x_{n+1} ;
2. 按 RZ / RCYLINDER / RSPHERE / 1D_Z 各自的坐标语义把位置与速度改写到沉积所需变量;
3. 必要时先做 crop_at_boundary(...);
4. 再用 Compute_shape_factor 与 Compute_shifted_shape_factor 在这个几何分支下生成对齐后的 old/new stencil.

也就是说，shape factor 在这里不是几何恢复之后“顺手再算的权重”，而是这条 implicit 几何链的最后离散化步骤。若把这两层拆开写，就会把 ShapeFactors.H 误解成和几何无关的通用插值库。

再进一步，1D_Z / RCYLINDER / RSPHERE 的几何分支还直接改写了 Jx/Jy/Jz 三个分量各自的写回合同，而不只是“先恢复什么坐标”。例如 implicit Esirkepov 在 1D_Z 下写成：

- Jx/Jy
 - 用 $0.5*(sz_{old} + sz_{new})$ 的 old/new 平均写回;
- Jz
 - 用 $sz_{old} - sz_{new}$ 的前缀累加承担唯一空间方向上的守恒输运。

而在 RCYLINDER / RSPHERE 下则反过来是：

- Jx
 - 实际承担径向主输运角色，走 $sx_{old} - sx_{new}$ 的守恒差分;
- Jy/Jz
 - 作为切向分量，只走 $0.5*(sx_{old} + sx_{new})$ 的横向平均写回。

对应地，Villasenor 在这两组几何里也保持同样的物理分工，只是把“守恒主分量”的写回改成 segment-local 的 cell weights，而把另外两个分量继续放在 node-average 支撑上。也就是说，几何分支真正改变的是“哪个分量承担连续性方程的主差分，哪个只沿横向平均写回”。

RZ 还要再多看一层：m=0 的 Jr/Jz 确实沿 XZ 主干继续用 $sx_{old}-sx_{new}$ 、 $sz_{old}-sz_{new}$ 的守恒差分，但 m>0 并不是简单把 mode-0 的三个分量统一乘上一个相位。源码在 ../warpx/Source/Particles/Deposition/CurrentDeposition.H:1000-1038 里把

- Jr 模态写成 $djr_{cplx} = 2 * sdx_i * xy_{mid}$,
- Jz 模态写成 $djz_{cplx} = 2 * sdz_k * xy_{mid}$,
- Jtheta 模态则单独写成包含 $xy_{new} / xy_{mid} / xy_{old}$ 、1/imode 和 Davidson 符号约定修正的 djt_{cplx} 。

因此更准确的说法是：RZ 的 mode-0 径向/轴向守恒结构与 XZ 同构，但 m>0 特别是 Jtheta 有自己独立的复模态重建合同，不能概括成“XZ 再做一次 Fourier 复制”。

另外，CurrentDeposition.H 的 kernel 写回对 RZ / RCYLINDER / RSPHERE 还不是最终物理电流密度。MultiParticleContainer::DepositC 在所有 species 沉积之后，会额外调用 WarpX::ApplyInverseVolumeScalingToCurrentDensity(...) (../warpx/Source/Particles/MultiParticleContainer.cpp:1400-1586，源码注释单 species 路径也有同样调用)。对应实现位于 ../warpx/Source/FieldSolver/WarpXPushFieldsEM.cpp:1400-1586，源码注释直接写明“the inverse volume factor was not included in the current deposition”。它做的事情包括：

- 先把轴附近负半径 guard cell 的沉积 wrap 回到轴上方;
- 对 RZ / RCYLINDER 的非轴点按 $2*\pi*r$ 缩放;
- 对 RSPHERE 的非轴点按 $4*\pi*r^2$ 型球壳体积因子缩放;
- 在轴上把 Jr/Jtheta 强制置零，而 Jz 则按 Verboncoeur 修正体积因子 axis_volume_factor 走专门处理。

这意味着柱/球对称几何里真正供场求解器消费的 J ，不是 kernel 原子加的直接输出，而是“守恒写回 + inverse-volume scaling”两层合同共同定义的结果。

因此第 5 章里更稳定的组织方式不能只按算法名字排目录，还必须同时保留：

1. 离散连续性合同；
2. Direct / Esirkepov / Villasenor / Vay 的实现差异；
3. implicit / RZ / 1D_Z / RCYLINDER / RSPHERE 的时间层与几何边界。

对正文来说，最重要的结论不是再重复列一遍全部 #if，而是明确：

1. Direct deposition 不自动保证

$$\frac{\rho^{n+1} - \rho^n}{\Delta t} + \nabla_h \cdot J = 0$$

2. Esirkepov / Villasenor 的第一性目标都是让这条离散连续性合同成立；
3. implicit 路线改写的是轨迹端点恢复方式，不是守恒目标本身；
4. Vay deposition 是显式-only，且当前只在 3D/XZ 这一组笛卡尔路径上有实现。

因此，Vay 应该被看作一个显式、笛卡尔、两阶段重组的专用沉积路径，而不是一般 current deposition kernel 的简单变种。

5.8 WarpXParticleContainer::DepositCharge() 入口

上面讲的是 Vay deposition 的实现拓扑；本地 regression 里还有一条更直接的验证入口：Examples/Tests/vay_deposition/。这组测试不是拿解析单粒子轨道去对照，而是只看经过一小段推进后，最终 full diagnostics 里是否仍满足

$$\frac{\max |\nabla \cdot E - \rho/\epsilon_0|}{\max |\rho/\epsilon_0|} < 10^{-3}.$$

也就是说，它真正测的是：

- algo.current_deposition = vay
- algo.maxwell_solver = psatd
- warpx.grid_type = collocated

这组 Vay 专用实现边界下，D-field 两阶段重组后的离散电荷守恒是否仍成立。它给了一个比 Langmuir 家族更窄、更直接的 Vay deposition 自证入口。

和它互补的另一组 regression 是 Examples/Tests/langmuir/ 里的 PSATD current-correction 变体。那组测试不是只看 divE-rho/\epsilonpsilon_0，而是两层断言一起做：

1. 先把 Ex/Ey/Ez 或 Ex/Ez 与解析 Langmuir-wave 场解比较；
2. 再由 analysis_utils.py 在特定组合下追加 divE-rho/\epsilonpsilon_0 检查。

更关键的是，这个 helper 明确写死了适用边界：

- current_correction
 - 始终检查，容差 1e-9
- current_deposition = vay
 - 始终检查，容差 1e-3
- current_deposition = esirkepov
 - 只在非 RZ 且非 PSATD 时检查

因此，Langmuir + current_correction 和 vay_deposition 两组 regression 的角色并不相同：

- Langmuir + current_correction
 - 是解析场解 + source consistency 的组合验证；
 - 典型输入还会显式打开 psatd.current_correction = 1 与 psatd.periodic_single_box_fft = 1。
- vay_deposition
 - 是更窄的 PSATD + collocated + Vay current deposition source-synchronization 验证；
 - 只断言离散 Gauss law，不再做解析波对照。

这正好也对应上一节的 SyncCurrentAndRho() 分叉：

- current-correction 变体对应 PSATD + periodic single box 下仍立即同步的那条路径；

- Vay 变体对应非 periodic-single-box 下 `current_fp_vay` 单独过滤、再交给后续 PSATD 同步链的那条专门路径。

tile 级 charge deposition 入口在 `../warpX/Source/Particles/WarpXParticleContainer.cpp:1502-1790`。它现在需要拆成两段读：1502-1607 是 component 检查、shared-memory 前置检查、tilebox、xyzmin 与 `time_shift_delta`；1713-1788 才是真正的 charge kernel 分派。

行号	操作
:1508-1512	检查 rho component 数量是否足够。
:1520-1525	shared-memory 路径下做非空粒子检查并取得 <code>ng_rho</code> 。
:1527-1556	shared-memory 路径下检查粒子 shape 与 guard cells。
:1558-1600	取得 species 电荷、profiling scope、tilebox 和 GPU/CPU 本地 <code>rho_fab</code> 。
:1602-1612	根据 <code>icomp</code> 计算 <code>time_shift_delta</code> ，再确定 xyzmin 和 <code>dinv</code> 。
:1713-1737	shared-memory charge deposition，根据 <code>WarpX::nox</code> 调用 <code>doChargeDepositionSharedShapeN<1..4>()</code> 。
:1744-1788	普通 charge deposition，重新建立 <code>ng_rho/tilebox/xyzmin</code> 后委托 <code>ablastr::particles::deposit_charge(...)</code> 。

`time_shift_delta` 对理解 rho component 很关键：`icomp==0` 表示旧时间层；`icomp==1` 表示新时间层。它和 `PhysicalParticleContainer::Evolve` 中 push 前/后两次 charge deposition 对应。

这里有一个比 current deposition 更容易讲混的分叉：`DepositCharge()` 不像 current deposition 那样在这个文件里直接按 `Esirkepov/Villasenor/Vay/Direct` 多算法展开。共享内存路径显式调用 `doChargeDepositionSharedShapeN<1..4>()`；普通路径则先交给 `ablastr::particles::deposit_charge(...)`，再由 ABLASTR 桥接到 `doChargeDepositionShapeN<1..4>()`。也就是说，charge deposition 的“主差别”首先不是算法名字，而是：

1. shared-memory 还是普通路径；
2. 旧/新时间层的 rho component；
3. CPU thread-local 暂存还是 GPU 直接 alias 目标 rho。

这一点和 current deposition 很不一样。current deposition 的主入口主要负责算法选择；而普通 charge deposition 的主入口更像是在整理 bridge contract，把 `depos_lev/ref_ratio/icomp/xyzmin`、guard-cell 检查和 CPU/GPU 暂存策略都布置好，再统一交给 `ChargeDeposition.H`。

5.8.1 icomp 不只是分量号，而是旧/新 rho 的时间层合同

`WarpXParticleContainer::DepositCharge()` 的注释写得很明确：

- `icomp = 0`
— 沉旧值, before particle push
- `icomp = 1`
— 沉新值, after particle push

而源码真正把这个时间层差异落到几何上的地方是：

```
const amrex::Real dt = warpx.getdt(lev);
const amrex::Real time_shift_delta = (icomp == 0 ? 0.0_rt : dt);
const amrex::XDim3 xyzmin = WarpX::LowerCorner(tilebox, depos_lev, time_shift_delta);
```

源码位置：`../warpX/Source/Particles/WarpXParticleContainer.cpp:1605-1607,1775-1776`。

也就是说，旧/新 rho component 的区别不只是 `MultiFab` 里“写第几块分量”，还会改变用于沉积的 tile 物理左下角参考时间层。后面的 shape kernel 本身不再判断“现在沉的是旧电荷还是新电荷”，因为 xyzmin 在桥接层已经按时间层对齐好了。

如果再把 `WarpX::LowerCorner(...)` 展开一层，这个 `time_shift_delta` 的真实作用会更清楚。`../warpX/Source/WarpX.cpp:3220-3238` 中，

```
const amrex::Real cur_time = warpx.gett_new(lev);
const amrex::Real time_shift = (cur_time + time_shift_delta - warpx.time_of_last_gal_shift);
amrex::Array<amrex::Real,3> galilean_shift = { warpx.m_v_galilean[0]*time_shift,
```



```
warpx.m_v_galilean[1]*time_shift,
warpx.m_v_galilean[2]*time_shift };
```

然后 `LowerCorner(...)` 才把这份 `galilean_shift` 加到 `grid_min` 上。换句话说, `icomp=0/1` 并不是简单地在注释里区分“old/new rho”, 而是真的通过 `time_shift_delta` 改写了 moving-window / Galilean 坐标下沉积 kernel 所看到的 tile 原点。对成书叙述来说, 这一点比“分量号不同”重要得多, 因为它说明 charge deposition 的旧/新时间层差异已经被压进了几何参考框架本身。

5.8.2 普通 charge deposition 的桥接合同在 ABLASTR, 而不在 kernel 本体

普通路径的桥接在 `../warpx/Source/ablastr/particles/DepositCharge.H:50-203`。它做了四件真正影响正文理解的事:

1. 把运行时 shape 阶数压成统一的 `particle_shape`, 因此调用点先 `assert WarpX::nox == WarpX::noy == WarpX::noz`。
2. 检查 `depos_lev` 只能是 `lev` 或 `lev-1`, 把 coarse-fine buffer 限定在这一级别差上。
3. 再做一次 `numParticlesOutOfRange(pti, range) == 0` 的 guard-cell 安全检查, 也就是 charge deposition 的 tile/guard 合法性并没有完全下沉到 `ChargeDeposition.H`。
4. 在 CPU/GPU 上切换不同的暂存策略:
 - GPU: `rho_fab` 直接 alias 到真实 `rho`
 - CPU: 先沉到 `local_rho`, 再 `lockAdd(...)` 回去

对应源码如下:

```
#ifdef AMREX_USE_GPU
amrex::MultiFab rhoi(*rho, amrex::make_alias, icomp*nc, nc);
auto & rho_fab = rhoi.get(pti);
#else
local_rho.resize(tb, nc);
local_rho.setVal(0.0);
auto & rho_fab = local_rho;
#endif
...
(*rho)[pti].lockAdd(local_rho, tb, tb, 0, icomp*nc, nc);
```

源码位置: `../warpx/Source/ablastr/particles/DepositCharge.H:157-170,200-202`。

这里也因此可以把外层和内层两套时间合同并排摆清:

1. `MultiParticleContainer::DepositCharge(relative_time)`
 - 通过 `PushX(relative_time)` 临时改写粒子位置数组;
 - 主要服务于“要在当前数组时间层之外取样”的场景;
2. `WarpXParticleContainer::DepositCharge(..., icomp, ...)`
 - 通过 `time_shift_delta` 改写 `LowerCorner(tilebox, ...)` 的 Galilean 参考原点;
 - 主要服务于“同一歩里 old/new rho component 各自该落在哪个移动网格参考框架”。

这两者叠加起来, 才构成了 WarpX 里 charge deposition 完整的时间语义。

因此第 5 章里更准确的调用链不是:

`DepositCharge -> ChargeDeposition.H`

而是:

```
WarpXParticleContainer::DepositCharge
-> 选择 shared-memory 或普通路径
-> 确定 icomp / xyzmin / depos_lev / ref_ratio
-> ABLASTR deposit_charge(...) 做 guard-check 与 CPU/GPU 暂存
-> ChargeDeposition.H 做 shape-factor 和原子加
```

5.8.3 shared-memory charge deposition 不是走 ABLASTR, 而是先变成 tile-binned 执行合同

如果把这一节再往 shared-memory 那半边压一层, 就会发现 `do_shared_mem_charge_deposition` 不是简单把普通 kernel 放进 shared memory, 而是先把整条调用链改写成一套 tile-binned 执行合同。对应源码在 `../warpx/Source/Particles/WarpXParticleContainer.cpp:1514` 与 `../warpx/Source/Particles/Deposition/ChargeDeposition.H:196-380`。

容器层做的第一件事不是调用 ABLASTR, 而是:

1. 仍先检查 `depos_lev` 只能是 `lev` 或 `lev-1`;
2. 仍先用 `shape_extent` 与 `ng_rho / rho->nGrowVect()` 检查粒子 `shape` 能否放进 `tile` 或 `guard cells`;
3. 然后把 `pti.validbox().grow(ng_rho)` 这块区域按 `WarpX::shared_tilesize` 切成 `bins`;
4. 为每个 `bin` 反推出对应 `tile box`, 再做一次 `reduction` 得到 `max_tbox_size`;
5. 最后才把 `bins + box + geom + max_tbox_size + shared_tilesize` 一起传给 `doChargeDepositionSharedShapeN<1..4>()`。

也就是说, `shared-memory charge deposition` 的主入口负责的已经不再只是“选 `shape` 阶数”, 而是把粒子先组织成

`DenseBins + tile boxes + max_tbox_size`

这一套可供 GPU `block` 或 CPU `tile` 复用的局部执行几何。

这一点和普通路径有实质差别。普通路径的主问题是 `icomp / xyzmin / local_rho` 如何经 ABLASTR 桥接到统一 `kernel`; `shared-memory` 路径的主问题则变成:

1. 每个 `tile` 内有哪些粒子;
2. 这个 `tile` 最多需要多大的局部沉积缓冲区;
3. 该缓冲区能否装进单个 `block` 的 `shared memory`。

`doChargeDepositionSharedShapeN<...>()` 本体也正按这个思路组织...`/warpX/Source/Particles/Deposition/ChargeDeposition` 里, 它先根据 `a_tbox_max_size` 构造一个 `sample tile`, 转成 `ix_type` 后再 `grow(depos_order)`, 据此计算

```
const auto npts = sample_tbox_x.numPts();
std::size_t shared_mem_bytes = npts*sizeof(amrex::Real);
const std::size_t max_shared_mem_bytes = amrex::Gpu::Device::sharedMemPerBlock();
WARPX_ALWAYS_ASSERT_WITH_MESSAGE(shared_mem_bytes <= max_shared_mem_bytes,
                                   "Tile size too big for GPU shared memory charge deposition");
```

这说明 `max_tbox_size` 不是旁枝信息, 而是 `shared-memory` 路径能否成立的硬约束: `WarpX` 先从所有实际 `tile` 里取一个逐方向最大包围盒, 再据此估算每个 `block` 需要的临时 `rho` 缓冲区大小; 若超出设备每个 `block` 的 `shared-memory` 预算, 就在 `kernel` 启动前直接 `abort`。

进入 GPU `kernel` 以后, 执行拓扑也和普通路径不同。`shared-memory` 版本不是对所有粒子直接做一个平铺 `ParallelFor`, 而是:

1. `one block per bin/tile`;
2. `block` 内线程按 `stride` 处理本 `tile` 的粒子;
3. 先在 `shared memory` 上分配 `tile-local buf`;
4. 先把 `buf` 清零, 再把当前 `tile` 粒子沉到 `buf`;
5. 最后才把 `buf` 原子加回全局 `rho_arr`。

因此 `shared-memory charge deposition` 虽然在数学上仍调用同样的 `shape-factor` 逻辑, 但它的工程合同已经明显不同于普通 ABLASTR 路径: 后者的 CPU/GPU 差别主要体现在 `local_rho` 还是 `alias` 目标场; 前者则把“`tile-local` 暂存”提升成了第一性执行结构, 并围绕它重新组织 `binning`、`tile geometry`、`shared-memory` 容量和 `block-to-tile` 对应关系。

这也解释了为什么 `shared-memory` 路径没有再走 ABLASTR `deposit_charge(...)`。它需要的不只是统一的 `shape` 调度, 而是:

- `DenseBins` 粒子重排;
- `shared_tilesize` 控制的 `tile` 划分;
- `max_tbox_size` 驱动的 `shared-memory` 预算检查;
- `one block per tile` 的专门 `launch` 拓扑。

这些信息都属于执行拓扑, 而不是普通 `charge kernel` 的纯桥接参数, 所以只能在 `WarpX` 容器层先整理完, 再直接进入 `doChargeDepositionSharedShapeN<...`

5.9 ChargeDeposition.H: 电荷沉积 kernel 的逐项结构

普通路径的中间桥接在 `../warpX/Source/ablastr/particles/DepositCharge.H:50-203`: 它接收 `WarpXParticleContainer` 的 `particle iterator`、本地/目标 `rho`、`ng_rho`、`depos_lev`、`ref_ratio` 与 `icomp/nc`, 再按 `WarpX::noz` 选择 `doChargeDepositionShapeN<1..4>`。最终的 `WarpX-specific shape kernel` 位于 `../warpX/Source/Particles/Deposition/ChargeDeposition.H:37-172`。

下面按 3D 主干摘出核心源码, 并保留 XZ/RZ 与 3D 的原始写入分支; 完整维度条件见原文件同一函数:

```
template <int depos_order>
void doChargeDepositionShapeN (const GetParticlePosition<Pidx>& GetPosition,
                              const amrex::ParticleReal * const wp,
                              const int* ion_lev,
                              amrex::FArrayBox& rho_fab,
```



```

        long np_to_deposit,
        const amrex::XDim3 & dinv,
        const amrex::XDim3 & xyzmin,
        amrex::Dim3 lo,
        amrex::Real q,
        [[maybe_unused]] int n_rz_azimuthal_modes)
{
    const bool do_ionization = ion_lev;
    const amrex::Real invvol = dinv.x*dinv.y*dinv.z;
    amrex::Array4<amrex::Real> const& rho_arr = rho_fab.array();
    amrex::IntVect const rho_type = rho_fab.box().type();

    amrex::ParallelFor(
        np_to_deposit,
        [=] AMREX_GPU_DEVICE (long ip) {
            amrex::Real wq = q*wp[ip]*invvol;
            if (do_ionization){
                wq *= ion_lev[ip];
            }

            amrex::ParticleReal xp, yp, zp;
            GetPosition(ip, xp, yp, zp);

            Compute_shape_factor< depos_order > const compute_shape_factor;
            const amrex::Real x = (xp - xyzmin.x)*dinv.x;

            amrex::Real sx[depos_order + 1] = {0._rt};
            int i = 0;
            if (rho_type[0] == NODE) {
                i = compute_shape_factor(sx, x);
            } else if (rho_type[0] == CELL) {
                i = compute_shape_factor(sx, x - 0.5_rt);
            }

            const amrex::Real y = (yp - xyzmin.y)*dinv.y;
            amrex::Real sy[depos_order + 1] = {0._rt};
            int j = 0;
            if (rho_type[1] == NODE) {
                j = compute_shape_factor(sy, y);
            } else if (rho_type[1] == CELL) {
                j = compute_shape_factor(sy, y - 0.5_rt);
            }

            const amrex::Real z = (zp - xyzmin.z)*dinv.z;
            amrex::Real sz[depos_order + 1] = {0._rt};
            int k = 0;
            if (rho_type[WARPX_ZINDEX] == NODE) {
                k = compute_shape_factor(sz, z);
            } else if (rho_type[WARPX_ZINDEX] == CELL) {
                k = compute_shape_factor(sz, z - 0.5_rt);
            }

            #if defined(WARPX_DIM_XZ) || defined(WARPX_DIM_RZ)
                for (int iz=0; iz<=depos_order; iz++){
                    for (int ix=0; ix<=depos_order; ix++){
                        amrex::Gpu::Atomic::AddNoRet(

```

```

        &rho_arr(lo.x+i+ix, lo.y+k+iz, 0, 0),
        sx[ix]*sz[iz]*wq);
    }
}
#elif defined(WARPX_DIM_3D)
    for (int iz=0; iz<=depos_order; iz++){
        for (int iy=0; iy<=depos_order; iy++){
            for (int ix=0; ix<=depos_order; ix++){
                amrex::Gpu::Atomic::AddNoRet(
                    &rho_arr(lo.x+i+ix, lo.y+j+iy, lo.z+k+iz),
                    sx[ix]*sy[iy]*sz[iz]*wq);
            }
        }
    }
#endif
};
}

```

这段代码对应的 3D 公式是

$$\rho_{i+\alpha, j+\beta, k+\gamma} \leftarrow \rho_{i+\alpha, j+\beta, k+\gamma} + q w_p \frac{1}{\Delta x \Delta y \Delta z} S_\alpha(x_p) S_\beta(y_p) S_\gamma(z_p).$$

源码里 `wq = q*wp[ip]*invvol` 已经包含体积归一化, 因此 `rho_arr` 存的是电荷密度而不是 cell 总电荷。`ion_lev` 非空时再乘电离态, 说明 field ionization species 的有效粒子电荷是在沉积时按粒子属性修正的。

这一层还有两个实现事实值得固定下来:

1. `rho_type = rho_fab.box().type()` 控制每个方向使用 node-centered 还是 cell-centered shape, 所以 charge deposition 并不是“永远拿一套 cell-centered $S(x)$ 到处乘”。
2. `ChargeDeposition.H` 本体看不到 `icomp`。旧/新时间层的 component 偏移已经在桥接层通过 `make_alias(..., icomp*nc, nc)` 或 `lockAdd(..., icomp*nc, nc)` 处理掉了。
3. `ChargeDeposition.H` 里看到的 `rho_fab` 也不总是最终那张 `rho`: GPU 路径直接 alias 目标 `MultiFab`, CPU 路径则可能只是 thread-local `local_rho`, 最后再 `lockAdd(...)` 回真实网格。

`Gpu::Atomic::AddNoRet` 是并行正确性必须条件: 不同粒子可能同时向同一个网格点沉积。没有 atomic, GPU/多线程下源项会出现竞态; 物理上表现为非确定的 ρ 和 $\mathbf{J}$ 误差。

这里还应再补一条维度语义, 否则很容易把 charge deposition 误读成“同一个 3D kernel 只是在低维时少掉几层循环”。源码其实不是这么组织的。../warpx/Source/Particles/Deposition/ChargeDeposition.H:77-136,334-400 里, kernel 会先按几何重写粒子坐标, 再决定到底保留哪几个方向的 shape:

- 1D_Z
 - 不再构造 x/y shape;
 - 只保留 `z = (zp - xyzmin.z)*dinv.z` 和 `sz[...]`;
 - 写回时也只循环 iz。
- RCYLINDER
 - 先把笛卡尔粒子位置压成 `rp = sqrt(xp*xp + yp*yp)`;
 - 再用 `x = (rp - xyzmin.x)*dinv.x` 生成径向 shape `sx[...]`;
 - z 方向 shape 在这个 kernel 分支里根本不再构造。
- RSPHERE
 - 更进一步, 把位置压成球半径 `rp = sqrt(xp*xp + yp*yp + zp*zp)`;
 - 同样只生成单个径向 `sx[...]`;
 - 写回时也只保留一维径向循环。

因此在 RCYLINDER / RSPHERE 下, `ChargeDeposition.H` 做的并不是“先在多维网格上沉积, 再由外层解释成柱/球几何”, 而是 kernel 一开始就把粒子位置改写成径向变量, 只在这一维上构造 support 并做原子加。换句话说:

1. 坐标压缩 已经发生在 kernel 内部;

2. **shape 维数降低** 不是后处理，而是前端事实；
3. 后面的 **inverse-volume scaling** 负责的是几何体积因子，不负责把一份“笛卡尔多维沉积结果”再翻译成柱/球坐标。

shared-memory 版本在这件事上和普通版本保持完全同构。doChargeDepositionSharedShapeN<...>() 的 WARPX_DIM_1D_Z / RCYLINDER / RSPHERE 分支同样先把坐标压成 z 或 r，再只构造对应的一维 sz 或 sx，最后沉到 tile-local buf。因此 shared-memory 路径改变的是执行拓扑，不改变这些几何分支的物理语义。

这里还值得再补一层 RZ 语义，否则读者容易把 charge deposition 误解成“把 2D kernel 原样照搬到柱坐标”。实际 WARPX_DIM_RZ 在写完 mode 0 之后，还会继续沿

$$e^{im\theta}$$

的实部/虚部分量，把 $m>0$ 的 azimuthal modes 显式写进额外 component。也就是说，RZ charge deposition 的 kernel 不只是决定 r-z 平面上的 node/cell shape 支撑，还在同一轮原子加里把 Fourier 模态结构也一并 materialize 到 rho_arr。这条事实和前面的 icomp*nc component 偏移一起看尤其重要：桥接层负责先把“旧/新时间层写到哪一块 component”整理好，而 ChargeDeposition.H 则在那块已经选定的 component 空间里，继续展开 mode 0 + $m>0$ 的 RZ 模态写回。

但这还不是 RZ charge deposition 的最后一步。对独立的 MultiParticleContainer::DepositCharge() 调用来说，所有 species 沉积完成后，../warpx/Source/Particles/MultiParticleContainer.cpp:643-647 还会统一执行

```
WarpX::GetInstance().ApplyInverseVolumeScalingToChargeDensity(rho[lev], lev);
```

也就是说，ChargeDeposition.H 写进去的首先仍是“尚未乘柱/球体积因子倒数”的局部源项；真正带上 $2\pi r$ 、 $4\pi r^2$ 这类几何体积语义的最终 rho，要到 species 汇总完以后才在容器层统一完成。这一点和前面 current deposition 的 inverse-volume scaling 是平行的：kernel 负责局部 shape 与守恒支持，几何体积修正则留到更外层统一做。

这条外层体积合同现在在运行级证据。项目用官方 test_rz_electrostatic_sphere 输入和正确的 warpx.rz executable 完成了 1 rank 运行；官方 analysis_electrostatic_sphere.py 给出三个轴向/径向采样的 L2 误差 0.02425、0.02425、0.01865，均低于 0.05，总能量变化也低于该 test 的 0.32% 门限。项目独立脚本 scripts/analyze_rz_charge_volume_contract.py 再把末态 plotfile 的 mode-0 rho 乘以柱坐标 cell volume

$$\Delta V_{ij} = 2\pi r_i \Delta r \Delta z$$

并与粒子权重乘电子电荷比较：rho 积分得到 $-1.0285648e-15$ C，粒子账本为 $-1.0263572e-15$ C，相对不一致为 $2.1509e-3 < 1e-2$ 。这不是对 ApplyInverseVolumeScalingToChargeDensity() 每个 cell 的逐行证明，但它把“最终 rho 已带上柱坐标体积语义”从源码推断提升成了可复现的全域 charge closure；报告位于 runs/stage-c-validation/rz_electrostatic_sphere/。运行时还暴露了一个可操作的编译边界：RZ 输入必须使用 warpx.rz，不能使用同为二维编译但几何合同为 Cartesian XZ 的 warpx.2d。

RZ 的非零模态也做了独立写回验证。由于官方 native test_rz_langmuir_multi 默认 warpx.n_rz_azimuthal_modes = 1，项目在 runs/stage-c-validation/rz_langmuir_multimode/ 建立了不修改 WarpX 的 case-local sibling：设为 3 个模态、打开 diag1.dump_rz_modes = 1，并把环向每单元粒子数提高到 6，以满足三模态装填约束。末态 plotfile 中 Er/Ez 的 $m=1$ 、 $m=2$ 实部和虚部均非零；把 theta=0 视为

$$F(r, z, 0) = F_0(r, z) + F_{1,\text{real}}(r, z) + F_{2,\text{real}}(r, z)$$

后，独立脚本 scripts/analyze_rz_langmuir_multimode_contract.py 得到 native Er 的重建相对误差 $3.05e-16$ 、Ez 的重建相对误差 $2.53e-16$ 。这验证的是 diagnostics/writeback 与模态重建合同；官方 analysis_rz.py 的单模解析场断言在该三模 sibling 上不适用，不能用它作为多模态结果的判据。报告保留三模 sibling 的解析场残差作为诊断信息，但不把它升级成独立的多模态精确解 gate。

因此普通 charge deposition 的更准确调用链还应再细一层：

```
WarpXParticleContainer::DepositCharge
```

- > 选择 shared-memory 或普通路径
- > 确定 icomp / xyzmin / depos_lev / ref_ratio
- > ABLASTR deposit_charge(...) 做 guard-check、CPU/GPU 暂存与 component 偏移
- > ChargeDeposition.H 做 node/cell shape、RZ modes 与原子加

这里还要把容器层接口的“做什么 / 不做什么”再拆开，否则很容易把 DepositCharge() 误读成一次调用就自动完成所有同步与边界修正。../warpx/Source/Particles/WarpXParticleContainer.cpp:1794-1923 实际上把这些职责分成几层开关：

- reset

- 只决定是否在本次 species 沉积前对 `rho->setVal(0., icomp*nc, nc, rho->nGrowVect());`
- 它不改变 shape kernel, 也不改变时间层语义。
- `local`
 - 只决定沉积后是否做 `SumBoundary(...)`;
 - `local = true` 时, 结果可以停留在本地 guard/valid 区布局, 不强制做跨 patch 通信。
- `apply_boundary_and_scale_volume`
 - 在 RZ / RCYLINDER / RSPHERE 下触发 `ApplyInverseVolumeScalingToChargeDensity(...)`;
 - 在非柱/球几何下则改为 `ApplyRhoFieldBoundary(...)`, 也就是按 PEC 之类边界条件去反射/修正 rho;
 - 它做的是沉积后的外层场语义整理, 不参与粒子到网格的局部 shape 写入。

如果再看多层入口 `WarpXParticleContainer::DepositCharge(const MultiLevelScalarField& rho, ..., interpolate_across_levels, ...)`, 还会再多一层:

- `interpolate_across_levels`
 - 只在每个 level 都沉完之后, 执行 `fine -> coarse` 的 `Coarsen(...) + ParallelAdd(...)`;
 - 它处理的是 AMR 层间一致性, 而不是单 level 内的 tile/guard/source kernel。

这样一来, `DepositCharge()` 这组接口的职责边界就更清楚了:

1. `DepositCharge(pti, ...)`
 - 负责单 tile、单批粒子的 bridge contract 与 kernel launch;
2. `DepositCharge(rho, lev, local, reset, apply_boundary_and_scale_volume, icomp)`
 - 负责单个 level 上的 reset、逐 tile 遍历、volume scaling / PEC 边界修正与 guard-cell 通信;
3. `DepositCharge(multilevel rho, ..., interpolate_across_levels, icomp)`
 - 负责多 level 汇总后是否继续做 fine-to-coarse average down。

这组分层还有一个对读者很重要的负面结论: **charge deposition kernel 本身并不承担 AMR average-down、PEC 边界反射、或全局 guard-cell 交换**。这些都属更外层的容器/通信语义。把这一点讲清以后, 第 5 章里“粒子怎样写入局部 ρ ”和“最终可用于 Maxwell/diagnostics 的 ρ 怎样整理完成”才不会混成一件事。

这里再往 AMR 邻接关系上压一层, 会看到 `DepositCharge()` 其实同时服务两种不同场景, 而它们的跨层语义不能混看。

第一种是**独立 charge 诊断/取样**。例如 `MultiParticleContainer::DepositCharge(rho, relative_time)` 里, WarpX 明确设置的是:

- `local = true`
- `reset = false`
- `apply_boundary_and_scale_volume = false`
- `interpolate_across_levels = false`

然后逐 species 往调用者给定的 rho 上累加。也就是说, 这条接口在这里的任务只是“把所有 species 的局部沉积加到同一张多层 rho 上”; 真正的跨 species 清零由最外层统一先做, 柱/球 inverse-volume scaling 也被推迟到 species 全部累加完成之后再统一做。它并不在 species 级别就把 fine level 自动 average-down 到 coarse level。

第二种是**运行时 AMR coarse-buffer source routing**。 `PhysicalParticleContainer::Evolve()` 里, 当 `has_buffer` 为真时, `PartitionParticlesInBuffers(...)` 会先把粒子数组重排成:

1. 前 `nfine_deposit` 个粒子沉到 fine patch 主场 `rho_fp`
2. 后 `np-nfine_deposit` 个粒子直接沉到 coarse buffer 场 `rho_buf`

对应源码就是:

```
DepositCharge(pti, wp, ion_lev, rho, 0, 0,
              np_to_deposit, thread_num, lev, lev);
...
DepositCharge(pti, wp, ion_lev, crho, 0, np_to_deposit,
              np-np_to_deposit, thread_num, lev, lev-1);
```

以及 push 后对 `icomp=1` 的完全平行调用。这里最重要的事实是: **buffer 粒子不是先沉到 fine rho_fp 再 restrict 到 coarse**。它们从一开始就以 `depos_lev = lev-1` 进入 `DepositCharge(...)`, 因此:

- `tile box` 会先被 `coarsen(pti.tilebox(), RefRatio(lev-1))`
- `dinv` 直接取 coarse level 的 `InvCellSize(lev-1)`
- `xyzmin` 也直接按 coarse level 几何和对应 `time_shift_delta` 计算

换句话说, `rho_buf` 不是事后从 `fine rho` 裁出来的一块通信缓存, 而是在 `coarse patch` 几何上直接生成的源项场。

这也解释了为什么前面那组多层接口参数不能直接替代运行时 `source synchronization`。运行时粗细层一致性真正依赖的是后面的 `SyncRho()` / `AddRhoFromFineLevelandSumBoundary(...)` 骨架: 先把 `fine rho_fp` coarsen 成 `rho_cp`, 再和 `rho_buf` 合并, 再通过临时 `coarse-side fine_lev_cp + OwnerMask` 去重后并回 `coarse` 主场。也就是说:

1. `DepositCharge(..., depos_lev = lev-1)` 解决的是“buffer 粒子该按哪一层几何直接沉积”;
2. `interpolate_across_levels` 解决的是“一个显式给定的多层 `rho` 是否要在函数尾做平均下传”;
3. `SyncRho()` 解决的才是运行时 `field-solver source` 使用前的 `coarse/fine/buffer` 一致性整理。

把这三层分开以后, 就不会误以为只要 `DepositCharge()` 有多层入口, 运行时所有 `rho_buf` 邻接问题都已经在那一层自动解决了。

最后还要把 `charge deposition` 与 `current deposition` 的 `implicit/守恒语义` 明确拆开。`ChargeDeposition.H` 的任务是把某一个已选定时间层上的粒子 `cloud` 写成 ρ , 它只消费当前位置、`ion_lev`、`rho_type`、`xyzmin`、`dinv` 和几何分支; 它并不恢复一步轨迹, 也不构造 `old/new shape difference`, 更不判断 `Esirkepov / Villasenor / Direct / Vay`。这些算法名属于 `current deposition` 的选择空间, 而不是 `charge deposition` 的选择空间。

因此 `DepositCharge()` 的“implicit 细节”不能按 `current deposition` 的 `implicit` 路径去读。对 ρ 来说, 真正需要固定的是:

1. 旧/新时间层由外层 `icomp=0/1` 与 `time_shift_delta -> LowerCorner(...)` 表达;
2. 粒子位置若需要额外时间平移, 由更外层 `MultiParticleContainer::DepositCharge(relative_time)` 临时 `PushX(...)` 表达;
3. `coarse-buffer` 粒子若要沉到粗层, 由 `depos_lev = lev-1` 直接改写 `tilebox`、`dinv` 与 `xyzmin`;
4. `shared-memory` 分支只改变 `binning`、`tile-local buffer` 和回写拓扑, 不引入新的物理算法;
5. `PEC`、`guard exchange`、`inverse-volume scaling` 和 `AMR average-down` 都在容器/通信层完成, 不在 `ChargeDeposition.H` `kernel` 内完成。

本节的 `current-checkout source contract` 由 `scripts/audit_charge_deposition_bridge_contract.py` 固定验收。它不是替代物理推导的测试, 而是防止源码阅读中的关键桥接环节被误读或在后续版本漂移: 当前 13 个锚点覆盖 `icomp/time_shift_delta`、`ABLASTR` 的越界保护与 `CPU/GPU` 暂存、形状函数分派、`RZ` 模式以及 `atomic writeback`。

这组负面边界是本节的收口点: `charge deposition` 的局部 `kernel` 是“单时间层 `shape` 加权的源项采样器”, 而不是 `charge-conserving current mover`。离散连续性方程仍由后面的 `current deposition` 与 `SyncCurrentAndRho()` 共同承担; `DepositCharge()` 在这条链里提供的是与旧/新时间层、`AMR buffer` 和几何体积因子一致的 ρ^n/ρ^{n+1} 输入。

5.10 Direct current deposition: 非守恒但直观的速度加权沉积

`Direct current deposition` 的核心 `kernel` 是 `../warpx/Source/Particles/Deposition/CurrentDeposition.H:47-274`。

下面两段分别取自同一函数的前半段和写入段; 中间省略的是与 `x` 方向同构的 `y/z` 方向 `shape` 初始化。先看粒子电流权重和半步位置:

```
template <int depos_order>
AMREX_GPU_HOST_DEVICE AMREX_INLINE
void doDepositionShapeNKernel([[maybe_unused]] const amrex::ParticleReal xp,
                              [[maybe_unused]] const amrex::ParticleReal yp,
                              [[maybe_unused]] const amrex::ParticleReal zp,
                              const amrex::ParticleReal wq,
                              const amrex::ParticleReal vx,
                              const amrex::ParticleReal vy,
                              const amrex::ParticleReal vz,
                              amrex::Array4<amrex::Real> const& jx_arr,
                              amrex::Array4<amrex::Real> const& jy_arr,
                              amrex::Array4<amrex::Real> const& jz_arr,
                              amrex::IntVect const& jx_type,
                              amrex::IntVect const& jy_type,
                              amrex::IntVect const& jz_type,
                              const amrex::Real relative_time,
                              const amrex::XDim3 & dinv,
                              const amrex::XDim3 & xyzmin,
                              const amrex::Real invvol,
                              const amrex::Dim3 lo,
                              [[maybe_unused]] const int n_rz_azimuthal_modes)
```

```

{
    // wqx, wqy wqz are particle current in each direction
    #if defined(WARPX_DIM_RZ) || defined(WARPX_DIM_RCYLINDER)
        const amrex::Real xpmid = xp + relative_time*vx;
        const amrex::Real ypmid = yp + relative_time*vy;
        const amrex::Real rpmid = std::sqrt(xpmid*xpmid + ypmid*ypmid);
        const amrex::Real costheta = (rpmid > 0._rt ? xpmid/rpmid : 1._rt);
        const amrex::Real sintheta = (rpmid > 0._rt ? ypmid/rpmid : 0._rt);
        const amrex::Real wqx = wq*invvol*(+vx*costheta + vy*sintheta);
        const amrex::Real wqy = wq*invvol*(-vx*sintheta + vy*costheta);
        const amrex::Real wqz = wq*invvol*vz;
    #else
        const amrex::Real wqx = wq*invvol*vx;
        const amrex::Real wqy = wq*invvol*vy;
        const amrex::Real wqz = wq*invvol*vz;
    #endif

    Compute_shape_factor< depos_order > const compute_shape_factor;
    const double xmid = ((xp - xyzmin.x) + relative_time*vx)*dinv.x;
    double sx_node[depos_order + 1] = {0.};
    double sx_cell[depos_order + 1] = {0.};
    int j_node = 0;
    int j_cell = 0;
    if (jx_type[0] == NODE || jy_type[0] == NODE || jz_type[0] == NODE) {
        j_node = compute_shape_factor(sx_node, xmid);
    }
    if (jx_type[0] == CELL || jy_type[0] == CELL || jz_type[0] == CELL) {
        j_cell = compute_shape_factor(sx_cell, xmid - 0.5);
    }
}

```

relative_time 在显式路径通常是 $-0.5*dt$ 。由于 DepositCurrent() 被调用时粒子位置已经是 x^{n+1} ，这行

$$x_{\text{mid}} = \frac{x^{n+1} - x_{\text{min}} - \frac{1}{2}v_x \Delta t}{\Delta x}$$

把沉积位置移回半步。Direct deposition 的电流权重就是 $qw_p \mathbf{v}_p / \Delta V$ 。

最终写入数组的源码在 CurrentDeposition.H:211-274:

```

// Deposit current into jx_arr, jy_arr and jz_arr
#if defined(WARPX_DIM_XZ) || defined(WARPX_DIM_RZ)
    for (int iz=0; iz<=depos_order; iz++){
        for (int ix=0; ix<=depos_order; ix++){
            amrex::Gpu::Atomic::AddNoRet(
                &jx_arr(lo.x+j_jx+ix, lo.y+l_jx+iz, 0, 0),
                sx_jx[ix]*sz_jx[iz]*wqx);
            amrex::Gpu::Atomic::AddNoRet(
                &jy_arr(lo.x+j_jy+ix, lo.y+l_jy+iz, 0, 0),
                sx_jy[ix]*sz_jy[iz]*wqy);
            amrex::Gpu::Atomic::AddNoRet(
                &jz_arr(lo.x+j_jz+ix, lo.y+l_jz+iz, 0, 0),
                sx_jz[ix]*sz_jz[iz]*wqz);
        }
    }
#elseif defined(WARPX_DIM_3D)
    for (int iz=0; iz<=depos_order; iz++){
        for (int iy=0; iy<=depos_order; iy++){
            for (int ix=0; ix<=depos_order; ix++){

```

```

    amrex::Gpu::Atomic::AddNoRet(
        &jx_arr(lo.x+j_jx+ix, lo.y+k_jx+iy, lo.z+l_jx+iz),
        sx_jx[ix]*sy_jx[iy]*sz_jx[iz]*wqx);
    amrex::Gpu::Atomic::AddNoRet(
        &jy_arr(lo.x+j_jy+ix, lo.y+k_jy+iy, lo.z+l_jy+iz),
        sx_jy[ix]*sy_jy[iy]*sz_jy[iz]*wqy);
    amrex::Gpu::Atomic::AddNoRet(
        &jz_arr(lo.x+j_jz+ix, lo.y+k_jz+iy, lo.z+l_jz+iz),
        sx_jz[ix]*sy_jz[iy]*sz_jz[iz]*wqz);
    }
}
}
#endif
}

```

3D 形式可以写为

$$J_{x,i+\alpha,j+\beta,k+\gamma} \leftarrow J_{x,i+\alpha,j+\beta,k+\gamma} + qw_p v_x \frac{1}{\Delta V} S_{\alpha}^{J_x}(x_p^{n+1/2}) S_{\beta}^{J_y}(y_p^{n+1/2}) S_{\gamma}^{J_z}(z_p^{n+1/2}),$$

J_y, J_z 同理, 但使用各自 staggering 对应的 shape 数组 $sx_jy/sy_jy/sz_jy$ 与 $sx_jz/sy_jz/sz_jz$ 。Direct 路径的优点是简单, 缺点是这个公式没有强制把 ρ^n 和 ρ^{n+1} 的差精确写成离散散度。

这里还要补一条容易被略掉的接口边界: 即使到了 implicit 版本, direct deposition 的 contract 也没有升级成“恢复完整轨迹, 再按边界裁剪后沉积”。doDepositionShapeNImplicit(...) 只是把 γ^{-1} 改成由 u_n 与 $u_{n+1/2}$ 共同恢复, 然后把

```
const amrex::Real relative_time = 0._rt;
```

喂回同一个 doDepositionShapeNKernel(...)。源码见 ../warpX/Source/Particles/Deposition/CurrentDeposition.H:363-441。也就是说, direct 的 implicit 语义依旧是:

1. 选定一个时间中心位置;
2. 用该时间层速度形成 $qwv/\Delta V$;
3. 按当前 staggering shape 直接沉回 Jx/Jy/Jz。

它并不显式接收 domain_double、do_cropping, 也不把粒子轨迹当成一条可以在 PEC/PECInsulator 附近截断、再按 cell crossing 重分段的几何对象。这也是为什么在 near-boundary 守恒场景里, direct 不能简单充当 Villasenor 的“廉价替身”。

5.11 Esirkepov current deposition: 用新旧形函数差构造连续性方程

阅读 Esirkepov 论文时, 最容易发生的记号错位是把论文的方向分解、WarpX 的前缀累加变量和最终网格电流分量直接画等号。它们可以建立结构对应, 但不是同一个层次的对象。当前源码审计固定下表作为本章的记号入口:

论文层对象	WarpX 当前实现	读者应保留的边界
W^1: x 向 shape difference	sx_old-sx_new 沿 i 累加到 sdx_i, 再写入 Jx	这是 3D Esirkepov kernel 的方向分工, 不等于所有 RZ/XZ 数组布局
W^2: y 向 shape difference	sy_old-sy_new 沿 j 累加到 sdy_j, 再写入 Jy	RZ/XZ 中 out-of-plane 分量会使用不同的几何分支
W^3: z 向 shape difference	sz_old-sz_new 沿 k 累加到 sdz_k, 再写入 Jz	1D/2D/RZ 会减少实际循环维度, 不能照搬 3D 下标
old/new form factor	Compute_shape_factor 与 Compute_shifted_shape_factor 生成 sx/sy/sz old/new 数组	shifted shape 的首索引对齐是源码合同, 不是论文排版中的隐含步骤
transverse tensor-product factor	one_third/one_sixth 组成 old-old、old-new、new-old、new-new 混合平均	该对应由预印本与源码核对得到, 不表示 CPC 定稿已逐页比较
current normalization	invdtd.x/y/z = transverse inverse cell area / dt	不能把三个分量都简化成单独的 1/dt

这张表对应的 14 个源码锚点由 `scripts/audit_esirkepov_notation_contract.py` 验收, 报告见 `runs/stage-c-validation/esirkepov-` 它关闭的是论文记号到当前 `checkout` 变量的映射歧义, 不替代 `SyncCurrent()`、AMR coarse-fine、边界同步或全 `geometry/order` runtime regression。

Esirkepov 入口在 `../warpX/Source/Particles/Deposition/CurrentDeposition.H:675-723`:

```
template <int depos_order>
void doEsirkepovDepositionShapeN (const GetParticlePosition<PIdx>& GetPosition,
                                   const amrex::ParticleReal * const wp,
                                   const amrex::ParticleReal * const uxp,
                                   const amrex::ParticleReal * const uyp,
                                   const amrex::ParticleReal * const uzp,
                                   const int* ion_lev,
                                   const amrex::Array4<amrex::Real>& Jx_arr,
                                   const amrex::Array4<amrex::Real>& Jy_arr,
                                   const amrex::Array4<amrex::Real>& Jz_arr,
                                   long np_to_deposit,
                                   amrex::Real dt,
                                   amrex::Real relative_time,
                                   const amrex::XDim3 & dinv,
                                   const amrex::XDim3 & xyzmin,
                                   amrex::Dim3 lo,
                                   amrex::Real q,
                                   [[maybe_unused]] int n_rz_azimuthal_modes,
                                   const amrex::Array4<const int>& reduced_particle_shape_mask,
                                   bool enable_reduced_shape
                                   )
{
    bool const do_ionization = ion_lev;

    amrex::XDim3 const invdtd = amrex::XDim3{(1.0_rt/dt)*dinv.y*dinv.z,
                                              (1.0_rt/dt)*dinv.x*dinv.z,
                                              (1.0_rt/dt)*dinv.x*dinv.y};

    Real constexpr inv_c2 = PhysConst::inv_c2;
    Real constexpr one_third = 1.0_rt / 3.0_rt;
    Real constexpr one_sixth = 1.0_rt / 6.0_rt;

    enum eb_flags : int { has_reduced_shape, no_reduced_shape };
    const int reduce_shape_runtime_flag = (enable_reduced_shape && (depos_order>1)) ? has_reduced_shape : n
```

`invdtd.x=(1/dt)*dinv.y*dinv.z` 不是普通的 $1/\Delta t$ 。因为 J_x 位于 x-face, 离散连续性中 J_x 的差分还会除以 Δx , 所以 current 的量纲需要配合横截面积 $1/(\Delta y \Delta z)$ 。源码使用 `dinv.y*dinv.z/dt`, 后续再由差分 operator 处理 x 向差分。

粒子旧/新位置和 shape 数组在 `CurrentDeposition.H:724-935` 生成:

```
amrex::ParallelFor( TypeList<CompileTimeOptions<has_reduced_shape,no_reduced_shape>>{},
                    {reduce_shape_runtime_flag},
                    np_to_deposit, [=] AMREX_GPU_DEVICE (long ip, auto reduce_shape_control) {
                        Real const gaminv = 1.0_rt/std::sqrt(1.0_rt + uxp[ip]*uxp[ip]*inv_c2
                                                            + uyp[ip]*uyp[ip]*inv_c2
                                                            + uzp[ip]*uzp[ip]*inv_c2);

                        Real wq = q*wp[ip];
                        if (do_ionization){
                            wq *= ion_lev[ip];
                        }

                        ParticleReal xp, yp, zp;
```



```

GetPosition(ip, xp, yp, zp);

double const x_new = (xp - xyzmin.x + (relative_time + 0.5_rt*dt)*uxp[ip]*gaminv)*dinv.x;
double const x_old = x_new - dt*dinv.x*uxp[ip]*gaminv;
double const y_new = (yp - xyzmin.y + (relative_time + 0.5_rt*dt)*uyp[ip]*gaminv)*dinv.y;
double const y_old = y_new - dt*dinv.y*uyp[ip]*gaminv;
double const z_new = (zp - xyzmin.z + (relative_time + 0.5_rt*dt)*uzp[ip]*gaminv)*dinv.z;
double const z_old = z_new - dt*dinv.z*uzp[ip]*gaminv;

const Compute_shape_factor< depos_order > compute_shape_factor;
const Compute_shifted_shape_factor< depos_order > compute_shifted_shape_factor;
const Compute_shifted_shape_factor< 1 > compute_shifted_shape_factor_order1;

double sx_new[depos_order + 3] = {0.};
double sx_old[depos_order + 3] = {0.};
const int i_new = compute_shape_factor(sx_new+1, x_new );
const int i_old = compute_shifted_shape_factor(sx_old, x_old, i_new);

double sy_new[depos_order + 3] = {0.};
double sy_old[depos_order + 3] = {0.};
const int j_new = compute_shape_factor(sy_new+1, y_new);
const int j_old = compute_shifted_shape_factor(sy_old, y_old, j_new);

double sz_new[depos_order + 3] = {0.};
double sz_old[depos_order + 3] = {0.};
const int k_new = compute_shape_factor(sz_new+1, z_new );
const int k_old = compute_shifted_shape_factor(sz_old, z_old, k_new );

int dil = 1, diu = 1;
if (i_old < i_new) { dil = 0; }
if (i_old > i_new) { diu = 0; }
int djl = 1, dju = 1;
if (j_old < j_new) { djl = 0; }
if (j_old > j_new) { dju = 0; }
int dkl = 1, dku = 1;
if (k_old < k_new) { dkl = 0; }
if (k_old > k_new) { dku = 0; }

```

这里 x_{new} 与 x_{old} 是同一时间步轨迹的两端。显式路径中 $\text{relative_time} = -0.5 \cdot dt$ ，而粒子数组位置是 push 后位置，于是

$$x_{\text{new}} = x^{n+1} \left(-\frac{1}{2} \Delta t + \frac{1}{2} \Delta t \right) v_x = x^{n+1}, \quad x_{\text{old}} = x^{n+1} - v_x \Delta t = x^n.$$

最后的 3D 电流公式在 CurrentDeposition.H:955-989:

```

for (int k=dkl; k<=depos_order+2-dku; k++) {
  for (int j=djl; j<=depos_order+2-dju; j++) {
    amrex::Real sdxi = 0._rt;
    for (int i=dil; i<=depos_order+1-diu; i++) {
      sdxi += wq*invdtd.x*(sx_old[i] - sx_new[i])*
        one_third*(sy_new[j]*sz_new[k] + sy_old[j]*sz_old[k])
        + one_sixth*(sy_new[j]*sz_old[k] + sy_old[j]*sz_new[k]);
      amrex::Gpu::Atomic::AddNoRet( &Jx_arr(lo.x+i_new-1+i, lo.y+j_new-1+j, lo.z+k_new-1+k), sdxi);
    }
  }
}
for (int k=dkl; k<=depos_order+2-dku; k++) {
  for (int i=dil; i<=depos_order+2-diu; i++) {

```

```

    amrex::Real sdyj = 0._rt;
    for (int j=djl; j<=depos_order+1-dju; j++) {
        sdyj += wq*invdtd.y*(sy_old[j] - sy_new[j])*
            (one_third*(sx_new[i]*sz_new[k] + sx_old[i]*sz_old[k])
            +one_sixth*(sx_new[i]*sz_old[k] + sx_old[i]*sz_new[k]));
        amrex::Gpu::Atomic::AddNoRet( &Jy_arr(lo.x+i_new-1+i, lo.y+j_new-1+j, lo.z+k_new-1+k), sdyj);
    }
}
}
for (int j=djl; j<=depos_order+2-dju; j++) {
    for (int i=dil; i<=depos_order+2-diu; i++) {
        amrex::Real sdzk = 0._rt;
        for (int k=dkl; k<=depos_order+1-dku; k++) {
            sdzk += wq*invdtd.z*(sz_old[k] - sz_new[k])*
                (one_third*(sx_new[i]*sy_new[j] + sx_old[i]*sy_old[j])
                +one_sixth*(sx_new[i]*sy_old[j] + sx_old[i]*sy_new[j]));
            amrex::Gpu::Atomic::AddNoRet( &Jz_arr(lo.x+i_new-1+i, lo.y+j_new-1+j, lo.z+k_new-1+k), sdzk);
        }
    }
}
}

```

这不是 $q\mathbf{v}S$ 的 direct 形式。它用新旧 shape 的差构造电流。例如 J_x 内部累计量可概括为

$$\Delta J_x(i, j, k) \propto \sum_{i' \leq i} q \frac{S_x^n(i') - S_x^{n+1}(i')}{\Delta t \Delta y \Delta z} \overline{S_y S_z}(j, k),$$

其中横向平均 $\overline{S_y S_z}$ 在源码中由 one_third 和 one_sixth 组合给出。这种构造保证对 $J_x/J_y/J_z$ 做离散度时，望远镜求和会还原新旧电荷形函数差：

$$\nabla_h \cdot \mathbf{J}^{n+1/2} = -\frac{\rho^{n+1} - \rho^n}{\Delta t}$$

在同一 shape、同一网格中心和同一边界同步规则下成立。这就是 Esirkepov 路径比 direct 路径更适合作为显式电磁 PIC 默认守恒沉积的原因。

如果把这段源码和 Esirkepov 2001 的 paper-level 推导对起来，逻辑会更清楚。论文第 3 节先把单粒子位移导致的总电荷变化写成

$$W^1 + W^2 + W^3 = S(x + \Delta x, y + \Delta y, z + \Delta z) - S(x, y, z),$$

然后再要求三个方向的电流差分分别去承担 $W^1/W^2/W^3$ 。WarpX 没有显式保留 $W(i, j, k, m)$ 这个中间数组，但实现上的等价结构就是：

1. Compute_shifted_shape_factor(...) 先把 old/new shape 放到统一索引框架内；
2. sx_old-sx_new、sy_old-sy_new、sz_old-sz_new 分别承担三个方向的差分源；
3. one_third/one_sixth 横向平均把多维 tensor-product shape 的耦合补回。

因此源码里的 prefix-like sdxi/sdyj/sdzk 不是经验配方，而是论文 density decomposition 在二阶 spline/tensor-product family 下的一种直接程序化实现。

如果把这条对应再压到循环级，关系会更清楚。3D Esirkepov kernel 并不是先算一个总的 $S_{\text{new}} - S_{\text{old}}$ 再数值拆账，而是从三组 prefix loop 的拓扑上就已经把 $W^1/W^2/W^3$ 固定下来了。对固定的 j, k , J_x 那组循环实际在做

```
sdxi += (sx_old[i] - sx_new[i]) * yz_mixed_average(j, k);
```

其中 yz_mixed_average 正是 $1/3, 1/6, 1/6, 1/3$ 的 old/new 双横向组合；然后沿 i 方向把它累加并写回 J_x 。 J_y 和 J_z 两组循环只是把同一结构置换到 y, z 方向，于是分别承担 W^2, W^3 。也就是说：

- sdxi 前缀和承担 W^1 ；
- sdyj 前缀和承担 W^2 ；
- sdzk 前缀和承担 W^3 。

如果把论文里的八项结构再和这三组 loop 并排看，分工其实非常具体： W^1 总是把“x 方向 old/new difference”当成主变量，而把 y,z 两方向放进 $1/3, 1/6$ 的 mixed average； W^2 则把主变量换成 y 方向差分； W^3 再换成 z。所以源码里真正被“前缀累加”的不是一个抽象的三维总差分，而是三份已经在论文里各自分好工的方向性 shape 变化。这样读的时候，`sdxi/sdyj/sdzk` 就不只是三个长得很像的循环，而是：

- `sdxi`: 先提取 x 向直接变化，再让 y,z 耦合平均去补横向几何；
- `sdyj`: 先提取 y 向直接变化，再让 x,z 去补横向几何；
- `sdzk`: 先提取 z 向直接变化，再让 x,y 去补横向几何。

这条对应第 5 章很关键，因为它说明 Eq. (23) 在源码里并没有“蒸发成一堆经验循环”，而是被压缩进了三组方向前缀累加的循环骨架本身。

更进一步，项目内预印本已经足够把 Eq. (23) 的结构写清：每个方向的 W^m 都是八个 old/new corner-like shape 值的线性组合，而且只出现两种系数 $1/3$ 与 $1/6$ 。这说明 WarpX 里显式写出来的 `one_third / one_sixth` 不是局部数值调味，而是论文唯一性分解的直接遗留物；当前源码中横向平均项的结构，并不是“为了让公式看起来对称”，而是为了让三方向分解在加总后精确回到总的 shape 差分。

这里还应把论文的 claim 再说硬一点。Esirkepov 2001 并没有把 density decomposition 当成众多可选配方之一，而是明确声称：在线性、零位移退化、坐标置换对称和总差分守恒这些自然条件下，这就是定义粒子相关电流的唯一允许过程。这样回头看 WarpX 的 `sdxi/sdyj/sdzk + one_third/one_sixth`，它们就不再只是“当前实现选用的一组常数”，而更接近一条被论文唯一性条件挑出来、随后在现代 tensor-product kernel 里程序化保存下来的结构。

不过这里也要把当前证据层级说清。第 5 章现在对 Esirkepov 的 paper-backed 论证，已经不再只是“源码猜测”，但它当前绑定的是项目内已经 materialize 的作者 arXiv 预印本 physics/9901047，而不是 Elsevier Computer Physics Communications 135(2) 的 publisher-formatted 定稿 PDF。也就是说，本章现在已经可以稳定依赖以下几层结论：

1. Eq. (23) 的 $W^1/W^2/W^3$ 结构与 $1/3, 1/6$ 系数；
2. density decomposition 的唯一性口径；
3. 二阶 spline / tensor-product form-factor 如何压成可编程局部算法；
4. 这些 paper-level 结构在 WarpX `sdxi/sdyj/sdzk + one_third/one_sixth` 中的程序化对应。

但本章仍不应提前声称“2001 CPC 发表版已逐行核对”。当前本机 access audit 只证明了两件事：ScienceDirect 的 article/PDF 端点存在，而当前命令行环境访问 PDF 仍返回 HTTP/2 403；相对地，arXiv 预印本在同日复核下继续可达。因此更准确的正文边界应是：Esirkepov 当前已经达到 preprint-backed + source-grounded，但还没有完成 publisher-PDF line-by-line compare。

一旦后续拿到 CPC 定稿，本章最值得优先做的不是泛泛“再看一遍论文”，而是按五个窄目标做 bounded compare：

1. title wording；
2. abstract wording；
3. section titles and numbering；
4. Eq. (23) 及其前后 density decomposition 公式；
5. second-order spline form-factor 那一节算法说明。

这样处理的好处是，本章当前可以继续放心用预印本支撑 Eq. (23) 到源码 loop 的主叙述，而不会把“尚未取得 publisher PDF”误写成“论文侧还完全不可引用”。

不过这五项里其实已有一项可以先靠本地资产落下最小结论：title wording 的确存在稳定差别。当前项目目录里的合法全文预印本标题是

- Exact charge conservation scheme for Particle-in-Cell simulations for a big class of form-factors

而本地 access-audit.md、DOI 记录与 WarpX bibliography 对应的 CPC 发表版标题则是

- Exact charge conservation scheme for Particle-in-Cell simulation with an arbitrary form-factor

这还不足以推出正文内容有何变化，但至少已经说明“预印本与发表版完全同题”这件事不能先验假定；后续 bounded compare 时，标题差异不是待发现项，而是已经确认存在、只是还未继续追踪到 abstract / section wording / equation typography 层。

截至 2026-07-11，这条 compare 线又可以把“已核实”和“仍未核实”拆得更干净。arXiv 页面明确记录预印本于 1999-01-26 提交，题名为 Exact charge conservation scheme for Particle-in-Cell simulations for a big class of form-factors，并标注为 13 页、无图、10 条参考文献；公开书目信息则确认 CPC 发表版为 Computer Physics Communications 135(2), 144-153 (2001)，题名为 Exact charge conservation scheme for Particle-in-Cell simulation with an arbitrary form-factor，DOI 为 10.1016/S0010-4655(00)00228-9。这已经足以把“发表版身份”和“当前本地全文资产”分开记录，但仍不足以推出发表版正文的逐式编辑差异。

本项目把这次 bounded compare 单独记在 notes/code-reading/particles/44-esirkepov-cpc-bounded-comparison.md。因此，本章当前可以更准确地写成：发表版书目信息已核实，预印本已完成 MinerU 和源码映射，Eq. (23) 到 `sdxi/sdyj/sdzk` 的主论证可以使用；但 abstract、section numbering、公式排版和 second-order spline 段落仍不能声称已经按 publisher PDF 逐页核过。

同样，当前预印本也已经足够把论文内部的 section 结构稳定绑定到第 5 章的主叙述，而不必等发表版 PDF 才能继续写。更准确地说：

1. **Section 2 Continuity equation in finite differences**
 - 先把离散 Maxwell + leapfrog mover 压成 $(\rho^{n+1}-\rho^n)/dt + \nabla_h \cdot J = 0$;
 - 这正对应本章先讲“为什么 direct qvS 不自动守恒”，再讲 source-side continuity contract 的必要性。
2. **Section 3 Density decomposition**
 - 先定义 $W^1/W^2/W^3$ ，再要求它们加总恢复总的 shape 差分；
 - 这正对应 WarpX 里 `sx_old-sx_new`、`sy_old-sy_new`、`sz_old-sz_new` 三组方向差分源，以及三组 prefix loop 的分工。
3. **Section 4 Computing of the current with second-order polynomial form-factor**
 - 把算法明确压成 `old shape -> push -> new shape -> difference -> directional current` 的可编码步骤；
 - 这正对应 `Compute_shifted_shape_factor(...)`、old/new arrays 同框、再到 `sdx/sdy/sdz` 前缀累加这条现代 kernel 骨架。

这层结构性对应很重要，因为它说明第 5 章当前并不是只拿 Eq. (23) 这一条孤立公式去贴源码，而是已经能把预印本的问题设定、唯一性分解、到二阶 spline 算法化三个层次，分别对回 WarpX 的守恒目标、directional decomposition、kernel skeleton。

为了避免把“论文结构已能对回源码”误读成“发表版逐页已核对”，当前证据矩阵固定如下：

论文层次	本地可用证据	WarpX 对应	当前证据等级
Section 2: 离散连续性方程	arXiv 预印本全文、MinerU Markdown	<code>SyncCurrentAndRho()</code> 、 <code>divE-rho/epsilon_0</code> regression	preprint-backed + source-grounded
Section 3: $W^1/W^2/W^3$ density decomposition	预印本公式与中文讲解	<code>sx_old-sx_new</code> 、 <code>sy_old-sy_new</code> 、 <code>sz_old-sz_new</code>	preprint-backed + source-grounded
Section 4: 二阶 spline 算法骨架	预印本算法段落	<code>compute_shifted_shape_factor</code> 、 <code>sdx/sdy/sdz</code> prefix loops	preprint-backed + source-grounded
CPC 发表版题名、卷期、页码、DOI 和公开摘要	ScienceDirect/公开书目元数据	<code>Esirkepovcpc01</code> bibliography key	publication-metadata verified
CPC 发表版 abstract、section numbering、Eq. (23) 排版、二阶 spline 文字	当前环境未取得 publisher PDF	暂无可绑定的逐页证据	open compare

这张表是本章当前的证据边界：前三行可以直接进入成书正文，第四行只能用于出版身份和引用信息，第五行不能写成已完成。

公式层还增加了一项可复现的负责任验证：`scripts/verify_esirkepov_density_decomposition.py` 用 10000 组确定性随机 old/new shape 分量检查

$$W^1 + W^2 + W^3 = (S_x^{old} + \Delta S_x)(S_y^{old} + \Delta S_y)(S_z^{old} + \Delta S_z) - S_x^{old} S_y^{old} S_z^{old}.$$

该脚本只验证论文 Eq. (23) 的代数分解，不替代 WarpX kernel、网格散度或端到端 regression；但它把 1/2 横向平均和 1/3 三重差分项的局部恒等式从“文字解释”提升为可重复执行的 formula-level check。用固定 seed 2001 的 10000 组样本运行时，最大残差为 $8.8818e-16 \leq 2e-15$ ；JSON/Markdown 证据归档于 `runs/stage-c-validation/esirkepov-density-decomposition/contract.{json,md}`。

公式层之外，又对当前同级 `../warpX checkout` 做了只读 source audit。`scripts/audit_esirkepov_source_contract.py` 检查 `CurrentDeposition.H` 中的 14 个锚点全部存在：包括 `doEsirkepovDepositionShapeN`、`Compute_shifted_shape_factor`、`invdtd`、`one_third/one_sixth`、`sdx/sdy/sdz`、三方向 old/new shape difference 和 `Jx/Jy/Jz` writeback。报告位于 `runs/stage-c-validation/esirkepov-source-contract/contract.{json,md}`；这证明当前源码仍 materialize 了正文所描述的 skeleton，但仍不是数值 kernel regression。

Villasenor 的组织方式则完全不同。`VillasenorDepositionShapeNKernel(...)` 在完成轨迹恢复和 boundary crop 之后，第一件事不是构造 shape difference，而是先统计整条轨迹的 `cell_crossings_x/y/z`，得到 `num_segments`，再按 crossing 逐段推进。3D 情况下，它甚至不会平均切段，而是每一轮都比较哪个方向先撞到下一条 crossing，并用最早发生的那个 crossing 定义当前段终点。也就是说，Villasenor 的第一性对象不是“一对 old/new shape 数组”，而是一条被真实 cell crossing 切开的粒子轨迹。

这条结构和 Villasenor-Buneman 1992 的 paper-level 写法也是一致的。原文先从最简单的 four-boundary case 出发，把一整步运动写成

$$J_{x1}, J_{x2}, J_{y1}, J_{y2}$$

四个局部 boundary flux；若一步跨过更多网格线，再继续拆成 seven-boundary / ten-boundary 的多段 four-boundary 子移动。这里真正应抓住的是它背后的组织原则：

- 第一性对象是局部 boundary flux，而不是全轨道 old/new shape difference；
- 遇到 crossing 就继续切段，而不是先做整轨道全局差分；
- 三维时再通过局部交叉项和分段权重，把严格守恒推广到 face-local stencil。

这也解释了为什么 Villasenor-Buneman 1992 会专门强调“不把一般位移拆成两次正交 move”。如果先把一条真实二维或三维轨迹硬拆成若干彼此独立的正交子移动，那么虽然也可能写出某种守恒公式，但它描述的已经不再是同一条几何扫描。Villasenor 真正坚持的是：每一段局部通量都必须来自粒子云这一步实际先撞到哪条 boundary 的几何事实；现代 WarpX 把这条历史动机改写成 earliest-crossing 判据，本质上正是在程序里拒绝“先假设可以正交拆分、再回头拼装”的思路。

所以 WarpX 现代源码虽然不再显式保留 four/seven/ten-boundary 这套手工分类，但 cell_crossings -> num_segments -> local this_J* writeback 这条段式结构，正是那篇 1992 论文的现代化程序版本。

可以把这层 paper-to-code 关系压缩成下面的流程。论文用 four-/seven-/ten-boundary 给若干典型几何子移动命名；WarpX 则先统计 crossing 数，再在同一个循环中重复执行“选最早 crossing、截断 segment、局部写回”的过程。因此图中的 four-boundary 不是某个固定的 WarpX 枚举值，seven/ten-boundary 也不是源码中的两个分支名，而是 repeated segmentation 可能产生的论文级几何结果。

flowchart TB

accTitle: Villasenor Segment Deposition

accDescr: The diagram maps the paper's boundary-crossing picture to WarpX's repeated earliest-crossing

```
orbit_start([One-step particle orbit]) --> count_crossings[Count cell crossings]
count_crossings --> set_segments[Set num_segments]
set_segments --> segment_check{More than one segment?}
segment_check -->|No| build_weights[Build cell/node weights]
segment_check -->|Yes| choose_crossing{Choose earliest crossing}
choose_crossing --> truncate_segment[Truncate current segment]
truncate_segment --> build_weights
build_weights --> write_local_flux[Write local this_J components]
write_local_flux --> remaining_segments{Segments remain?}
remaining_segments -->|Yes| choose_crossing
remaining_segments -->|No| sum_flux([Sum segment-local fluxes])
```

```
classDef process fill:#dbeafe,stroke:#2563eb,stroke-width:2px,color:#1e3a5f
classDef decision fill:#fef9c3,stroke:#ca8a04,stroke-width:2px,color:#713f12
classDef terminal fill:#dcfcec,stroke:#16a34a,stroke-width:2px,color:#14532d
```

```
class count_crossings,set_segments,build_weights,truncate_segment,write_local_flux process
class segment_check,choose_crossing,remaining_segments decision
class orbit_start,sum_flux terminal
```

论文级几何名称	它表达的内容	WarpX 中的运行时表示
four-boundary	一个 elementary local submove 的 boundary-flux 分配	一个 crossing-defined segment 的 this_J* 局部写回
seven-boundary	在基本子移动上增加 crossing 后的多边界几何	num_segments > 1 后重复执行 earliest-crossing loop；不是固定的 seven 分支
ten-boundary	更复杂的多 crossing 几何组合	同一 loop 继续累加更多 segment；具体 segment 数由端点 cell index 差决定

这张图也解释了源码注释中“valid for an arbitrary number of cell crossings”的含义：实现的可扩展性来自循环和 crossing 计数，而不是预先枚举有限个几何 case。论文中的 seven/ten-boundary 仍然适合用来帮助读者建立几何直觉，但阅读 WarpX 时应优先追踪 cell_crossings_*、num_segments、ns 和局部 this_J*，不要寻找同名的 case label。

更具体一点，论文里从 four-boundary 进入 seven-boundary 或 ten-boundary，本质上是在回答同一个问题：当前这一步里，粒子云先撞到哪一条新的网格边界？现代 WarpX 不再把答案写成手工 case table，而是直接在 kernel 里做“最早 crossing”判据。XZ/RZ 分支会比较当前候选 x-crossing 和 z-crossing 哪个先发生：

```
if (dzp == 0. || std::abs(dxp_seg) < std::abs(dxp/dzp*dzp_seg)) {
    Xcell = x0_new;
    dzp_seg = dzp/dxp*dxp_seg;
    z0_new = z0_old + dzp_seg;
} else {
    Zcell = z0_new;
    dxp_seg = dxp/dzp*dzp_seg;
    x0_new = x0_old + dxp_seg;
}
```

而 3D 分支则把同一个逻辑扩成三方向竞争：比较 x/y/z 三个候选 crossing 中谁最早发生，就用那个方向来定义当前 segment 的终点。这样一来，论文里“先切成 seven-boundary 还是 ten-boundary 的哪一段”这件事，在今天的 WarpX 代码里已经被改写成了一个更通用的程序判据：

1. 为每个方向构造下一条候选 crossing；
2. 比较谁最早发生；
3. 用最早 crossing 截断当前 segment；
4. 把余下轨迹继续送回同一套局部沉积循环。

所以 seven-/ten-boundary 在现代实现里不再是离散命名的几何 case，而是 repeated earliest-crossing segmentation 的自然结果。

如果再把二维 four-boundary 公式和今天的 XZ/RZ kernel 对一下，关系还可以写得更硬一点。论文里的

$$J_{x1} + J_{x2} = \Delta x, \quad J_{y1} + J_{y2} = \Delta y$$

说明二维 Villasenor 的纵向总输运，本质上仍由该方向位移本身控制；几何信息真正改变的是“这份输运怎样在两条局部 boundary 之间分配”。WarpX 当前 XZ/RZ segment kernel 恰好保留了这层结构：Jx 与 Jz 都写成“该段主方向输运量”乘上横向 old/new node weights 的简单平均，

```
this_Jx = wx*sx_cell[i]*(sz_old_node[k] + sz_new_node[k])/2 * seg_factor_x;
this_Jz = wz*sz_cell[k]*(sx_old_node[i] + sx_new_node[i])/2 * seg_factor_z;
```

也就是说，二维情形里最核心的 four-boundary 几何仍然能读成：

1. 主方向总输运由 dx_seg 或 dz_seg 决定；
2. 横向 old/new 平均决定它落在哪两条局部 boundary 上、各占多少；
3. crossing 变多时，就继续把整步拆成多个 obey 同一规则的局部 segment。

这正是论文里“two-boundary split of one directional flux”到现代 kernel 的直接对应关系。

如果把对应再压到 Eq. (6)-(9) 本身，可以写得更细。论文里的

$$J_{x1} = \Delta x \left(\frac{1}{2} - y - \frac{1}{2} \Delta y \right), \quad J_{x2} = \Delta x \left(\frac{1}{2} + y + \frac{1}{2} \Delta y \right)$$

可以直接读成：“同一段 x 向总输运 Δx ，按横向旧位置 y 和横向位移 Δy 改写成上下两条 boundary 上的两份局部 flux”。WarpX 今天的 XZ/RZ kernel 不再显式把这两份 flux 分别命名成 J_{x1}, J_{x2}，而是把同一件事改写成

$$\text{cell-based support in } x \times \frac{S_{\text{old}}^{(z)} + S_{\text{new}}^{(z)}}{2} \times \frac{dt_{\text{seg}}}{dt}.$$

其中：

1. sx_cell[i] 承担论文里“当前这段 flux 落在哪个主方向 cell support 上”；

2. $\frac{1}{2}(\text{sz}_{\text{old}} + \text{sz}_{\text{new}})$ 承担论文里由 $y, \Delta y$ 决定的“两条 boundary 怎样分流”;
3. $\text{seg_factor_x} = \text{dt}_{\text{seg}}/\text{dt} = \text{dx}_{\text{seg}}/\text{dx}$ 则把这一段局部输运从整步 Δx 缩回当前 crossing-defined 子移动。

而与之完全对称的另外两式

$$J_{y1} = \Delta y \left(\frac{1}{2} - x - \frac{1}{2} \Delta x \right), \quad J_{y2} = \Delta y \left(\frac{1}{2} + x + \frac{1}{2} \Delta x \right)$$

则说明论文并不是把 x 向和 y 向分开用两套不同逻辑处理; 它坚持的是同一条局部几何规则: 某一方向的总输运由该方向位移给出, 而它分到哪两条相邻 boundary, 则由正交方向上的旧位置与位移共同决定。换到今天的 WarpX 语言里, 这正对应 XZ/RZ kernel 中 J_x/J_z 彼此镜像的两条写法: 主方向分量落在 cell-based support 上, 正交方向只通过 old/new 节点权重平均来决定两边界分流, 而不会再额外引入第二套独立守恒机制。

若再贴近 Eq. (6)–(9) 本身, 四个通量的角色可以直接读成:

- J_{x1} : x 向输运落到“下侧”那条局部 boundary 的份额;
- J_{x2} : 同一份 x 向输运落到“上侧”那条局部 boundary 的份额;
- J_{y1} : y 向输运落到“左侧”那条局部 boundary 的份额;
- J_{y2} : 同一份 y 向输运落到“右侧”那条局部 boundary 的份额。

于是 $\frac{1}{2} \Delta y$ 这类因子, 本质上不是在给 Δx 再乘一个神秘修正, 而是在回答: 当 charge cloud 沿 x 方向推进时, 它有多少面积扫过了上/下两条相邻 boundary。WarpX 当前 XZ/RZ kernel 用 old/new node average 来写这件事, 只是把论文里显式的几何宽度改写成了现代 shape-weight 语言; 它保留下来的核心物理量, 仍然是“哪一条局部边界分到多少横向扫掠面积”。

所以从 paper-level 读到 code-level, 真正保持不变的不是表面符号, 而是这条组织关系: 先有主方向输运, 再有横向分流, 最后才由 crossing segmentation 决定这一份局部 flux 属于哪一段真实轨迹。

两篇论文还有一条共同的实现边界, 值得在这里顺手点明。Villasenor 1992 在 2D 讨论里直接把 timestep 约束和 Courant condition 连在一起; Esirkepov 2001 第 4 节则要求 $|\Delta x|, |\Delta y|, |\Delta z|$ 不超过单个网格步长。它们说法不同, 但物理边界一致: 这两条严格守恒沉积都默认 one-step orbit 仍是局部对象。WarpX 现代实现对此条前提的处理方式, 则是把它拆到 dt/CFL、implicit/suborbit endpoint reconstruction, 以及 cell_crossings $\rightarrow$ segment loop 这些程序结构里, 而不再在正文里单独保留一个“几何 case table”。

更进一步, 每个 segment 内部也不是只拿段长乘平均速度。源码会同时构造两组权重:

- 以段中心 $x_0_{\text{bar}}/y_0_{\text{bar}}/z_0_{\text{bar}}$ 为输入的 cell-based weights, 由 Compute_shape_factor<depos_order-1> 生成;
- 以段两端 $x_0_{\text{old}} \rightarrow x_0_{\text{new}}, y_0_{\text{old}} \rightarrow y_0_{\text{new}}, z_0_{\text{old}} \rightarrow z_0_{\text{new}}$ 为输入的 node-based old/new weights, 由 Compute_shape_factor_pair<depos_order> 生成。

然后对当前段直接形成局部 $\text{this_Jx}/\text{this_Jy}/\text{this_Jz}$:

```
this_Jx = wx*sx_cell[i]*( ... )*seg_factor_x;
this_Jy = wy*sy_cell[j]*( ... )*seg_factor_y;
this_Jz = wz*sz_cell[k]*( ... )*seg_factor_z;
```

这里的 seg_factor_ 本质上就是当前段的 $\text{dt_seg}/\text{dt}$ 。因此 Villasenor 的守恒组织方式不是 Esirkepov 那种沿方向累加 $\text{sdxi}/\text{sdyj}/\text{sdzk}$ 的 whole-orbit prefix sum, 而是:

1. 用 crossing 把整条轨迹切成多个局部 segment;
2. 对每个 segment 单独构造 cell/node 权重;
3. 把该段的局部输运量直接写回 $\text{this_Jx}/\text{this_Jy}/\text{this_Jz}$;
4. 最后由所有 segment 的局部输运求和闭合整条轨迹的离散守恒。

这里还可以再压实一层。3D kernel 里这三个 seg_factor

```
seg_factor_x = dxp_seg/dxp;
seg_factor_y = dyp_seg/dyp;
seg_factor_z = dzp_seg/dzp;
```

并不是抽象的“分段修正系数”, 而是当前局部 segment 在三个方向上分别占整步总位移的比例。也就是说, WarpX 在每个方向上做的不是“把整条轨迹的通量平均分给若干 segment”, 而是:

1. 先由 earliest-crossing 规则确定当前 segment 的真实终点;
2. 再把这一段在 $x/y/z$ 三个方向上实际走过的位移, 分别写成整步位移的分数;
3. 最后用这些方向分数去缩放当前段的 $\text{this_Jx}/\text{this_Jy}/\text{this_Jz}$ 。

这条结构和论文里 seven-/ten-boundary 的子移动通量是同一个物理意思：局部段只承担自己那一小段真实扫掠所对应的那部分 face flux，而不会提前替后续 segment“多沉一截”。因此 `seg_factor_x/y/z` 不是附带修正项，而是现代 WarpX 用程序方式保存 Villasenor 局部守恒子移动语义的关键部件。

这里还有一个容易被误读的细节。`one_third / one_sixth` 并不只出现在 Esirkepov 路径里；Villasenor 的每个 segment 也在横向两个方向上使用同样的 $1/3, 1/6$ 组合，对 old/new node weights 做局部平均。区别不在于“是否使用这组系数”，而在于它们服务的对象完全不同：

- 在 Esirkepov 里，这组系数属于整条轨迹 old/new shape difference 的唯一分解；
- 在 Villasenor 里，这组系数属于单个 segment 内横向 old/new node weights 的局部 face-flux 平均。

对 Villasenor 1992 的 3D 推导来说，这条差别还有一层更具体的意义。论文里专门冒出来的 $\Delta x \Delta y \Delta z / 12$ ，强调的是三维局部通量不再是三个方向彼此独立的简单并列，而会出现真正的 mixed-direction coupling。WarpX 当前 3D kernel 虽然不再把这类项显式写成单个 $\Delta x \Delta y \Delta z / 12$ 单项式，但它并没有把这层耦合抹掉；相反，这层耦合正是通过每个方向电流里那组

```
old*old * one_third
old*new * one_sixth
new*old * one_sixth
new*new * one_third
```

的双横向 old/new 混合平均被程序化保存下来的。现代源码中的 `one_third/one_sixth` 在 Villasenor 3D 路径里不只是“平滑一下横向权重”，而是在离散实现层面承担了论文 3D 交叉耦合项的角色：它保证每个方向的局部 face flux 在做双横向平均时，仍然保留 old/new 端点之间的混合信息，而不是把三维守恒退化三个互不相干的一维沉积。

如果把论文 Eq. (36) 本身也放进来，这层对应还能再硬一点。那一式给出的 x 向四个 face contribution 不是四份彼此独立的 Δx ，而是

1. 一份 $+\Delta x, \bar{\eta}, \bar{\zeta}$;
2. 两份带负号的 $-\Delta x \Delta y \Delta z / 12$ 修正；
3. 以及一份带正号的 $+\Delta x \Delta y \Delta z / 12$ 修正。

它真正表达的是： x 向 face flux 要同时感受到 y/z 两个横向方向的局部体积重叠，因此四个相邻 x -face 上的份额既有“横向平均面积”主项，也有“旧端点与新端点不能简单分离”的 mixed-direction coupling 修正。WarpX 当前 3D kernel 虽然把这层结构改写成 `old*old / old*new / new*old / new*new` 四项 old/new 混合平均，但这四项的符号与权重组织，承担的正是同一种职责：让每个 `this_Jx/this_Jy/this_Jz` 在分到四个局部横向角点时，不会丢掉论文 Eq. (36) 里那条 $+, -, -, +$ 型耦合信息。

因此两条算法虽然都继承了同一类 tensor-product 守恒平均结构，但一个把它组织成 whole-orbit decomposition，另一个把它组织成 segment-local flux closure。这解释了为什么两段 kernel 看起来都会出现 `one_third/one_sixth`，但循环骨架、support 范围和几何语义仍然截然不同。

这里同样值得把当前证据边界讲清。和 Esirkepov 那条线不同，Villasenor-Buneman 1992 在本项目里当前不只是 preprint-backed，而是已经基于本机现成 full-text PDF 与 MinerU 资产 materialize 到论文目录，因此第 5 章对 Villasenor 的 paper-backed 论证不再需要退回“只有源码、没有论文”的口径。当前正文已经可以稳定依赖的层次包括：

1. “不把一般位移拆成正交 move”这条历史动机；
2. four-/seven-/ten-boundary move 的局部 boundary-flux 组织；
3. cell_crossings -> num_segments -> local this_J* writeback 与论文几何 case 的现代对应；
4. XZ/RZ 下 directional transport * (old+new)/2 * dt_seg/dt 这条 four-boundary 到 segment kernel 的直接映射；
5. 3D 路径里 `one_third/one_sixth` 与 $\Delta x \Delta y \Delta z / 12$ 类交叉耦合的程序化对应。

这条线当前已经完成四边界、重复分段和三维交叉项的第一轮公式级审计；仍未完成的是论文图示逐图回填、记号统一，以及所有现代 geometry/order 分支的逐项等价性审查。因此本章现在对 Villasenor 最稳妥的证据等级是 **paper-backed + source-grounded + formula-audited**，而不是把尚未完成的出版级图示精修误写成公式缺口。

5.11.1 Villasenor-Buneman 公式级审计：四边界、重复分段与三维交叉项

项目内的 Villasenor-Buneman 1992 full-text PDF 与 MinerU Markdown 现在已经完成第一轮公式级核对。论文以局部原点为最近的 cell-boundary 交点，对单位方形粒子的四边界运动写出：

$$\begin{aligned} J_{x1} &= \Delta x \left(\frac{1}{2} - y - \frac{1}{2} \Delta y \right), & J_{x2} &= \Delta x \left(\frac{1}{2} + y + \frac{1}{2} \Delta y \right), \\ J_{y1} &= \Delta y \left(\frac{1}{2} - x - \frac{1}{2} \Delta x \right), & J_{y2} &= \Delta y \left(\frac{1}{2} + x + \frac{1}{2} \Delta x \right). \end{aligned}$$

这四式的核心不是某个固定阶数的 shape kernel，而是“主方向位移 × 横向扫描宽度的旧/新平均”。因此在当前 WarpX XZ/RZ kernel 中，对应关系应读成：主方向的 displacement 或 cell weight，乘以横向 old/new node weight 的平均，再乘 `seg_factor = dt_seg/dt`。现代源码还要额外承载 arbitrary shape order、几何分支、boundary crop 和 segment-local writeback，所以不能把源码中的表达式当成论文四式的逐字复制。

论文对 seven-boundary 和 ten-boundary 也没有另起两套独立电流公式。seven-boundary 先按第一次 complementary-mesh crossing 把轨迹分成两段，例如：

$$\Delta x_1 = \frac{1}{2} - x, \quad \Delta y_1 = \frac{\Delta y}{\Delta x} \Delta x_1, \quad qquad \Delta x_2 = \Delta x - \Delta x_1, \quad qquad \Delta y_2 = \Delta y - \Delta y_1.$$

随后对两段分别套用四边界公式；ten-boundary 则重复同一过程三次。这个映射正好解释了 WarpX 当前的 `cell_crossings_* -> num_segments -> earliest-crossing -> local this_J*` 循环：论文中的几何名称是结果分类，源码中的可扩展性来自“截断一段、写回一段、继续推进”的循环，而不是七/十边界的固定分支标签。

下面的示意图把这层对应关系压成读者侧流程。它不是 Villasenor-Buneman 论文原图，也不表示源码里存在 `seven_boundary` 或 `ten_boundary` 两个分支；它只把论文的结果分类和 WarpX 当前循环骨架放在同一张图中：

flowchart LR

```

A[" 论文: four-boundary move"] --> B[" 第一次 complementary-mesh crossing"]
B --> C[" 写回 segment-local this_J*"]
C --> D[" 轨迹是否还有剩余位移? "]
D -->| " 是" | E[" 更新局部原点与 residual displacement"]
E --> B
D -->| " 否" | F[" 论文分类: 完成 four-boundary move"]
E -. " 两段结果" .-> G["seven-boundary case"]
E -. " 三段结果" .-> H["ten-boundary case"]

```

读图时应把实线理解成现代实现的执行顺序，把虚线理解成论文中的结果分类。也就是说，seven-/ten-boundary 不是额外的物理守恒律，而是同一局部四边界构造在一条轨迹上重复执行后出现的几何计数；这也是为什么源码只需要一个可重复的 crossing loop，就能覆盖论文中多个 case。

三维部分还给出了一个不能省略的交叉项。对某个 x-face，论文写成：

$$\Phi_x = \Delta x \bar{\eta} \bar{\zeta} + \frac{\Delta x \Delta y \Delta z}{12},$$

其余三个 x-face 按横向因子和交叉项符号变化，y/z 分量由循环置换得到。论文明确指出 $\Delta x \Delta y \Delta z / 12$ 是三维新增项。WarpX 当前 3D Villasenor kernel 不把它保留为一个独立的单项式，而是通过 `one_third/one_sixth` 组成的四个 old/new 横向权重乘积表达同一类 mixed-direction coupling。因而“源码没有显式的 /12”不能被解释成“三维交叉耦合不存在”。

这次审计的完整公式与证据边界记录于 `notes/code-reading/particles/45-villasenor-formula-level-audit.md`。当前可以把 Villasenor 线标记为 **paper-backed + source-grounded + formula-audited**；尚未完成的只是论文图示逐图回填、记号统一和所有现代 geometry/order 分支的逐项等价性审查。

这里还应补一条维度差异，否则读者容易误以为 Villasenor 在所有几何里都完全按同一平均式沉积。实际并不是这样。WarpX 的 3D kernel 中，Jx/Jy/Jz 三个分量都要面对真正的双横向耦合，因此都会写成 `cell-based weight * (old/new node weights 的 1/3, 1/6 组合) * seg_factor`。但在 XZ/RZ kernel 里，in-plane 的 Jx/Jz 只需要处理单个横向方向，所以退化成 `(old+new)/2` 的简单平均；只有 out-of-plane 的 Jy 仍保留 1/3, 1/6 组合。换句话说：

- 2D/XZ/RZ：主平面输运更接近论文 four-boundary 的“两条局部边界分流”；
- 3D：每个方向都必须面对双横向耦合，因此每个分量都需要完整的 tensor-product 局部平均。

这条维度差异，正是 Villasenor 1992 二维公式和 WarpX 三维通用 kernel 之间最重要的实现延伸。

这也是为什么源码注释会强调 Villasenor “results in a tighter stencil”。它更紧，不是因为放弃了高阶修正。恰恰相反，`depos_order >= 3` 时，源码仍会对 cell-based weights 做

$$\frac{4S_{\text{bar}} + S_{\text{old}} + S_{\text{new}}}{6}$$

式的 higher-order 修正；只是这些修正现在被局部化到了每个 segment 内，而不是像 Esirkepov 那样围绕整条 old/new shape difference 一次性组织。更稳定的结论可以直接写成：

- **Esirkepov**: 把整条轨迹压成 old/new shape arrays, 并在统一索引框架内做方向前缀累加;
- **Villasenor**: 把整条轨迹按真实 cell crossing 切段, 再让每个 segment 的局部输运逐段闭合守恒。

两条路径满足的是同一条离散连续性方程, 但“守恒是怎样被压进代码里的”完全不同。

为了把这条差异固定成可复查的出版级摘要, 可以把论文对象、源码对象和验证边界并排写成下表:

对照层	Esirkepov 2001	Villasenor-Buneman 1992	读者应保留的边界
第一性对象	整条轨迹两端的 tensor-product shape difference	crossing-defined segment 的局部 face flux	两者都服务离散连续性, 但不是同一 个局部变量
论文结构	$W^1 + W^2 + W^3$ density decomposition; 二阶 form-factor 使用 mixed averages	four-boundary flux; 多 crossing 递归成 seven-/ten-boundary 子移 动; 3D 有 mixed-direction cross term	论文 case 名称不能直接当作现代 源码分支名
WarpX 入口	doEsirkepovDepositionShapeNKKernelDeposit	doVillasenorDepositionShapeNKKernelDeposit 及 explicit/implicit 前端	前者由 DepositCurrent() 分派, 不能由 DepositCharge() 推断
源码循环	sx_old-sx_new、 sy_old-sy_new、 sz_old-sz_new 加 sdx/sdy/sdzk prefix accumulation	cell_crossings_* -> num_segments -> earliest-crossing -> this_J* segment loop	一个是 whole-orbit directional decomposition, 一个是 segment-local closure
1/3, 1/6 的职责	论文唯一性分解中的横向 old/new mixed average	单个 segment 的局部横向 face-flux average	数值常数相同不代表算法组织相同
几何支持	old/new shape 先在同一索引 框架中对齐; 需要 shifted shape	轨迹按真实 crossing 裁剪和分段; 支持域随 segment 局部化	near-boundary 语义不能把 Direct 当作任一路径的替身
当前验证	verify_esirkepov_density 的代数恒等式, 加 Langmuir/current- correction 的端到端 Gauss-law gate	45-composition-formula-level 的公式审计, 加 RZ/3D kernel 源码映射	公式恒等式和 kernel bitwise equivalence 或所有 geometry/order 的端到端证明

表中最后一列是本章的出版边界: 它允许读者在一页内看清“论文中的守恒对象如何进入源码”, 同时阻止两个常见的过度推断。第一, one_third/one_sixth 只是共享的 tensor-product 平均结构, 不意味着 Villasenor 就是 Esirkepov 的另一种循环写法; 第二, 公式恒等式和源码行级映射只能证明局部结构, 不能自动升级成所有维度、边界、shape order 和 implicit/suborbit 分支都已逐项等价。

这条局部公式边界现在也有独立的可执行检查: scripts/verify_villasenor_formula_contract.py 用固定 seed 的 10000 组二维 crossing 轨迹和三维中点/位移样本, 先验证四边界公式满足

$$J_{x1} + J_{x2} = \Delta x, \quad J_{y1} + J_{y2} = \Delta y,$$

再把任意轨迹按所有 cell crossing 切成 segment, 验证各段位移之和仍恢复整条轨迹; 同时验证 Eq.(36) 四个 face contribution 的三维交叉项和体积分数差分闭合。二维残差最大为 4.440892098500626e-16, 三维 face-sum 与 volume-closure 残差最大为 1.7763568394002505e-15, 最大 crossing 数为 6。报告位于 runs/stage-c-validation/villasenor_formula_contract/contract.{j 它把 Eq.(6)-(9)、Eq.(36) 与 repeated segmentation 的代数/几何层落成了可重复证据, 但仍不替代 WarpX kernel 的 bitwise、边界或全量 Gauss-law regression。

在公式审计之外, 本轮又对当前 ../warpx checkout 做了只读源码合同核对。scripts/audit_villasenor_source_contract.py 的 16 个锚点全部通过, 覆盖 VillasenorDepositionShapeNKKernel、explicit/implicit entryptoint、三方向 cell_crossings_* 计数、num_segments 循环、final-segment/continuation 分支、seg_factor_* 和 this_Jx/this_Jy/this_Jz 写回。报告位于 runs/stage-c-validation/villasenor-source-contract/contract.{json,md}; 它说明正文中的 crossing-driven segment skeleton 与当前源码仍一致, 但仍不替代数值 kernel regression。

本轮又把 implicit Villasenor 从源码和公式层推进到官方 2-rank 运行级证据。官方 test_2d_theta_implicit_jfnk_vandb 使用 shape=2、周期边界和 theta-implicit Newton/JFNK; 上游 analysis_vandb_jfnk_2d.py 与独立 scripts/analyze_implicit_villasenor

均通过, 最大总能量相对变化为 $4.0980\text{e-}15 < 2\text{e-}14$, Gauss-law RMS 为 $9.2951\text{e-}16 < 2\text{e-}15$, 末态网格为 40x40, 所有输出字段有限。报告归档于 runs/stage-c-validation/implicit_villasenor_2d_jfnk_mpi2/contract.{json,md}。

同一条独立 contract 又读取官方 test_2d_theta_implicit_jfnk_vandb_cropping: 该 sibling 把 shape 提升到 4、网格缩小到 16x16, 并打开 near-boundary cropping; 官方 analysis 与独立读取均通过, 末态 Gauss-law 最大绝对误差为 $8.2275\text{e-}14 < 1\text{e-}13$, RMS 为 $3.0023\text{e-}14$ 。这两条结果可以支持“2D implicit Villasenor 的普通 shape=2 路径和 shape=4 boundary-cropping 路径均有运行级守恒证据”, 但不能外推到所有 geometry/order。

同一 family 的 test_2d_theta_implicit_jfnk_vandb_filtered 只把 warpx.use_filter 打开为 1, 保留 shape=2、周期边界和 JFNK 配置不变。官方 analysis 与要求显式确认 filter 输入的独立 contract 均通过: 最大总能量相对变化 $3.8931\text{e-}15 < 2\text{e-}14$, Gauss-law RMS $5.1401\text{e-}16 < 2\text{e-}15$, 且末态字段全部 finite。于是当前 2D 证据不只覆盖“Villasenor 能守恒”, 还覆盖了 implicit current 同步之后再经过 field filter 的组合路径。

官方 test_2d_theta_implicit_jfnk_vandb_picmi 又提供了同一物理合同的 Python 前端路径。项目使用 Python-enabled build_py 的 pywarpx.picmi 运行 2-rank 输入脚本; PICMI 生成的 inputs2d_from_PICMI 和最终 warpx_used_inputs 均明确包含 algo.current_deposition = "villasenor"、algo.evolve_scheme = "theta_implicit_em" 与 algo.particle_shape = 2。官方输入脚本 producer 与独立 contract 均完成, 最大总能量相对变化 $4.0980\text{e-}15 < 2\text{e-}14$, Gauss-law RMS $9.5730\text{e-}16 < 2\text{e-}15$, 末态字段全部 finite。该运行日志还出现 newton.linear_solver unused-input 提示; 它来自 PICMI 生成配置的拼写边界, 不影响本次由 GMRESLinearSolver materialize 的实际运行, 但不能把该 warning 误写成前端完全无 unused-input。

维度边界也必须如实记录: 官方 RZ theta-implicit Villasenor 输入要求 newton.linear_solver=petsc_ksp, 当前 build_full binary 未启用 AMREX_USE_PETSC, 因此在 NewtonSolver::Define() 初始化阶段直接拒绝, 未进入物理计算。随后只用命令行把同一输入的线性求解器覆盖为 amrex_gmres 做 control; 它仍未进入物理时间推进, 而是在 WarpX::InitData() -> ThetaImplicitEM::Define() -> InitializeCurlCurlBCMasks() 触发 SIGILL。因此当前 RZ blocker 不只是 PETSc 缺失, 还包含 arm64 build_full 的 RZ theta-implicit boundary-mask 初始化失败。官方 1D planar-pinch sibling 则在 Newton 后的粒子边界处理路径出现 SIGILL, 只产出初始诊断帧。三者均记录为构建/运行边界, 而不是伪造为 Villasenor physics failure 或 pass。

运行级 case	主要分支	独立结果	证据边界
test_2d_theta_implicit_jfnk_vandb=2、周期、energy + Gauss law	官方 + independent, 2-rank	4.0980e-15 energy、9.2951e-16 Gauss RMS, PASS	官方 + independent, 2-rank
test_2d_theta_implicit_jfnk_vandb_cropping	near-boundary cropping	8.2275e-14 max charge error, PASS	官方 + independent, 未含强 energy ledger
test_2d_theta_implicit_jfnk_vandb_filtered	warpx.use_filter=1	3.8931e-15 energy、5.1401e-16 Gauss RMS, PASS	官方 + independent, 显式确认 filter 输入
test_2d_theta_implicit_jfnk_vandb_picmi	Python GMRESLinearSolver	4.0980e-15 energy、9.5730e-16 Gauss RMS, PASS	Python-enabled build, 保留 unused-input warning
test_rz_theta_implicit_dyaxis/insulator	RZ blocker	PETSc 官方路径在 NewtonSolver::Define() 拒绝; amrex_gmres control 在 InitializeCurlCurlBCMasks() SIGILL	未进入物理计算
test_1d_theta_implicit_planar-pinch	1D planar-pinch	Newton 后 SIGILL, 仅初始帧	不作为通过证据

5.11.2 Esirkepov 运行级维度证据: 1D、2D 与 3D Langmuir

前面的公式恒等式和源码合同只能证明局部结构; 为了避免把它们误写成端到端证据, 本轮又直接运行了 WarpX 官方 Langmuir 输入。官方 1D 和 3D 输入本身显式设置 algo.current_deposition = esirkepov; 官方 2D 测试卡默认是 direct, 本项目在 case-local 副本中只将这一项覆盖为 esirkepov, 并补上官方 analysis 所需的 rho/divE 输出字段, 因此它是可复查的验证 sibling, 而不是 WarpX 上游已注册的 2D Esirkepov regression。

两条运行都使用当前 checkout 对应的 native binary、2 个 MPI rank 和 OMP_NUM_THREADS=1。官方 analysis 负责理论 Langmuir 场误差与其内置 charge-conservation gate; 项目独立的 scripts/analyze_esirkepov_langmuir_contract.py 则重新读取最

终 plotfile, 检查 Ex/Ey/Ez/Bx/By/Bz/jx/jy/jz/rho/divE 的有限性, 并独立计算

$$\epsilon_{\text{charge}} = \frac{\|\text{divE} - \rho/\epsilon_0\|_{\infty}}{\|\rho/\epsilon_0\|_{\infty}}.$$

结果如下:

case-local 证据	维度/网格	官方理论场 gate	独立 charge gate	结论
runs/stage-c-validation1D/2D/3D shape=1/2/3/4	1D/2D/3D shape=1/2/3/4	1.026e-2 / < 0.05	8.3450e-12 < 1e-11	PASS
runs/stage-c-validation1D/2D/3D shape=1/2/3/4	1D/2D/3D shape=1/2/3/4	2.201e-2 / 0.0503	3.5650e-12 < 1e-11	PASS
runs/stage-c-validation1D/2D/3D shape=1/2/3/4	1D/2D/3D shape=1/2/3/4	3.402e-2 / 0.0502	3.1326e-12 < 1e-11	PASS
runs/stage-c-validation1D/2D/3D shape=1/2/3/4	1D/2D/3D shape=1/2/3/4	4.632e-2 / 0.0502	4.5607e-12 < 1e-11	PASS
runs/stage-c-validation1D/2D/3D shape=1/2/3/4	1D/2D/3D shape=1/2/3/4	6.216e-2 / 0.0503	4.8972e-12 < 1e-11	PASS
runs/stage-c-validation1D/2D/3D shape=1/2/3/4	1D/2D/3D shape=1/2/3/4	8.404e-2 / < 0.05	1.3029e-12 < 1e-11	PASS
runs/stage-c-validation1D/2D/3D shape=1/2/3/4	1D/2D/3D shape=1/2/3/4	3.608e-2 / 0.0503	L0 0.8828、L1 1.2005	BOUNDARY
ratio 4, CKC/filter				

这些证据把第 5 章的 Esirkepov 运行覆盖推进到 1D/2D/3D + 2D shape=1/2/3/4。shape=4 的 0.07 场误差阈值来自官方 analysis_2d.py 对测试名中 particle_shape_4 的分支, 而不是本项目临时放宽; shape=1/2/3 仍使用 0.0503, 所有 shape 都使用独立 1e-11 charge residual gate。shape=0 的尝试在当前 WarpX.cpp:1450 初始化断言处拒绝, 源码合同只允许 particle_shape=1..4, 因此记录为 unsupported boundary, 而不是失败的 physics case。MR overlay 的理论场 gate 通过, 但逐层 reader contract 在 L0/L1 分别得到 0.8828/1.2005, 故当前只标记为 BOUNDARY, 不把它升级成 AMR 守恒通过; 新增的 15-anchor AMR source contract 已证明路由/同步源码骨架存在, 新增的 7-anchor Python observability audit 也证明 generic register API 存在, 但两者都不能替代中间场与 route-count 专门验证。当前 1-4 阶运行证据仍不能推出 AMR buffer、边界裁剪、RZ/RCYLINDER/RSPHERE 或 implicit 分支都已逐项等价, 也不能替代尚未取得的 CPC publisher-PDF bounded compare。2D case 的 direct -> esirkepov 覆盖和 rho/divE 诊断字段只存在于本项目 case-local 输入副本中, 不能写成上游官方注册回归。独立 contract 的 JSON/Markdown 结果分别归档在七个 case-local 目录中, producer 与官方 analysis 日志也保留在同目录。

5.12 沉积不只回 rho/J: WarpX 还把线性响应矩阵和统计矩交回网格

如果只盯 DepositCharge() 和 DepositCurrent(), 会把本章的边界讲窄。WarpX 里至少还有两条同样属于 particle-to-grid deposition 的源项支线:

1. mass matrices deposition
 - 服务 implicit/JFNK 的线性化电流响应;
 - 不是 Maxwell 方程这一时间步要直接消费的 current_fp。
2. temperature / variance deposition
 - 服务 per-species 温度和速度方差统计;
 - 不是 rho/J 的附属输出, 而是一条独立的 weighted-moment deposition 主线。

这三条线共享同一套 particle-to-grid 工程骨架:

- shape order
- tilebox / buffer
- guard-cell 安全检查
- boundary sum / fill-boundary

但物理语义不同。

5.12.1 mass matrices: 沉的是 J(E) 的线性响应, 不是当前步的 Maxwell 源项

MultiParticleContainer::DepositMassMatrices() 会先把

- MassMatrices_X

- MassMatrices_Y
- MassMatrices_Z

三组方向块清零,再逐 species 调 `pc->DepositMassMatrices(fields, lev, dt)`。源码见 `../warpX/Source/Particles/MultiParticleContainer.cpp`。

species 级的 `PhysicalParticleContainer::DepositMassMatrices()` 则取:

- Bfield_aux
- 九个矩阵块 Sxx..Szz

再调用 `WarpXParticleContainer::DepositMassMatrices(...)`,源码见 `../warpX/Source/Particles/PhysicalParticleContainer.cpp`。

因此这条线的第一性对象不是

$$\rho, \quad \mathbf{J},$$

而是 implicit 线性化里近似

$$\mathbf{J}(\mathbf{E}) \approx \mathbf{J}_0 + \mathbf{M}(\mathbf{E} - \mathbf{E}_0)$$

所需的粒子响应块。也正因为它不是普通 current deposition 的平移版,源码一开始就拒绝:

- Esirkepov
- Vay
- colocated grid

只保留:

- Villasenor
- Direct

两条 mass-matrix deposition 路径, 见 `../warpX/Source/Particles/WarpXParticleContainer.cpp:1131-1365`。

5.12.2 temperature / variance deposition: 沉的是样本数、加权均值和去均值二次矩

species 打开 `do_temperature_deposition` 后, `PhysicalParticleContainer::AllocData()` 会按 `current_fp` 的 `box/stagger/guard` 规格额外分配 `T_<species>` 三个方向场, 并创建 `VarianceAccumulationBuffer`。源码见 `../warpX/Source/Particles/PhysicalParticleContainer.cpp:363-387`。

真正的多物种入口不在主 `Evolve()` 里, 而是 `MultiParticleContainer::DepositTemperatures()`:

1. 找出 `T_<species>`;
2. 清零;
3. 调 `pc->AccumulateVelocitiesAndComputeTemperature(T_vf, relative_time)`;

见 `../warpX/Source/Particles/MultiParticleContainer.cpp:645-670`。

这条线的工作前提也比普通 J 沉积更窄。 `DepositTemperature()` 直接要求:

- `current_deposition_algo == Direct`
- `push_type == Explicit`
- 关闭 shared-memory current deposition

否则 abort, 见 `../warpX/Source/Particles/PhysicalParticleContainer.cpp:1844-1858`。

内部统计对象不是“直接沉温度”, 而是:

- `n`
- `w`
- `wv`
- `w(v-\bar{v})^2`

更具体地, 当前实现硬编码走 `DOUBLE_PASS`:

1. 第一遍沉 sample count、权重和、加权速度和;

2. boundary sum;
3. 第二遍用第一遍得到的 $\bar{v}$ 再沉去均值二次矩;
4. 再按

$$\text{var} = \frac{n}{(n-1) \sum w} \sum w(v - \bar{v})^2$$

做 unbiased normalization;

5. 最后乘 m/k_B 变成 Kelvin。

源码位置:

- `../warpX/Source/Particles/PhysicalParticleContainer.cpp:1982-2188`
- `../warpX/Source/Particles/Deposition/TemperatureDeposition.H:25-115`
- `../warpX/Source/Particles/Deposition/VarianceAccumulationBuffer.cpp:79-118`

因此, 这条温度沉积主线更准确地说是:

- particle-to-grid weighted-moment deposition
- 再经过 boundary sum / filter / 归一化
- 最后得到 $T_x/T_y/T_z$

而不是“顺手从 J 推一个温度”。

5.12.3 这一节回头修正了本章的总边界

到这里, 本章里“粒子把东西交回网格”至少已经分成三类:

1. ρ/J
 - 服务 Maxwell 源项与离散连续性;
2. `MassMatrices_*`
 - 服务 implicit/JFNK 的线性响应近似;
3. `T_<species>` / variance buffers
 - 服务统计矩和温度 diagnostics。

它们的共同点是都共享 shape、tilebox、guard-cell 和 boundary-sum 的 deposition 工程骨架; 不同点是物理语义完全不同。

5.13 沉积后为什么还要同步

沉积 kernel 只把粒子贡献放到本地数组、tile、本 level 或 buffer 中。它还不能保证场求解器马上可以使用这些源项。主循环在 `../warpX/Source/Evolve/WarpXEvolve.cpp:555-561` 调 `SyncCurrentAndRho()`, 源码注释直接把它定义成:

- filter
- exchange guard cells
- interpolate across MR levels
- apply boundary conditions

但如果只停在这句注释, 本章会把真正的 source-synchronization 合同讲窄。更准确的分层如下。

5.13.1 PSATD 与 FDTD 的同步时序不同

`SyncCurrentAndRho()` 位于 `../warpX/Source/Evolve/WarpXEvolve.cpp:768-837`。

- PSATD + periodic single box
 - 即使启用了 current correction 或 Vay deposition, 也立即同步;
 - 若是 Vay, 则同步的名字就是 `current_fp_vay`, 不是 `current_fp`。
- PSATD + 非 periodic single box
 - 只有在
 - * 没开 `current_correction`
 - * 且不是 Vay deposition 时才立刻 `SyncCurrent("current_fp") + SyncRho()`;
 - 否则把更完整的同步推迟到 `PushPSATD`。
- FDTD
 - 直接 `SyncCurrent("current_fp") + SyncRho()`。

因此 source synchronization 的时序本身就是 solver-algorithm contract 的一部分，不是沉积后永远立刻做的固定尾声。

这里还有一条容易被一句话摘要吃掉的实现边界：PSATD + 非 periodic single box + Vay deposition 并不是“什么都不做，等后面统一处理”，而是会先对 `current_fp_vay` 单独做 filter，而且当前源码旁边直接留着注释：

```
// TODO This works only without mesh refinement
```

也就是说，`current_fp_vay` 在这条分支里当前仍带着“只假定无 MR”的现实边界。对第 5 章来说，这一点很重要，因为它说明 source synchronization 不只受 Maxwell solver 与 deposition algorithm 影响，还受当前实现是否已经覆盖 AMR 组合情形影响。

5.13.2 SyncCurrent() 的骨架：coarsen、buffer、owner mask、filter、SumBoundary

SyncCurrent() 的底层在 `../warpX/Source/Parallelization/WarpXComm.cpp:1174-1376`。

它的关键结构不是一句“通信电流”，而是：

1. 若开 `do_current_centering`
 - 先把 `current_fp_nodal` 中心化回 `staggered current_fp`。
2. finest-to-coarsest 迭代
 - 先把未过滤、未求和的 `J_fp` coarsen 成 `J_cp`。
3. 若有 `current_buf`
 - 先把 `J_cp` 加到 `buffer`，再统一作为跨 level 的通信源。
4. coarse level 接收 finer 源项时
 - 先 `ParallelAdd` 到临时 `fine_lev_cp`
 - 再用 `OwnerMask(...)` 去掉 nodal overlap / periodic overlap 的 double counting
 - 然后才并入本 level `J_fp`
5. 最后对每个 level 再做：
 - `ApplyFilterMF(...)`
 - `SumBoundaryJ(...)`

这里尤其要注意 owner-mask 这一步：它说明 fine-to-coarse 源项并不能直接并进当前 level `J_fp`，否则 nodal overlap 点会被重复加两次。

这里可以把源码里的三个中间对象明确分开：

1. `J_cp`
 - 表示当前 fine level 先 coarsen 下来的 coarse representation；
2. `mf_comm`
 - 表示“这一轮真正要跨 level 发送的源项”；
 - 若存在 `current_buf`，它其实 alias 到 `J_buffer`，也就是“buffer + coarsened current”的合并体；
 - 若不存在 `current_buf`，才直接等于 `J_cp`；
3. `fine_lev_cp`
 - 表示 coarse level 接收完 fine 源项后的临时落点；
 - 它不是最终要保留的场，而只是为了在并回 `J_fp` 之前先做 owner-mask 去重。

这层区分很重要，因为它说明 WarpX 的 fine/coarse 同步并不是“fine 直接往 coarse 主场里加一遍”，而是：

```
J_fp(fine)
-> coarsen 成 J_cp
-> 若有 current_buf 则先并进 buffer，得到 mf_comm
-> ParallelAdd 到 coarse 侧临时 fine_lev_cp
-> 用 OwnerMask 去重
-> 再并回 coarse 的 J_fp
```

只有这样，coarse-fine transition zone 和 nodal/periodic overlap 才不会在同一张 coarse 主场上被重复计数。

这里还有一个容易被“通信电流”四个字掩盖掉的结构：如果打开 `do_current_centering`，SyncCurrent() 的第一步并不是通信，而是先把 `current_fp_nodal` 中心化回 `staggered current_fp`。也就是说，某些 current 甚至在同层 box 间求和之前，都还没有处在 field solver 最终消费的 staggering 上。对成书叙述来说，这比简单说“同步后电流可用”更准确，因为它明确区分了：

1. nodal/staggered 源项重排；
2. same-level / fine-coarse 一致性；
3. filter 与 guard-cell 求和；
4. 边界条件。

SyncCurrent() 的最后一步也不只是“把所有 ghost cells 累加一遍”。SumBoundaryJ() 会先从 get_ng_depos_J() 出发, 再叠加 current centering 与 bilinear filter 真正需要的 stencil 宽度, 最后只对那部分 guard region 做 WarpXSumGuardCells(...)。因此这里的 same-level 同步范围并不是整个 nGrow, 而是“沉积和后续 filter 真正需要的最小安全区”。

5.13.3 SyncRho() 平行但不完全等于 SyncCurrent()

SyncRho() 位于 ../warpX/Source/Parallelization/WarpXComm.cpp:1385-1451, 其高层结构和平行于 SyncCurrent():

1. rho_fp -> rho_cp coarsen;
2. 若有 rho_buf, 先和 rho_cp 合并;
3. coarse level 也通过临时 fine_lev_cp + OwnerMask 去重后再并回 rho_fp;
4. 最后对每个 level 调 ApplyFilterandSumBoundaryRho(...).

但它和 current 不完全相同:

- 没有 do_current_centering
- 过滤和求和由 ApplyFilterandSumBoundaryRho(...) 统一处理

后者在 ../warpX/Source/Parallelization/WarpXComm.cpp:1677-1692 中:

- 若 use_filter
 - 先把 filtered rho 写进临时 rf
 - 再用 WarpXSumGuardCells(rho, rf, ...) 把 filtered 值并回
- 否则直接 WarpXSumGuardCells(rho, ...)

所以 SyncCurrent() 与 SyncRho() 虽然共享 fine/coarse + buffer + owner-mask 的大骨架, 但具体 filter/sum 实现并不一样。

更进一步说, rho 这条线比 current 少了一层 staggering 重排, 但并没有因此退化成“更简单的数组相加”。ApplyFilterandSumBoundaryRho(...) 先决定是否构造 filtered 临时 rf, 然后再用 WarpXSumGuardCells(...) 把 filtered 或 unfiltered 结果合并回去。也就是说, rho 的 source synchronization 也同样把“物理要不要 filter”写进了同步合同本身, 而不是在同步完成后额外修饰。

rho 路径里也有和 current 完全平行的“临时接收层”问题: fine level 传下来的 rho_cp 或 rho_buf + rho_cp 并不是直接加回 coarse rho_fp, 而是同样先落到临时 fine_lev_cp, 再通过 OwnerMask(...) 选出 coarse patch 上真正该保留的那一份。也就是说, fine/coarse 去重并不是 current 独有技巧, 而是 source synchronization 对所有守恒源项都共享的一条骨架。

5.13.4 SyncCurrentAndRho() 的最后一步其实是边界条件

顶层同步完成后, SyncCurrentAndRho() 还会继续对:

- fine patch 的 rho_fp/current_fp
- coarse patch 的 rho_cp/current_cp

调用:

- ApplyRhofieldBoundary(...)
- ApplyJfieldBoundary(...)

源码见 ../warpX/Source/Evolve/WarpXEvolve.cpp:815-836。

因此这条函数的完整职责不是:

- “做完 guard-cell 通信就结束”

更准确地说, 这里调用的也不是抽象的“边界后处理”。WarpXEvolve.cpp 在注释里直接写的是:

- Reflect charge and current density over PEC boundaries, if needed.

而 ../warpX/Source/BoundaryConditions/WarpX_PEC.H 又把底层语义钉得更死:

- ApplyReflectiveBoundarytoRhofield(...): 把沉积到 PEC 外侧的电荷密度反射回计算域;
- ApplyReflectiveBoundarytoJfield(...): 把沉积到 PEC 外侧的电流密度反射回计算域。

这意味着 ApplyRhofieldBoundary(...) / ApplyJfieldBoundary(...) 在 source synchronization 里承担的不是“最后顺手修一下边界层”, 而是把已经完成 same-level 求和、fine/coarse 合并、filter 和 OwnerMask 去重后的源项, 再按实际场边界几何物化成可供 Maxwell solver 消费的 patch-local rho/J。如果边界是 PEC, 那一步会把落到导体外侧或 guard 区的沉积量, 按镜像位置与分量符号规则折回域内; 如果不是需要反射的边界, 则这里才退化成相对平直的 boundary pass。

还有一个容易被一句话略过的实现点：WarpX 并不是只对 fine patch 做这件事，而是按 patch 类型分别处理：

- level lev 上始终对 fine patch 的 rho_fp/current_fp 调 PatchType::fine
- 当 lev > 0 时，再对 coarse patch 的 rho_cp/current_cp 调 PatchType::coarse

也就是说，source synchronization 的终点不是“得到一份全局一致的守恒源项”，而是“得到 fine/coarse 两套都已经与边界条件相容的源项 MultiFab”。这对后面的场推进很关键，因为 solver 消费的是 patch-local rho/J，不是一个抽象的、尚未带边界语义的守恒和。

而是：

1. solver/algorithm 条件分叉；
2. same-level 与 fine/coarse 源项同步；
3. filter；
4. boundary reflection / mirror / PEC-compatibility。

所以完整源项路径更准确地写成：

```
particle trajectory
-> tile-level charge/current deposition
-> species sum
-> MultiFab/local or buffer accumulation
-> filter / guard-cell exchange / AMR synchronization
-> boundary conditions
-> field solver
```

漏掉：

- owner-mask 去重、
- fine/coarse buffer 合并、
- filter 的时序、
- 或最后的 ApplyRhofieldBoundary/ApplyJfieldBoundary

都会把 WarpX 的 source synchronization 合同讲错。

5.13.5 本章对应的两条关键 regression，分别在验证哪一层 source-synchronization 合同

到这里，source synchronization 的源码主链已经闭合；对应的本地 regression 也可以更明确地挂回这条链，而不只当成零散 smoke test。

第一条是 Examples/Tests/langmuir/ 里的 PSATD + current_correction 变体。它不是只看 $\text{divE} - \rho/\epsilon_0$ ，而是两层断言同时成立：

1. Ex/Ey/Ez 或 Ex/Ez 仍要解析 Langmuir-wave 场解匹配；
2. analysis_utils.py 在 current_correction 路径下还会追加 $\text{divE} - \rho/\epsilon_0$ 检查，容差固定放宽到 $1e-9$ 。

因此它验证的不是“某个 deposition kernel 单独正确”，而是：

- Esirkepov deposition
- PSATD current correction
- 立即同步后的源项一致性

这三层组合后，解析波解和离散 Gauss law 都没有被破坏。

第二条是 Examples/Tests/vay_deposition/。这组测试更窄，也更直接：它不再对照解析波，只断言

$$\frac{\max |\nabla \cdot E - \rho/\epsilon_0|}{\max |\rho/\epsilon_0|} < 10^{-3}.$$

在当前输入里，这条断言专门落在

- algo.current_deposition = vay
- algo.maxwell_solver = psatd
- warpx.grid_type = collocated

这组实现边界上。结合 `SyncCurrentAndRho()` 的源码分叉, `vay_deposition` 真正测到的是: `current_fp_vay` 在 `filter-first`、后续再交给 `PSATD` 同步链的专门路径里, 最终是否仍能保住离散 `Gauss law`。

所以对本章来说, 这两组 `regression` 的职责应分开理解:

- `Langmuir + current_correction`
 - 更宽, 测 `physics solution + source consistency`;
- `vay_deposition`
 - 更窄, 测 `specialized source-synchronization path` 本身。

这样第 5 章的验证闭环就不再只是“有些测试目录可以参考”, 而是已经能说清: 哪一条 `regression` 在替本章的哪一层 `source contract` 做断言。

5.14 形函数、guard cells 与稳定性

更高阶 `shape` 会影响三个方面:

1. `gather/deposition stencil` 更宽, 需要更多 `guard cells`;
2. 粒子噪声降低, 但局部性和通信成本增加;
3. 在边界、`AMR coarse-fine interface`、`PML` 和 `embedded boundary` 附近, `shape` 的截断或修正会影响守恒。

源码中 `current deposition` 和 `charge deposition` 都会检查 `shape` 是否能放进 `guard cells`。例如 `current deposition` 在 `WarpXParticleContainer.cpp:416-446` 计算 `shape_extent` 和 `range`; `shared-memory charge deposition` 在 `WarpXParticleContainer.cpp:1527-1556` 做类似检查, 普通 `charge deposition` 则经 `Source/ablastr/particles/DepositCharge.H` 的桥接路径处理本地 `tile` 与 `guard` 区。这些检查不是性能细节, 而是物理离散化安全条件。

当前 `checkout` 的 `geometry/order` 分派可以压成下面这张证据表。表中的“源码覆盖”来自 `scripts/audit_deposition_geometry_order_contract` 的 53 个锚点; “运行证据”只列已经实际运行过的组合, 不能由源码入口自动推导。

路径	源码覆盖	当前运行证据	仍未关闭的边界
<code>DepositCharge()</code> <code>ordinary/shared</code>	<code>1D_Z</code> , <code>XZ</code> , <code>RZ</code> , <code>RCYLINDER</code> , <code>RSPHERE</code> , <code>3D</code> ; <code>shape</code> <code>1/2/3/4</code>	<code>1D/2D/3D</code> <code>charge/Gauss-law siblings</code> ; <code>2D shape 1/2/3/4</code> ; <code>RZ</code> <code>charge/inverse-volume</code>	<code>RCYLINDER/RSPHERE</code> 的逐阶 <code>charge/Gauss-law</code> <code>runtime</code> 矩阵仍不完整
<code>Direct current</code>	<code>1D_Z</code> , <code>XZ</code> , <code>RZ</code> , <code>RCYLINDER</code> , <code>RSPHERE</code> , <code>3D</code> ; <code>shape</code> <code>1/2/3/4</code> ; <code>implicit</code> 入口	既有 <code>Langmuir</code> 、 <code>Vay/Direct</code> 相关回归和源码 <code>contract</code>	不能据此推出所有几何、边界裁剪和 <code>implicit</code> 组合等价
<code>Esirkepov</code>	<code>shape 1/2/3/4</code> ; 显式与 <code>implicit skeleton</code>	<code>1D/2D/3D Langmuir</code> ; <code>2D</code> <code>shape 1/2/3/4</code> ; <code>RCYLINDER/RSPHERE</code> <code>shape 1/2/3/4</code> 径向 <code>Er</code> ; <code>RZ</code> <code>Er/Ez field PASS</code> ; <code>2D MR</code> 为 <code>BOUNDARY</code>	<code>RZ charge residual</code> 为 <code>BOUNDARY</code> ; <code>RCYLINDER/RSPHERE</code> 的 <code>charge/Gauss-law</code> 与完整 <code>AMR route-count</code> 仍未形成 强 <code>runtime</code> 闭环
<code>Villasenor</code>	<code>shape 1/2/3/4</code> ; 显式与 <code>implicit skeleton</code>	<code>2D implicit native</code> , <code>filtered</code> , <code>shape=4</code> <code>cropping</code> , <code>PICMI</code> ; 公式级 <code>contract</code>	<code>RZ</code> 因 <code>PETSc/build</code> 边界未 形成运行级证据, 其他几何/阶数组 合仍需逐项核对
<code>Vay</code>	<code>shape 1/2/3/4</code>	既有 <code>vay_deposition</code> <code>regression</code>	几何与边界裁剪的全组合覆盖仍未完 成

因此, 本节现在可以安全地说“当前源码提供了哪些分派入口”, 但不能把它缩写成“所有入口都已验证”。尤其是 `RCYLINDER/RSPHERE` 只在 `source contract` 中确认了编译期 `geometry branch`; 它们与 `RZ` 的坐标压缩、逆体积和模式写回语义不能互相替代。

`RZ + Esirkepov` 还需要单独保留一个诊断边界: 当前 `2-rank case` 的 `Er/Ez field contract` 通过, 但同面 `divE-rho/epsilon0 residual` 为 `3.593e-3`, 而官方 `analysis_utils.py` 本来也明确跳过 `RZ Esirkepov` 的强 `charge gate`。原因不能简单归结为“kernel 已经错误”: `DivEFunctor` 与 `RhoFunctor` 分别从场求解器和重新沉积的 `charge density` 构造诊断量, 再经过各自的 `node/cell`, `mode` 与插值路径。完整源码语义与可复现命令见 `notes/code-reading/particles/46-rz-esirkepov-charge-boundary.md`; 本书将其标为 `BOUNDARY`, 不把 `field PASS` 升级为守恒 `PASS`。

同一条证据又做了 `warpx.do_dive_cleaning=1/0` 的 paired control: 全局 charge residual 从 $3.593\text{e-}3$ 增至 $9.693\text{e-}2$, 约为 26.98 倍; 第一径向 cell 之外的 residual 则为 $4.293\text{e-}4/6.540\text{e-}12$ 。两个 case 的全局最大值都由 axis cell 主导, Er/Ez field error 也改变为 $2.427\text{e-}2/4.941\text{e-}3$ 。这说明边界同时对 axis treatment 和 cleaning 路径敏感, 但不能据此把 cleaning 认定为唯一根因; 比较结果归档为 `AXIS_DOMINATED_CLEANING_SENSITIVE_DIAGNOSTIC_BOUNDARY`。

进一步只切换 `boundary.verboncoeur_axis_correction`: 在 `64x128, particle_shape=1` case 中, 开启时 charge residual 为 $3.593\text{e-}3$, 关闭时降为 $5.513\text{e-}12$, 且 off-axis residual 为 $1.720\text{e-}12$; 两者 Er/Ez field error 都低于 0.12。因此这个 `shape=1` case-local control 可以恢复双 gate, 但不足以要求修改 WarpX 默认值; 完整低分辨率对照和参数语义见 `notes/code-reading/particles/46-rz-esirkepov-charge-boundary.md`。

在默认轴修正保持开启的条件下, RZ Esirkepov Langmuir 的 `shape=1/2/3/4` field runtime coverage 也已补齐: Er 最大相对误差分别为 $1.075\text{e-}2/5.703\text{e-}2/8.694\text{e-}2/1.167\text{e-}1$, 全部低于 0.12; 对应 charge residual 为 $3.593\text{e-}3/4.306\text{e-}3/4.341\text{e-}3/4.433\text{e-}3$, 且最大值均由 axis cell 主导。汇总报告见 `runs/stage-c-validation/esirkepov_langmuir_rz_shape-matrix/contract.{json,md}`; 因此这里关闭的是 field shape coverage 缺口, 不是 RZ charge boundary。

将同样的 `shape=2/3/4` case 切换到 `verboncoeur_axis_correction=false` 后, charge residual 降到 $2.202\text{e-}12/2.103\text{e-}12/2.063\text{e-}12$, 但 Er field error 升到 $0.132/0.173/0.213$, 超过 0.12。因此轴修正对照不是一个可以全局套用的“修复开关”, 而是随 shape 改变的 charge/field tradeoff; 矩阵归档于 `runs/stage-c-validation/esirkepov_langmuir_rz_shape-axis-matrix/contract.{json,md}`。

随后把 `shape=1` 的同一 paired case 加密到 `128x256`: correction-on 的 Er/Ez field error 为 $2.797\text{e-}2/3.049\text{e-}2$, axis charge residual 降至 $1.520\text{e-}3$; correction-off 的 Er/Ez 为 $2.809\text{e-}2/3.662\text{e-}2$, charge residual 为 $9.353\text{e-}12$, field/charge 双 gate 均通过。四格汇总见 `runs/stage-c-validation/esirkepov_langmuir_rz_axis-resolution-comparison/contract.{json,md}`。脚本为 `scripts/summarize_rz_esirkepov_axis_resolution_contract.py`。这组结果只支持 resolution/axis-correction/shape interaction 的分层解释, 不替代跨 geometry/order 的统一强守恒合同。

同一汇总脚本随后参数化到 `shape=2`。 `64x128` correction-off 的 Er/Ez field error 为 $0.1323/1.445\text{e-}2$, charge residual 为 $2.202\text{e-}12$; 加密到 `128x256` 后, correction-off 的 Er/Ez 降至 $9.318\text{e-}3/7.775\text{e-}3$, charge residual 为 $9.644\text{e-}12$, field/charge 双 gate 均通过。correction-on 的高分辨率 field error 为 $9.321\text{e-}3/5.154\text{e-}3$, 但 axis charge residual 仍为 $2.177\text{e-}3$ 。因此 `shape=2` 的 coarse field failure 可以归为分辨率边界, 但 correction-on charge boundary 仍未闭合; 报告见 `runs/stage-c-validation/esirkepov_langmuir_rz_shape2_axis-resolution-comparison/contract.{json,md}`, 详细说明见 `notes/code-reading/particles/53-rz-esirkepov-shape2-resolution-contract.md`。

最后把 correction-off 的相同 resolution 对照扩展到 `shape=3/4`。三档 coarse Er field error 为 $0.1323/0.1734/0.2134$, 而 `128x256` refined sibling 降至 $9.318\text{e-}3/1.113\text{e-}2/1.365\text{e-}2$; 对应 refined charge residual 为 $9.644\text{e-}12/6.086\text{e-}12/6.724\text{e-}12$, 三档 field/charge 双 gate 全部通过。`scripts/summarize_rz_esirkepov_shape_resolution_family.py` 将这三条 case 收成 family contract, 报告见 `runs/stage-c-validation/esirkepov_langmuir_rz_shape-resolution-family/contract.{json,md}`。这关闭的是 higher-shape coarse field boundary 的分辨率诊断, 不关闭 correction-on axis charge residual, 也不外推到其他 geometry、AMR 或 implicit 路径。

最后将 `shape=2/3/4` 的 correction-on refined siblings 也纳入同一完整矩阵: Er/Ez field errors 分别为 $9.321\text{e-}3/5.154\text{e-}3$ 、 $9.342\text{e-}3/6.392\text{e-}3$ 和 $1.079\text{e-}2/7.454\text{e-}3$, 均通过 0.12 field gate; 但 axis charge residual 仍为 $2.177\text{e-}3/2.353\text{e-}3/2.552\text{e-}3$ 。相比之下, correction-off refined charge residual 为 $9.644\text{e-}12/6.086\text{e-}12/6.724\text{e-}12$ 。完整矩阵由 `scripts/summarize_rz_esirkepov_shape_resolution_family.py` 生成, 报告见 `runs/stage-c-validation/esirkepov_langmuir_rz_axis-correction-family/contract.{json,md}`。因此当前第 5 章可以明确区分: 高阶 shape refined field 已闭合, correction-off charge 在 case-local sibling 上闭合, 而 correction-on axis charge 仍是独立诊断边界。

为了避免把不同诊断量混成一个结论, v0.47 又对上述 `shape=2/3/4` correction-on refined sibling 直接读取 `rho`、`rho_electrons` 和 `rho_ions`。末态 `rho=(rho_electrons+rho_ions)` 的相对差分别为 $1.303\text{e-}14/1.228\text{e-}14/1.343\text{e-}14$, 说明 species decomposition 在 rho-side 已达到机器精度; 三个 case 的 integrated-rho 时间序列漂移分别为 $2.371\text{e-}6/2.729\text{e-}6/3.354\text{e-}6$, 只作为可复核观测记录。这个结果不能替代 `divE-rho`、current closure 或完整 Gauss-law contract: 同一批 case 的 axis `divE-rho` residual 仍为 $2.177\text{e-}3/2.353\text{e-}3/2.552\text{e-}3$ 。脚本为 `scripts/analyze_rz_esirkepov_rho_observable.py`, 报告见 `runs/stage-c-validation/esirkepov_langmuir_rz_rho-observable/contract.{json,md}`。

同一诊断也已扩展到 `RCYLINDER/RSPHERE shape=1`: 默认 correction 下 charge residual 为 $4.711\text{e-}3/4.166\text{e-}2$, 关闭后为 $3.505\text{e-}12/2.420\text{e-}11$ 。这表明 `RCYLINDER` 的 axis correction off 可以恢复当前强 gate, 而 `RSPHERE` 虽明显改善仍略超 $1\text{e-}11$; 两者均不支持直接修改全局默认值, 完整对照见 `runs/stage-c-validation/esirkepov_radial_charge_axis-comparison/contract.{json,md}`。

`RSPHERE` 的 `64/128` resolution paired control 进一步显示: correction on 的 residual 为 $4.166\text{e-}2/1.390\text{e-}2$, correction off 为 $2.420\text{e-}11/9.843\text{e-}11$; 四个 field gate 都通过, 但四个 charge gate 都未闭合。因此这条证据只能说明 axis/resolution 组合敏感, 不能替代正式收敛研究或作为全局默认参数修改依据。

这组径向结果的源码合同现在也已单独验收: `scripts/audit_radial_axis_volume_contract.py` 固定了 `boundary.verboncoeur_axis_correction`。

的默认值和解析入口，确认 RZ/RCYLINDER 使用 1/3 对 1/4、RSPHERE 使用 1/4 对 1/8 的轴体积因子，并确认 `ApplyInverseVolumeScalingToChargeDensity()` 在 `rho_fp` 与 `rho_buf` 路径中的调用时机。因而本节的准确边界是：径向 `field shape coverage` 已有运行证据，`charge residual` 的轴体积/诊断耦合也有源码映射，但尚未形成跨 `geometry`、`shape`、`resolution` 的统一强守恒合同。

5.15 本章结论

沉积的物理底线是离散连续性方程。WarpX 的工程实现把它拆成多层：

```
flowchart TD
    A["PhysicalParticleContainer::Evolve"] --> B["rho component 0 before push"]
    A --> C["PushPX"]
    C --> D["x and u advanced"]
    D --> E["DepositCurrent relative_time=-0.5 dt"]
    D --> F["rho component 1 after push"]
    E --> G["WarpXParticleContainer::DepositCurrent"]
    F --> H["WarpXParticleContainer::DepositCharge"]
    G --> I["Esirkepov / Villasenor / Vay / Direct"]
    H --> J["charge deposition shape kernels"]
    I --> K["SyncCurrentAndRho"]
    J --> K
    K --> L["field solver"]
```

对当前项目推进而言，本章已经可以用这张责任矩阵把 `current kernel`、`charge kernel` 和同步层的边界稳定地交给读者；`DepositCharge()` / `ABLASTR` / `ChargeDeposition.H` 的主要职责边界也已达到 v0.40 内部阶段闭合。剩余工作应集中在出版级证据和表达，而不是继续扩大局部 `kernel` 的职责：

1. 取得合法的 CPC publisher PDF 后，完成 Esirkepov 2001 的 abstract、section numbering、Eq. (23) 和二阶 spline 段落的 bounded compare；
2. 在取得合法 CPC publisher PDF 后，完成 Esirkepov 2001 与预印本的 bounded compare；
3. 对本章源码路径、公式编号和宽表格做最终出版级精修，并继续补足尚未覆盖的 `geometry/order` 分支，再转向后续尚未闭合的成书模块。

5.16 练习与源码定位

1. 连续性方程题：从 `rho` 的 old/new 时间层出发，解释为什么 Direct current deposition 不能自动保证 $\Delta t \operatorname{div}_h \mathbf{J} = \rho_{\text{old}} - \rho_{\text{new}}$ ，而 Esirkepov/Villasenor 必须引入轨迹或 crossing 信息。
2. 源码定位题：分别定位 `DepositCharge()`、`DepositCurrent()` 和 `SyncCurrentAndRho()` 的入口，给每个函数写出一个“它负责什么”和一个“它不负责什么”的边界。
3. 公式复核题：运行 `python scripts/verify_esirkepov_density_decomposition.py`，再对照 `notes/code-reading/particles/4` 说明公式级恒等式通过为什么仍不能替代端到端 Gauss-law regression。
4. Villasenor 几何题：运行 `python scripts/verify_villasenor_formula_contract.py --samples 10000`，解释四边界 flux closure、crossing-split displacement closure 和 Eq.(36) 三维 volume closure 分别验证了哪一层，为什么仍不能推出 `CurrentDeposition.H` 的所有 `geometry/order` 分支等价。

6. 电磁场求解器

本章当前依据的 WarpX 源码版本是：

- 只读源码路径：`../warpx`
- 分支：`pkuHEDPbranch`
- commit：`8c488b1a9`

v0.6 校准说明：本章已把场推进主入口、FDTD stencil、PSATD/JRhom 谱推进、PML damping 和 regression 入口重新核到上述 checkout，并补入读者侧分派图和求解器对照表。旧版草稿里仍保留较长的理论和代码精读段落；若后续 WarpX 更新，优先重核下表中的入口，再更新小节中的源码块。

主题	当前入口	v0.6 证据边界
主时间步分支	../warpx/Source/Evolve/WarpXEvolve.cpp:564-644	Symplectic4thOrderRho() 后, PSATD 走 PushPSATD(), FDTD 走 EvolveB(dt/2) -> EvolveE(dt) -> EvolveB(dt/2)
PSATD 推进	../warpx/Source/FieldSolver/WarpXPushFieldsEM.cpp:771-943	PushFieldsEM() 中 Vay 14-43, J/rho 谱变换、PSATDPushSpectralFields()、PML push 和边界回填
FDTD B/E 分派	../warpx/Source/FieldSolver/WarpXPushFieldsFDTD.cpp:945-1045	PushFieldsFDTD() 中 m_fDTD_solver_fp/cp, 并在 PML 打开时调用 EvolveBPML() / EvolveEPML()
FDTD kernel	../warpx/Source/FieldSolver/FiniteDifferenceKernel.cpp:53-225 与 EvolveE.cpp:55-235	FiniteDifferenceKernel 中 Upward/Downward 差分算子
SpectralSolver 分派	../warpx/Source/FieldSolver/SpectralSolver.cpp:26-143	PushFieldsSpectralSolver 中 first-order JRhom、second-order JRhom 选择具体 PsatdAlgorithm*
JRhom 外层循环	../warpx/Source/Evolve/WarpXEvolve.cpp:343-1008	粒子位置 13-19, 1008 粒子位置 13-19, 谱推进并回填平均场
PML split fields	../warpx/Source/BoundaryConditions/WarpXEvolvePML.cpp:46-365、 PML.cpp:582-1197	ns/PMLComponent 中 damping 因子、电流 damping、常规场与 PML 场交换
验证入口	../warpx/Examples/Tests/pml/	Yee、CKC、PSATD、Galilean PSATD、RZ PSATD 和 restart PML regression

flowchart TD

```

A["OneStep_nosub: particles pushed and J/rho synchronized"] --> B{"algo.maxwell_solver"}
B -->|"Yee / CKC / HybridPIC / ECT"| C["FDTD branch"]
C --> D["EvolveF/G half step"]
D --> E["EvolveB(dt/2)"]
E --> F["FillBoundaryB"]
F --> G["EvolveE(dt) or MacroscopicEvolveE(dt)"]
G --> H["FillBoundaryE"]
H --> I["EvolveF/G half step"]
I --> J["EvolveB(dt/2)"]
J --> K{"do_pml"}
K -->|"yes"| L["DampPML and fill moving-window guards"]
K -->|"no"| M["safe guard-cell fill if requested"]
B -->|"PSATD"| N["PushPSATD"]
N --> O["Correct/Vay-transform J and rho"]
O --> P["FFT E/B and optional F/G"]
P --> Q["SpectralSolver::pushSpectralFields"]
Q --> R["Inverse FFT and optional averaged fields"]
R --> S{"PML enabled"}
S -->|"yes"| T["PML::PushPSATD or PML_RZ::PushPSATD"]
S -->|"no"| U["Apply field boundaries"]
A --> V{"psatd.JRhom"}
V -->|"enabled"| W["OneStep_JRhom: skip normal deposition, redeposit J/rho over subintervals"]
W --> Q

```

求解器路径	输入/开关	主源码入口	数值含义	读者检查点
Yee FDTD	algo.maxwell_solver = yee, staggered grid	FiniteDifferenceSolver/FiniteDifferenceCartesianYeeAlgorithm.H EvolveE.cpp、	交错/错开更新, 受 CFL 限制	EvolveB(dt/2) -> EvolveE(dt) -> EvolveB(dt/2) 是否和主循环顺序一致
CKC FDTD	algo.maxwell_solver = ckc	CartesianCKCAlgorithm.H 经同一 EvolveB/E 模板实例化	扩展 Stencil 降低数值色散	B 更新是否走 CKC 的横向加权 Upward 算子
Nodal FDTD	collocated/nodal grid	CartesianNodalAlgorithm.H 经同一 EvolveB/E 模板实例化	与 Yee 交错, 差分算子变为中心形式	UpwardDx 与 DownwardDx 是否退化为一中心差分
标准 PSATD	algo.maxwell_solver = psatd, v_galilean = 0	WarpX::PushPSATD()、Fourier SpectralSolver.cpp、Maxwell 线性部分 PsatdAlgorithmGalilean.cpp	空间解析推进	J/rho 是否先完成 current correction 或 Vay deposition
Galilean PSATD	psatd.v_galilean 非零	SpectralSolver.cpp:75-82 PsatdAlgorithmGalilean.cpp	Galilean 坐标中降低 frame NCI	current correction 是否使用 Galilean 连续性公式
PML FDTD	field boundary 为 PML 且非 PSATD 路径	EvolveBPML.cpp、EvolveEPML.cpp、WarpXEvolvePML.cpp	split-field curl 更新加 sigma damping	split components 是否经 PML::Exchange() 回填常规场
PML PSATD	PSATD 与 PML 同时打开	PML::PushPSATD()、PML 区域单独谱推进或 PsatdAlgorithmPml.cpp、PML_RZ.cpp	PML 谱推进	PML push 是否发生在主域 PSATDPushSpectralFields() 之后
JRhom PSATD	psatd.JRhom = CL1/LQ4/...	WarpX::OneStep_JRhom()、PsatdAlgorithmJRhom.cpp	个 PIC 步内多次沉积 J/rho 并用多项式源项推进	是否禁用 Vay/Galilean/current correction, 并按子区间重沉积

电磁 PIC 的场求解器离散 Maxwell 方程。显式 FDTD 的经典代表是 Yee 算法：电场和磁场在空间上交错，在时间上也交错。抽象地写，

$$\begin{aligned}\mathbf{B}^{n+1/2} &= \mathbf{B}^n - \frac{\Delta t}{2} \nabla_h \times \mathbf{E}^n, \\ \mathbf{E}^{n+1} &= \mathbf{E}^n + c^2 \Delta t \nabla_h \times \mathbf{B}^{n+1/2} - \frac{\Delta t}{\epsilon_0} \mathbf{J}^{n+1/2}, \\ \mathbf{B}^{n+1} &= \mathbf{B}^{n+1/2} - \frac{\Delta t}{2} \nabla_h \times \mathbf{E}^{n+1}.\end{aligned}$$

这正对应 WarpX::OneStep_nosub 中的 FDTD 路径：EvolveB(dt/2)、EvolveE(dt)、EvolveB(dt/2)。本机源码位置是 ../warpx/Source/Evolve/WarpXEvolve.cpp:606-643，其中三次核心推进调用位于 :612、:617 和 :628。

WarpX 的场推进封装在 ../warpx/Source/FieldSolver/WarpXPushFieldsEM.cpp。其中：

- WarpX::EvolveB 在 :945-996：按 level 和 patch type 调用 FDTD solver 或 PML solver 的 B 更新。
- WarpX::EvolveE 在 :999-1045 起：按 level 和 patch type 调用 E 更新，并处理 PML 和电荷守恒相关字段。
- WarpX::PushPSATD 在 :771-943：处理 PSATD current correction、Vay deposition、谱空间 transform、PML push 和边界回填。

真正的 FDTD stencil 在 Source/FieldSolver/FiniteDifferenceSolver/ 中，例如 EvolveB.cpp、EvolveE.cpp、EvolveBPML.cpp、EvolveEPML.cpp。当前已新增第一篇源码精读 notes/code-reading/fieldsolver/00-fieldsolver-dispatch.md，开始逐块展开这些文件。

WarpX::EvolveB() 的顶层路由在 ../warpx/Source/FieldSolver/WarpXPushFieldsEM.cpp:945-996：

```

void
WarpX::EvolveB (int lev, PatchType patch_type, amrex::Real a_dt, SubcyclingHalf subcycling_half, amrex::Real
{
    // Evolve B field in regular cells
    if (patch_type == PatchType::fine) {
        m_fdttd_solver_fp[lev]->EvolveB( m_fields,
                                          lev,
                                          patch_type,
                                          m_flag_info_face[lev], m_borrowing[lev], a_dt );
    } else {
        m_fdttd_solver_cp[lev]->EvolveB( m_fields,
                                          lev,
                                          patch_type,
                                          m_flag_info_face[lev], m_borrowing[lev], a_dt );
    }
}

```

FiniteDifferenceSolver::EvolveB() 的 Cartesian 主 kernel 在 ../warpX/Source/FieldSolver/FiniteDifferenceSolver/Evo

```

Bx(i, j, k) += dt * T_Algo::UpwardDz(Ey, coefs_z, n_coefs_z, i, j, k)
              - dt * T_Algo::UpwardDy(Ez, coefs_y, n_coefs_y, i, j, k);

By(i, j, k) += dt * T_Algo::UpwardDx(Ez, coefs_x, n_coefs_x, i, j, k)
              - dt * T_Algo::UpwardDz(Ex, coefs_z, n_coefs_z, i, j, k);

Bz(i, j, k) += dt * T_Algo::UpwardDy(Ex, coefs_y, n_coefs_y, i, j, k)
              - dt * T_Algo::UpwardDx(Ey, coefs_x, n_coefs_x, i, j, k);

```

这就是

$$\partial_t \mathbf{B} = -\nabla_h \times \mathbf{E}.$$

FiniteDifferenceSolver::EvolveE() 的 Cartesian 主 kernel 在 ../warpX/Source/FieldSolver/FiniteDifferenceSolver/Evo

```

Ex(i, j, k) += c2 * dt * (
    - T_Algo::DownwardDz(By, coefs_z, n_coefs_z, i, j, k)
    + T_Algo::DownwardDy(Bz, coefs_y, n_coefs_y, i, j, k)
    - PhysConst::mu0 * jx(i, j, k) );

```

它对应

$$\partial_t \mathbf{E} = c^2 (\nabla_h \times \mathbf{B} - \mu_0 \mathbf{J}).$$

如果开启 do_dive_cleaning, EvolveE() 还会加入 grad(F); EvolveF.cpp 更新

$$\partial_t F = \nabla_h \cdot \mathbf{E} - \rho / \epsilon_0.$$

如果开启 do_divb_cleaning, EvolveG.cpp 更新

$$\partial_t G = c^2 \nabla_h \cdot \mathbf{B},$$

并由 EvolveB.cpp 中的 +grad(G) 反馈到磁场。

PSATD 的思路不同: 在 Fourier 空间中, Maxwell 方程的线性部分可以在一个时间步内解析积分。这样可以显著降低数值色散, 尤其适合激光等离子体加速、boosted frame 和长距离传播问题。代价是并行分解、边界、PML、current correction 和多层 AMR 的实现更复杂。WarpX 的 OneStep_nosub 对 PSATD 单独分支: ../warpX/Source/Evolve/WarpXEvolve.cpp:578-604 调用 PushPSATD、PML damping 和谱场回填。

电磁求解器的稳定性首先受 CFL 条件约束。对标准 Yee 网格，时间步必须小于电磁波跨越网格的稳定上限。WarpX 输入中可通过 `warpx.cfl` 控制 CFL 系数；Langmuir 示例使用 `warpx.cfl = 0.8`，uniform plasma 示例使用 `warpx.cfl = 1.0`。

一个常见误区是只把 field solver 当作 Maxwell 方程更新器。真实代码还必须处理：

- guard cell 填充；
- nodal point 同步；
- PML 阻尼；
- current filtering；
- divergence cleaning；
- embedded boundary；
- macroscopic medium；
- PSATD 的谱空间电流校正。

因此，写“场求解器正确”不能只看 `EvolveE.cpp` 和 `EvolveB.cpp`。必须同时检查它们在主循环中的调用时间、输入的 `J` 是否已同步、边界和 guard cells 是否处在正确状态。

6.1 FDTD 差分算子：Yee、Nodal 与 CKC

`notes/code-reading/fieldsolver/01-fdtd-evolve-e-b.md` 已经把 `T_Algo::Upward/Downward` 展开到算法头文件。Yee 的 `UpwardDx` 和 `DownwardDx` 分别是 staggered forward/backward difference：

```
return inv_dx*( F(i+1,j,k,ncomp) - F(i,j,k,ncomp) );
```

```
return inv_dx*( F(i,j,k,ncomp) - F(i-1,j,k,ncomp) );
```

Nodal grid 没有 Yee 那种 E/B 空间交错，所以 `UpwardDx` 是中心差分，`DownwardDx` 直接调用同一个函数：

```
return 0.5_rt*inv_dx*( F(i+1,j,k,ncomp) - F(i-1,j,k,ncomp) );
```

CKC 的关键差异是 `Upward` 使用横向邻点加权扩展 stencil，而 `Downward` 保持局部 backward difference。这对应理论文档中的

$$D_t \mathbf{B} = -\nabla^* \times \mathbf{E}, \quad D_t \mathbf{E} = \nabla \times \mathbf{B} - \mathbf{J}.$$

因此同一个 `EvolveB.cpp` 模板 kernel 在传入 `CartesianYeeAlgorithm`、`CartesianNodalAlgorithm` 或 `CartesianCKCAlgorithm` 时，会得到不同的离散 curl。

本章当前引用的核心文献线索是 Yee、GodfreyJCP2014_PSATD、Lehe2016、VayJCP2013。后续需要将 PSATD 相关论文用 MinerU 转换成 Markdown，并补一节 Galilean PSATD 与数值 Cherenkov 不稳定性的推导。

6.2 FDTD PML split-field 更新

PML 的目标是在计算区域边缘吸收入射电磁波。Berenger PML 的基本做法不是简单给整个场乘阻尼，而是把场分量拆成不同方向的 split components，并对这些分量施加匹配吸收。WarpX 的 FDTD PML 第一层实现见 `notes/code-reading/fieldsolver/02-fdtd-pml.md`。

PML split components 的 component 编号定义在 `../warpx/Source/BoundaryConditions/PMLComponent.H:8-18`：

```
/* In WarpX, the split fields of the PML (e.g. Eyx, Eyz) are stored as
 * components of a MultiFab (e.g. component 0 and 1 of the MultiFab for Ey)
 * The correspondence between the component index (0,1) and its meaning
 * (yx, yz, etc.) is defined in the present file */
```

```
struct PMLComp {
    enum { xy=0, xz=1, xx=2,
          yz=0, yx=1, yy=2,
          zx=0, zy=1, zz=2,
          x=0, y=1, z=2 }; // Used for the PML components of F
};
```

因此，`Ex(i,j,k,PMLComp::xy)` 不是另一个物理电场分量，而是 `E_x` 中由 `y` 方向 curl 项驱动的 split component。这个存储约定贯穿 `EvolveBPML.cpp`、`EvolveEPML.cpp` 和 `EvolveFPML.cpp`。

`EvolveBPML()` 明确把非 Cartesian FDTD PML 排除掉：


```

#if defined(WARPX_DIM_RZ) || defined(WARPX_DIM_RCYLINDER) || defined(WARPX_DIM_RSPHERE)
    amrex::ignore_unused(fields, patch_type, level, dt, dive_cleaning);
    WARPX_ABORT_WITH_MESSAGE(
        "PML only implemented in Cartesian geometry.");
#else

```

这是一条功能边界：当前读到的 FDTD PML kernel 不能拿来解释 RZ 或 spherical 几何的 PML。进入 Cartesian 后，B 的 split update 仍然复用 Yee/Nodal/CKC 的 T_Algo::UpwardDz* 差分模板。例如 Bx 的更新为：

```

Bx(i, j, k, PMLComp::xz) += dt * (
    T_Algo::UpwardDz(Ey, coefs_z, n_coefs_z, i, j, k, PMLComp::yx)
    + T_Algo::UpwardDz(Ey, coefs_z, n_coefs_z, i, j, k, PMLComp::yz)
    + UpwardDz_Ey_yy);

Bx(i, j, k, PMLComp::xy) -= dt * (
    T_Algo::UpwardDy(Ez, coefs_y, n_coefs_y, i, j, k, PMLComp::zx)
    + T_Algo::UpwardDy(Ez, coefs_y, n_coefs_y, i, j, k, PMLComp::zy)
    + UpwardDy_Ez_zz);

```

它仍对应普通 Maxwell 方程中的

$$\partial_t B_x = \partial_z E_y - \partial_y E_z,$$

但 E_y 和 E_z 已经被拆成 PML components，所以源码必须把相关 components 求和。UpwardDz_Ey_yy 和 UpwardDy_Ez_zz 只在开启 PML divergence cleaning 时加入。

EvolveEPML() 做相反的 curl(B) 更新，并在 PML 中可选加入 F 修正和粒子电流项：

```

Ex(i, j, k, PMLComp::xz) -= c2 * dt * (
    T_Algo::DownwardDz(By, coefs_z, n_coefs_z, i, j, k, PMLComp::yx)
    + T_Algo::DownwardDz(By, coefs_z, n_coefs_z, i, j, k, PMLComp::yz) );
Ex(i, j, k, PMLComp::xy) += c2 * dt * (
    T_Algo::DownwardDy(Bz, coefs_y, n_coefs_y, i, j, k, PMLComp::zx)
    + T_Algo::DownwardDy(Bz, coefs_y, n_coefs_y, i, j, k, PMLComp::zy) );

```

这对应

$$\partial_t E_x = c^2(\partial_y B_z - \partial_z B_y),$$

只是每个参与 curl 的磁场分量也以 split components 形式存储。若 pml_has_particles 为真，EvolveEPML.cpp 还会读取 pml_j_fp/cp 并调用 push_ex_pml_current 等 helper，使 PML 中传播的粒子电流进入电场更新。

最后，EvolveFPML() 更新 PML divergence-cleaning 标量：

```

F(i, j, k, PMLComp::x) += dt * (
    T_Algo::DownwardDx(Ex, coefs_x, n_coefs_x, i, j, k, PMLComp::xx)
    + T_Algo::DownwardDx(Ex, coefs_x, n_coefs_x, i, j, k, PMLComp::xy)
    + T_Algo::DownwardDx(Ex, coefs_x, n_coefs_x, i, j, k, PMLComp::xz) );

```

普通区域的 F 方程含有 $-\rho/\epsilon_0$ 项；当前 PML F kernel 只读到 split E 的 divergence 累积。PML 中吸收系数、sigma profile、damping 因子和 current damping 的细节不在这三个 field update 文件中，下一步要继续进入 BoundaryConditions/PML.cpp 和 BoundaryConditions/PML_current.H。

6.3 PML sigma profile、damping 与电流源项

notes/code-reading/fieldsolver/03-pml-damping-current.md 已经继续展开 BoundaryConditions/PML.cpp 和 WarpXEvolvePML.cpp。PML 的吸收 profile 由 FillLo() / FillHi() 生成：

```

Real offset = static_cast<Real>(glo-i);
p_sigma[i-slo] = fac*(offset*offset);
p_sigma_cumsum[i-slo] = (fac*(offset*offset*offset)/3._rt)/v_sigma;

```

```

if (i <= ohi+1) {
    offset = static_cast<Real>(glo-i) - 0.5_rt;
    p_sigma_star[i-sslo] = fac*(offset*offset);
    p_sigma_star_cumsum[i-sslo] = (fac*(offset*offset*offset)/3._rt)/v_sigma;
}

```

对应的 profile 是

$$\sigma(s) = Cs^2, \quad \int_0^s \sigma(s') ds' = \frac{Cs^3}{3}.$$

SigmaBox::ComputePMLFactorsE/B() 再把它转成指数阻尼:

```

p_sigma_star_fac[idim][i] = std::exp(-p_sigma_star[idim][i]*dt);
p_sigma_fac[idim][i] = std::exp(-p_sigma[idim][i]*dt);

```

DampPML_Cartesian() 在每个 PML tile 上调用 warpx_damp_pml_ex/ey/ez 和 warpx_damp_pml_bx/by/bz。这些 kernel 根据场的 staggered 位置选择 sigma_fac 或 sigma_star_fac, 对 split components 逐方向相乘。例如 Exy 乘 y 方向阻尼, Exz 乘 z 方向阻尼; 若开启 do_pml_dive_cleaning, Exx 也乘 x 方向阻尼。

若 PML 中有粒子电流, push_ex_pml_current() 会把 J_x 按横向 sigma 比例分给 Exy 和 Exz:

```

alpha_xy = sigjy[k-ylo]/(sigjy[k-ylo]+sigjz[l-zlo]);
alpha_xz = sigjz[l-zlo]/(sigjy[k-ylo]+sigjz[l-zlo]);
Ex(j,k,l,PMLComp::xy) = Ex(j,k,l,PMLComp::xy) - mu_c2_dt * alpha_xy * jx(j,k,l);
Ex(j,k,l,PMLComp::xz) = Ex(j,k,l,PMLComp::xz) - mu_c2_dt * alpha_xz * jx(j,k,l);

```

而 DampJPML() 使用的是 sigma_cumsum_fac, 不是场 damping 的 sigma_fac:

```

damp_jx_pml(i, j, k, pml_jxfab, sigma_star_cumsum_fac_j_x,
            sigma_cumsum_fac_j_y, sigma_cumsum_fac_j_z,
            xs_lo, y_lo, z_lo);

```

这说明 WarpX 对 PML 电流采用沿吸收层积分后的 damping, 而不是简单的局部 $e^{-\sigma \Delta t}$ 。

最后, PML::Exchange() 把 split fields 求和再与常规场交换:

```

MultiFab totpmlmf(pml.boxArray(), pml.DistributionMap(), 1, 0);
MultiFab::LinComb(totpmlmf, 1.0, pml, 0, 1.0, pml, 1, 0, 1, 0);
if (ncp == 3) {
    MultiFab::Add(totpmlmf, pml, 2, 0, 1, 0);
}

```

所以 PML 的完整链条是: 常规场进入 PML split component, PML 内 curl 更新, 乘 sigma damping, split components 求和回填常规边界。只看 EvolveEPML.cpp 或只看 DampPML() 都不足以说明 PML 的实际边界条件。

6.4 非 Cartesian FDTD: RZ、RCYLINDER 与 RSPHERE

notes/code-reading/fieldsolver/04-noncartesian-fdtd.md 继续把 FDTD 从 Cartesian 推到 WarpX 的编译几何分支。FiniteDifferenceSolver.cpp 中, RZ/RCYLINDER 和 RSPHERE 不走 Cartesian CKC/Nodal 分支, 而是只接受 Yee/HybridPIC:

```

#if defined(WARPX_DIM_RZ) || defined(WARPX_DIM_RCYLINDER)
    m_dr = cell_size[0];
    m_nmodes = WarpX::n_rz_azimuthal_modes;
    m_rmin = WarpX::GetInstance().Geom(0).ProbLo(0);
    if (fdtd_algo == ElectromagneticSolverAlgo::Yee ||
        fdtd_algo == ElectromagneticSolverAlgo::HybridPIC ) {
        CylindricalYeeAlgorithm::InitializeStencilCoefficients( cell_size,
            m_h_stencil_coefs_r, m_h_stencil_coefs_z );
    }

```

cylindrical Yee 的核心算子不是 $\partial_r F$, 而是

$$\frac{1}{r} \frac{\partial(rF)}{\partial r}.$$

源码中对应:

```
return 1._rt/r * inv_dr*( (r+0.5_rt*dr)*F(i+1,j,k,comp) - (r-0.5_rt*dr)*F(i,j,k,comp) );
```

RZ 的 azimuthal mode 展开使 ∂_θ 变成 im , 所以实部/虚部会互相耦合。EvolveBCylindrical() 中 Br 的高阶 mode 更新为:

```
Br(i, j, 0, 2*m-1) += dt*(
    T_Algo::UpwardDz(Etheta, coefs_z, n_coefs_z, i, j, 0, 2*m-1)
    - m * Ez(i, j, 0, 2*m )/r );
Br(i, j, 0, 2*m ) += dt*(
    T_Algo::UpwardDz(Etheta, coefs_z, n_coefs_z, i, j, 0, 2*m )
    + m * Ez(i, j, 0, 2*m-1)/r );
```

轴上 $r = 0$ 不能直接除以 r , 源码显式使用正则化条件。例如 $Etheta(r=0, m=1) = -i Er(r=0, m=1)$:

```
Etheta(i, j, 0, 2*m-1) = Er(i, j, 0, 2*m );
Etheta(i, j, 0, 2*m ) = -Er(i, j, 0, 2*m-1);
```

这类条件是 RZ FDTD 的物理核心: 轴上的场必须满足坐标正则性, 而不是普通网格差分的延伸。

RSPHERE 使用 spherical operator:

$$\frac{1}{r^2} \frac{\partial(r^2 F)}{\partial r}.$$

源码为:

```
return 1._rt/(r*r) * inv_dr*( rph*rph*F(i,j,k,comp) - rmh*rmh*F(i-1,j,k,comp) );
```

对应的 EvolveFSpherical() 在轴上用 $6*Er/dr$ 正则化:

```
F(i, j, 0, 0) += dt * (
    - rho(i, j, 0, rho_shift) * inv_epsilon0
    + 6._rt*Er(i, j, 0, 0)/dr);
```

这说明非 Cartesian FDTD 不能作为 Cartesian FDTD 的坐标名替换来讲; 必须把 metric factors、mode coupling 和 axis regularization 都写入公式和源码解读。

6.5 PSATD 谱求解主流程

notes/code-reading/fieldsolver/05-psatd-spectral-flow.md 开始进入 PSATD。理论上, PSATD 把 Maxwell 方程写到 Fourier 空间:

$$\frac{\partial \tilde{\mathbf{E}}}{\partial t} = i\mathbf{k} \times \tilde{\mathbf{B}} - \tilde{\mathbf{J}}, \quad \frac{\partial \tilde{\mathbf{B}}}{\partial t} = -i\mathbf{k} \times \tilde{\mathbf{E}}.$$

在一个时间步内对线性部分解析积分, 得到含

$$C = \cos(k\Delta t), \quad S = \sin(k\Delta t)$$

的更新式。源码入口是 WarpX::PushPSATD(), 当前位于 ../warpX/Source/FieldSolver/WarpXPushFieldsEM.cpp:771-943:

```
// FFT of E and B
PSATDForwardTransformEB();

// FFT of F and G
if (WarpX::do_dive_cleaning) { PSATDForwardTransformF(); }
if (WarpX::do_divb_cleaning) { PSATDForwardTransformG(); }
```

```
// Update E, B, F, and G in k-space
PSATDPushSpectralFields();
```

```
// Inverse FFT of E, B, F, and G
PSATDBackwardTransformEB();
```

在此之前, PushPSATD() 会根据 current_correction、Vay deposition、periodic single box 和 mesh refinement 分支处理 J/rho:

```
PSATDForwardTransformJ(current_fp_string, current_cp_string);
PSATDForwardTransformRho(rho_fp_string, rho_cp_string, 0, rho_old);
PSATDForwardTransformRho(rho_fp_string, rho_cp_string, 1, rho_new);

::PSATDCurrentCorrection(finest_level, spectral_solver_fp, spectral_solver_cp);
```

```
PSATDBackwardTransformJ(current_fp_string, current_cp_string);
```

v0.5 需要特别记住两个实现边界。第一, fft_periodic_single_box 分支会在 :791-837 内完成 current correction 或 Vay deposition 的 k-space 处理; 非 periodic single box 分支在 :839-899 里还会在 correction/Vay 后调用 SyncCurrent()、SyncRho() 或 SumBoundaryJ()。第二, 真正的场推进顺序是 PSATDForwardTransformEB() :901-902、可选 RZ PML push :904-907、F/G transform :909-911、PSATDPushSpectralFields() :913-914、再做 E/B/F/G 反变换 :916-926; 最后才进入每层 PML push 和物理边界条件 :928-940。

SpectralSolver 本身只负责建立 k-space、spectral field storage 和选择具体算法。当前分派入口是 ../warpx/Source/FieldSolver/SpectralS

```
const SpectralKSpace k_space= SpectralKSpace(realspace_ba, dm, dx);

m_spectral_index = SpectralFieldIndex(
    update_with_rho, fft_do_time_averaging, time_dependency_J, time_dependency_rho,
    dive_cleaning, divb_cleaning, pml);

void SpectralSolver::pushSpectralFields(){
    algorithm->pushSpectralFields( field_data );
}
```

所以真正的 PSATD 更新在 PsatdAlgorithm* 子类中。以 PsatdAlgorithmGalilean.cpp 为例, 标准 PSATD 也通过 v_galilean=0 的情形进入同一套形式:

```
fields(i,j,k,Idx.Ex) = T2 * C * Ex_old
                      + I * c2 * T2 * S_ck * (ky * Bz_old - kz * By_old)
                      + X4 * Jx - I * (X2 * rho_new - T2 * X3 * rho_old) * kx;
```

(ky*Bz-kz*By) 是谱空间 curl 的 x 分量。X1/X2/X3/X4/T2 是下一节需要展开的系数, 它们把标准 PSATD、Galilean PSATD、是否使用 rho、是否时间平均等分支折叠成统一更新式。

PSATD 还必须处理 staggered 网格位置。SpectralFieldData::ForwardTransform() 在实空间场不是 nodal 时乘相位因子:

```
if (!is_nodal_0) { spectral_field_value *= shift0_arr[i]; }
if (!is_nodal_1) { spectral_field_value *= shift1_arr[j]; }
if (!is_nodal_2) { spectral_field_value *= shift2_arr[k]; }
```

相位因子来自

```
pshift[i] = amrex::exp( I*sign*pk[i]*0.5_rt*t_dx_idim);
```

即 $e^{\pm ik\Delta x/2}$ 。这一步是把 Yee staggered 数据映射到谱算法假定的位置; 没有这一步, 谱空间 curl 与实空间场的位置会错半格。

6.6 标准/Galilean PSATD 系数和 current correction

notes/code-reading/fieldsolver/06-psatd-galilean-current-correction.md 继续展开 PsatdAlgorithmGalilean.cpp。标准 PSATD 是 Galilean 实现的 $v_G = 0$ 极限; 源码中

$$w_c = \mathbf{k}_c \cdot \mathbf{v}_G, \quad T_2 = e^{i w_c \Delta t}.$$

w_c 必须使用 centered modified k:

```
const amrex::Real w_c = kx_c[i]*vg_x +
#if defined(WARPX_DIM_3D)
    ky_c[j]*vg_y + kz_c[k]*vg_z;
#else
    kz_c[j]*vg_z;
#endif
```

基础振荡系数是:

```
C(i,j,k) = std::cos(om_s * dt);

if (om_s != 0.)
{
    S_ck(i,j,k) = std::sin(om_s * dt) / om_s;
}
else
{
    S_ck(i,j,k) = dt;
}
```

T2(i,j,k) = theta_c * theta_c;

其中 om_s = c*|k_s|。X1-X4 把电流和电荷项折叠进统一更新式: 源码显式处理 $k = 0$ 和 $w_c = 0$ 极限, 避免除零。

current correction 的源码与官方参数文档逐项对应。标准分支为:

```
fields(i,j,k,Idx.Jx_mid) = Jx - (k_dot_J - I * (rho_new - rho_old) / dt)
    * kx / (k_norm * k_norm);
```

这正是

$$\hat{\mathbf{J}}_{corr} = \hat{\mathbf{J}} - \left(\mathbf{k} \cdot \hat{\mathbf{J}} - i \frac{\hat{\rho}^{n+1} - \hat{\rho}^n}{\Delta t} \right) \frac{\mathbf{k}}{k^2}.$$

Galilean 分支为:

```
const Complex rho_old_mod = rho_old * amrex::exp(I * k_dot_vg * dt);
const Complex den = 1._rt - amrex::exp(I * k_dot_vg * dt);

fields(i,j,k,Idx.Jx_mid) = Jx - (k_dot_J - k_dot_vg * (rho_new - rho_old_mod) / den)
    * kx / (k_norm * k_norm);
```

这里 rho_old_mod 是 $\rho^n \theta^2$, den 是 $1 - \theta^2$ 。因此 current correction 的目的不是平滑电流, 而是投影掉违反谱空间连续性方程的纵向误差, 使修正后的电流与 rho_old/rho_new 相容。

6.6.1 v0.26 X1-X4 系数的源码公式闭环

v0.26 把 notes/code-reading/fieldsolver/17-psatd-x-coefficients.md 回填到正文。这里的范围限定为 Cartesian PsatdAlgorithmGalilean.cpp: 标准 PSATD 是 v_galilean=0 的极限, Galilean PSATD 则通过

$$\omega_c = \mathbf{k}_c \cdot \mathbf{v}_{gal}, \quad T_2 = \exp(i\omega_c \Delta t)$$

把旧时刻场和旧时刻电荷项移到 Galilean 网格。源码使用 centered modified k 计算 ω_c , 而使用 spectral solver 的 modified k 计算

$$\omega_s = c|\mathbf{k}_s|, \quad C = \cos(\omega_s \Delta t), \quad S_{ck} = \frac{\sin(\omega_s \Delta t)}{\omega_s}.$$

S_ck 在 $\omega_s = 0$ 时直接取 Δt 。这个零模分支非常重要, 因为 X1-X4 里既有 $\omega_s^2 - \omega_c^2$, 也有 $\theta_c^* - \theta_c$ 这样的分母, 不能用一个通式硬算所有模式。

四个系数在源码中的角色如下：

系数	Galilean 通式或标准极限	更新式中的位置	解释
X1	通式为 $(1 - \text{theta2_c} * C + i * w_c * \text{theta2_c} * S_ck) / (\epsilon_0 * (\omega_s^2 - w_c^2))$; 零模为 $dt^2 / (2 \epsilon_0)$	B_new 中的 $i * X1 * (k \times J)$	电流通过 Faraday/Ampere 耦合对磁场的贡献
X2	w_c != 0 时由 $c^2 * (\text{theta_c_star} * X1 - \text{theta_c} * \text{tmp}) / (\text{theta_c_star} - \text{theta_c})$ 给出; 标准非零模为 $c^2 * (dt - S_ck) / (\epsilon_0 * dt * \omega_s^2)$	E_new 中 $-i * X2 * \rho_new * k$	新时刻电荷对纵向电场的贡献
X3	w_c != 0 时由 $c^2 * (\text{theta_c_star} * X1 - \text{theta_c_star} * \text{tmp}) / (\text{theta_c_star} - \text{theta_c})$ 给出; 标准非零模为 $c^2 * (dt * C - S_ck) / (\epsilon_0 * dt * \omega_s^2)$	E_new 中 $+i * T2 * X3 * \rho_old * k$	旧时刻电荷经 Galilean 相位后的纵向贡献
X4	$i * w_c * X1 - \text{theta2_c} * S_ck / \epsilon_0$; 标准 PSATD 为 $-S_ck / \epsilon_0$	E_new 中的 $X4 * J$	电流对电场的直接源项

因此, Cartesian standard/Galilean PSATD 的核心更新可以压缩成:

$$\mathbf{E}^{n+1} = T_2 C \mathbf{E}^n + i c^2 T_2 S_{ck} (\mathbf{k} \times \mathbf{B}^n) + X_4 \mathbf{J} - i (X_2 \rho^{n+1} - T_2 X_3 \rho^n) \mathbf{k},$$

$$\mathbf{B}^{n+1} = T_2 C \mathbf{B}^n - i T_2 S_{ck} (\mathbf{k} \times \mathbf{E}^n) + i X_1 (\mathbf{k} \times \mathbf{J}).$$

这两个式子也说明为什么本章不能把 X1-X4 写成“稳定化系数”。它们首先是 PSATD 对源项积分的解析系数: X1/X4 管电流源项, X2/X3 管电荷源项, T2 管 Galilean 相位。nci_psatd_stability regression 里的 NCI 能量比和 Gauss-law 检查, 是这些系数、current correction、filter、有限阶 modified k 和输入参数共同作用后的结果, 而不是某一个 X 系数单独承担的验证。

本节还要保留三个边界。第一, time averaging 使用 Psi* 和 Y* 系数, 不能只靠 X1-X4 描述平均场输出。第二, JRhom 使用 PsatdAlgorithmJRhomFirstOrder/SecondOrder, 源项时间依赖和系数体系不同。第三, RZ、Galilean RZ、PML PSATD 都有各自的系数语义, 不能把 Cartesian PsatdAlgorithmGalilean.cpp 的 X1-X4 直接搬过去。

6.6.2 v0.27 time-averaging Psi/Y 系数的源码公式闭环

v0.27 继续收束 notes/code-reading/fieldsolver/18-psatd-time-averaging-coefficients.md。这一节的重点不是普通 E/B 推进, 而是 psatd.do_time_averaging=1 时写入 Ex_avg...Bz_avg 的 average-field 输出。源码边界很明确: PsatdAlgorithmGalilean.cpp:76-85 只有在 time_averaging 打开时才分配 Psi1_coef、Psi2_coef 和 Y1-Y4, 而:98-101 又断言 time averaging 必须配合 psatd.update_with_rho=1。

这说明 time averaging 不是一个只依赖电流的后处理开关。平均电场里实际含有 rho_new/rho_old:

```
fields(i,j,k,Idx.Ex_avg) = Psi1 * Ex_old
                          - I * c2 * Psi2 * (ky * Bz_old - kz * By_old)
                          + Y4 * Jx + (Y2 * rho_new + Y3 * rho_old) * kx;
```

对应的向量形式可以写成:

$$\langle \mathbf{E} \rangle = \Psi_1 \mathbf{E}^n - i c^2 \Psi_2 (\mathbf{k} \times \mathbf{B}^n) + Y_4 \mathbf{J} + (Y_2 \rho^{n+1} + Y_3 \rho^n) \mathbf{k},$$

$$\langle \mathbf{B} \rangle = \Psi_1 \mathbf{B}^n + i \Psi_2 (\mathbf{k} \times \mathbf{E}^n) + i Y_1 (\mathbf{k} \times \mathbf{J}).$$

这里的 avg 仍在谱空间。WarpXEvolve.cpp:998-1007 随后调用 PSATDScaleAverageFields(1/(2*dt)) 和 PSATDBackwardTransformEBA 把谱空间平均场变成实空间的 Efield_avg/Bfield_avg, 供后续 gather/通信/诊断路径使用。

InitializeSpectralCoefficientsAveraging() 使用的共享量和 X1-X4 相似, 但积分区间不同。它定义

$$\omega_s = c|\mathbf{k}_s|, \quad \omega_c = \mathbf{k}_c \cdot \mathbf{v}_{gal},$$

并使用

$$\theta_c = e^{i\omega_c \Delta t/2}, \quad \theta_c^2 = e^{i\omega_c \Delta t}, \quad \theta_c^3 = e^{i3\omega_c \Delta t/2},$$

以及

$$C_1 = \cos(\omega_s \Delta t/2), \quad C_3 = \cos(3\omega_s \Delta t/2), \quad S_1 = \frac{\sin(\omega_s \Delta t/2)}{\omega_s}, \quad S_3 = \frac{\sin(3\omega_s \Delta t/2)}{\omega_s}.$$

于是旧场平均权重为:

$$\Psi_1 = \frac{\theta_c^3(\omega_s^2 S_3 + i\omega_c C_3) - \theta_c(\omega_s^2 S_1 + i\omega_c C_1)}{\Delta t(\omega_s^2 - \omega_c^2)},$$

$$\Psi_2 = \frac{\theta_c^3(C_3 - i\omega_c S_3) - \theta_c(C_1 - i\omega_c S_1)}{\Delta t(\omega_s^2 - \omega_c^2)}.$$

源码里 Psi1 的零模标准分支为 1, Psi2 的零模标准分支为 -dt。这个负号不能孤立解读, 因为平均电场更新式里它前面还有 -i*c2, 平均磁场更新式里则是 +i。

Y1-Y4 的角色也要分开。源码先构造内部辅助量

$$\Psi_3 = \begin{cases} -i(\theta_c^3 - \theta_c)/(\Delta t \omega_c), & \omega_c \neq 0, \\ 1, & \omega_c = 0, \end{cases}$$

然后把

$$Y_1 = \frac{1 - \Psi_1 - i\omega_c \Psi_2}{\epsilon_0(\omega_s^2 - \omega_c^2)}$$

放入平均磁场的 $iY_1(\mathbf{k} \times \mathbf{J})$ 项。通用非零分支下,

$$Y_2 = \frac{ic^2(\epsilon_0 \omega_s^2 Y_1 - \Psi_3 + \Psi_1)}{\epsilon_0 \omega_s^2 (\theta_c^2 - 1)}, \quad Y_3 = \frac{ic^2(\Psi_3 - \Psi_1 - \epsilon_0 \theta_c^2 \omega_s^2 Y_1)}{\epsilon_0 \omega_s^2 (\theta_c^2 - 1)}.$$

Y2 和 Y3 分别乘 rho_new 与 rho_old, 因此它们是 average-field 的 charge endpoint 系数。最后

$$Y_4 = \frac{\Psi_2 + i\epsilon_0 \omega_c Y_1}{\epsilon_0}$$

是平均电场中的直接电流项。这样读, Y1 和 Y4 都与电流有关, 但一个进入磁场旋度源, 一个进入电场直接源; Y2/Y3 则与电荷端点有关。

这也给后续写作立了一个防混写规则: Cartesian PsatdAlgorithmGalilean.cpp 的 average-field Y1-Y4、JRhom second-order 的 Y1-Y8、RZ/Galilean RZ 的平均场系数和 PML PSATD 的 C1-C25 是不同算法族内的局部命名。正文可以比较它们服务的积分对象, 但不能因为名字相同就合并成一张“PSATD Y 系数表”。

6.6.3 v0.17 文献闭环: Lehe et al. 2016 的 Galilean PSATD

v0.17 为本节补入第一篇 PSATD/Galilean/NCI 核心论文闭环:references/06_stability_filtering_nci/2016_LehePRE2016_Elimination讲解.md。该笔记基于本地 PDF、MinerU Markdown 和论文图片, 按论文顺序记录 Galilean 坐标、离散连续性方程、PIC cycle、二维稳定性分析和准柱坐标扩展。

这篇论文对本章最重要的判断是: Galilean PSATD 不是 moving window 的同义词。它首先把坐标改成

$$\mathbf{x}' = \mathbf{x} - \mathbf{v}_{gal}t,$$

于是 Maxwell 方程中的时间导数变成

$$(\partial_t - \mathbf{v}_{gal} \cdot \nabla'),$$

并且一个时间步内的电流近似也从“固定实验室网格点上常量”改为“固定 Galilean 网格点上常量”。这正是标准 PSATD 与 Galilean PSATD 在 NCI 行为上分开的根源。

对应到 current correction, Galilean 离散连续性方程不再是简单的

$$-i \frac{\hat{\rho}^{n+1} - \hat{\rho}^n}{\Delta t} + i\mathbf{k} \cdot \hat{\mathbf{J}}^{n+1/2} = 0,$$

而是带有移动网格相位:

$$-i \frac{\hat{\rho}^{n+1} - \theta^2 \hat{\rho}^n}{\Delta t} + i\mathbf{k} \cdot \hat{\mathbf{J}}^{n+1/2} = 0, \quad \theta = \exp(i\mathbf{k} \cdot \mathbf{v}_{gal} \Delta t / 2).$$

这可以直接解释上面的源码: rho_old_mod = rho_old * exp(i*k_dot_vg*dt) 不是任意相位技巧, 而是离散连续性方程在 Galilean 网格上的旧时刻电荷项。WarpX 官方 boosted-frame 文档 ../warpX/Docs/source/theory/boosted_frame.rst 引用的 bf-LehePRE2016 正是这个推导来源: ../warpX/Examples/Tests/nci_psatd_stability/analysis_galilean.py 则把论文中的稳定性判断变成 regression gate, 用最终电场能量相对不稳定参考值的比例检查 NCI 是否被压住, 并在 current-correction 分支继续检查 Gauss law。

因此, 本书后面讲 psatd.use_default_v_galilean 时, 不能把它写成经验开关。它的理论含义是: 在 boosted-frame 或均匀流动等离子体问题中, 让 Galilean 网格速度接近背景漂移速度 $\mathbf{v}_0$, 使背景等离子体在数值网格中近似静止, 从而移除主要 alias resonance。论文的二维稳定性分析和 Warp/FBPIC 数值实验都显示, 最大增长率只在 $\mathbf{v}_{gal} \approx \mathbf{v}_0$ 附近降到零; 取反方向的 Galilean 速度并不会自动稳定。

6.6.4 v0.18 文献闭环: Kirchen et al. 2016 的 boosted-frame 应用证据

v0.18 继续补入 Kirchen et al. 2016:references/06_stability_filtering_nci/2016_KirchenPOP2016_Stable_discrete_representation讲解.md。这篇论文和 Lehe et al. 2016 的分工不同: Lehe 论文负责 Galilean PSATD 的推导、离散连续性方程和稳定性分析; Kirchen 论文负责说明该离散表示如何落到 Lorentz boosted-frame 等离子体加速 workflow。

在 boosted frame 中, 实验室系静止的背景等离子体会以

$$\mathbf{v}_{plasma} = -\beta c \mathbf{e}_z$$

穿过计算域。Kirchen et al. 的关键选择就是让 Galilean 网格速度取同一个漂移速度:

$$\mathbf{v}_{gal} = \mathbf{v}_{plasma} = -\beta c \mathbf{e}_z.$$

这样背景等离子体在 Galilean 网格上静止, 而激光、电子束等 elongated quantities 相对网格以修正后的速度传播。这个表述能把 WarpX 官方 boosted-frame 文档中的 bf-KirchenPOP2016 引用落成一句实际操作原则: psatd.use_default_v_galilean 应理解为把 warpX.gamma_boost 推导出的背景漂移速度交给 Galilean PSATD, 而不是打开一个无物理来源的稳定化滤波器。

Kirchen 论文的应用图也给出一个读者侧验证顺序。第一, 固定所有数值参数, 只在 $\mathbf{v}_{gal}=0$ 和 $\mathbf{v}_{gal}=-\beta c$ 之间切换; 标准 PSATD 出现强 NCI, Galilean PSATD 稳定。第二, 把 boosted-frame 结果 back-transform 回实验室系, 比较 E_z 、 E_y 、激光腰斑、脉冲长度、电子束能量、能散和归一化发射度; 结果在亚百分比量级内一致。也就是说, 本节不能只写“Galilean PSATD 稳定”, 还要写“稳定后仍保持加速器物理量”。

这也解释了 `nci_psatd_stability` regression 和更高层 `boosted-frame example` 之间的关系：前者用均匀流动等离子体和电场能量比做最小稳定性 gate；Kirchen 论文提供的是应用层证据，说明同一 Galilean 表示在激光等离子体加速的 `boosted-frame workflow` 中既抑制 NCI，又保持实验室系物理可比性。

6.6.5 v0.19 文献闭环：Godfrey et al. 2014 的 PSATD NCI 策略谱系

v0.19 补入 Godfrey, Vay, Haber 2014:references/06_stability_filtering_nci/2014_GodfreyJCP2014_Numerical_stability 文讲解.md。这篇论文在本章中的位置应放在 Lehe/Kirchen 之前的理论基线：它不是 Galilean PSATD 论文，而是固定网格 PSATD 的 NCI 色散分析和抑制策略谱系。

论文的起点是 PSATD 场推进本身。谱空间中定义

$$C = \cos(k\Delta t), \quad S = \sin(k\Delta t),$$

电场更新包含旋度场项、电流项和纵向投影项：

$$\mathbf{E}^{n+1} = C\mathbf{E}^n - iS \frac{\mathbf{k} \times \mathbf{B}^n}{k} - \frac{S}{k} \mathbf{J}^{n+1/2} + (1-C) \frac{\mathbf{k}\mathbf{k} \cdot \mathbf{E}^n}{k^2} + \left(\frac{S}{k} - \Delta t \right) \frac{\mathbf{k}\mathbf{k} \cdot \mathbf{J}^{n+1/2}}{k^2}.$$

PSATD 的真空电磁模没有普通 FDTD Courant 限制，但相对论束流 PIC 仍有 NCI，因为离散粒子-网格系统中存在空间 alias 和时间 alias。Godfrey et al. 把这一点写成离散色散矩阵：

$$\det \mathcal{D}(\omega, \mathbf{k}) = 0,$$

其中束流 alias 可写作

$$k'_z = k_z + m_z \frac{2\pi}{\Delta z},$$

而危险的共振位置满足类似

$$k_x^r = \left[\left(\left(k_z + m_z \frac{2\pi}{\Delta z} \right) v - p \frac{2\pi}{\Delta t} \right)^2 - k_z^2 \right]^{1/2}.$$

这给本章一个很重要的写作边界：`warpx.use_filter = 1` 不是随意的数值润色，而是针对高 k alias 和短波共振的 filter/smoothing 家族；但 filter 也不是万能的，因为小 k 非共振增长可能需要 current scaling、插值阶数、时间步或表示方式共同处理。Godfrey 论文中的 current scaling 写成

$$\mathbf{J} = \zeta : \mathbf{J}_e, \quad \zeta = \text{diag}(\zeta_z, \zeta_x, \zeta_y),$$

它是 NCI 色散矩阵中的 k 依赖电流因子。这里必须和 WarpX regression 里的 `psatd.current_correction` 分开：`psatd.current_correction` 首先是连续性方程/Gauss law 约束路径，`analysis_galilean.py` 在 current-correction 分支还会检查 `max|divE-rho/eps0|` 相对误差；Godfrey 的 ζ 则是为压低 NCI 增长率而设计的 current scaling。

Godfrey 2014、Lehe 2016、Kirchen 2016 合起来形成一个清楚的策略谱系。Godfrey 论文讲 fixed-grid PSATD 中如何用数字滤波、三次插值、current scaling 和时间步选择降低 NCI；Lehe 论文讲 Galilean PSATD 如何通过移动坐标/源项表示，在 $v_{gal} \approx v_0$ 时从表示层面消除均匀漂移 NCI；Kirchen 论文讲这个 Galilean 表示如何落到 boosted-frame LPA workflow 并保持回变换后的物理量一致。对应到 WarpX，`nci_psatd_stability` 的 `warpx.use_filter = 1`、`psatd.current_correction`、`psatd.do_time_averaging` 和 `psatd.JRhom` 应被写成不同机制的 regression 入口，而不是同一个“稳定化开关”的不同名字。

6.6.6 v0.20 源码闭环: WarpX PSATD/NCI 机制对照表

v0.20 继续把上一节的策略谱系落回 WarpX 源码。结论先写清楚: 当前 WarpX 中和 NCI 稳定性相关的 filter、current correction、finite-order PSATD、Galilean representation 和 JRhom 是五组不同机制; 其中源码里叫 `NCIGodfreyFilter` 的路径也不是 `nci_psatd_stability` 输入卡里常见的 `warpX.use_filter = 1`。

机制	触发入口	当前源码证据	实际操作	和 Godfrey 2014 的关系
实空间 bilinear/binomial-like filter	普通 Cartesian explicit/PSATD 路径中 <code>warpX.use_filter = 1</code> , 默认值由 WarpX.cpp 根据 <code>evolve_scheme</code> 和几何决定	Source/WarpX.cpp:825-842 读取 <code>warpX.use_filter</code> 、 <code>warpX.use_filter_compensation</code> 、 <code>warpX.filter_npass_each_dir</code> 和 <code>warpX.filter_npass_each_dir</code> Source/Initialization/WarpXInitData.cpp:1203-1208 初始化 <code>bilinear_filter</code> ; Source/Filter/BilinearFilter.cpp:24-87 构造逐方向 stencil; Source/Filter/Filter.cpp:38-130 应用 tensor-product stencil; Source/Parallelization/WarpXComm.cpp:1479-1496 和 1677-1693 对 <code>current/rho</code> 做 filter 与边界求和	多方向中心权重 0.5、相邻权重 0.25 的对称 1D stencil, 再按方向组合成 3D stencil, 再按方向组合成 3D stencil, 再按方向组合成 3D stencil	属于 Godfrey 论文中的 digital filtering/smoothing 家族, 可以压制高 k alias, 但不是论文中的 current correction
RZ PSATD k-space binomial filter	RZ/RCYLINDER/RSPHERICAL 几何且 PSATD 时, WarpX.cpp 把 <code>use_filter</code> 转成 <code>use_kspace_filter</code>	Source/WarpX.cpp:853-858 在 RZ PSATD 下设置 <code>use_kspace_filter = use_filter</code> 并关闭 <code>use_filter</code> ; Source/WarpX.cpp:3123-3125 调用 <code>SpectralSolver::InitFilter()</code> ; Source/FieldSolver/SpectralSolver/SpectralBinomialFilter.cpp:18-38 构造 k-space filter; Source/FieldSolver/SpectralSolver/SpectralFieldDataRZ.cpp:800-848 初始 化并应用 radial/z filter; Source/FieldSolver/WarpXPushFieldsEM.cpp:514-538 与 612-625 在 spectral J/rho 上应用	对 $1 - \sin^2(k\Delta/2)$ 的幂次 filter, 补偿打开时再乘一个低阶补偿因子	仍是 spectral filtering 家族; 和 Cartesian 实空间 bilinear filter 入口相同, 但实现位置、作用空间和 RZ 语义不同
FDTD Godfrey gather filter	<code>use_fDTD_nci_corr</code> , 不是 <code>warpX.use_filter</code>	Source/Initialization/WarpXInitData.cpp:1203-1208 初始化两组 <code>NCIGodfreyFilter</code> ; Source/Filter/NCIGodfreyFilter.cpp:29-138 固定 z 向 5 点 stencil 并按 gather 类型查表插值; Source/Particles/PhysicalParticleContainer.cpp:552-563 与 896-970 在粒子 gather 前过滤场; Source/Parallelization/GuardCellManager.cpp:355-365 为 gather 增加 z 向 guard cells	Godfrey 表系数构造的 z 向 5 点 filter, 分别作用 Bx/By/Ez 和 Bx/By/Ez 组合	这是 70-119 NCI corrector 的 gather-side filter; 不是把它误读成 PSATD <code>nci_psatd_stability</code> 中 <code>warpX.use_filter = 1</code> 的 filter

这张表给本章一个新的写作规则: 以后看到 `filter` 必须先问它是实空间 source filter、RZ spectral filter, 还是 FDTD gather-side

Godfrey filter。三者都能和 NCI 稳定性有关，但它们不共享同一条源码路径，也不该在书中被统一写成“打开 Godfrey filter”。

`psatd.current_correction` 的边界也需要更精确。WarpX 在 `Source/WarpX.cpp:1653-1682` 根据 `current deposition, divE cleaning` 和用户输入决定默认值，并在 `Source/FieldSolver/WarpXPushFieldsEM.cpp:791-805、840-866` 把 `J、rho` 变换到 `spectral` 空间后调用 `PSATDCurrentCorrection()`。在标准连续性投影分支，核心形式可以写成

$$\mathbf{J}_{corr} = \mathbf{J} - \frac{\mathbf{k} \cdot \mathbf{J} - i(\rho^{n+1} - \rho^n)/\Delta t}{\mathbf{k} \cdot \mathbf{k}} \mathbf{k}.$$

`Source/FieldSolver/SpectralSolver/SpectralAlgorithms/PsatdAlgorithmJRhomSecondOrder.cpp:544-610` 中的 JRhom second-order current correction 就是这个投影式结构，并且源码断言它只在 `J` 常数、`rho` 线性时间依赖的组合下实现。Galilean 与 comoving 分支仍是连续性投影，只是连续性方程本身包含移动坐标相位。`Source/FieldSolver/SpectralSolver/SpectralAlgorithms/PsatdAlg` 中，当 $\mathbf{k}_c \cdot \mathbf{v}_{gal} \neq 0$ 时，源码先构造

$$\rho_{gal}^n = \rho^n \exp(i \mathbf{k}_c \cdot \mathbf{v}_{gal} \Delta t),$$

再用

$$\mathbf{k} \cdot \mathbf{J} - \frac{\mathbf{k}_c \cdot \mathbf{v}_{gal} (\rho^{n+1} - \rho_{gal}^n)}{1 - \exp(i \mathbf{k}_c \cdot \mathbf{v}_{gal} \Delta t)}$$

替换标准分支里的连续性残差。`Source/FieldSolver/SpectralSolver/SpectralAlgorithms/PsatdAlgorithmComoving.cpp:418-503` 也沿用相同思想，只是速度来自 comoving formulation。由此可见，`psatd.current_correction` 是让 `spectral current` 满足离散连续性/Gauss-law 约束的 projection，不是 Godfrey 2014 中为降低 NCI 增长率引入的 $\zeta(k)$ current scaling。

finite-order PSATD 是另一条独立轴线。`Source/WarpX.cpp:1557-1590` 读取 `psatd.nox/noy/noz`，字符串 "inf" 会转成 -1，但源码随后断言：如果不是 `psatd.periodic_single_box_fft = 1`，每个方向的 FFT order 必须是正整数。因此普通 multi-box PSATD 不能写成“无限阶全局谱求解器”。`Source/WarpX.cpp:3164-3182` 把 `nox_fft/noy_fft/noz_fft、periodic_single_box、update_with_rho、fft_do_time_averaging、solution type` 和 `J/rho time dependency` 传给 `SpectralSolver`；`Source/FieldSolver/SpectralSolver/SpectralSolver.cpp:60-107` 再按 PML、comoving、Galilean、first-order JRhom、second-order standard/JRhom 的优先级选择具体 algorithm。

这也解释了 `Examples/Tests/nci_psatd_stability` 的输入卡为什么不能只按“PSATD”一个词归类。`inputs_base_2d` 打开 `warpx.use_filter = 1`，并设置 `psatd.nox/noy/noz = 16`；`inputs_base_3d` 同样打开 `warpx.use_filter = 1`，但有限阶为 8。这些 regression 不是无限阶 periodic single-box PSATD 的泛化代表，而是有限阶 PSATD、filter、Galilean/current-correction/JRhom 分支的组合测试。

regression / analysis	代表输入	脚本判据	本章应如何表述
analysis_galilean.py 普通 Galilean / averaged Galilean	inputs_test_2d_galilean_psatd、inputs_test_3d_galilean_psatd、inputs_test_rz_galilean_psatd	读取最终 plotfile 的 <code>psi0/Ez</code> ，用电场能量除以脚本 <code>psatd</code> 稳定参考能量，并要求低于维度/分支容差	这是 NCI 抑制的强 regression gate，但主判据是能量比，不是色散关系重建
analysis_galilean.py current-correction 分支	inputs_test_2d_galilean_psatd、inputs_test_3d_galilean_psatd、inputs_test_rz_galilean_psatd 等	除电场能量外，还读取 <code>divE0</code> ， <code>psatd.current_correction</code> 检查 <code>max divE-rho/eps0 </code> 的相对误差	这条 gate 同时覆盖稳定性和 Gauss-law/continuity projection；它不能证明 Godfrey $\zeta(k)$ current scaling 已实现
RZ Galilean current-correction paired runtime	test_rz_galilean_psatd_current-correction、test_rz_galilean_psatd_current-correction-psb	if PSB 2-connection, <code>7.5405e-4 > 3e-4</code> ；PSB single-box single-rank energy <code>8.163e-11</code> ，charge <code>2.642e-11 < 1e-9</code> 均通过	当前项目级证据分层为 CHARGE_BOUNDARY 与 PASS；PSB 的严格 charge gate 不能外推到非 PSB 多 rank
analysis_psatd_CC1.py	inputs_test_3d_uniform_psatd	电场能量 <code>psa6e6R</code> 后要求 <code>CC1 1e-8</code>	这是 JRhom CC1 的 NCI 能量 gate，没有额外 charge-conservation assert

regression / analysis	代表输入	脚本判据	本章应如何表述
checksum-only RZ JRhom	<code>test_rz_psatd_JRhom_LL2</code>	CMake 中 <code>analysis=OFF</code> , 只走 checksum	只能写成 workflow/output regression, 不能写成强 NCI 物理论证
project-level RZ JRhom first-stage helper	同一 <code>test_rz_psatd_JRhom_LL2</code> 的 2-rank repeated/MPI ledger	<code>scripts/analyze_rz_jrhom</code> 这是可重复的 <code>baseline finite + energy</code> 通过, <code>ll2-no-timeavg-cleaning</code> reference 被拒绝	<code>scripts/analyze_rz_jrhom</code> 这是可重复的 <code>validation/handoff</code> evidence; 尚未改变上游 CMake 的 <code>analysis=OFF</code>

本版还把这 `local validation` 收成可重复的交接链: `scripts/build_rz_jrhom_first_stage_patch.py` 从 `rz-jrhom-reference-scan-mpl` 重建 helper、unified diff、provenance、submission packet、PR draft 和 bundle; 随后 `audit`、`report`、`preview` 与 `stage --dry-run` 对只读目标 checkout 进行一致性检查。当前实际状态仍是 `unstaged: analysis_rz_jrhom.py` 尚未写入 `../warpX`, `test_rz_psatd_JRhom_LL2` 的 CMake 行仍为 `analysis=OFF`。因此这里交付的是可审阅、可复现的 handoff asset, 不是已经进入 WarpX upstream CI 的 patch。对 bundle helper 的直接复核中, MPI=2 baseline 返回通过, `ll2-no-timeavg-cleaning` 返回拒绝; 两者与独立 first-stage contract 的 `baseline_ratio=0.9770894022295227`、`reference_ratio=1.0` 一致。

把这些源码和 regression 合起来, v0.20 对 Godfrey 2014 策略谱系的落点是:

1. filtering/smoothing 在 WarpX 中至少有实空间 bilinear filter 和 RZ spectral binomial filter 两条实现;
2. 源码名为 `NCIGodfreyFilter` 的路径属于 FDTD gather-side NCI corrector, 不能直接外推到 PSATD filter;
3. `psatd.current_correction` 是连续性投影, 不是 Godfrey $\zeta(k)$ current scaling;
4. finite-order PSATD 由 `psatd.nox/noy/noz` 和 `periodic_single_box_fft` 共同限定, NCI tests 中常见的是有限阶分支;
5. Galilean PSATD 是表示层面的移动坐标策略, JRhom 是源项时间积分策略, 二者和 filter/current correction 可以组合出不同 regression 入口, 但不能合并成同一个物理机制。

6.7 PSATD-JRhom: 多次源项沉积与一阶/二阶谱更新

`notes/code-reading/fieldsolver/07-psatd-jrhom.md` 把 PSATD-JRhom 从主循环到谱算法做了第一轮完整精读。物理上, JRhom 处理的是一个 PIC 时间步内 J 和 rho 不一定满足“电流常量、电荷线性”的假设。WarpX 使用 `psatd.JRhom` 字符串指定时间依赖:

```
std::string JRhom_input;
pp_psatd.query("JRhom", JRhom_input);
if (!JRhom_input.empty()) {
    WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
        JRhom_input.length() >= 3,
        "psatd.JRhom = '" + JRhom_input + "' input string is too short to parse."
    );
    m_JRhom = true;
    // parse time dependency of J from first character
    if (JRhom_input[0] == 'C') {
        time_dependency_J = TimeDependencyJ::Constant;
    }
    else if (JRhom_input[0] == 'L') {
        time_dependency_J = TimeDependencyJ::Linear;
    }
    else if (JRhom_input[0] == 'Q') {
        time_dependency_J = TimeDependencyJ::Quadratic;
    }
}
```

第一个字符控制 J, 第二个字符控制 rho, 后续数字控制子区间数 m。例如 CL1 是标准 PSATD 源项假设, LQ4 表示 J 分段线性、rho 分段二次, 并把大时间步拆成 4 个子区间。谱求解器分配时会把内部时间步改成

```
amrex::Real solver_dt = dt[lev];
if (WarpX::m_JRhom) { solver_dt /= static_cast<amrex::Real>(WarpX::m_JRhom_subintervals); }
```

因此 `PsatdAlgorithmJRhom*` 内部看到的是 $\delta t = \Delta t/m$ 。

JRhom 的外层 PIC loop 不走普通 PushPSATD()。它先推进粒子，但跳过普通沉积：

```
// Push particle from  $x^{\{n\}}$  to  $x^{\{n+1\}}$ 
//           from  $p^{\{n-1/2\}}$  to  $p^{\{n+1/2\}}$ 
```

```
const bool skip_deposition = true;
PushParticlesandDeposit(cur_time, skip_deposition);
```

当前源码入口是 ../warpx/Source/Evolve/WarpXEvolve.cpp:843-1008。初始化阶段先把 E/B/F/G 变换到谱空间 :866-869, 按需清零平均场 :871-872, 再对 rho 做初始沉积和 FFT :874-889, 并对非 constant J 做第一次沉积和 FFT :892-905。

随后在每个子区间按时间依赖类型重新沉积 J/rho:

```
const int n_deposit = WarpX::m_JRhom_subintervals;
const amrex::Real sub_dt = dt[0] / static_cast<amrex::Real>(n_deposit);
const int n_loop = (WarpX::fft_do_time_averaging) ? 2*n_deposit : n_deposit;

for (int i_deposit = 0; i_deposit < n_loop; i_deposit++)
{
    if (time_dependency_J != TimeDependencyJ::Constant) { PSATDMoveJNewToJOld(); }

    const amrex::Real t_deposit_current = (time_dependency_J == TimeDependencyJ::Linear) ?
        (i_deposit-n_deposit+1)*sub_dt : (i_deposit-n_deposit+0.5_rt)*sub_dt;

    const amrex::Real t_deposit_charge = (time_dependency_rho == TimeDependencyRho::Linear) ?
        (i_deposit-n_deposit+1)*sub_dt : (i_deposit-n_deposit+0.5_rt)*sub_dt;
```

线性依赖使用子区间端点，常量和二次依赖使用中点；二次依赖还会额外沉积一次，形成 old/mid/new 三个时间层：

```
if (time_dependency_J == TimeDependencyJ::Quadratic)
{
    PSATDMoveJNewToJMid();
    mypc->DepositCurrent( m_fields.get_mr_levels_alldirs(current_string, finest_level), dt[0], t_deposit_
    SyncCurrent("current_fp");
    PSATDForwardTransformJ("current_fp", "current_cp");
}
```

谱数组的 old/mid/new component 由 SpectralFieldIndex 分配：

```
if (time_dependency_J == TimeDependencyJ::Quadratic)
{
    Jx_old = c++; Jy_old = c++; Jz_old = c++;
    Jx_new = c++; Jy_new = c++; Jz_new = c++;
    Jx_mid = c++; Jy_mid = c++; Jz_mid = c++;
}
else if (time_dependency_J == TimeDependencyJ::Linear)
{
    Jx_old = c++; Jy_old = c++; Jz_old = c++;
    Jx_new = c++; Jy_new = c++; Jz_new = c++;
}
```

二阶 JRhom kernel 把这些时间层组合成多项式系数：

```
const Complex a_jx = (J_quadratic) ? (Jx_new - 2._rt * Jx_mid + Jx_old) : 0._rt;
const Complex b_jx = (J_linear || J_quadratic) ? (Jx_new - Jx_old) : 0._rt;
const Complex c_jx = (J_linear) ? (Jx_new + Jx_old)/2._rt : Jx_mid;

const Complex a_rho = (rho_quadratic) ? (rho_new - 2._rt * rho_mid + rho_old) : 0._rt;
const Complex b_rho = (rho_linear || rho_quadratic) ? (rho_new - rho_old) : 0._rt;
const Complex c_rho = (rho_linear) ? (rho_new + rho_old)/2._rt : rho_mid;
```

这对应每个子区间内

$$\tilde{\mathbf{J}}(t) = \mathbf{a}_J \tau^2 + \mathbf{b}_J \tau + \mathbf{c}_J, \quad \tilde{\rho}(t) = a_\rho \tau^2 + b_\rho \tau + c_\rho.$$

电场更新式以 Ex 为例：

```
fields(i,j,k,Idx.Ex) = C * Ex_old
+ I * c2 * S_ck * (ky * Bz_old - kz * By_old)
+ Y3 * a_jx + Y2 * b_jx - S_ck/ep0 * c_jx
+ I * c2 * kx * sum_rho;
```

这里 (ky*Bz-kz*By) 是谱空间 curl(B), Y3/Y2/S_ck 分别积分二次、一次和常量电流源项, sum_rho 则是电荷密度多项式带来的纵向修正。

磁场更新式同样含有 k x J 的多项式积分：

```
fields(i,j,k,Idx.Bx) = C * Bx_old
- I * S_ck * (ky * Ez_old - kz * Ey_old)
- I * Y1 * (ky * a_jz - kz * a_jy)
+ I * Y5 * (ky * b_jz - kz * b_jy)
+ I * Y4 * (ky * c_jz - kz * c_jy );
```

JRhom 的支持边界也必须写清楚：源码禁止 Vay deposition 与 JRhom 组合，默认关闭 JRhom current correction，并禁止 Galilean PSATD：

```
if (current_deposition_algo == CurrentDepositionAlgo::Vay) {
    WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
        m_JRhom == false,
        "Vay deposition not implemented with JRhom algorithm");
}
```

```
if (m_JRhom) { current_correction = false; }
```

在当前 OneStep_JRhom() 中，二次 J 的中点沉积位于 ../warpx/Source/Evolve/WarpXEvolve.cpp:941-947, rho 的 old/new/mid 处理位于 :949-975, 每个子区间的谱推进位于 :984-985。若开启 time averaging, 平均场在 :997-1007 缩放并反变换回实空间。

```
if (m_JRhom)
{
    WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
        v_galilean_is_zero,
        "PSATD-JRhom algorithm not implemented with Galilean PSATD"
    );
}
```

所以 JRhom 不是“普通 PSATD 上再加一个系数表”，而是重排了源项采样和谐推进：粒子先完整推进，源项在多个相对时刻沉积，谱场按 old/mid/new 保存源项时间层，最后在每个子区间用解析积分推进 E/B/F/G。

6.7.1 v0.28 JRhom second-order Y1-Y8 系数的源码公式闭环

v0.28 把 notes/code-reading/fieldsolver/19-psatd-jrhom-y-coefficients.md 补成独立图谱。这个图谱的第一条规则是防混写：PsatdAlgorithmJRhomSecondOrder.cpp 的 Y1-Y8 是实系数，属于 JRhom 多项式源项积分；它们不是上一节 Cartesian Galilean/standard PSATD average-field 分支里的 complex Y1-Y4。

源码分配边界在 PsatdAlgorithmJRhomSecondOrder.cpp:59-77。Y1-Y5 总是分配，Y6-Y8 只在 time_averaging 打开时分配：

```
Y1_coef = SpectralRealCoefficients(ba, dm, 1, 0);
Y2_coef = SpectralRealCoefficients(ba, dm, 1, 0);
Y3_coef = SpectralRealCoefficients(ba, dm, 1, 0);
Y4_coef = SpectralRealCoefficients(ba, dm, 1, 0);
Y5_coef = SpectralRealCoefficients(ba, dm, 1, 0);

if (time_averaging)
{
```

```

Y6_coef = SpectralRealCoefficients(ba, dm, 1, 0);
Y7_coef = SpectralRealCoefficients(ba, dm, 1, 0);
Y8_coef = SpectralRealCoefficients(ba, dm, 1, 0);
}

```

第二条规则是时间步边界。SpectralSolver.cpp 在 JRhom 打开时把 solver_dt 除以 m_JRhom_subintervals, 所以 PsatdAlgorithmJRhomSecondOrder 里看到的 Δt 是每个 JRhom 子区间长度, 而不是外层 PIC 大步长。

二阶类先把 old/mid/new 源项写成局部多项式:

$$\mathbf{J}(\tau) = \mathbf{a}_J \tau^2 + \mathbf{b}_J \tau + \mathbf{c}_J, \quad \rho(\tau) = a_\rho \tau^2 + b_\rho \tau + c_\rho.$$

源码对应为:

```

const Complex a_jx = (J_quadratic) ? (Jx_new - 2._rt * Jx_mid + Jx_old) : 0._rt;
const Complex b_jx = (J_linear || J_quadratic) ? (Jx_new - Jx_old) : 0._rt;
const Complex c_jx = (J_linear) ? (Jx_new + Jx_old)/2._rt : Jx_mid;

const Complex a_rho = (rho_quadratic) ? (rho_new - 2._rt * rho_mid + rho_old) : 0._rt;
const Complex b_rho = (rho_linear || rho_quadratic) ? (rho_new - rho_old) : 0._rt;
const Complex c_rho = (rho_linear) ? (rho_new + rho_old)/2._rt : rho_mid;

```

共享谱量仍是

$$\omega_s = c|\mathbf{k}_s|, \quad C = \cos(\omega_s \Delta t), \quad S_{ck} = \frac{\sin(\omega_s \Delta t)}{\omega_s},$$

并在 $\omega_s = 0$ 时取 $S_{ck} = \Delta t$ 。Y1-Y5 的非零模公式为:

$$\begin{aligned}
Y_1 &= \frac{(1-C)(8-\omega_s^2 \Delta t^2) - 4S_{ck}\omega_s^2 \Delta t}{2\epsilon_0 \Delta t^2 \omega_s^4}, \\
Y_2 &= \frac{2(C-1) + S_{ck}\omega_s^2 \Delta t}{2\epsilon_0 \Delta t \omega_s^2}, \quad Y_3 = \frac{S_{ck}\omega_s(8-\omega_s^2 \Delta t^2) - 4(1+C)\omega_s \Delta t}{2\epsilon_0 \Delta t^2 \omega_s^3}, \\
Y_4 &= \frac{1-C}{\epsilon_0 \omega_s^2}, \quad Y_5 = \frac{(1+C)\Delta t - 2S_{ck}}{2\epsilon_0 \Delta t \omega_s^2}.
\end{aligned}$$

零模分支分别是:

$$Y_1 = -\frac{\Delta t^2}{12\epsilon_0}, \quad Y_2 = 0, \quad Y_3 = -\frac{\Delta t}{6\epsilon_0}, \quad Y_4 = \frac{\Delta t^2}{2\epsilon_0}, \quad Y_5 = -\frac{\Delta t^2}{12\epsilon_0}.$$

这些系数在 ordinary field push 里分工如下。Y3/Y2/S_ck 分别积分二次、一次、常量电流对电场的贡献:

```

fields(i,j,k,Idx.Ex) = C * Ex_old
+ I * c2 * S_ck * (ky * Bz_old - kz * By_old)
+ Y3 * a_jx + Y2 * b_jx - S_ck/ep0 * c_jx
+ I * c2 * kx * sum_rho;

```

Y1/Y5/Y4 则进入磁场中的 $\mathbf{k} \times \mathbf{J}$ 多项式积分:

```

fields(i,j,k,Idx.Bx) = C * Bx_old
- I * S_ck * (ky * Ez_old - kz * Ey_old)
- I * Y1 * (ky * a_jz - kz * a_jy)
+ I * Y5 * (ky * b_jz - kz * b_jy)
+ I * Y4 * (ky * c_jz - kz * c_jy);

```

同一组 Y1/Y5/Y4 还出现在电荷纵向组合里:

```
const Complex sum_rho = Y1 * a_rho - Y5 * b_rho - Y4 * c_rho;
```

如果打开 `dive_cleaning`, `F` 也复用这组源项积分:

```
fields(i,j,k,Idx.F) = C * F_old + S_ck * I * k_dot_E
+ I * ( Y1 * k_dot_ddJ - Y5 * k_dot_dJ - Y4 * k_dot_J_mid )
+ Y3 * a_rho + Y2 * b_rho - S_ck/ep0 * c_rho;
```

`Y6-Y8` 只属于 time-averaged field 累计。非零模公式为:

$$Y_6 = \frac{\Delta t^3 \omega_s^3 - 3\Delta t^2 \omega_s^3 S_{ck} - 12\Delta t \omega_s (1 + C) + 24\omega_s S_{ck}}{6\epsilon_0 \Delta t^2 \omega_s^5},$$

$$Y_7 = \frac{\Delta t \omega_s^2 S_{ck} + 2C - 2}{2\epsilon_0 \Delta t \omega_s^4}, \quad Y_8 = \frac{\Delta t - S_{ck}}{\epsilon_0 \omega_s^2}.$$

零模分支为:

$$Y_6 = \frac{\Delta t^3}{30\epsilon_0}, \quad Y_7 = -\frac{\Delta t^3}{24\epsilon_0}, \quad Y_8 = \frac{\Delta t^3}{6\epsilon_0}.$$

源码注释明确说 average-field 是累加形式, 因为 `JRhom` 可配合 sub-cycling:

```
fields(i,j,k,Idx.Ex_avg) += S_ck * Ex_old
+ I * c2 * ep0 * Y4 * (ky * Bz_old - kz * By_old)
- I * c2 * kx * (Y6 * a_rho + Y7 * b_rho + Y8 * c_rho)
+ ( Y1 * a_jx - Y5 * b_jx - Y4 * c_jx );
```

因此 v0.28 对第 6 章的写作边界是: `Y1-Y5` 是 `JRhom` ordinary field push 的源项积分, `Y6-Y8` 是 `JRhom` time averaging 的累计积分; 二者都不应和 Cartesian Galilean average-field 的 `Y1-Y4` 合并。后续如果继续拆 `RZ/Galilean RZ`, 也要按 algorithm class 和 field layout 单独建表。

6.8 RZ PSATD: Hankel transform、azimuthal modes 与 Ep/Em

`notes/code-reading/fieldsolver/08-psatd-rz-hankel.md` 进入 RZ spectral solver。RZ PSATD 不能理解成“二维 PSATD”。它使用 azimuthal mode decomposition:

$$F(r, z, \theta) = \sum_m \Re(F_m(r, z)e^{im\theta}),$$

并且每个实空间 MultiFab 的 component 数为

$$n_{\text{comps}} = 2n_{\text{modes}} - 1.$$

源码中:

```
utils::parser::queryWithParser(pp_warpx, "n_rz_azimuthal_modes", n_rz_azimuthal_modes);
WARPX_ALWAYS_ASSERT_WITH_MESSAGE( n_rz_azimuthal_modes > 0,
    "The number of azimuthal modes (n_rz_azimuthal_modes) must be at least 1");
```

```
ncomps = n_rz_azimuthal_modes*2 - 1;
```

RZ spectral solver 入口选择标准、Galilean 或 PML 算法:

```
if (with_pml) {
    PML_algorithm = std::make_unique<PsatdAlgorithmPmlRZ>(
        k_space, dm, m_spectral_index, n_rz_azimuthal_modes, norder_z, grid_type, dt);
}
if (v_galilean[2] == 0) {
    algorithm = std::make_unique<PsatdAlgorithmRZ>(
```



```

        k_space, dm, m_spectral_index, n_rz_azimuthal_modes, norder_z, grid_type, dt,
        update_with_rho, fft_do_time_averaging, time_dependency_J, time_dependency_rho, dive_cleaning, dive
    } else {
        algorithm = std::make_unique<PsatdAlgorithmGalileanRZ>(
            k_space, dm, m_spectral_index, n_rz_azimuthal_modes, norder_z, grid_type, v_galilean, dt, update_w
    }

```

RZ 的 k_z 来自 z 向 FFT, 而 k_r 来自径向 Hankel transform 的 Bessel roots. SpectralKSpaceRZ 只构造 z 向 k :

```

const int i_dim = 1;
const bool only_positive_k = false;
k_vec[i_dim] = getKComponent(dm, realspace_ba, i_dim, only_positive_k);

```

Hankel transformer 为每个 mode 建立三套 transform:

```

for (int mode=0 ; mode < m_n_rz_azimuthal_modes ; mode++) {
    dht0[mode] = std::make_unique<HankelTransform>(mode, mode, m_nr, rmax);
    dhtp[mode] = std::make_unique<HankelTransform>(mode+1, mode, m_nr, rmax);
    dhtm[mode] = std::make_unique<HankelTransform>(mode-1, mode, m_nr, rmax);
}

```

标量用 dht0. 横向矢量场先组合为

$$F_p = \frac{F_r - iF_\theta}{2}, \quad F_m = \frac{F_r + iF_\theta}{2},$$

源码中对应:

```

// temp_p = (F_r - I*F_t)/2
// temp_m = (F_r + I*F_t)/2
F_r_physical_array(i,j,k,mode_r) = 0.5_rt*(r_real + t_imag);
F_r_physical_array(i,j,k,mode_i) = 0.5_rt*(r_imag - t_real);
F_t_physical_array(i,j,k,mode_r) = 0.5_rt*(r_real - t_imag);
F_t_physical_array(i,j,k,mode_i) = 0.5_rt*(r_imag + t_real);

```

然后分别做 dhtp/dhtm:

```

dhtp[mode]->HankelForwardTransform(F_r_physical, mode_r, G_p_spectral, mode_r);
dhtm[mode]->HankelForwardTransform(F_t_physical, mode_r, G_m_spectral, mode_r);

```

所以 PsatdAlgorithmRZ 中的 E_p/E_m 不是 E_x/E_y , 而是由 E_r/E_θ 组合出的谱分量:

```

int const Ep_m = Idx.Ex + Idx.n_fields*mode;
int const Em_m = Idx.Ey + Idx.n_fields*mode;
int const Ez_m = Idx.Ez + Idx.n_fields*mode;
int const Bp_m = Idx.Bx + Idx.n_fields*mode;
int const Bm_m = Idx.By + Idx.n_fields*mode;
int const Bz_m = Idx.Bz + Idx.n_fields*mode;

```

RZ PSATD 的电场更新以 $E_p/E_m/E_z$ 为变量:

```

fields(i,j,k,Ep_m) = C*Ep_old
    + S_ck*(-c2*I*kr/2._rt*Bz_old + c2*kz*Bp_old - inv_ep0*Jp)
    + 0.5_rt*kr*rho_diff;
fields(i,j,k,Em_m) = C*Em_old
    + S_ck*(-c2*I*kr/2._rt*Bz_old - c2*kz*Bm_old - inv_ep0*Jm)
    - 0.5_rt*kr*rho_diff;
fields(i,j,k,Ez_m) = C*Ez_old
    + S_ck*(c2*I*kr*Bp_old + c2*I*kr*Bm_old - inv_ep0*Jz)
    - I*kz*rho_diff;

```

RZ 的谱散度写成

$$\nabla \cdot \mathbf{E} \rightarrow k_r(E_p - E_m) + ik_z E_z.$$

源码中 update_with_rho=0 时用它重构电荷项:

```
Complex const divE = kr*(Ep_old - Em_old) + I*kz*Ez_old;
Complex const divJ = kr*(Jp - Jm) + I*kz*Jz;
```

```
rho_diff = (X2 - X3)*PhysConst::epsilon_0*divE - X2*dt*divJ;
```

RZ current correction 也沿这个谱散度结构投影:

```
Complex const F = - ((rho_new - rho_old)/dt + I*kz*Jz + kr*(Jp - Jm))/k_norm2;
```

```
fields(i,j,k,Jp_m) += +0.5_rt*kr*F;
fields(i,j,k,Jm_m) += -0.5_rt*kr*F;
fields(i,j,k,Jz_m) += -I*kz*F;
```

反变换后, RZ 还要按 mode 对称性填充轴下 guard cells:

```
if (i < 0) {
    ii = -i - 1;
    if (icomp == 0) {
        // Mode zero is symmetric
        sign = +1._rt;
    } else {
        // Odd modes are anti-symmetric
        const auto imode = (icomp + 1)/2;
        sign = static_cast<amrex::Real>(std::pow(-1._rt, imode));
    }
}
```

这说明 RZ PSATD 的“正确性”同时依赖三件事: Hankel/Bessel 谱基、Ep/Em 横向矢量代数、以及轴上/轴下 mode 对称性。把 Cartesian PSATD 的 kx,ky,kz 公式机械删去一个方向, 得不到 WarpX 的 RZ 实现。

6.8.1 v0.29 RZ/Galilean RZ PSATD 系数边界

v0.29 把 notes/code-reading/fieldsolver/20-psatd-rz-galilean-rz-coefficients.md 补成 RZ 系数边界图谱。第一条结论先写清楚: RZ 标准 PSATD 的 X1-X3/X5-X6 和 Galilean RZ 的 X1-X4/Theta2/T_rho 不能和 Cartesian 同名系数直接合并, 因为 RZ 的谱基、横向字段和 current correction 都不同。

PsatdAlgorithmRZ.cpp:43-55 总是分配实系数 C/S_ck/X1-X3, 只有 linear-J time averaging 才分配 X5/X6。standard RZ 的非零模公式为:

$$C = \cos(ck\Delta t), \quad S_{ck} = \frac{\sin(ck\Delta t)}{ck},$$

$$X_1 = \frac{1-C}{\epsilon_0 c^2 k^2}, \quad X_2 = \frac{1-S_{ck}/\Delta t}{\epsilon_0 k^2}, \quad X_3 = \frac{C-S_{ck}/\Delta t}{\epsilon_0 k^2},$$

其中 $k = \sqrt{k_r^2 + k_z^2}$ 。零模分支为:

$$C = 1, \quad S_{ck} = \Delta t, \quad X_1 = \frac{\Delta t^2}{2\epsilon_0}, \quad X_2 = \frac{c^2 \Delta t^2}{6\epsilon_0}, \quad X_3 = -\frac{c^2 \Delta t^2}{3\epsilon_0}.$$

这些系数进入 Ep/Em/Ez 更新时, 径向符号并不是 Cartesian 向量式的简单投影:

```
fields(i,j,k,Ep_m) = C*Ep_old
    + S_ck*(-c2*I*kr/2._rt*Bz_old + c2*kz*Bp_old - inv_ep0*Jp)
    + 0.5_rt*kr*rho_diff;
```

```
fields(i,j,k,Em_m) = C*Em_old
+ S_ck*(-c2*I*kr/2._rt*Bz_old - c2*kz*Bm_old - inv_ep0*Jm)
- 0.5_rt*kr*rho_diff;
```

standard RZ 的 time averaging 支持边界很窄: 只允许 psatd.time_dependency_J=linear。若 time averaging 配合非线性 J, 源码直接 abort。linear-J 分支中, X5/X6 的非零模公式为:

$$X_5 = \frac{c^2}{\epsilon_0} \left[\frac{S_{ck}}{\omega^2} - \frac{1-C}{\omega^4 \Delta t} - \frac{\Delta t}{2\omega^2} \right], \quad X_6 = \frac{c^2}{\epsilon_0} \left[\frac{1-C}{\omega^4 \Delta t} - \frac{\Delta t}{2\omega^2} \right],$$

其中 $\omega = ck$ 。零模分支为

$$X_5 = -\frac{c^2 \Delta t^3}{8\epsilon_0}, \quad X_6 = -\frac{c^2 \Delta t^3}{24\epsilon_0}.$$

X5/X6 在 average-field 更新中分别乘 old/new rho 和 old/new J:

```
fields(i,j,k,Ep_avg_m) += S_ck * Ep_old
+ c2 * ep0 * X1 * (kz * Bp_old - I * kr * 0.5_rt * Bz_old)
- kr * 0.5_rt * (X5 * rho_old + X6 * rho_new) + X3/c2 * Jp - X2/c2 * Jp_new;
```

Galilean RZ 又是另一套边界。PsatdAlgorithmGalileanRZ.cpp 只使用 m_v_galilean[2], 也就是轴向速度:

```
const amrex::Real vz = m_v_galilean[2];
amrex::Real const kv = kz*vz;
```

它分配 C/S_ck 两个实系数, 以及 complex X1-X4/Theta2/T_rho。令

$$k_v = k_z v_z, \quad \nu = \frac{k_v}{ck}, \quad \theta = e^{ik_v \Delta t/2},$$

则源码中

$$\Theta_2 = \theta^2, \quad T_\rho = \begin{cases} -\Delta t, & k_z = 0, \\ \frac{1-\theta^2}{ik_z v_z}, & k_z \neq 0. \end{cases}$$

一般分支 nu != 1 && nu != 0 先构造

$$x_1 = \frac{\theta^* - C\theta + ik_v S_{ck} \theta}{1 - \nu^2},$$

再定义

$$X_1 = \frac{\theta x_1}{\epsilon_0 c^2 k^2}, \quad X_2 = \frac{x_1 - \theta(1-C)}{(\theta^* - \theta)\epsilon_0 k^2}, \quad X_3 = \frac{x_1 - \theta^*(1-C)}{(\theta^* - \theta)\epsilon_0 k^2},$$

$$X_4 = ik_v X_1 - \frac{\theta^2 S_{ck}}{\epsilon_0}.$$

当 nu == 0, Galilean RZ 回到 standard RZ 的 X1-X3, 并取 X4=-S_ck/epsilon0。当 nu == 1, 源码有专门极限分支, 避免 $1 - \nu^2$ 和 $\theta^* - \theta$ 分母退化。零模分支则取 Theta2=1, X4=-dt/epsilon0。

Galilean RZ 的更新式与 standard RZ 使用相同 Ep/Em layout, 但旧场和 curl 项乘 T2=Theta2, 电流直接用 X4:

```
fields(i,j,k,Ep_m) = T2*C*Ep_old
+ T2*S_ck*(-c2*I*kr/2._rt*Bz_old + c2*kz*Bp_old)
+ X4*Jp + 0.5_rt*kr*rho_diff;
```

电荷项中旧时刻 rho 也带 Galilean 相位:

```
if (update_with_rho) {
    rho_diff = X2*rho_new - T2*X3*rho_old;
} else {
    rho_diff = T2*(X2 - X3)*myeps0*divE + T_rho*X2*divJ;
}
```

RZ current correction 沿 $E_p/E_m/J_p/J_m$ 的谱梯度方向修正, 而不是 Cartesian 的 $J_x/J_y/J_z$ 投影。standard RZ 使用

```
Complex const F = - ((rho_new - rho_old)/dt + I*kz*Jz + kr*(Jp - Jm))/k_norm2;
fields(i,j,k,Jp_m) += +0.5_rt*kr*F;
fields(i,j,k,Jm_m) += -0.5_rt*kr*F;
fields(i,j,k,Jz_m) += -I*kz*F;
```

Galilean RZ 则把 rho_old 乘 theta2, 并用 Galilean 连续性残差替换普通时间差分。两条 RZ 路径都显式不支持 Vay deposition。

所以 v0.29 对第 6 章的边界补充是: RZ 标准 PSATD、RZ time averaging、Galilean RZ 和 Cartesian/Galilean/JRhom/PML 都应各自成表。它们可以共享 C/S_ck 这类基础三角函数, 但不能共享同一组更新式语义。

6.8.2 v0.30 comoving PSATD 系数与验证边界

v0.30 把 notes/code-reading/fieldsolver/21-psatd-comoving-coefficients.md 补成 regular-domain comoving PSATD 的系数图谱。这个分支在源码中不是 Galilean PSATD 的小选项, 而是 SpectralSolver.cpp 里优先级最高的 regular-domain algorithm:

```
if (v_comoving[0] != 0. || v_comoving[1] != 0. || v_comoving[2] != 0.)
{
    algorithm = std::make_unique<PsatdAlgorithmComoving>(...);
}
else if (v_galilean[0] != 0. || v_galilean[1] != 0. || v_galilean[2] != 0.)
{
    algorithm = std::make_unique<PsatdAlgorithmGalilean>(...);
}
```

WarpX.cpp 解析参数时进一步保证 v_comoving 与 v_galilean 互斥。psatd.use_default_v_comoving=1 只能在 warpx.gamma_boost 已设置时使用, 并只自动填 z 向 normalized velocity:

```
m_v_comoving[2] = -std::sqrt(1._rt - 1._rt / (gamma_boost * gamma_boost));
```

随后源码把 m_v_comoving 乘以 PhysConst::c。因此正文要区分输入卡中的“光速单位速度”和 algorithm 内部的 SI 速度。comoving 还要求 direct current deposition, 强制 psatd.update_with_rho=1, 并且不支持 Esirkepov/Villasenor、Vay deposition 或非默认 J constant / rho linear 时间依赖。

PsatdAlgorithmComoving 分配 C/S_ck 两个实系数, 以及 complex X1-X4/Theta2:

```
C_coef = SpectralRealCoefficients(ba, dm, 1, 0);
S_ck_coef = SpectralRealCoefficients(ba, dm, 1, 0);
X1_coef = SpectralComplexCoefficients(ba, dm, 1, 0);
X2_coef = SpectralComplexCoefficients(ba, dm, 1, 0);
X3_coef = SpectralComplexCoefficients(ba, dm, 1, 0);
X4_coef = SpectralComplexCoefficients(ba, dm, 1, 0);
Theta2_coef = SpectralComplexCoefficients(ba, dm, 1, 0);
```

这里最容易漏掉的是两套波数的分工: C/S_ck 使用 finite-order modified wave number,

$$\omega_{\text{mod}} = c|\mathbf{k}_{\text{mod}}|, \quad C = \cos(\omega_{\text{mod}}\Delta t), \quad S_{ck} = \frac{\sin(\omega_{\text{mod}}\Delta t)}{\omega_{\text{mod}}},$$

而 comoving 相位使用 infinite-order $\mathbf{k}$ 与 comoving velocity:

$$k_v = \mathbf{k} \cdot \mathbf{v}_c, \quad \omega = c|\mathbf{k}|, \quad \nu = -\frac{k_v}{\omega},$$

$$\theta = e^{i\nu\omega\Delta t/2}, \quad \Theta_2 = \theta^2.$$

一般非退化分支满足 $\nu \neq \omega_{\text{mod}}/\omega$ 、 $\nu \neq -\omega_{\text{mod}}/\omega$ 且 $\nu \neq 0$ 。源码先构造

$$x_1 = \frac{\omega^2}{\omega_{\text{mod}}^2 - \nu^2 \omega^2} (\theta^* - \theta C + i\nu\omega\theta S_{ck}),$$

再定义

$$X_1 = \frac{x_1}{\epsilon_0 \omega^2},$$

$$X_2 = \frac{c^2 (x_1 \omega_{\text{mod}}^2 - \theta(1-C)\omega^2)}{(\theta^* - \theta)\epsilon_0 \omega^2 \omega_{\text{mod}}^2},$$

$$X_3 = \frac{c^2 (x_1 \omega_{\text{mod}}^2 - \theta^*(1-C)\omega^2)}{(\theta^* - \theta)\epsilon_0 \omega^2 \omega_{\text{mod}}^2},$$

$$X_4 = i\nu\omega X_1 - \frac{\theta S_{ck}}{\epsilon_0}.$$

ordinary field push 仍使用 Cartesian Ex/Ey/Ez/Bx/By/Bz layout。电场中 X4 乘电流, X2/X3 分别乘 new/old charge density:

```
fields(i,j,k,Idx.Ex) = C*Ex_old + S_ck*c2*I*(ky_mod*Bz_old - kz_mod*By_old)
+ X4*Jx - I*(X2*rho_new - X3*rho_old)*kx_mod;
```

磁场中 X1 乘 current curl:

```
fields(i,j,k,Idx.Bx) = C*Bx_old - S_ck*I*(ky_mod*Ez_old - kz_mod*Ey_old)
+ X1*I*(ky_mod*Jz - kz_mod*Jy);
```

当 $\nu == 0$ 时, comoving X1-X4 退到 standard PSATD 形式:

$$X_1 = \frac{1-C}{\epsilon_0 \omega_{\text{mod}}^2}, \quad X_2 = \frac{c^2(1-S_{ck}/\Delta t)}{\epsilon_0 \omega_{\text{mod}}^2}, \quad X_3 = \frac{c^2(C-S_{ck}/\Delta t)}{\epsilon_0 \omega_{\text{mod}}^2}, \quad X_4 = -\frac{S_{ck}}{\epsilon_0}.$$

当 $\nu == \omega_{\text{mod}}/\omega$ 或 $\nu == -\omega_{\text{mod}}/\omega$, 源码用 tmp1/tmp2 与半步相位写出专门极限, 避免 $\omega_{\text{mod}}^2 - \nu^2 \omega^2$ 分母退化。knorm_mod 与 knorm 任一为零时也有单独分支; 完全零模时:

$$C = 1, \quad S_{ck} = \Delta t, \quad \Theta_2 = 1, \quad X_1 = \frac{\Delta t^2}{2\epsilon_0}, \quad X_2 = \frac{c^2 \Delta t^2}{6\epsilon_0}, \quad X_3 = -\frac{c^2 \Delta t^2}{3\epsilon_0}, \quad X_4 = -\frac{\Delta t}{\epsilon_0}.$$

comoving current correction 也有自己的 continuity 残差。若 $\mathbf{k} \cdot \mathbf{v}_c \neq 0$, 源码使用

```
const Complex theta = amrex::exp(- I * k_dot_v * dt * 0.5_rt);
const Complex den = 1._rt - theta * theta;
fields(i,j,k,Idx.Jx_mid) = Jx
- (kmod_dot_J + k_dot_v * theta * (rho_new - rho_old) / den)
* kx_mod / (knorm_mod * knorm_mod);
```

若 $\mathbf{k} \cdot \mathbf{v}_c = 0$, 则退回普通 continuity correction:

```
fields(i,j,k,Idx.Jx_mid) = Jx
- (kmod_dot_J - I * (rho_new - rho_old) / dt)
* kx_mod / (knorm_mod * knorm_mod);
```

验证边界同样要写窄。当前 Examples/Tests/nci_psatd_stability/CMakeLists.txt 注册了 test_2d_comoving_psatd_hybrid, 输入卡使用 algo.current_deposition=direct、psatd.use_default_v_comoving=1、warpx.gamma_boost=13.、warpx.grid_type=hybrid, 但 analysis 字段是 OFF, 只接了

```
"analysis_default_regression.py --path diags/diag1000400"
```

所以本书目前只能把它称为 comoving boosted-frame hybrid PSATD 的 checksum regression, 而不能声称它已经提供和 analysis_galilean.py 同等级的 NCI 增长率或稳定性强判据。

v0.30 之后, 第 6 章的 PSATD 系数边界已经形成一套明确分层: Cartesian Galilean X1-X4、average-field Psi/Y、JRhom Y1-Y8、RZ/Galilean RZ X/Theta/T_rho、comoving X1-X4/Theta2 和 PML C1-C25 都按 algorithm class 分开; 后续工作应从“验证强度”继续推进, 而不是继续把同名符号并表。

6.8.3 v0.31 comoving PSATD regression analysis 方案

v0.31 把 notes/code-reading/fieldsolver/22-psatd-comoving-regression-analysis-plan.md 补成 test_2d_comoving_psa 的 analysis 升级方案。当前 CMake 注册是:

```
add_warpx_test(
    test_2d_comoving_psatd_hybrid
    2
    2
    inputs_test_2d_comoving_psatd_hybrid
    OFF
    "analysis_default_regression.py --path diags/diag1000400"
    OFF
)
```

这表示它现在只有 checksum 自动消费者。输入卡本身已经是很明确的 boosted-frame hybrid comoving producer:

```
algo.maxwell_solver = psatd
algo.current_deposition = direct
psatd.use_default_v_comoving = 1
psatd.current_correction = 0
warpx.grid_type = hybrid
warpx.gamma_boost = 13.
warpx.do_moving_window = 1
warpx.use_filter = 1
diag1.fields_to_plot = Ex Ey Ez Bx By Bz jx jy jz rho
```

因此 v0.31 的第一条边界是: 现有输出能支持 Ex/Ey/Ez/Bx/By/Bz/jx/jy/jz/rho 的 analysis, 但不能直接支持 divE-rho/epsilon0 的 charge-conservation gate, 因为 divE 没有输出。

可直接落地的第一阶段 analysis 应包括三类检查。第一, finite field sanity: 所有输出字段都必须有限, 避免 checksum 之外的 NaN/Inf 被掩盖:

```
for name in ["Ex", "Ey", "Ez", "Bx", "By", "Bz", "jx", "jy", "jz", "rho"]:
    arr = all_data["boxlib", name].squeeze().v
    assert np.all(np.isfinite(arr))
```

第二, 沿用 Galilean analysis 的电场能量 proxy:

$$U_E = \sum \frac{\epsilon_0}{2} (E_x^2 + E_y^2 + E_z^2).$$

但 energy_ref_comoving 不能借用 analysis_galilean.py 中的 Galilean reference; 它必须来自 comoving reference run 或 CI baseline 的明确记录。初版脚本可以写成:

```
energy = np.sum(scc.epsilon_0 / 2 * (Ex**2 + Ey**2 + Ez**2))
err_energy = energy / energy_ref_comoving
assert err_energy < tol_energy
```

第三，增加局部异常 spike sanity:

$$R_E = \frac{\max |\mathbf{E}|}{\text{p99}(|\mathbf{E}|) + \delta}.$$

这个 ratio 可以发现单点谱异常或边界异常，但仍只是异常探测，不是 NCI 增长率拟合。

如果后续要把这个 regression 推到更强的 Gauss-law 证据，输入卡必须先增加：

```
diag1.fields_to_plot = Ex Ey Ez Bx By Bz jx jy jz rho divE
```

然后才能定义

$$\epsilon_G = \frac{\|\nabla \cdot \mathbf{E} - \rho/\epsilon_0\|_\infty}{\max(\|\nabla \cdot \mathbf{E}\|_\infty, \|\rho/\epsilon_0\|_\infty, \delta)}.$$

由于当前 comoving 输入卡设置 psatd.current_correction=0，这条 Gauss-law gate 的语义应写成“末态 Gauss-law drift diagnostic”，不能写成“current-correction correctness”。

v0.31 对读者的实际增量是把“checksum-only”拆成一条可执行的升级路线：

层级	当前状态	可支持的说法
checksum	已存在	末态 plotfile 与 baseline 一致
finite field sanity	可直接实现	捕捉 NaN/Inf，不是物理强判据
electric energy ceiling	需要 reference 标定	可作为稳定性 proxy，不是增长率拟合
spike ratio	需要 reference 标定	捕捉局部异常，不替代能量判据
Gauss-law drift	需要输出 divE	验证末态 Gauss-law drift，需和 current_correction=0 语义分开

所以后续真正提交 WarpX 侧 patch 时，最小完整包应包含 analysis_comoving.py、CMake wiring、必要时的 divE 输出变更，以及 energy_ref/tol_energy/spike_ratio_ref 的 reference 来源说明。

6.8.4 comoving PSATD reference 标定与 patch 收口条件

仅把 analysis_comoving.py 的脚本骨架写出来，还不够形成可提交的 WarpX regression patch。真正难的部分是 reference 标定：energy_ref、tol_energy 和 spike_ratio_ref 不能随手抄一次本机输出，而要像 analysis_galilean.py 那样有清楚的物理语义和 provenance。

这里最值得借用的是 Galilean 那条现成模式。analysis_galilean.py 不是把稳定运行和自身比较，而是把稳定配置的末态电场能量和一个已知不稳定对照配置比较；analysis_psatd_CC1.py 则展示了另一种更窄的写法：单独给某一条 test 固定一个 energy_ref，只保留能量比 gate。comoving 的第一版 patch 更接近第二种形式，但其 reference 语义仍应继承第一种思路，也就是先明确“如果关闭当前 comoving 稳定化关键开关，末态电场能量会膨胀到什么量级”。

这意味着 comoving 不能直接借用 Galilean 分支里现成的 energy_ref。当前 test_2d_comoving_psatd_hybrid 同时打开了 hybrid grid、moving window、boosted frame、filter 和 laser/plasma/beam 场景，算法路径和 producer surface 都不同；如果把别的 family 的 reference 常量搬过来，就会把“算法不同”和“reference 不同”混成一件事。更稳妥的做法，是先在当前稳定输入上冻结一份 stable ledger，至少记下 diag1000400 末态的电场能量、 $\max(|\mathbf{E}|)$ 与 $\text{p99}(|\mathbf{E}|)$ ；随后再构造一个只关闭 comoving 关键稳定化机制的 sibling 输入，从同一张 diag1000400 取出 energy_ref_unstable。这样脚本里的能量 gate 才真正有“稳定配置相对不稳定对照被压低了多少”的解释力。

局部 spike gate 则不应和能量 gate 共用同一种 reference 口径。它的用途更像异常探测，而不是 NCI 增长率 proxy，所以应优先绑定 stable envelope：拿当前稳定 baseline 的 spike_ratio = $\max(|\mathbf{E}|) / \text{p99}(|\mathbf{E}|)$ 作为上界，必要时再乘一个有来源说明的 safety factor。相反，Gauss-law drift 仍属于第二阶段工作。只有当输入卡真的把 divE 加入 fields_to_plot，并且正文明确区分“current correction 断言”和“current_correction = 0 情形下的末态 drift 观测”之后，才适合把它加入 patch。

v0.32 的实际推进，是把上面这套标定流程至少跑到一本地 audit 的粒度。当前 PIC-tutor 已在只读 ../warpX 前提下，用 inputs_test_2d_comoving_psatd_hybrid 先跑出 stable baseline，再用命令行覆盖 psatd.use_default_v_comoving=0 与 psatd.v_comoving=0 跑出 sibling，对两张 diag1000400 生成 runs/fieldsolver-validation/comoving-reference-ledgers/comoving-comparing-stable-vs-no-comoving.{md,json}。这一步确认了两件事：第一，override 后的运行日志里不再出现 comoving velocity 行，说明它确实切到了非 comoving 分支；第二，当前单进程样本并没有出现“关闭 comoving 后电场能量显著抬高”的预期，反而得到

`stable_over_unstable_energy_ratio = 1.0469608718245416`, 也就是 `stable baseline` 的电场能量略高于 `no-comoving sibling`。

这个结果直接决定了 `patch` 策略边界。当前 `local sample` 仍然能提供 `spike_ratio_ref_stable = 1.1103719982074416` 这样的 `stable envelope` 候选值, 也能证明 `ledger/provenance` 工具链是通的; 但它还不能让我们诚实地把 `energy_ref_unstable = 7.786411776882875e+14` 宣称作为最终 `CI gate`, 因为这个“unstable reference”并没有在本地样本上表现出比 `stable baseline` 更高的电场能量。

更重要的是, 后续本地 `control experiment` 进一步收紧了这个判断。用同一台机器、同样的 `warpx.numprocs='1 1'` 覆盖方式, 对 `WarpX` 已经具备成熟 `energy gate` 的 `test_2d_galilean_psatd_hybrid` 再做一组 `stable / no-Galilean sibling` 对照, 当前仓库在 `runs/fieldsolver-validation/galilean-reference-ledgers/galilean-stable-vs-no-galilean.{md,json}` 中得到 `stable_over_unstable_energy_ratio = 0.8418992628444483`: 也就是 `stable Galilean baseline` 的电场能量明显低于禁用 `Galilean` 的 `sibling`。这个 `control` 说明“本机缺 `MPI`, 所以单进程样本看不出 `unstable contrast`”并不是一个足够强的解释; 至少对 `Galilean family` 而言, 单进程本地设置仍然能复现预期的 `unstable-energy ordering`。

这里还有一个更硬的事实。当前 `no-comoving` 和 `no-galilean` 两个 `sibling` 的末态 `electric_energy`、`e_mag_max`、`e_mag_p99` 和 `spike_ratio` 在本地 `ledger` 中已经重合到 `1e-14` 相对误差量级; 换句话说, 一旦把 `v_comoving` 或 `v_galilean` 都压成零, 它们在这组 `2D hybrid boosted-frame` 运行里就实际汇合到同一条 `standard-PSATD branch`。于是问题就不再是“comoving 的 unstable sibling 还没找到”, 而是“当前可识别的 `shared unstable branch` 对 `Galilean family` 提供了有效 `energy ordering`, 却没有对 `comoving family` 提供同样的 `ordering`”。

因此, 更严格的说法应当是: 本地 `audit` 已经验证了参数分支和 `ledger` 提取流程, 但 `comoving no-comoving sibling` 当前没有给出可用的 `unstable-energy contrast`, 这更像是 `comoving family` 本身未必适合直接沿用 `Galilean` 式 `energy gate` 语义, 而不只是并行环境缺失。真正写入 `WarpX analysis_comoving.py` 的实现方向, 接下来至少要先回答两个问题之一: 要么重新选择更有解释力的 `comoving sibling`; 要么明确承认 `comoving` 的第一阶段 `patch` 应该以 `finite/spike` 或其他更合适的判据为主, 把 `energy gate` 降级为待更多样本后再定。

这条更保守的路线现在已经有了脚本原型, 而不再只是文字判断。`PIC-tutor` 当前新增了 `scripts/analysis_comoving.py`: 它保持 `WarpX regression helper` 的读取方式, 但把 `gate` 拆成三层接口, 默认始终执行 `finite-field sanity`, 可按 `ledger` 启用 `spike gate`, 而 `energy gate` 只有在显式传入参数时才会启用。更关键的是, 这个原型已经用现有本地样本做过自校验: `stable comoving baseline` 能通过 `finite + spike`, 而 `no-comoving sibling` 会在同一 `spike_ratio_ref_stable` 阈值下失败。于是第一个真正可执行的第一阶段 `patch` 形态已经变得具体起来: 即便后续继续搁置 `energy gate`, `comoving family` 也已经具备一条可落地的 `finite + spike analysis` 路线。

本轮又把这组判别收成了独立的 `reader-side contract`: `scripts/analyze_comoving_first_stage_contract.py` 同时读取 `stable baseline` 和 `no-comoving reference`, 要求前者通过、后者被同一 `spike ceiling` 拒绝。实际结果为 `stable spike_ratio=1.1103719982074416`, `reference spike_ratio=1.1119614945212388`, 阈值 `1.1114823702056489`; 两组输出字段都保持 `finite`, 只有 `reference` 未通过 `spike gate`, 因此这不是“两个 case 都能跑”的弱 `smoke`, 而是对第一阶段 `gate` 判别力的最小正/负对照。报告归档于 `runs/fieldsolver-validation/comoving-reference-ledgers/comoving-first-stage-contract.{json,md}`。同时, 能量 `gate` 仍明确关闭: 当前本地 `calibration` 没有形成可解释的 `unstable-energy oracle`。

为了再往前逼近真正的 `WarpX patch` 目录结构, 当前仓库还把这条路线进一步压成了一个最小 `draft helper`: `notes/code-reading/fieldsolver/analysis_comoving_first_stage_patch_draft.md` 说明其 `intended wiring` 和候选 `SPIKE_RATIO_MAX` 来源。这份 `draft helper` 的目标不是替代本地原型, 而是回答另一个更实际的问题: 如果下一轮真的要往 `Examples/Tests/nci_psatd_stability/` 提 `patch`, 最小、最克制、最不夸大证据边界的第一版文件长什么样。当前答案是: 只带 `finite + spike`, 不带 `energy gate`。现在这份 `helper` 及其 `comoving_first_stage_patch.diff` 也已经不再是手工维护的孤立草稿, 而是可以通过 `scripts/build_comoving_first_stage_patch.py` 从 `comoving-stable-vs-no-comoving.json ledger` 自动重建, 从而把阈值来源、`helper` 内容和 `CMake wiring` 绑定回同一份 `reference` 证据。

这一轮模块收口又往前走了一步: 生成链现在不只输出 `helper.diff` 和 `provenance note`, 还会自动生成 `comoving_first_stage_submission_packet.md`、`comoving_first_stage_pr_draft.md`, 以及镜像 `WarpX` 目录结构的 `comoving_first_stage_bundle/`。前者固定 `scope`、`review claim`、`checklist` 和 `follow-up boundary`; `PR draft` 直接给出可复用的 `title`、`summary`、`out-of-scope` 列表与 `reviewer checklist`; `staging bundle` 则把 `analysis_comoving.py`、`patch diff` 和随附说明收成可直接复制到另一个 `worktree` 的目录包。进一步地, `PIC-tutor` 现在还补了 `scripts/stage_comoving_first_stage_patch.py`, 可以对目标 `WarpX worktree` 先做 `dry-run`, 再自动复制 `helper` 并只改写 `test_2d_comoving_psatd_hybrid` 那一段 `CMake analysis wiring`, 而不碰其他 `test block`; `scripts/audit_comoving_first_stage_patch.py` 又把同一个目标 `worktree` 的状态判成 `unstaged / partial / staged` 三档, 避免后续连“现在到底装没装进去”都只能靠人工 `grep`; `scripts/report_comoving_first_stage_patch.py` 则把这份状态和下一条推荐命令写成 `markdown` 预检报告, 方便后续接续者直接接手; 而 `scripts/preview_comoving_first_stage_patch.py` 再往前一步, 把目标 `checkout` 将会发生的 `helper/CMake` 改动直接打印成 `unified diff`。换句话说, 当前 `PIC-tutor` 已经把 `comoving` 第一阶段 `patch` 从“本地可验证的 `helper` 草案”继续推进成“可直接整理成 `upstream` 提交描述并交给真实 `WarpX worktree` 的 `handoff bundle`”。这不会自动证明 `energy gate` 已经成立, 但它把第一阶段 `patch` 应该如何被诚实地提交、评审和后续拆分, 收成了更稳定的文字资产。

本次重建和复核把上述 `handoff` 链重新跑通: `scripts/build_comoving_first_stage_patch.py` 由 `stable/no-comoving`

ledger 生成 bundle, audit/report/preview/stage --dry-run 均成功; 直接执行 bundle helper 时 stable 返回 0、no-comoving 返回 1, 独立 contract 也确认 stable spike_ratio=1.1103719982074416 低于 1.1114823702056489, reference 1.1119614945212388 被拒绝。目标 checkout 仍保持 unstaged, 所以本节的结论是“交接包可复现且可审阅”, 不是“WarpX upstream 已接入”。

因此, comoving reference 标定这块现在的闭合条件可以重新表述为: 第一, 正文诚实记录当前 analysis=OFF、第一阶段 finite/energy/spike gate 和第二阶段 divE gate 之间的证据等级差异; 第二, stable ledger、no-comoving sibling 和 provenance note 已经 materialize, 且已确认当前单进程样本不足以直接推出最终 energy_ref_unstable; 第三, 下一步若继续做 WarpX patch, 就该把重心放在更贴近 upstream regression 的 repeated/MPI contrast, 而不是继续停留在抽象方案描述。达到这一步, 模块本身已经足够支撑一个“local calibration audit”版本号, 但还不足以声称 WarpX 侧强 analysis gate 已经定稿。

在这之后, 当前仓库又沿 v_comoving 本身做了一轮更窄的 sibling 扫描, 结果记录在 notes/code-reading/fieldsolver/25-psatd-comoving-vel 与 runs/fieldsolver-validation/comoving-reference-ledgers/comoving-velocity-scan.{md,json}。这轮扫描刻意不改 filter、moving window、hybrid grid 或 deposition, 只比较五条 velocity 路径: stable default selector、显式写回默认 v_comoving、半速 v_comoving、零 v_comoving 和反号 v_comoving。结果有三点最关键。第一, 显式默认 v_comoving 与 stable baseline 的 electric_energy 和 spike_ratio 在舍入误差内完全重合, 说明 default selector 本身不是隐藏变量。第二, half-default-beta 与 zero-comoving 都没有把末态电场能量抬高到 stable 之上, 反而分别降到 0.9880x 和 0.9551x stable, 因此“只沿 velocity 自身减弱 comoving”仍然给不出可用的 local energy-reference sibling。第三, 反号 v_comoving 会把 spike_ratio 推到 1.0622x stable, 但同时把 electric_energy 压到 0.8028x stable。这说明在 comoving family 内部, spike 与 energy 已经可以明确解耦: 它是一个更坏的局部异常候选, 却不是一个更高能量的末态候选。于是第一阶段 patch 的收口逻辑反而更清楚了: finite + spike 继续是当前最有本地证据支撑的主 gate, 而 energy gate 若还要争取, 就不应继续停留在本机 v_comoving 数值扫描, 而应转向更接近 upstream regression 的 repeated/MPI contrast。

和这条 comoving 主线并行的后备收口方向, 现在也可以写得更硬了: notes/code-reading/fieldsolver/26-rz-psatd-validation-strong-cr 已经把 RZ PSATD 当前真正的强 validation 主线收成一张判据表。当前最强的 RZ PSATD regression 并不是 test_rz_psatd_JRhom_LL2, 而是三条分开的 active family: 第一, test_rz_galilean_psatd* 用 analysis_galilean.py 提供 RZ NCI suppression 与 current_correction/periodic_single_box_fft 的 charge gate; 第二, test_rz_langmuir_multi_psatd* 用 analysis_rz.py + analysis_utils.py 提供解析 Er/Ez 波形与部分 charge-conservation gate; 第三, test_rz_pml_psatd 用 analysis_pml_psatd_rz.py 提供 radial PML 残余场上界。相对地, test_rz_psatd_JRhom_LL2 当前仍只有 checksum。

如果把这条 fallback 线也按“能支撑什么论断”拆开, 当前最稳妥的写法可以先固定成下面四层:

层级	代表测试	当前主判据	正文里允许写到的强度
强 NCI 抑制	test_rz_galilean_psatd*	analysis_galilean.py 的末态全域 field-energy gate; current_correction 分支再加 divE-rho/\epsilon_0 gate	可直接写成 RZ Galilean PSATD 对 drifting-plasma NCI 的强 regression
强解析场 / 局部守恒	test_rz_langmuir_multi_psatd*	analysis_rz.py 的解析 Er/Ez 对照; 部分 sibling 再加 analysis_utils.py 守恒 gate	可写成 RZ PSATD 在 Langmuir 小振幅问题上的波形正确性和部分守恒
RZ Langmuir current-correction runtime	test_rz_langmuir_multi_psatd*	analysis_rz.py 与 current_correction contract 均通过; Er/Ez=1.0542e-1/1.9313e-2, charge residual 5.4781e-14	当前项目级 1-rank evidence 可同时支撑解析场与同面 charge-conservation; 不外推到 RZ JRhom LL2
Standard RZ Langmuir PSATD runtime	test_rz_langmuir_multi_psatd*	analysis_rz.py 与 independent field contract 均通过; Er/Ez=1.1617e-1/1.5194e-2, < 0.12, current_correction=0	可支撑 standard RZ PSATD 的解析场与 filter workflow; charge gate 在该 sibling 中不适用
RZ Langmuir PSATD-JRhom CL4 runtime	test_rz_langmuir_multi_psatd*	analysis_rz.py 与 independent field contract 均通过; Er/Ez=1.0994e-1/6.4303e-2, < 0.12, current_correction=0	可支撑 RZ PSATD-JRhom CL4 的解析场与 filter workflow; charge gate 在该 sibling 中不适用

三条结果已由 `scripts/summarize_rz_langmuir_psatd_family.py` 收成 `family matrix`: 三者都使用 RZ + PSATD + direct, 网格维度均为 [64,128,1], 解析场 gate 全部通过; 只有 `current-correction` 行带 `charge=PASS`, standard 与 JRhom CL4 行明确标成 NOT_APPLICABLE。这张矩阵是 family-level runtime evidence, 不是所有 geometry/order 组合的收敛研究。| 强 PML 残余场 | `test_rz_pml_psatd` | `analysis_pml_psatd_rz.py` 的全域残余 Er/Ez 上界 | 可写成 RZ PSATD + radial PML 的吸收残余场控制 | | `checksum-only workflow` | `test_rz_psatd_JRhom_LL2` | `analysis=OFF`, 只保留 final checksum | 只能写成 JRhom LL2 的输出/流程回归, 不能写成独立稳定性强判据 |

这张表的意义不在于再重复一遍 CMake, 而是把“RZ PSATD 已经有哪些强 gate、哪些还没有”收成一本书里可直接引用的证据等级表。这样如果 comoving 第一阶段 patch 线暂时不直接上提, 第 6 章接下来的最合理推进就不是继续堆外围 patch 工具, 而是补 RZ JRhom LL2 的独立 main analysis, 或至少继续保持这条线的表述纪律: 已有强 gate 的 family 就按强 gate 写, 没有 main analysis 的 family 就明确写成 checksum-only workflow。

沿着这条缺口继续往前压, 当前仓库又把 `test_rz_psatd_JRhom_LL2` 下一步到底该补哪类 analysis 单独判定了一次, 结果记录在 `notes/code-reading/fieldsolver/27-rz-jrhom-ll2-analysis-direction.md`。关键结论是: 这条 RZ JRhom 路线虽然也带 `update_with_rho + do_time_averaging`, 但它并不像 `langmuir/analysis_rz.py` 那样站在一个小振幅、周期、解析可写的 modal scaffold 上。相反, 它当前的 producer 形状是 moving window、rigid driver、driver_back、continuous-injection plasma_e/plasma_p、damped longitudinal boundary, 再叠 JRhom_LL2 + div cleaning 的 application workflow; 而 diagnostics 只输出 Er Ez Bt jr jz rho rho_driver rho_plasma_e rho_plasma_p, 并没有 divE、phi 或可直接回代理论波形的 reduced diagnostic。这意味着如果第一步就强行走 Langmuir 式解析 Er/Ez gate, 反而要先额外定义“理论上应该长什么样”。

在当前证据下, 它更像 `analysis_psatd_CC1.py` 或 `analysis_galilean.py` 那类 stability-style gate 候选, 而不是解析场 gate 候选。最实际的第一阶段主判据, 应先尝试补一个未态 field-energy route: 先在同一 workflow 上寻找最小改动的 reference sibling, 确认是否存在稳定的 energy ordering; 若 ordering 成立, 再决定把第一阶段脚本收成 `finite + energy`, 还是 `finite + energy + spike`。换句话说, `test_rz_psatd_JRhom_LL2` 现在最需要补的不是另一张解析场表, 而是一条和 `nci_psatd_stability` family 口径一致的独立 stability-style main analysis。

这一步现在也已经不再只是口头计划。当前仓库新增了 `scripts/build_rz_psatd_reference_ledger.py` 与 `scripts/scan_rz_jrhom_reference` 并在 `notes/code-reading/fieldsolver/28-rz-jrhom-reference-sibling-scan.md` 里固定了第一批最小改动 sibling: 保持当前 workflow 不变, 只比较 JRhom 从 LL2 -> CL1、关闭 `do_time_averaging`、关闭 `divE/divB cleaning`, 以及它们的组合。这样下一步就可以直接去跑 `diag1000025` 的 energy/spike ordering, 而不需要再手工拼接 reference sibling 和 ledger 汇总流程。

这轮本地扫描现在已经给出第一批真正可用的排序。当前 `diag1000025` surface 上, `ll2-no-timeavg-cleaning` 同时给出最高 electric_energy 和最高 spike_ratio, 因此它是第一阶段最像 unstable reference 的候选; 相对地, 单独关掉 `divE/divB cleaning` 并不会把能量抬高, `cl1-no-timeavg-no-cleaning` 虽然会略抬高 spike, 但能量仍低于 baseline, 而 `cl1-timeavg-cleaning` 更是被 `PsatdAlgorithmRZ.cpp` 的源码断言直接拒绝, 因为 `RZ_psatd.do_time_averaging=1` 只支持 J 线性时间依赖。这意味着下一步已经不必继续泛泛地“搜索 reference sibling”, 而是可以直接围绕 baseline 与 `ll2-no-timeavg-cleaning` 这对候选去写第一阶段 `finite + energy` helper, 并把 spike 作为增强项保留。

这条 helper 现在也已经落成脚本原型, 记录在 `notes/code-reading/fieldsolver/29-rz-jrhom-first-stage-helper.md`。当前新增的 `scripts/analysis_rz_jrhom.py` 保持了和本地 `analysis_comoving.py` 类似的接口形态, 但换成 RZ 字段口径: 它始终检查 Er/Ez/Bt/jr/jz/rho 的 finite 性, 默认从 `rz-jrhom-reference-scan.json` 读取 baseline `baseline-jrhom-ll2-timeavg-cleaning` 与 `unstable reference ll2-no-timeavg-cleaning`, 并把

$$\text{err_energy} = \frac{E_{\text{plotfile}}}{E_{\text{ref}}}$$

作为主 gate, 再用当前 stable/reference 比值乘一个很小的 safety factor 自动导出 `tol_energy`。按当前 ledger, baseline 样本满足 `err_energy = 9.7708651502456867 \times 10^{-1}`, 而 reference sibling 自身会因为 `err_energy = 1` 在同一阈值下失败。这说明 `test_rz_psatd_JRhom_LL2` 这条线已经不只是“知道下一步该写 helper”, 而是已经具备了一个可直接运行的第一阶段 `finite + energy` 原型。

与此同时, spike 仍被刻意保留为第二层增强项, 而不是第一阶段默认主合同。原因不是它没有分辨力, 而是当前最需要先回答的问题, 是这条 RZ JRhom workflow 能否像 `analysis_psatd_CC1.py` 那样先拥有一条稳定性主 gate。现在答案已经是肯定的: 至少在 `diag1000025` 这一张 active checksum surface 上, 当前仓库已经把这条主 gate 收成了可运行脚本和可追溯阈值来源。后续若 `repeated/MPI` 设置或更长时间窗继续支持同样 ordering, 再决定是否把这条 helper 真正上提到 `WarpX Examples/Tests/nci_psatd_stability/`, 以及是否把 spike 从可选增强项升级成正式第二 gate。

这一轮继续往 upstream 方向推进时, 还得到了一条同样重要的执行边界, 记录在 `notes/code-reading/fieldsolver/30-rz-jrhom-input-numprocs` `test_rz_psatd_JRhom_LL2` 的输入卡本身写着 `warpX.numprocs = 1 2`, 而 `CMakeLists.txt` 里它也注册成 `nprocs = 2`。当前仓库先把 `scan_rz_jrhom_reference_candidates.py` 扩成可显式切换 `--numprocs-override` 和 `--command-prefix-str`

脚本, 再用 `--numprocs-override none` 做输入卡原生审计。第一轮 `plain single-process` 调用确实统一触发了 `warpx.numprocs`, `if specified ... number of processes` 断言, 说明单进程 `direct invocation` 与输入卡原生 `1 x 2 decomposition` 的进程数合同不匹配; 但这条线没有停在“缺 launcher”上。进一步搜索本机环境后, 当前已经在另一个 Conda 环境里找到可用的 `mpiexec -n 2`, 并用同一脚本真正重跑出 `rz-jrhom-reference-scan-mpi2.{md,json}`。结果更关键: `baseline`、`ll2-no-timeavg-cleaning`、`ll2-timeavg-no-cleaning` 和 `cl1-no-timeavg-no-cleaning` 都能在 2 ranks 下落出 `diag1000025`, 而且 `repeated/MPI` 下的 `energy` 排序与本机 1 1 快速样本完全一致, 仍然是 `ll2-no-timeavg-cleaning > baseline > cl1-no-timeavg-no-cleaning > ll2-timeavg-no-cleaning`。这意味着当前 `finite + energy helper` 已经不再只是本机单进程样本的临时口径, 而是有了更贴近 `upstream regression` 形状的 2-rank 复核支持。它当然还没有自动变成 WarpX `upstream` 的正式 `analysis`, 但至少“当前排序只是单进程偶然产物”这条怀疑, 现在已经没有证据支撑了。

更进一步, 当前仓库已经把这条 `repeated/MPI` 已验证的 `helper` 又往前收成了一套真正可交接的 `handoff` 资产, 记录在 `notes/code-reading/fieldsolver/31-rz-jrhom-first-stage-patch-draft.md`。具体来说, `scripts/build_rz_jrhom_first_stage_patch.py` 现在会从 `rz-jrhom-reference-scan-mpi2.json` 自动重建六类产物: `analysis_rz_jrhom_first_stage_draft.py`、`rz_jrhom_first_stage_patch.diff`、`rz_jrhom_first_stage_provenance_note.md`、`rz_jrhom_first_stage_submission_patch.py`、`rz_jrhom_first_stage_pr_draft.md` 和 `rz_jrhom_first_stage_bundle/`。它们的作用和边界都很明确: 第一阶段 `patch` 只声称 `finite + energy`, 仍保留现有 `checksum`, 不引入新的 `diagnostics surface`, 也不把 `spike gate` 一起塞进第一版草案。这样一来, RZ JRhom LL2 这条线现在已经不只是“有一个本地可运行 `helper`”, 而是已经具备了 `helper`、`diff`、`review` 口径和 `bundle` 目录一体化的上提草案。

但仅有 `bundle` 还不够, 因为真正的接续问题不是“能不能再生成一份 `diff`”, 而是“目标 WarpX `checkout` 当前到底是什么状态, 这份 `bundle` 会对它造成什么精确改动”。因此当前仓库又补了 `notes/code-reading/fieldsolver/32-rz-jrhom-target-checkout-workflow.md`, 把这一轮工程闭合点压到 `target-checkout workflow`: `scripts/preview_rz_jrhom_first_stage_patch.py` 只读打印 `unified diff`, `scripts/audit_rz_jrhom_first_stage_patch.py` 把目标 `worktree` 判成 `unstaged / partial / staged` 三档, `scripts/report_rz_jrhom_first_stage_patch.py` 自动生成 `markdown` 预检报告, 而 `scripts/stage_rz_jrhom_first_stage_patch.py` 则在显式传入 `--warpx-root` 的前提下支持 `dry-run` 和最小写入 `staging`。它们共同绑定的写入面很克制: 只新增 `Examples/Tests/nci_psatd_stability/analysis_rz_jrhom.py`, 并只把 `test_rz_psatd_JRhom_LL2` 的 `analysis` 行从 `OFF` 改成 "`analysis_rz_jrhom.py diags/diag1000025`", 不碰任何其他 `test block`、`checksum` 行或 `dependency` 行。这样第 6 章现在已经不只是“说明如何设计这条 `helper`”, 而是已经把“如果要把它交给另一个 WarpX `checkout`, 应当先如何预览、审计、报告和 `staging`”收成了可直接执行的工程流程。换句话说, 当前这条线的下一步已经不再是继续打磨 `handoff` 文本, 而是非常具体的两种选择: 要么在目标 `checkout` 上按当前口径做 `preview/report/dry-run/stage`, 并准备实际提交流程; 要么把它作为一个已经闭合的 `first-stage boundary` 暂时冻结, 转去推进下一个成书模块。

截至 2026-07-12, 对相邻目标 `checkout ../warpx` 的只读审计结果是: 整体状态为 `unstaged`; `Examples/Tests/nci_psatd_stability/analysis_rz_jrhom.py` 尚不存在; `test_rz_psatd_JRhom_LL2` 的 CMake `analysis` 行仍是 `OFF`。本轮重新运行 `audit`、`dry-run` 和 `unified preview`, 三者均成功, 精确改动仍只有两项: 新增第一阶段 `finite + energy helper`, 以及把该 `test` 的 `analysis` 行改成 "`analysis_rz_jrhom.py diags/diag1000025`". 因此当前项目可以声称“RZ JRhom first-stage `handoff bundle` 和目标 `checkout workflow` 已准备好”, 但不能声称 `patch` 已经写入 WarpX, 更不能声称 `upstream regression` 已经接入。

这次审计报告已写入 `notes/code-reading/fieldsolver/rz_jrhom_first_stage_target_report.md`, 运行级 JSON/Markdown 快照另归档于 `runs/fieldsolver-validation/rz-reference-ledgers/rz-jrhom-first-stage-target-audit.md`。它把 `bundle` 状态、`helper` 状态、CMake 状态和下一条推荐命令固定在同一份可接续记录中; 后续若要实际 `staging`, 仍需显式传入目标 `checkout` 并先执行 `dry-run`, 当前书稿工作本身不修改 `../warpx`。

6.8.5 PSATD/场求解器算法族决策矩阵

前面的各节分别解释了系数、源项时间依赖和验证脚本, 但读者在实际阅读输入卡或源码时还需要一个横向索引。下面这张表只做算法族导航, 不把同名系数强行合并; “验证强度”描述的是本项目当前 `checkout` 中能直接找到的证据等级, 不等于算法理论上的完整正确性。

算法族	主要谱基/空间表示	源项时间模型	粒子沉积与组合限制	代表系数或字段	当前可用验证证据
FDTD / Yee / CKC	实空间有限差分 stencil	按离散时间层推进 E/B/J	依赖对应的 current deposition、guard-cell 和边界链; 不进入 PSATD 的谱系	Yee/CKC 差分算子、split-field PML 系数	NCI FDTD、PML 和部分解析场 regression 可提供强判据
Cartesian standard PSATD	3D Fourier k 空间	常规 J/rho 时间依赖与可选 current correction	与 grid_type、current_deposition filter 和 periodic FFT 组合受约束	C、S _{ck} 、K ⁻² 、X ₁ -X ₄	Langmuir、NCI energy、Gauss-law 和部分 PML regression

算法族	主要谱基/空间表示	源项时间模型	粒子沉积与组合限制	代表系数或字段	当前可用验证证据
Cartesian Galilean PSATD	Fourier k 空间 + Galilean moving frame	以 v_galilean 修 正相位和源项积分	与 current correction、Vay、 JRhomb、implicit 等存在反向兼容约束	Galilean X1-X4 、 Theta2 、 average-field Psi/Y	analysis_galilean.py 提供较成熟的 NCI/energy 与部 分 charge gate
Cartesian comoving PSATD	regular-domain Fourier k 空间 + v_comoving	comoving 相位、 rho 参考层和 current correction	当前要求 direct deposition、 update_with_rho=1 不支持 Esirke- pov/Villasenor/Vayresidual 和 JRhom 组合	comoving X1-X4 、 Theta2 、 comoving continuity	当前有 local finite + spike 原型; energy reference 尚未形成可靠 ordering
Cartesian PSATD-JRhomb	Fourier k 空间 + 子区间推进	J/rho 在一个 PIC 步内按 con- stant/linear/quadratic 形式分段采样	不支持 Vay 和 Galilean PSATD; 外层 OneStep_JRhomb() 改写沉积与谱推进时序	JRhomb Y1-Y8 、 E/B/F/G 子区间更 新	analysis_psatd_CC1.py 有强 NCI/energy 入口; 其余家族需按时 间模型分别验证
RZ standard / Galilean PSATD	k_z FFT + k_r Hankel/Bessel roots; Ep/Em/Ez 字段布局	standard 或轴向 Galilean 相位/源项 积分	RZ mode 对称性、 轴上 guard、 current correction 和 time-averaging 有独立限制; 不支持 Vay	standard X1-X3/X5-X6 ; Galilean X1-X4/Theta2/T_rho	RZ Langmuir、 RZ Galilean NCI、RZ PML 提供强判据
PSATD PML	split-field spectral/PML 表 示	吸收层内使用专用 split-field 时间推进	受 PML profile、 方向、RZ 分支和 C1-C25 系数合同约 束	C1-C25 、PML split fields	Cartesian/RZ PML residual-field analysis; 论文闭环 仍待 Lee/Vay 全文

读这张表时应先区分三种“选择”：第一是谱基或空间离散 (FDTD、Cartesian Fourier、RZ Hankel/Fourier)；第二是源项时间模型 (普通、Galilean/comoving、JRhom)；第三是沉积与同步组合 (Direct、charge-conserving current、Vay、current correction、filter)。例如，JRhom 的 Y1-Y8 不能因为名字与 Galilean average-field 的 Y1-Y4 相似就合并解释；同样，RZ 的 Ep/Em 不是 Cartesian Ex/Ey 的简单改名。

验证列也应按证据等级阅读。**analysis_galilean.py**、Langmuir 解析场和 PML residual-field analysis 已经能对相应 family 提供较强的物理断言；**test_rz_psatd_JRhomb_LL2** 的上游 CMake 仍是 checksum/workflow 入口，但本地 repeated/MPI ledger 已支持一个通过正/负对照的 **finite + energy** helper，二者必须分开写。comoving 同理：本地 ledger 已验证分支选择和 **finite + spike** 方向，却还没有证明可复用 Galilean 式 **energy_ref**。这张矩阵因此同时是算法选择表和证据边界表。

本轮又把这条 helper 收成独立的正/负 contract: **scripts/analyze_rz_jrhomb_first_stage_contract.py** 直接读取真实 2-rank repeated/MPI 末态 plotfile，要求 baseline 被 energy ceiling 接受，l12-no-timeavg-cleaning reference 被同一 ceiling 拒绝。实际 baseline energy ratio 为 0.9770894022295227，reference ratio 为 1.0，由 1.001 safety factor 导出的阈值为 0.9780664916317521；六个字段 Er/Ez/Bt/jr/jz/rho 在两组 plotfile 中均 finite。报告位于 **runs/fieldsolver-validation/rz-reference-ledgers/rz-jrhomb-first-stage-contract.{json,md}**。这仍是 project-level repeated/MPI validation，不等于 WarpX upstream CMake 已经接入 analysis; spike 只记录、不作为第一阶段主 gate。

6.9 静电与静磁求解器

绑定精读笔记: **notes/code-reading/fieldsolver/09-electrostatic-magnetostatic.md**。

WarpX 的 electrostatic 路径不再用 Maxwell curl 方程推进场，而是在每一步从当前粒子/流体源项重新解椭圆方程。最基本的 lab-frame 静电模式是

$$\nabla^2\phi = -\rho/\epsilon_0, \quad \mathbf{E} = -\nabla\phi.$$

如果启用 `labframe-electromagnetostatic`, 还会解磁矢势:

$$\nabla^2 \mathbf{A} = -\mu_0 \mathbf{J}, \quad \mathbf{B} = \nabla \times \mathbf{A}.$$

如果启用 `relativistic`, WarpX 对每个 species 用平均速度 $\beta = \langle \mathbf{v} \rangle / c$ 解修正 Poisson 方程:

$$[\nabla^2 - (\beta \cdot \nabla)^2] \phi = -\rho / \epsilon_0,$$

并按

$$\mathbf{E} = -\nabla \phi + \beta(\beta \cdot \nabla \phi), \quad \mathbf{B} = -\frac{1}{c} \beta \times \nabla \phi$$

重建场。这个模式不能把所有 species 的电荷先相加, 因为不同 species 的平均速度不同。

参数入口在 `WarpX::ReadParameters()`。一旦 `warpX.do_electrostatic` 不是 `none`, Maxwell solver 会被关闭:

```
pp_warpX.query_enum_sloppy("do_electrostatic", electrostatic_solver_id, "-");
// if an electrostatic solver is used, set the Maxwell solver to None
if (electrostatic_solver_id != ElectrostaticSolverAlgo::None) {
    electromagnetic_solver_id = ElectromagneticSolverAlgo::None;
}
```

这句代码是模型边界: 静电 PIC 不传播光波, 也不描述激光/辐射传播。它用瞬时 Poisson 解代替全 Maxwell 更新。

solver 对象在 `WarpX::WarpX()` 构造期选择:

```
if ((WarpX::electrostatic_solver_id == ElectrostaticSolverAlgo::LabFrame)
    || (WarpX::electrostatic_solver_id == ElectrostaticSolverAlgo::LabFrameElectroMagnetostatic))
{
    m_electrostatic_solver = std::make_unique<LabFrameExplicitES>(nlevs_max);
}
else if (electrostatic_solver_id == ElectrostaticSolverAlgo::LabFrameEffectivePotential)
{
    m_electrostatic_solver = std::make_unique<EffectivePotentialES>(nlevs_max);
}
else
{
    m_electrostatic_solver = std::make_unique<RelativisticExplicitES>(nlevs_max);
}
```

每步入口是 `WarpX::ComputeSpaceChargeField()`。如果 `reset_fields=true`, E/B 先被清零, 再由静电/静磁求解器把自洽场加回:

```
if (reset_fields) {
    // Reset all E and B fields to 0, before calculating space-charge fields
    ABLASTR_PROFILE("WarpX::ComputeSpaceChargeField::reset_fields");
    for (int lev = 0; lev <= max_level; lev++) {
        for (int comp=0; comp<3; comp++) {
            m_fields.get(FieldType::Efield_fp, Direction{comp}, lev)->setVal(0);
            m_fields.get(FieldType::Bfield_fp, Direction{comp}, lev)->setVal(0);
        }
    }
}

m_electrostatic_solver->ComputeSpaceChargeField(
    m_fields, *mypc, myfl.get(), max_level );
```

6.9.1 Poisson 边界条件

PoissonBoundaryHandler 读取 boundary.potential_lo_x/hi_x/... 和 warpx.eb_potential(x,y,z,t), 再把 field boundary 转成 AMReX linear operator boundary。Multigrid 支持 periodic、PEC/Dirichlet、Neumann; open/PML 会被拒绝:

```
WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
    (WarpX::field_boundary_lo[idim] != FieldBoundaryType::Open &&
     WarpX::field_boundary_hi[idim] != FieldBoundaryType::Open &&
     WarpX::field_boundary_lo[idim] != FieldBoundaryType::PML &&
     WarpX::field_boundary_hi[idim] != FieldBoundaryType::PML) ,
    "Open and PML field boundary conditions only work with "
    "warpx.poisson_solver = fft."
);
```

Dirichlet 电势由 setPhiBC() 写入 nodal phi 的物理边界:

```
if (dirichlet_flag[2*idim] && iv[idim] == domain.smallEnd(idim)){
    phi_arr(i,j,k) = phi_bc_values_lo[idim];
}
if (dirichlet_flag[2*idim+1] && iv[idim] == domain.bigEnd(idim)) {
    phi_arr(i,j,k) = phi_bc_values_hi[idim];
}
```

因此边界电势是解空间上的约束, 而不是加到 ρ 的体源项。

6.9.2 Lab-frame 静电

LabFrameExplicitES::ComputeSpaceChargeField() 先取出 rho_fp/rho_cp/phi_fp/Efield_fp, 把所有粒子电荷和流体电荷沉积到总 rho:

```
const MultiLevelScalarField rho_fp = fields.get_mr_levels(FieldType::rho_fp, max_level);
const MultiLevelScalarField rho_cp = fields.get_mr_levels(FieldType::rho_cp, max_level, skip_lev0_coarse_p);
const MultiLevelScalarField phi_fp = fields.get_mr_levels(FieldType::phi_fp, max_level);
const MultiLevelVectorField Efield_fp = fields.get_mr_levels_alldirs(FieldType::Efield_fp, max_level);

mpc.DepositCharge(rho_fp, 0.0_rt);
if (mfl) {
    const int lev = 0;
    mfl->DepositCharge(fields, *rho_fp[lev], lev);
}
```

随后同步电荷密度:

```
const Vector<std::unique_ptr<MultiFab> > rho_buf(num_levels);
auto & warpx = WarpX::GetInstance();
warpx.SyncRho( rho_fp, rho_cp, amrex::GetVecOfPtrs(rho_buf) );

lab-frame 中 beta=0, 所以 computeE() 退化为普通的 E=-grad(phi):
const std::array<Real, 3> beta = {0._rt};

setPhiBC(phi_fp, warpx.gett_new(0));
```

```
computePhi(rho_fp, phi_fp, beta, self_fields_required_precision,
           self_fields_absolute_tolerance, self_fields_max_iters,
           self_fields_verbosity, is_igf_2d_slices, Efield_fp);
```

6.9.3 Relativistic self fields

Relativistic solver 对每个 species 分别求自场。核心差异是先沉积单 species 电荷, 再用该 species 的全局平均速度设置 beta:

```

bool const local_average = false; // Average across all MPI ranks
std::array<ParticleReal, 3> beta_pr = pc.meanParticleVelocity(local_average);
std::array<Real, 3> beta;
for (int i=0 ; i < static_cast<int>(beta.size()) ; i++) {
    beta[i] = beta_pr[i]/PhysConst::c; // Normalize
}

```

然后用 species 自己的 self-field solver 参数求势并加场:

```

computePhi( amrex::GetVecOfPtrs(rho), amrex::GetVecOfPtrs(phi),
            beta, pc.self_fields_required_precision,
            pc.self_fields_absolute_tolerance, pc.self_fields_max_iters,
            pc.self_fields_verbosity, is_igf_2d_slices);

```

```

computeE( Efield_fp, amrex::GetVecOfPtrs(phi), beta );
computeB( Bfield_fp, amrex::GetVecOfPtrs(phi), beta );

```

computeE() 的 nodal 3D Ex 代码正是 $(\beta_i\beta_j - \delta_{ij})\partial_j\phi$:

```

Ex_arr(i,j,k) +=
    +(beta_x*beta_x-1._rt)*0.5_rt*inv_dx*(phi_arr(i+1,j ,k )-phi_arr(i-1,j ,k ))
    + beta_x*beta_y      *0.5_rt*inv_dy*(phi_arr(i ,j+1,k )-phi_arr(i ,j-1,k ))
    + beta_x*beta_z      *0.5_rt*inv_dz*(phi_arr(i ,j ,k+1)-phi_arr(i ,j ,k-1));

```

computeB() 在 beta=0 时立即返回; 否则按 $-\beta \times \nabla\phi/c$ 生成磁场:

```

if ((beta[0] == 0._rt) && (beta[1] == 0._rt) && (beta[2] == 0._rt)) { return; }

```

```

Bx_arr(i,j,k) += PhysConst::inv_c * (
    -beta_y*inv_dz*0.5_rt*(phi_arr(i,j ,k+1)-phi_arr(i,j ,k-1))
    +beta_z*inv_dy*0.5_rt*(phi_arr(i,j+1,k )-phi_arr(i,j-1,k )));

```

6.9.4 Effective potential

Effective potential solver 把 Poisson 方程改为 variable-coefficient elliptic solve.它先分配 cell-centered effective_potential_sigma:

```

fields.alloc_init(
    warpx::fields::FieldType::effective_potential_sigma, /*level=*/ 0,
    convert(rho->boxArray(), IntVect(AMREX_D_DECL(0,0,0))),
    rho->DistributionMap(), 1, IntVect(AMREX_D_DECL(0,0,0)), 1.0_rt
);

```

ComputeSigma() 中的核心因子是

$$\frac{C_{EP}}{4}\omega_{ps}^2\Delta t^2 = \frac{C_{EP}}{4}\frac{q_s n_s}{m_s \epsilon_0}\Delta t^2.$$

源码写作:

```

auto mult_factor = (
    C_SI * warpx.getdt(lev) * warpx.getdt(lev) / (4._rt * PhysConst::epsilon_0)
);

```

每个 species 对 sigma 的贡献为:

```

auto const q = std::abs(pc->getCharge());
auto const mult_factor_pc = mult_factor * q / pc->getMass();

```

```

sigma_arr(i, j, k, 0) += time_filter_param * mult_factor_pc * rho_cc;

```

最后调用专门的 variable-coefficient solver:

```

ablastr::fields::computeEffectivePotentialPhi(
    sorted_rho,
    sorted_phi,
    *sigma,
    required_precision,
    absolute_tolerance,
    max_iters,
    verbosity,
    warpx.Geom(),
    warpx.DistributionMap(),
    warpx.boxArray(),
    WarpX::grid_type,
    false,
    EB::enabled(),
    WarpX::do_single_precision_comms,
    warpx.refRatio(),
    post_phi_calculation,
    *m_poisson_boundary_handler,
    warpx.gett_new(0),
    eb_farray_box_factory
);

```

6.9.5 Magnetostatic vector Poisson

labframe-electromagnetostatic 的磁场不是 Maxwell curl 更新，而是解 vector Poisson 后取 curl。入口明确要求无 mesh refinement:

```

WARPX_ALWAYS_ASSERT_WITH_MESSAGE(this->max_level == 0,
    "Magnetostatic solver not implemented with mesh refinement.");

```

```

AddMagnetostaticFieldLabFrame();

```

它先清零并沉积所有 species 的电流:

```

for (int lev = 0; lev <= max_level; lev++) {
    for (int dim=0; dim < 3; dim++) {
        m_fields.get(FieldType::current_fp, Direction{dim}, lev)->setVal(0.);
    }
}

for (int ispecies=0; ispecies<mypc->nSpecies(); ispecies++){
    WarpXParticleContainer& species = mypc->GetParticleContainer(ispecies);
    if (!species.do_not_deposit) {
        species.DepositCurrent(
            m_fields.get_mr_levels_alldirs(FieldType::current_fp, finest_level),
            dt[0], 0.);
    }
}

```

再同步电流，设置矢量边界，调用 vector Poisson solver:

```

SyncCurrent("current_fp");

```

```

setVectorPotentialBC(m_fields.get_mr_levels_alldirs(FieldType::vector_potential_fp_nodal, finest_level));

```

```

computeVectorPotential(
    m_fields.get_mr_levels_alldirs(FieldType::current_fp, finest_level),
    m_fields.get_mr_levels_alldirs(FieldType::vector_potential_fp_nodal, finest_level),
    magnetostatic_solver_required_precision, magnetostatic_solver_absolute_tolerance,

```



```
    magnetostatic_solver_max_iters, magnetostatic_solver_verbosity
);
```

矢势的 PEC 边界条件按分量区分: 法向 A_n 用 Neumann, 切向 A_t 用 Dirichlet:

```
if ( WarpX::field_boundary_lo[idim] == FieldBoundaryType::PEC ) {
    if (ndotA) {
        lobc[adim][idim] = LinOpBCType::Neumann;
        dirichlet_flag[adim][idim*2] = false;
    } else {
        lobc[adim][idim] = LinOpBCType::Dirichlet;
        dirichlet_flag[adim][idim*2] = true;
    }
}
```

Poisson solve 后, EBCalcBfromVectorPotentialPerLevel::operator() 从 MLMG 取每个 A 分量的梯度, 再按 curl 组合成 B。例如 A_x 对 B_y/B_z 的贡献:

```
mlmg[0]->getGradSolution({buf_ptr});

// Interpolate dAx/dz to By grid buffer, then add to By
this->doInterp(*m_grad_buf_e_stag[lev][2],
              *m_grad_buf_b_stag[lev][1]);
MultiFab::Add(*m_b_field[lev][1]), *(m_grad_buf_b_stag[lev][1]), 0, 0, 1, 0 );

// Interpolate dAx/dy to Bz grid buffer, then subtract from Bz
this->doInterp(*m_grad_buf_e_stag[lev][1],
              *m_grad_buf_b_stag[lev][2]);
m_grad_buf_b_stag[lev][2]->mult(-1._rt);
MultiFab::Add(*m_b_field[lev][2]), *(m_grad_buf_b_stag[lev][2]), 0, 0, 1, 0 );

逐项合起来就是
```

$$B_x = \partial_y A_z - \partial_z A_y, \quad B_y = \partial_z A_x - \partial_x A_z, \quad B_z = \partial_x A_y - \partial_y A_x.$$

因此这一路径的正确性依赖三个环节同时一致: 电流沉积/同步给出正确 J , vector Poisson 解出符合边界条件的 A , post callback 再把 curl A 插值到 Bfield_fp 的实际 staggering。

6.10 Hybrid PIC: 广义 Ohm 定律与 B 场 RK 子步

notes/code-reading/fieldsolver/12-hybrid-pic-model-deep-dive.md 把 WarpX 的 kinetic-fluid hybrid solver 从模型参数到 kernel 做了第一轮深拆。这个路径不使用 Maxwell-Ampere 方程推进电场, 而是把电子视为流体、离子仍作为 kinetic particles, 用广义 Ohm 定律求电场:

$$\mathbf{E} = -\frac{1}{en_e} (\mathbf{J}_e \times \mathbf{B} + \nabla P_e) + \eta \mathbf{J} - \eta_h \nabla^2 \mathbf{J}.$$

其中准中性假设给出 $\rho = e n_e$, 总电流由忽略位移电流后的 Ampere 定律给出:

$$\mu_0 \mathbf{J} = \nabla \times \mathbf{B}.$$

电子电流不直接沉积, 而是由

$$\mathbf{J}_e = \mathbf{J} - \mathbf{J}_i - \mathbf{J}_{\text{ext}}$$

得到。源码里的 Jfield 已经在 CalculatePlasmaCurrent() 阶段扣除了外部电流, 所以 Ohm kernel 中只显式出现 $\mathbf{J} - \mathbf{J}_i$ 。

6.10.1 参数与辅助场

HybridPICModel 保存 hybrid solver 的所有模型参数和辅助场接口:

```
class HybridPICModel
{
public:
    HybridPICModel ();
    void ReadParameters ();
    void AllocateLevelMFs (... ) const;
    void InitData (const ablastr::fields::MultiFabRegister& fields);
    void GetCurrentExternal ();
    void CalculatePlasmaCurrent (... ) const;
    void HybridPICSolveE (... ) const;
    void BfieldEvolveRK (... );
    void FieldPush (... );
    void CalculateElectronPressure () const;
```

它要求用户指定电子温度, 并在非等温多方闭合时要求 `n0_ref`:

```
utils::parser::queryWithParser(pp_hybrid, "gamma", m_gamma);
if (!utils::parser::queryWithParser(pp_hybrid, "elec_temp", m_elec_temp)) {
    Abort("hybrid_pic_model.elec_temp must be specified when using the hybrid solver");
}
const bool n0_ref_given = utils::parser::queryWithParser(pp_hybrid, "n0_ref", m_n0_ref);
if (m_gamma != 1.0 && !n0_ref_given) {
    Abort("hybrid_pic_model.n0_ref should be specified if hybrid_pic_model.gamma != 1");
}
```

电子温度从 eV 转成 J 后参与压力计算:

```
// convert electron temperature from eV to J
m_elec_temp *= PhysConst::q_e;
```

Hybrid PIC 需要额外场来存储电子压强、时间插值用的 ρ/J_i 、Ampere 电流和外部电流:

```
fields.alloc_init(FieldType::hybrid_electron_pressure_fp,
    lev, amrex::convert(ba, rho_nodal_flag),
    dm, ncomps, ngRho, 0.0_rt);

fields.alloc_init(FieldType::hybrid_rho_fp_temp,
    lev, amrex::convert(ba, rho_nodal_flag),
    dm, ncomps, ngRho, 0.0_rt);

fields.alloc_init(FieldType::hybrid_current_fp_plasma, Direction{0},
    lev, amrex::convert(ba, jx_nodal_flag),
    dm, ncomps, ngJ, 0.0_rt);
```

InitData() 还会记录 J/B/E 的 index type。这个细节决定 HybridPICSolveE.cpp 里每个物理量如何从自身 staggering 插值到 Ex/Ey/Ez 的位置:

```
amrex::IntVect Jx_stag = fields.get(FieldType::current_fp, Direction{0}, 0)->ixType().toIntVect();
amrex::IntVect Bx_stag = fields.get(FieldType::Bfield_fp, Direction{0}, 0)->ixType().toIntVect();
amrex::IntVect Ex_stag = fields.get(FieldType::Efield_fp, Direction{0}, 0)->ixType().toIntVect();

Jx_IndexType[idim] = Jx_stag[idim];
Bx_IndexType[idim] = Bx_stag[idim];
Ex_IndexType[idim] = Ex_stag[idim];
```

6.10.2 顶层 field update

Hybrid field update 的主入口是 `WarpX::HybridPICEvolveFields()`。它目前硬性限制为单 level:

```

WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
    finest_level == 0,
    "Ohm's law E-solve only works with a single level.");

```

进入这个函数时，粒子已经被推到 x^{n+1} 。接着沉积 ρ^{n+1} 和 $J_i^{n+1/2}$ ：

```

// The particles have now been pushed to their t_{n+1} positions.
// Perform charge deposition at t_{n+1} and current deposition at t_{n+1/2}.
HybridPICDepositRhoAndJ();

```

```

// Get the external current
m_hybrid_pic_model->GetCurrentExternal();

```

因为上一步末尾保存了 ρ^n 和 $J_i^{n-1/2}$ ，源码可以构造中间时间层。 J_i^n 由两侧半步平均得到：

```

MultiFab::LinComb(
    *current_fp_temp[lev][idim],
    0.5_rt, *current_fp_temp[lev][idim], 0,
    0.5_rt, *m_fields.get(FieldType::current_fp, Direction{idim}, lev), 0,
    0, 1, current_fp_temp[lev][idim]->nGrowVect()
);

```

$\rho^{n+1/2}$ 同样由 ρ^n 和 ρ^{n+1} 平均：

```

MultiFab::LinComb(
    *rho_fp_temp[lev], 0.5_rt, *rho_fp_temp[lev], 0,
    0.5_rt, *m_fields.get(FieldType::rho_fp, lev), 0, 0, 1,
    rho_fp_temp[lev]->nGrowVect()
);

```

第一个半步用 E^n 推 $B^n \rightarrow B^{n+1/2}$ ，第二个半步用 $E^{n+1/2}$ 推 $B^{n+1/2} \rightarrow B^{n+1}$ 。最后，为了得到 E^{n+1} ，代码把 ion current 外推到 $n+1$ ：

```

MultiFab::LinComb(
    *current_fp_temp[lev][idim],
    -1._rt, *current_fp_temp[lev][idim], 0,
    2._rt, *m_fields.get(FieldType::current_fp, Direction{idim}, lev), 0,
    0, 1, current_fp_temp[lev][idim]->nGrowVect()
);

```

由于此时 $\text{current_fp_temp} = J_i^n = 0.5(J_i^{n-1/2} + J_i^{n+1/2})$ ，上式就是

$$J_i^{n+1} = \frac{3}{2}J_i^{n+1/2} - \frac{1}{2}J_i^{n-1/2}.$$

6.10.3 Ampere 电流与 Ohm kernel

每个 B 场 RK stage 都通过 FieldPush() 闭合一次 $B \rightarrow J \rightarrow E \rightarrow B$ ：

```

// Calculate J = curl x B / mu0 - J_ext
CalculatePlasmaCurrent(Bfield, eb_update_E);
// Calculate the E-field from Ohm's law
HybridPICSolveE(Efield, Jfield, Bfield, rhofield, eb_update_E, true);

```

```

// Push forward the B-field using Faraday's law
warpX.EvolveB(dt, subcycling_half, t_old);
warpX.FillBoundaryB(ng, nodal_sync);

```

CalculatePlasmaCurrent() 先用 finite-difference solver 从 B 计算 Ampere 电流，再减去外部电流：

```

warpX.get_pointer_fdt_d_solver_fp(lev)->CalculateCurrentAmpere(
    current_fp_plasma, Bfield, eb_update_E, lev
);

```

```

if (m_has_external_current) {
    ablastr::fields::VectorField current_fp_external =
        warpx.m_fields.get_all_dirs(FieldType::hybrid_current_fp_external, lev);
    for (int i=0; i<3; i++) {
        current_fp_plasma[i]->minus(*current_fp_external[i], 0, 1, 1);
    }
}

```

HybridPICSolveECartesian() 先把 J、J_i 和 B 插值到 nodal grid, 并计算 Hall 项:

```

// calculate enE = (J - Ji) x B
enE_nodal(i, j, k, 0) = (
    (jy_interp - jiy_interp) * Bz_interp
    - (jz_interp - jiz_interp) * By_interp
);
enE_nodal(i, j, k, 1) = (
    (jz_interp - jiz_interp) * Bx_interp
    - (jx_interp - jix_interp) * Bz_interp
);
enE_nodal(i, j, k, 2) = (
    (jx_interp - jix_interp) * By_interp
    - (jy_interp - jiy_interp) * Bx_interp
);

```

随后在每个 E 分量所在的 grid staggering 上除以 rho, 加入压力梯度、阻性和超阻性项。以 Ex 为例:

```

const Real rho_val = Interp(rho, nodal, Ex_stag, coarsen, i, j, k, 0);

if (rho_val < rho_floor && holmstrom_vacuum_region) {
    Ex(i, j, k) = 0._rt;
} else {
    const Real grad_Pe = (!solve_for_Faraday) ?
        T_Algo::UpwardDx(Pe, coefs_x, n_coefs_x, i, j, k)
        : 0._rt;

    const auto enE_x = Interp(enE, nodal, Ex_stag, coarsen, i, j, k, 0);
    const auto rho_val_limited = std::max(rho_val, rho_floor);

    Ex(i, j, k) = (enE_x - grad_Pe) / rho_val_limited;
}

```

solve_for_Faraday 是一个重要分支: 如果这个 E 只用于更新 B, 则 $\text{curl}(\text{grad } Pe)=0$, 压力梯度对 Faraday 更新没有贡献, 所以源码跳过它; 如果最终要求输出 E^{n+1} , 才把纵向压力项加回。

阻性和超阻性只在 solve_for_Faraday=true 时加入:

```

Ex(i, j, k) += eta(rho_val, jtot_val) * Jx(i, j, k);

auto nabla2Jx = T_Algo::Dxx(Jx, coefs_x, n_coefs_x, i, j, k)
    + T_Algo::Dyy(Jx, coefs_y, n_coefs_y, i, j, k)
    + T_Algo::Dzz(Jx, coefs_z, n_coefs_z, i, j, k);

Ex(i, j, k) -= eta_h(rho_val, btot_val) * nabla2Jx;

```

因此源码完整对应

$$\eta \mathbf{J} - \eta_h \nabla^2 \mathbf{J}.$$

6.10.4 电子压力与外部矢势 split field

电子压力闭合是多形式:

```
static amrex::Real get_pressure (amrex::Real const n0,
                                amrex::Real const T0,
                                amrex::Real const gamma,
                                amrex::Real const rho) {
    return n0 * T0 * std::pow((rho/PhysConst::q_e)/n0, gamma);
}
```

也就是

$$P_e = n_0 T_{e0} \left(\frac{n_e}{n_0} \right)^\gamma.$$

外部矢势由 ExternalVectorPotential 管理。用户提供空间矢势 $\mathbf{A}(x,y,z)$ 和时间函数 $s(t)$, 代码从中构造

$$\mathbf{B}_{\text{ext}} = s(t) \nabla \times \mathbf{A}, \quad \mathbf{E}_{\text{ext}} = -\frac{ds}{dt} \mathbf{A}.$$

源码用中心差分计算时间因子:

```
const amrex::Real scale_factor_B = m_A_time_scale[i](t);

const amrex::Real sf_l = m_A_time_scale[i](t-0.5_rt*dt);
const amrex::Real sf_r = m_A_time_scale[i](t+0.5_rt*dt);
const amrex::Real scale_factor_E = -(sf_r - sf_l)/dt;
```

然后累加到 hybrid 外部场:

```
AddExternalFieldFromVectorPotential(E_ext[lev], scale_factor_E, A_ext[lev],
    warpx.GetEBUpdateEFlag()[lev]);
AddExternalFieldFromVectorPotential(B_ext[lev], scale_factor_B, curlA_ext[lev],
    warpx.GetEBUpdateBFlag()[lev]);
```

在 HybridPICEvolveFields() 中, 若启用 split external fields, 推进前先从总 $\mathbf{B}$ 中减去外部 $\mathbf{B}$, 结束时再把外部 $\mathbf{E}/\mathbf{B}$ 加回。这一点防止 Ohm solver 把外部驱动场误当成自洽 plasma response。

这一路径的实现边界也很明确: 目前 field solve 只支持单 level; RZ Ohm solver 只支持 $m=0$; `n_floor` 是除以密度时的硬下限; `holmstrom_vacuum_region` 会在低密度区置零 $\mathbf{E}$ 以抑制真空区波动。Hybrid PIC 的正确性不能只检查 HybridPICsSolveE.cpp, 还必须同时检查沉积时间层、Ampere current、RK 子步、外部场分裂和边界填充是否一致。

6.10.5 Fluids/ 与 HybridPICModel 不是同一条流体链

这里有一个很容易混淆的边界: WarpX 当前 worktree 里同时存在

- Source/Fluids/*
- FieldSolver/FiniteDifferenceSolver/HybridPICModel/*

它们都带有“fluid”语义, 但职责完全不同。

HybridPICModel 是 field solver 内部的电子流体闭合。它不维护一套独立的 species state, 也不通过 WarpXFluidContainer 推进电子。它做的是:

1. 从 $\text{curl } \mathbf{B} / \mu_0$ 得到总 plasma current;
2. 扣掉外部电流和 kinetic ion current;
3. 用剩下的电子流体电流加上电子压强闭合广义 Ohm 定律, 求出 $\mathbf{E}$;
4. 再用 Faraday 定律和 RK 子步推进 $\mathbf{B}$ 。

而 Fluids/ 里的 MultiFluidContainer -> WarpXFluidContainer 则是另一条 runtime layer。它真正维护的是每个 fluid species 的 nodal

$$(N, NU_x, NU_y, NU_z),$$

并在每个 PIC step 里执行:

1. 从 Efield_aux/Bfield_aux gather 主场;
2. 复用粒子侧 UpdateMomentumHigueraCary(...) 加 Lorentz source;
3. 用 AdvectivePush_Muscl() 做 cold-fluid 守恒更新;
4. 把 qN 和 qNU/\gamma 再沉积回普通 rho_fp/current_fp。

所以 Fluids/ 更像“额外 cold-fluid species 参与普通场沉积”，而不是“hybrid solver 的电子闭合实现”。这也是为什么:

- WarpX.cpp 里 do_fluid_species 和 electromagnetic_solver_id == HybridPIC 是两套独立 existence gate;
- Fluids/ 会在 moving window 下整体平移并对新暴露 nodal box 重新 InitData();
- 最直接的 validation 入口是 langmuir_fluids 这类 cold-fluid regression, 而不是 ohm_solver_*。

6.11 FieldSolver regression 判据: 从 analysis 脚本反读物理检查量

前面各节解释了场求解器的离散方程和源码路径。还需要回答一个实际问题: WarpX 自己怎样判断这些 solver 没有坏掉? 答案不只在 CMakeLists.txt 里。Examples/Tests/*/CMakeLists.txt 告诉我们跑哪些输入文件和 checksum; 真正带物理含义的判据在 analysis*.py 中。

因此本节只讨论源码/脚本判据, 不把本地运行结果混入结论。FieldSolver 相关测试大致分三类:

- 明确 assert 物理量: NCI 场能、PSATD Gauss law、静电球 L2 误差、隐式能量守恒、Newton/GMRES 迭代数。
- 半物理半回归: Hybrid Ohm solver 的 RZ normal modes 和 ion beam instability 会比较谱采样或增长率 RMS 的历史值。
- 可视化加 checksum: Landau damping、magnetic reconnection、Cartesian Ohm EM modes、cylinder compression 主要生成物理图像并依靠 checksum 自动发现输出漂移。

6.11.1 NCI FDTD: 场能增长是否被 corrector 压住

../warpX/Examples/Tests/nci_fDTD_stability/analysis_ncicorr.py 的核心检查是读 plotfile 中的 Ex、Ez、By, 计算

$$\mathcal{E}_{\text{NCI}} = \sum_{\text{grid}} (E_x^2 + E_z^2 + c^2 B_y^2).$$

源码判据是:

```
use_MR = re.search("nci_correctorMR", fn) is not None
```

```
if use_MR:
    energy_corrector_off = 5.0e32
    energy_threshold = 1.0e28
else:
    energy_corrector_off = 1.5e26
    energy_threshold = 1.0e24
```

```
ex = ad0["boxlib", "Ex"].v
ez = ad0["boxlib", "Ez"].v
by = ad0["boxlib", "By"].v
energy = np.sum(ex**2 + ez**2 + scc.c**2 * by**2)
```

```
assert energy < energy_threshold
```

这条 regression 的输入骨架也值得写清。inputs_base_2d 不是空白模板, 而是已经固定了:

- 2D periodic drifting plasma;
- algo.current_deposition = esirkepov;
- algo.particle_shape = 3;
- warpX.use_filter = 1;
- warpX.cfl = 1;

- `warpx.do_subcycling = 1;`
- 电子和离子都沿漂移方向取无量纲动量 `u_z = 1000;`
- `base` 层已经打开 `particles.use_fdttd_nci_corr = 1.`

`inputs_test_2d_nci_corrector` 只是把它固定为单层 `amr.max_level = 0`, 而 `inputs_test_2d_nci_corrector_mr` 则切到 `amr.max_level = 1` 并用 `warpx.fine_tag_lo/hi` 把整个域提升到 refined level。也就是说, 这两条并不是“是否开 corrector”的 AB 对照, 而是“同一 corrected drifting-plasma 骨架”在 non-MR 与 MR 配置下的稳定性检查。

还要明确一个当前源码树边界: `analysis_ncicorr.py` 试图用

```
use_MR = re.search("nci_correctorMR", fn) is not None
```

来切换 non-MR 的 `1e24` 阈值与 MR 的 `1e28` 阈值, 但当前 `CMakeLists.txt` 给两条活跃测试传入的参数都写成 `diags/diag1000600`。因此, 从可见注册层看, MR 分支并没有被单独显式选通。保守的结论应是:

- non-MR 强断言是直接可见并可证实的;
- MR 变体的验证目标明确是“mesh refinement 下也要压制 NCI”, 但 `1e28` 那条阈值分支目前更像 `analysis` 脚本中的预留区分逻辑, 而不是注册参数层已直接证明的独立入口。

这个量不是严格写成 SI 形式的电磁能, 而是 NCI 增长指示量。脚本把 corrector 关闭时的 benchmark 能量量级也打印出来, 说明测试要捕捉的是“数值 Cherenkov 不稳定性是否被压低很多个数量级”。它对应前面 FDTD EvolveE/B、NCI corrector/filter 和边界/同步状态的组合效果, 而不是单独验证某一行 curl stencil。

6.11.2 NCI PSATD: 电场能量比与 Gauss law

PSATD 的 NCI 稳定性测试在 `../warpx/Examples/Tests/nci_psatd_stability/analysis_galilean.py`。脚本先从 `warpx_used_inputs` 判断维度、current correction、time averaging 和 single-box FFT, 然后设置不同 reference energy 与容差。

核心源码是:

```
energy = np.sum(scc.epsilon_0 / 2 * (Ex**2 + Ey**2 + Ez**2))
err_energy = energy / energy_ref
assert err_energy < tol_energy

if current_correction:
    divE = all_data["boxlib", "divE"].squeeze().v
    rho = all_data["boxlib", "rho"].squeeze().v / scc.epsilon_0
    err_charge = np.amax(np.abs(divE - rho)) / max(np.amax(divE), np.amax(rho))
    assert err_charge < tol_charge
```

这里 `energy_ref` 不是解析电磁能, 而是同一测试在不稳定设置下的参考能量。例如 Galilean PSATD case 的参考来自 `psatd.v_galilean=(0,0,0)`, averaged Galilean case 的参考来自关闭 time averaging。判断是

$$\frac{\sum \epsilon_0 |\mathbf{E}|^2 / 2}{\mathcal{E}_{\text{unstable,ref}}} < \text{tol_energy}.$$

如果打开 `psatd.current_correction`, 脚本还检查离散 Gauss law:

$$\epsilon_\rho = \frac{\|\nabla_h \cdot \mathbf{E} - \rho / \epsilon_0\|_\infty}{\max(\|\nabla_h \cdot \mathbf{E}\|_\infty, \|\rho / \epsilon_0\|_\infty)} < \text{tol_charge}.$$

这正对应前面 PSATD 章节中的两件事: `PsatdAlgorithmGalilean` 通过移动坐标系降低 NCI; current correction 通过谱空间投影修正 J, 使更新后的 E 与 rho 满足 Gauss law。

6.11.3 Maxwell hybrid QED: 真空修正后的相速度偏移

`../warpx/Examples/Tests/maxwell_hybrid_qed/analysis.py` 不是在检查辐射反作用、光子发射或 Breit-Wheeler 产额, 而是在检查 hybrid-QED 修正后的 Maxwell 色散关系。输入文件固定了:

- `warpx.grid_type = collocated`
- `algo.maxwell_solver = psatd`

- warpx.use_hybrid_QED = 1
- warpx.quantum_xi = 1.e-23

并直接用 parser 外场构造一个叠加在静态背景场 E_s 上的高斯包络平面波：

```
warpx.Ey_external_grid_function(x,y,z) = "exp(-z**2/L**2)*cos(2*pi*z/wavelength) + Es"
warpx.Bx_external_grid_function(x,y,z) = "-sqrt((1+(12*xi*Es**2)/epsilon0)/(1+(4*xi*Es**2)/epsilon0))*exp(-"
```

analysis 从最终 Ey(x_mid,z) 线抽取脉冲峰值位置，进而计算模拟相速度：

```
EyQED = EyQED_2d[EyQED_2d.shape[0] // 2, :]
z_end = dsQED.domain_left_edge[1].v + np.argmax(EyQED) * dz
phase_velocity_pic = (z_end - z_start) / dsQED.current_time.v
```

然后与输入里同一组 Es、xi 对应的理论相速度比较：

```
phase_velocity_theory = scc.c / np.sqrt(
    (1.0 + 12.0 * xi * Es**2 / scc.epsilon_0) / (1.0 + 4.0 * xi * Es**2 / scc.epsilon_0)
)
error_percent = (
    100.0 * np.abs(phase_velocity_pic - phase_velocity_theory) / phase_velocity_theory
)
assert error_percent < 1.25
```

因此它验证的是：

$$v_{\phi}^{\text{PIC}} \approx v_{\phi}^{\text{hybrid QED}} = \frac{c}{\sqrt{(1 + 12\xi E_s^2/\epsilon_0)/(1 + 4\xi E_s^2/\epsilon_0)}}.$$

这条 test 的定位应当是“field solver / Maxwell hybrid QED / vacuum-dispersion benchmark”，而不是宽泛的“QED processes”。它和第 4 章那类粒子 QED regression 的差别很大：这里没有粒子事件统计，只有带 vacuum-polarization 修正的场传播速度。

6.11.4 静电球：解析场 L2 误差与能量守恒

../warpx/Examples/Tests/electrostatic_sphere/analysis_electrostatic_sphere.py 检查均匀带电电子球的库仑展开。球半径满足

$$\ddot{r} = \frac{a}{r^2}, \quad a = \frac{q_e q_{\text{tot}}}{4\pi\epsilon_0 m_e}.$$

脚本把解析反函数写成：

```
def v_exact(r):
    return np.sqrt(q_e * q_tot / (2 * pi * e_mass * epsilon_0) * (1 / r_0 - 1 / r))

def t_exact(r):
    return np.sqrt(r_0**3 * 2 * pi * e_mass * epsilon_0 / (q_e * q_tot)) * (
        np.sqrt(r / r_0 - 1) * np.sqrt(r / r_0)
        + np.log(np.sqrt(r / r_0 - 1) + np.sqrt(r / r_0))
    )

r_end = fsolve(func, r_0)[0]

def E_exact(r):
    return np.sign(r) * (
        q_tot / (4 * pi * epsilon_0 * r**2) * (abs(r) >= r_end)
        + q_tot * abs(r) / (4 * pi * epsilon_0 * r_end**3) * (abs(r) < r_end)
    )
```

然后沿三条坐标轴抽取 WarpX 电场，避开靠近边界的区域，计算相对 L2 误差：


```
L2_error = np.sqrt(sum((E_exact_grid - E_grid) ** 2)) / np.sqrt(
    sum((E_exact_grid) ** 2)
)
```

```
assert L2_error_x < l2_tolerance
assert L2_error_y < l2_tolerance
assert L2_error_z < l2_tolerance
```

普通 case 的 `l2_tolerance=0.05`, `emass_10` case 放宽到 0.096。如果粒子 openPMD 诊断里有 `phi`, 脚本还检查势能释放和总能量守恒:

```
assert Ep_f < 0.7 * Ep_i
assert abs((Ek_i + Ep_i) - (Ek_f + Ep_f)) < energy_fraction * (
    Ek_i + Ep_i
)
```

这组判据直接覆盖 `ElectrostaticSolvers` 的 Poisson solve、边界处理、粒子 `phi` 诊断和粒子-场能量一致性。与 NCI 测试不同, 它有明确解析解, 因此最适合作为静电求解器章节的物理闭环例子。

这里正好可以接回 Birdsall-Langdon 第一分卷 4-9 到 4-10 的两个老判断。第一, Poisson stencil 的“局部更高阶”不自动意味着整套 PIC 离散系统更准确, 因为真正决定误差的是 mover、shape、field differencing 和 solver 合起来的系统合同, 而不是某一个局部公式单独最优。第二, 在已经用 finite-size particles 和网格差分改写了短程库仑作用之后, 场能量更基础的记账式是

$$\text{ESE} \propto \sum_k \rho_k \phi_k^*$$

而不是简单把

$$\sum_k |E_k|^2$$

当成无条件等价的替代。因为一旦离散系统把 $\rho \rightarrow \phi \rightarrow E$ 的合同改成了 K^2 、 κ 、smoothing 和 staggered differencing 的版本, $|E_k|^2$ 与 $\rho_k \phi_k^*$ 的比值就会带上额外的离散因子。第 6 章后面遇到 electrostatic sphere、Pierce diode 和其它静电 benchmark 时, 应优先把 `rho`、`phi`、field solve 和总能量账本放在同一个检查框架里, 而不是只看场图像。

Chapter 8 则把这条判断推进成更系统的 finite-grid 理论: $K(k)$ 、 $\kappa(k)$ 、 $S(k)$ 和 alias sum 不是分散在不同实现角落里的“修正项”, 而是直接一起进入 grid-modified dielectric function。换句话说, field solver 在 PIC 里不是单独决定色散的; 它总是和 particle shape、sampled density、force interpolation 共同定义一条离散色散关系。所以本章讨论 Poisson、spectral solve、smoothing 和 field differencing 时, 不能只按“求解器精度”排序, 而必须同时追问这些算子会怎样改写 alias branches、Langmuir dispersion 和 warm-plasma damping 的数值边界。

Chapter 9/10 进一步把这条判断分成两半。第一, finite Δt 也会制造自己的 time aliases, 因此 numerical heating 不一定只是 spatial-grid aliases 的副产品; 当某条 plasma branch 靠近 $\pi/\Delta t$ 一带的时间 alias 时, branch-coupling 本身就能触发高噪声和非物理增热。第二, 若想得到 exact energy conservation, 关键不是“把 E 算得更准”, 而是让离散 Poisson 解、 $\sum_j \rho_j \phi_j$ 场能量账本和粒子受力共享同一套 reciprocity 合同。也正因为如此, energy-conserving 路线和 momentum-conserving 路线不是同一求解器上可自由互换的小选项, 而是两套不同的离散系统组织方式: 前者优先保证总能量账本, 后者更自然地保留零总力/动量结构, 而 long-wavelength dispersion、self-force 与 alias errors 则必须另行逐项审查。

Chapter 12 再把这件事推进到统计层。Birdsall 在 12-3 到 12-7 里说明: PIC 里的 thermal noise、Debye shielding、field correlation 和 numerical heating 不能只被看成“粒子数有限导致的随机噪声”, 而应写成带有 $S(k_p)$ 、 $\epsilon(k, \omega)$ 和时间 alias comb ω_g 的 fluctuation spectrum。此时 $1/2 \int \rho \phi$ 又一次成为更基础的能量变量, 因为它直接把 $(\rho^2)_{\{k, \omega\}}$ 通过 K^2 接到 field-energy density 上。更关键的是, Birdsall 进一步把 grid effects 写成 effective kinetic collision operator, 并用 H-theorem 说明: space-time grid 可以在 Maxwellian 本应最大熵的情形下继续制造 entropy, 这正是 nonphysical heating、drift drag 和 velocity diffusion 的统一统计图像。因此本章后面凡是谈 Poisson、PSATD、smoothing、Langmuir damping、uniform-plasma noise floor 或 NCI/stability 时, 都不该只问“色散关系对不对”, 还要追问这套离散合同在 $(\rho^2)_{\{k, \omega\}}$ 、 $1/2 \int \rho \phi$ 、drift / diffusion 和 entropy production 这几类观测量上会留下什么数值病灶。

Chapter 13 则把这条统计图像压成了更直接的工程尺度。第一, thermal plasma 的 heating/cooling 不能只按 N_D 或 CFL 粗略估; 它还显式依赖 $\lambda_D/\Delta x$ 、 $v_t \Delta t/\Delta x$ 、shape order, 以及 mover 自身的 phase error。第二, damped equations of motion 既可能抑制某些高频 branch, 也可能通过 nonresonant drag term 制造 nonphysical cooling, 所以“总能量下

降”并不自动代表数值健康。第三, Hockney 的 2d2v 长时间实验说明: 真正有设计价值的量往往不是单独的 collision time τ_s 或 heating time τ_H , 而是二者的比值 τ_H/τ_s , 以及它如何沿着 $v_t/\Delta t \approx \Delta x/2$ 这类 optimum path 随 $\lambda_D/\Delta x$ 和 particle shape 改变。也就是说, 本章后面只写“某求解器稳定”还不够; 更严谨的说法应当是, 它把 thermal-plasma 观测窗口放在了多少个 collision times 之前, 或者把 nonphysical heating / cooling 推迟到了多久之后。

Dawson 1983 对本章的补充则更偏“为什么 electrostatic solver 会自然长成这个样子”。这篇综述不是从 Poisson 方程本身出发, 而是先从 finite-size particles 与 coarse-grained density 讲起, 然后把 shape factor $\rightarrow$ charge sharing / multipole expansion $\rightarrow$ uniform-grid FFT $\rightarrow$ Fourier-space Poisson solve $\rightarrow$ inverse FFT $\rightarrow$ gather back to particle 写成标准 electrostatic particle-model contract。这样一来, 本章讨论 electrostatic / spectral 路线时就不该把 FFT-Poisson 看成孤立求解器技巧, 而应把它视为和 particle shape、source representation、field interpolation 同时定义的一整条离散系统。

同一篇综述对 electromagnetic / Darwin 路线也给出了一条很适合保留的高层边界。对 full electromagnetic model, 时间步首先受最高频 light mode 与 CFL 限制; 主动截断高频 k modes 的理由, 不只是“算得更快”, 而是把弱耦合短波 branch 从建模目标里剔除, 从而把时间分辨率留给真正关心的大尺度 collective physics。与此同时, Dawson 还明确批评了 space/time filtering 的根本不对称: 空间方向早已有 particle size、k-mode truncation 与 Fourier solve 这类成熟手段, 但时间方向并没有真正等价的 ω -space filtering, 因此 large-time-step / time-averaged routes 仍然只是不同程度的时间滤波妥协。

对 Darwin model, 他的判断更直接: 这条路线的目标不是更完整地逼近 Maxwell, 而是主动删去 displacement current 和不关心的 radiation branch, 以便在 Alfvén waves、pinches、ion-cyclotron 这类低频磁化问题上摆脱 light-wave time-step 限制。但它也不能靠“把 Maxwell 少一项再原样 leapfrog”获得, 因为直接这样做会因 different-current mutual inductance 而数值不稳定, 必须重新组织 transverse-field 方程。这正好说明本章后面遇到的 electrostatic、full EM、implicit、hybrid 或 Darwin-like low-frequency routes, 并不是同一个 solver 家族上的小微调, 而是针对不同 branch-retention 目标做出的不同模型组织。

Dawson 1983 在 numerical stability 小节里又给了一个比 Birdsall 更概括的总结: particle simulation 里的两类型数值不稳定, 本质上都来自 stroboscopic sampling。空间离散会把连续 density spectrum 投影到有限 field Fourier modes 上, 从而制造 spatial aliasing; 有限 Δt 则会把高频 branch 重新折叠成低频有效 branch, 形成 time aliasing。这样看, finite-size particles、short-wavelength cutoff 和足够小的 time step 并不只是零散经验, 而是在分别压制这两类 alias resonance。这个高层说法对本章很有用, 因为它把 electrostatic、full EM、PSATD、implicit 和后面所有 time-filtering / space-filtering 取舍都放回了同一组数值病灶。

同一篇综述在 Tests of the statistical theory of plasmas 的入口又补了一条很有用的校验边界: 对一维 electrostatic sheet model, 因其力律简单、无需 grid, 可以直接跟踪 point-particle dynamics 到接近 machine accuracy, 文中甚至给出长时间能量守恒到 10^{-12} 量级的代表性代码。这意味着后面凡是讨论 gridded electrostatic model 的 drag、diffusion、field fluctuations 或 transport coefficients, 都不该只拿解析理论作唯一标尺; 更基础的比较对象还包括这类无 grid、近 exact 的 particle benchmark。换句话说, Poisson solver、particle shape 和 field interpolation 的数值副作用, 很多时候应被理解成“相对于更 fundamental particle model 多引入了多少统计输运偏差”, 而不只是“场图看起来是否平滑”。

这条统计理论主线再往下走, 还有两个对本章特别实用的测量合同。第一, velocity diffusion 不是一条单斜率直线: Dawson 1983 明确把它分成 short-time 的 $\langle \Delta v^2 \rangle \propto \tau^2$ 阶段和 decorrelation 之后的近线性增长阶段。这意味着 diffusion coefficient 只有在进入 random-impulse regime 后才有稳定解释。第二, thermal field fluctuation 的第一层合同不是整张场图, 而是每个 Fourier mode 的 time-averaged modal energy; 对 point particles, 它满足 $KT/2$ 型 equipartition, 而 finite-size particle shape 会系统改写这一 fluctuation level。于是本章后面不论讨论 electrostatic noise floor、shape order, 还是 smoothing / spectral filtering, 都应追问它们怎样改写 modal fluctuation spectrum, 而不是只看总场能量是否变小。

再进一步, Dawson 1983 还明确说明: thermal-plasma wave diagnostics 至少要分成 power spectrum、time correlation 和 magnetized peak taxonomy 三层。power spectrum 的第一价值是把 Debye-cloud random continuum 和 collective plasma spike 分开, 而不是单纯“看哪里有峰”; 同时 $\Delta\omega \lesssim 1/T$ 又说明有限 run length 会直接限制谱结构的可解释性。对有外磁场的体系, 谱图里还会出现 Bernstein harmonics、upper-hybrid peak、可动离子时的 ion-cyclotron / lower-hybrid peaks, 以及 $\omega=0$ 的 convective-cell / charged-flux-tube 结构。于是本章后面讨论噪声底、shape order、smoothing、spectral filtering 或 magnetized fluctuation 时, 都不应只写“能量更小/更稳定”, 而应继续问: 谱是在 continuum 还是 discrete spike 上被改写、相关时间有多长、以及被改写的是哪一类 mode family。

除了均匀带电球, 当前本地 examples 里还有一个更偏工程器件侧、但同样有理论对照的静电强基准: Examples/Physics_applications/pierce_diode/。它把两平行板间的 1D Pierce diode 直接设到 Child-Langmuir 极限, 输入里:

- warpx.do_electrostatic = labframe
- boundary.potential_lo_z = 0
- boundary.potential_hi_z = extractor_voltage
- ions.flux = J_CL/q_e

也就是把注入通量直接固定成理论空间电荷限制电流。analysis 随后读取 openPMD 中的 phi、E_z、rho、j_z 和离子 z/u_z, 并把 phi(z) 与 J(z) 和 Child-Langmuir 理论解比较, 要求相对误差都低于 20%。

所以 `pierce_diode` 的意义不是泛泛的“静电应用案例”，而是：

- Poisson solver
- fixed-potential conducting boundaries
- 连续粒子注入通量
- space-charge-limited diode steady profile

这四层在同一个 1D 理论基准下被同时闭合。

6.11.5 隐式 EM: 能量、Gauss law 与求解器迭代数

隐式 solver 的 regression 不是只看场图像,而是直接读 reduced diagnostics.../warpx/Examples/Tests/implicit/analysis_1d.py 对 1D Picard case 做总能量漂移检查:

```
field_energy = np.loadtxt("diags/reducedfiles/field_energy.txt", skiprows=1)
particle_energy = np.loadtxt("diags/reducedfiles/particle_energy.txt", skiprows=1)

total_energy = field_energy[:, 2] + particle_energy[:, 2]
delta_E = (total_energy - total_energy[0]) / total_energy[0]
max_delta_E = np.abs(delta_E).max()

if re.match("test_1d_semi_implicit_picard", test_name):
    tolerance_rel = 2.5e-5
elif re.match("test_1d_theta_implicit_picard", test_name):
    tolerance_rel = 1.0e-14

assert max_delta_E < tolerance_rel
```

本项目已在当前 checkout 上完成 `inputs_test_1d_theta_implicit_picard` 的单进程复现。运行产物位于 `/Volumes/PHILIPS/programs/PIC/` 共 101 个 reduced-diagnostic 样本; 项目脚本 `scripts/analyze_implicit_theta_picard_contract.py` 与官方 `analysis_1d.py` 均通过:

$$\max \left| \frac{W(t) - W(0)}{W(0)} \right| = 3.4784001 \times 10^{-15} < 10^{-14}.$$

这条结果把第 3 章的 implicit 时间合同接到了第 6 章的 field-solver gate: 粒子能量与场能量必须作为一个总账本检查, 不能只看某一类能量。还要注意官方 `analysis_1d.py` 通过 CMake 测试目录名选择容差; 直接在任意自定义目录执行会出现 `tolerance_rel` 未定义, 因此项目保留了独立、不依赖目录名的合同分析脚本。

`theta_implicit_picard` 要求接近机器精度, `semi_implicit_picard` 允许更大的能量误差。对 exactly energy-conserving implicit EM, `analysis_implicit.py` 还检查 Gauss law RMS:

本项目随后在同一 checkout 上复现了 sibling `inputs_test_1d_semi_implicit_picard`。它同样输出 101 个 reduced-energy 样本, 但采用半隐式 EM 的实际容差合同:

scheme	$\max \Delta W / W_0 $	tolerance	status
theta-implicit Picard	$3.4784001 \times 10^{-15}$	10^{-14}	PASS
semi-implicit Picard	2.2569031×10^{-6}	2.5×10^{-5}	PASS

两条结果均由官方 `analysis_1d.py` 和项目独立脚本 `scripts/analyze_implicit_picard_energy_contract.py` 复核。这里不能把两档容差读成分析脚本不一致: 它们对应的是两个不同的时间离散/场推进合同。`theta-implicit` 分支在该基准上把粒子与场总账本压到机器精度量级; `semi-implicit` 分支则以 $2.5e-5$ 作为官方允许的能量漂移上界。运行产物分别归档于 `runs/stage-c-validation/implicit_theta_picard/` 和 `runs/stage-c-validation/implicit_semi_picard/`。

```
drho = (rho - epsilon_0 * divE) / e / ne0
drho2_avg = (drho**2).sum() / (nX * nY * nZ)
drho_rms = np.sqrt(drho2_avg)

assert drho_rms < tolerance_rel_charge
```

归一化形式是

$$\epsilon_{\text{Gauss,rms}} = \left[\frac{1}{N} \sum_i \left(\frac{\rho_i - \epsilon_0 (\nabla_h \cdot \mathbf{E})_i}{en_{e0}} \right)^2 \right]^{1/2}.$$

如果只看这些 diagnostics,很容易把 implicit case 误解成“普通场推进外加一个 nonlinear solver 黑箱”。实际上源码里,Gauss law 和能量误差背后还隐含着一条更具体的线性化装配链。../warpx/Source/FieldSolver/ImplicitSolvers/ImplicitSolver.cpp:771-788 的 PreLinearSolve() 在线性求解前会:

```
m_WarpX->DepositMassMatrices();

if (m_use_mass_matrices_jacobian) {
    FinishMassMatrices();
    SaveE();
}

if (m_use_mass_matrices_pc) {
    SyncMassMatricesPCAndApplyBCs();
    const amrex::Real theta_dt = m_theta*m_dt;
    SetMassMatricesForPC( theta_dt );
}
```

这几步不是普通缓存刷新,而是在把粒子响应拆成两套不同对象:

- current_fp_non_suborbit = J_0;
- MassMatrices_X/Y/Z = dJ/dE 的完整局域响应;
- MassMatrices_PC 是从主质量矩阵裁剪、通信、施边界、再乘上 $c^2 \mu_0 \theta \Delta t$ 后供 preconditioner 使用的近似系数场。

随后 ../warpx/Source/FieldSolver/ImplicitSolvers/ImplicitSolver.cpp:105-125 和 :144-356 把 linear stage 的电流明确写成

$$J(E) = J_{\text{suborbit}} + J_0 + MM(E - E_0),$$

其中 E_0 由 Efield_fp_save 保存。ComputeJfromMassMatrices() 不是抽象矩阵乘法,而是在每个 Jx/Jy/Jz 分量的真实 staggered grid 上,对 Ex/Ey/Ez-E0 做局域 stencil 卷积。再往下, ../warpx/Source/NonlinearSolvers/MatrixPC.H:300-318、JacobiPC.H:286-317 和 CurlCurlMLMGPC.H:275-308 分别把 MassMatrices_PC 当成稀疏矩阵条目、局域 Jacobi 权重或 MLMG 的 beta 系数来消费。

因此这些 implicit regression 实际同时在检查三层东西:

1. 粒子推进和 J_0/MM/J_{\rm suborbit} 分拆是否一致;
2. Jacobian 近似是否真的围绕同一个 E_0 线性化;
3. preconditioner 拿到的 MassMatrices_PC 是否已经是边界、通信和物理系数都正确处理过的线性算子系数。

再往下一层,Newton 真正迭进 GMRES / PETSc 的也不是 R(U) 本身,而是

$$F(U) = U - b - R(U).$$

../warpx/Source/NonlinearSolvers/NewtonSolver.H:454-468 的 EvalResidual() 明确写成:

```
m_ops->ComputeRHS( m_R, a_U, a_time, a_iter, false );

// Compute residual: F(U) = U - b - R(U)
a_F.Copy(a_U);
a_F -= m_R;
a_F -= a_b;
```

而 matrix-free Jacobian ../warpx/Source/NonlinearSolvers/JacobianFunctionMF.H:198-234 再用有限差分构造方向作用:

```

m_Z.linComb( 1.0, m_Y0, eps, a_dU ); // Z = Y0 + eps*dU
m_ops->ComputeRHS(m_R, m_Z, m_cur_time, -1, true );

// dF = dU - (R(Z)-R(Y0))/eps
a_dF.linComb( 1.0, a_dU, eps_inv, m_RO );
a_dF.increment(m_R, -eps_inv);

```

因此 WarpX 的 JFNK 线性 solve 实际是在做

$$J_F(U_0) \delta U \approx \delta U - \frac{R(U_0 + \epsilon \delta U) - R(U_0)}{\epsilon},$$

而不是手工显式装配整块 Jacobian。接着 WarpX_PETSc.cpp:174-190,300-318 只把这两个本地回调接进 PETSc:

- applyMatOp 调 a_linop->apply(...)
- applyNativePC 调 a_linop->precond(...)

也就是说, PETSc 在这里不是重新定义物理, 而只是消费 WarpX 本地已经定义好的 residual、Jacobian 方向作用和 preconditioner apply。

若再切到 pc_petsc 这条支线, 结构还要再分一次: ../warpx/Source/FieldSolver/ImplicitSolvers/StrangImplicitSpectralEM.cpp:1 里, Strang split implicit spectral EM 的 nonlinear 右端不是 curl-curl 场更新, 而是直接

$$R(U) = -\frac{\Delta t}{2} \mu_0 c^2 J^{n+1/2},$$

因为 source-free Maxwell 部分已经由前后两次 spectral advance 吃掉。对应源码是:

```

a_RHS.Copy(FieldType::current_fp, warpx::fields::FieldType::None, allow_type_mismatch);
amrex::Real constexpr coeff = PhysConst::c2 * PhysConst::mu0;
a_RHS.scale(-coeff * 0.5_rt*m_dt);

```

而当 PETSc 不走 PCSHELL, WarpX_PETSc.cpp:115-140,342-391 会在 SNES Jacobian callback 里触发一次 assemblePCMatrix(), 把 WarpX 本地 preconditioner 近似搬进显式 Mat P。这条链的关键不是 Jacobian A 被显式装配, 而是:

- A 仍然是 shell matrix, 继续通过 apply() 做 matrix-free Jacobian;
- 只有 P 被单独装配成 sparse matrix 给 PETSc PC 使用。

对应初始化代码是:

```
KSPSetOperators( m_ksp->obj, this->m_A->obj, this->m_P->obj );
```

MatrixPC::Assemble() 则把这块 P 写成

$$P \approx I + \nabla \times (\alpha \nabla \times \cdot) + M_{PC},$$

其中单位阵、curl-curl stencil 和 MassMatrices_PC 都通过 insertOrAdd() 累加到同一行的列条目里。也就是说, pc_petsc 路径不是“把整个 implicit 求解显式矩阵化”, 而只是把 preconditioner 近似显式矩阵化, 再交给 PETSc 处理。

若再往下追一层, ../warpx/Source/FieldSolver/ImplicitSolvers/WarpXSolverDOF.cpp:19-207 说明 MatrixPC 的每一行并不是抽象的“Ex/Ey/Ez 某个分量块”, 而是先由 WarpXSolverDOF 给 staggered Efield_fp 的每个有效点分配一对 {local, global} 自由度编号。这个编号还不是对整个 MultiFab 无差别铺开, 而是先经过 getFieldDotMaskPointer(...) 取回的 dot-mask 裁剪: 只有 mask 为真的位置才进入线性系统, 其他位置的 local/global 槽都保留为 invalid。于是 MatrixPC::Assemble() 里的

```

const int ridx_l = dof_arr(i,j,k,0);
const int ridx_g = dof_arr(i,j,k,1);
if (ridx_l < 0) { return; }

```

实际意思就是: 这一条矩阵行只对应一个被 dot-mask 接受的 staggered 电场自由度, 而 ridx_l 决定它在本 rank 的行号, ridx_g 决定它在全局稀疏矩阵里的真实列号。

在此基础上, ../warpx/Source/NonlinearSolvers/MatrixPC.H:319-809 再按几何把这一行写成局域 stencil。共有三层叠加:

1. 先无条件写单位对角 I;

2. 若 $\text{thetaDt} > 0$, 再写 `curl(alpha curl .)` 的离散条目;
3. 若 `m_include_mass_matrices=true`, 最后再把 `MassMatrices_PC` 的同分量局域窗口写进去。

不同几何的差异主要体现在第二层。1D Z 几何下只有横向 E_x/E_y 行带三点二阶差分, E_z 不带 curl-curl; XZ / RZ 下 $\text{dir}=0,2$ 不只是本分量三点模板, 还会额外跨到横向分量写四个 mixed-derivative 角点条目, 对应二维的 $\partial_x \partial_z$ 交叉导数; 3D 下每个分量行都会同时耦合到另外两个分量, 在两个横向方向上写二阶项和 mixed-derivative 项; RCYLINDER 则没有这类跨分量 mixed derivative, 但径向二阶项都显式带有 $1/\rho$ 这类圆柱几何因子。所有这些条目都通过 `insertOrAdd()` 合并到同一行里, 并逐项乘上 `BC_mask_Edir_arr(...)`, 所以边界条件不是事后再修, 而是在矩阵条目生成时就已经嵌入 stencil。

而 `BC_mask_Edir_arr(...)` 本身也不是临时判断得到的布尔开关, 而是 `../warpx/Source/FieldSolver/ImplicitSolvers/ThetaImplicit` 在 `pc_petsc` 模式下预先分配并写好的系数场。`InitializeCurlCurlBCMasks()` 会根据几何维度先决定每个 E 分量需要多少类 mask, 然后再把 PEC、PMC、Silver-Mueller、PECInsulator 甚至轴线 None 的边界重构系数直接写进这些分量里。所以 `MatrixPC::Assemble()` 在边界上不是“先写标准 stencil, 再删条目”, 而是直接把已经改写好的离散系数乘进对角项、邻点项和 mixed-derivative 项。

`MassMatrices_PC` 这边也有类似的“前处理后消费”结构。`../warpx/Source/FieldSolver/ImplicitSolvers/ImplicitSolver.cpp:470-7` 先按 deposition 算法、shape 和 `mass_matrices_pc_width` 得到完整的 Jacobian mass-matrix 窗口 `m_ncomp_xx/yy/zz`, 再裁出只供 preconditioner 使用的 `m_ncomp_pc_xx/yy/zz`。当前这一步只保留 `xx/yy/zz` 三个同分量块, 不显式保留 `xy/xz/...` 交叉块; 随后 `PreLinearSolve()` 再对 `MassMatrices_PC` 做同步、J 边界处理和 $c^2/\mu_0 \theta \Delta t$ 缩放。因此 `MatrixPC::Assemble()` 读到的 `sigma_ii_arr` 已经不是原始粒子沉积结果, 而是一个经过窗口裁剪、通信和边界条件处理的 diagonal-block 近似。

最后一步 `../warpx/Source/NonlinearSolvers/WarpX_PETSc.cpp:342-389,468-490` 说明 `pc_petsc` 的矩阵提交流程是: WarpX 先按本 rank 的 `m_ndofs_l` 创建 Mat P 的 local 行块, 再由 `assemblePCMatrix()` 从 `MatrixPC` 取回 device 端的行存数组, 拷回 host, 逐行调用 `MatSetValues()`, 最后统一 `MatAssemblyBegin/End`。所以这条链的并行 ownership 仍然完全跟着 `WarpXSolverDOF` 的 local/global 编号走, PETSc 只负责把这些本地行提交拼成全局 sparse matrix, 而不重新定义行列的物理含义。

这些实现细节之所以值得追到测试层, 是因为 `Examples/Tests/implicit/analysis_petsc_matrix.py` 给了它们一个非常强的硬断言: `inputs_test_2d_curl_curl_petsc_pc`, `inputs_test_rz_curl_curl_petsc_pc` 和 `inputs_test_rcylinder_curl_curl_petsc_pc` 都把 `jacobian.pc_type = pc_petsc` 和 `pc_petsc.type = lu` 放在一起, 然后直接要求

```
assert total_gmres_iters == num_steps
assert total_newton_iters == num_steps
```

也就是每个时间步恰好只需要 1 次 Newton 和 1 次 GMRES。由于 LU 在这里是精确线性求解器, 这组 regression 实际不是在检验长期物理解, 而是在把 `WarpXSolverDOF` 编号、`MatrixPC::Assemble()` 几何条目、`curl2_BC_mask`、`MassMatrices_PC` 和 `assemblePCMatrix()` 提交链整体当成一个“应当等价于精确 PC”的矩阵装配系统来验收。只要这条断言失败, 就更应该先怀疑矩阵条目生成或提交, 而不是先怀疑 Maxwell 方程本身。

同一条结构判据现在应明确绑定到三条 benchmark 名, 而不再混成一个笼统的 implicit checksum 桶:

- `test_2d_curl_curl_petsc_pc`
- `test_rz_curl_curl_petsc_pc`
- `test_rcylinder_curl_curl_petsc_pc`

它们共享同一个 `analysis_petsc_matrix.py`, 区别只在几何维度和 `MatrixPC` 的装配分支。

相比之下, `Examples/Tests/implicit/analysis_planar_pinch.py` 的验证口径更综合。它一边读取 `newton_solver.txt` 约束平均 GMRES/Newton 和 Newton/step 迭代数, 一边把 `field_energy.txt`、`particle_energy.txt` 和 `poynting_flux.txt` 合成完整能量账本, 再用 plotfile 里的 `divE` 与 `rho` 做 Gauss 定律 RMS 误差检查。因此 planar pinch 这一组 regression 不只是 solver smoke test, 而是在同时检验: implicit 粒子-场耦合是否保持能量守恒, 边界 Poynting flux 是否正确计入能量账本, preconditioner 是否维持可接受效率, 以及最终的电荷约束是否仍在机器精度附近。

另外一组经常被忽略但同样关键的 regression 是 `analysis_vandb_jfnk_2d.py` 与 `analysis_vandb_jfnk_2d_cropping.py`。前者把 `theta_implicit_em + newton + Villasenor` 放在 2D periodic thermal plasma 下, 只检查两件事: 总能量机器精度守恒, 以及

$$\rho - \epsilon_0 \nabla \cdot E$$

的 RMS 误差仍在机器精度附近。它因此更像是对 `ImplicitPushPX.cpp`、`CurrentDeposition.H` 中 `Villasenor` 路径、以及 `SyncCurrentAndRho()` 后续消费链的专项守恒验收, 而不是对 `pc_petsc` 的矩阵装配验收。`inputs_test_2d_theta_implicit_jfnk_vandb_filt` 只是把 `warpx.use_filter = 1` 打开, 然后继续用同一个 analysis, 这等于给 filter 路径加了一条“不能破坏能量守恒和 Gauss 定律”的强回归约束。

analysis_vandb_jfnk_2d_cropping.py 则更偏向边界和粒子轨道纠正路径。对应输入把 PEC 场边界、absorbing 粒子边界、particles.crop_on_PEC_boundary = 1、implicit_evolve.particle_suborbits = 1 和 algo.current_deposition = villasenor 同时打开，但 analysis 只保留一个最大局部误差断言：

```
assert drho_max < tolerance_max_charge
```

这表明它关心的不是闭域总能量，而是：当 particle cropping、PEC 边界、suborbit fallback 和 Villasenor charge-conserving deposition 一起出现时，局部 Gauss 定律还能不能被守住。换句话说，这个 regression 实际把第 5 章的 charge-conserving deposition、第 4 章的 implicit suborbit fallback，以及第 7 章的 PEC/cropping 边界语义绑成了一条共同的验证链。

因此 Examples/Tests/implicit/ 现在至少应分成五条不同验证线，而不应继续被写成一个统一的 implicit checksum 桶：

1. analysis_1d.py 验证 1D Picard 周期热等离子体的总能量漂移，其中 semi_implicit_picard 容差较宽，theta_implicit_picard 要求机器精度；
2. analysis_implicit.py 验证周期/对称边界下 exactly energy-conserving implicit EM 的总能量和 Gauss-law RMS 同时达到机器精度；
3. analysis_2d_psatd.py 验证 strang_implicit_spectral_em + psatd 的 spectral split 没有破坏总能量守恒；
4. analysis_planar_pinch.py 验证 planar pinch 的能量账本、边界 Poynting flux、平均 Newton/GMRES 迭代数和 Gauss 定律；
5. analysis_vandb_jfnk_2d.py 与 analysis_vandb_jfnk_2d_cropping.py 分别验证 JFNK + Villasenor 周期热等离子体守恒，以及 PEC cropping / suborbit fallback 组合下的局部 Gauss-law 约束。

这样看，inputs_test_2d_theta_implicit_jfnk_vandb_filtered 和 inputs_test_2d_theta_implicit_jfnk_vandb_picmi.py 也都不再是“新的 implicit 物理 benchmark”，而只是把同一条 JFNK/Villasenor 守恒合同分别延伸到 filter 路径和 PICMI front-end 映射路径。

PSATD 这边的验证树也应采用同样的分层，而不是把所有 nci_psatd_stability tests 都写成一个统一的“PSATD / spectral solver”桶。当前至少应分成三类：

1. analysis_galilean.py 族：2D/3D/RZ 的普通 Galilean、current_correction、current_correction + periodic_single_box_fft，以及 averaged Galilean 与其 hybrid-grid 版本。共同判据是把最终电场能量与一个已知不稳定参考值比较，并在 current_correction=1 时额外检查 divE-rho/\epsilon_0 的相对误差。
2. analysis_psatd_CC1.py：单独覆盖 test_3d_uniform_plasma_psatd_JRhom_CC1，验证 JRhom = CC1 配合 do_divb_cleaning/do_dive_cleaning 后的 NCI 抑制。
3. checksum-only 基线：test_2d_comoving_psatd_hybrid、test_2d_galilean_psatd_hybrid 和 test_rz_psatd_JRhom_LL2 当前都没有独立 analysis，应诚实记录为 workflow/输出基线，而不是强稳定性断言。

Planar pinch case 还必须把边界 Poynting flux 纳入能量账本：

```
dE = Efields + Eplasma + dE_poynting
rel_net_energy = np.abs(dE - dE[0]) / Eplasma
assert max_rel_net_energy < rel_net_energy_tol
```

```
assert total_gmres_iters / total_newton_iters < gmres_iters_tol
assert total_newton_iters / num_steps < newton_iters_tol
```

这说明隐式 field solver 的正确性至少有三层：离散能量守恒、Gauss law 约束、非线性/线性求解器效率。PETSc matrix 测试进一步把求解器结构变成硬断言：

```
assert total_gmres_iters == num_steps
assert total_newton_iters == num_steps
```

当 LU 作为精确求解器或预条件器时，每个时间步只需要 1 次 Newton 和 1 次 GMRES；如果这个断言失败，问题更可能出在矩阵装配、DOF 映射、PETSc bridge 或预条件器，而不是 Maxwell 方程本身。

6.11.6 Hybrid Ohm solver: 哪些是强判据，哪些只是输出回归

Hybrid Ohm solver 的测试更接近物理 benchmark。ohm_solver_em_modes/analysis_rz.py 先对 $E_\theta(r, z, t)$ 做径向 Hankel 投影、轴向 Fourier transform 和时间 Fourier transform：

```
def transform_spatially(data_for_transform):
    interp = RegularGridInterpolator(
        (info.z, info.r), data_for_transform, method="linear"
```

```
)
data_interp = interp((zg, rg))

Fmz = np.einsum("ijkl,kl->ij", proj, data_interp)
Fmn = fft.fftshift(fft.fft(Fmz, axis=1), axes=1)
return Fmn
```

```
F_kw = fft.fftshift(fft.fft(results, axis=0), axes=0)
```

它会画出 fast/slow branch 和热共振线。显式 assert 只比较谱上固定采样点：

```
amps = np.abs(F_kw[2, 1, len(kz) // 2 - 2 : len(kz) // 2 + 2])
assert np.allclose(
    amps, np.array([55.65891974, 31.29213566, 70.13683876, 15.395433])
)
```

所以这不是完整色散关系拟合，而是谱结构回归。Ion beam R instability 更接近增长率 benchmark：对 $B_y(z, t)$ 做空间 FFT，追踪 $m=4, 5, 6$ 模，并用 Munoz et al. 2018 Fig. 12a 的增长率在 $10 < t\Omega_i < 40$ 内拟合：

```
gamma4 = 0.1915611861780133
gamma5 = 0.20087036355662818
gamma6 = 0.17123024228396777
idx = np.where((t_grid > 10) & (t_grid < 40))

A4 = np.exp(np.mean(np.log(np.abs(field_kt[idx, 4] / sim.B0)) - t_points * gamma4))
m4_rms_error = np.sqrt(
    np.mean(
        (np.abs(field_kt[idx, 4] / sim.B0) - A4 * np.exp(t_points * gamma4)) ** 2
    )
)

assert np.isclose(m4_rms_error, 1.546, atol=0.01)
```

脚本注释说明这些容差来自测试创建时的误差，不是从理论直接推导出的严格误差上界。因此失败时不能只看 assert，需要重跑 full benchmark 并人工比较增长到饱和前的理论趋势。

另外几类 Hybrid Ohm 测试没有显式 assert：

- ohm_solver_ion_Landau_damping/analysis.py 画出 $|E_z(k_m, t)|/|E_z(k_m, 0)|$ 与 $\exp(-\gamma t)$ 的比较， γ 来自 Munoz et al. 2018 Fig. 14b 插值。
- ohm_solver_magnetic_reconnection/analysis.py 输出重联率

$$R(t) = \frac{\langle E_y \rangle}{v_A B_0}.$$

- ohm_solver_em_modes/analysis.py 对 Cartesian parallel/perpendicular EM modes 做二维 FFT 谱图。
- ohm_solver_cylinder_compression 在 CMake 中 analysis=OFF，只有 checksum。

这给本章一个重要限制：Hybrid PIC 章节不能把所有 regression 都写成“物理判据已严格验证”。更准确的说法是：RZ normal modes 和 ion beam instability 有脚本级硬断言；Landau damping、magnetic reconnection、Cartesian EM modes 和 cylinder compression 主要提供物理图像与输出回归线索。

6.11.7 具体 regression 入口索引

上面的验证讨论按物理检查量组织。实际维护时，还需要知道哪些 regression 入口正在覆盖这些检查。下表按当前 `../warpX/Examples/Tests` 的 CMake 与 analysis 脚本整理，目的是让读者能从正文回到可运行测试，而不是只停留在抽象“有验证”的说法上。

family	代表输入 / 测试名	analysis 入口	主要判据	本章对应的源码风险
PML FDTD Yee	pml/inputs_test_2d_pml_yee	pml/analysis_pml_yee diags/diag1000300	本步场能量除以初始激光能量, 反射率相对理论值误差 < 5%	EvolveBPML/EPML、sigma damping、PML::Exchange() 是否能吸收而不反射
PML FDTD CKC	pml/inputs_test_2d_pml_ckc	pml/analysis_pml_ckc diags/diag1000300	同样检查反射率, 理论参考值不同, 误差 < 5%	CKC stencil 与 split-field PML 是否组合正确
PML PSATD / Galilean	pml/inputs_test_2d_pml_psatd inputs_test_2d_pml_xdiag1000300	pml/analysis_pml_psatd diags/diag1000300	先要求 iteration 50 的能量与脚本常量一致到 1e-14, 再要求最终反射率 < 1e-6	PushPSATD() 后的 PML::PushPSATD()、谱场回填和 PML 边界是否正确
PML restart	pml/inputs_test_2d_pml_restart inputs_test_2d_pml_*_pml_restart	pml/analysis_default_restart diags/diag1000300	重启前后最终 plotfile 一致	PML split fields 和场 guard cells 是否可 checkpoint/restart
RZ PML PSATD	pml/inputs_test_rz_pml_psatd	pml/analysis_pml_psatd diags/diag1000500	3. max Ez / Ez < 2.0	PML_RZ::PushPSATD()、RZ spectral PML 和径向边界吸收
Galilean PSATD NCI	nci_psatd_stability/inputs_test_2d_3d_galilean_psatd_nci	nci/analysis_psatd_nci diags/diag1000300	之比小于维度/分支容差; current correction 分支还检查 max divE-rho/eps0 相对误差	Galilean PsatdAlgorithm 是否抑制 boosted-frame NCI, 并保持 Gauss law
Averaged Galilean PSATD	inputs_test_2d_averaged_galilean_psatd inputs_test_3d_averaged_galilean_psatd	inputs_test_2d_averaged_galilean_psatd inputs_test_3d_averaged_galilean_psatd	同样用电场能量比检查稳定性, time averaging 分支容差更宽	fft_do_time_averaging 的平均场回填是否稳定
PSATD-JRhom NCI	inputs_test_3d_uniform_plasma_psatd_JRhom_NCI	inputs_test_3d_uniform_plasma_psatd_JRhom_NCI diags/diag1000300	场能量除以 66e6 后 < 1e-8	OneStep_JRhom() 多次沉积与 PsatdAlgorithmJRhom* 源项积分是否抑制 NCI
RZ PSATD-JRhom smoke	inputs_test_rz_psatd_JRhom_smoke	inputs_test_rz_psatd_JRhom_smoke checksum	只有最终 plotfile checksum	RZ JRhom 当前是输出回归入口, 不应写成物理强判据
Langmuir FDTD / PSATD 2D	langmuir/inputs_test_langmuir_psatd_2d	langmuir/analysis_psatd_2d diags/diag1000080	E _x /E _z 与解析 Langmuir 场最大相对误差 < 0.0503; 四阶形函数分支 < 0.07; 部分分支追加 charge conservation	场求解器、沉积、gather 和 guard-cell 同步在解析 plasma wave 中是否闭合
Langmuir 3D / div cleaning	langmuir/inputs_test_langmuir_psatd_3d	langmuir/analysis_psatd_3d diags/diag1000040	网格场与粒子出场均与解析场比较, 误差 < 5e-2; div-cleaning 分支还检查 dF/dt = divE-rho/eps0 到 1e-2	3D PSATD/FDTD、粒子场诊断和 divergence cleaning 的一致性
Langmuir RZ / RZ PSATD	langmuir/inputs_test_langmuir_psatd_rz	langmuir/analysis_psatd_rz diags/diag1000080	E _x /E _z 与 RZ 解析 Langmuir 场误差 < 0.12, 并检查 RZ 粒子过滤诊断	RZ field solver、RZ PSATD/current correction/JRhom 和诊断过滤是否共同正确

这个索引表也暴露了一个写作边界: 有些 regression 是物理强判据, 例如 Langmuir 解析场、PML 反射率、NCI 电场能量比; 有些只是 checksum 或 restart 路径, 例如 RZ PSATD-JRhom smoke 和部分 PML restart。正文讨论“验证链”时要区分这两类证据, 不能把 checksum 说成完整物理验证。

6.12 练习与运行验证

1. **solver 分派题**: 给定 `algo.maxwell_solver`、`psatd.JRhom`、`m_implicit_solver` 和 `AMR subcycling` 四个开关, 使用第 2 章决策图判断它们分别会落到哪一个 `OneStep` 入口, 并列出一个不允许的组合。
2. **源码定位题**: 从 `EvolveB/EvolveE`、`PushPSATD`、`ImplicitSolver::ComputerRHS` 中各选一个入口, 指出它消费的是 `J/rho`、谱空间历史源项还是 `nonlinear residual`。
3. **最小运行题**: 复现 `runs/stage-c-validation/implicit_theta_picard/` 与 `implicit_semi_picard/` 的独立能量合同, 解释两者为什么分别使用 `1e-14` 与 `2.5e-5` 容差, 而不能只比较“是否通过”。

6.12.1 本章验证链的结论

综合这些脚本, `FieldSolver` 的验证链可以这样归纳:

求解器路径	analysis 量	主要检查
FDTD + NCI corrector	$\sum(E_x^2 + E_z^2 + c^2 B_y^2)$	数值 Cherenkov 是否被抑制
PSATD Galilean/current correction	电场能量比、 $\nabla \cdot E - \rho/\epsilon_0$	NCI 抑制和 Gauss law
Electrostatic Poisson	均匀球解析 E_r 的三轴 L2、粒子势能/动能	Poisson 场、边界和能量一致性
Implicit EM	总能量漂移、Gauss RMS、Newton/GMRES 迭代数	隐式离散守恒和求解器结构
Hybrid Ohm	谱采样、增长率 RMS、阻尼/重联图像、checksum	Ohm solver 的物理 benchmark 和输出回归

这也决定后续写作方式: 场求解器的“正确性”不能只靠某一个 `assert`, 而要把连续方程、离散公式、源码时间层、边界/同步和 regression analysis 合起来看。否则, 单独贴 `EvolveE.cpp` 的 `curl` 更新式, 仍然无法证明真实 `WarpX` field solver 在完整 PIC loop 中保持物理一致。

7. 边界条件、PML 与 AMR

v0.8 源码基线: 本章这一轮按相邻 `../warpx` 的 `pkuHEDPbranch / 8c488b1a9` 重新校准入口。本版先完成边界、PML、guard-cell 与 AMR 的源码入口地图; PML 公式细化、AMR regrid 后场数组重建和 regression 判据仍留到下一版继续闭合。

边界条件在 PIC 中同时作用于场和粒子。场边界控制 Maxwell 方程如何在计算域边缘闭合; 粒子边界控制宏粒子离开、反射、吸收、周期穿越或被记录的方式。二者不能混为一谈。

`WarpX` 官方理论文档把 PML、PEC、PMC、Silver-Mueller、周期边界和嵌入边界放在 `Docs/source/theory/boundary_conditions.rst`。源码入口主要是:

- `../warpx/Source/BoundaryConditions/`
- `../warpx/Source/Particles/ParticleBoundaries.cpp`
- `../warpx/Source/Particles/ParticleBoundaries_K.H`
- `../warpx/Source/Evolve/WarpXEvolve.cpp::HandleParticlesAtBoundaries`

当前边界源码精读已建立两篇基础笔记:

- `notes/code-reading/boundary/00-field-boundary-parameters.md`
- `notes/code-reading/boundary/01-pml-data-and-update.md`

随后又补入:

- `notes/code-reading/boundary/02-pec-insulator-silver-mueller.md`
- `notes/code-reading/boundary/03-boundary-parameter-table.md`
- `notes/code-reading/boundary/04-silver-mueller-internal-stencil.md`
- `notes/code-reading/embedded-boundary/00-eb-initialization.md`
- `notes/code-reading/embedded-boundary/01-face-extensions.md`
- `notes/code-reading/embedded-boundary/02-particle-scraping-and-deposition-near-eb.md`

这两篇笔记分别覆盖“参数如何进入 `WarpX`”和“PML 如何变成真实 `split fields / sigma arrays`”。对于边界模块, 这个切分比直接按文件顺序扫描更有效, 因为边界问题天然跨参数解析、主循环分派、场数组镜像和粒子沉积四层。

7.0 v0.8 源码入口地图

本章后续不能只按“边界条件”这个名词归类，因为 WarpX 中的边界语义会穿过参数解析、场数组 guard cell、PML split field、粒子删除/反射/记录、诊断和 AMR 重建。v0.8 先把读代码的入口固定如下：

问题	当前入口	v0.8 读法
field 与 particle 边界解析顺序	../warpx/Source/WarpX.cpp:274-296	MakeWarpX() 先读 field boundary，再由 field periodic 掩码约束 particle boundary，最后才构造 WarpX 单例。
field boundary 参数与 periodic 一致性	../warpx/Source/BoundaryConditions/FieldBoundaryTypes.cpp:21-30	进入 FieldBoundaryType::Default，周期方向必须 lo/hi 成对闭合。
particle boundary 参数与 field periodic 继承	../warpx/Source/Particles/ParticleBoundaryTypes.cpp:18-97	boundary.particle_lo/hi, field periodic 会把相同方向的 particle boundary 改成 periodic；构造函数层默认仍是 absorbing。
E/B 物理边界施加	../warpx/Source/BoundaryConditions/EC/ECBoundaryTypes.cpp:51-255	Silver-Mueller 和轴边界集中在这里分派；Silver-Mueller 只挂在 level 0 的 B first half-push。
rho/J 镜像与导体内清零	../warpx/Source/BoundaryConditions/EC/ECBoundaryTypes.cpp:257-302	boundary 会触发 rho/J 的反射边界处理；PECInsulator 还会在导体内清零平行分量。
PML 数据与推进	../warpx/Source/BoundaryConditions/PML/PML.cpp、WarpXEvolvePML.cpp、PML_current.H	PML 不是一个单独边界开关，而是一组 split fields、sigma/kappa 系数、current damping 和推进分支。第 6 章已经覆盖场求解器侧入口，本章继续补边界侧语义。
FillBoundary、PML exchange 与 guard cell 检查	../warpx/Source/Parallelization/WarpXCommFillBoundary.cpp:703-716	WarpXCommFillBoundary() 会进入 PML exchange/fill 和普通 MultiFab::FillBoundary，并在 guard 数不足时直接断言。
guard-cell 数量预算	../warpx/Source/Parallelization/GuardCellManager.cpp:351-400 :300-390	GuardCellManager::get_ncil、NCI、moving window、subcycling、safe mode 和 implicit 分支共同决定；不是 AMR 后临时补的常数。
AMR/load-balance 后边界 buffer 与场数组重建	../warpx/Source/Parallelization/WarpXRegrid.cpp:402-430	boundary buffer；RemakeLevel() 按原 nGrowVect() 重建 field MultiFab，并在 EB 路径使用
boundary scraping 诊断	../warpx/Source/Diagnostics/BoundaryScrapingDiagnostic.cpp:97-126、 ../warpx/Source/Particles/ParticleBoundariesBuffer.cpp	BoundaryScrapingDiagnostic 不只是删除，可能升 level，再由 scraping diagnostics 输出。

把这些入口串起来，边界章节的主线应当是：

flowchart TD

```

A["WarpX::MakeWarpX()"] --> B["parse_field_boundaries()"]
B --> C["periodicity array"]
C --> D["parse_particle_boundaries()"]
B --> E["ApplyE/B field boundary"]
E --> F["PEC / PMC / PECInsulator / Silver-Mueller / axis"]
E --> G["Apply rho/J reflective boundary"]
E --> H["PML split fields and current damping"]
H --> I["WarpXComm FillBoundary and PML exchange"]

```

```

I --> J["GuardCellManager guard budgets"]
J --> K["AMR regrid / RemakeLevel / EB factory"]
D --> L["Handle particle boundaries and buffers"]
L --> M["BoundaryScrapingDiagnostics"]

```

因此，后续解释边界时要同时回答三类问题：输入参数如何被约束，场和粒子的运行时边界动作在哪里发生，以及这些动作怎样与 PML、guard cell、AMR 和 diagnostics 互相交叉。

7.1 field / particle 边界不是两套彼此独立的输入

WarpX::MakeWarpX() 在构造单例之前，先解析 field boundary，再从中提取 periodic 掩码，最后才解析 particle boundary：

```

std::tie(field_boundary_lo, field_boundary_hi) =
    warpx::boundary_conditions::parse_field_boundaries();

const auto is_field_boundary_periodic =
    warpx::boundary_conditions::get_periodicity_array(field_boundary_lo, field_boundary_hi);

std::tie(particle_boundary_lo, particle_boundary_hi) =
    warpx::particles::parse_particle_boundaries(is_field_boundary_periodic);

```

源码位置：../warpx/Source/WarpX.cpp:284-291。

这意味着 particle 边界不是“读完自己就结束”，而是依赖 field 边界的第二阶段配置。其后果有两条：

1. 某方向若 field 是 periodic，则 particle 必须两侧都 periodic；
2. 如果用户根本没写 boundary.particle_lo/hi，periodic 的 field 方向会自动把 particle 边界改成 periodic，而不是保留 absorbing。

对应的 field consistency 检查在 FieldBoundaries.cpp：

```

WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
    (is_lo_periodic == is_hi_periodic),
    "field boundary must be consistenly periodic in both lo and hi");

```

源码位置：../warpx/Source/BoundaryConditions/FieldBoundaries.cpp:27-33。

而 field/particle 联合一致性检查在 ParticleBoundaries.cpp：

```

WARPX_ALWAYS_ASSERT_WITH_MESSAGE(
    (particle_boundary_lo[idim] == ParticleBoundaryType::Periodic) &&
    (particle_boundary_hi[idim] == ParticleBoundaryType::Periodic),
    "field and particle boundary must be periodic in both lo and hi");

```

源码位置：../warpx/Source/Particles/ParticleBoundaries.cpp:43-46。

因此，periodic 在 WarpX 中的真实语义不是“某一侧做周期延拓”，而是“整根坐标轴拓扑闭合”。

若只想先找参数入口而不立刻读实现，当前最适合查的是 notes/code-reading/boundary/03-boundary-parameter-table.md，它已经把 boundary.field_*、boundary.particle_*、boundary.potential_*、PECInsulator parser.particles.crop_on_PEC_boundaries 和 PML 参数的依赖关系汇总成总表。

7.2 电磁 field boundary 的顶层分派

field boundary 参数解析完成后，真正把边界施加到场数组的入口在 WarpXFieldBoundaries.cpp。ApplyEfieldBoundary() 顶层首先按边界类型分派：

```

if (::isAnyBoundary<FieldBoundaryType::PEC>(field_boundary_lo, field_boundary_hi)) {
    PEC::ApplyPECToEfield(...);
}

if (::isAnyBoundary<FieldBoundaryType::PMC>(field_boundary_lo, field_boundary_hi)) {
    PEC::ApplyPECToBfield(...);
}

```

```

if (::isAnyBoundary<FieldBoundaryType::PECInsulator>(field_boundary_lo, field_boundary_hi)) {
    pec_insulator_boundary->ApplyPEC_InsulatorToEfield(...);
}

```

源码位置: `../warpx/Source/BoundaryConditions/WarpxFieldBoundaries.cpp:55-161`。

`ApplyBfieldBoundary()` 除了 `PEC/PMC/PECInsulator` 外, 还处理 `Silver-Mueller`:

```

if (lev == 0) {
    if (subcycling_half == SubcyclingHalf::FirstHalf) {
        if (::isAnyBoundary<FieldBoundaryType::Absorbing_SilverMueller>(field_boundary_lo, field_boundary_hi)) {
            m_fdt_d_solver_fp[0]->ApplySilverMuellerBoundary(...);
        }
    }
}

```

源码位置: `../warpx/Source/BoundaryConditions/WarpxFieldBoundaries.cpp:231-239`。

这说明 `Silver-Mueller` 不是通用 field boundary post-process, 而是挂在 `Yee/FDTD` 的 `B first half-push` 上的专用边界。

继续往下看其内部实现时, 还要再补一层认识: 它更新的不是域内最后一层 `B`, 而是物理域外第一层 `guard cell`, 并且按 `Yee` 交错把切向 `E` 递推到切向 `B`。这一点已经单独整理在 `notes/code-reading/boundary/04-silver-mueller-internal-stencil.md`。

7.3 PEC / PMC 不只是场边界, 也是沉积对称性

官方理论文档对 `PEC` 的定义是: 边界上切向 `E` 与法向 `B` 为零; `guard` 区对场做奇偶镜像; `rho` 和平行电流的边界处理还取决于粒子边界是 `reflecting` 还是 `absorbing`。见 `../warpx/Docs/source/theory/boundary_conditions.rst:275-323`。

这意味着 `PEC/PMC` 的章节写法不能只停留在“某些分量置零”:

- 对 `E/B`, 要讲边界值和 `guard-cell` 镜像;
- 对 `rho/J`, 要讲镜像沉积与 `image charge / reflective deposition`;
- 对粒子, 要讲 `ApplyBoundaryConditions()` 和沉积语义如何配套。

这些细节现在已经在 `notes/code-reading/boundary/02-pec-insulator-silver-mueller.md` 里拆开, 后续可直接据此继续回填本章的 `PEC/PMC/PECInsulator` 小节。

当前本地 checkout 里, `PMC` 还有一条很直接的场级 regression: `Examples/Tests/pec/inputs_test_3d_pmc_field`。它在 `z` 方向设 `PMC`、在局部区域初始化正弦 `Ey/Bx` 波包, 然后用 `analysis_pec.py` 检查反射后的 `standing wave` 是否达到理论上的 `constructive interference` 振幅 $\pm 2E_{in}$ 。因此这条测试验证的不是抽象“`PMC` 边界存在”, 而是 `PMC` 通过交换 `PEC` 的 `E/B` 角色后, 反射相位与站波振幅仍满足理论合同。

`Silver-Mueller` 的 regression 则是另一条完全不同的口径。 `Examples/Tests/silver_mueller/analysis.py` 直接读取最终 `Full diagnostics`, 并要求所有场分量在脉冲离域后都满足

$$|E| < 0.01 \text{ V/m},$$

而这些输入里的入射激光峰值量级约为 10 V/m 。所以这组测试检验的不是“反射后应形成某种驻波”, 而是

$$|E_{\text{reflected}}| \ll |E_{\text{incident}}|.$$

当前本地 family 里共有四条最小基准:

- `test_1d_silver_mueller`
- `test_2d_silver_mueller_x`
- `test_2d_silver_mueller_z`
- `test_rz_silver_mueller_z`

它们分别覆盖 `1D` 轴向出射、`2D x` 向出射、`2D z` 向出射, 以及 `RZ z` 向出射; 其中 `RZ` 版本还同时把 `r_lo = none` 的轴线正则性和 `absorbing_silver_mueller` 开放边界放到同一最小回归里。

`PEC` 与 `PECInsulator` 的 regression 边界也应和上面区分开。当前本地 `pec family` 里至少有两组强 analysis:

1. `test_3d_pec_field` 与 `test_3d_pec_field_mr` 这两条分别用 `analysis_pec.py` 和 `analysis_pec_mr.py` 检查反射后 standing-wave 的 E_{y_max}/E_{y_min} 是否接近理论 $\pm 2E_{in}$ 。单级版本容差为 1%，MR 版本放宽到 5%。因此它们真正验证的是 PEC 场边界反射后的波振幅合同，而不是抽象“边界条件被支持”。
2. `test_2d_pec_field_insulator_implicit` 与 `..._restart` 这两条走 `analysis_pec_insulator_implicit.py`，把 `fieldenergy.txt` 与 `poyntingflux.txt` 合成完整能量账本，并要求相对误差低于 10^{-13} 。因此它们真正验证的是 `pec_insulator` parser 边界、`implicit` 场推进和边界 Poynting flux reduced diagnostics 一起构成的精确能量记账合同。`..._restart` 版本说明从 checkpoint 恢复后同一合同仍成立，但它并不是独立的逐字段 restart 对照。

相比之下，`test_3d_pec_particle` 当前没有独立 analysis，仍应诚实记录为粒子侧 gather/deposition 的 checksum 基线，而不是被误写成强物理 benchmark。

PML 的物理目标是吸收入射电磁波，使开放边界尽量不反射。经典 PML 思想来自 Berenger。WarpX 在 `OneStep_nosub` 的场推进后处理 PML: FDTD 分支中，场推进后若 `do_pml` 为真，执行 `DampPML()` 并填充 E/B/F/G 的 moving-window guard cells。PSATD 分支中也有单独的 PML damping。

7.4 PML 在 WarpX 里是独立子域，不是单个边界公式

PML 类本身管理的是一整套 PML 子域、split fields 和阻尼系数缓存：

```
class PML
{
public:
    PML (... ,
        int ncell, int delta, ...,
        int pml_has_particles, int do_pml_in_domain,
        ...,
        bool do_pml_dive_cleaning, bool do_pml_divb_cleaning,
        ...);
```

源码位置：`../warpX/Source/BoundaryConditions/PML.H:137-156`。

真正承载阻尼 profile 的核心数据结构是 `SigmaBox`，其中缓存了：

- `sigma / sigma_star`
- `sigma_cumsum / sigma_star_cumsum`
- `sigma_fac / sigma_star_fac`
- `sigma_cumsum_fac / sigma_star_cumsum_fac`

源码位置：`../warpX/Source/BoundaryConditions/PML.H:46-76`。

`SigmaBox` 的 profile 在 `PML.cpp` 中按离边界距离平方增长：

```
Real offset = static_cast<Real>(glo-i);
p_sigma[i-slo] = fac*(offset*offset);
...
offset = static_cast<Real>(glo-i) - 0.5_rt;
p_sigma_star[i-sslo] = fac*(offset*offset);
```

源码位置：`../warpX/Source/BoundaryConditions/PML.cpp:83-92`。

所以 `warpX.pml_delta` 控制的是阻尼增长深度，而不是简单的总厚度。

7.5 PML split field 与 PML 电流

在主循环里，PML 阻尼入口是 `WarpX::DampPML()`，Cartesian 实际工作函数是 `DampPML_Cartesian()`。见 `../warpX/Source/BoundaryConditions`。

这个函数先取出 `pml_E`、`pml_B`、`sigba` 和每个分量的 stagger 信息，然后把它们送进 `warpX_damp_pml_ex/ey/ez/bx/by/bz`。例如 `Ex` 的 split 分量阻尼：

```
if (sy == 0) {
    Ex(i,j,k,PMLComp::xy) *= sigma_star_fac_y[j-ylo];
} else {
```

```

    Ex(i,j,k,PMLComp::xy) *= sigma_fac_y[j-ylo];
}

```

源码位置: `../warpx/Source/BoundaryConditions/WarpX_PML_kernels.H:77-82`。

这说明 PML 不是给整个 E_x 统一乘一个阻尼系数, 而是对 E_{xy} 、 E_{xz} 这类 split components 按其离散位置和方向分别阻尼。

如果进一步允许粒子进入 PML, 即 `warpx.pml_has_particles = 1`, 那么粒子电流还要按 split 方式注入 PML 电场。`push_ex_pml_current()` 的形式是:

```

alpha_xy = sigjy[k-ylo]/(sigjy[k-ylo]+sigjz[l-zlo]);
alpha_xz = sigjz[l-zlo]/(sigjy[k-ylo]+sigjz[l-zlo]);
Ex(j,k,l,PMLComp::xy) = Ex(j,k,l,PMLComp::xy) - mu_c2_dt * alpha_xy * jx(j,k,l);
Ex(j,k,l,PMLComp::xz) = Ex(j,k,l,PMLComp::xz) - mu_c2_dt * alpha_xz * jx(j,k,l);

```

源码位置: `../warpx/Source/BoundaryConditions/PML_current.H:27-36`。

也就是说, PML 中的 J_x 不是直接加到“整体 E_x ”上, 而是要分摊到与阻尼方向一致的 split components 上。

PML 的 regression 入口也不能继续混成单一 checksum 桶。当前最稳定的五条验证线是:

1. test_2d_pml_x_yee
 - analysis_pml_yee.py
 - 从最终全场重建总电磁能量
 - 计算反射率 $R = E_{end} / E_{start}$
 - 要求其相对理论 $5.683000058954201e-07$ 的误差低于 5%
2. test_2d_pml_x_ckc
 - analysis_pml_ckc.py
 - 做同一反射率检查
 - 但理论值变成 $1.8015e-06$
3. test_2d_pml_x_psatd 与 test_2d_pml_x_galilean
 - 共享 analysis_pml_psatd.py
 - 先从 diag1000050 复算初始电磁能量并要求与硬编码参考值一致到 $1e-14$
 - 再要求最终反射率低于 $1e-6$
 - 因而这里验证的是 PSATD/Galilean PSATD + PML 的低反射率合同
4. test_rz_pml_psatd
 - analysis_pml_psatd_rz.py
 - 不比较能量比, 而是在脉冲离域后直接要求域内 $\max(|E_r|, |E_z|) < 2$
 - 它真正验证的是 RZ radial PML 的残余场衰减
5. test_2d_pml_x_yee_restart 与 test_2d_pml_x_psatd_restart
 - 复用顶层 Examples/analysis_default_restart.py
 - 逐字段比较 restart 与非 restart 输出
 - 因而这两条是在测 PML + solver 场景的 restart 可重复性, 而不是新物理吸收判据

相比之下, test_3d_pml_psatd_dive_divb_cleaning 当前 analysis=OFF。它把 psatd + pml + do_dive_cleaning + do_divb_cleaning + do_pml_dive/divb_cleaning 组合在一起, 但目前只应诚实记录为 workflow/output checksum 基线, 不能写成与上面四条同等级的强吸收 benchmark。

7.5.1 v0.9 边界 regression 入口索引

上面的讨论已经说明: 同样叫“边界测试”, 证据强度并不一样。CMakeLists.txt 只能说明一个输入卡被注册成 regression; 真正能说明物理合同的是 analysis*.py。v0.9 先把第 7 章需要回填的入口收成下表, 后续再逐条扩写成正文。

family	代表输入 / 测试名	analysis 入口	主要判据	本章对应的源码风险
粒子 domain boundary	boundaries/inputs_test_3d_particle_boundary	src/analysis/analysis_particle_boundaries.py diags/diag1000008; 独立 scripts/analyze_particle_boundaries_contract.py	analysis 与独立合 同通过; reflecting position/velocity 误 差 0.4/0.6 position 绝对误差 4.44e-16、velocity 误差 0, absorbing 由 3 -> 1 保留保底粒子	parse_particle_boundaries() ParticleBoundaries_K.H::appl domain boundary contract 和粒子删除/重分 布顺序
Thermal particle boundary	particle_thermal_boundary/inputs_test_3d_particle_thermal_boundary	src/analysis/analysis_particle_thermal_boundary.py 独立 scripts/analyze_particle_thermal_boundary_contract.py	final/reference 9.8450e-6 < 5e-5、 粒子能相对漂移 1.7030% < 2%	rethermalization、 PMCeller.py boundary、 ParticleEnergy/FieldEnergy reduced ledger 与热边 界能量注入
Absorbing particle boundary + thermalizer	particle_absorbing_boundary/inputs_test_3d_particle_absorbing_boundary	src/analysis/analysis_particle_absorbing_boundary.py 独立 scripts/analyze_particle_absorbing_boundary_contract.py	ParticleHistogram2D; Silver-Mueller + uz=[-5,-1] 尾部权重 2.15096e20 < 3.2e20	ParticleHistogram2D、 ParticleThermalizer、 parser-derived histogram bin window
PEC / PMC 场反射	pec/inputs_test_3d_pec_field_reflection inputs_test_3d_pmc_field_reflection	src/analysis/analysis_pec.py diags/diag1000125 或 diag1000134 独立	反射后 Ey_max/Ey_min 相对 理论 $\pm 2E_{in}$ 的误差 < 1%; PMC 复用同一站波 振幅合同	WarpXFieldBoundaries.cpp 中 PEC/PMC 对 E/B 的角色分派、 guard-cell 镜像和 FDTD 半步相位
3D PEC field reflection	pec/inputs_test_3d_pec_field_reflection	src/analysis/analysis_pec.py diags/diag1000125; 独立 scripts/analyze_3d_pec_field_reflection_contract.py	官方和独立 analysis 通 过; Ey_max/Ey_min 相对 +/-2e5 V/m 误差 0.559%/0.632% 1%	3D Yee standing-wave、z 两端 PEC、周期横向边界和反 射振幅
3D PEC + MR field reflection	pec/inputs_test_3d_pec_mr_field_reflection	src/analysis/analysis_pec_mr.py diags/diag1000125; 独立 scripts/analyze_3d_pec_mr_field_reflection_contract.py	官方和独立 analysis 通 过; 两层 AMR 上 Ey_max/Ey_min 误差 0.006%/13.564% 5%; producer 有 first-level-only projection-cleaner warning	PEC 反射、 coarse/fine field synchronization、 MBycovering-grid consumer 和清理器适用 边界
3D PMC field reflection	pec/inputs_test_3d_pmc_field_reflection	src/analysis/analysis_pec.py diags/diag1000134; 独立 scripts/analyze_3d_pmc_field_reflection_contract.py	官方和独立 analysis 通 过; Ey_max/Ey_min 相对 +/-2e5 V/m 误差 0.615%/0.690% 1%	3D Yee standing-wave、z 两端 PMC、周期横向边界; 与 PEC 共用振幅 consumer, 但边界分量 语义不同
PEC 粒子近边界场抑制 + local periodic control	pec/inputs_test_3d_pec_particle_near_boundary_suppression + local periodic control	src/analysis/analysis_pec_particle_near_boundary_suppression.py + local periodic control 独立	PEC/periodic 全场 particle value 0.0070491<0.01, Ey 比值 0.0022796<0.01; 粒 子数和末态状态一致	PEC 近边界场边界与粒子 source 的组合输出; 支撑 场抑制对照, 不等同于直接 gather/deposition kernel instrumentation

family	代表输入 / 测试名	analysis 入口	主要判据	本章对应的源码风险
PECInsulator 显式 boundary drive	pec/inputs_test_2d_parallel_implicit-0FFInsulator	scripts/analyze_pec_insulator_2d_parallel_implicit-0FF.py	初态场全零; 末态上边界中段 By=0 相对输入 $3.3e-3$ 误差 $1.54\% < 5\%$; 下边界 By=0	PEC_Insulator panstrac 局部 area_x_hi、时间驱动 By_x_hi 和 Yee field-boundary application; 不等于 implicit energy ledger
PECInsulator implicit / restart	pec/inputs_test_2d_parallel_implicit-0FFInsulator..._restart	pec/analysis_insulator_implicit-0FF.py diags/diag1000020	与 poyntingflux.txt 合成能量账本, 要求相对误差 $< 1e-13$; restart 版本继承同一合同并依赖前置测试	pec_insulator、 implicit EM、边界 Poynting flux reduced diagnostic 与 checkpoint 恢复的一致性
2D PECInsulator implicit energy	pec/inputs_test_2d_parallel_implicit-0FFInsulator	pec/analysis_insulator_implicit-0FF.py diags/diag1000020; 个 field- 独立 scripts/analyze_pec_insulator_implicit-0FF.py	theta-implicit Picard、 PECInsulator 局部边界 contraction- energy/PoyntingFlux reduced diagnostics checkpoint 恢复、隐式 solver continuation、 reduced ledger 时间序列化	theta-implicit Picard、 PECInsulator 局部边界 contraction- energy/PoyntingFlux reduced diagnostics checkpoint 恢复、隐式 solver continuation、 reduced ledger 时间序 列化
2D PECInsulator implicit restart	pec/inputs_test_2d_parallel_implicit-0FFInsulator..._restart	pec/analysis_insulator_implicit-0FF.py diags/diag1000020; 个 field- 独立 scripts/analyze_pec_insulator_implicit-0FF.py --case-name ..._restart	与 poyntingflux.txt 合成能量账本, 要求相对误差 $< 1e-13$; restart 版本继承同一合同并依赖前置测试	pec_insulator、 implicit EM、边界 Poynting flux reduced diagnostic 与 checkpoint 恢复的一致性
PML FDTD	pml/inputs_test_2d_parallel_implicit-0FFInsulator	pml/analysis_pml_yee.py diags/diag1000300	本库电磁能量除以初始激光 能量得到反射率; Yee 理论 值 $5.683000058954201e-07$ CKC 理论值 $1.8015e-06$, 相对误差 均 $< 5\%$	PML split fields、 DampPML()、 Yee/CKC stencil 与 PML::Exchange() 的组合
PML PSATD / Galilean	pml/inputs_test_2d_parallel_implicit-0FFInsulator	pml/analysis_pml_psatd.py diags/diag1000050	先用 diag1000050 复 算初始能量并要求相对误差 $< 1e-14$, 再要求最终反 射率 $< 1e-6$	PushPSATD() 后 PML push、谱场回填、 Galilean 坐标和 PML 交界
PML restart	pml/inputs_test_2d_parallel_implicit-0FFInsulator..._restart	pml/analysis_pml_psatd.py diags/diag1000050	先用 diag1000050 复 算初始能量并要求相对误差 $< 1e-14$, 再要求最终反 射率 $< 1e-6$	PML split field、 guard cells、 checkpoint/restart 序列化
RZ PML PSATD	pml/inputs_test_rz_parallel_implicit-0FFInsulator	pml/analysis_pml_psatd.py diags/diag1000500	先用 diag1000500 复 算初始能量并要求相对误差 $< 1e-14$, 再要求最终反 射率 $< 1e-6$	PML_RZ、RZ spectral PML、径向 PML 与轴 线 none 边界的组合
PML cleaning smoke	pml/inputs_test_3d_parallel_implicit-0FFInsulator	pml/analysis_pml_psatd.py diags/diag1000500	先用 diag1000500 复 算初始能量并要求相对误差 $< 1e-14$, 再要求最终反 射率 $< 1e-6$	do_pml_dive_cleaning/do_pml_ 的 workflow 覆盖, 但缺少强物理断言

family	代表输入 / 测试名	analysis 入口	主要判据	本章对应的源码风险
particles in PML	particles_in_pml/inputs_tests_3d_pml/analyze_particles_in_pml	particles_in_pml_has_particles=1、in-domain PML、 scripts/analyze_particles_in_pml.py	2.554e-4<3e-4、2D 3D 单层 4.326<10 均 通过; 3D MR 官方 signed 106.435<110 通过但 absolute 110.399>110 失败	damping、AMR fine patch 与粒子穿出后的残 余场; 官方 analysis 使用 有符号最大值, 不能替代绝 对值审计
RZ Silver-Mueller open boundary	silver_mueller/inputs_tests_rz_silver_mueller/analyze_rz_silver_mueller	silver_mueller_analysis 通过; diags/diag1000500; max(Er , Et , Ez) =	110.399>110 失败	RZ Yee + absorbing Silver-Mueller field boundary、激光脉冲出 场
3D EB particle scrape	particle_boundary_scrape/inputs_tests_3d_particle_boundary_scrape/analyze_particle_boundary_scrape	particle_boundary_scrape_analysis 通过; diags/diag1000060	子, step 60 主容器中为 0; 说明粒子在撞到 EB 后 被删除/记录	EB by ParticleBoundaryBuffer、 save_particles_at_eb、 embedded boundary 刮擦与主容器删除
3D EB absorption with PEC	particle_boundary_production/inputs_tests_3d_particle_boundary_production/analyze_particle_boundary_production	particle_boundary_production_analysis 通过; diags/diag1000060	producer 与官方 analysis 通过; step 40 612 个、step 60 0 个电 子	PEC on boundary + cubic EB + absorbing particle path; 与全 none field-boundary 的 particle scrape case 分开记录
RZ EB scraping	scraping/inputs_tests_scraping/analyze_rz_scraping	scraping_analysis 通过; diags/diag1000037	本主容器剩 512 个粒子; 每个输出 iteration 都满 足 remaining + scraped = initial, 最终 scraped+remaining 的 id 集合等于初始 id 集 合	RZ EB 刮擦、 BoundaryScraping- Diagnostics openPMD 输出、 scraped buffer flush 后的一致性
RZ EB scraping filter	scraping/inputs_tests_scraping/analyze_rz_scraping_filter	scraping_filter_analysis 通过; diags/diag1000037	本主容器剩 512 个粒子; 由 于只记录 z > 0, 检查 2 * scraped + remaining = initial, 并要求 scraped 粒子中 z <= 0 数量为 0	BoundaryScrapingDiagnostics 的 per-species plot_filter_function、 scraped buffer 选择性 输出
RZ EB flux injection	flux_injection/inputs_tests_flux_injection/analyze_rz_flux_injection	flux_injection_analysis 通过; diags/diag1000020; 独立 scripts/analyze_rz_flux_injection.py	总权重相对误差 0.2156% < 1%, 最小 fid/R=0.997896 通过 向/两切向速度分布均通过 histogram gate	EB surface emission、 RZ-to-3D particle reconstruction、法/切 向/两切向速度分布均通过 布和粒子权重闭合
3D EB flux injection	flux_injection/inputs_tests_flux_injection/analyze_3d_flux_injection	flux_injection_analysis 通过; diags/diag1000020; 独立 scripts/analyze_3d_flux_injection.py	总权重相对误差 0.3655% < 1%, 最小 fid/R=0.996380 通过 向/两切向速度分布均通过 histogram gate	3D surface emission、局 部正交基构造、法/切向分 布和粒子权重闭合

family	代表输入 / 测试名	analysis 入口	主要判据	本章对应的源码风险
2D EB flux injection	flux_injection/inputs_test_2d_flux_injection	scripts/analyze_2d_flux_injection.py	EB 注入的总权重相对误差 0.0606% < 1%，最小	2D only EB surface emission、二维切向基构造、法/切向分
RZ PICMI EB mirror reflection	particle_boundary_interaction/inputs_test_rz_mirror_reflection	scripts/analyze_rz_mirror_reflection.py	EB buffer 并重注入; x 相对误差 1.7867% < 2%, y=0	ParticleBoundaryBufferWrapper、afterstep callback、反射和剩余时间推进
RZ PICMI secondary emission	secondary_ion_emission/inputs_test_rz_secondary_emission	scripts/analyze_rz_secondary_emission.py	EB buffer、128x128/256x256 controls 为 0.9977%/0.6646% 并通过	ParticleBoundaryBufferWrapper、callback、随机热 kick 与球面 impact-point 回溯; 误差对分辨率敏感
RZ PICMI spacecraft charging	spacecraft_charging/inputs_test_rz_spacecraft_charging	scripts/analyze_rz_spacecraft_charging.py	phi_min 约 0 -> 误差 3.551% < 4%、tau 相对误差 16.561% < 20%	ParticleBoundaryBufferWrapper、跨 species 收集电荷、在线改写导体势、Poisson field correction 和 charging curve
2D PICMI domain reflection buffer	particle_boundary_reflection/inputs_test_2d_domain_reflection	scripts/analyze_2d_domain_reflection.py	boundary-buffer 断言; final plotfile checksum analysis=OFF	boundary-buffer wrapper、reflection model、domain boundary scrape timestamp 与 checksum surface 分离
Cartesian 2D Silver-Mueller x	silver_mueller/inputs_test_2d_silver_mueller_x	scripts/analyze_silver_mueller_cartesian.py	通过; max(Ex , Ey , Ez) = 3.516e-3, 3.912e-3 V/m < 0.01 V/m	ApplySilverMuellerBoundary() 在 level 0 B first half-push 生效的开放边界同
Cartesian 2D Silver-Mueller z	silver_mueller/inputs_test_2d_silver_mueller_z	scripts/analyze_silver_mueller_cartesian.py	通过; max(Ex , Ey , Ez) = 3.516e-3, 9.149e-4 V/m < 0.01 V/m, 与	ApplySilverMuellerBoundary() 在 level 0 B first half-push 生效的开放边界同
Cartesian 1D Silver-Mueller	silver_mueller/inputs_test_1d_silver_mueller	scripts/analyze_silver_mueller_cartesian.py	通过; max(Ex , Ey , Ez) = 3.887e-8, 0 V/m < 0.01 V/m	1D 双侧 Gaussian pulse 离域后场
Silver-Mueller open boundary	silver_mueller/inputs_test_2d_silver_mueller_open_boundary	scripts/analyze_silver_mueller_cartesian.py	所有场分量都低于脚本阈值, 检验反射场远小于入射场	ApplySilverMuellerBoundary() 只在 level 0 B first half-push 生效的开放边界同

这个表的用途不是替代正文推导, 而是给后续写作设定证据等级。强 analysis 可以支撑“验证某个物理合同”; restart analysis 支撑“恢复后状态一致”; checksum-only 只能支撑“当前输出没有漂移”。第 7 章后续扩写时, 必须在每条边界结论旁边标明它属于哪一类证据。

本轮对官方 test_3d_particle_boundaries 做了 2-rank 复现, 官方 analysis.py 以 exit code 0 结束; 独立脚本进一步从初末态按粒子 ID 排序并回代自由飞行轨迹。reflecting 粒子的镜像位置最大绝对误差为 0、速度反号归一化误差为 0; periodic 粒子的位置

回绕最大绝对误差为 $4.4409\text{e-}16$ 、速度保持误差为 0; absorbing species 从初始 3 个粒子变为最终保留的 1 个保底粒子。报告归档于 runs/stage-c-validation/particle_boundaries_mpi2/contract.{json,md}。这条证据覆盖的是 domain boundary kernel 的解析几何合同, 不等同于 EB scraping 或 transition-zone route-count 验证。

RZ PICMI 镜面反射 case 也已完成真实 Python-enabled WarpX 2-rank 复现。此前 build_full 的 WarpX_PYTHON=OFF 只允许记录环境边界; 本轮使用现有 build_py 配置完成 WarpX_PYTHON=ON、RZ pybind bindings 和 amrex.space2d import smoke test, 再运行 inputs_test_rz_particle_boundary_interaction_picmi.py。官方 analysis.py 与独立脚本均通过: 末态数值坐标为 $(x,y,z)=(0.03307856,0,-0.20299489)$, 解析镜面反射坐标为 $(0.03321447,0,-0.20668779)$, x/z 相对误差分别为 0.4092% 和 1.7867%。这条证据直接闭合了“EB buffer -> Python after-step callback -> 法向镜像 -> 剩余 dt-delta_t 重注入”的运行链; 运行时依赖、Python-enabled build 和报告归档于 runs/stage-c-validation/particle_boundary_interaction_rz_mpi2/。

同一 Python-enabled build 还完成了 test_2d_particle_reflection_picmi 的 1-rank 运行。官方输入脚本在 sim.step(10) 后直接通过 ParticleBoundaryBufferWrapper 断言: z_{hi} buffer 有 63 个粒子且每个 stepScraped=4, z_{lo} buffer 有 67 个粒子且每个 stepScraped=8; 脚本以 exit code 0 结束。由于上游 CMake 对这条路径保持 analysis=OFF, 这里的强证据是 input-level Python assertion, 不能把 final plotfile checksum 误写成同等级的解析物理 analysis。报告归档于 runs/stage-c-validation/particle_reflection_picmi_2d/contract.{json,md}。

官方 test_rz_flux_injection_from_eb 也已完成当前 checkout 的 2-rank 复现。上游 analysis 检查总注入量、EB 外侧位置和法向/两切向速度分布, 全部通过; 独立 reader-side contract 从同一末态 plotfile 重新计算, 得到 75347 个电子宏粒子、总权重相对误差 $0.2156\% < 1\%$ 、最小 $r/R=0.997896 > 0.98$, 三组加权 histogram 的 residual 也分别低于上游阈值。该证据闭合的是“sphere EB surface emission -> RZ particle reconstruction -> distribution/weight consumer”组合链, 不等同于 secondary-emission callback 或 transition-zone route-count 证明。报告归档于 runs/stage-c-validation/rz_flux_injection_from_eb_mpi2/contract.{json,md}。

同一 flux_injection_from_eb family 的 3D sibling 也已用 2-rank producer 复现。官方 analysis 与独立 scripts/analyze_3d_flux_injection 均通过: 801308 个电子宏粒子、总权重相对误差 $0.3655\% < 1\%$ 、最小 $r/R=0.996380 > 0.98$, 法向和两条切向速度 histogram 均低于对应阈值。RZ 和 3D 两条结果共同支持 sphere EB surface-emission 的权重/几何/分布组合合同, 但不等同于粒子边界 buffer route-count 验证。报告归档于 runs/stage-c-validation/3d_flux_injection_from_eb_mpi2/contract.{json,md}。

2D cylinder sibling 随后也完成 2-rank 复现。官方 analysis 与独立 scripts/analyze_2d_flux_injection_from_eb_contract.py 均通过: 50235 个电子宏粒子、总权重相对误差 $0.0606\% < 1\%$ 、最小 $r/R=0.998653 > 0.98$, 法向和两条切向速度 histogram 均通过。至此 flux_injection_from_eb 的 RZ sphere、3D sphere 和 2D cylinder 三条几何路径都有同等级的 producer/analysis/reader-side 证据; 这仍不等同于 transition-zone route-count 或 secondary-emission callback 证明。报告归档于 runs/stage-c-validation/2d_flux_injection_from_eb_mpi2/contract.{json,md}。

官方 test_rz_spacecraft_charging_picmi 也已完成 Python-enabled 2-rank 复现。1000 步 producer 通过 ParticleBoundaryBufferWrapper 跨 species 汇总 EB 收集电荷, 并在 afterEsolve callback 中修正场、更新 set_potential_on_eb; 官方 analysis.py 和独立 scripts/analyze_spacecraft_charging_contract.py 对同一 101 个 openPMD 时间样本拟合出 $v_0=-145.9723$ 、 $\tau=3.63045\text{e-}6$ s, 相对参考误差分别为 3.551% 和 16.561%, 均在 gate 内。该证据支持当前 charging callback/application workflow 的组合合同, 不等同于长期浮动电势收敛或一般 EB route-count 证明。报告归档于 runs/stage-c-validation/spacecraft_charging_rz_mpi2/contract.{json,md}。

RZ Silver-Mueller 开放边界也已完成官方 2-rank 复现。500 步 Yee/RZ 激光脉冲 producer 后, 官方 analysis.py 与独立 scripts/analyze_rz_silver_mueller_contract.py 都检查末态 covering grid 的 $E_r/E_t/E_z$; 最大绝对值分别为 $6.690\text{e-}3$ 、 $3.043\text{e-}3$ 、 $8.990\text{e-}4$ V/m, 均低于 0.01 V/m。这条证据支撑的是 RZ Silver-Mueller field boundary 的短时低残余场合同, 不替代 PML 反射率、长期吸收效率或不同入射角的系统扫描。报告归档于 runs/stage-c-validation/rz_silver_mueller_z_mpi2/contract.{json,md}。

Cartesian 2D x 向 sibling 也已完成官方 2-rank、500 步复现。官方 silver_mueller/analysis.py 与独立 scripts/analyze_silver_mueller 在 diag1000500 上均通过: $\max|E_x|=9.1492743\text{e-}4$ 、 $\max|E_y|=3.5158070\text{e-}3$ 、 $\max|E_z|=3.9117365\text{e-}3$ V/m, 最大值 $3.9117365\text{e-}3 < 0.01$ V/m。这条结果把同一短时残余场合同扩展到 Cartesian x 向双侧开放边界; 仍不替代 2D z 向、1D sibling 的逐一复现, 也不等同于长期反射率或入射角扫描。报告归档于 runs/stage-c-validation/silver_mueller_2d_x_mpi2/contract.{json,md}。

Cartesian 2D z 向 sibling 随后按同一 2-rank、500 步配置完成复现。官方 analysis 与独立 contract 均通过: $\max|E_x|=3.9117365\text{e-}3$ 、 $\max|E_y|=3.5158069\text{e-}3$ 、 $\max|E_z|=9.1492744\text{e-}4$ V/m, 最大值 $3.9117365\text{e-}3 < 0.01$ V/m。x/z 两个 case 的分量峰值呈置换对称, 说明当前短时 residual-field consumer 对法向方向切换保持一致; 这仍不是长期反射率、不同入射角或 1D sibling 的系统扫描。报告归档于 runs/stage-c-validation/silver_mueller_2d_z_mpi2/contract.{json,md}。

Cartesian 1D sibling 也完成了官方 2-rank、500 步复现。官方 silver_mueller/analysis.py 与同一独立 Cartesian contract 在 diag1000500 上均通过: $\max|E_x|=3.8865209\text{e-}8$ 、 $\max|E_y|=3.8865209\text{e-}8$ 、 $\max|E_z|=0$ V/m, 最大值远低于 0.01 V/m。至此 1D、2D x/z 与 RZ 都有同一量纲阈值下的短时 residual-field evidence; 这仍不等于长期吸收效率或角度/频率扫描。报告归档于 runs/stage-c-validation/silver_mueller_1d_mpi2/contract.{json,md}。

3D PEC field sibling 也已完成 2-rank 复现。官方 analysis_pec.py 与独立 scripts/analyze_3d_pec_field_contract.py

在 diag1000125 上均通过: $E_{y_max}=1.988811e5$ V/m, $E_{y_min}=-1.987356e5$ V/m, 相对理论 $\pm 2e5$ V/m 的误差为 0.559%/0.632%。独立脚本另外记录 z 两端 cell-centered field 的边界最大值, 但不把它当成额外 pass gate, 因为上游合同实际检查的是站波振幅。该证据支撑的是 3D Yee + PEC 反射组合, 不等同于 PEC 粒子 gather/deposition 或 PECInsulator implicit 能量合同。报告归档于 runs/stage-c-validation/pec_field_3d_mpi2/contract.{json,md}。

PEC+MR sibling 随后完成 2-rank 复现。官方 analysis_pec_mr.py 与独立 scripts/analyze_3d_pec_field_mr_contract.py 均通过: 两层 AMR 的 $E_{y_max}=1.917992e5$ V/m, $E_{y_min}=-1.928917e5$ V/m, 相对 $\pm 2e5$ V/m 的误差为 4.100%/3.554%, 在上游放宽后的 5% gate 内。producer 同时报告 Projection Div Cleaner 只清理 first AMR level 的 warning; 本书把它保留为实现适用边界, 不将其误写成 PEC reflection analysis 失败。报告归档于 runs/stage-c-validation/pec_field_mr_3d_mpi2/contract.{json,md}。

PMC sibling 也已完成 2-rank 复现。官方 analysis_pec.py 与独立 scripts/analyze_3d_pmc_field_contract.py 在 diag1000134 上均通过: $E_{y_max}=1.987693e5$ V/m, $E_{y_min}=-1.986197e5$ V/m, 相对 $\pm 2e5$ V/m 的误差为 0.615%/0.690%。这条结果说明当前 PMC 配置下同一 standing-wave amplitude consumer 处于官方 gate 内; 由于上游 analysis 不直接断言独立磁场边界分量, 本书不把它升级为完整 PMC 分量级证明。报告归档于 runs/stage-c-validation/pmc_field_3d_mpi2/contract.{json,md}。

2D PECInsulator implicit case 也已完成 2-rank 复现。官方 analysis_pec_insulator_implicit.py 与独立 scripts/analyze_pec_insulator_implicit_contract.py 从 diags/reducedfiles/fieldenergy.txt 和 poyntingflux.txt 重建同一能量账本, 21 个样本的最大归一化差为 $1.2678e-15$, 通过 $1e-13$ gate。该证据支持“局部 PECInsulator 边界注入的场能由 Poynting flux 精确闭合”的当前隐式短时合同, 不等同于 restart continuation、长期稳定性或一般 PEC field amplitude 证明。报告归档于 runs/stage-c-validation/pec_insulator_implicit_2d_mpi2/contract.{json,md}。

对应的 restart sibling 也已从 chk000010 接续完成 2-rank 运行。官方 analysis 与独立 contract 对 continuation 的 10 个 ledger 样本均通过, 场能从 $3.633236e-5$ 延续到 $8.551785e-5$ J, 最大归一化能量差仍为 $1.2678e-15$ 。这条证据支持 checkpoint 恢复后 PECInsulator implicit energy/Poynting ledger 的连续性, 但不替代完整 restart 前后逐字段一致性审计。报告归档于 runs/stage-c-validation/pec_insulator_implicit_restart_2d_mpi2/contract.{json,md}。

同一 Python-enabled build 还运行了官方 test_rz_secondary_ion_emission_picmi。producer 正常完成 3 步并在最终 openPMD iteration 生成 2 个电子, 说明 ParticleBoundaryBufferWrapper 到 Python secondary-emission callback 的调用链已被触发; 但 64x64 默认基线的官方 analysis.py 和独立 contract 都拒绝几何 gate: 第一个电子回溯到最近离子 impact point 的相对距离为 3.6038%, 超过 2%, 第二个为 0.8237%。隔离 debug 输入显示, 失败 parent ion 的 EB buffer 交点在电子回溯前已经相对解析球面交点偏离约 3.1% 半径, 当前网格为 $dr=0.03125$, $dz=0.0625$, 球半径 0.2。随后不修改上游输入地运行 128x128 与 256x256 controls, 最大误差分别降到 0.9977% 和 0.6646%, 两者均通过官方 2% gate。因而当前最准确的成书结论是“callback/count wiring 已通过; 默认 64x64 geometry 保留 BOUNDARY; refined controls 支持分辨率敏感性诊断”, 而不是把整条 secondary-emission regression 写成通过。报告归档于 runs/stage-c-validation/secondary_ion_emission_rz_resolution/resolution-trend.{json,md}, 原始诊断归档于 runs/stage-c-validation/secondary_ion_emission_rz/geometry-diagnosis.md。

为区分实现错误与网格分辨率效应, 又在不修改上游输入的前提下把同一 RZ PICMI case 的 nr/nz 从 64/64 提高到 128/128。官方 analysis 与独立 contract 均通过: 两个电子的最大回溯相对误差降为 0.9977% < 2%, 相对 64x64 基线的最大误差下降约 3.61 倍。该对照支持“当前失败主要受 EB 交点离散分辨率控制”的诊断, 但不能把 64x64 baseline 改写成通过, 也不能替代对默认测试分辨率和上游容差的正式决策。对照报告归档于 runs/stage-c-validation/secondary_ion_emission_rz_resolution/resolution-comparison.{json,md}。

再增加 256x256 后, 最大回溯相对误差为 0.6646%, 官方和独立 gate 继续通过; 三档结果为 64x64: 3.6038%、128x128: 0.9977%、256x256: 0.6646%。相分辨率的经验误差阶约为 1.85 和 0.59, 这里只作为描述性趋势, 不能包装成正式 convergence order: 样本只有三档, 且粒子事件、callback 和分析容差均未改变。趋势报告归档于 runs/stage-c-validation/secondary_ion_emission_rz_resolution/resolution-comparison.{json,md}。

thermal boundary 也已完成官方 2-rank、2000 步复现。官方 analysis.py 和独立 reduced-ledger 脚本均通过: EF/EN 各有 201 个样本, 按上游语义以第 10 步场能作为参考, 末态场能比值为 9.0906979、末态场能 $9.8450169e-6 < 5e-5$, 粒子能从 0.0138667572 变为 0.0141029132, 相对漂移 1.7030369% < 2%。这支持“当前短时 thermal boundary workflow 的能量注入处于官方 gate 内”, 不支持“系统已达到热平衡”或“长期统计收敛”。报告归档于 runs/stage-c-validation/particle_thermal_boundary_mpi2/contract.{json,md}。

同一轮还运行了 particle_boundary_process 的 3D absorption 分支。当前 checkout 的 2-rank producer 与官方 analysis_absorption.py 均以 exit code 0 结束, step 40 保留 612 个电子, step 60 主容器为 0。它与前面的 particle_boundary_scrape 使用相同的 cubic EB 粒子推进几何, 但把 domain field boundary 固定为 PEC, 因此本书将其作为“PEC 场边界 + EB 吸收”的组合 workflow 单独记录, 而不把两个 case 合并成一个更强的单项物理结论。运行目录为 runs/stage-c-validation/particle_boundary_process_absorption_mpi2/。

此外, 官方 test_1d_particle_absorbing_boundary 已用当前重建 binary 完成 1-rank、8000 步复现, 官方 analysis.py 通过。独立脚本从实际生成的 warpx_used_inputs 读取 ParticleHistogram2D bin 定义, 再计算末态 iteration=8000 的关注窗口 $z=[0,50]$ um, $uz=[-5,-1]$; 输出 shape 为 1000x1000, 选择 bin 为 $uz=[250,317]$, $z=[667,1000]$, 尾部总权重 $2.1509559e20 < 3.2e20$ 。这条证据支持 thermalizer 对指定反向高速电子尾部的短时压制, 不等于完整 Maxwellian 分布或长期热化收敛证明。报告归档于 runs/stage-c-validation/particle_absorbing_boundary_1d/contract.{json,md}。

本轮又对官方 `test_3d_particle_scrape` 做了当前 checkout 的 2-rank 复现。producer 和官方 `analysis_scrape.py` 均以 exit code 0 结束；独立 reader-side 脚本 `scripts/analyze_particle_scrape_contract.py` 进一步读取 `diag1000020/40/60`，得到电子数 612/612/0，第 20 到 40 步总宏粒子权重相对差 0.0，第 40 步 $z_{\max} = -8.6943652e-5$ m 仍低于 EB 下边界 $-8.65e-5$ m，且所有粒子仍在 domain 内。该证据把“中间态未撞击、预撞击权重不变、末态主容器为空”收成一条可重放的时间序列合同，但由于该官方 case 没有把 scraped buffer 作为独立 plotfile species 暴露出来，不能升级为 scraped particle ID 集合守恒证明。报告归档于 `runs/stage-c-validation/particle_boundary_scrape_mpi2/contract.{json,md}`。

官方 `test_2d_particles_in_pml` 也已按当前 checkout 的 2-rank 配置完成 180 步复现。输入打开 `warpx.pml_has_particles=1`、`warpx.do_pml_in_domain=1` 和 `warpx.do_pml_j_damping=1`，两个反向运动的单粒子在域内 PML 中被吸收；官方 `analysis_particles_in_pml.py` 在 `diag1000180` 上给出 $\max_{\text{Efield}} = 2.5542538436684726e-4$ ，通过 2D 单层 $<3e-4$ 的残余场 gate。独立脚本 `scripts/analyze_particles_in_pml_contract.py` 不复用上游脚本的有符号 $\max(E_x, E_y, E_z)$ ，而是对 $E_x/E_y/E_z$ 分别取绝对值最大，得到 $\max|E_x| = 2.5542538436684726e-4$ 、 $\max|E_y| = 0$ 、 $\max|E_z| = 1.225051864499787e-4$ ，同样通过。该证据直接支持“粒子进入 PML 后，PML current damping 与 split-field 更新没有在主域留下超阈值残余电场”的当前 2D 单层合同；不等同于 3D/AMR sibling、粒子轨迹逐粒子守恒或 upstream checksum benchmark。

对应的 3D 单层输入也已完成 2-rank、120 步复现。官方 `analysis` 给出 $\max_{\text{Efield}} = 4.325973103094924 < 10$ ，扩展后的独立 contract 在 `128x64x64` finest covering grid 上得到 $\max|E_x| = 4.325973103094924$ 、 $\max|E_y| = 1.9812378771324655$ 、 $\max|E_z| = 1.9812378771325192$ ，同样通过。这里仍然验证的是粒子离开域后主域残余场，而不是粒子逐轨迹或 PML 反射率。

3D AMR sibling 的结果需要单独保留判据边界。官方 `test_3d_particles_in_pml_mr` 运行 200 步、两层 AMR 后，官方脚本按有符号 $\max(E_x, E_y, E_z)$ 得到 `106.43539539129057 < 110` 并通过；但同一 `256x256x256` finest covering grid 上，独立绝对值 contract 得到 $E_{x\min} = -110.3993781372607$ 、 $E_{x\max} = 106.43539539129057$ ，所以 $\max|E| = 110.3993781372607 > 110$ 。进一步的 signed/absolute/level decomposition 显示：初始帧三个分量全为零，末态唯一越过阈值的是边界侧 PML 区域的负向 E_x ； E_y/E_z 的正负峰值绝对值都约为 102.5971，level 0/level 1 covering-grid 读取给出相同极值。因此当前上游 consumer 对负向峰值不敏感，不能把“官方 gate 通过”直接升级成全场残余场强合同；本书将 AMR sibling 记为可运行、官方判据通过但强化审计失败的边界证据，不据此修改上游分析或容差。报告见 `runs/stage-c-validation/particles_in_pml_3d_mr_mpi2/signed-absolute-level-contract.{json,md}`，源码判据审计仍见 `runs/stage-c-validation/particles-in-pml-analysis-source-contract/`。

PEC 粒子 case 原本只有 checksum consumer；本轮用同一官方输入复制出一个仅将 `boundary.field_lo/hi` 从 `pec periodic periodic` 改为 `periodic periodic periodic` 的 local control，并以 2 ranks 运行到 step 20。独立脚本 `scripts/analyze_pec_particle_contract.py` 比较两个末态 plotfile：PEC 的全场 $\max|E| = 4.459437205311129$ V/m，periodic control 为 `632.622746012735` V/m，比值 `0.0070491 < 0.01`；切向 E_y 的比值为 `0.0022796 < 0.01`。electron/proton 各保留 1 个，末态位置和动量差的最大绝对值为 $8.94e-18$ ，说明对照主要改变的是近边界场，而不是粒子推进状态。该合同提升了原 checksum-only case 的可解释性，但由于 plotfile 不暴露粒子 gather 时刻的局部 E 样本，仍不应写成直接的 gather/deposition kernel 证明。报告归档于 `runs/stage-c-validation/pec_particle_3d_mpi2/contract.{json,md}`，periodic control 输入和报告在同级 `pec_particle_3d_periodic_control/`。

显式 PECInsulator baseline 也已完成当前 checkout 的 2-rank、10 步复现。输入在 x_{hi} 边界只对 $2.25e-2 \leq z \leq 2.75e-2$ 的局部段施加 $B_y(x_{\text{hi}}(y, z, t))$ ，并在 x_{lo} 使用 Neumann、z 方向使用 periodic。官方 CMake 没有 analysis，独立脚本 `scripts/analyze_pec_insulator_explicit_contract.py` 从初末两个 plotfile 直接读取场：初态六个 E/B 分量全为零；末态 B_y 在上边界 active 的 16 个 cell-centered z 单元中最大为 $3.249284802456277e-3$ ，相对 ramp 饱和值 $3.3e-3$ 的误差为 1.5368%，低于 5% gate；下边界对应行 $\max|B_y| = 0$ 。该证据支撑的是显式局部 PECInsulator boundary-drive 的空间定位和幅值响应，不是 implicit case 中 `FieldEnergy + PoyntingFlux` 的机器精度能量闭合；两者必须分开引用。报告归档于 `runs/stage-c-validation/pec_insulator_explicit_2d_mpi2/contract.{json,md}`。

7.5.2 v0.10 四条强 analysis 路径的正文化

第一条强路径是 `boundaries/inputs_test_3d_particle_boundaries` 对粒子 domain boundary 的闭合检查。输入卡把几何盒设为 $[-1, 1]^3$ ，并故意把三类粒子边界拆到三个方向：x 方向 reflecting，y 方向 absorbing，z 方向 periodic。`analysis.py` 不是只看最终 plotfile checksum，而是同时读取初始 `diag1000000` 和最终 `diag1000008`，先从初态粒子位置、动量和相对论速度外推自由飞行轨道，再分别套用镜像、删除和周期回绕规则。最终断言也对应三种语义：absorbing 分支只剩一个没有越界的保底粒子；reflecting 分支的速度符号反转并且位置满足

$$x_{\text{after}} = 2x_{\min/\max} - x_{\text{free}},$$

periodic 分支的速度保持不变，位置按盒长回绕；两个位置误差都要求小于 $1e-15$ 。这条 regression 因而直接覆盖 `ParticleBoundaries_K.H::apply_boundaries` 中 `Absorbing -> particle_lost`、`Reflecting -> mirror + change_sign_u` 和周期边界由 AMReX geometry 处理后的最终轨道结果。写作时可以把它作为 domain particle boundary 的最小物理合同：不是“边界参数被解析了”，而是越界粒子的生死、镜像和回绕都被解析轨道重新计算过。

第二条强路径是 `pec/analysis_pec.py` 和 `pec/analysis_pec_mr.py` 对 PEC/PMC 场反射的站波振幅检查。输入卡发射一个正弦波打到物理边界；脚本在反射后读取 E_y ，把理论阈值设为 $E_{th} = 2 E_{in}$ ，然后分别比较 E_{y_max} 与 $+E_{th}$ 、 E_{y_min} 与 $-E_{th}$ 。单层 3D PEC/PMC 的容差是 1%，MR 版本在 level-0 covering grid 上放宽到 5%。这个判据的要点是，它并不把“边界类型字符串为 `pec/pmc`”当作结论，而是要求反射波和入射波叠加上正确的站波振幅。源码上，`WarpXFieldBoundaries.cpp` 会把 PEC/PMC 映射到 `PEC::ApplyPECToEfield()` 与 `PEC::ApplyPECToBfield()` 的不同角色组合：PEC 对切向电场施加导体边界，PMC 则通过 E/B 角色互换实现磁导体边界。MR 路径额外说明同一物理合同在 `coarse/fine patch` 和物理边界同时存在时仍能保持到较宽容差内。

第三条强路径是 `particles_in_pml/analysis_particles_in_pml.py`。这组输入卡打开 `warpx.pml_has_particles=1`、`warpx.do_pml_in_domain=1` 和 `warpx.do_pml_j_damping=1`，让一对带相反电荷、相反动量的粒子从中心向两侧穿出。脚本不检查粒子本身是否存在于 PML，而是在粒子离开后读取 `finest level covering grid`，取

$$\max(E_x, E_y, E_z)$$

作为上游残余场判据：2D 单层要求 $< 3e-4$ ，2D MR 要求 $< 6e-4$ ，3D 单层要求 < 10 ，3D MR 要求 < 110 。这不是全场范数，负向峰值可能被忽略。项目脚本 `scripts/analyze_particles_in_pml_contract.py` 另取每个分量的绝对值最大值作为更严格的 reader-side 审计；2D 单层、2D MR 和 3D 单层均同时通过，但 3D MR 的 E_x 负向峰值使强化 gate 失败。这些阈值看起来不统一，但它们对应的是不同维度和 refinement 下一件事：粒子电流进入 in-domain PML 后，PML split-field 电流注入和 `DampJPML()` 必须把伪电荷留下的残余场压到可接受范围内。源码链条是 `WarpX.cpp` 解析三个 PML 粒子开关，`WarpXEvolve.cpp` 在有 PML 粒子的路径中执行 `CopyJPML()` 和 `DampJPML()`，随后 `DampPML()` 衰减 split fields；`PML_current.H` 则把 $J_x/J_y/J_z$ 分摊到对应 split components。因而这条 test 应写成“粒子穿出 PML 后的残余场合同”，而不是普通 PML 平面波吸收率合同。

2D AMR sibling 也已完成当前 checkout 的 2-rank,300 步复现。官方 analysis 在 `diag1000300` 上给出 $\max_Efield=3.661057413095795e-4 < 6e-4$ ；独立脚本在 `256x256x1 finest covering grid` 上对 $E_x/E_y/E_z$ 取绝对值最大，得到 $\max|E_x|=3.661057413095795e-4$ 、 $\max|E_y|=0$ 、 $\max|E_z|=3.2881867835369167e-4$ ，同样通过。该结果把 2D 单层和 MR 的残余场证据闭合；3D MR 仍因 signed-vs-absolute 差异保留为边界案例。

本轮又对这条 signed-vs-absolute 差异做了源码级审计和运行级分解。`scripts/audit_particles_in_pml_analysis_contract.py` 对上游 `analysis_particles_in_pml.py` 的 `max(Ex_array.max(), Ey_array.max(), Ez_array.max())`、阈值断言和缺少 `abs()` 的事实逐项通过，同时确认项目独立脚本确实执行 `np.max(np.abs(...))`。新增的 `scripts/analyze_particles_in_pml_signed_absol` 又核对了初末帧、逐分量正负峰值、物理位置和 level 0/1 读取，确认 `Ex_min=-110.3993781372607` 不是读取器把符号弄反，而是上游 consumer 没有把边界侧负向峰值纳入 gate。最小上游修正建议仍是把官方 consumer 改成逐分量绝对值最大值，但本项目不直接修改 `../warpx`；在重新运行并决定阈值前，该 case 仍只能写成“官方 gate 通过、强化 absolute gate 失败”的判据边界。源码报告归档于 `runs/stage-c-validation/particles-in-pml-analysis-source-contract/`，运行报告见 `runs/stage-c-validation/particles_in_pml_3d_mr_mpi2/signed-absolute-level-contract.{json,md}`，详细说明见 `notes/code-reading/particles/51-particles-in-pml-signed-vs-absolute-contract.md` 和 `57-particles-in-pml-signed-absolute-level-contract.md`。

第四条强路径是 RZ embedded-boundary scraping。`scraping/inputs_test_rz_scraping` 在 RZ 几何中用 `eb_implicit_function = "(x**2-0.1**2)"` 构造半径 0.1 的圆柱 EB，把电子初始化为 $0.15 < r < 0.2$ 并给负径向动量，同时打开 `electron.save_particles_at_eb=1` 和 `BoundaryScraping` openPMD 诊断。`analysis_rz.py` 的主断言有两层：最终主容器必须剩 512 个粒子；每个输出 iteration 都满足 `remaining + scraped = initial`，最终还要把 scraped buffer 和主容器里的粒子 id 合并、排序，与初始 id 集合完全一致。`analysis_rz_filter.py` 进一步打开 `diag3.electron.plot_filter_function = "z > 0"`，于是强合同变成 `2 * scraped + remaining = initial`，并且 scraped 输出里不能出现 $z \leq 0$ 的粒子。源码上，`ParticleBoundaryBuffer.cpp::gatherParticlesFromEmbeddedBoundaries()` 负责把穿过 EB 的粒子转入边界 buffer 并附加 `stepScraped/timeScraped/nx/ny/nz` 等属性；`BoundaryScrapingDiagnostics.cpp` 负责按 interval 写出 `particles_at_eb` 并 flush buffer；`ParticleDiag.cpp` 解析 per-species `plot_filter_function`。因此 RZ scraping 的正文重点应是主容器删除、scraped buffer 记录和诊断过滤三者的守恒关系。

粒子边界处理位于 `WarpXEvolve.cpp` 行 713-766。主要步骤是连续通量注入、`particlescraper` callback、`mypc->ApplyBoundaryConditions()`、domain boundary buffer 收集、粒子重分布、嵌入边界刮擦和排序。对 laser-solid、TNSA/RPA、moving window、open boundary 等案例，这一段决定了粒子是否会在边界产生非物理堆积或丢失。

本轮又运行了官方 RZ `test_rz_scraping` 与 `test_rz_scraping_filter`，均使用当前 checkout 对应的 `warpx.rz` binary 和 2 MPI ranks；两个官方 `analysis_rz*.py` 均以 exit code 0 结束。独立脚本 `scripts/analyze_rz_scraping_contract.py` 从 `diags/diag2` 与 `diags/diag3/particles_at_eb` 的 openPMD series 逐 iteration 重建守恒账本：完整 case 初始 6656 个粒子，末态剩 512、scraped 6144，每个 iteration 都满足 `remaining + scraped = 6656`，初始权重与末态剩余加 scraped 权重的相对误差为 0.0，初末粒子 ID 并集完全一致；过滤 case 末态仍剩 512，记录 3072 个 $z>0$ scraped 粒子，每个 iteration 满足 `2*scraped + remaining = 6656`，且 scraped 输出没有 $z \leq 0$ 粒子。过滤 case 的 ID 集合守恒不宣称，因为被过滤掉的另一半 scraped

buffer 没有出现在该输出 series 中。报告分别归档于 runs/stage-c-validation/rz_scraping_mpi2/contract.{json,md} 和 runs/stage-c-validation/rz_scraping_filter_mpi2/contract.{json,md}。

这里还要特别区分两类“边界参数”。boundary.particle_lo/hi 控制的是粒子越界后是否 Open/Absorbing/Reflecting/Thermal/Periodic, 真正执行在 ../warpx/Source/Particles/ParticleBoundaries_K.H 的 apply_boundaries() 中;而 particles.crop_on_PEC_bound 并不在这里直接删粒子,它只是在 WarpXParticleContainer::DepositCurrent().DepositCharge() 和 ImplicitPushPX.cpp 里生成 do_cropping 标志,告诉 Villasenor / implicit suborbit 这些轨道恢复与沉积 kernel: 当 field boundary 是 PEC 或 PECInsulator 时,边界外那段轨迹要不要在几何上截断。也就是说,一个参数决定“粒子是否还活着”,另一个参数决定“活着或刚越界的粒子,其轨迹是否还允许继续穿过 PEC 外侧参与沉积”。

这也解释了为什么 analysis_vandb_jfnk_2d_cropping.py 要把 PEC 场边界、absorbing 粒子边界、particles.crop_on_PEC_boundary = 1、implicit_evolve.particle_suborbits = 1 和 algo.current_deposition = villasenor 一起打开: 它不是在测单个参数,而是在测 ApplyBoundaryConditions -> do_cropping -> suborbit Villasenor deposition -> boundary buffer -> deleteInvalidParticles() 这条组合链还能不能维持局部 Gauss 定律。

7.5.3 v0.21 PSATD PML 源码闭环: 普通 PML、RZ PML 与 regression 边界

v0.21 继续把 PSATD PML 从“表格里一个验证项”扩成源码闭环。先看主时间步顺序。WarpX::PushPSATD() 在 Source/FieldSolver/WarpXPushFieldsEM.cpp:901-902 先处理 current correction、Vay deposition 或普通 J/rho spectral transform; 随后在 :901-902 对主域 E/B 做 forward transform。RZ 几何有一个特殊插入点: Source/FieldSolver/WarpXPushFieldsEM.cpp:904-907 会在主域 F/G transform 和 PSATDPushSpectralFields() 之前调用

```
if (pml_rz[lev0]) {
    pml_rz[lev0]->PushPSATD(lev0, m_fields, get_spectral_solver_fp(lev0));
}
```

普通 Cartesian/2D/3D PML 则不同: 主域 E/B/F/G 在 :913-926 完成 spectral push 和 backward transform 之后, Source/FieldSolver/WarpXPushFieldsEM.cpp:928-940 才逐 level 调 pml[lev]->PushPSATD(m_fields, lev), 然后再施加物理 E/B 边界条件。因此本书后续不能把 PML PSATD 写成“主域 PSATD 公式多乘一个阻尼因子”。更准确的说法是: 主域 PSATD、RZ PML spectral push、普通 PML spectral push 和物理边界条件是四个相邻但不同的 runtime stage。

普通 PML 的分配阶段也说明它是一套独立 spectral 子域。Source/BoundaryConditions/PML.cpp:785-817 在 algo.maxwell_solver = psatd 时先把 PML field guard cells 提升到 FFT stencil 所需范围; Source/BoundaryConditions/PML.cpp:913-936 随后为 PML box 单独构造 SpectralSolver。传给这套 PML spectral solver 的关键 flags 是:

flag	PML 构造时的值	含义
in_pml	true	SpectralSolver.cpp:60-65 会优先选择 PsatdAlgorithmPml, 不再走普通 comoving/Galilean/JRhom 分派树
periodic_single_box	false	PML 子域不是全局 periodic single-box spectral solve
update_with_rho	false	PML spectral push 不走普通主域 rho update 语义
fft_do_time_averaging	false	PML 子域不生成主域 time-averaged field 输出
v_galilean	继承 WarpX::m_v_galilean	Galilean PML 不是另一个 solver 类, 而是 PsatdAlgorithmPml 内部的相位因子分支
v_comoving	{0,0,0}	PML 子域不走 comoving PSATD

这组 flags 解释了一个容易误读的地方: inputs_test_2d_pml_x_galilean 虽然打开了 psatd.v_galilean = 0. 0. 0.99, 但 PML 子域仍由 PsatdAlgorithmPml 处理;Source/FieldSolver/SpectralSolver/SpectralAlgorithms/PsatdAlgorithmPml.cpp: 只把非零 Galilean velocity 记录成 m_is_galilean, 然后在更新式中使用 T2 相位因子。它不是把 PML 子域改派到普通主域的 PsatdAlgorithmGalilean。

PML::PushPSATD() 的实际工作也不是直接推进整体 Ex/Ey/Ez/Bx/By/Bz。Source/BoundaryConditions/PML.cpp:1310-1325 先取 pml_E_fp/pml_B_fp,可选取 pml_F_fp/pml_G_fp,再分别对 fine patch 和 coarse patch 调 PushPMLPSATDSinglePatch()。该函数在 Source/BoundaryConditions/PML.cpp:1329-1365 对 split components 做 forward transform, 例如 Ex 会被拆成 Exy 与 Exz, By 会被拆成 Byx 与 Byz; 开启 warpx.do_pml_dive_cleaning 时还会额外 transform Exx/Eyy/Ezz 和

Fx/Fy/Fz。后面的 PsatdAlgorithmPml.cpp:142-180 再根据是否同时开启 dive_cleaning/divb_cleaning 来重组 E/B/F/G 并选择不同公式分支。

普通 PML PSATD 的核心系数仍来自真空 spectral wave propagator.Source/FieldSolver/SpectralSolver/SpectralAlgorithms/Psa
先构造

$$k^2 = k_x^2 + k_y^2 + k_z^2, \quad C = \cos(c|k|\Delta t), \quad S_{ck} = \frac{\sin(c|k|\Delta t)}{c|k|},$$

再用 C1-C9 组织 longitudinal/transverse 投影;无 divergence cleaning 时,Source/FieldSolver/SpectralSolver/SpectralAlgorithms
继续构造 C10-C22 并更新 split E/B components; 有 do_pml_dive_cleaning 和 do_pml_divb_cleaning 时,
Source/FieldSolver/SpectralSolver/SpectralAlgorithms/PsatdAlgorithmPml.cpp:285-320 开始走 C23-C25
与 F/G 耦合分支。这说明 PML PSATD 里的 split-field 更新仍是 spectral Maxwell update, 只是状态变量不再是主域整体场, 而是
PML 子域的 split components。

RZ PML 是另一条更窄的专用路径。Source/BoundaryConditions/PML_RZ.cpp:39-69 只分配 Er/Et 和 Br/Bt 的 PML
fields; Source/BoundaryConditions/PML_RZ.cpp:195-216 对这四个 PML fields 做 RZ spectral forward transform、
spec_solver.pushSpectralFields(doing_pml=true) 和 backward transform。真正的 RZ PML algorithm 在
Source/FieldSolver/SpectralSolver/SpectralAlgorithms/PsatdAlgorithmPmlRZ.cpp:19-34 分配 C_coef 与
S_ck_coef, 在 :115-159 按

$$|k| = \sqrt{k_r^2 + k_z^2}, \quad C = \cos(c|k|\Delta t), \quad S_{ck} = \frac{\sin(c|k|\Delta t)}{c|k|}$$

生成系数; 更新式在 :100-109 只耦合 Er/Et 的 PML components 与主域 Bz、Br/Bt 的 PML components 与主域 Ez。同一文件
:161-172 明确把 RZ PML 中的 current correction 与 Vay deposition 设为未实现。这也是为什么 inputs_test_rz_pml_psatd
显式固定 psatd.current_correction = 0。

把源码放回 regression, 证据等级应写成三层:

regression	输入侧真实分支	analysis 消费面	自动断言	证据等级
test_2d_pml_x_psatd	inputs_base_2d 上 覆写 algo.maxwell_solver = psatd、 warpx.cfl = 0.7071067811865475、只消费 do_pml_divb/dive_cleaning = 0、 psatd.current_correction = 0、 psatd.update_with_rho = 1	analysis_pml_psatd. 先读 diags/diag1000050、 再读 diags/diag1000300; 只消费 Er/Et/Ez/Bx/By/Bz	第 50 步能量与 7.282940112203595e-08 的相对误差 < 1e-14; 末 态反射率 < 1e-6	2D plain PSATD + PML 的强低反射率 gate
test_2d_pml_x_galilean	inputs_base 上覆写 psatd.v_galilean = 0. 0. 0.99、 warpx.grid_type = collocated, 并打开 do_pml_divb/dive_cleaning = 1	同一 analysis_pml_psatd. 但根据 cwd 中的 galilean 切到能量 oracle diags/diag1000300; 只消费	第 50 步能量 oracle < 1e-14; 末态反射率 < 1e-6 4.3876202529614e-08	2D Galilean PSATD + collocated + PML div-cleaning 的强低反 射率 gate

regression	输入侧真实分支	analysis 消费面	自动断言	证据等级
test_rz_pml_psatd	RZ 几何、 boundary.field_hi = pml periodic、 warpx.pml_ncell = 10、 algo.maxwell_solver = psatd、单电子从轴上 径向外冲	analysis_pml_psatd_maxp、 只读末态 diags/diag1000500 的 Er/Ez	$\max(E_r), \max(E_z) < 2.0$	RZ radial PML 的末 态残余场强 gate, 不是能 量反射率 gate
test_3d_pml_psatd_divE_cleanup	3D PML cleaning Gaussian laser、 psatd.nox/noy/noz = 8、主域和 PML 的 do_dive/divb_cleaning = 1	CMake 中 analysis=OFF,只跑 analysis_default_regression.py --path diags/diag1000100	只有 checksum	workflow/output baseline, 不能写成 divergence-cleaning 强物理断言

这张表也解释了 v0.21 对第 7 章的修正: PML PSATD 的强证据目前主要来自 2D plain/Galilean 的低反射率 gate 和 RZ 的残余场 gate; 3D cleaning 组合虽然覆盖了更复杂的开关组合, 但原始 CMake consumer 只有 checksum, 不能支撑“divergence cleaning 在 3D PML 中已被物理强验证”的说法。本轮新增 scripts/analyze_pml_3d_cleaning_contract.py, 让官方输入额外输出并消费原生 divE/divB, 并与关闭 do_pml_dive_cleaning/do_pml_divb_cleaning 的单进程 control 做同一 plotfile 对照。结果显示 clean case 的 core normalized Gauss residual 为 0.771579, control 为 1.271171; 但 core normalized divB 为 1.863595 对 0.146012, clean/control 比值 12.7633, 并未呈现单调改善。因而这次 analysis 的价值是把“不能升级为强 gate”的边界变成了可复核负/对照证据, 而不是把 divE/divB 的存在误写成 cleaning 通过。报告位于 runs/stage-c-validation/pml_psatd_3d_cleaning_contract.{json,md}。真正的强结论仍需要 solver-native spectral residual、F/G 直接输出或专门 upstream analysis, 并需要在可比 MPI 配置下复现。

源码语义审计进一步解释了为什么这条对照不能直接升级为 cleaning 效率结论: PSATD divE 由 WarpX::ComputeDivE() 调用谱空间 ComputeSpectralDivE, 而 native divB 由 DivBFuncor 创建 cell-centered MultiFab 后调用 warpx_computedivb 的有限差分路径; DivEFuncor 还会按 staggered/nodal 位置先生成临时盒再 coarsen/interpolate。PML 内部真正的 cleaning state 是 F/G split fields, 但官方 input 没有输出它们。该源码-诊断映射已固化于 notes/code-reading/fieldsolver/33-pml-cleaning-diagnostic-semantics 因此 clean/control report 的 divE 改善与 divB 变差应继续作为 reader-side boundary evidence, 而不是“cleaning 失败”或“cleaning 通过”。

随后使用本机 MPICH launcher 和 FI_PROVIDER=tcp 按官方 2-rank 形状重新运行同一输入, 两个 MPI processes 成功写出 diag1000100, 并新增 scripts/analyze_pml_3d_mpi_consistency.py 做 1-rank/2-rank reader-side 对照。rho 的相对 L-infinity 差为 $9.31e-13$, 但 Ex/Ey/Ez 最大相对差分别为 6.38%/5.43%/3.76%, divE 为 7.17%; 因此这条证据证明官方 2-rank producer 在当前环境可运行, 却不支持把单进程结果直接外推成 rank-invariant field contract。报告位于 runs/stage-c-validation/pml_psatd_3d_mpi_consistency.{json,md}, 运行命令和 launcher 边界记录在 case-local run-command.md。

为避免把上述 rank-dependent 差异混入 cleaning 开关比较, 又在同一 2-rank 分解上运行了关闭两个 PML cleaning 开关的 control。scripts/analyze_pml_3d_cleaning_contract.py 给出 clean/control core normalized Gauss residual 0.764365/1.274899, 但 core normalized divB 1.879771/0.146534, 比值 12.8282; 这与单进程对照的 12.7633 同方向。因此当前最稳妥的结论是: PML cleaning 路径在该官方输入上可运行, 且 divE 指标有改善, 但 native cell-centered divB 诊断没有显示改善; 强 cleaning physics gate 仍关闭。2-rank clean/control 报告位于 runs/stage-c-validation/pml_psatd_3d_cleaning_contract_mpi2.{json,md}。

2D Cartesian PSATD-PML 也已按官方 CMake 的 2-rank 形状复跑。官方 analysis_pml_psatd.py 与独立 scripts/analyze_pml_cartesian 均通过: diag1000050 初始能量相对误差为 $1.27e-15 < 1e-14$, diag1000300 末态反射率为 $9.4704449758e-7 < 1e-6$ 。这使 Cartesian 低反射率证据从原先的单进程 project-level summary 扩展到真实 2-rank producer; 仍需保留“本地复现不替代上游 CMake checksum”的边界。报告位于 runs/stage-c-validation/pml_psatd_2d_mpi2/contract.{json,md}。

随后按官方 inputs_test_2d_pml_x_psatd_restart 的兄弟目录布局, 从 chk000150 以 2 ranks 接续到 diag1000300。官方 analysis_default_restart.py 与独立 scripts/analyze_pml_restart_contract.py 均通过, 对 Bx/By/Bz/Ex/Ey/Ez/divE/rho 八个 field 的逐数组比较给出最大绝对误差和最大相对误差均为 0.0, $1e-12$ restart gate 通过。该结果验证的是 PML + PSATD 状态序列化后的恢复一致性, 不是新的低反射率判据; 命令、输入和报告归档于 runs/stage-c-validation/test_2d_pml_x_psatd_restart/。

7.5.4 v0.22 PML 理论文献闭环：从 Berenger/APML 到 PSATD split-field 系数

v0.22 继续补 v0.21 留下的理论短板：PML 的“完全匹配”首先是连续介质和离散格式之间的合同，不是源码里某个 PML 类名自动保证的性质。WarpX 官方理论文档 Docs/source/theory/boundary_conditions.rst 的 PML 小节给出了这条合同的最小推导。它先从 Berenger 1994 的 2D TE split-field 写法开始：

$$\begin{aligned} \epsilon_0 \frac{\partial E_x}{\partial t} + \sigma_y E_x &= \frac{\partial H_z}{\partial y}, & \epsilon_0 \frac{\partial E_y}{\partial t} + \sigma_x E_y &= -\frac{\partial H_z}{\partial x}, \\ \mu_0 \frac{\partial H_{zx}}{\partial t} + \sigma_x^* H_{zx} &= -\frac{\partial E_y}{\partial x}, & \mu_0 \frac{\partial H_{zy}}{\partial t} + \sigma_y^* H_{zy} &= \frac{\partial E_x}{\partial y}, & H_z &= H_{zx} + H_{zy}. \end{aligned}$$

这组式子解释了 WarpX 代码里为什么有 E_{xy}/E_{xz}、E_{yx}/E_{yz}、B_{xy}/B_{xz} 这样的 split component：它们不是新的物理分量，而是把某个场分量按 curl 驱动方向拆开。普通场求解器只需要 E_x/E_y/E_z 和 B_x/B_y/B_z；PML 内部要记住“这个分量来自哪个方向的导数”，才能对入射方向相关的吸收剖面施加阻尼。

WarpX 文档随后把 Berenger PML 推广成 APML 形式，引入 $c_x/c_y/c_x^*/c_y^*$ 和广义电导。连续平面波分析的要点是：若

$$c_x = c_x^*, \quad c_y = c_y^*, \quad \bar{\sigma}_x = \bar{\sigma}_x^*, \quad \bar{\sigma}_y = \bar{\sigma}_y^*, \quad \frac{\sigma_x}{\epsilon_0} = \frac{\sigma_x^*}{\mu_0}, \quad \frac{\sigma_y}{\epsilon_0} = \frac{\sigma_y^*}{\mu_0},$$

则 PML 介质的阻抗与真空阻抗匹配，连续极限下任意频率和入射角都不反射。这个条件对应的是“perfectly matched”四个字真正的含义：波进入一层各向异性的吸收介质，而不是在边界上被强行截断。文档也直接提醒，离散化后这种无反射性质不会自动保持，所以才需要后面的指数型 leapfrog 更新和 regression 反射率检查。

WarpX Docs/source/refs.bib 为这条谱系提供了四个重要锚点：

key	作用	对本书当前用途
Berengerjcp94	原始 PML 论文，给出电磁波吸收的 split-field 思路	解释 PMLComp 和 FDTD PML split-field 的理论起点
Vay2002 / Vaycpc04	asymmetric PML 与 mesh refinement 中的 PML 应用	连接 APML、AMR 边界和 WarpX 文档中的 generalized PML 推导
LeeCPC2015	高阶有限差分 pseudo-spectral Maxwell solver 中的 PML 效率	连接第 6/7 章的 PSATD PML 和 PsatdAlgorithmPml.cpp
Vay2000	波方程吸收层边界条件	作为 PML/ABC 早期理论支线，辅助区分 open boundary 与 PML

本次还新增了 references/08_boundaries_pml_geometry/2016_LeeVayACP2016_Efficiency_of_the_PML_with_high-order_FD 作为 Lee/Vay PML 文献取证目录。AIP 官方页面给出题名、作者、DOI 和会议记录；但本机在 2026-06-29 访问官方 PDF 端点时返回 HTTP 403，所以该目录只保存 README.md、download-log.md 和中文讲解骨架，尚未保存 PDF 或 MinerU Markdown。这里的写作边界必须写清：v0.22 已经建立“应读哪篇文献”和“它应该解释哪段源码”的索引，但还没有完成 Lee/Vay 论文的逐段公式抽取。

把理论放回当前源码，最容易混淆的是两套系数。FDTD PML 的阻尼系数来自 Source/BoundaryConditions/PML.cpp 的 SigmaBox。pml_delta、pml_ncell、sigma/sigma_star、sigma_fac、sigma_cumsum_fac 决定 split components 在实空间里如何按方向衰减。第 6 章已经把这条链写成 SigmaBox -> ComputePMLFactors* -> DampPML_Cartesian() -> PML::Exchange()。

PsatdAlgorithmPml.cpp 中的 C1-C25 属于另一层：它们是 PML 子域内部的 pseudo-spectral Maxwell propagator。该文件的初始化函数先在谱空间为每个模式计算

$$C = \cos(c|\mathbf{k}|\Delta t), \quad S_{ck} = \frac{\sin(c|\mathbf{k}|\Delta t)}{c|\mathbf{k}|}, \quad K^{-2} = \frac{1}{|\mathbf{k}|^2}.$$

在 pushSpectralFields() 内，C1-C9 由 k_x^2 、 k_y^2 、 k_z^2 和 1-C 组成，作用是把 split components 重新投影到 longitudinal/transverse 耦合结构中。例如 $C1=(k_x^2 + k_y^2 + k_z^2)/k^2$ ， $C4=k_x^2(C-1)/k^2$ ， $C9=k_x k_y(1-C)/k^2$ 。这些系数只告诉谱空间 Maxwell 解在一个时间步内如何旋转/投影，不包含 PML 的空间吸收剖面。

C10-C22 继续把 S_{ck} 和 $dt-S_{ck}$ 与磁场/电场交叉耦合项组合起来。例如 C10 含有 $i c^2 k_x k_y k_z (dt-S_{ck})/k^2$, 而 C17-C22 含有某一方向的 k 乘以“一个方向平方乘 dt 、另外两个方向平方乘 S_{ck} ”的组合。它们只在未开启 PML divergence cleaning 时用于 split E/B 更新。开启 `dive_cleaning && divb_cleaning` 后, 源码改走 $C23=i c^2 k_x S_{ck}$ 、 $C24=i c^2 k_y S_{ck}$ 、 $C25=i c^2 k_z S_{ck}$, 并把 F/G split components 纳入同一谱推进。

因此 v0.22 的结论可以精确写成三层:

层	主要文件	本层回答的问题	不应混写成
连续 PML 理论	Docs/source/theory/boundary_conditions/PMLs/refs.bib 中 Berenger/Vay/Lee 条目	边界条件和 PMLs 与真空阻抗匹配, 为什么需要 split fields	WarpX 当前具体输入参数的默认值
实空间 PML profile/damping	PML.cpp、SigmaBox、WarpXEvolvePML.cpp、PML_current.H	PML 层数、sigma profile、场/电流阻尼如何在网格上生成和應用	PSATD 谱推进中的 C1-C25
PSATD PML 谱推进	PML::PushPSATD()、PsatdAlgorithmPml.cpp、PsatdAlgorithmPmlRZ.cpp	PML split components 如何在 pseudo-spectral 子域中推进, Galilean 相位和 cleaning 分支如何进入	FDTD PML 的 sigma_fac 或普通主域 PsatdAlgorithmGalilean

这也给下一轮工作定了一个更具体的边界: 要完成真正的 PSATD PML 文献闭环, 不再读一遍 PML.cpp, 而是取得 Lee/Vay 论文 PDF 后, 把高阶有限差分与 pseudo-spectral solver 的 PML 反射率/效率公式和 PsatdAlgorithmPml.cpp 的 split-field propagator 一一对照; 若论文公式和当前 WarpX 源码已有偏离, 还要标明这是实现演化、符号约定差异, 还是本书理解仍不完整。

7.5.5 v0.23 LeeCPC2015 获取审计与公式映射准备

v0.23 先处理一个出版级写作问题: 计划里写“取得 Lee/Vay CPC 或 AIP 论文授权 PDF, 执行 MinerU 转换”, 但当前环境不能把这一步伪装成已经完成。对技术书稿来说, 这不是小问题。如果正文声称“根据论文逐段推导”, 而项目里只有 DOI、摘要页或元数据, 那么后续审校会无法复查公式来源。

本轮把 Lee/Vay 主引用从 v0.22 的 AIP 会议版线索中拆出来, 单独建立 references/08_boundaries_pml_geometry/2015_LeeCPC2015_Efficiency 理由很简单: WarpX Docs/source/refs.bib 真正引用的是 CPC 2015 论文 LeeCPC2015, DOI 为 10.1016/j.cpc.2015.04.004; AIP Conference Proceedings 2016 版本 DOI 10.1063/1.4965625 是相关会议版, 但不是 WarpX refs.bib 里的主键。

获取审计的结果如下:

来源	v0.23 检查结果	写作结论
WarpX Docs/source/refs.bib	LeeCPC2015 指向 Computer Physics Communications 194, 1-9, DOI 10.1016/j.cpc.2015.04.004	这是本书应优先采用的主引用
OpenAlex	标记 CPC 文章为 green OA, OA URL 指向 OSTI 1246488	说明理论上存在 accepted-manuscript 线索, 但还不等于项目已拿到 PDF
Crossref	记录 Elsevier TDM 链接, 并显示 accepted-manuscript license 在 2016-05-21 后生效	可作为授权状态线索, 但不是可直接转换的全文
OSTI 页面/API	https://www.osti.gov/pages/bibliography/1246488 和 API 返回书目信息、Publisher's Accepted Manuscript 类型、citation links	前246488暴露 PDF/full text 文件
OSTI purl 猜测	servlets/purl/1246488 及 .pdf 变体返回 HTTP 404	不能把 OSTI 记录当成已下载全文
ScienceDirect PDF	本机 curl PDF 端点返回 HTTP 403	需要机构访问或浏览器授权会话
Elsevier API PDF	无授权请求返回 406/最小元数据	不能绕过授权取全文
AIP 会议版 PDF	AIP PDF 端点本机仍返回 HTTP 403	只能作为官方元数据线索, 暂不能作为 MinerU 输入

因此, v0.23 对 LeeCPC2015 的状态定义为: 书目已核实、开放获取线索已定位、全文尚未取得。项目 manifest 中对应记录使用 metadata_only, 而不是 downloaded。这条边界要保持到后续真的有 PDF、MinerU Markdown、图片和逐段中文讲解为止。

在全文未到位前，本书仍可以先做“公式映射准备”。映射准备不是替代论文阅读，而是定义拿到全文后要验证什么：

后续论文段落应核查的主题	当前 WarpX 源码锚点	需要回答的问题
高阶 finite-difference Maxwell solver 与 PML 反射率	EvolveBPML.cpp、EvolveEPML.cpp、PML.cpp	论文中的 FDTD/高阶 FD 结论是否对应 WarpX Yee/CKC/nodal PML regression
pseudo-spectral Maxwell solver 的 PML 处理 PML 厚度、profile、入射角和反射率	PML::PushPSATD()、 PsatdAlgorithmPml.cpp Examples/Tests/pml/analysis_pml_*.cpp	论文是否给出和当前 C1-C25 同构的 split-field spectral propagator WarpX regression 的 final reflectivity gate 是否覆盖论文关心的参数空间
Galilean / moving-frame 情况	PsatdAlgorithmPml.cpp 的 T2=exp(i w_c dt)	论文若未覆盖 Galilean PML，本书不能把 Galilean 分支归因给 LeeCPC2015
divergence cleaning 进入 PML spectral update	C23-C25 与 F/G split components	论文是否讨论 cleaning；若没有，应明确这是 WarpX 实现扩展而非论文公式

这让 v0.23 的下一步变得很具体。拿到 PDF 后，不能只做“论文已下载”的资产提交，而要核对四件事：第一，论文是否真的推导 pseudo-spectral PML 更新式；第二，论文使用的符号和当前 WarpX C1-C25 是否同构；第三，论文效率/反射率实验是否能解释 WarpX analysis_pml_psatd.py 的 $< 1e-6$ gate；第四，论文没有覆盖的 WarpX 分支，例如 Galilean 相位和 divergence-cleaning F/G，要在正文里单独标成实现侧扩展。

7.5.6 v0.24 PsatdAlgorithmPml.cpp 的 C1-C25 系数图谱

v0.24 在全文仍不可下载的条件下继续推进源码侧闭环。新增的 notes/code-reading/fieldsolver/16-psatd-pml-coefficient-atlas.md 把 PsatdAlgorithmPml.cpp 中的系数拆成三组，并把每组和 regression 证据对应起来。这个笔记的用途很明确：它不是 LeeCPC2015 的替代品，而是后续拿到全文后检查论文公式与 WarpX 实现是否同构的源码底稿。

所有 C1-C25 都只在 $knorm \neq 0$ 的谱模式上定义。共享谱量是：

$$k^2 = k_x^2 + k_y^2 + k_z^2, \quad C = \cos(c|\mathbf{k}|\Delta t), \quad S_{ck} = \frac{\sin(c|\mathbf{k}|\Delta t)}{c|\mathbf{k}|}, \quad K^{-2} = \frac{1}{k^2}.$$

若 psatd.v_galilean 非零，还会在 InitializeSpectralCoefficients() 中计算

$$T_2 = \exp(i\mathbf{k}_c \cdot \mathbf{v}_{gal}\Delta t),$$

并在每个 split component 更新外层整体相乘。这里的 T2 是 Galilean 相位，不是 PML 阻尼因子；PML 的 sigma profile 仍来自 PML.cpp / SigmaBox。

第一组是 C1-C9，负责投影和 split-component 几何耦合：

系数	源码表达式	作用
C1	(kx2*C + ky2 + kz2)/k^2	x 加权的对角投影
C2	(kx2 + ky2*C + kz2)/k^2	y 加权的对角投影
C3	(kx2 + ky2 + kz2*C)/k^2	z 加权的对角投影
C4-C6	k_i^2(C-1)/k^2	同轴修正项
C7	ky*kz(1-C)/k^2	yz 交叉投影
C8	kx*kz(1-C)/k^2	xz 交叉投影
C9	kx*ky(1-C)/k^2	xy 交叉投影

第二组是 C10-C22，只在 !dive_cleaning && !divb_cleaning 分支中存在。它们把 dt-S_ck 或 S_ck 与电磁交叉耦合项组合起来：

系数段	结构	用途
C10	i c^2 kx ky kz (dt-S_ck)/k^2	三方向对称交叉项，反复出现在 split E/B 更新中

系数段	结构	用途
C11-C16	$i c^2$ 乘一个平方波数、一个线性波数和 $dt-S_{ck}$	方向特定的交叉项；磁场更新使用相应 $/c^2$ 版本
C17-C22	$i c^2 k_i [k_j^2 dt + (k_l^2 + k_i^2) S_{ck}] / k^2$	更强的方向耦合项，连接某一 split 电场/磁场与横向整体场

第三组是 C23-C25，只在 `dive_cleaning` && `divb_cleaning` 分支中存在：

系数	源码表达式	用途
C23	$i c^2 k_x S_{ck}$	x 方向 F/G 与 E/B split components 的 coupling
C24	$i c^2 k_y S_{ck}$	y 方向 F/G 与 E/B split components 的 coupling
C25	$i c^2 k_z S_{ck}$	z 方向 F/G 与 E/B split components 的 coupling

这三组系数和 regression 的关系也要分清：

regression	命中的系数组	analysis 证据	当前不能推出的结论
<code>test_2d_pml_x_psatd</code>	no-cleaning 分支, C1-C22	<code>analysis_pml_psatd.py</code> 检查第 50 步能量 oracle 和最终反射率 $< 1e-6$	不能证明每个系数逐项正确，只能证明组合后的低反射率结果
<code>test_2d_pml_x_galilean</code>	cleaning 分支, T2、C1-C9、C23-C25	同一 analysis，切换 Galilean energy oracle，最终反射率 $< 1e-6$	不能单独证明 F/G 残差，因为脚本只读 E/B 能量
<code>test_3d_pml_psatd_dive_dive_cleaning</code>	3D cleaning 分支	CMake 中 <code>analysis=OFF</code> ，只有默认 checksum	不能写成 3D PML cleaning 的强物理验证
<code>test_rz_pml_psatd</code>	RZ 专用 <code>PsatdAlgorithmPmlRZ.cpp</code> ，不是 Cartesian C1-C25	$\max(E_r , E_z) < 2.0$	不能拿来证明 Cartesian <code>PsatdAlgorithmPml.cpp</code>

这样一来，即使 LeeCPC2015 全文暂时不可得，第 7 章也已经具备一个可审校的源码判据：以后论文到位时，若论文给出 pseudo-spectral PML 更新公式，就逐项对照 C1-C25；若论文只给反射率和效率曲线，就把论文作为性能/精度证据，把 C1-C25 保持为 WarpX 实现公式，二者不能混为一谈。

7.5.7 v0.25 LeeCPC2015 的论文-源码公式核对清单

v0.25 继续尝试 Lee/Vay PML 论文的全文路径。CPC DOI、OSTI、eScholarship、ScienceDirect 和 AIP 直接 PDF 端点仍没有给出本机可下载的授权 PDF；AIP pubs.aip.org 的直接 PDF URL 在浏览器 UA 下也返回 Cloudflare HTTP 403。因此，本版仍不执行 MinerU，也不写逐段论文讲解。新增的价值在于把后续全文到位后的核对任务从“读论文”拆成具体、可验收的条目：

核对层	论文到位后要找的内容	当前 WarpX/PIC-tutor 锚点	当前可写结论
PML profile	σ 剖面、PML 厚度、入射波和反射率测量方式	<code>PML.cpp</code> 、 <code>SigmaBox.*</code> 、 <code>inputs_base_2d</code>	只能说明 WarpX PML profile 的实现位置，不能假定论文参数完全同构
高阶 FD PML	finite-difference 阶数、stencil 和 PML 离散处理	<code>EvolveBPML.cpp</code> 、 <code>EvolveEPML.cpp</code> 、Yee/CKC regression	可写 WarpX 有 FDTD/CKC PML regression，论文参数空间仍待全文确认
PSATD PML 更新	是否推导 split-field spectral propagator	<code>PML::PushPSATD()</code> 、 <code>PsatdAlgorithmPml.cpp</code> :1956-1960	若论文不给等价更新式，C1-C25 不能继续标为 WarpX 实现公式

核对层	论文到位后要找的内容	当前 WarpX/PIC-tutor 锚点	当前可写结论
C1-C9	longitudinal/transverse 投影或矩阵块	16-psatd-pml-coefficient	当前源码不明, 等待论文符号对照
C10-C22	no-cleaning split E/B cross coupling	plain PSATD PML 分支和 test_2d_pml_x_psatd	regression 证明组合低反射率, 不证明逐系数
C23-C25	divergence-cleaning F/G 是否进入论文公式	cleaning 分支和 test_2d_pml_x_galilean	若论文未覆盖, 应写为 WarpX 实现扩展
Galilean T2	moving/Galilean frame	T2=exp(i k_c dot v_gal dt)	若论文未覆盖, 不能归因给 LeeCPC2015
RZ PML	PML 是否在论文中出现 是否讨论 cylindrical geometry	PsatdAlgorithmPmlRZ.cpp、test_rz_pml_psatd	RZ regression 不能证明 Cartesian C1-C25

这个清单已经落到论文专属目录:

references/08_boundaries_pml_geometry/2015_LeeCPC2015_Efficiency_of_the_PML_with_high-order_FD_and_pseudo-
式核对清单.md

写作上要遵守一个很硬的边界: 论文全文没到位时, 最多只能写“WarpX 当前源码如何实现 PSATD PML, 以及 regression 如何间接验证组合效果”, 不能写“LeeCPC2015 推导了 C1-C25”。等 PDF 到位后, 真正的完成标准不是把 PDF 放进目录, 而是逐项填完上面的表: 论文符号、源码变量、测试证据、正文结论四者必须同列出现。

7.5.8 v0.40 LeeCPC2015 accepted-manuscript 资产与源码映射

截至 2026-07-11, eScholarship 49m2k3vj 已提供可下载的 7 页 accepted/submitted manuscript。本项目已将 PDF、MinerU Markdown、13 张抽取图片和中文讲解放入该论文专属目录, 论文内容因此从 metadata_only 推进到 **accepted-manuscript-backed + source-grounded**。当前本地全文明确覆盖四条可回写主线:

论文内容	本地可引用的结论	WarpX 对应层	当前限制
PML split-field medium	电/磁 conductivity 按 $\sigma/\epsilon_0 = \sigma/\mu_0$ 匹配, 连续极限保持真空阻抗	PML.cpp、 Evolve*PML.cpp 的 split-field/damping	论文的二维 TE 代表式不能直接覆盖 WarpX 所有几何
高阶 FDTD 到 PSTD 极限	PML 反射率随波长、入射角和 stencil 阶数变化; PSTD 可视 为高阶 FDTD 的无限阶极限	FDTD PML 与 PML::PushPSATD() 的 solver 分层	论文讨论的是 PSTD/高阶 solver, 不等于当前 C1-C25 每一 项
staggered-grid PSTD	Fourier 导数外的 $\exp(-ik \Delta/2)$ 因子负责 staggered-grid shift	PML::PushPSATD() 和谱空间 split-field 更新	需要继续按当前 WarpX 字段布 局核对相位约定
reflection recurrence	用相邻切片的 r_j/t_j 递推整层 PML 反射系数, 数值结果与解析 积分比较	analysis_pml_psatd.py 的能量/反射率消费	regression 证明组合反射率, 不 证明 C1-C25 逐系数正确

这一步允许第 7 章把 LeeCPC2015 从“只有获取审计”升级为“有全文支撑的第一轮 PML 理论解释”, 但仍保留版本边界: 本地 PDF 是 eScholarship accepted/submitted manuscript, 不是 publisher-formatted CPC PDF; C1-C25、Galilean T2、PML divergence-cleaning F/G 和 RZ PML 仍需分别标成 WarpX 实现侧或专用扩展, 不能直接归因给论文原始公式。

7.5.9 v0.40 PSATD-PML 的 Cartesian/RZ 并列复现

为了把上面的源码和 regression 地图推进到可复查的运行证据, 项目又对官方 pml 输入做了单进程复现。Cartesian inputs_test_2d_pml_x_psatd 使用 warpx.2d 运行, 官方 analysis_pml_psatd.py 与独立脚本 scripts/analyze_pml_psatd_contract.py 都通过: diag1000050 初始场能量相对参考值误差为 $1.45e-15$, 末态总能量与初始能量之比, 即反射率, 为 $9.4704e-7 < 1e-6$ 。这条结果支持“主域 PSATD 与普通 Cartesian PML 组合满足低反射率合同”, 但不证明 C1-C25 每个系数逐项正确。

同一份独立报告还读取官方 inputs_test_rz_pml_psatd 的 RZ 末态 plotfile。历史 1-rank 项目级结果为 $\max|Er|=1.4793$ 、 $\max|Ez|=0.4841$; 本轮进一步按官方 CMake 的 2-rank 注册复现, analysis_pml_psatd_rz.py 与独立 scripts/analyze_rz_pml_psatd_con

均通过,得到 $\max|E_r|=1.0316$ 、 $\max|E_z|=0.5695$,合并残余场最大值 $1.0316 < 2$ 。这条证据测的是径向脉冲离域后的 PML_RZ 残余场衰减,不应与 Cartesian 反射率数值直接比较;2-rank 报告归档于 `runs/stage-c-validation/pml_rz_psatd_mpi2/contract.{json,md}`。

7.6 Embedded boundary 先是几何初始化和辅助标记系统

前面讨论的 PML、PEC、PMC、Silver-Mueller 都作用在计算域外边界上,而 embedded boundary 的第一步不是“给某个边界类型分派更新公式”,而是先把几何对象嵌入到 AMReX cut-cell 数据结构。

运行时总开关在 `EmbeddedBoundary/Enabled.cpp`:

```
std::string eb_implicit_function;
bool eb_enabled = pp_warpx.query("eb_implicit_function", eb_implicit_function);
```

```
std::string eb_stl;
eb_enabled |= pp_eb2.query("geom_type", eb_stl);
```

源码位置: `../warpx/Source/EmbeddedBoundary/Enabled.cpp:25-30`。

这说明 EB 的启用条件不是单一布尔量,而是:

1. 编译时必须有 `AMREX_USE_EB`;
2. 运行时必须提供 `warpx.eb_implicit_function`, 或者走 `eb2.geom_type / eb2.stl_file` 的 AMReX EB2 参数路径。

几何真正初始化在 `WarpX::InitEB()`:

```
if (!impf.empty()) {
    auto eb_if_parser = utils::parser::makeParser(impf, {"x", "y", "z"});
    ParserIF const pif(eb_if_parser.compile<3>());
    auto gshop = amrex::EB2::makeShop(pif, eb_if_parser);
    amrex::EB2::Build(gshop, Geom(maxLevel()), maxLevel(), maxLevel()+20);
} else {
    amrex::ParmParse pp_eb2("eb2");
    if (!pp_eb2.contains("geom_type")) {
        std::string const geom_type = "all_regular";
        pp_eb2.add("geom_type", geom_type);
    }
    amrex::EB2::Build(Geom(maxLevel()), maxLevel(), maxLevel()+20);
}
```

源码位置: `../warpx/Source/WarpXInitEB.cpp:78-95`。

这里的结构很清楚:

- 若给出 `warpx.eb_implicit_function`, `WarpX` 自己负责把解析函数包装成 `ParserIF` 再交给 AMReX `EB2::GeometryShop`;
- 若没有隐式函数,就沿用 `eb2.*` 参数,由 AMReX `EB2` 直接构几何;
- `maxLevel()+20` 是为了让 `EB2` 尽量向粗层 coarsen, 服务 multigrid, 而不是某个物理精度参数。

EB 几何建立后, `WarpX` 还会把 signed distance 场填进 field registry:

```
const amrex::EB2::IndexSpace& eb_is = amrex::EB2::IndexSpace::top();
for (int lev=0; lev<=maxLevel(); lev++) {
    const amrex::EB2::Level& eb_level = eb_is.getLevel(Geom(lev));
    auto const eb_fact = fieldEBFactory(lev);
    amrex::FillSignedDistance(*m_fields.get(FieldType::distance_to_eb, lev), eb_level, eb_fact, 1);
}
```

源码位置: `../warpx/Source/WarpXInitEB.cpp:106-112`。

所以 `distance_to_eb` 不是附加诊断,而是后续 scraping、近壁沉积和 cut-cell 处理可以直接复用的几何辅助场。

更关键的是, `EmbeddedBoundaryInit.*` 在初始化阶段就为后续 solver / deposition 准备了几类辅助标记:

- `MarkReducedShapeCells`: 若高阶 shape 可能覆盖到部分或完全 cut cell, 就强制降成一阶沉积;
- `MarkUpdateCellsStairCase`: 对非 ECT solver, 只要 field 自由度邻接到非 regular cell, 就停止更新;

- MarkUpdateECellsECT / MarkUpdateBCellsECT: ECT solver 不看 stair-case 邻域, 而是直接看对应 edge length 或 face area 是否为零。

例如 MarkReducedShapeCells() 的混合区域逻辑是:

```
if ( !flag(i_cell, j_cell, k_cell).isRegular() ) {
    reduce_shape = 1;
}
```

源码位置: ../warpx/Source/EmbeddedBoundary/EmbeddedBoundaryInit.cpp:98-103。

而 MarkUpdateCellsStairCase() 的核心是:

```
if ( !flag(i_cell, j_cell, k_cell).isRegular() ) {
    eb_update_flag = 0;
}
```

源码位置: ../warpx/Source/EmbeddedBoundary/EmbeddedBoundaryInit.cpp:206-210。

这说明 EB 在 WarpX 里的第一层实现不是“直接改 Maxwell 更新式”, 而是先把 cut-cell 几何转换成:

1. 粒子沉积是否降阶;
2. 场自由度是否允许更新;
3. edge/face 几何量是否还存在。

当前这一层已经单独整理在 notes/code-reading/embedded-boundary/00-eb-initialization.md, 而 WarpXFaceExtensions.cpp 的 face extension 稳定性标志、intrusion 判据和 cut-face 修正则继续整理在 notes/code-reading/embedded-boundary/01-face-extensions.md。

7.7 Embedded boundary 的 face extension: 把不稳定 cut face 变成 enlarged face

对 ECT solver 来说, 仅仅知道某个 face 被 cut 还不够, 因为部分 cut face 的有效面积可能小到破坏稳定性。WarpX 的处理不是简单禁用这些 face, 而是尝试把它们扩成 enlarged face。

初始化侧先在 MarkExtensionCells() 中定义两个标志:

```
flag_ext_face_data(i, j, k) = int(S(i, j, k) < S_stab && S(i, j, k) > 0);
if(flag_ext_face_data(i, j, k)){
    flag_info_face_data(i, j, k) = 0;
}
if(int(S(i, j, k) > 0 && !flag_ext_face_data(i, j, k))) {
    flag_info_face_data(i, j, k) = 1;
}
```

源码位置: ../warpx/Source/EmbeddedBoundary/EmbeddedBoundaryInit.cpp:426-433。

其中:

- flag_ext_face = 1 表示这个 cut face 本身不稳定, 必须扩展;
- flag_info_face = 0 表示它当前是借方面;
- flag_info_face = 1 表示它是可出借面积的稳定面;
- 在 extension 过程中, 被别人侵入的 lender 会再改成 2。

真正的 extension 在 WarpX::ComputeFaceExtensions() 里按三步进行:

```
::init_borrowing(m_borrowing[maxLevel()], Bfield);
ComputeOneWayExtensions();
ComputeEightWaysExtensions();
::shrink_borrowing(m_borrowing[maxLevel()], Bfield);
```

源码位置: ../warpx/Source/EmbeddedBoundary/WarpXFaceExtensions.cpp:514-529。

第一步是 one-way extension, 只允许从一个正交邻居一次性借满所需面积 S_ext。如果存在这样的 lender, 就直接把 lender 的 S_mod 扣掉 S_ext, 把 borrower 的 S_mod 增加 S_ext, 并把 lender 标成 2。见 ../warpx/Source/EmbeddedBoundary/WarpXFaceExtensions.cpp:653

第二步是 eight-ways extension。若单邻居借不满, 就在 3x3 邻域内筛选所有可用 lender, 按原始 face 面积比例分摊:

```
const amrex::Real patch = S_ext * ::GetNeigh(S, i, j, k, i_n, j_n, idim) / denom;
```

源码位置: `../warpx/Source/EmbeddedBoundary/WarpXFaceExtensions.cpp:830-831`。

但 WarpX 还会反复剔除那些按该比例借出后会把自己 S_{mod} 减成非正的邻居, 因此 eight-ways 不是机械加权, 而是“保证性的面积分摊”。见 `../warpx/Source/EmbeddedBoundary/WarpXFaceExtensions.cpp:820-846`。

如果 one-way 和 eight-ways 都失败, 就进入 BCK fallback:

```
if (flag_ext_face_max_lev_idim(i, j, k)) {
    S(i, j, k) = ::ComputeSStab<idim>(i, j, k, lx, ly, lz, dx, dy, dz);
    flag_info_face_max_lev_idim(i, j, k) = -1;
}
```

源码位置: `../warpx/Source/EmbeddedBoundary/WarpXFaceExtensions.cpp:196-200`。

源码注释说明这是 Benkler-Chavannes-Kuster correction, 精度低于常规 ECT extension, 但仍优于纯 staircasing。

extension 的借用关系不会散落在若干数组里, 而是统一压进 FaceInfoBox:

```
struct FaceInfoBox {
    amrex::Gpu::DeviceVector<Neighbours> neigh_faces;
    amrex::Gpu::DeviceVector<amrex::Real> area;
    amrex::Gpu::DeviceVector<int> inds;
    amrex::BaseFab<int> size;
    amrex::BaseFab<int*> inds_pointer;
```

源码位置: `../warpx/Source/EmbeddedBoundary/WarpXFaceInfoBox.H:15-28`。

它记录的是“这个 enlarged face 向哪些邻居借了多少面积”。后续 ECT B 更新时, `WarpX::EvolveB()` 把 `m_flag_info_face` 和 `m_borrowing` 直接送进 solver:

```
m_fdt_d_solver_fp[lev]->EvolveB( m_fields,
                                   lev,
                                   patch_type,
                                   m_flag_info_face[lev], m_borrowing[lev], a_dt );
```

源码位置: `../warpx/Source/FieldSolver/WarpXPushFieldsEM.cpp:971-975`。

在 `FiniteDifferenceSolver::EvolveBCartesianECT()` 中, 不稳定 face 会先聚合 enlarged face 的有效电荷:

```
Venl_dim(i, j, k) = Rho(i, j, k) * S(i, j, k);
...
Venl_dim(i, j, k) += Rho(ip, jp, kp) * borrowing_dim_area[ind];
...
rho_enl = Venl_dim(i, j, k) / S_mod(i, j, k);
```

源码位置: `../warpx/Source/FieldSolver/FiniteDifferenceSolver/EvolveB.cpp:307-339`。

因此, WarpX 的 face extension 不是“把几何修漂亮一点”, 而是直接决定 ECT solver 如何构造 enlarged face 的有效 rho 并推进 B。

7.8 Embedded boundary 的粒子侧: signed-distance 判定、默认吸收与 scraped buffer

对粒子来说, embedded boundary 的关键并不是 face extension, 而是“什么时候认定粒子已经撞进了 EB, 以及撞进去之后怎样处理”。

`ParticleScraper.H` 的核心逻辑非常直接:

```
ablastr::particles::compute_weights<amrex::IndexType::NODE>(
    xp, yp, zp, plo, dxi, i, j, k, W);
amrex::Real const phi_value = ablastr::particles::interp_field_nodal(i, j, k, W, phi);

if (phi_value < 0.0)
{
    ...
    amrex::RealVect normal = DistanceToEB::interp_normal(i, j, k, W, ic, jc, kc, Wc, phi, dxi);
    DistanceToEB::normalize(normal);
    ...
}
```

```
f(ptd, ip, pos, normal, engine);
}
```

源码位置: `../warpx/Source/EmbeddedBoundary/ParticleScrapper.H:181-208`。

这说明 WarpX 的粒子撞墙检测并不是拿粒子坐标去直接查解析几何, 而是:

1. 在 `nodal distance_to_eb` 上插值;
2. 若 `phi_value < 0`, 认定粒子进入了需要刮擦的 EB 区域;
3. 再用 `DistanceToEB::interp_normal()` 从 `signed-distance` 的离散梯度重建法向。

`DistanceToEB.H` 本身也只做这件事。它的 `interp_normal()` 对 `phi` 做差分加权, `normalize()` 再把法向单位化。也就是说, 粒子侧法向来自 `signed-distance` 场, 而不是 STL 面片直接投影。

默认处理器在 `ParticleBoundaryProcess.H` 里只有两个最小版本:

```
struct NoOp { ... };

struct Absorb {
    ...
    amrex::ParticleIDWrapper{ptd.m_idcpu[i]}.make_invalid();
}
```

源码位置: `../warpx/Source/EmbeddedBoundary/ParticleBoundaryProcess.H:12-33`。

因此, 当前主源码链上的默认 EB 粒子边界语义其实很朴素: 不是复杂反射模型, 而是“把撞进 EB 的粒子标成 `invalid`”。真正删除发生在后续 `deleteInvalidParticles()` 或 `Redistribute()`, 而不是 `Absorb()` 本身立即擦除数据。

这一逻辑会在多处触发:

- `WarpXParticleContainer` 在 `Redistribute()` 后立刻做一轮 EB 吸收;
- `MultiParticleContainer::ScrapeParticlesAtEB()` 可以对所有 species 统一刮擦;
- `AddParticles.cpp` 在新增粒子或 flux 注入后, 也会先对临时容器做 `scrapeParticlesAtEB(..., Absorb())`。

因此 WarpX 的策略是“新粒子一旦进入容器, 就尽快排除已经落在 EB 内部的非法粒子”。

如果用户想把这些 scraped 粒子保留下来做诊断, 就可以设置:

- `<species_name>.save_particles_at_eb = 1`

官方文档说明这会撞到 EB 的粒子复制到 scraped particle buffer, 可供 `BoundaryScrapingDiagnostic` 或 Python 接口使用。见 `../warpx/Docs/source/usage/parameters.rst:1890-1924`。

更重要的是, `ParticleBoundaryBuffer.cpp` 在把粒子放进 EB buffer 时, 不是简单复制当前状态, 而是用二分法沿粒子轨迹回溯到 `phi = 0` 的真实交点:

```
amrex::Real const dt_fraction = amrex::bisection( 0.0, 1.0,
    [=] (amrex::Real dt_frac) {
        ...
        UpdatePosition(x_temp, y_temp, z_temp, ux, uy, uz, -dt_frac*dt, mass);
        ...
        amrex::Real const phi_value = ablastr::particles::interp_field_nodal(i, j, k, W, phiarr);
        return phi_value;
    });
```

源码位置: `../warpx/Source/Particles/ParticleBoundaryBuffer.cpp:92-104`。

随后它还会记录 scraping 发生的 step、时间偏移、真实时间和表面法向。也就是说, scraped particle buffer 保存的不是“死前最后一帧粒子”, 而是“与 EB 表面交点处的粒子诊断样本”。

因此, embedded boundary 的粒子侧主链可以概括为:

1. `distance_to_eb` 判定粒子是否进入 EB;
2. `DistanceToEB` 重建边界法向;
3. 默认 `Absorb` 仅把粒子标成 `invalid`;
4. 后续删除逻辑真正清除粒子;

5. 若启用 `save_particles_at_eb`, 则 `ParticleBoundaryBuffer` 回溯到 $\phi=0$ 交点并记录 `scraped` 事件。

`Examples/Tests/embedded_circle/` 给这条链提供了一个很实用但证据层级较弱的本地入口。它不是 `electrostatic_sphere_eb` 那类解析 ϕ/Er 强基准, 也不是 `point_of_contact_eb` 那类直接检查交点几何的强 `analysis`, 而是一个 2D circular EB workflow baseline:

- `eb_implicit_function` 定义圆形导体
- `eb_potential = -10` 进入电静求解
- 电子/氦离子都先 `initialize_self_fields = 1`
- 双物种 `background_mcc` 持续运行
- 两个 species 都打开 `save_particles_at_eb = 1`
- `diag3` 用 `BoundaryScraping` openPMD 写出 `scraped` 粒子

当前 `CMakeLists.txt` 中这条 `test` 没有独立 `analysis.py`, 只保留 `checksum helper`。因此它在本章里更适合承担:

- EB geometry + electrostatic + MCC + `BoundaryScraping` 的联合 workflow 基线

而不是承担解析电势或表面碰撞物理的强验证。

`domain boundary buffer` 的收集时机也要一起记住。`WarpXEvolve.cpp` 里先执行 `mypc->ApplyBoundaryConditions()`, 随后立刻调用 `m_particle_boundary_buffer->gatherParticlesFromDomainBoundaries(*mypc, cur_time)`, 最后才在 EB 路径后统一 `deleteInvalidParticles()`。因此 `buffer` 记录依赖的是“粒子还在容器里、但已经越界或即将失效”的中间态, 而不是从被删除后的粒子列表回溯出来。对 `save_particles_at_xlo/.../eb`、`BoundaryScrapingDiagnostic` 和 Python `buffer` 接口来说, 这个顺序决定了 `scraped` 数据为什么能同时保留 `step`、时间偏移和边界法向。

周期边界有一个关键规则: 如果某个方向的场边界是 `periodic`, 该方向粒子边界也必须 `periodic`。这个规则在过去本机验证中已经用本地运行、源码和文档确认过; 非周期边界则不要求 `field` 和 `particle` 边界字面一致。正式章节需要把对应源码错误消息和参数验证写入本章。

AMR 的目标是把计算资源集中在物理上需要高分辨率的区域。但 PIC 的 AMR 比流体 AMR 更难, 因为宏粒子穿过 `refinement interface` 时会看到不同网格上的插值场, 容易产生 `self-force`、短波反射和电荷/电流不一致。`WarpX` 官方 `Docs/source/theory/amr.rst` 特别强调两个问题: `mesh refinement interface` 附近的 `spurious self-force`, 以及电磁波在粗细网格界面处的反射和放大。

在真正进入 `coarse-fine interface` 之前, 还必须先把并行层的 `guard-cell` 通信模型看清。`WarpX` 不是“所有字段都统一 `FillBoundary` 一次”这么简单, 而是把通信语义分成两类:

- E/B/F/G/Aux 这类场变量: `guard cells` 用 `FillBoundary` 风格的复制/同步;
- J/rho 这类源项: 重叠区域必须做 `SumBoundary` / `ParallelAdd` 风格的累加, 因为粒子靠近 `box` 边缘时, 同一 (i,j,k) 可能同时被沉积到一个 `box` 的 `guard` 区和另一个 `box` 的 `valid` 区。

这一层的统一预算由 `Parallelization/GuardCellManager.*` 管理。它先区分:

- `ng_alloc *`: 每类 `MultiFab` 实际分配多少 `guard cells`;
- `ng_FieldSolver`、`ng_FieldGather`、`ng_UpdateAux`、`ng_MovingWindow` 等: PIC 循环不同阶段真正交换多少。

这些配额不是拍脑袋常数, 而是共同受以下因素控制:

- 粒子 `shape` 阶数与 `subcycling`;
- `moving window` 位移;
- Galilean / `comoving` 修正;
- NCI / `bilinear filter`;
- FDTD stencil 或 PSATD 局部 FFT stencil。

例如 `SyncCurrent()` 的源码注释就明确说明, 多层 AMR 下不能简单把 `finer coarse-patch current` 直接 `ParallelAdd` 到当前 level 的 `fine patch`, 因为 `nodal overlap` 会双计数; `WarpX` 为此引入临时 `fine_lev_cp` 和 `OwnerMask` 去重。也就是说, `coarse-fine current` 同步本身就已经是 AMR 物理一致性的一部分, 而不是纯粹 MPI 细节。

当前这一层已经单独整理在 `notes/code-reading/parallelization/00-guard-cell-model.md`。

继续往下看 `Parallelization/WarpXComm.cpp`, 会发现 `WarpX` 对 `current / rho` 的 `coarse-fine` 同步并不是简单的“restrict 一下再加回去”。`SyncCurrent()` 的大段注释明确说明:

- `finest level` 先把 `fine-patch current restriction` 到同层 `coarse patch`;
- 若有 `current buffer`, 则 `coarse-patch current` 先并入 `buffer`, 再把 `buffer` 当作更粗层的通信源;
- 更粗层接收 `finer` 数据时, 先写到临时 `fine_lev_cp`;
- 由于 `nodal` 点可能在多个 `box` 中重叠, 不能直接加回 `J_fp`, 而要借助 `OwnerMask` 只让 `owner box` 接管该点数据。

因此, WarpX 的 coarse-fine source 同步真实更接近:

restriction -> optional buffer merge -> temporary receive -> owner-mask de-dup -> same-level SumBoundary

而不是单一的 restriction/prolongation 二步法。

rho 路径在 SyncRho() 中基本平行, 只是 bilinear filter 与 SumBoundary 被折叠成 ApplyFilterandSumBoundaryRho()。这意味着 J 和 rho 虽然共享 AMR 同步框架, 但在 filter 实现上仍有细微差异。

这一层已经单独整理在 notes/code-reading/parallelization/01-current-rho-sync-paths.md, 继续顺着 WarpXRegrid.cpp 往下读时, 问题就不再是“数据如何同步”, 而会转成“DistributionMapping 改变后, fields、particles、EB、boundary buffer 和 diagnostics 如何整体重建”。

WarpXRegrid.cpp 的顶层入口是:

```
void
WarpX::CheckLoadBalance (int step)
{
    if (step > 0 && load_balance_intervals.contains(step+1))
    {
        LoadBalance();
        ResetCosts();
    }
    if (!costs.empty())
    {
        RescaleCosts(step);
    }
}
```

源码位置: ../warpX/Source/Parallelization/WarpXRegrid.cpp:49-63。

这说明 WarpX 的 load balance 不是“到点直接重分布”, 而是:

1. 周期性检查当前 step 是否命中 load_balance_intervals;
2. 若命中, 则执行 LoadBalance();
3. 之后清零 costs;
4. timer 模式下还会继续对 costs 做 running-average 式重标定。

LoadBalance() 内部先按 level 构造候选 DistributionMapping, 支持:

- SFC
- knapsack

然后比较 currentEfficiency 和 proposedEfficiency, 只有满足:

proposedEfficiency > load_balance_efficiency_ratio_threshold*currentEfficiency

时才真正采纳新映射。源码位置: ../warpX/Source/Parallelization/WarpXRegrid.cpp:82-143。

因此, WarpX 当前策略更接近“收益足够大才搬”, 而不是粗暴地按固定周期强制改 rank 图。

更关键的是 RemakeLevel() 的边界。它当前只支持:

- BoxArray 不变;
- DistributionMapping 改变。

因为函数内部直接写着:

```
if (ba == boxArray(lev)) {
    ...
} else
{
    WARPX_ABORT_WITH_MESSAGE("RemakeLevel: to be implemented");
}
```

源码位置: `../warpX/Source/Parallelization/WarpXRegrid.cpp:174-176,283-286`。

这意味着当前这里讨论的还不是“任意 AMR regrid”，而是“同一 patch 拓扑下的 rank 重映射”。

一旦某个 level 真的采纳新映射，WarpX 重建的远不只是粒子容器。RemakeLevel() 里会依次重做：

- `m_fields.remake_level(lev, dm)`: field registry 的 level 场数据；
- EB 相关的 `m_eb_reduce_particle_shape`、`m_eb_update_E/B`、ECT `m_borrowing`；
- `m_field_factory[lev]` 与 `InitializeEBGridData(lev)`；
- PSATD 的 fine/coarse spectral solver real-space 容器；
- `m_accelerator_lattice[lev]` $\rightarrow$ `InitElementFinder(...)`；
- `current_buffer_masks` / `gather_buffer_masks` 与 `BuildBufferMasks()`；
- `multi_diags` $\rightarrow$ `InitializeFieldFunctors(lev)`。

源码范围: `../warpX/Source/Parallelization/WarpXRegrid.cpp:178-290`。

随后若至少有一个 level 完成了 load balance, WarpX 才统一做：

```
mypc->Redistribute();
mypc->defineAllParticleFiles();
m_particle_boundary_buffer->redistribute();
reduced_diags->LoadBalance();
```

源码位置: `../warpX/Source/Parallelization/WarpXRegrid.cpp:149-159`。

因此，WarpX 的 load balance 不是“先搬粒子再说”，而是一次多子系统一致提交：

```
candidate DM -> efficiency check -> remake field/EB/solver/masks -> redistribute particles ->
redistribute boundary buffer -> refresh diagnostics
```

这一层现在已经单独整理在 `notes/code-reading/parallelization/02-regrid-and-load-balance.md`。

7.7.1 v0.11 guard-cell、通信和 regrid 的闭环

把上面的源码入口连起来看，AMR 章节最容易误读的一点是：guard_cells 不是一个单独的“安全余量”，而是一份同时约束内存分配、通信宽度和 regrid 重建的合同。GuardCellManager.H 明确把它拆成两组字段：ng_alloc_EB/J/Rho/F/G 表示各类 MultiFab 实际分配的 guard cells；ng_FieldSolver、ng_FieldGather、ng_UpdateAux、ng_MovingWindow、ng_afterPushPSATD 等则表示 PIC 循环不同阶段真正需要交换的宽度。也就是说，分配量回答“数组能容纳多少”，阶段量回答“这一次通信应该交换多少”。

GuardCellManager.cpp::Init() 的计算顺序体现了这个区别。它先从粒子 shape 阶数 nox 起步；若多层 AMR 且开启 subcycling, 则因为 fine level 粒子在重分布前会做两次 push, 先额外加一层；若使用 Galilean 或 comoving PSATD, 再额外加一层；随后把 E/B guard cells 取成偶数, 便于 coarse-fine 插值。J/rho 则先允许非偶数, 因为 J 只需要 fine-to-coarse 插值。之后, moving window 会用 refinement ratio 把所有相关方向的 guard cells 抬高；显式电磁路径还会按 $\text{ceil}(c \, dt / dx)$ 给 rho 和 J 追加粒子位移距离, JRhom/time-averaging 会把等效位移时间从 0.5 dt、dt 推到 2 dt。隐式路径则不再用光速 Courant 估算, 而是用 particle_max_grid_crossings 给 J/rho 定界。

这说明 guard-cell 预算本身已经含有物理时间尺度：rho 要覆盖一个 PIC iteration 前后两次 charge 状态, J 要覆盖半步或 JRhom 特例下的一整步/两步位移。后面的 filter 和 solver 又继续加约束：bilinear filter 会扩展 J/rho 分配；F/G 为 moving window、CKC 和 div cleaning 保留自己的 guard cells；PSATD 则按 FFT stencil 计算 ngFFT, 并允许用户用 psatd.nx_guard/ny_guard/nz_guard 覆盖。进入 PSATD 分支后, WarpX 为了避免临时 parallel copy, 会把 ng_alloc_EB/J/Rho/F/G 统一抬到所有字段所需 guard cells 的最大值。这样做牺牲了一些内存, 但换来后续 field solver、deposition 和 FFT stencil 在同一组 box 上可直接通信。

通信层 WarpXComm.cpp 消费的是这些阶段量, 而不是重新推导 guard cells。FillBoundaryE() 与 FillBoundaryB() 对每个 level 都先处理 fine patch; 若 lev > 0, 再处理 coarse patch。每个 patch 内部先让 PML 与 valid domain 做 Exchange(), 再填 PML 自己的 guard cells; RZ+FFT PML 还有独立的 pml_rz $\rightarrow$ FillBoundaryE/B()。随后才轮到 valid-domain field 的 FillBoundary。如果启用 single-precision communications, WarpX 会显式断言 `ng <= nGrowVect()`, 然后在 safe mode 下把 nghost 提升到整个已分配 nGrowVect(); 否则只交换调用者传入的阶段量 ng。如果不走 single-precision wrapper, 则根据 nodal_sync 选择 AMReX 的 FillBoundaryAndSync 或普通 FillBoundary。因此, m_safe_guard_cells 的含义不是“分配更多 guard cells”, 而是“每次通信都交换所有已分配 guard cells”; 真正的分配量早在 GuardCellManager 里定好了。

source 项的同步是另一套语义。SyncCurrent() 的源码注释把单层和多层路径分得很清楚：单层时, 先可选 filter, 再 SumBoundary, 使 box 边缘或周期边界上的重复沉积回到“好像只有一个 box”的结果；多层时, 从 finest level 往 coarse level 走, fine-patch J 先 coarsen 到同层 coarse patch, 若有 current buffer 就先并入 buffer, 再把 buffer 或 coarse patch 作为通信源送到更粗 level。更粗 level 接收时不能直接 ParallelAdd 到 J_fp, 因为 nodal 点可能属于多个 box; WarpX 先建临时 fine_level_cp, 再用 OwnerMask

只让 owner box 把该点加回主 J_fp. SyncRho() 复用同一套 owner-mask 思路, 只是把 rho 的 filter 和 SumBoundary 收在 ApplyFilterandSumBoundaryRho() 里。

因此, AMR 里的 field 通信和 source 同步可以按下面的最小图式理解:

flowchart LR

```

A["GuardCellManager Init"] --> B["ng_alloc_*: allocated ghost cells"]
A --> C["ng_FieldSolver / ng_FieldGather / ng_UpdateAux"]
C --> D["WarpXComm FillBoundary E/B/F/G/Aux"]
B --> E["nGrowVect checks and safe mode full exchange"]
D --> F["PML Exchange and valid-domain FillBoundary"]
A --> G["ng_depos_J / ng_depos_rho"]
G --> H["SyncCurrent / SyncRho"]
H --> I["coarsen, optional buffer merge, OwnerMask de-dup, SumBoundary"]
B --> J["WarpXRegrid RemakeLevel"]
J --> K["remake fields, EB factory, spectral solvers, masks, diagnostics"]

```

WarpXRegrid.cpp::RemakeLevel() 则是这份合同在 load balance 后的重建端。它在 BoxArray 不变、DistributionMapping 改变时才工作; 若 patch 拓扑真的改变, 当前代码直接 WARPX_ABORT_WITH_MESSAGE("RemakeLevel: to be implemented")。对已有 MultiFab, 局部 lambda 会读取旧对象的 nGrowVect(), 再用新 DistributionMapping 调 AllocInitMultiFab(), 这保证重建后字段仍保留原有 guard-cell 分配宽度。EB 路径进一步用 guard_cells.ng_FieldSolver.max() 创建 EBFabFactory 的几何 ghost 宽度, 并重新初始化 EB grid data。PSATD 路径会把 real-space box 转成 cell-centered, 按 getngEB() 增长后重建 fine/coarse spectral solver; buffer mask、accelerator lattice、diagnostic field functor 也都重新绑定到新的 ba/dm。

这一层给 v0.11 的成书结论是: WarpX 的 AMR/load-balance 不是“换一个 MPI rank 分布”这么窄。它必须同时保持三类一致性: guard-cell 分配和阶段交换的一致性, PML/valid-domain field communication 与 source SumBoundary 语义的一致性, 以及 regrid 后 field registry、EB factory、spectral solver、buffer mask、particle boundary buffer 和 diagnostics 的指针/布局一致性。后续讲 coarse-fine substitution 公式时, 只有先承认这三层 runtime 合同, 读者才不会把 AMR 误解成单纯的插值公式。

但 regrid 仍然没有回答 refinement interface 最关键的物理问题: 粒子在 coarse-fine 界面附近到底应该看哪套场, patch 内外源项的粗细解又如何合成。WarpX 在 Docs/source/theory/amr.rst 里给出的主公式是:

$$F(a) = F(r) + I[F(s) - F(c)].$$

这里:

- r 是 refined patch 上的 fine 解;
- c 是与 refined patch 对应的 coarse patch 解;
- s 是 parent grid 上对应区域的 coarse subset;
- a 是粒子最终 gather 的 auxiliary grid;
- I 是 coarse-to-fine 插值算子。

物理含义不是“粗细解做平均”, 而是:

1. parent coarse grid 的 F(s) 已经包含 patch 内外全部源的粗网格响应;
2. coarse patch F(c) 只包含 patch 内源在粗网格上的响应;
3. F(s)-F(c) 提供 patch 外源的 coarse 背景场;
4. 再把这部分插值到 fine 网格, 加到 patch 内源的 fine 解 F(r) 上;
5. 得到粒子真正应该看的 full solution F(a)。

这条理论公式在 WarpX.H 中直接被写成源码注释:

```

// This function does aux(lev) = fp(lev) + I(aux(lev-1)-cp(lev)).
// Caller must make sure fp and cp have ghost cells filled.
void UpdateAuxiliaryData ();

```

源码位置: ../warpx/Source/WarpX.H:649-652。

也就是说, WarpX 的 substitution strategy 不是某个隐藏在 solver 内部的后处理, 而是显式通过 UpdateAuxiliaryData*() 构造 E/Bfield_aux, 然后让粒子从 aux gather。

在 PhysicalParticleContainer::Evolve() 里, 粒子真正读取的是:

```
amrex::MultiFab & Ex = *fields.get(FieldType::Efield_aux, Direction{0}, lev);
amrex::MultiFab & Ey = *fields.get(FieldType::Efield_aux, Direction{1}, lev);
amrex::MultiFab & Ez = *fields.get(FieldType::Efield_aux, Direction{2}, lev);
amrex::MultiFab & Bx = *fields.get(FieldType::Bfield_aux, Direction{0}, lev);
amrex::MultiFab & By = *fields.get(FieldType::Bfield_aux, Direction{1}, lev);
amrex::MultiFab & Bz = *fields.get(FieldType::Bfield_aux, Direction{2}, lev);
```

源码位置: `../warpX/Source/Particles/PhysicalParticleContainer.cpp:479-484`。

这说明 `aux` 不是诊断容器, 而是粒子-场耦合的实际工作场。

`UpdateAuxiliaryDataSameType()` 中最核心的电磁路径可以概括成三步:

1. `ParallelCopy` 把 `aux(lev-1)` 拷到临时 `dE/dB`;
2. `MultiFab::Subtract` 减去 `cp(lev)`;
3. `warpX_interp(...)` 把残差插值到 fine grid, 并加到 `fp(lev)` 上得到 `aux(lev)`。

所以源码上的真实结构就是:

```
parent full solution -> subtract coarse patch -> interpolate residual -> add onto fine patch
```

而 `WarpXComm_K.H` 的 kernel 最终确实写成了:

```
arr_aux(j,k,l) = arr_fine(j,k,l) + res;
```

或在 momentum-conserving gather 的 nodal 版本里写成:

```
arr_aux(j,k,l) = tmp + (fine - coarse);
```

源码位置: `../warpX/Source/Parallelization/WarpXComm_K.H:83,238`。

这两种形式本质上是同一条 substitution 公式, 只是后一种还要先把 staggered fine/coarse 场转成 nodal 量。

理论文档里另一个同样重要的思想是 transition zone。WarpX 并没有要求 patch 边缘的粒子始终从 fine patch gather、也不要求它们始终在 fine patch 上 deposit。相反, WarpX.H 明确提供了两个控制量:

```
/// With mesh refinement, particles located inside a refinement patch, but within
/// #n_field_gather_buffer cells of the edge of the patch, will gather the fields
/// from the lower refinement level instead of the refinement patch itself
static int n_field_gather_buffer;
/// With mesh refinement, particles located inside a refinement patch, but within
/// #n_current_deposition_buffer cells of the edge of the patch, will deposit their charge
/// and current onto the lower refinement level instead of the refinement patch itself
static int n_current_deposition_buffer;
```

源码位置: `../warpX/Source/WarpX.H:340-347`。

这就是 `amr.rst` 里“extra transition cells are added around the effective refined area”的运行时入口。

对应的运行时数据结构不是抽象开关, 而是 `gather_buffer_masks` 与 `current_buffer_masks` 两组 `iMultiFab::BuildBufferMasksInBox()` 的逻辑要求一个 cell 的 `ngbuffer` 邻域必须完全留在 patch interior, mask 才为 1; 否则记为 0。因此:

- 1 表示足够远离 coarse-fine 边界的 interior;
- 0 表示进入 transition zone, 需要按 buffer 规则处理。

最后, `PhysicalParticleContainer::Evolve()` 并不是在 gather kernel 内即时判断这些 masks, 而是先调用 `PartitionParticlesInBuffers()` 重新排列粒子。该函数先按较大的 buffer 宽度做一次稳定分区, 再按较小的 buffer 宽度做第二次稳定分区, 从而分别得到:

- `nfine_gather`
- `nfine_current`

也就是说, WarpX 允许:

- 某些粒子仍在 fine patch 上 gather;
- 但已经切换到 lower level 上 deposit;

或者相反。这正是 coarse-fine transition zone 可控化的关键。

因此, WarpX 当前的 refinement interface 方案不是单一插值步骤, 而是三层配合:

1. UpdateAuxiliaryData*() 负责 substitution, 构造 full solution;
2. gather/current buffer masks 把界面附近显式定义为 transition zone;
3. PartitionParticlesInBuffers() 把粒子按 fine / lower-level gather-deposit 路径稳定分区。

这一层现在已经单独整理在 notes/code-reading/parallelization/03-amr-coarse-fine-substitution.md。

不过, 只把 substitution 公式和 transition zone 讲清, 还不够解释 WarpX 为什么能同时在 CPU 和 GPU 上复用同一套 coarse-fine 通信逻辑。真正支撑它的, 是 WarpXComm.cpp 与 WarpXComm_K.H 的分层执行模型。

WarpXComm_K.H 并不是一个“大而全”的通信模块, 而更像一组只负责单点算术的 device kernels。例如 coarse-fine same-type substitution 的最终形式就是:

```
arr_aux(j,k,l) = arr_fine(j,k,l) + res;
```

而 stag-to-nodal 的 substitution / gather 版本则写成:

```
arr_aux(j,k,l) = tmp + (fine - coarse);
```

源码位置: ../warpx/Source/Parallelization/WarpXComm_K.H:83,238。

这说明 WarpXComm_K.H 的角色不是遍历 MultiFab, 而是只定义“一个点上该怎么算”。外层的 box/tile 遍历、通信和 launch 语义全部留在 WarpXComm.cpp。

WarpXComm.cpp 在这方面的骨架非常统一。无论是 current centering、UpdateAuxiliaryData*(), 还是 coarse-fine owner-mask 去重, 基本都沿着同一模式组织:

1. MFIter(..., TilingIfNotGPU()) 遍历 FAB / tile;
2. 用 array(mfi) / const_array(mfi) 取出 Array4 轻量视图;
3. 再通过 amrex::ParallelFor(...) launch device lambda。

例如最早的 nodal-to-staggered current centering 就是:

```
for (MFIter mfi(dst, TilingIfNotGPU()); mfi.isValid(); ++mfi)
{
    const Box bx = mfi.growntilebox();
    auto const& src_arr = src.const_array(mfi);
    auto const& dst_arr = dst.array(mfi);
    amrex::ParallelFor(bx, [=] AMREX_GPU_DEVICE (int j, int k, int l) noexcept
    {
        warpx_interp(...);
    });
}
```

源码位置: ../warpx/Source/Parallelization/WarpXComm.cpp:95-112。

这里 TilingIfNotGPU() 的含义很直接:

- 在 CPU 路径上启用 tiling, 改善缓存局部性;
- 在 GPU 路径上通常不再额外 tile, 而是直接按 box launch。

外面再套一层:

```
#pragma omp parallel if (Gpu::notInLaunchRegion())
```

就形成了 WarpX/AMReX 非常典型的双栈执行壳:

- 非 GPU launch 区域: OpenMP 并行外层 tile 循环;
- GPU 路径: 由 ParallelFor 驱动 device kernel, 不再主机侧重复并行。

因此, WarpX 的 GPU portability 不是“把 kernel 改成 CUDA”这么简单, 而是靠

MFIter -> Array4 -> ParallelFor

这一层稳定抽象撑起来的。

通信层本身也有执行策略分叉。FillBoundaryE/B 中,若 do_single_precision_comms 为真,WarpX 走 ablastr::utils::communication::FillBoundaryE/B 的包装路径;否则直接走 AMReX 原生:

```
amrex::FillBoundaryAndSync_nowait(vec_mf, period);
amrex::FillBoundaryAndSync_finish(vec_mf);
```

或

```
amrex::FillBoundary_nowait(vec_mf, period);
amrex::FillBoundary_finish(vec_mf);
```

源码位置: ../warpX/Source/Parallelization/WarpXComm.cpp:811-834,893-916。

这里 nodal_sync 不是装饰参数,而是决定是否在 fill 之后继续对 nodal overlap 做同步;m_safe_guard_cells 则决定只交换调用者要求的 ng,还是直接把所有已分配 guard cells 都交换掉。

也就是说, WarpX 的并行层执行模型在工程上可以概括为:

1. WarpXComm_K.H 负责点值 kernel;
2. WarpXComm.cpp 负责 MFilter/Array4/ParallelFor 组织;
3. FillBoundary / ParallelCopy / ParallelAdd 再按通信精度、nodal sync 和 guard-cell 安全策略分支。

这一层现在已经单独整理在 notes/code-reading/parallelization/04-warpXcomm-kernel-execution-model.md。

7.7.2 v0.12 coarse-fine substitution 与 transition zone 的证据图

把 v0.11 的 runtime 合同再往读者侧收一步,可以把 AMR coarse-fine 路径画成下面这张图。它的目的不是替代源码,而是把 UpdateAuxiliaryData*()、WarpXComm_K.H、buffer masks 和 particle gather/deposition 分区放到同一个视图里。

flowchart TD

```
A["Parent full solution F(s): E/Bfield_aux on level lev-1"] --> B["ParallelCopy into dE/dB on coarse patch"]
B --> C["Coarse-patch solution F(c): E/Bfield_cp on lev"]
C --> D["MultiFab::Subtract: d = F(s)-F(c)"]
D --> E["Fine-patch solution F(r): E/Bfield_fp on lev"]
E --> F["Fine full solution F(a): E/Bfield_aux on lev"]
F --> G["Fine interior particles gather from lev"]
G --> H["Optional copy to E/Bfield_cax for lower-level gather"]
H --> I["gather_buffer_masks / current_buffer_masks"]
I --> J["PartitionParticlesInBuffers"]
J --> K["Transition-zone particles gather from lev-1 cax"]
K --> L["Fine interior particles deposit to current_fp/rho_fp"]
L --> M["Transition-zone particles deposit to current_buf/rho_buf"]
M --> N["AddCurrent/AddRho from fine level and SumBoundary"]
```

这张图里最核心的方向是从左到右:先构造 parent full solution 与 coarse patch 的差,再把这份“patch 外源的 coarse 背景”插值到 fine patch,最后加到 fine patch 自己的解上。UpdateAuxiliaryDataSameType() 对 E/B 都按这条路走:level 0 直接把 fp 或 time-averaged fp 拷到 aux; level > 0 时,用 ParallelCopy() 从 aux(lev-1) 取 parent full solution 到临时 dE/dB,可选复制一份到 E/Bfield_cax,再减去 E/Bfield_cp,最后通过 warpx_interp() 加回 E/Bfield_fp 得到 E/Bfield_aux。如果 momentum-conserving gather 需要 staggered-to-nodal, UpdateAuxiliaryDataStagToNodal() 会进入 WarpXComm_K.H 的另一组 kernel,但数学骨架仍是 tmp + (fine - coarse)。

transition zone 则决定粒子是否真的使用这套 fine-level full solution。PhysicalParticleContainer::Evolve() 开头先取 current_buffer_masks 与 gather_buffer_masks,再判断是否存在 current_buf/rho_buf 或 E/Bfield_cax。如果存在 buffer,它调用 PartitionParticlesInBuffers(),先按较大的 buffer 宽度稳定分区,再在较大 buffer 内按较小 buffer 二次稳定分区,得到 nfine_gather 与 nfine_deposit。随后:

- 前 nfine_gather 个粒子从 E/Bfield_aux gather, gather_lev = lev;
- 后面的 gather-buffer 粒子从 E/Bfield_cax gather, gather_lev = lev-1;
- 前 nfine_deposit 个粒子把 charge/current 沉积到 rho_fp/current_fp;
- 后面的 deposition-buffer 粒子沉积到 rho_buf/current_buf,并把 deposit level 标成 lev-1。

这解释了为什么 `n_field_gather_buffer` 和 `n_current_deposition_buffer` 必须分开。一个粒子可以因为离 refinement edge 太近而从 lower level gather, 但仍在 fine patch 上 deposit; 也可以反过来。源码上这不是概念描述, 而是由同一批粒子数组重新排序后用不同的 [offset, count) 区间喂给 `PushPX()`、`DepositCharge()` 和 `DepositCurrent()`。

v0.12 还要把 regression 证据分层记录下来。当前可直接支撑 AMR/coarse-fine 叙述的入口可以先分成四类:

证据入口	覆盖的 AMR 机制	analysis 强度	适合写入正文的结论
langmuir/inputs_test_2d_langmuir_mr_refinement、 ..._mr_anisotropic、 ..._mr_maxlevel2	coarse-fine field gather、 E/Bfield_aux 对解析 Langmuir 场的闭合	强 analysis: analysis_2d.py 在 level-0 covering grid 上比较 Ex/Ez 解析场, 主误差 < 0.0503, 并按 路径叠加 charge conservation	可以作为 MR coarse-fine field solution 没有破坏 Langmuir 主模的正文证据
langmuir/inputs_test_2d_langmuir_mr_subcycling、 ..._mr_subcycling1、 ..._mr_subcycling2	langmuir_mr_subcycling gather 下的 staggered-to-nodal substitution kernel	强 analysis 同样使用 analysis_2d.py 与 checksum; 输入显式设置 algo.field_gathering = momentum-conserving	可以支撑 WarpXComm_K.H 中 tmp + (fine - coarse) 这条 kernel 不是孤立代码, 而是 被 MR Langmuir 路径覆盖
particles_in_pml/inputs_test_2d_particles_in_pml、 ..._pml_particles_in_pml、 ..._pml_particles_in_pml_3d	PML + particle current damping 后的 finest-level residual field	强 analysis: analysis_particles_in_pml.py 在 finest covering grid 上取 max(Ex,Ey,Ez), MR 容差为 2D < 6e-4、3D < 110	可以作为 MR/PML/particle- current 组合路径的残余场证据, 但不是 coarse-fine substitution 的专门 benchmark
subcycling/inputs_test_2d_subcycling_mr、 ..._subcycling_mr、 ..._subcycling_mr_3d	deposit_on_main_grid、 n_current_deposition_buffer=0 n_field_gather_buffer=0 的 MR workflow	analysis=OFF + checksum; 独立 for=0 inputs/analyze_subcycling_mr、 inputs/analyze_subcycling_mr_3d	两层 AMR 64x256、E/B/J finite、moving-window signature confirmed by coarse dz=7.8125e-8 m、 最终四 species 存在; 仍不能写成 transition-zone 物理强断言

这张表的边界很重要: langmuir MR 系列可以支持“MR 下解析场仍在容差内”; particles_in_pml_mr 可以支持“MR+PML+ 粒子离域后残余场在阈值内”; subcycling_mr 现在可以进一步支持“两层 AMR + moving window workflow 的输出完整性和几何位移与光速时间一致到一个 coarse cell”, 但仍不能写成 transition-zone 物理强断言。如果后续要把 transition zone 写成更强的物理验证, 需要新增或找到直接检查 `n_field_gather_buffer/n_current_deposition_buffer` 分区结果、E/Bfield_cax gather 路径和 `current_buf/rho_buf` 回灌结果的 analysis, 而不是只引用 checksum。

7.7.3 v0.14 transition-zone validation 应该直接检查什么

v0.14 先把上面那句“需要更强 validation”落成可执行的检查清单。当前本地 ../warpx 里, transition-zone 不是一个单独 diagnostics 名称, 而是四段源码合同叠在一起:

1. startup 根据 `n_current_deposition_buffer`、`n_field_gather_buffer` 和 `particles.deposit_on_main_grid` 决定是否分配 buffer 资源;
2. BuildBufferMasks() 把 fine patch 的 interior / buffer zone 写成 `current_buffer_masks` 与 `gather_buffer_masks`;
3. PartitionParticlesInBuffers() 根据两张 mask 改写 `nfine_deposit` 与 `nfine_gather`, 并重新排列粒子数组;
4. PhysicalParticleContainer::Evolve() 根据这两个分界, 把粒子分别送入 E/Bfield_aux、E/Bfield_cax、`current_fp/rho_fp` 和 `current_buf/rho_buf`。

源码第一段在 WarpX.cpp::AllocLevelData() 与 AllocLevelMFs()。若有 species 打开 `deposit_on_main_grid` 且 `n_current_deposition_buffer == 0`, 源码会先把 current buffer 强制补到 1, 目的只是激活 buffer 路径; 注释同时说明这个 1 并不代表真实几何宽度, 因为 `deposit_on_main_grid` 会把所有相关粒子都送到主网格沉积。随后, 负的 `n_current_deposition_buffer` 会回退到 `guard_cells.ng_alloc_J.max()`, 负的 `n_field_gather_buffer` 会回退到 `n_current_deposition_buffer + 1`。资源分配则更硬: 只要 `lev > 0` 且 gather buffer、current buffer 或 species gather-from-main-grid 任一条件成立, WarpX 就会先 coarsen fine-level BoxArray; 其中 gather 分支分配 `Efield_cax/Bfield_cax` 与 `gather_buffer_masks`, current 分支分配 `current_buf/rho_buf` 与 `current_buffer_masks`。

第二段在 `WarpX.cpp::BuildBufferMasks()`。函数对每个 level 的 current pass 与 gather pass 分别取 `ngbuffer`, 先用 `AMReX BuildMask()` 建出 guard mask, 再在 `BuildBufferMasksInBox()` 中检查每个 cell 周围 `[-ngbuffer, +ngbuffer]` 邻域: 只要邻域里有一个 guard-mask cell 是 0, 当前 cell 的 buffer mask 就写成 0; 只有整个邻域都留在 interior 时才写成 1。因此 validation 不能只看“是否有 refined patch”, 而应直接检查 mask 的 0/1 拓扑是否随 `n_current_deposition_buffer` 与 `n_field_gather_buffer` 改变。

第三段在 `Particles/Sorting/Partition.cpp::PartitionParticlesInBuffers()`。函数先选较大的 buffer mask 做第一次 stable partition, 把较大 buffer 外的粒子放到前段; 如果两个 buffer 宽度不同, 再在较大 buffer 内用较小 mask 做第二次 stable partition。最终得到两个可以不同的分界:

- `nfine_gather`: 前段粒子从 fine-level E/Bfield_aux gather;
- `nfine_deposit`: 前段粒子向 fine-level current_fp/rho_fp 沉积。

这里还有两条会覆盖 mask 结果的强制路由: `m_deposit_on_main_grid && lev > 0` 会把 `nfine_current` 直接压成 0; `m_gather_from_main_grid && lev > 0` 会把 `nfine_gather` 直接压成 0。这意味着 dedicated validation 至少要分三类 case: 普通 buffer-width case、`deposit_on_main_grid` case、`gather_from_main_grid` case。把这三类混成一条 MR checksum, 会掩盖真正的分支行为。

第四段在 `PhysicalParticleContainer::Evolve()`。源码先读 `CurrentBufferMasks()` 和 `GatherBufferMasks()`, 再由 `has_J_buf`、`has_E_cax` 得到 `has_buffer`。若 `has_buffer && !do_not_push`, 才调用 `PartitionParticlesInBuffers()`。之后, 真正写网格和 gather 的调用都以 `[offset, count)` 体现分界:

路径	源码动作	validation 应直接观测的量
fine charge deposition	<code>DepositCharge(..., rho_fp, 0, 0, np_to_deposit, lev, lev)</code>	<code>rho_fp</code> 只接收前 <code>nfine_deposit</code> 段
coarse-buffer charge deposition	<code>DepositCharge(..., rho_buf, 0, np_to_deposit, np-np_to_deposit, lev, lev-1)</code>	<code>rho_buf</code> 接收尾段粒子, 目标 level 是 <code>lev-1</code>
fine gather/push	<code>PushPX(..., 0, np_gather, lev, lev)</code>	前 <code>nfine_gather</code> 段从 E/Bfield_aux 读场
coarse gather/push	<code>PushPX(..., nfine_gather, np-nfine_gather, lev, lev-1)</code>	尾段粒子从 E/Bfield_cax/Bfield_cax 读场
fine current deposition	<code>DepositCurrent(..., 0, np_to_deposit, lev, lev)</code>	<code>current_fp</code> 只接收前 <code>nfine_deposit</code> 段
coarse-buffer current deposition	<code>DepositCurrent(..., current_buf, np_to_deposit, np-np_to_deposit, lev, lev-1)</code>	<code>current_buf</code> 接收尾段粒子, 随后参与 coarse-level sync

最后, `current_buf/rho_buf` 不是沉积终点。subcycling 路径在 `WarpXEvolve.cpp` 里会把 `current_buf` 和 `rho_buf` 交给 `AddCurrentFromFineLevelandSumBoundary()`、`AddRhoFromFineLevelandSumBoundary()`; 常规同步路径也会在 `WarpXComm.cpp::SyncCurrent()` 中把 coarse patch current 与 `current_buf` 相加, 再把别名 `mf_comm` 传入跨层通信。`SyncRho()` 同理把 `rho_buf` 作为第三个 multilevel scalar field 传入。

把这些源码合同放回当前 regression, 可以得到一个更严格的证据边界:

现有入口	能证明什么	不能证明什么	v0.14 结论
<code>langmuir/*multi_mr*</code>	MR 后 Ex/Ez 解析场仍在容差内, 并有部分路径叠加 charge conservation	不直接输出 mask、 <code>nfine_*</code> 或 <code>*_buf/*_cax</code> 分支命中情况	仍是强 AMR 场解证据, 但不是 transition-zone branch test
<code>particles_in_pml/*_mr</code>	MR+PML+ 粒子离域后的 finest-level residual field 在阈值内	不区分 residual field 来自 PML damping、particle current damping 还是 buffer routing	是组合路径强检查, 不是专门的 buffer 分区检查

现有入口	能证明什么	不能证明什么	v0.14 结论
subcycling/inputs_test_2d_subcycling_1	deposit_on_main_grid、moving window、CKC MR workflow	独立完整性 contract: final level/dimensions、E/B/J finite、moving-window shift 和 species presence; 输入仍显式 n_current_deposition_buffer=0、n_field_gather_buffer=0	可当 workflow/geometry baseline; 仍不能当 transition-zone 物理强断言

因此，若后续要补一条真正的 transition-zone validation，最小方案应当不是再加一条末态 checksum，而是新增一个可观测分区的测试面。一个可执行的设计是：

1. 使用 2D MR 小盒子，固定一个 level-1 patch，并把一组单粒子放在 patch interior、gather buffer、current buffer 和 patch 外侧的已知位置；
2. 分别运行 `n_field_gather_buffer > n_current_deposition_buffer`、`n_current_deposition_buffer > n_field_gather_buffer`、二者相等、`deposit_on_main_grid`、`gather_from_main_grid` 五类输入；
3. 在 diagnostics 或测试专用输出中记录每类粒子的场采样结果与沉积目标，至少能区分 `E/Bfield_aux` vs `E/Bfield_cax`、`rho/current_fp` vs `rho/current_buf`；
4. analysis 不只检查最终 plotfile checksum，而应断言每组粒子的 `gather_lev`、`depos_lev` 或等价可观测结果与预期一致；
5. 若不希望修改 WarpX 诊断接口，可以先用构造策略：让 fine aux 与 cax 上的某个场分量取明显不同的常数或解析值，再用单粒子动量变化反推出它到底从哪张场 gather；沉积侧则用空间上隔离的单粒子权重，在 coarse-level sync 后检查对应 cell 的 `rho/J` 是否只出现在预期 coarse/fine 面。

这条路线的价值是把 validation 从“AMR 组合案例跑通”提高到“每个 transition-zone branch 被直接命中”。它也给后续写作设了边界：在这类 dedicated test 出现之前，本章只能把 `nfine_gather/nfine_deposit`、`E/Bfield_cax` 和 `current_buf/rho_buf` 写成源码审计结论，不能把现有 MR regression 夸写成已经逐项验证这些分支。

7.7.4 v0.15 dedicated transition-zone 测试草案

v0.15 把上一节的检查清单继续推进到测试草案层。这里的目标不是在 PIC-tutor 里修改 WarpX，而是给后续 WarpX regression 提供一个足够具体的设计：输入卡应如何最小化，粒子应如何放置，analysis 应该断言哪些路由，以及现有 diagnostics 到底能观察到哪里。

建议新增一个独立 test family，例如：

```
Examples/Tests/amr_transition_zone/
inputs_test_2d_transition_zone_buffers
inputs_test_2d_transition_zone_deposit_on_main_grid
inputs_test_2d_transition_zone_gather_from_main_grid
analysis_transition_zone.py
```

对应 CMake wiring 不应只挂 checksum，而应让主 analysis 先执行路由断言，再追加默认 regression checksum。形式上可以是：

```
add_warpx_test(
    test_2d_transition_zone_buffers 2 2
    inputs_test_2d_transition_zone_buffers
    "analysis_transition_zone.py diags/diag1000001 buffers"
    "analysis_default_regression.py --path diags/diag1000001"
    OFF)
```

`deposit_on_main_grid` 与 `gather_from_main_grid` 两条 sibling 也应复用同一个 analysis，只把 mode 参数改成 `deposit_on_main_grid` 或 `gather_from_main_grid`。这样可以避免三条测试各自发展出不同的判据口径。

最小输入卡不需要复杂物理场景。它应像 `single_particle`、`multiple_particles`、`laser_on_fine` 这些 regression 一样，把几何、粒子和 diagnostics 固定到可预测状态：

```
max_step = 1
geometry.dims = 2
amr.n_cell = 32 32
amr.max_level = 1
amr.ref_ratio = 2
```

```

amr.blocking_factor = 8
amr.max_grid_size = 16
warpx.fine_tag_lo = -4.0 -4.0
warpx.fine_tag_hi = 4.0 4.0

algo.current_deposition = esirkepov
algo.charge_deposition = standard
algo.field_gathering = energy-conserving
algo.particle_shape = 1

warpx.n_current_deposition_buffer = 1
warpx.n_field_gather_buffer = 2

particles.species_names = p_interior p_gather_only p_both
p_interior.injection_style = MultipleParticles
p_gather_only.injection_style = MultipleParticles
p_both.injection_style = MultipleParticles

diagnostics.diags_names = diag1
diag1.diag_type = Full
diag1.intervals = 1
diag1.fields_to_plot = rho jx jy jz Ex Ey Ez part_per_cell
diag1.particle_fields_to_plot = x y ux uy route_x route_y
diag1.particle_fields_species = p_interior p_gather_only p_both
diag1.particle_fields.route_x(x,y,z,ux,uy,uz) = x
diag1.particle_fields.route_y(x,y,z,ux,uy,uz) = y

```

上面只是骨架,真实输入卡还要按 WarpX 当前 parser 语法补齐 geometry.prob_lo/hi、边界条件、粒子 multiple_particles_pos_x/y、动量和权重等字段。关键是使用 deterministic MultipleParticles, 而不是随机分布; 粒子坐标要分别落在 fine patch interior、只命中较宽 gather buffer 的区域、只命中较宽 current buffer 的区域、两个 buffer 都命中的区域。do_not_push 不适合作为主测试开关, 因为 PhysicalParticleContainer::Evolve() 只有在 has_buffer && !do_not_push 时才调用 PartitionParticlesInBuffers(); 它最多只能用于额外 control species, 不能用于要验证 transition-zone partition 的主粒子。

buffer-width case 至少要三条:

case	buffer 设置	目标
gather wider	n_field_gather_buffer = 2、 n_current_deposition_buffer = 1	产生“coarse gather + fine deposit”的粒子
current wider	n_field_gather_buffer = 1、 n_current_deposition_buffer = 2	产生“fine gather + coarse-buffer deposit”的粒子
equal width	n_field_gather_buffer = 1、 n_current_deposition_buffer = 1	确认两个分界一致时不会出现 only-gather 或 only-deposit 中间层

再加两条强制路由 case:

case	输入开关	目标
main-grid deposit	particles.deposit_on_main_grid = p_deposit_main	不管 current mask 如何, nfine_deposit 应被压成 0
main-grid gather	particles.gather_from_main_grid = p_gather_main	不管 gather mask 如何, nfine_gather 应被压成 0

预期粒子分区可以先写成以下判据表。这里 M_J 表示 current_buffer_masks, M_G 表示 gather_buffer_masks; 1 是 fine-interior, 0 是 buffer/guard 邻域。

粒子组	M_G	M_J	预期 gather	预期 deposit
interior	1	1	fine E/Bfield_aux	fine rho_fp/current_fp
gather-only buffer	0	1	coarse E/Bfield_cax	fine rho_fp/current_fp
deposit-only buffer	1	0	fine E/Bfield_aux	coarse rho_buf/current_buf
both-buffer	0	0	coarse E/Bfield_cax	coarse rho_buf/current_buf
deposit_on_main_grid species	任意	任意	按 gather mask 或 species 设置	强制 coarse/main-grid deposit
gather_from_main_grid species	任意	任意	强制 coarse/main-grid gather	按 current mask 或 species 设置

analysis 可分两层写。第一层完全使用现有 diagnostics: 用 diag1 读取粒子位置、动量、权重和 ρ/J , 再用已知粒子坐标与权重反推出哪些 coarse/fine cell 收到了沉积。若要反推 gather 路由, 应让 fine aux 与 coarse cax 上某个场分量在 transition zone 内产生可区分的解析值, 然后通过单步动量变化判断粒子到底从哪张场 gather。这个方案不需要改 diagnostics, 但它是间接证据, 容易被场插值、同步和最终 plotfile 合并面模糊。

第二层是更干净的 dedicated instrumentation。建议在测试专用路径中输出四类量:

输出	粒度	用途
current_buffer_mask、 gather_buffer_mask	level/grid field	直接断言 mask 拓扑随 buffer width 改变
nfine_gather、nfine_deposit	species/tile reduced table	直接断言 stable partition 分界
gather_route、deposit_route	particle diagnostic 或 route-count table	直接断言每个粒子组命中的目标层
rho_buf/current_buf routed sums	reduced diagnostics	在 coarse sync 前确认 buffer deposition 没被最终 plotfile 合并掩盖

这组 instrumentation 不必变成用户长期依赖的物理诊断。它可以是 regression-only reduced output, 或挂在 debug/test 选项后面。关键是主 analysis 要断言 route counts, 而不是只比较最终 plotfile checksum。否则即使某个 route 悄悄从 E/Bfield_cax 退回 E/Bfield_aux, 末态场也可能因为同步或数值容差而没有明显漂移。

因此, v0.15 对 dedicated transition-zone validation 的结论是: 最小 test 可以用现有输入语法和 Full diagnostics 起步, 但完整 branch proof 需要额外暴露 mask、partition 分界或 route id。书稿层面后续应把这条草案作为第 7 章的“测试设计合同”, 等真正实现或找到对应 WarpX regression 后, 再把它升级成 verified validation。

7.7.5 v0.16 transition-zone regression patch 计划

v0.16 继续把上面的草案拆成 WarpX 侧可以实施的 patch 计划。本项目仍不修改 `../warpx`, 但计划必须钉到真实源码入口, 否则后续实现时很容易把 route-count 做成与粒子实际推进脱节的旁路统计。

先确定扩展点。当前 reduced diagnostics 的类型分派集中在 `../warpx/Source/Diagnostics/ReducedDiags/MultiReducedDiags.cpp`: 构造函数读取 `warpx.reduced_diags_names`, 再按每个 `<rd_name>.type` 从 `reduced_diags_dictionary` 里创建具体类。所有 reduced diagnostics 继承 `ReducedDiags`, 基类负责 `path`、`extension`、`intervals`、`separator`、`precision`、表头文件创建和 `WriteToFile()` 的通用文本输出。`ParticleNumber.cpp` 是一个最小模板: 构造时按 species 数量建立列, `ComputeDiags()` 中循环 `MultiParticleContainer` 的 species 并把结果写入 `m_data`。因此, transition-zone route-count 最自然的第一阶段实现不是改 Full diagnostics, 而是新增一个 reduced diagnostic, 例如:

```
Source/Diagnostics/ReducedDiags/TransitionZoneRoutes.H
Source/Diagnostics/ReducedDiags/TransitionZoneRoutes.cpp
```

并在:

Source/Diagnostics/ReducedDiags/MultiReducedDiags.cpp
Source/Diagnostics/ReducedDiags/CMakeLists.txt

里注册 TransitionZoneRoutes 类型。

route-count 的观测点也必须放对。PhysicalParticleContainer::Evolve() 里，源码先取 current_masks、gather_masks、has_J_buf、has_E_cax，再对每个 tile 调用 PartitionParticlesInBuffers()。这个调用返回后，局部变量里同时存在：

- np: 该 tile 粒子数；
- nfine_deposit: 仍向 fine rho_fp/current_fp 沉积的前段粒子数；
- nfine_gather: 仍从 fine E/Bfield_aux gather 的前段粒子数；
- has_J_buf: 是否真的存在 current_buf/rho_buf；
- has_E_cax: 是否真的存在 E/Bfield_cax/Bfield_cax；
- species 成员 m_deposit_on_main_grid 与 m_gather_from_main_grid: 是否覆盖 mask 路由。

这一瞬间是 route-count 的唯一强观测点。再往后，rho_buf/current_buf 会参与 coarse sync，最终 plotfile 可能已经把 buffer 贡献混到 coarse/fine 输出面里；再往前，nfine_deposit/nfine_gather 还不存在。因此，patch 计划应在 PartitionParticlesInBuffers() 之后、第一次 DepositCharge() 之前插入一个轻量 hook。它不应改变粒子顺序，也不应读取最终场数据。

第一阶段可以新增一个小型运行时累加器，目标不是用户诊断，而是 regression instrumentation。接口形状可以是：

```
TransitionZoneRouteCounter::Record(
    species_name, lev, tile_id, np,
    nfine_gather, nfine_deposit,
    has_E_cax, has_J_buf,
    m_gather_from_main_grid, m_deposit_on_main_grid);
```

如果不希望引入全局单例，也可以把累加器挂到 MultiReducedDiags 或 WarpX 实例上；但无论挂在哪里，都要满足三条约束：

1. Record() 只在对应 reduced diagnostic 被启用时做实际工作，默认运行不能付出额外同步成本；
2. GPU/OMP tile 循环里不能直接做重 I/O，最多写线程本地或 rank-local counters；
3. ComputeDiags() 或 WriteToFile() 时再做 MPI reduce，并由 IO rank 输出文本表。

建议输出列不要过细到每个粒子 id，先做 species/level 聚合即可：

列	含义
step, time	继承 reduced diagnostic 基类口径
lev	refinement level, transition-zone validation 主要检查 lev=1
<species>_np	本 step 该 species 在该 level 被统计的粒子数
<species>_fine_gather	nfine_gather 累加值
<species>_coarse_gather	np - nfine_gather 累加值，只有 has_E_cax 或 main-grid gather case 应非零
<species>_fine_deposit	nfine_deposit 累加值
<species>_coarse_deposit	np - nfine_deposit 累加值，只有 has_J_buf 或 main-grid deposit case 应非零
<species>_forced_gather_main	m_gather_from_main_grid 命中计数
<species>_forced_deposit_main	m_deposit_on_main_grid 命中计数

如果要区分 tile 级 geometry，可以额外输出 tile_count、tiles_with_coarse_gather、tiles_with_coarse_deposit，但不建议第一版输出所有 tile 行。route-count 的 regression 目标是确认分支命中和数量正确，不是做性能诊断。

第二阶段才考虑 mask 输出。current_buffer_masks 与 gather_buffer_masks 是 iMultiFab，当前可通过 WarpX::CurrentBufferMasks(lev) 和 WarpX::GatherBufferMasks(lev) 取到，但普通 fields_to_plot 不会自动暴露它们。若要验证 mask 拓扑，有两条路线：

路线	修改点	优点	风险
field functor / debug field surface	在 Full diagnostics 初始化 field functors 时把 mask 转成可输出 MultiFab 名称, 如 <code>current_buffer_mask</code> 、 <code>gather_buffer_mask</code>	analysis 可直接用 yt 读 mask 图	需要处理 iMultiFab -> MultiFab、level 命名和非 AMR case
reduced mask summary	在 TransitionZoneRoutes 里额外统计 mask interior/buffer cell 数	修改小, 输出稳定	不能直接检查空间拓扑, 只能检查数量

v0.16 的建议是先做 reduced route-count, 再按需要补 reduced mask summary; 整张 mask plotfile 可以留到第二个 PR。原因很直接: 真正的 branch proof 先要证明粒子是否走了 coarse gather / coarse deposit, 而不是先证明 mask 图长得对。

基于这个 instrumentation, 后续 WarpX patch 可以拆成五个可 review 的提交:

patch	文件范围	验证点
1. reduced diagnostic skeleton	TransitionZoneRoutes.H/.cpp、MultiReducedDiags.cpp、CMakeLists.txt	输入 <code>warpX.reduced_diags_names = TZR、TZR.type = TransitionZoneRoutes</code> 后能生成带表头的 TZR.txt
2. route-count hook	PhysicalParticleContainer::Evolve 附近或独立 helper	若无 AMR 或未启用 TZR 时无输出变化; 启用后 <code>np = fine_gather + coarse_gather = fine_deposit + coarse_deposit</code>
3. regression inputs	Examples/Tests/amr_transition_zone_buffers_width	case 和两类 main-grid override case 都能稳定命中预期 route
4. analysis	Examples/Tests/amr_transition_zone_buffers_width	在 TZR.txt 断言每个 species 的 route counts 与预期表一致, 并追加 plotfile checksum
5. CMake wiring	Examples/Tests/amr_transition_zone_buffers_width 与上级注册	在 CMakeLists.txt 新增 test regression, 不依赖 slow 标签

analysis_transition_zone.py 的断言应比 v0.15 草案更具体。对于 gather wider case, 至少要出现:

```
p_interior: coarse_gather = 0, coarse_deposit = 0
p_gather_only: coarse_gather > 0, coarse_deposit = 0
p_both: coarse_gather > 0, coarse_deposit > 0
```

对于 current wider case, 至少要出现:

```
p_deposit_only: coarse_gather = 0, coarse_deposit > 0
p_both: coarse_gather > 0, coarse_deposit > 0
```

对于 main-grid override case, 强断言应写成:

```
p_deposit_main: fine_deposit = 0, coarse_deposit = np
p_gather_main: fine_gather = 0, coarse_gather = np
```

这比只看 ρ/J 的最终分布更稳, 因为它直接绑定 PartitionParticlesInBuffers() 的输出边界。后续若再加 field-level 物理检查, 可以在同一 analysis 里继续读取 diag1000001, 把 $\rho/jx/jy/jz$ 的非零 cell 与 route-count 做交叉校验; 但那应是第二层检查, 不应替代 route-count 主断言。

最后要把 patch 的负面边界也写进计划: 这条 instrumentation 不应成为用户长期依赖的物理输出, 不应在默认运行中引入 GPU 同步, 不应绕过 m_deposit_on_main_grid/m_gather_from_main_grid 的真实逻辑, 也不应把 do_not_push case 作为主 validation。do_not_push 会跳过 PartitionParticlesInBuffers(), 只能当 control, 不适合作为 transition-zone branch proof。

因此, v0.16 的接续目标已经从“设计一个测试”变成“写出一组可 review 的 WarpX patch”。下一步若要真正实现, 就应在 WarpX 仓库中开独立分支, 先做 TransitionZoneRoutes skeleton 和 route-count hook, 再把 Examples/Tests/amr_transition_zone 接入 CI。

源码上, AMR 与 subcycling 的关键入口在 WarpXEvolve.cpp:

- OneStep 行 469-492: 有 mesh refinement 时决定是否进入 subcycling。
- OneStep_sub1 行 1060 起: 当前 subcycling 只支持两个 level, refinement ratio 为 2。
- SyncCurrentAndRho 行 768-837: 沉积后做跨层同步和边界处理。
- Source/Parallelization/: guard cell、通信、regrid、sum guard cells 等。

本章的实践建议: 初学 WarpX 时先用 `amr.max_level = 0` 读懂单层 PIC 主循环, 再进入 AMR。否则同一个物理误差可能来自 pusher、deposition、field solver、guard cells、coarse-fine interpolation 或 PML, 排查成本会急剧上升。

后续要补的实验是: 用 Examples/Tests/langmuir 单层案例建立基线, 再选择一个 mesh-refinement 测试比较 refinement interface 附近的电荷守恒和场能量变化。

本章下一轮应继续补两块:

1. WarpX_PEC.cpp 与 PEC_Insulator.cpp 的镜像、置零和 rho/J 反射规则;
2. BoundaryConditions/ 与 Parallelization/ 交界处更细的 coarse-fine / guard-cell 例外分支, 尤其是 PML、time-averaged fields 与 solver-specific 限制。

关于第 3 块, 现已在 notes/code-reading/particles/04-amr-gather-deposition-buffers.md 与第 5 章补齐: PartitionParticlesInBuffers() 之后, 粒子如何分别进入 E/Bfield_aux、E/Bfield_cax、current_fp/current_buf、rho_fp/rho_buf 这几条 kernel 路径, 现在已经闭环。

7.7.6 练习与源码定位

1. **边界分派题**: 给定一个 field boundary 和一个 particle boundary, 分别沿 `parse_field_boundaries()`、`parse_particle_boundaries()` 定位它们如何进入 solver/particle container; 说明 periodic 继承为什么不能只看输入字符串。
2. **PML 证据题**: 对照 `pml/analysis_pml_yee.py`、`analysis_pml_psatd.py` 和 RZ analysis, 区分反射率强判据、末态 residual 判据和 checksum-only 证据。
3. **AMR route 题**: 阅读 `BuildBufferMasks()` 与 `PartitionParticlesInBuffers()`, 画出一个粒子分别进入 fine gather、coarse gather、fine deposit 和 coarse deposit 的条件; 说明为什么当前没有 dedicated route-count regression 时不能声称每条 route 已被单独验证。

7.7.7 v0.40 transition-zone 当前证据状态

为了避免把设计草案写成已完成 regression, 本章把当前状态收束成四层:

证据层	当前状态	可以支持的正文表述	不能支持的表述
源码合同	已按当前 WarpX checkout 核对 BuildBufferMasks()、PartitionParticlesInBuffers()、E/Bfield_aux/cax、rho/current_fp/buf 和 coarse-fine sync	可以解释 gather/deposit buffer 如何分区、如何选择	不能据此声称每个 route 已被专门 analysis 逐项命中
间接 regression	langmuir/*multi_mr*、particles_in_pml/*_mr 等已有解析场或残余场强判据	可以写 MR/coarse-fine 与 PML 组合路径在现有测试中的整体表现	不能把整体末态误差等同于 nfine_gather/nfine_deposit route proof
dedicated route-count 设计	v0.15/v0.16 已有 reduced diagnostic、hook、输入卡和 CMake patch 设计, 但未进入当前 ../warpx	可以作为 WarpX 侧可 review 的测试设计合同	不能称为当前 CI 已启用的 transition-zone regression
目标 checkout 状态	当前未发现已接入的 TransitionZoneRoutes 实现或 amr_transition_zone 专门 test family	可以明确记录下一条实现路径和文件边界	不能声称 route-count instrumentation 已实现或已运行

本轮又对相邻 WarpX checkout 做了只读的 live source contract audit。新增 `scripts/audit_transition_zone_source_contract.py`, 在 commit 8c488b1a9 上逐项找到 WarpX.H 的 `buffer-width` 控制, WarpX.cpp 的 `BuildBufferMasks()/BuildBufferMasksInBox()`、`Partition.cpp` 的 `stablePartition`、`PhysicalParticleContainer.cpp` 的 `nfine_deposit/nfine_gather` 与 `Efield_cax/current_buf/rho_buf` 路由, 以及 WarpXComm.cpp 的 `SyncCurrent/SyncRho` 回灌锚点; 五组源码检查全部通过。同时, 脚本确认当前 `Source/` 与 `Examples/Tests/` 中仍没有 `TransitionZoneRoutes` 或 `amr_transition_zone`。报告位于 `runs/stage-c-validation/transition-zone-source-contract.{json,md}`。这一步把“源码已核”变成了可重复的 checkout-level audit, 但仍明确不是运行级 route-count regression。

因此, v0.40 的第 7 章对 transition zone 应采用“源码已核、间接验证已核、专门 route proof 待实现”的证据等级。下一步真正进入 WarpX 时, 应先实现 reduced diagnostic skeleton 和 `PartitionParticlesInBuffers()` 后的轻量 hook, 再接入五类最小输入; 在此之前, 本书不会把现有 MR checksum 或 residual-field analysis 改写成 branch-level validation。

本轮又把这条“下一步”压成可 review 的 implementation packet: `notes/code-reading/particles/50-transition-zone-route-count-i` 固定了 `PhysicalParticleContainer::Evolve()`、`SyncCurrent/SyncRho` 的插入点, 以及 `nfine/nbuffer`、`weight`、`rho/J`、`coarsened-fine`、`owner-mask` 和 `post-sync` 的最小 reduced schema。该 packet 仍未写入当前 `../warpX`, 因此它是接续开发入口, 不是已启用的 CI regression。官方 `test_2d_subcycling_mr` 也已完成当前 checkout 的 2-rank、250 步运行。独立脚本 `scripts/analyze_subcycling_mr_contract.py` 读取初末 `diag1000000/diag1000250`: 末态为两层 AMR、64x256x1, E/B/J 在 finest covering grid 上全部 finite; moving window 的实际 z 位移为 $1.9453125e-5$ m, 而 $c*t=1.9529297e-5$ m, 误差 $7.6171875e-8$ m, 不超过 `coarse dz=7.8125e-8` m; 最终 `driver/beam/plasma_e/plasma_p` 粒子数分别为 10000/10000/30218/31872。初始诊断帧尚未包含连续注入的两个 plasma species, 脚本将其记录为初始化时序而非失败。该 contract 只支撑 subcycling+MR+moving-window 的运行完整性和几何时间一致性, 不支撑 transition-zone route-count、fine/coarse 电荷守恒或粒子 gather/deposition 分区证明。报告归档于 `runs/stage-c-validation/subcycling_2d_mr_mpi2/contract.{json,md}`。

8. 诊断、验证与案例

PIC 程序的可信度来自验证, 而不是来自输入文件能跑完。一个最小验证闭环需要回答:

- 初始条件是否表达了目标物理问题;
- 网格、粒子数、时间步是否分辨关键尺度;
- 输出量是否足以检查守恒律和不稳定性;
- 结果是否能和解析解、benchmark、regression 或文献对比;
- 源码路径是否确实是本次运行用到的路径。

本书当前第一批推荐案例是 Langmuir wave、uniform plasma 和 LWFA/PWFA。

在进入具体案例前, 可以先记住 Dawson 1983 对 diagnostics 的一个老判断: simulation 的目标是 physics essence, 而不是 detail。也就是说, diagnostics 的价值不在于“把所有字段和粒子都写出来”, 而在于能否把大规模数值状态压成可解释的 observables、谱、守恒量和 reader-side 证据。对二维和三维模型, 这种 diagnostics / visualization / postprocessing 的难度甚至可能不低于模型本身。这条判断和当前 WarpX worktree 的结构很一致: full diagnostics、reduced diagnostics、back-transformed diagnostics、checkpoint 以及 openPMD/plotfile reader-side analysis, 都不该只按 writer 类型分类, 而应按“是否真正提炼出目标 physics”来理解。

同一篇综述还给了 diagnostics 的另一条很有价值的组织方式: 先分 `measurements related to particle motion`, 再分 `measurements related to waves`。前者典型的是 `distribution function`、`phase space`、`drag`、`velocity diffusion`; 后者典型的是 `field fluctuation level`、`time correlations`、`power spectrum` 与 `nonuniform-plasma normal modes`。这种分法比“plotfile/reduced/openPMD/BTD”更接近物理问题本身, 因为它直接对应读者真正要问的量: 是想测输运系数、相关时间、噪声底、谱线, 还是想重建某个本征模的空间结构。后面各案例如果只停在“输出了哪类文件”, 而不说明它到底在测哪一类物理量, diagnostics 章节就会失焦。

Dawson 1983 后面的统计理论 examples 又把这条 diagnostics 思路压得更具体: 这些 `drag`、`diffusion`、`field-fluctuation` 和 `correlation measurements`, 不只是“可以输出的量”, 而是 simulation 用来直接检验 subtle plasma statistics 的观测合同。作者甚至特意把一维 electrostatic sheet model 提出来当高精度 benchmark, 因为它不需要 grid、可把 point-particle dynamics 跟到近 machine accuracy。于是 diagnostics 章节里有一条很值得保留的边界: reader-side analysis 的对照对象不一定只有解析式, 也可以是更 fundamental、近 exact 的 particle model。这一点对后面理解 noisy thermal backgrounds、transport coefficients 和 fluctuation measurements 特别重要。

把这条统计 diagnostics 再压实一点, Dawson 给出的最小测量合同其实已经很完整:

- drag
 - 不是看单粒子轨道, 而是固定窄速度窗口后测群体平均速度衰减;
- velocity diffusion
 - 不是任意时段都能读系数, 而要先识别 τ^2 的 short-time regime 和 decorrelation 后的近线性 regime;

- field fluctuations
 - 不是先看整张场图, 而是先看每个 k mode 的 time-averaged modal energy 是否满足热平衡与 shape-modified fluctuation 预期。

这条合同对当前 worktree 的意义很直接: 后面不论是 uniform_plasma 的 noisy thermal background、Langmuir family 的 fluctuation floor, 还是 thermal-plasma energy/stability families, 都更应该被组织成“这些 reader-side measurements 能否稳定恢复理论里真正关心的统计量”, 而不是“导出了哪些字段文件”。

如果再往前推进一层, Dawson 的 wave-side diagnostics 还要求继续区分:

- power spectrum:
 - 是 Debye-cloud random continuum 还是 collective plasma spike;
- time correlations:
 - 对应的 wave memory / decorrelation time 多长;
- magnetized peaks:
 - 是 Bernstein, upper-hybrid, ion-cyclotron, lower-hybrid, 还是 $\omega=0$ 的 convective-cell / charged-flux-tube 结构。

这对本章的直接约束是: thermal / noisy plasma diagnostics 不该只停在 field RMS 或总场能量上, 而应继续追问谱线形状、linewidth、相关时间和 peak taxonomy。否则我们只能知道“有噪声”, 却不知道噪声究竟来自随机 continuum、热平衡模、磁化谐波, 还是低频结构化 cells。

对 nonuniform plasma, Dawson 又把这条 diagnostics 合同推进了一步: reader-side analysis 的目标不只是标出某个 ω 上“有一条峰”, 而是重建该峰对应的空间波函数。做法是先记录 $\phi(\mathbf{r}, t)$ 、 $\mathbf{E}(\mathbf{r}, t)$ 或 $\mathbf{B}(\mathbf{r}, t)$; 若系统在某个方向上均匀, 就先沿该方向 Fourier 分解, 再在剩余坐标上分析 $\phi(k_x, y, \omega)$ 这类量。对离散谱线 ω_1 , 可以把信号分别与 $\sin \omega_1 t$ 和 $\cos \omega_1 t$ 做相关积分, 从而恢复 mode amplitude 和 phase profile。这里有个很硬的 measurement boundary: 积分窗口 T 必须短于该 mode 的 damping time, 否则初始 coherent oscillation 衰减后、由随机粒子运动重新激发的任意相位会把空间相位结构洗掉; 长运行应拆成多个短窗口再平均, 而不是简单延长一次积分。对连续谱也不能一概当噪声处理, 因为其中既可能出现局域在某一小块等离子体区域的 localized oscillations, 也可能只是 random particle motion 的 continuum; 后者就必须继续测 $\delta \mathbf{v}(\mathbf{v}, x, \omega)$ 这类 kinetic quantity, 而不能只停在势场或电场谱图。

这一点又和 noisy start / quiet start 的工程边界连在一起。Dawson 明确指出, 对 weak instability, random start 的主要问题不只是“图更吵”, 而是它会直接限制增长率测量的动态范围: 给定 k 模的初始涨落通常是 $N^{-1/2}$ 量级, 而弱不稳定最终可能只长到不到百分之一到几个百分点, 于是总共可用的指数增长窗口只有有限的 γt 。作者给出的数量级判断是 $\gamma t \sim \frac{1}{2} \ln N$; 即便 $N=10^5$, 典型也只有大约 5 个 e-foldings, 因此增长率往往只能测到二十个百分点量级, 对更弱的不稳定性甚至会被 natural noise 直接淹没。更具体地说, 纯随机空间加载还会强烈过激发 small- k long-wavelength electrostatic modes, 因为它没有体现 Debye shielding 和局域电中性; 这说明 quiet-start 或 cell-neutral loading 的意义不只是“让初值更平滑”, 而是把 weak-effect measurements 的可识别动态范围从噪声底里救出来。

再往实现层压一步, Dawson 给的 quiet-start recipe 也不是抽象建议, 而是明确的 phase-space construction: 把相空间切成 cells, 把每个空间 cell 内的目标速度分布 $P(v)$ 归一到该 cell 的粒子数, 再把 $P(v)$ 分成等面积小区间, 每个区间放一个粒子并赋予相应代表速度。对任意目标分布, 还可以先构造 cumulative map $y(v) = \int_{-\infty}^v P(v') dv'$, 再用其反函数把 $[0, 1]$ 上的均匀变量映射成所需速度分布。这说明 diagnostics 一侧讨论 noisy/quiet starts 时, 不能只写“quiet start 降噪”, 还要看到它真正交换掉了什么: 它用更规则的有限粒子 phase-space covering 换取更大的 weak-effect dynamic range, 但简单的 equal-area placement 对 tail 或低密度关键区域的分辨能力有限, 于是后面才需要 weighted particles / many-size electrons 继续补这条短板。

Langmuir wave

入口: `../warpx/Examples/Tests/langmuir/inputs_test_1d_langmuir_multi`

关键设置:

- max_step = 80
- geometry.dims = 1
- boundary.field_lo = periodic
- boundary.field_hi = periodic
- algo.field_gathering = energy-conserving
- algo.current_deposition = esirkepov
- 电子和正电子两个 species, 密度 $n_0 = 2.e24$

这个例子适合检查等离子体振荡频率、沉积和 Gauss 定律误差。输入中定义

$$\omega_p = \sqrt{\frac{2n_0 e^2}{\epsilon_0 m_e}},$$

这里的因子 2 来自电子和正电子两种可动带电粒子的对称贡献。动量微扰用正负相反的三角函数给出，使两个 species 产生相反响应。

图 8-1 给出本地 81 个快照验证树末态的数值电场与解析场对照。两幅图使用同一纵轴尺度：左图为 simulation，右图为 theory。这个图只承担 reader-side 的波形 sanity check；严格的场误差、最终 Gauss-law 误差和频率拟合数值仍以紧随其后的报告和 gate 为准。

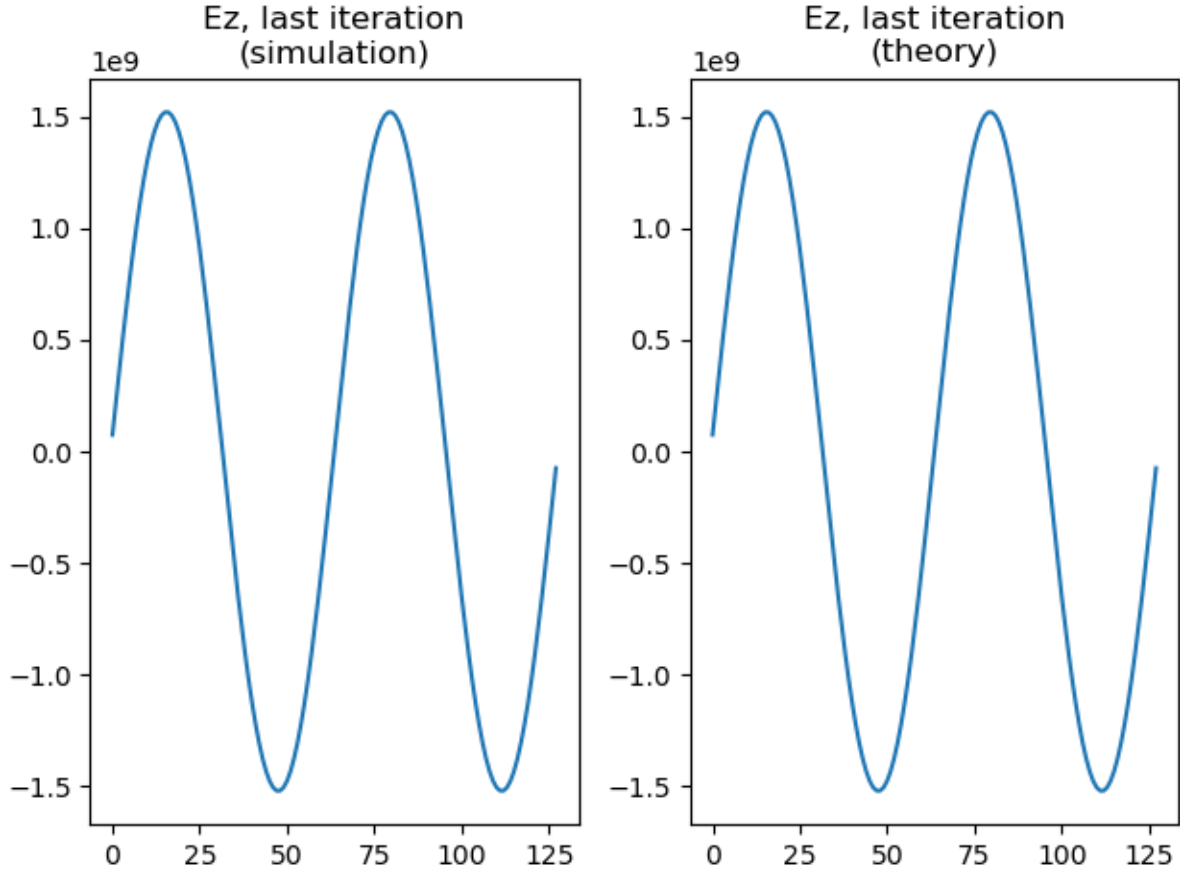


图 8-1 的数据来自 runs/stage-c-validation/langmuir_frequency_fit/langmuir_multi_1d_analysis.png，由官方 Langmuir 输入和 scripts/analyze_langmuir_frequency_fit.py 生成；图像被复制到书稿专属 manuscript/assets/figures/ 目录，避免正文依赖运行目录中的临时文件。

当前本地 Langmuir 验证树已经比这个 1D 入口更大。1D/2D/3D/RZ 原生输入族分别复用 analysis_1d.py、analysis_2d.py、analysis_3d.py、analysis_rz.py，因此共享同一个“解析场解逐点比较”的主合同；其中 3D 版本还额外检查 selective particle output 和 openPMD 粒子位置上的 Ex/Ey/Ez 场采样。analysis_utils.py 又把 charge-conservation 检查做成条件分支，只在 Esirkepov、Vay deposition 或 PSATD current-correction 这些适用组合下强制比较 divE 与 ρ/ϵ_0 。与之并列的 langmuir_fluids 则是另一棵冷流体验证树：它不只看 E，还把 J 和 rho 一起与解析冷流体解比较。需要单独记住的是，2D/3D/RZ 的 PICMI 变体目前大多仍是 analysis=OFF 的前端 + checksum scaffold，不应和原生输入的强大物理断言混成同一等级。

从应用综合章的角度，Langmuir wave 的价值不只是“有一个 textbook 解析解”，而是它把当前 worktree 里的四条主线挂到了同一个最小物理问题上：

1. 初始化
 - NUniformPerCell
 - profile = constant
 - parse_momentum_function 共同决定冷等离子体微扰如何进入粒子。
2. 粒子推进与沉积
 - energy-conserving gather

- Esirkepov, Vay deposition
 - momentum-conserving gather 在这个最小问题上暴露 $\text{divE-rho}/\epsilon_0$ 误差。
3. 场求解
- FDTD
 - PSATD
 - current_correction
 - JRhom 都能在同一解析波形下做最小分支验证。
4. 诊断与 reader-side analysis
- plotfile/full diagnostics
 - openPMD
 - selective particle output
 - on-particle Ex/Ey/Ez 都已经现有成 analysis 脚本消费。

这也是为什么 Examples/Tests/langmuir/ 在当前项目里不是“一个普通回归目录”，而是应用综合章最适合先收口的第一条主线。它已经把：

冷等离子体解析振荡

```
-> parser 初始化
-> gather / pusher / deposition / solver
-> diagnostics / openPMD reader
-> MR / PSATD / current correction / JRhom / PICMI / fluids 分支
```

压在了同一个最小问题上。

到 2026-05-18 为止，这条主线已经不只停在源码和 analysis 脚本层，也有了第一条本地运行记录。当前在

- /Volumes/PHILIPS/programs/PIC/PIC-tutor/runs/stage-c-validation/langmuir_1d

用

```
env OMP_NUM_THREADS=1 FI_PROVIDER=tcp \
/Volumes/PHILIPS/programs/PIC/warpx/build_full/bin/warpx.1d.MPI.OMP.DP.PDP.OPMD.FFT.EB.QED.GENQEDTABLES
/Volumes/PHILIPS/programs/PIC/warpx/Examples/Tests/langmuir/inputs_test_1d_langmuir_multi
```

完成了真实运行，并生成 diags/diag1000080。虽然官方 analysis_1d.py 因本机缺 matplotlib/yt 没有原样跑通，但它的核心断言已按同一公式手工复现：

- 解析场相对误差 $\text{error_rel} = 1.70\text{e-}3 < 5\text{e-}2$
- $\text{divE-rho}/\epsilon_0$ 相对误差 $8.35\text{e-}12 < 1\text{e-}11$

所以 Langmuir wave 现在在本项目里已经是运行级强基准，而不只是“源码上看起来应该能验证”的强基准。

Uniform plasma

入口：../warpx/Examples/Physics_applications/uniform_plasma/inputs_test_2d_uniform_plasma

关键设置：

- max_step = 10
- geometry.dims = 2
- 周期场边界
- 单电子 species
- 常密度 $1.\text{e}25$
- gaussian 动量分布, $u_x_{th} = u_y_{th} = u_z_{th} = 0.01$

这个例子更适合检查并行分解、粒子噪声、诊断输出和性能路径。因为物理结构简单，任何明显的非均匀场增长、粒子丢失或能量异常都容易被发现。

但当前本地 uniform_plasma 目录里的 regression 边界需要写得更精确。test_2d_uniform_plasma 和 test_3d_uniform_plasma 在 CMakeLists.txt 中都没有独立 analysis，实际只依赖顶层 Examples/analysis_default_regression.py 提供 checksum 基线；因此它们更像是“full diagnostics / 并行噪声 / 最小 workflow 稳定性”基准，而不是独立的热等离子体物理 hard assert。真正的强断言在 test_3d_uniform_plasma_restart：它从 chk000006 恢复，再用 Examples/analysis_default_restart.py 逐字段比较 restart 与非 restart 输出，要求相对误差低于 $1\text{e-}12$ 。另外，名字里带 uniform_plasma 的 test_3d_uniform_plasma_psatd_JRhom_CC1 并不属于这个应用目录，而是 nci_psatd_stability 里的 PSATD 稳定性回归，它检查的是 JRhom=CC1 + div cleaning 后电场能量是否足够小，从而证明 NCI 被压制，而不是均匀热等离子体本身的统计性质。

这意味着 `uniform_plasma` 在应用综合章里最准确的角色不是“单一 physics benchmark”，而是最小热背景 workflow：

1. 均匀、周期、单 species 的 thermal background
 - 给粒子噪声和并行划分提供最低复杂度基线；
2. full diagnostics
 - 给 plotfile/openPMD 风格输出提供最小 reader-side 骨架；
3. checkpoint/restart
 - 给 `analysis_default_restart.py` 提供一条极干净的 field-level reproducibility 基准。

如果把 TODO 里的“噪声、能量、性能和诊断”拆开，当前工作树里的证据来源其实是分层的：

- 噪声、性能背景、writer/checkpoint:
 - 主要来自 `Examples/Physics_applications/uniform_plasma/`
- 热等离子体总能量强断言：
 - 主要来自相邻 `Examples/Tests/energy_conserving_thermal_plasma/`
- PSATD/JRhom 稳定性强断言：
 - 主要来自相邻 `Examples/Tests/nci_psatd_stability/`

因此 `uniform_plasma` 的真正价值，不是它自己包含了所有强 analysis，而是它把：

均匀热背景

-> 粒子噪声 / 并行稳定性
-> full diagnostics / checkpoint
-> restart reproducibility
-> 与能量守恒、PSATD 稳定性测试树的边界

压成了项目里第二条最适合成文的应用主线。

到 2026-05-18 为止，这条主线也已有一条最小本地运行记录。当前在

- `/Volumes/PHILIPS/programs/PIC/PIC-tutor/runs/stage-c-validation/uniform_plasma_2d`

用

```
env OMP_NUM_THREADS=1 FI_PROVIDER=tcp \  
/Volumes/PHILIPS/programs/PIC/warpx/build_full/bin/warpx.2d.MPI.OMP.DP.PDP.OPMD.FFT.EB.QED.GENQEDTABLES \  
/Volumes/PHILIPS/programs/PIC/warpx/Examples/Physics_applications/uniform_plasma/inputs_test_2d_uniform_
```

完成了真实运行，并生成 `diags/diag1000010`。但这里必须保持验证分级：

- 主程序运行成功，只能证明 workflow、writer、最小噪声背景和输出路径正常；
- 官方 regression 本来就只有 `analysis_default_regression.py --path diags/diag1000010` 这一层 checksum；
- 这轮历史运行记录没有复跑 checksum 脚本；当前环境已有 yt，但仍缺 `openpmd_viewer`，所以 openPMD 分支仍未在本地验证。

所以 `uniform_plasma` 当前在本项目里的最准确状态仍然是：

- 已有运行级 baseline；
- 尚不是独立物理解强断言；
- 它的强 physics closure 仍需借相邻 `restart`、`energy_conserving_thermal_plasma` 和 `nci_psatd_stability` 三棵树来补齐。

2026-07-12: checkpoint/restart 的真实证据

为了把上面的 restart 说明从源码和 CMake wiring 推进到运行级证据，本项目在 `runs/stage-c-validation/uniform_plasma_3d_mpi2/` 复现了同一组 3D 输入：基线从第 0 步运行到第 10 步，并在第 6 步写出 `diags/chk000006`；restart sibling 从该 checkpoint 继续运行到第 10 步。两条路径最终都写出 `diags/diag1000010`。

官方 `../warpx/Examples/analysis_default_restart.py` 与项目内 `scripts/analyze_uniform_plasma_restart.py` 都对两个末态 plotfile 的 level-0 covering grid 逐字段比较。共比较 37 个 field，包含 `Bx/By/Bz`、`Ex/Ey/Ez`、`jx/jy/jz`、`rho`，以及 `electrons` 的粒子位置、动量、权重和粒子 ID；独立 reader-side 对照的最大绝对误差为 $2.4414\text{e-}4$ ，最大相对误差为 $2.8631\text{e-}16$ ，通过官方 $1\text{e-}12$ 容差。绝对误差来自量纲较大的场/电流数组，不能脱离相对误差单独解释。

这一证据有两个必须同时保留的边界。第一，它直接证明的是 checkpoint 状态恢复、粒子/场续跑和末态 diagnostics 的 reproducibility，不是热平衡能量守恒或某个解析波的 physics gate。第二，WarpX 的 CMake 注册把该测试配置为 2-rank MPI；本轮已用本机 MPICH `mpiexec -n 2` 按官方兄弟目录布局真实补跑。官方 `analysis_default_restart.py` 对 37 个 field 全部通过，独立 reader-side 对照的最大相

对误差为 $2.8631\text{e-}16 < 1\text{e-}12$ 。但仓库 checksum API 的 rank-specific 聚合参考与本地 2-rank producer 不一致，最大相对差为 $3.20\text{e-}2$ ；因此这里应写成“2-rank restart reproducibility evidence”，同时保留 checksum 非通过边界，不能把逐字段 restart pass 扩大成 checksum pass。

为解释这一 checksum 边界，又用同一当前 WarpX binary 和官方输入分别生成 1-rank、2-rank 的非 restart 基线，并运行 `scripts/analyze_uniform_plasma_mpi_consistency.py`。两套 producer 的粒子总权重完全一致；field energy 相对差为 $1.9379\text{e-}2$ ，particle kinetic energy 相对差为 $8.9170\text{e-}4$ ，total energy 相对差为 $6.2269\text{e-}4$ ，physical-field 最大 L2 相对差为 1.0185。因此该 thermal/randomized uniform-plasma case 的 rank-invariant field contract 明确不成立，checksum 差异不能被解释成 restart 失败；本项目只把 2-rank restart 的 plotfile-to-plotfile 一致性写成通过证据。1-rank/2-rank 报告位于 `runs/stage-c-validation/uniform_plasma_3d_mpi2/uniform-plasma-mpi-consistency.{json,md}`。

图 8-11 将这条边界压成两个可读的面板：左图显示 2-rank 相对 1-rank 的全局能量比，右图显示 B/E/J/rho 各组 physical field 的最大 L2 相对误差。左图说明粒子动能和总能量仍接近，但右图说明逐场 rank-invariant gate 并未成立；虚线是参考值或 $1\text{e-}12$ machine-level gate，不是本案例已经通过的物理阈值。

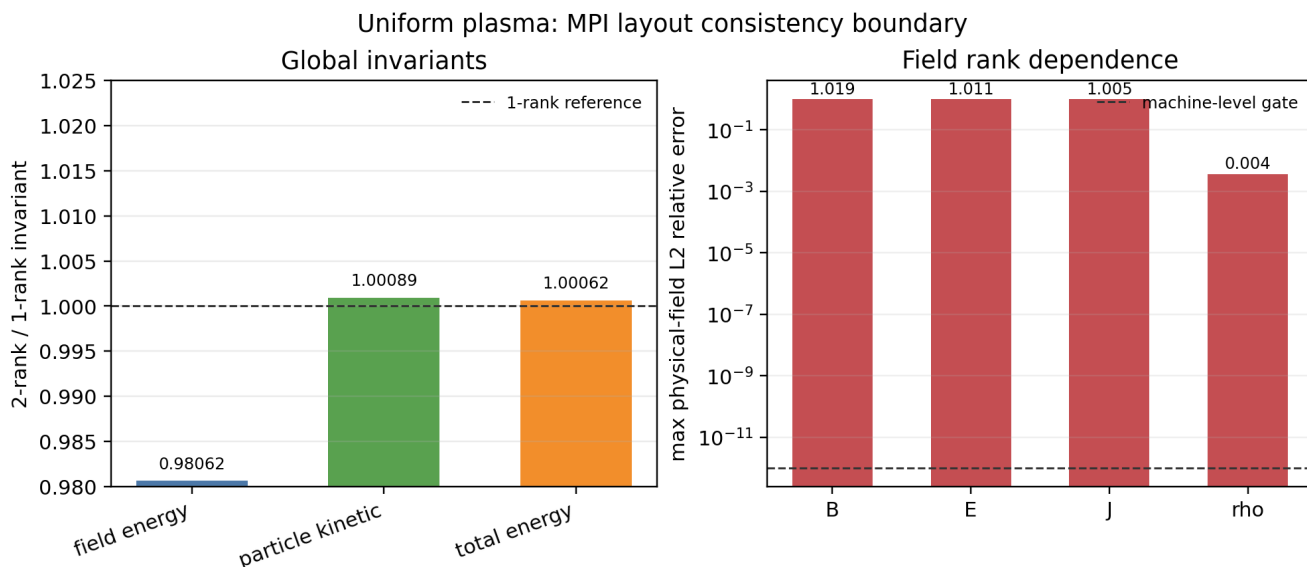


图 8-11 由 `scripts/plot_uniform_plasma_mpi_consistency.py` 从 `uniform-plasma-mpi-consistency.json` 重新生成；该图展示的是并行证据边界，不是新的强 physics benchmark。

LWFA/PWFA

应用综合章的下一条主线不应再写成单独的 `plasma_acceleration`。当前本地 worktree 里，更准确的组织方式是把：

- `Examples/Physics_applications/laser_acceleration/`
- `Examples/Physics_applications/plasma_acceleration/`

并排视作同一类 wakefield acceleration runtime architecture 的两个分支：

1. LWFA
 - laser-driven
2. PWFA
 - beam-driven

它们共享的不是统一的 physics hard assert，而是非常相近的工程关注点：

- moving window
- boosted frame
- diagnostics
- mesh refinement
- PICMI/native front-end split

这一节现在还需要保留一条更早的文献边界：当前已经开始精读的 Tajima-Dawson 1979 并不是现代 `laser_acceleration` family 的 analysis blueprint，而是这条应用线最早期的 scaling baseline。它把 `laser pulse` $\rightarrow$ `ponderomotive wake` $\rightarrow$ `trapping`

-> acceleration 这条最小物理闭环压得很硬，并给出

$$v_p = v_g^{EM} = c \sqrt{1 - \frac{\omega_p^2}{\omega^2}},$$

$$L_t = \frac{\lambda_w}{2} = \frac{\pi c}{\omega_p},$$

$$eE_L \cong mc\omega_p, \quad \gamma_{\max} \simeq 2 \frac{\omega^2}{\omega_p^2}, \quad l_a \cong 2 \frac{\omega^2 c}{\omega_p^3}.$$

这些式子最适合在本章里承担 LWFA earliest scaling baseline 的角色：它们解释为什么 wake phase velocity、dephasing、加速长度和 underdense-plasma driver 是同一条物理主线；但它们并不直接验证当代 WarpX 的 moving window、boosted frame、mesh refinement、openPMD 或 PICMI 前端实现。

同时，这篇文章也不是只有解析公式。当前精读已经确认它还给出了一个最小 relativistic electromagnetic PIC demonstration: 1 1/2-D、one spatial dimension、three velocity/field dimensions、Gaussian finite-size particles、固定离子背景，并通过扫描 ω/ω_p 去对照最早期 scaling。文中数值结果至少压实了三件事：

1. wake longitudinal field 可达到

$$E_L \sim 0.6 \frac{mc\omega_p}{e},$$

即冷等离子体 wave-breaking 级上限的大约 60%；

2. driver spectrum 会裂成多峰，作者明确解释为 successive / multiple forward Raman scattering，并把它和 photon deceleration、wake emission 联系起来；
3. simulation 中的最大电子能量随 $(\omega/\omega_p)^2$ 的变化基本贴合解析式，只是在高端开始受有限系统大小和周期边界污染。

这进一步说明：Tajima-Dawson 1979 能作为 LWFA 的 earliest scaling 与 minimal EM-PIC demonstration 文献入口，但它仍不能替代现代 WarpX laser_acceleration family 的 runtime regression 合同。

文末还要再保留三条降级边界。第一，原文的 feasible within present-day technology 只是 1979 年语境下的工程可行性判断，后面立刻又承认 short-pulse shaping 仍需改进。第二，作者明确保留了 $\Delta\omega = \omega_p$ 的 two-laser / beat-wave alternative，因此这篇文章更准确地支撑的是早期 wakefield family，而不是今天单一路径的单脉冲 LWFA。第三，pulsar atmosphere / cosmic-ray source 的段落只应当看作历史语境下的 speculative extrapolation，不能进入现代 WarpX 应用合同。

LWFA: laser_acceleration 是 runtime matrix, 不是统一 wake benchmark

laser_acceleration/README.rst 明确写的是 laser-wakefield acceleration，但 Analyze 章节仍是 TODO。配合 CMakeLists.txt 可以看出，当前这组目录更像是：

- 1D/2D/3D/RZ
- boosted frame
- moving window
- refined patch
- PICMI / Python callback
- openPMD diagnostics

这些路径的 LWFA runtime matrix。

当前只有三条局部强 analysis：

1. analysis_1d_fluid_boosted.py
 - 检查 boosted 1D 冷流体 $E_z/J_z/\rho/V_z$ 是否贴理论；
2. analysis_refined_injection.py
 - 检查 refined injection 的总粒子数和 refinement-edge 前方 ρ 均匀性；
3. analysis_openpmd_rz.py
 - 检查 RZ openPMD diagnostics 的 mesh shape、species ordering 和 $\rho_{<species>}$ 物理中心。

其余大多数 active tests 都是 analysis = OFF 加 checksum baseline。因此，当前 laser_acceleration 目录更适合在本章承担：

- moving-window / boosted LWFA skeleton

- diagnostics / openPMD / MR / PICMI 路径覆盖

而不是完整的 wake amplitude、beam energy gain 或 laser diffraction 强 benchmark。

PWFA: plasma_acceleration 是 workflow matrix, 不是解析 wake benchmark

plasma_acceleration/README.rst 明确写的是 beam-driven wakefield acceleration, 而不是 generic laser-plasma acceleration。这一点非常重要, 因为它决定了这里的 driver 不是 laser antenna, 而是 relativistic bunch。

当前目录的另一个硬边界是:

- Analyze 仍是 TODO
- Visualize 仍是 TODO
- 3D PICMI boosted-frame 等价性也被 README.rst 自己标成 TODO

同时, CMakeLists.txt 中所有 active tests 都是:

```
OFF # analysis
"analysis_default_regression.py --path ..."
```

因此 plasma_acceleration 当前最准确的角色是:

- PWFA workflow matrix
- beam-driven wakefield application baseline

它覆盖了:

- moving window
- boosted frame
- rigid bunch
- density ramp / plasma channel
- particles.use_fdt_dnci_corr = 1
- mesh refinement
- hybrid grid
- PICMI front-end

但当前并没有目录内统一的 wakefield physics hard assert。尤其要保留一个源码树边界: 3D PICMI 输入文件目前仍未像 native 输入那样真正使用 boosted frame, 所以不能把它说成 native boosted PWFA 的等价前端。

当前 worktree 中这条应用主线真正成立的结论

把 LWFA/PWFA 重新收束之后, 当前本地证据支持的最强结论是:

1. laser_acceleration
 - 是 LWFA runtime matrix
 - 强 analysis 只覆盖局部合同
2. plasma_acceleration
 - 是 PWFA workflow matrix
 - 当前 active tree 全部 checksum-only
3. 二者共享的主线不是统一 benchmark, 而是:
 - moving window
 - boosted frame
 - diagnostics
 - MR
 - PICMI/native front-end split

这也意味着, 后续书稿如果从应用角度组织 wakefield acceleration, 最自然的章节结构不是简单按目录分, 而是:

```
wakefield acceleration runtime architectures
-> laser-driven branch (LWFA)
-> beam-driven branch (PWFA)
```

Laser ion / plasma mirror / RPA/TNSA

这一条应用主线必须写得比目录名更谨慎。当前本地 worktree 里真正可落到 application tree 的 laser-target 入口只有两个：

- Examples/Physics_applications/laser_ion/
- Examples/Physics_applications/plasma_mirror/

而 RPA/TNSA 当前并没有独立应用目录或回归树，只是：

- laser_ion/README.rst 背后的物理机制标签；
- Docs/source/glossary.rst 里的术语定义。

laser_ion: 最强的 laser-target application entry

laser_ion 的真实角色不是“已经证明某条 ion-acceleration scaling”，而是：

- Gaussian laser
- planar solid-density target
- full diagnostics
- time-averaged diagnostics
- reduced diagnostics
- PICMI front-end

这条组合工作流的本地入口。

当前它在 CI 里的最硬断言来自 analysis_test_laser_ion.py，检查的是：

- diagInst 最后 5 个瞬时 Ez snapshot 的时间平均
- 与 diagTimeAvg 的原位 time-averaged Ez 是否逐点一致

因此它当前最强的 regression 合同是：

- diagnostics time-average consistency

而不是：

- TNSA cutoff energy
- RPA threshold
- ion conversion efficiency

README 里的 analysis_histogram_2d.py 和 plot_2d.py 仍然很重要，但它们当前属于 user-facing post-processing helper，不是活跃 CI regression 本体。

还要再保留一个细边界：laser_ion 确实已经有 PICMI 版输入，但其 reduced diagnostics 能力和 native 版并不完全对齐，例如 PICMI 脚本里仍留有 ParticleHistogram2D 的 TODO。因此更准确的说法是：

- PICMI 已覆盖主工作流与 analysis_test_laser_ion.py 合同；
- 但前端能力还没有完全追平 native input。

plasma_mirror: laser-solid surface-plasma workflow baseline

plasma_mirror 当前应用语义很明确：

- laser-solid interaction
- surface plasma
- planar overdense target

但验证层级明显更弱：

- 只有 test_2d_plasma_mirror
- analysis = OFF
- checksum helper
- 没有 PICMI
- README.rst 的 Analyze/Visualize 仍是 TODO

因此它更准确的角色是：

- laser-solid surface-plasma workflow baseline

而不是：

- reflectivity benchmark
- high-harmonic benchmark

RPA/TNSA：当前属于物理解释层，不属于本地应用目录层

这条边界如果不写清，很容易把文献中的机制标签误写成当前 worktree 已有的独立本地 examples。当前最强、也最保守的结论只能是：

1. laser_ion
 - 是激光打固体平面靶的本地应用骨架；
2. RPA/TNSA
 - 是理解这类骨架时需要引入的机制标签；
3. 当前 Examples/ 中
 - 没有独立 rpa_*
 - 也没有独立 tnsa_* application tree。

因此，这条应用综合章在当前 worktree 里更准确的组织方式是：

```
laser-target applications
-> laser_ion
-> plasma_mirror
-> RPA/TNSA as mechanism labels, not standalone local trees
```

Capacitive discharge

这条应用线在当前 worktree 里不该再混成普通 collision/* 附属条目。它更准确的角色是：

- PIC-MCC low-temperature plasma application tree

因为它同时把以下对象接到同一应用骨架上：

- parallel-plate electrostatic discharge
- background_mcc
- 可选 DSMC ionization 分支
- PICMI front-end
- Python callback Poisson solver
- Turner benchmark profile 对照

1D PICMI 是当前最强的 Turner benchmark 入口

当前最强的两条 active tests 是：

- test_1d_background_mcc_picmi
- test_1d_dsmc_picmi

它们共用同一条脚本：

- inputs_base_1d_picmi.py

这不是普通薄输入卡，而是完整的 benchmark driver：

1. 选择 Turner case N=1..4
2. 组装 1D electrostatic grid
3. 可选安装 Python level Poisson solver callback
4. 打开 background_mcc
5. 可选把 ionization 切成 DSMC
6. 累积离子密度并写出 ion_density_case_N.npy

当前 CI --test --pythonsolver 模式下还会显式确认：

- callback solver 已经实际运行；
- he_ions 的 z 坐标访问链可用。

因此这条应用树在工程上也不只是低温等离子体 benchmark，同时还是本地最直接的：

- PICMI + Python callback Poisson solver

应用入口之一。

当前最硬断言是 **case-1 ion-density profile**

`analysis_1d.py` 和 `analysis_dsmc.py` 当前都直接读取：

- `ion_density_case_1.npy`

并与内置 Turner case-1 参考离子密度 profile 做 `allclose`。

因此当前 worktree 里最强的 physics contract 是：

- final averaged ion density profile
- against Turner benchmark case 1

而不是笼统的“有 MCC test”或“有 DSMC test”。

DSMC 版不是孤立小 test，而是同一 benchmark scaffold 的分支

这两条 1D tests 共享同一应用骨架，区别只在于 collision realization：

1. `test_1d_background_mcc_picmi`
 - `background_mcc`
 - external Python Poisson solver callback
 - Turner case-1 profile 对照
2. `test_1d_dsmc_picmi`
 - 把 ionization 切到 DSMC 分支
 - 仍回到同一 Turner case-1 profile 对照

因此更准确的表述是：

- DSMC 分支已经在同一低温等离子体 benchmark scaffold 里被强对照覆盖

而不是只证明“DSMC can run”。

2D native / PICMI 当前仍主要是 workflow baseline

与 1D 强对照相比，当前 2D 分支只有：

- `test_2d_background_mcc`
- `test_2d_background_mcc_picmi`

并且两者都是：

- `analysis = OFF`
- checksum helper

因此它们当前只能诚实记成：

- 2D capacitive-discharge workflow baseline

而不是 2D Turner 强 benchmark。另一个必须保留的边界是：

- `test_2d_background_mcc_dp_psp`

当前整条 `add_warpx_test(...)` 仍被注释掉，所以它只能作为遗留分支记录，不能再冒充活跃 test。

既有 `plasma_acceleration` 目录边界

入口：`../warpx/Examples/Physics_applications/plasma_acceleration/inputs_test_3d_plasma_acceleration_boosted`

这一组也需要避免被过度解读。当前本地 `plasma_acceleration` family 在 `CMakeLists.txt` 中所有活跃 tests 都是 `analysis = OFF`，只复用目录内的 `analysis_default_regression.py` 做 checksum。因此它们不是“PWFA 解析 benchmark”，而是应用 workflow 基线。

但它们并不空泛。原生输入和 PICMI 输入合起来，已经覆盖了：

- moving window
- boosted frame
- rigid-injected driver/beam
- `particles.use_fdt_nci_corr = 1`
- level-1 refined patch / `add_refined_region(...)`
- momentum-conserving gather 分支
- `grid_type = hybrid`
- field / particle diagnostics

因此这组例子当前真正承担的角色是：给 beam-driven wakefield acceleration 的 runtime matrix 保留稳定输出基线，而不是直接对 wake amplitude、dephasing 或 beam loading 做强物理断言。还有一个需要显式保留的源码树边界是：README.rst 目前明确写着 3D PICMI 版“应该像原生输入一样使用 boosted frame，但仍是 TODO”。所以 `inputs_test_3d_plasma_acceleration_picmi.py` 当前只能诚实记成 non-boosted PICMI scaffold，不能误写成原生 boosted PWFA 的等价前端。

Magnetic reconnection

当前 worktree 里，磁重联这条应用线最准确的入口不是一般 Fluids/，而是：

- Examples/Tests/ohm_solver_magnetic_reconnection/

它依赖的是：

- `picmi.HybridPICSolver`
- `HybridPICModel`

也就是：

- kinetic ions
- electron-fluid Ohm closure
- Faraday + RK 子步推进 B

而不是 WarpXFluidContainer 那条额外 cold-fluid species runtime layer。这个边界必须写死，因为已有源码笔记已经明确：

- Fluids/
 - 自己维护 nodal N/NU
 - gather 主场
 - 再把 ρ/J 沉积回普通场寄存器；
- HybridPICModel
 - 则是在 field solver 内部从总电流、离子电流和电子闭合关系反推出 E。

因此 magnetic_reconnection 在应用综合章里的正确角色是：

- hybrid-PIC space-plasma application

而不是：

- fluid/PIC coupling demo

force-free sheet + reduced FieldProbe

`inputs_test_2d_ohm_solver_magnetic_reconnection_picmi.py` 当前不是薄输入卡，而是完整应用 driver。它同时定义了：

- 2D Cartesian 几何
- x 周期、z 方向 dirichlet/reflecting
- force-free-sheet 解析初始 $B_x/B_y/B_z$
- plasma_resistivity
- substeps
- kinetic ion loading
- reduced diagnostic FieldProbe

其中最重要的 diagnostics 不是 full plotfile，而是：

- `plane.dat`

它来自 X 点附近的 reduced FieldProbe, 专门供 reader-side analysis 提取重联率。

当前 analysis 是 observable extraction, 不是强 assert

analysis.py 当前直接从 plane.dat 读取 E_y, 构造:

$$R(t) = \frac{\langle E_y \rangle}{v_A B_0}.$$

然后输出:

- reconnection_rate.png

在非 --test 模式下还会进一步生成:

- mag_reconnection.mp4

但这条脚本没有显式数值 assert。因此它当前只能被诚实归类为:

- physics-informed visualization / observable extraction

而不是:

- hard numerical benchmark

checksum 仍是 active coverage 的另一半

CMakeLists.txt 里这条 test 还会同时跑:

- analysis_default_regression.py --path diags/diag1000020

所以当前 active coverage 的真实结构是:

1. analysis.py
 - 提取重联率并可视化;
2. checksum helper
 - 兜底历史输出稳定性。

这也解释了它和邻近 ohm_solver_* 条目的分工:

- ohm_solver_em_modes、ion_beam_instability
 - 更偏局部 solver correctness 的强 regression;
- magnetic_reconnection
 - 更偏 hybrid-PIC 代表性物理案例和输出回归。

因此, 这条应用线在当前书稿里最准确的结论应写成:

```
magnetic_reconnection
= HybridPICModel application line
= force-free-sheet + reduced FieldProbe + reconnection-rate extraction
= physics-informed visualization + checksum
!= scalar hard-assert benchmark
```

Beam-beam / luminosity / FEL / ion extraction

这一条应用综合主线最容易被误写成单一“束流例子”列表, 但当前 worktree 里的四类入口其实承担着不同层级的合同:

- DifferentialLuminosity
- beam_beam_collision
- free_electron_laser
- ion_beam_extraction
- accelerator_lattice

更准确的组织方式应是:

```
beam and accelerator applications
-> luminosity diagnostics benchmark
-> collider-QED workflow baseline
-> FEL boosted-frame radiation benchmark
-> electrostatic ion-source extraction
-> accelerator-lattice optics regression
```

DifferentialLuminosity: reduced-diagnostic 强谱基准

Examples/Tests/diff_lumi_diag/ 当前是这条应用线里最强的 diagnostics regression。它的 analysis.py 不是普通画图脚本，而是同时读取：

- 一维文本表 DifferentialLuminosity_beam1_beam2.txt
- 二维 openPMD 网格 DifferentialLuminosity2d_beam1_beam2/

然后直接构造两束 Gaussian beams 的解析 luminosity 谱：

- dL/dE
- $d^2L/dE_1 dE_2$

再与 diagnostics 做显式误差比较和 assert。因此它的角色必须写成：

- reduced-diagnostic strong benchmark

而不是一般的 beam application helper。

beam_beam_collision: collider-QED 应用骨架

与 DifferentialLuminosity 相比，Examples/Physics_applications/beam_beam_collision/ 当前证据层要弱得多：

- active regression 只有 checksum helper；
- 没有独立 analysis.py；
- plot_fields.py / plot_reduced.py 只是后处理可视化脚本。

但它并不空泛。它当前真正把这些路径绑在一起：

- warpx.do_electrostatic = relativistic
- 两束 125 GeV 电子/正电子 Gaussian bunch 对撞
- initialize_self_fields = 1
- Quantum Synchrotron
- Breit-Wheeler
- ColliderRelevant
- ParticleNumber
- openPMD full diagnostics

因此它最准确的定位是：

- collider-QED application baseline

而不是 luminosity 强谱基准。

free_electron_laser: boosted rigid-beam + undulator + BTD 的强 benchmark

free_electron_laser 当前不是普通 laser example，因为它本质上没有 laser antenna。已有本地笔记已经压实：

- 核心是 RigidInjectedParticleContainer
- particles.By_external_particle_function(...) 提供 undulator 外加粒子磁场
- BackTransformed diagnostics 与 boosted-frame full diagnostics 的一致性

当前最强断言来自 analysis_fel.py：

1. 对 $\log(E_x^2)$ 的线性增长区做拟合，要求 gain length 接近 0.22 m；
2. 在 lab-frame 与 boosted-frame diagnostics 上做 FFT，要求 radiation wavelength 满足 undulator 理论值。

因此它当前最准确的角色是：

- boosted rigid-beam radiation benchmark

这条应用线也正好和 Dawson 1983 里的经典 FEL 例子接上。那篇综述把 free-electron laser 专门当作 relativistic electromagnetic particle model 的代表问题: lab frame 下给出

$$\lambda \simeq \frac{\lambda_0}{2\gamma^2},$$

而在 beam frame 下又把同一过程重写成 pump electromagnetic wave 衰变成 electromagnetic wave 加 plasma wave 的 Raman-like 参数不稳定, 并要求满足

$$k_{\text{pump}} = k_{\text{EM}} + k_p, \quad \omega_{\text{pump}} = \omega_{\text{EM}} + \omega_p(k_p).$$

它的历史 simulation 结果还明确展示了 matching-condition 谱证据、约 36% 的 longitudinal current 下降、约 30% 的束流能量转成辐射, 以及 $2\lambda_0$ backward mode 的危险性。对本章来说, 这组文献证据的作用不是替代当前 `analysis_fel.py`, 而是把 WarpX 这条 boosted rigid-beam + undulator + BTB benchmark 放回更早的 relativistic EM-PIC 谱系里理解。

如果再把图像层次压得更明确, Dawson 1983 这条 FEL 历史线已经形成一个很完整的 diagnostics contract:

- Fig.54
 - 给最小装置和 lab-frame / beam-frame 两种物理图像;
- Fig.55
 - 用 EM / electrostatic spectra 检查 matching condition;
- Fig.56
 - 用 EM energy、electrostatic energy 和 longitudinal current 的同步演化检查 gain、beam degradation 与 saturation;
- Fig.57
 - 再把 trapping-based efficiency estimate

$$\eta = \frac{\gamma_0 - \gamma_{\text{ph}}}{\gamma_0 - 1}$$

及其大- γ 近似

$$\eta \simeq \omega_{po}(2k_0 c \gamma^{3/2})^{-1}$$

与 simulation 对照。

也就是说, 这组经典图像已经把 mechanism verification、nonlinear saturation 和 rough efficiency scaling 串成一条最小 reader-side 论证链。当前 WarpX `free_electron_laser` 的价值, 则是把这条历史论证链重新落到 boosted-frame implementation、BTB 重建和 gain-length / wavelength regression 上。

ion_beam_extraction: EB electrostatic extraction 的强应用入口

`Examples/Physics_applications/ion_beam_extraction/` 当前是这条应用线里最直接的 electrostatic + embedded-boundary beam-source application。

它把:

- plasma source
- `boundary.potential_*`
- `warpx.eb_potential(x,y,z,t)`
- electrostatic solver
- embedded-boundary electrode geometry
- 持续 boundary injection

接成了一条完整链。

当前 `analysis_ion_beam_extraction.py` 会直接检查抽出离子束尾部能量是否接近 40 keV, 因此它不是 checksum baseline, 而是:

- electrostatic EB extraction strong application check

`accelerator_lattice`: beamline optics 的强回归层

如果只写 `collider`、`FEL` 和 `extraction`, 这条总节还少了一层真正对应加速器模块本身的强验证。当前最自然的入口是:

- `Examples/Tests/accelerator_lattice/`

这里的 `analysis.py` 会重新读回:

- `lattice.elements`
- `line*`
- `drift*`
- `quad*`

然后按解析 `hard-edged quadrupole optics` 逐段积分, 并要求最终粒子轨道和解析解足够接近。它同时覆盖:

- `lab frame`
- `boosted frame`
- `moving window`

所以它在这条总节里最准确的角色是:

- `beamline optics strong regression`

诊断在源码中的位置

`WarpX::Evolve` 中诊断不是附加脚本, 而是时间步的一部分:

- 行 173: `multi_diags->NewIteration()`。
- 行 323-330: 判断是否需要为诊断同步粒子速度。
- 行 337-344: `reduced diagnostics` 和 `full diagnostics` 的计算、打包、写出。
- 行 374-382: 最终时间步或中断时 `flush last timestep`。

源码目录包括:

- `../warpX/Source/Diagnostics/`
- `../warpX/Regression/Checksum/`
- `../warpX/Examples/analysis_default_regression.py`

更底层地看, `Source/Diagnostics` 顶层其实分成四层角色:

1. `MultiDiagnostics` 负责读取 `diagnostics.diags_names`, 按 `<diag>.diag_type` 把每个 `diagnostics` 实例化成 `FullDiagnostics`、`BTDiagnostics` 或 `BoundaryScrapingDiagnostics`, 并在主循环里统一分派。
2. `Diagnostics` 提供统一模板骨架: `InitData()`、`FilterComputePackFlush()`、`ComputeAndPack()`、`Flush()`。也就是说, 所有 `diagnostics` 都要经过“先决定是否 `compute/pack`, 再决定是否 `flush`”的同一套阶段。
3. `FullDiagnostics` 负责把 `fields_to_plot`、`particle_fields_to_plot` 和 `species` 输出需求映射成具体 functors, 再把结果堆叠进输出 `MultiFab`。
4. `ParticleDiag` 不是真正的粒子数据缓冲区, 而是每个 `species` 的输出配置对象: 它记录变量选择、`random_fraction` / `uniform_stride` / `parser filter`、附加粒子场请求, 以及粒子来源容器指针。

一个关键实现边界是: 普通 `FullDiagnostics` 的粒子输出, 并不像 `back-transformed diagnostics` 那样先 `pack` 到独立粒子 `buffer`。它更多是把 `ParticleDiag` 作为 `species` 句柄和过滤配置传给 `writer`; 真正的粒子变量裁剪和过滤发生在 `FlushFormatPlotfile.cpp` / `WarpXOpenPMD.cpp` 的写出阶段。只有 `BTDiagnostics` 这类带 `snapshot buffering` 的 `diagnostics`, 才会真正分配 `m_particles_buffer` 和 `ComputeParticleDiagFunctor`。

这也是为什么 `Diagnostics::ComputeAndPack()` 虽然同时有 `field-functor loop` 和 `particle-functor loop`, 但对普通 `full diagnostics` 来说, 后者默认是空的。不要把“所有 `diagnostics` 都有独立粒子 `pack` 阶段”当成 `WarpX` 的统一事实。

继续往下拆, 会看到 `diagnostics` 模块其实还有另一条容易混淆的边界。 `ComputeDiagFunctors/` 是字段计算层: `JFunctor`、`RhoFunctor`、`PhiFunctor` 直接把 `j/rho/phi` 这类量写进 `diagnostics MultiFab`; `ParticleReductionFunctor` 虽然读取粒子, 但它输出的仍然是 `cell-centered MultiFab`, 因此也属于字段计算层, 而不是粒子 `writer`。

真正的粒子过滤发生在 `writer` 阶段。无论是 `WarpXOpenPMD.cpp` 还是 `FlushFormatPlotfile.cpp`, 都会先从 `ParticleDiag` 取出:

- `random_fraction`
- `uniform_stride`
- `parser filter`

- geometry filter

然后创建一个临时粒子容器 `tmp`，通过 `tmp.copyParticles(...)` 把通过过滤的粒子复制进去，再把 `tmp` 写出。因此 `ParticleDiag` 构造阶段只是记录过滤规则；真正应用过滤是在 `flush` 的时候。

`phi`、`Ex/Ey/Ez`、`Bx/By/Bz` on particles 也不属于 `field functor` 层，而是 `writer` 在过滤后的 `tmp` 上再做一次 `gather`。`ParticleIO.cpp` 明确限制这些附加粒子场只允许 `diag_type = Full`，因为对 `BackTransformed` 或 `BoundaryScraping` 这类带粒子缓冲区的 diagnostics 来说，粒子被写出的时间并不等于粒子被收集的时间，此时再 `gather` 场会产生时间层错配。

`ReducedDiags/` 又是另一套平行体系。它不继承 `Diagnostics`，也不走 `MultiFab + ParticleDiag + FlushFormat` 这条主线，而是由 `MultiReducedDiags` 读 `warpx.reduced_diags_names` 后，按 `<reduced_diag_name>.type` 分派到 `FieldEnergy`、`ParticleEnergy`、`ParticleHistogram`、`FieldProbe`、`LoadBalanceCosts` 等具体类型。它们共享的核心抽象不是字段/粒子快照，而是一段 `m_data` 向量和统一的表格写出协议：第一列是 `step`，第二列是时间，后面各列是 `reduced quantity`。

这类 `reduced diagnostics` 里，很多类型是“到点现算即写”，例如 `FieldEnergy` 和 `ParticleEnergy`；但也有例外，例如 `FieldPoyntingFlux` 会维护跨时间步累积的积分量，因此还要实现 `WriteCheckpointData()` / `ReadCheckpointData()`，把内部状态写进 `checkpoint` 并在 `restart` 时恢复。也就是说，`diagnostics` 模块里并不是只有 `BTDiagnostics` 才有跨步状态，部分 `reduced diagnostics` 也有。

`checkpoint format` 本身也不能等同于普通 `diagnostics` 格式。`FlushFormatCheckpoint.cpp` 实际写出的是 `WarpX` 的 `restart state`: `E/B`、`coarse/fine patch fields`、同步后的 `current`、`PML` 数据、粒子状态，以及 `reduced diagnostics` 额外的 `checkpoint` 数据。它并不是把某个 `diagnostics` 的 `m_mf_output` 换一种文件格式落盘，而是直接序列化运行态。

`BTDiagnostics` 则是另一类“有状态机”的 `diagnostics`。它几乎每一步都可能执行 `DoComputeAndPack()`，把 `cell-centered` 后的场切成一片片 `lab-frame slice`，逐步填进 `snapshot buffer`；`DoDump()` 判断的也不是单纯的时间间隔，而是“当前 `buffer` 是否已满”“最后一个有效 `z-slice` 是否已经填满”以及“结束时是否需要强制冲刷剩余 `buffer`”。因此 `BTD` 不能被理解成另一种普通 `full diagnostics`，它本质上是一套 `slice / buffer` 的累积和 `flush` 机制。

再往 `reduced diagnostics` 的具体类型里看，会发现它们虽然共用 `ReducedDiags` 骨架，但实现形态并不统一。`FieldProbe` 内部维护的不是一个标量数组，而是一套专门的 `FieldProbeParticleContainer`: `point/line/plane` 三种几何最终都会在 `InitData()` 中转成一批 `probe particles`，再在 `ComputeDiags()` 里对 `Efield_aux/Bfield_aux` 做 `gather`。因此它测到的不是推进器主寄存器的原始 `fp` 场，而是粒子侧真正会看到的 `aux` 场；`do_moving_window_FP` 也不是事后回推场，而是直接平移这批 `probe particles`。若 `integrate = 1`，它还会在每一步把采样值乘 `dt` 累加到 `probe-particle SoA` 中，到输出步才写出累计量。

`ParticleHistogram` 和 `ParticleHistogram2D` 又是另一种 `reduced diagnostics`。前者是 `parser` 驱动的一维 `weighted histogram`：对每个粒子先算 `histogram_function(t,x,y,z,ux,uy,uz)`，再按 `floor((f-bin_min)/bin_size)` 落 `bin`，默认累加粒子权重 `w`，最后在 `MPI` 归约之后再做 `max_to_unity` 或 `area_to_unity` 归一化。后者虽然还挂在 `ReducedDiags/` 下，但 `writer` 已完全变成 `openPMD mesh` 输出：`histogram_function_abs`、`histogram_function_ord` 决定二维坐标，`value_function` 决定每个粒子向该 `bin` 累加什么值，并且 `writer` 会连同轴标签、`bin spacing`、`global offset` 和 `parser` 字符串一起写进 `openPMD` 元数据。因此二维 `histogram` 不是“多一列文本表”，而是真正的带坐标二维诊断场。

`LoadBalanceCosts` 和 `LoadBalanceEfficiency` 则属于性能/并行态 `diagnostics`。前者输出粒度是 `box`，而不是 `rank`：每个 `box` 会写 `cost`、`proc`、`lev`、`i_low/j_low/k_low`、`num_cells`、`num_macro_particles`，`GPU` 运行时还会附带 `gpu_ID`，再通过额外的 `MPI_Gatherv` 收集 `hostname`。`Heuristic` 模式下它会先调用 `ComputeCostsHeuristic()` 重建 `heuristic cost`；`Timers` 模式下则直接导出当前 `timers` 成本。因此它真正暴露的是 `WarpX` `load-balance` 决策所依据的 `box-level` 负载分布，而不只是一个抽象效率数字。`LoadBalanceEfficiency` 相比之下非常薄，它只是把 `warpx.getLoadBalanceEfficiency(lev)` 的结果按 `level` 写出来，用于快速检查某次重分配前后是否更均衡。

对应的 `regression` 也不是同一种口径。`analysis_reduced_diags_impl.py` 会从 `full plotfile` 重新计算 `FieldEnergy`、`ParticleEnergy`、`FieldReduction` 等 `compact observable`，再与 `reduced diagnostics` 文本结果比较，所以它主要验证 `reduced diagnostics` 与 `full-state reference` 是否一致。`analysis_reduced_diags_load_balance_costs.py` 则完全不读 `plotfile`，而是直接从 `LBC.txt` 重建每个 `rank` 的总成本，并只断言 `load-balance` 之后

$$\text{efficiency}_{\text{before}} < \text{efficiency}_{\text{after}}$$

这说明 `reduced diagnostics` 在 `WarpX` 里既有“物理量压缩输出”的一面，也有“把并行运行态暴露给后处理”的一面，不能把它们都当成同一种小型文本统计表。

如果把 `diagnostics` 再按 `writer` 分成一层，会看到 `diag_type` 和 `format` 是两套独立分派。`MultiDiagnostics` 先按 `diag_type` 把对象构造成 `FullDiagnostics`、`BackTransformed` 或 `BoundaryScrapingDiagnostics`；之后 `Diagnostics::InitDataBeforeRestart()` 再按 `format` 选择：

- FlushFormatPlotfile
- FlushFormatOpenPMD
- FlushFormatCheckpoint

这三条 writer 路径虽然共用 flush 调度时机,但服务目标已经不同。plotfile 路径会把 diagnostics 已经 pack 好的 cell-centered MultiFab 通过 WriteMultiLevelPlotfile(...) 写出;若打开 plot_raw_fields = 1,还会额外把原始 staggered/raw fields 写进 raw_fields/ 子目录,因此它既能给普通分析用,也能给底层网格调试用。openPMD 路径在 fields 侧同样写 diagnostics 视图,但在粒子侧比 plotfile 更强:它可以在 writer 中对过滤后的临时粒子容器再次 gather phi、Ex/Ey/Ez、Bx/By/Bz。不过这项能力只允许 diag_type = Full,因为 ParticleIO.cpp 明确禁止对 BackTransformed 或 BoundaryScraping 这类“收集时刻和写出时刻不一致”的缓冲型 diagnostics 再去 gather 场。

checkpoint 路径则完全不是“另一种 diagnostics 文件格式”。FlushFormatCheckpoint::WriteToFile() 基本不消费 m_mf_output,而是直接从 warpx.m_fields 序列化真实运行态:

- Efield_fp/Bfield_fp
- E_old
- synchronized current_fp/current_cp
- coarse patch fields
- time-averaged fields
- PML 数据
- 完整 species 与 lasers 粒子状态
- distribution mapping
- reduced diagnostics 的 checkpoint state

因此 checkpoint 的真正对象是 restart persistence,而不是用户筛选后的诊断视图。也正因为如此,FullDiagnostics::ReadParameters() 对 format = checkpoint 做了比文档更强的源码约束:不能自定义 fields_to_plot、不能裁剪 diag_lo/diag_hi、不能做 coarsening_ratio、不能指定 species 子集,也不能开 raw fields。它要求的是“全量可恢复状态”,不是“最小可读输出”。

从现有本地例子看,这三类 writer 的最小输入骨架也已经比较稳定:

- 普通 plotfile: 只写 diag1.diag_type = Full 和一组 fields_to_plot 即可,format 缺省就是 plotfile。
- openPMD: 在 full diagnostics 上再加 diag1.format = openpmd 和 diag1.openpmd_backend = h5/bp*,laser_ion 已经给出了带 field filtering 的最小可复用骨架。
- checkpoint: 通常并行放一个 diag1 和一个 chk,后者写 chk.diag_type = Full、chk.format = checkpoint;重启则用 amr.restart = "../.../chk000XXX" 接回。

对本章当前最相关的 reduced diagnostics,也已经能直接从本地 examples 抽出最小运行入口:

- FieldProbe: Examples/Tests/reduced_diags/inputs_test_3d_reduced_diags 和 laser_ion 都给了 point/line 的最小参数骨架。
- ParticleHistogram2D: laser_ion 已经给了 histogram_function_abs/ord 与 value_function = "w" 的二维相空间例子。
- LoadBalanceCosts: Docs/source/usage/workflows/plot_distribution_mapping.rst 与 Examples/Tests/reduced_diags 已经构成最小“生成 + 画图/验效” workflow。

如果要看 FieldProbe 的强 analysis regression,本地还有一组比这些“最小骨架”更直接的条目: Examples/Tests/field_probe/. 它不是只检查文件格式,而是把 line FieldProbe 接到单缝衍射 benchmark 上。analysis 会从 FP_line.txt 读出 step 500 的积分电磁通量,再与解析 sinc^2 衍射包络比较,并要求平均相对误差小于 2.5%。按当前 checkout 的官方输入真实运行后,这条线暂时不能作为“已通过”的例子: 1-rank 和官方 2-rank MPI 配置产生完全一致的 FP_line.txt,但 analysis.py 的平均误差都是 3.6703%,最大选点误差为 10.0843%。项目内的 scripts/analyze_field_probe_diffraction.py 已复现同一选点范围和分母,并将结果归档到 runs/stage-c-validation/field_probe_2d/ 与 field_probe_2d_mpi2b/。

这里的失败本身是诊断章节需要保留的证据。它说明 reduced diagnostic 的 writer 合同已经接通: 201 个 probe 点、step 500 的积分通量和 1/2-rank 一致性都成立;但这还不足以证明 FieldProbe 的物理量与解析衍射包络一致。随后将网格从官方的 $\lambda/16$ 加密到 $\lambda/32$,并在相同物理时间的 step 1000 取样,官方同口径误差降为 0.3533%,最大选点误差为 1.0414%,通过 2.5% gate。对照报告位于 runs/stage-c-validation/field_probe_resolution_comparison.md。

因此当前最稳妥的成书结论是:原始 coarse case 的“输出链通过、解析 physics gate 未通过”是真实结果;网格加密后的通过结果支持 coarse-grid 离散误差是主因,但不能把 refined case 的结果反写成原始官方输入已通过。关闭 filter 只能把误差略降至 3.5910%,而 interp_order=0 在当前分支产生零通量,后者应作为另一个需要单独审计的 raw-field gather 边界,而不是有效的物理改进方案。

图 8-3 将这条边界画成两个并排面板：左图是官方 analysis 使用的平均误差和 2.5% gate，右图是 40 个选点中的最大误差。颜色只表示当前报告的 pass/fail 状态；refined 的通过来自 $\lambda/32$ 、相同物理时间的 step 1000 对照，不是对 coarse 输入的重写。



图 8-3 由 `scripts/plot_field_probe_resolution.py` 从 `runs/stage-c-validation/field_probe_resolution_comparison.js` 重新生成。

同一层里，本地还有两组更偏束流诊断的强 regression。Examples/Tests/collider_relevant_diags/ 不是普通 reduced-output 烟雾测试，而是把 ColliderRelevant 与 ParticleExtrema 并排打开，然后用解析粒子样本逐项核对 `chi_min/max/ave`、`theta_x/theta_y` 的 `min/ave/max/std`，再从 full openPMD 的 `rho_beam_e/rho_beam_p` 重建 `dL/dt` 与 reduced output 交叉验证。也就是说，这组例子验证的不是“表格写出来了”，而是 collider-oriented reduced quantities 的定义和聚合合同本身。

Examples/Tests/diff_lumi_diag/ 则把 reduced diagnostics 进一步推进到带解析谱对照的束流物理量：一维 DifferentialLuminosity 文本表和二维 DifferentialLuminosity2D openPMD 网格同时输出，analysis 直接构造两束高斯束流碰撞的解析 `dL/dE` 与 `d2L/dE1dE2`，再分别比较 1D/2D diagnostics。对本书来说，这组例子非常有价值，因为它把“reduced diagnostics 可以是纯文本列，也可以是 openPMD 网格”这件事，用同一个物理 benchmark 明确落地了。

与之相邻、但等级更弱的一组是 Examples/Physics_applications/beam_beam_collision/。这组例子同样使用 collider 场景、Quantum Synchrotron、Breit-Wheeler、ColliderRelevant 与 ParticleNumber reduced diagnostics，但当前活跃 regression 只有 analysis_default_regression.py 提供 checksum，没有独立 analysis.py。因此它更准确地是一个 collider-QED application baseline：验证 relativistic electrostatic self-field、beamstrahlung、coherent pair generation 和 reduced diagnostics 的联合工作流能否稳定接通，而不是对 luminosity 谱或 Yakimenko 2019 结果做强物理断言。目录里的 plot_fields.py 和 plot_reduced.py 也只是说明用户应如何可视化 $|E|/|B|$ 、次级对密度、每个 beam particle 的 photon 数与 NLBW pair 数；它们不应被当成 regression analysis。

Examples/Tests/reduced_diags/ 本体当前则应再拆成两棵验证树。test_3d_reduced_diags 走 analysis_reduced_diags_impl.py：它不是只看 reduced text 文件能不能写出来，而是从 full plotfile 重新计算 FieldEnergy、ParticleEnergy、FieldMomentum、ParticleMomentum、FieldMaximum、RhoMaximum 和 parser 驱动的 FieldReduction，再与 EF/EP/PF/PP/MF/MR/NP/FR_*/Edotj.txt 逐项对照。除 field energy 因 staggered-vs-cell-centered 差异放宽到 0.3 之外，其余量默认要求 $1e-12$ 。因此这条 regression 真正验证的是 compact observable 的物理定义和 full-state reference 一致性。

与之并列的 test_3d_reduced_diags_load_balance_costs_* 则完全不是物理场解对照。analysis_reduced_diags_load_balance_costs 根本不读 plotfile，而是直接从 LBC.txt 重建每个 rank 的总成本，再只断言 load balance 前后的效率满足 `efficiency_before < efficiency_after`。因此这组 tests 真正验证的是 LoadBalanceCosts 是否把并行运行态忠实暴露给 reduced output，而不是某个固定电磁场或粒子分布是否被精确重现。

这里还要特别写清两个当前边界。第一，test_3d_reduced_diags_load_balance_costs_timers_psatd 这个名字在当前本地 checkout 里并没有真的把 solver 切到 psatd；它的 input 仍然只是 `inputs_base_3d + algo.load_balance_costs_update = Timers`。第二，test_3d_reduced_diags_single_precision 当前也只看到 analysis_reduced_diags_impl.py 里预留了 `single_precision=True` 的放宽容差代码路径，但没有看到 active CMake test/input。因此这两条都不能被夸大成当前活跃的强 regression。

这一层补上以后，第 8 章里 diagnostics 的主线已经不只是“有哪些类”，而是能同时回答：

1. 这个 diagnostics 对象属于哪条计算主线；
2. flush 时走哪种 writer；
3. 最终落盘的是 diagnostics 视图、标准化交换格式，还是 restart 运行态。

如果进一步按“落盘后怎么读”来分，这三类 writer 也已经形成稳定分工。plotfile 典型目录是：

```
diag1NNNNNN/  
  Header  
  Level_*/  
  <species>/  
  warpx_job_info  
  WarpXHeader  
  raw_fields/    # optional
```

主读取工具是 yt, WarpX 文档里的标准入口就是：

```
import yt  
ds = yt.load('./diags/plotfiles/plt00000/')
```

如果只是要常规 field/particle 分析、AMR-aware 后处理，plotfile + yt 仍然是最顺手的默认组合。只有当打开 plot_raw_fields = 1 想看原始 staggered 网格时，才需要额外走 Tools/PostProcessing/read_raw_data.py 这条 raw reader。

openPMD 的目录则更扁平，典型是：

```
diag1/  
  paraview.pmd  
  openpmd_%06T.h5|bp5|bp4|json
```

fields/particles 的层级主要在文件内部，而不是目录树展开。读者侧主工具是：

- openPMD-viewer
- openPMD-api

前者适合快速浏览和 Jupyter 交互，后者适合保留完整 metadata、做并行/chunk 读取以及与外部数据生态对接。文档还专门提醒：yt 也能读一部分 openPMD HDF5，但没有 mesh refinement 支持，因此不能把它当成 plotfile 读法的完全替代。

checkpoint 则根本不是“分析目录”，而是 WarpX 自己的 restart contract。其目录会更像：

```
chkNNNNNN/  
  WarpXHeader  
  warpx_job_info  
  Level_*/  
  <species>/  
  <lasers>/
```

里面包含的是 E/B 主寄存器、current、coarse patch、PML、完整 species 和 lasers，以及 distribution mapping 和 reduced diagnostics checkpoint state。它的首要读取者不是 Python 数据分析工具，而是：

```
amr.restart = "../.../chk000XXX"
```

也就是说，checkpoint 的主用途是“接着跑”和“恢复”，不是“直接分析”。这一点和 plotfile/openPMD 必须明确分开。

因此，对输出格式的选择可以直接收敛成三条经验：

1. 想做常规物理分析，优先 plotfile。
2. 想做标准化交换、粒子上附加场、RZ mode 或 richer metadata，优先 openPMD。
3. 想保留完整运行态并支持 restart，只能用 checkpoint。

当前本地 restart/ 目录里还有两条很窄但很有代表性的 PICMI regression，把这三条经验压到了更细的接口层。inputs_test_2d_id_cpu_read_picmi.py 虽然也挂了 checkpoint 组件，但当前强断言其实是脚本内直接读取 pti["idcpu"] 并用 unpack_ids/unpack_cpus 验证粒子标识解包合同；inputs_test_2d_runtime_components_picmi.py 则把 picmi.Checkpoint(...), amr.restart=... 参数解析和动态 newPid runtime component 放在一起，证明 checkpoint front-end 接线与 runtime-attribute 写入合同可以共存，但对应的 test_2d_runtime_components_picmi_restart 仍是 FIXME scaffold。这说明 checkpoint/PICMI 这条线已经有最小 regression 证明“前端能接上”，但还没有把“restart 后动态 runtime attrs 仍完全一致”升级成活跃强断言。

BackTransformed diagnostics 还有一条当前本地很值得保留的 RZ 强基准: Examples/Tests/btd_rz/。它不是只检查 RZ BTD 目录结构,而是从 back_rz openPMD 文件读取 back-transformed 轴上场剖面,直接拟合 boosted-frame Gaussian laser 还原到 lab frame 后的振幅、波长、包络持续时间和相位中心。因此这组例子说明: RZ BackTransformed diagnostics 已经不仅有 writer 合同,还有明确的物理重建合同。

checkpoint/restart 这条线也有两类当前本地很值得区分的最小基准。test_3d_acceleration / test_3d_acceleration_restart 是最严格的一类: analysis 逐字段比较 restart 与非 restart plotfile, 要求最大相对误差低于 $1e-12$, 因此它真正验证的是 acceleration 基线上的 restart 可重复性, 而不是某个独立“加速物理”现象。test_3d_eb_picmi 则更像一条前端 scaffold: 它把 PICMI、embedded boundary、checkpoint 与 amr.restart=... 放进同一个最小脚本中, 但当前活跃 test 仍主要依赖 checksum, 显式 restart 变体还停在 FIXME。因此这条线目前证明的是“EB + PICMI + checkpoint 配线能接上”, 而不是“EB restart 后所有状态都已有独立强断言覆盖”。

这条 restart_eb 边界还需要再强调一层: 目录里虽然已经放着 analysis_default_restart.py, 而且注释掉的 test_3d_eb_picmi_restart 也明确准备好了

```
amr.restart = "../test_3d_eb_picmi/diags/chk000030"
```

与逐字段 restart 对照的调用方式, 但当前活跃注册仍只有 test_3d_eb_picmi 本体, 且 analysis = OFF。因此在正文里更准确的说法应是:

1. restart_eb 已经有完整的“未来强 restart regression”脚手架;
2. 当前活跃 CI 仍只证明 EB + PICMI + checkpoint 输出链稳定;
3. 还不能把这条线表述成“EB restart 已完成 field-level reproducibility 验证”。

同一个 restart/ 目录里还有一条更细、但很容易被误归到“纯 PSATD benchmark”的分支: test_3d_acceleration_psatd*。这些输入并不是单独拿出来验证谱色散关系, 而是把同一条 3D boosted acceleration workflow 切到:

- algo.maxwell_solver = psatd
- psatd.use_default_v_galilean = 1

并在另一支里再额外打开:

- psatd.do_time_averaging = 1

然后继续复用同一个 analysis_default_restart.py 做逐字段 restart 对照。也就是说, 这几条回归真正证明的是:

1. PSATD + Galilean acceleration workflow 的 checkpoint/restart 可重复性;
2. time-averaged PSATD update 打开后, 同一 workflow 的 restart 可重复性。

它们不是新的色散理论强基准, 而是“更复杂 solver path 仍能完整写盘并无漂移恢复”的 diagnostics/restart 合同。

BoundaryScrapingDiagnostics 与 Python scraped-particle buffer

边界诊断是这一章里一个容易误解的特殊分支。它不是普通 full diagnostics 的“少画几列字段”, 而是单独的 diag_type=BoundaryScraping。

MultiDiagnostics.cpp 读到这个 diag_type 时, 会直接构造:

```
alldiags[i] = std::make_unique<BoundaryScrapingDiagnostics>(i, diags_names[i], diags_types[i]);
```

而 BoundaryScrapingDiagnostics::ReadParameters() 又立刻把默认 field 输出关掉:

```
m_varnames_fields = {};  
m_varnames = {};  
m_num_buffers = AMREX_SPACEDIM*2;  
if (eb_enabled) { m_num_buffers += 1; }
```

这说明它的输出对象不是普通场变量, 而是每个边界各自那份 ParticleBoundaryBuffer。

进一步看 InitializeParticleBuffer(), 它对每个 species、每个 boundary 都直接取:

```
WarpXParticleContainer::Base* bnd_buffer =  
    particle_buffer.getParticleBufferPointer(species_name, i_buffer);  
m_output_species[i_buffer].push_back(ParticleDiag(m_diag_name, species_name, pc, bnd_buffer));
```

所以 BoundaryScrapingDiagnostics 不是重新扫主粒子容器, 而是直接消费前面边界处理阶段已经收集好的 scraped event。

这个 diagnostics 目前还有两个硬边界:

- 必须编译 openPMD;
- <diag>.format 必须是 openpmd。

写出时, Flush(i_buffer) 会把每个边界单独写到:

```
const std::string file_prefix =  
    m_file_prefix + "/particles_at_" + particle_buffer.boundaryName(i_buffer);
```

因此目录天然会分成 particles_at_xlo、particles_at_zhi、particles_at_eb 等子目录。更关键的是, 写完后它立即:

```
particle_buffer.clearParticles(i_buffer);
```

也就是说, BoundaryScrapingDiagnostics 对 ParticleBoundaryBuffer 是“写出并消费”的语义, 而不是只读观察。

Python 接口走的是同一份底层状态。Source/Python/WarpX.cpp 只是把 WarpX 单例里的:

```
wx.GetParticleBoundaryBuffer()
```

直接暴露给 sim.extension.get_particle_boundary_buffer()。高层 ParticleBoundaryBufferWrapper 再把它封成:

- get_particle_boundary_buffer_size(...)
- get_particle_boundary_buffer(...)
- get_particle_scraped_this_step(...)
- clear_buffer()

其中 get_particle_scraped_this_step() 并不是单独的“本步队列”, 它只是用 stepScraped == getistep(level) 对累计 buffer 做一次筛选。官方 Python 文档也明确要求用户手动 clear_buffer(), 否则内存会持续增长。

因此, 边界 scraped particle 的消费侧必须记住三个事实:

1. BoundaryScrapingDiagnostics 只写粒子, 不写场, 而且当前只支持 openPMD。
2. Python wrapper 和 diagnostics 共用同一个 ParticleBoundaryBuffer, 不是两份独立副本。
3. diagnostics flush 后会自动清空对应 boundary buffer; Python 路径则需要用户自己清空。

这一点对二次发射、探测器统计和边界通量诊断尤其关键, 因为它决定了“什么时候读到哪些粒子”, 本质上取决于 buffer 的消费时机, 而不只是取决于边界物理本身。

固定模板案例页

为了避免 diagnostics 章节一直停留在“原理说明”, 现在把四类最常用输出都压成同一模板:

1. 最小输入片段。
2. 典型目录树。
3. 读取入口。
4. 适用场景。

模板 A: plotfile

最小输入片段:

```
diagnostics.diams_names = diag1
```

```
diag1.diag_type = Full  
diag1.intervals = 10  
diag1.fields_to_plot = Ex Ey Ez Bx By Bz rho  
diag1.write_species = 1
```

典型目录树:

```
diags/diag1/  
  diag1000000/  
    Header  
    Level_0/  
    <species>/  
    warpx_job_info  
    WarpXHeader  
    raw_fields/    # optional
```


读取入口:

```
import yt
ds = yt.load("./diags/diag1/diag1000000/")
```

适用场景:

- 常规 field/particle 分析。
- 需要 yt 的 AMR-aware 工作流。
- 需要额外读取 raw_fields/ 做 staggered-grid 调试。

模板 B: openPMD

最小输入片段:

```
diagnostics.diags_names = diag1

diag1.diag_type = Full
diag1.format = openpmd
diag1.openpmd_backend = h5
diag1.intervals = 10
diag1.fields_to_plot = Ex Ey Ez Bx By Bz rho
diag1.write_species = 1
```

如果需要 writer 阶段再把场 gather 到粒子:

```
diag1.plot_phi = 1
diag1.plot_E = 1
diag1.plot_B = 1
```

典型目录树:

```
diags/diag1/
  paraview.pmd
  openpmd_000000.h5
  openpmd_000010.h5
```

读取入口:

```
from openpmd_viewer import OpenPMDTimeSeries
ts = OpenPMDTimeSeries("./diags/diag1/")
```

适用场景:

- 标准化数据交换。
- richer metadata 或 openPMD 生态工具链。
- full diagnostics 下的 phi / E / B on particles。

模板 C: checkpoint

最小输入片段:

```
diagnostics.diags_names = chk

chk.diag_type = Full
chk.format = checkpoint
chk.intervals = 100
```

重启入口:

```
amr.restart = ./diags/chk/chk000100
```

典型目录树:

```
diags/chk/
  chk000100/
    WarpXHeader
    warpx_job_info
    Level_0/
    <species>/
    <lasers>/
```

读取入口:

首要读取者不是 Python, 而是 WarpX 本体的 restart。

适用场景:

- 中断后续跑。
- 完整运行态持久化。
- reduced diagnostics 的 checkpoint state 恢复。

模板 D: BoundaryScraping/openPMD

本地最清楚的真实骨架来自 thomson_parabola_spectrometer:

```
diagnostics.diagnostics_names = screen

screen.diag_type = BoundaryScraping
screen.format = openpmd
screen.intervals = 1

hydrogen1_1.save_particles_at_zhi = 1
carbon12_6.save_particles_at_zhi = 1
carbon12_4.save_particles_at_zhi = 1
```

典型目录树:

```
diags/screen/
  particles_at_zhi/
    paraview.pmd
    openpmd_000001.h5
    openpmd_000002.h5
    ...
```

如果是 embedded boundary scraping, 则目录会变成 particles_at_eb/。

读取入口:

```
from openpmd_viewer import OpenPMDTimeSeries
series = OpenPMDTimeSeries("./diags/screen/particles_at_zhi/")

point_of_contact_eb/analysis.py 也用同一路径读取:

ts_scraping = OpenPMDTimeSeries("./diags/diag2/particles_at_eb/")
```

适用场景:

- 探测器或屏幕 hit 记录。
- 吸收边界通量统计。
- EB 接触点位置和法向验证。
- 需要把 scraped particles 落成持久文件而不是只在 Python callback 里临时消费。

和前三类相比, 这一类还要额外记住:

1. 只支持 openPMD。
2. 只写粒子, 不写场。
3. species 必须先开 save_particles_at_xlo/.../eb。
4. writer filter 在 flush 时生效, 因此 plot_filter_function 里的 t 是写出时间, 不是撞边界时间。

thomson_parabola_spectrometer 还需要再往前走一步理解。它不只是 BoundaryScraping/openPMD 的模板例子，也是 active 强 analysis: CMakeLists.txt 同时运行 analysis.py 和 checksum helper。analysis.py 会从 screen/particles_at_zhi/ 读取 detector hits，再从 diag0 的初始 full diagnostic 读取 uz/id/mass，按粒子 id 回连出每个 detector hit 对应的初始能量，最终在 screen 平面上重建按 species 和入射能量着色的离子分离图。因此这组例子真正验证的是：

1. BoundaryScraping 在 zhi 屏面的 detector-hit 持久化
2. openPMD 读取键
3. id 跨 diagnostics 回连
4. 解析 E_x/B_x 场驱动下的 test-particle TPS optics

所以它不应再被归到笼统的 PEC / conducting boundary 桶里。

这样整理后，第 8 章里关于 writer 的内容已经不再只是抽象分类，而是能直接给出“要什么输出，就怎么写输入、怎么找目录、怎么读文件”。

Python 边界 buffer 的最小消费模板

如果不想先把 scraped particles 写成 openPMD，而是直接在 Python 里消费 ParticleBoundaryBuffer，本地 examples 说明这条路也已经很稳定，而且至少有两种典型模式。

模式 A：运行结束后统一检查

particle_boundary_scrape 给的是最小自检骨架。species 只要打开：

```
electrons = picmi.Species(
    ...,
    warpx_save_particles_at_xhi=1,
    warpx_save_particles_at_eb=1,
)
```

模拟结束后直接：

```
from pywarpx import particle_containers
```

```
particle_buffer = particle_containers.ParticleBoundaryBufferWrapper()
n = particle_buffer.get_particle_boundary_buffer_size("electrons", "eb")
weights = particle_buffer.get_particle_boundary_buffer("electrons", "eb", "w", 0)
total_weight = sum(w.sum() for w in weights)
particle_buffer.clear_buffer()
```

这里三个实现细节必须记住：

1. get_particle_boundary_buffer_size() 给的是累计到当前时刻的 scraped 数，不是本步增量。
2. get_particle_boundary_buffer() 返回的是“每个 tile 一条数组”的列表，不会自动拼平成一块大数组。
3. Python 路径不会自动清空 buffer，必须自己 clear_buffer()。

模式 B：callback 或在线控制里的即时事件流

spacecraft_charging 给的是“文件化 writer 和 Python 在线消费并存”的例子。它一边开：

```
part_scraping_boundary_diag = picmi.ParticleBoundaryScrapingDiagnostic(
    name="diag2",
    period=-1,
    species=[electrons, protons],
    warpx_format="openpmd",
)
```

一边又在 Python 里直接读同一个 EB buffer：

```
particle_buffer = ParticleBoundaryBufferWrapper()
weights = particle_buffer.get_particle_boundary_buffer(species, "eb", "w", 0)
sum_weights_over_tiles = sum([w.sum() for w in weights])
ntot = float(mpi.COMM_WORLD.allreduce(sum_weights_over_tiles, op=mpi.SUM))
```

这里它把 `period=-1` 设成只在末尾 flush 一次，目的就是在模拟中途保留完整 in-memory buffer，供 Python 侧累计计算 spacecraft 收集到的净电荷。

它的 analysis 还会继续从 full diagnostics 里抽取每个输出步的最小势值，拟合

$$\phi(t) = v_0 (1 - e^{-t/\tau}),$$

并要求拟合得到的 `v0` 和 `tau` 分别落在 4% 与 20% 的容差内。于是这组例子在第 8 章里最准确的定位就不是“演示 Python 能访问 buffer”，而是：

- boundary buffer 在线消费
- Python 动态改写 EB potential
- 以及 electrostatic diagnostics 最终能否给出正确的 charging 时间尺度

三者联动的应用级 regression。

如果逻辑只想处理“本步刚撞边界的粒子”，更直接的高层接口是：

```
weights_this_step = particle_buffer.get_particle_scraped_this_step(
    "electrons", "eb", "w", 0
)
```

它本质上只是按 `stepScraped == current_step` 对累计 buffer 做过滤，但这已经足够支撑：

- secondary emission;
- 每步边界通量统计;
- callback 驱动的边界反应模型。

这和 `BoundaryScrapingDiagnostics` 的分工

现在 diagnostics 和 Python 两条路径的边界已经可以写得很清楚：

1. `BoundaryScrapingDiagnostics` 负责文件化持久输出，flush 后自动清空对应 boundary buffer。
2. `ParticleBoundaryBufferWrapper` 负责 Python 即时访问，默认不会自动清空。
3. 两者共用同一份 `ParticleBoundaryBuffer`，不是两份副本。

因此，如果 diagnostics 设成频繁 flush，Python 侧看到的累计事件就会被 writer 周期性截断；如果 diagnostics 只在末尾 flush，甚至根本不开 writer，Python 才能把它当成长时间累积的事件池来用。

Python field wrapper 的最小强回归

`python_wrappers/inputs_test_2d_python_wrappers_picmi.py` 覆盖的是另一条 Python 接口线。它既不是粒子 callback，也不是单纯 writer smoke test，而是直接验证：

- `sim.fields.get(...)`
- `MultiFabRegister`
- `valid-domain` 字段
- PML split fields
- `divergence-cleaning` 标量

能否在 Python 侧被稳定访问。

这条 regression 在 `CMakeLists.txt` 里没有独立 `analysis.py`：

```
add_warp_x_test(
    test_2d_python_wrappers_picmi
    ...
    OFF
    "analysis_default_regression.py --path diags/diag1000100"
    OFF
)
```

所以如果只看 CMake，会误以为它只是 checksum-only。实际上强断言全部写在 input 脚本内部。脚本在

```
sim.initialize_inputs()
sim.initialize_warpx()
```

之后，直接抓出：

```
Ex = sim.fields.get("Efield_fp", dir="x", level=0)
...
Expml = sim.fields.get("pml_E_fp", dir="x", level=0)
...
Fpml = sim.fields.get("pml_F_fp", level=0)
Gpml = sim.fields.get("pml_G_fp", level=0)
```

然后给 valid domain 里的 E/B/F/G 填一个平滑 unit pulse，推进 100 步，最后不是看图片，而是逐分量检查 benchmark：

```
def check_values(benchmark, data, comp, rtol, atol):
    passed = np.allclose(
        benchmark, np.sum(np.abs(data[(), (), comp])), rtol=rtol, atol=atol
    )
    assert passed
```

这里 `data[(), (), comp]` 的意思也很关键：

- `()` 取 valid + ghost 全范围；
- `comp` 用来访问 PML split-field 的不同 component。

脚本会依次断言：

- Ex/Ey/Ez
- Bx/By/Bz
- F/G
- pml_E_fp
- pml_B_fp
- pml_F_fp
- pml_G_fp

的每个相关 component 都与固定 benchmark 一致，而且若干应为零的 component 也显式要求保持为零。于是这条 test 的真实定位应当是：

- Python field wrapper / PML split-field access

而不是过粗的 Python API / callbacks。它验证的是 `pywarpx` 经由 `MultiFabRegister` 暴露出来的非 owning `MultiFab` 视图，在 valid domain、ghost cell、PML 和 cleaning 字段这几层上都没有被包装错。

本地运行注意

本机过去运行短 WarpX 案例时，MPI/OFI 路径可能受网络接口影响；小规模复现实验可优先设置：

`FI_PROVIDER=tcp`

这不是物理参数，只是本机 MPI transport 的稳定性处理。正式实验记录必须把环境变量、binary 路径、输入文件、输出目录和分析脚本写入对应章节。

2026-07-11 reader-side 验证增量

本轮把本章末尾的两个后续任务推进到真实运行产物：

1. `runs/stage-c-validation/langmuir_frequency_fit/` 使用同一份 Langmuir 官方输入，只把 `diag1/openpmd.intervals` 从 40 改为 1，重新运行 80 步，得到 81 个逐步快照；
2. `scripts/analyze_langmuir_frequency_fit.py` 对 Ez 的目标空间模做投影，并用两正交分量拟合时间频率，同时逐快照计算 `divE-rho/epsilon_0`；
3. `scripts/analyze_uniform_plasma_conservation.py` 用 `yt` 读取 `uniform-plasma` 初末 `plotfile`，统计粒子数、粒子总权重、场能、粒子动能和总能量。

Langmuir 结果记录在 `runs/stage-c-validation/langmuir_frequency_fit/langmuir-frequency-fit.md`：

- 81 个快照；
- 解析 `omega_p=1.128292045086e14`，拟合值为 `1.128697661742e14`；

- 相对频率误差 $3.595\text{e-}4$;
- 官方 `analysis_1d.py` 在同一族运行上给出最大相对误差 $1.70\text{e-}3$ 、最终 `divE-rho/epsilon_0` 误差 $8.35\text{e-}12$ ，均通过原始阈值;
- 逐步 reader-side 扫描的最大守恒误差为 $3.149\text{e-}10$ ，发生在中间快照，因此不能把“每一步都满足官方 $1\text{e-}11$ 阈值”写成已验证事实。

Uniform-plasma 结果记录在 `runs/stage-c-validation/uniform_plasma_2d/uniform-plasma-conservation.md`:

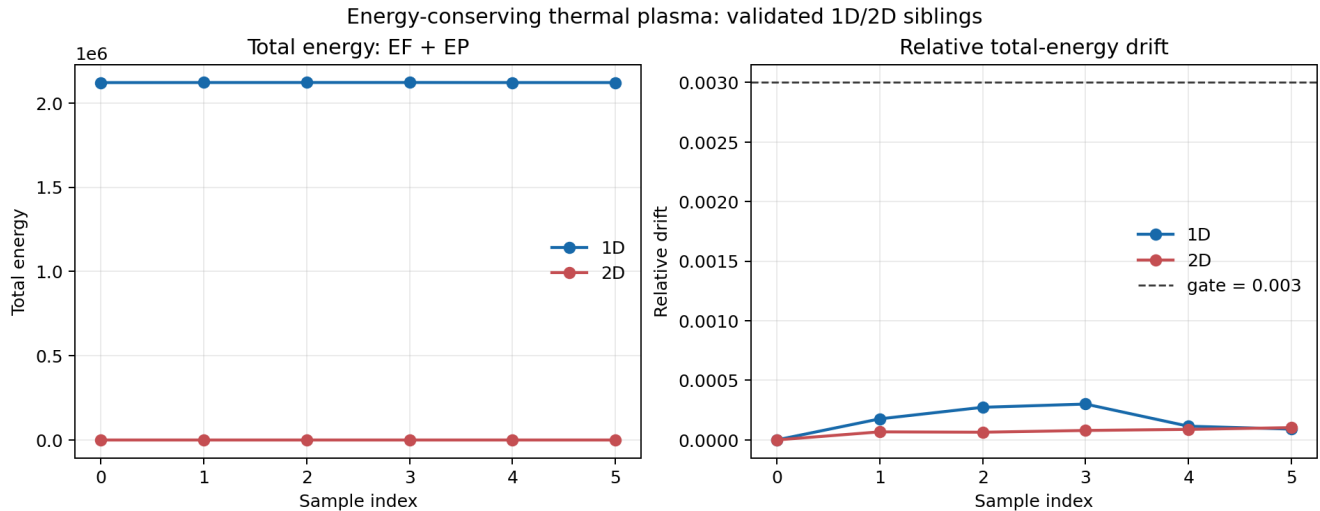
- 初末粒子数均为 65536;
- 粒子总权重相对变化为 0;
- 初始场能量为零，所以场能相对变化率有意标记为 `undefined (zero baseline)`;
- 粒子动能变化约 7.06%，总能量变化约 1.97%。

随后又把同一输入副本延长到 100 步，并每 10 步写出一个 `plotfile`；长时间序列报告位于 `runs/stage-c-validation/uniform_plasma_2d_long/uniform-plasma-conservation.md`。粒子总权重在全部 11 个快照中保持不变，末态总能量相对初态变化 $1.387\text{e-}2$ ，时间序列中的最大绝对相对偏差为 $2.518\text{e-}2$ 。这比单看 10 步终点更能说明当前 `workflow` 的短时热背景统计范围，但仍不足以构成热平衡能量守恒 `gate`；后者应与 `energy_conserving_thermal_plasma` 的专门 `analysis` 合同绑定。

这条专门合同已经在本轮真实运行中闭合。`runs/stage-c-validation/energy_conserving_thermal_plasma_2d/` 使用官方 2D 输入运行 500 步，产生 `EF.txt` 和 `EP.txt` 六个 reduced-energy 样本；官方 `Examples/Tests/energy_conserving_thermal_plasma/analysis_2d.py` 通过，新增的 `scripts/analyze_energy_conserving_thermal_plasma.py` 也复现同一 `EF+EP` 计算，得到最大总能量相对漂移 $1.031\text{e-}4$ ，低于官方 $3.000\text{e-}3$ 阈值。由此可以把两类证据明确分开：`uniform-plasma` 负责粒子数、I/O 和热背景 `workflow` 统计，`energy-conserving-thermal-plasma` 才负责 `energy-conserving gather` 的强能量漂移 `gate`。

同一官方 family 的 1D sibling 也完成了复现：`runs/stage-c-validation/energy_conserving_thermal_plasma_1d/` 的 500 步运行和官方 `analysis` 均通过，最大漂移为 $3.009\text{e-}4$ 。`scripts/compare_energy_conserving_thermal_plasma_family.py` 将 1D/2D 两份报告汇总为 `runs/stage-c-validation/energy_conserving_thermal_plasma_family.md`，两者共享 `EF+EP`、0.003 阈值和 6 个采样点的验证合同，但不把两种几何的物理轨迹误写成数值等价。

图 8-2 把这两份 JSON 报告中的 `EF+EP` 和归一化漂移放在同一张图里。左图保留不同几何的能量尺度差异，右图直接与共同的 0.003 `gate` 对照；因此读者可以同时看到“能量在场能与粒子能之间交换”和“总能量误差仍被 `gate` 约束”这两个不同层次。图表由 `scripts/plot_energy_conserving_thermal_plasma.py` 从 `case-local JSON` 重新生成。



2026-07-12: reduced diagnostics 与 full-state reference 对照

本轮按官方 2-rank 配置真实运行 `Examples/Tests/reduced_diags/inputs_test_3d_reduced_diags`，末态为 `diags/diag1000200`。官方 `analysis_reduced_diags.py` 从该 `plotfile` 重新计算粒子能量/动量、场能/动量、场最大值、`rho` 最大值、粒子数以及 `FR_Max/FR_Min/FR_Integral/Edotj` 等 `parser-driven FieldReduction`，再逐项与 `EP/EF/PP/PF/MF/MR/NP/FR_*/Edotj.txt` 对照。

共比较 60 个 reduced observable，官方 `analysis` 通过。除 field energy 外，最大相对误差为 $4.125\text{e-}13$ ；field energy 的相对误差为 $2.483\text{e-}1$ ，仍低于官方为 staggered Yee reduced energy 与 cell-centered `plotfile` reference 设置的专用 0.3 容差。项目内 `scripts/analyze_reduced_diags_contract.py` 保存了官方 `analysis` 的逐项摘要，报告位于 `runs/stage-c-validation/reduced_diags_3d_mpi2/reduced-diags-contract.md`。

这条证据的准确含义是“compact reduced observable 与 full-state reference 的定义和 writer 输出一致”，不是说 60 个量都构成独立物理守恒定律。尤其 field energy 的 24.8% 误差必须保留其 staggered/cell-centered 离散表示边界，不能被误读成普通量的数值精度。

图 8-4 将这两个误差层分开画出：左图在对数尺度下显示 59 个非 field-energy observable 的排序误差，右图单独显示 staggered field-energy 的 0.2483 误差与 0.3 专用 gate。这样图表本身就保留了“普通量接近机器精度、field energy 仍有离散表示差异”的证据结构，而不是只给出一个混合最大值。

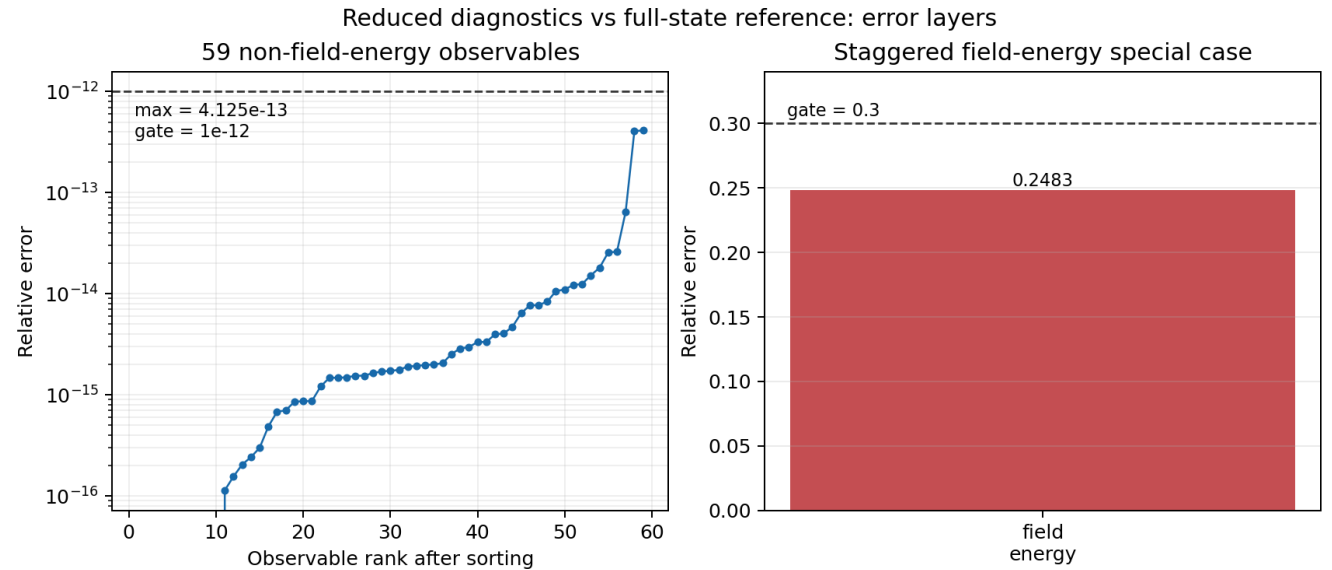


图 8-4 由 scripts/plot_reduced_diags_error_layers.py 从 runs/stage-c-validation/reduced_diags_3d_mpi2/reduced-d 重新生成。

LoadBalanceCosts: 性能诊断的 efficiency gate

同一 reduced_diags family 还包含一条不读取 plotfile 的性能诊断分支: inputs_base_3d 使用 128 x 32 x 128 网格、16 个 box、algo.load_balance_intervals = 2, 并由 LoadBalanceCosts 将每个 box 的 cost、rank、level、几何位置和粒子数写入 LBC.txt。官方 analysis 按 rank 汇总 box cost, 再计算

$$\eta = \frac{1}{N_{\text{rank}}} \sum_r \frac{C_r}{\max_{r'} C_{r'}}.$$

它比较第 1 行 (load balance 前) 与第 2 行 (load balance 后), 要求 eta_after > eta_before。本地按官方 2-rank 配置分别运行 Heuristic 与 Timers 两个 sibling:

cost source	before	after	result
Heuristic	0.625252	1.000000	PASS
Timers	0.744780	0.996162	PASS

报告位于 runs/stage-c-validation/load_balance_costs_heuristic_mpi2/load-balance-costs.md 和 runs/stage-c-validation/load_balance_costs_timers_mpi2/load-balance-costs.md。因此 LoadBalanceCosts 的强合同不是“有一个 LBC 文本文件”，而是它能把 box-level cost 通过 MPI 汇总成 rank-level efficiency, 并观察到重分配后的效率改善；这属于性能/并行态验证，不应与 reduced_diags 的 compact physical observable 对照混成同一类 gate。

图 8-6 将两条 cost source 的 rank-level efficiency 直接画成 before/after 对照。Heuristic 从 0.625252 提升到 1.0, Timers 从 0.744780 提升到 0.996162；图表表达的是负载重分配后的性能改善，不是场或粒子物理精度。

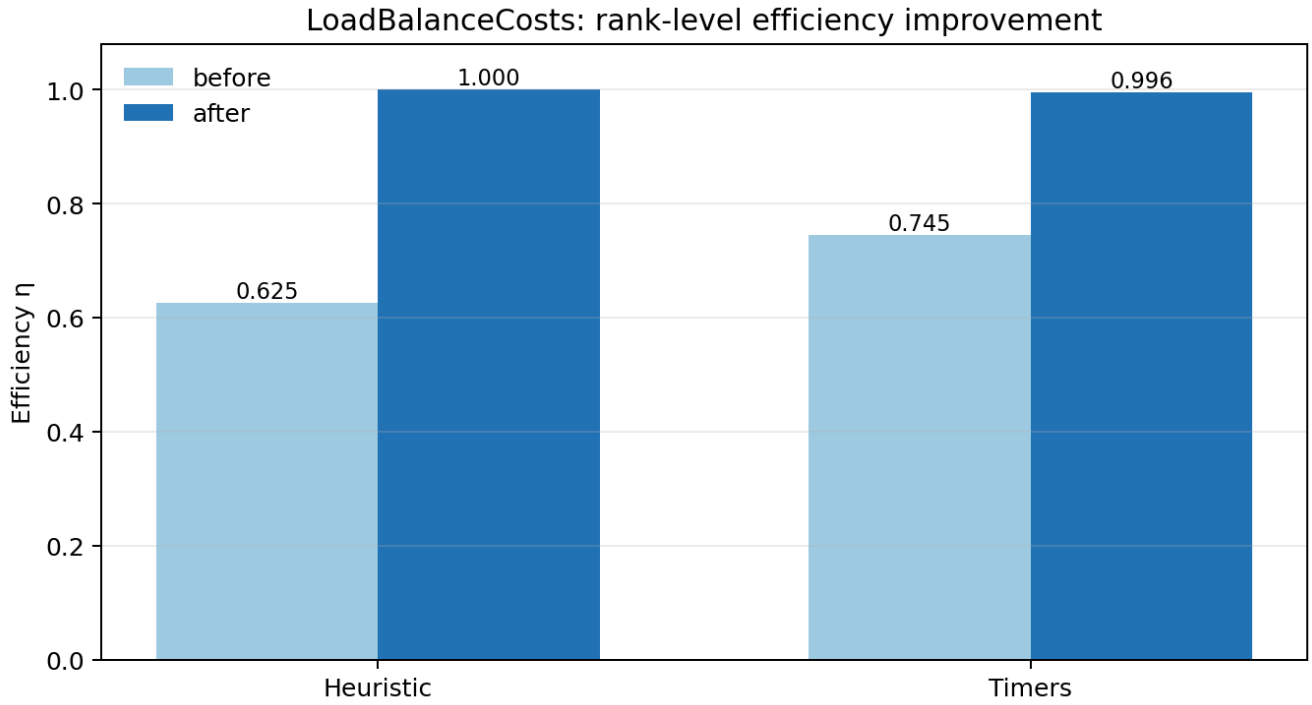


图 8-6 由 `scripts/plot_load_balance_efficiency.py` 从两份 `load-balance-costs.json` 重新生成。

ColliderRelevant: 束流统计与 luminosity-rate 聚合合同

`Examples/Tests/collider_relevant_diags/` 提供了另一条强 reduced-diagnostics regression。它在同一 3D、2-rank 输入中同时输出 `ColliderRelevant_beam_e_beam_p`、`ParticleExtrema_beam_e/beam_p` 和 openPMD full state。官方 `analysis.py` 使用输入中的三个解析宏粒子样本，逐项检查每个 beam 的 `chi_min/max/ave`、位置均值/标准差和 `theta_x/theta_y` 的 `min/ave/max/std`，并把 `ParticleExtrema` 与 `ColliderRelevant` 交叉核对；随后从 `rho_beam_e`、`rho_beam_p` 和粒子电荷重建

$$\frac{dL}{dt} = 2c \Delta V \sum_i \frac{\rho_{e,i}}{q_e} \frac{\rho_{p,i}}{q_p}.$$

本地运行归档于 `runs/stage-c-validation/collider_relevant_diags_3d_mpi2/`：两个 openPMD iteration 都得到 $dL/dt = 4.42662301265625e8$ ，与 reduced text 的对应两行完全一致；`ColliderRelevant` 为 2 行/33 列，两个 `ParticleExtrema` 文件各为 2 行，官方 `analysis` 与项目内 `scripts/analyze_collider_relevant_contract.py` 均通过。该结果验证的是 collider-oriented quantity 的定义、统计和聚合 writer 合同，不等于已经完成 `diff_lumi_diag` 的解析谱 benchmark，也不等于 `beam_beam_collision` 的 QED 应用级物理复现。

图 8-7 将两个 openPMD iteration 的 dL/dt 交叉结果叠加显示。两个 reader-side reconstruction 点与 `ColliderRelevant` reduced 点完全重合，且 JSON 报告给出的相对误差均为 0；这张图验证的是聚合定义和 writer/reader 对齐，不是 luminosity 随束流演化的独立动力学 benchmark。

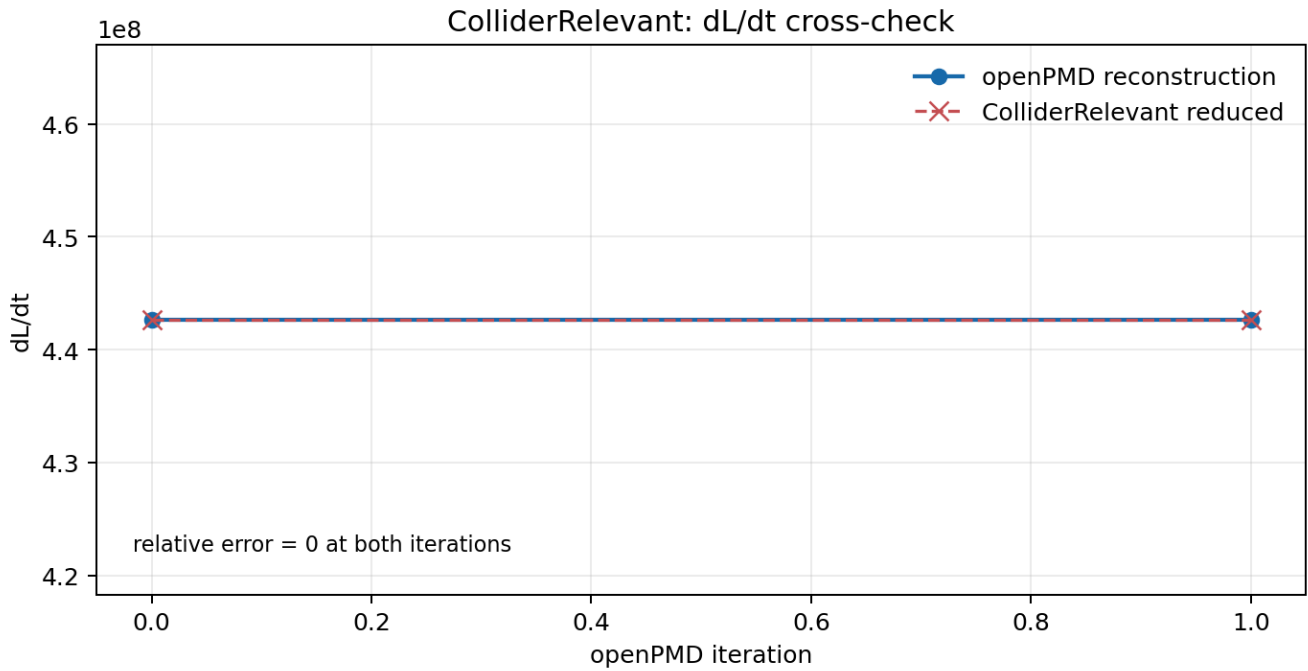


图 8-7 由 `scripts/plot_collider_luminosity_consistency.py` 从 `collider-relevant-contract.json` 重新生成。

DifferentialLuminosity: 1D/2D 解析谱与 AMR 对照

`Examples/Tests/diff_lumi_diag/` 把 reduced diagnostics 推进到能量微分 luminosity 的解析 benchmark。共享的 `inputs_base_3d` 设定两束相向高斯束，1D `DifferentialLuminosity` 输出总能量谱，2D `DifferentialLuminosity2D` 输出两个入射束能量的二维网格；官方 `analysis.py` 分别用高斯束解析式比较末态 1D 与 2D 结果。这个分析依赖 `openpmd_viewer` 读取 2D openPMD series，而不是把二维数据误当作普通文本列。

本地按官方 2-rank 配置完成三组 sibling，三组都在 step 80 输出 128 个 1D 能量 bin 和 128×128 的 2D 网格：

case	AMR max level	1D error / tolerance	2D error / tolerance	result
leptons	0	0.8903% / 2.0%	2.6890% / 4.0%	PASS
leptons + AMR	1	0.9796% / 2.0%	3.0042% / 4.0%	PASS
photons	0	2.0119% / 2.1%	4.9327% / 6.0%	PASS

报告位于 `runs/stage-c-validation/diff_lumi_diag_leptons_mpi2/diff-lumi-contract.md`、`diff_lumi_diag_leptons_mr_m` 和 `diff_lumi_diag_photons_mpi2/diff-lumi-contract.md`，项目脚本为 `scripts/analyze_diff_lumi_contract.py`。这组结果补上了当前束流诊断链中此前缺少的解析谱 physics gate；AMR sibling 的意义是验证中心细化区域仍能保持相同的 reduced luminosity 定义，而不是宣称 AMR 与 uniform-grid 轨迹逐点相同。

图 8-5 将三组 sibling 的 1D/2D 相对误差和各自 gate 并列展示。photons 使用报告中独立的 2.1%/6.0% 容差，不能直接套用 leptons 的阈值；三组柱状值均低于对应虚线 gate，图表只表达解析谱误差合同，不把 reduced writer 的文件形状误写成额外物理结论。

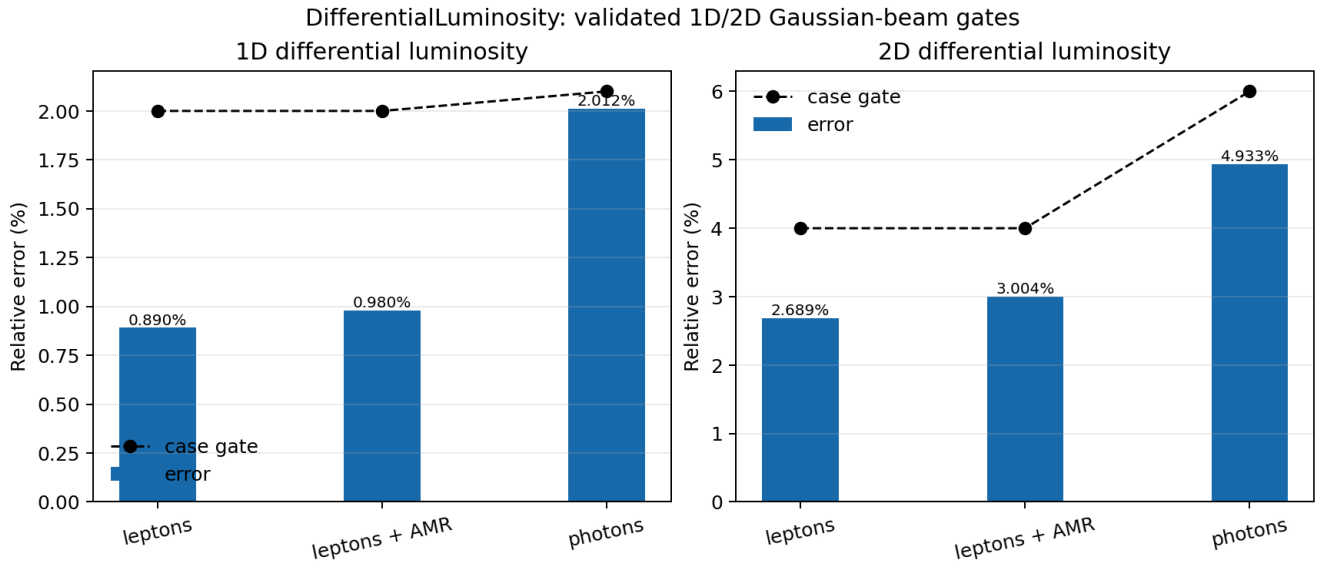


图 8-5 由 `scripts/plot_diff_lumi_errors.py` 从三份 `diff-lumi-contract.json` 重新生成。

ParticleHistogram2D: 二维 openPMD writer 合同

当前 checkout 没有单独的 ParticleHistogram2D CMake regression; 本地采用 `Examples/Physics_applications/laser_ion/` 的官方 2-rank application 作为完整 producer。输入同时配置 `PhaseSpaceIons` 和 `PhaseSpaceElectrons` 两个二维 histogram: 两者都使用 `z` 作为 abscissa, `uz` 作为 ordinate, 1000 x 1000 bins, `value_function = w`, 而 `electrons` 还增加 `sqrt(x*x+y*y) < 1e-6` filter。官方 CMake analysis 只负责 time-averaged field 与 instantaneous field 的一致性, 因此不能单独被写成 histogram physics gate; 项目脚本 `scripts/analyze_particle_histogram2d_contract.py` 对 histogram series 做独立 writer 检查。

本地运行归档于 `runs/stage-c-validation/laser_ion_histogram2d_mpi2/`。两个 series 都写出 0 和 100 两个 BP5 iteration, 数据形状均为 1000 x 1000, axis labels 为 `uz/z`, 所有数据有限且存在非零 bin; 官方 time-average analysis 通过。 `PhaseSpaceIons.txt` 与 `PhaseSpaceElectrons.txt` 的大小均为 0, 这是预期结果: `ParticleHistogram2D::WriteToFile()` 绕过基类逐行文本 writer, 直接创建 `reducedfiles/<name>/openpmd_%T.<backend>` series, 因此空 .txt companion 不能被误判成 histogram 丢失。

这条证据的边界也需要保留: 它证明二维 histogram 的配置、openPMD layout、轴元数据和 writer 输出链成立, 不等于已经用独立解析分布证明 laser-ion phase-space 的物理收敛性; 后者仍需要更高分辨率/粒子数以及针对相空间分布的物理参考结果。

在不改变这条边界的前提下, 项目又用 `scripts/analyze_particle_histogram2d_moments.py` 对 BP5 数组做了 reader-side 加权统计。 `PhaseSpaceIons` 的总权重从 iteration 0 的 $3.9975794429219594e18$ 到 iteration 100 的 $3.997579442921919e18$, 相对变化约 $1.0e-14$; `std(z)` 保持在 $1.47204e-6$ m, 而 `std(uz)` 从数值零增长到 $4.78558e-4$ 。受径向 filter 影响的 `PhaseSpaceElectrons` 总权重从 $5.30929e17$ 变为 $5.32861e17$, `std(uz)` 从 0.196998 变为 0.199300 。这些统计量把“相空间图发生了什么变化”从视觉判断推进成了可复现的 weighted-moment 摘要, 但仍不能替代更高分辨率/粒子数的 convergence study。报告位于 `runs/stage-c-validation/laser_ion_histogram2d_mpi2/particle-histogram2d-moments.{json,md}`。

随后又做了一个匹配物理时间的 producer 对照: baseline 使用 384×512 网格, `dt=1.083064693e-16` s, 100 步, refined 使用 768×1024 网格, `dt=5.415323467e-17` s, 200 步; 两者最终时间差只有 $3.31e-29$ s, `scripts/analyze_particle_histogram2d_resolution.py` 对两个 BP5 series 的总权重、`std(z)` 和 `std(uz)` 设置了 $1e-3/1e-2/5e-2$ 的局部稳定性阈值, ions/electrons 均通过: `std(z)` 相对差为 $5.51e-4/2.02e-4$, `std(uz)` 相对差为 $2.04e-3/4.47e-2$ 。这是“网格加密后 reader-side 加权宽度仍稳定”的项目级证据, 不是严格的物理收敛阶证明; 两套 producer 都是单进程, 运行结束时的 OFI MPI_Finalize 环境尾噪声不影响已写出的 BP5 数据读取。

同一脚本还记录了 1×1 particles-per-cell 的负对照。它的 ions 宽度仍接近 baseline, 但 electrons 总权重相对差为 $1.9471e-3$, 超过 $1e-3$ 阈值, 因此 `weighted_width_stability` 被拒绝。这个负结果保留了当前最重要的边界: 网格分辨率敏感性已经有局部通过项, 粒子数统计收敛仍未完成。完整 JSON/Markdown 产物位于 `runs/stage-c-validation/laser_ion_histogram2d_resolution/`。

为判断该负结果是否只是单个低采样点, 又补做了 $1 \times 1/2 \times 2/4 \times 4/8 \times 8$ 四档 particles-per-cell 序列, 并用 `scripts/analyze_particle_histogram2d_p` 做相邻档位比较。 $1 \times 1 \rightarrow 2 \times 2$ 时 electrons 总权重差为 $1.9471e-3$, 稳定性 gate 失败; $2 \times 2 \rightarrow 4 \times 4$ 时降至 $4.2685e-4$, $4 \times 4 \rightarrow 8 \times 8$ 时进一步降至 $3.6534e-4$, ions/electrons 的总权重、`std(z)`、`std(uz)` 均通过局部阈值。这支持“增加粒子数后该 reader-side 统计总体更稳定”的方向性判断, 但它仍是单进程、单一激光离子 case 的局部矩合同, 不足以给出正式收敛阶或上游 regression gate。 8×8 producer 的 MPI 收尾仍出现本机 OFI MPI_Finalize 尾噪声, 但 BP5 输出和独立读取均完整; 趋势报告位于

runs/stage-c-validation/laser_ion_histogram2d_resolution/particle-count-trend-8x8.{json,md}.

图 8-8 直接从 BP5 series 读取 PhaseSpaceIons 与 PhaseSpaceElectrons 的 iteration 0/100 数据，并按每个面板的非零 bin 裁剪显示范围。它展示的是 writer 实际落盘的 $uz-z$ 相空间结构和随 iteration 的变化；由于每个面板使用独立对数颜色归一化，颜色不能用于跨 species 或跨 iteration 的绝对产额比较。完整数组仍为 1000×1000 ，空 .txt sidecar 也仍是预期的 writer 路径边界。

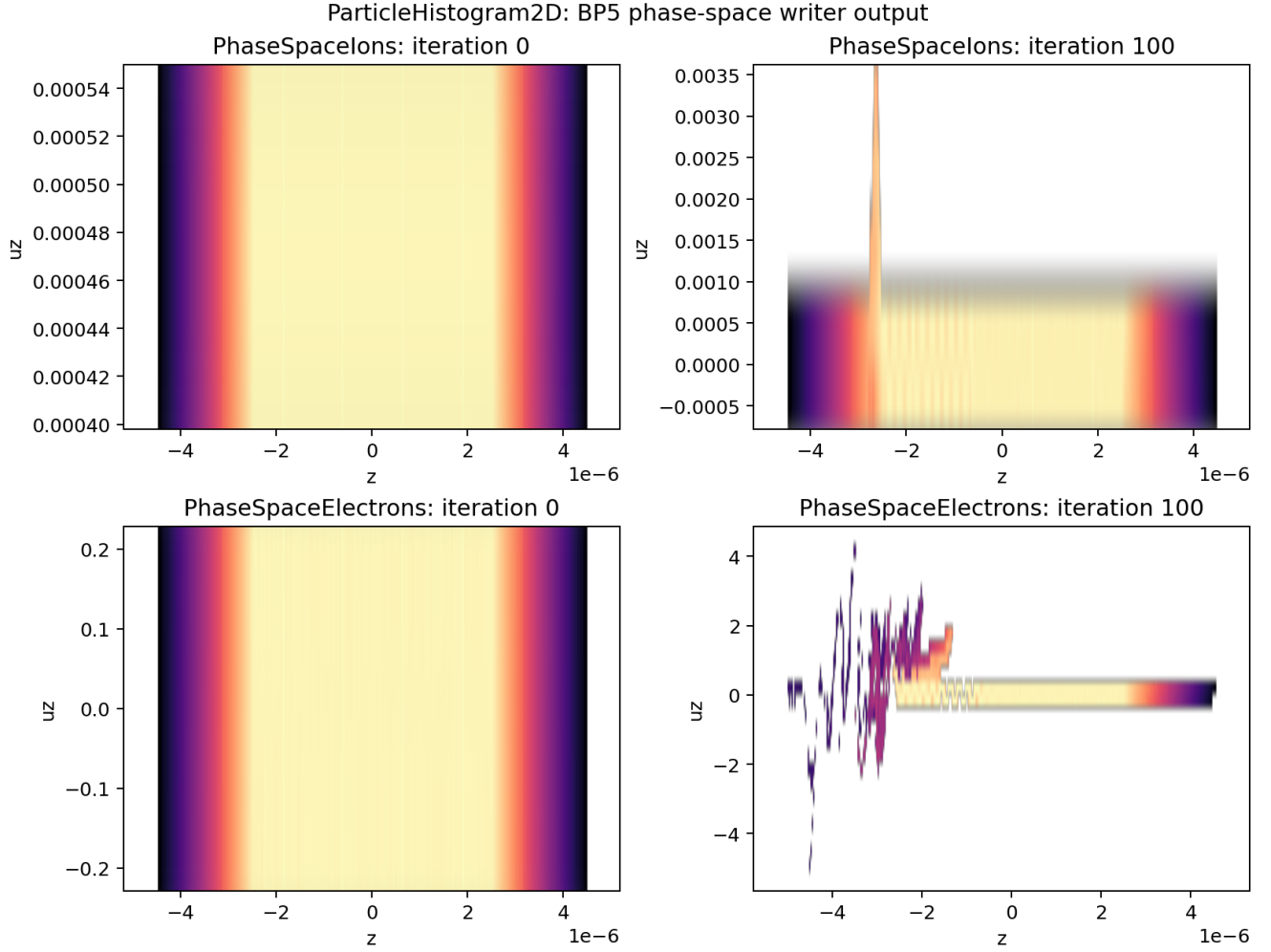


图 8-12 用同一份四档 pairwise contract 把粒子数敏感性归一化到各自局部 gate: 虚线是 gate 边界，纵轴小于 1 表示通过。1x1 $\rightarrow$ 2x2 的电子总权重仍越过边界，2x2 $\rightarrow$ 4x4 和 4x4 $\rightarrow$ 8x8 则落在边界以内；因此图支持“采样增加后局部统计趋稳”，但不把单一 case 的 reader-side 矩合同写成正式收敛阶。

ParticleHistogram2D particle-count sensitivity

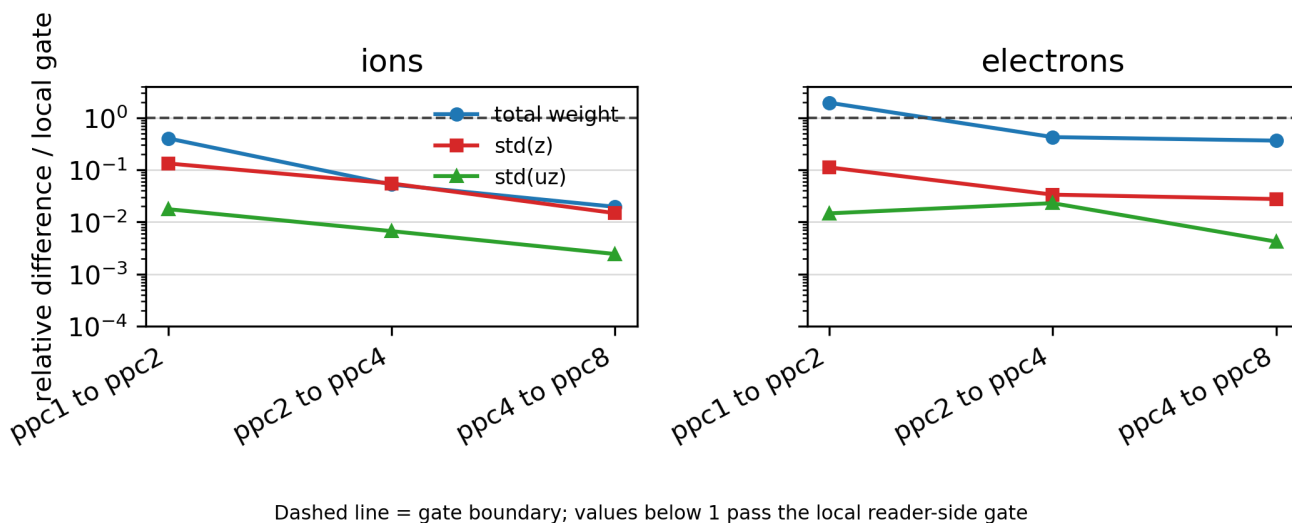


图 8-8 由 `scripts/plot_particle_histogram2d.py` 从 `runs/stage-c-validation/laser_ion_histogram2d_mpi2/` 的 BP5 series 重新生成。

BeamRelevant: 束流矩与截断高斯束合同

BeamRelevant 是文本型束流诊断, 和 **ColliderRelevant** 的逐粒子 `chi/theta` 统计不同。3D 路径固定输出 22 个物理量: 位置与动量均值、`gamma` 均值、位置/动量/`gamma` rms、三方向 emittance、Twiss `alpha/beta` 以及总 `charge`; 连同步列和时间列共 24 列。其实现先按粒子权重做并行归约, 再从二阶矩构造 rms、emittance 和 Twiss 量, 因此最小验证应同时检查 schema、权重聚合和几何分布, 而不是只检查文件存在。

本地保留官方 `initial_distribution` 中 `beam` 的参数, 构造了只包含该 `beam` 与 `bmmntr = BeamRelevant` 的 3D、1-rank、`max_step=0` sibling。 `scripts/analyze_beam_relevant_contract.py` 对 `bmmntr.txt` 做独立解析: 输出为 1 行/24 列; `z_cut=2` 的截断高斯束 `charge` 期望值为 $-9.544997\text{e-}21$ C, 实测为 $-9.544980\text{e-}21$ C, 相对误差 $1.77\text{e-}6$; 横向 rms 为 $0.249884/0.249765$ m, 纵向 rms 为 0.220356 m, 均通过 2% gate; 均值、emittance、Twiss 相关输出均有限且满足正值边界。

图 8-9 将这个初始化-only contract 的两个主要物理量画出来: 左图是实际总 `charge` 相对于截断高斯期望值的比值, 右图是三个位置 rms 与解析目标的对照。图中没有把单行输出扩展成虚假的时间演化; `gamma`、emittance 和 Twiss 量仍按报告中的 `finite/positive checks` 处理。

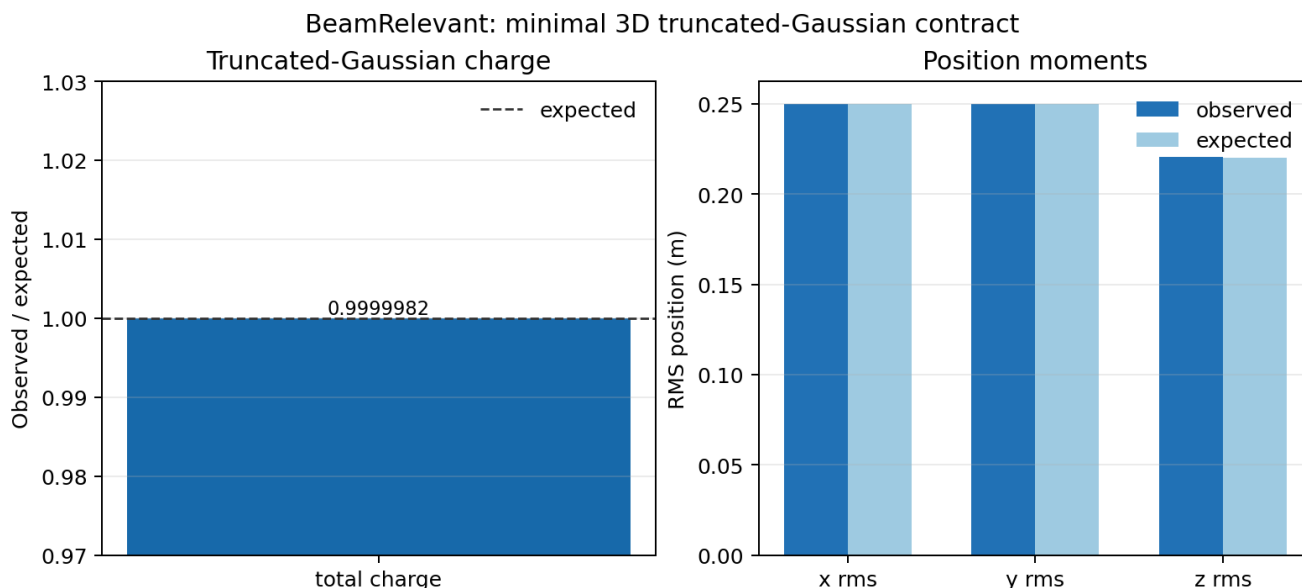


图 8-9 由 `scripts/plot_beam_relevant_contract.py` 从 `beam-relevant-contract.json` 重新生成。

Native external-file Gaussian beam: 输入文件路径与束斑物理合同

`gaussian_beam/CMakeLists.txt` 中的 `native test_3d_focusing_gaussian_beam_from_openpmd` 当前仍引用目录内不存在的 `analysis.py`。项目因此保留官方 `registration` 缺口，同时直接运行其 `prepare` 脚本和 `native input`，使用 1 rank producer 生成 `plotfile` 与 BP5 openPMD 输出，再用 `scripts/analyze_gaussian_beam_focus_contract.py` 独立读取 `iteration 0` 的 `x/y/z/w`。

运行结果为 1,999,966 个宏粒子、总权重 $1.999966e10$ 、81 个有效 `z` slice；按 focal-distance 理论包络计算的最大相对误差为 $\sigma_x = 3.0515e-2 < 0.051$ 、 $\sigma_y = 3.6214e-2 < 0.038$ 。官方 `analysis_focusing_beam.py` 也对同一输出正常结束。这个结果补足的是 `native external-file` 的项目级 `physics gate`，不应被表述为 WarpX upstream CMake 的 `analysis.py` 已经恢复；其证据目录为 `runs/stage-c-validation/gaussian_beam_native_openpmd/run/`。

图 8-10 直接从同一 BP5 iteration 0 重建每个 `z` slice 的加权 σ_x 、 σ_y ，并与 focal-distance 理论包络叠加。它展示的是 `native external-file producer` 的真实粒子输出与理论束斑合同，不是对缺失的 upstream `analysis.py` 的替代提交。

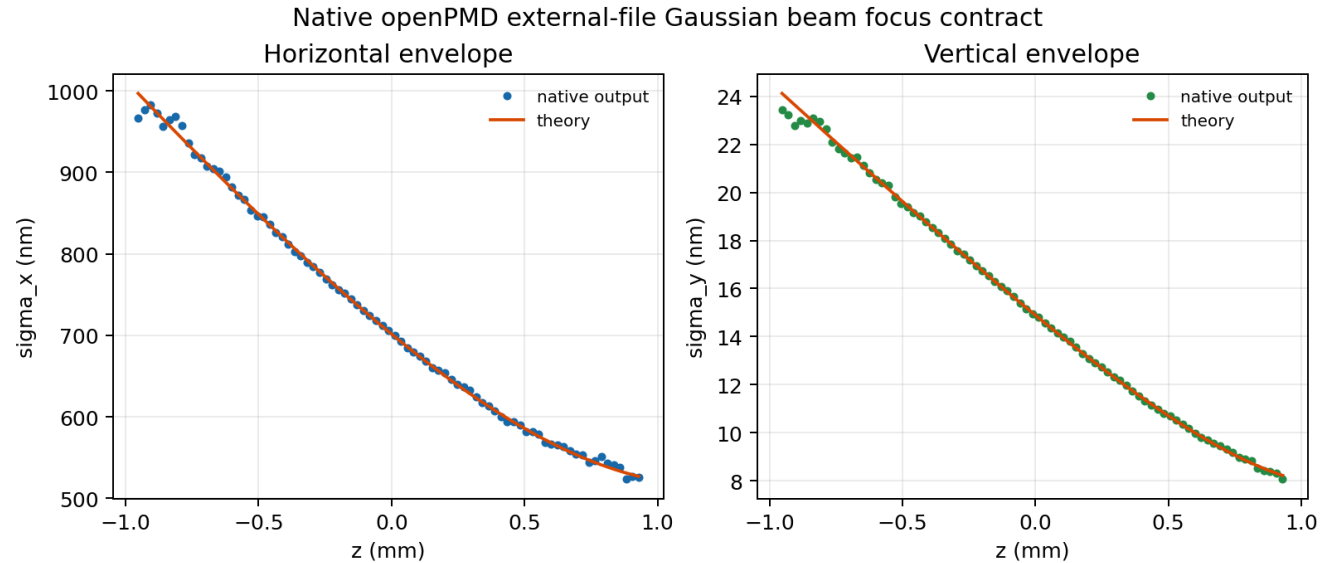


图 8-10 由 `scripts/plot_gaussian_beam_focus_contract.py` 从 `runs/stage-c-validation/gaussian_beam_native_openpmd/` 重新生成。

完整官方 `Examples/Tests/initial_distribution/ input` 现已用当前 `../warpX checkout` 重建后的 `binary` 复现。producer 和官方 `analysis.py` 均以 `exit code 0` 结束，10 类分布的最大相对差为 $1.8931e-2 < 0.02$ 。仓库 `checksum` 默认 `rtol=1e-9` 观察到最大相对差 $3.18e-3$ ，反映随机采样而非初始化失败；在显式记录的 `rtol=5e-3` `sampling tolerance` 下通过。因此本项可以从“`binary/source mismatch 未完成`”升级为“`官方分布 analysis 通过、随机 checksum 有条件通过`”，但不宣称确定性 $1e-9$ `checksum` 相等。证据目录为 `runs/stage-c-validation/initial_distribution_full_current/`。

第 8 章验证矩阵：命令、产物与 gate

本章的运行证据统一遵循同一目录约定：WarpX 原始输入只读复制到 `runs/stage-c-validation/<case>/inputs_test`，运行目录保存完整输出，项目脚本只读取该目录并生成 JSON/Markdown 摘要。下面的 MPIEXEC、WARPX 和 PYTHON 取决于本机环境；本轮实际使用的是 `MPICH launcher`、`warpX.3d/2d.MPI.OMP.DP.PDP.OPMD.FFT.EB.QED.GENQEDTABLES` 和 Python 环境中的 `yt/openpmd_api/openpmd_viewer`。

验证线	producer / MPI	项目级复现命令	主要 gate	证据目录
Langmuir	官方 1D, 1 rank	<code>python scripts/analyze_langmuir_1d_1rank.py ...</code>	场误差 $1.70e-3$ 、最终守恒量 $1.35e-2$ 误差 $3.595e-4$	<code>runs/stage-c-validation/langmuir_1d_1rank/</code>

验证线	producer / MPI	项目级复现命令	主要 gate	证据目录
Uniform plasma restart	官方 3D, 2 ranks	python scripts/analyze_uniform_plasma_restart.py ...	37 个 field; 最大相对误差 $2.8631e-12$; checksum 参考另有 rank-specific 差异	runs/stage-c-validation/unif
Uniform plasma MPI consistency	官方 3D, 1/2 ranks	python scripts/analyze_uniform_plasma_mpi_consistency.py ...	粒子总权重一致; energy 相对差分别为 $1.9379e-2/8.9170e-4/6.2269e-4$; 不通过 rank-invariant field gate	runs/stage-c-validation/unif
Energy-conserving thermal plasma	官方 1D/2D, 1 rank	python scripts/compare_energy_conserving_thermal_plasma_family.py ...	共同 EF+EP 漂移 < 0.03	runs/stage-c-validation/ener
FieldProbe	官方 coarse/refined, 1/2 rank	python scripts/compare_field_probe_resolution.py ...	coarse 3.6703% 失败; refined 0.3533% 通过	runs/stage-c-validation/fiel
Reduced observables	官方 3D, 2 rank	python scripts/analyze_reduced_observables_contract.py ...	60 项; 非 field-energy < 0.3	runs/stage-c-validation/redu
LoadBalanceCosts	Heuristic/Timers, 2 rank	python scripts/analyze_load_balance_costs_contract.py ...	eta_after > eta_before	runs/stage-c-validation/load
ColliderRelevant	官方 3D, 2 rank	python scripts/analyze_collider_relevant_contract.py ...	chi/theta/ParticleExtremum 与 dir/cte 一致	runs/stage-c-validation/coll
DifferentialLuminosity	leptons/AMR/photons, 2 rank	python scripts/analyze_diff_lumi_contract.py ...	1D/2D 高斯束解析谱	runs/stage-c-validation/diff
ParticleHistogram2D	laser-ion, 2 rank	python scripts/analyze_particle_histogram_2d_contract.py ...	BP5 0/100、 100×100 有 2 个非零数据	runs/stage-c-validation/lase
BeamRelevant	最小 3D, 1 rank	python scripts/analyze_beam_relevant_contract.py ...	24 列、截断 Gaussian	runs/stage-c-validation/beam
Full initial distribution	官方 3D, 1 rank	PYTHONPATH=.../Tools/PostProcess python .../initial_distribution_analysis.py	分布与 cases 一致; $1.8931e-2 < 0.02$; rtol=5e-3 通过	runs/stage-c-validation/init
Native Gaussian external file	gaussian_beam native, 1 rank	python scripts/analyze_gaussian_beam_focus_contract.py ...	81 z slices; sigma-x 3.8%	runs/stage-c-validation/gaus
RZ electrostatic sphere	官方 RZ, 1 rank	python scripts/analyze_rz_charge_volume_contract.py ...	官方轴向场 L2 gate; 全 charge-volume/rho-volume/particle-charge mismatch < 1%	runs/stage-c-validation/rz_e
RZ Langmuir multimode	case-local RZ sibling, 1 rank, 3 modes	python scripts/analyze_rz_langmuir_multimode_contract.py ...	m=1/2 实虚分量非零; field/writeback reconstruction < $3.1e-16$	runs/stage-c-validation/rz_l

这张表中的“通过”只表示对应列出的 gate 通过。例如 FieldProbe 的 coarse 输入仍然是失败证据, 完整 initial-distribution 的随机 checksum

也不等价于确定性 $1e-9$ 回归；这样读者可以从同一张表直接区分强 physics analysis、writer/schema contract、性能 gate 和采样统计边界。

当前证据等级应写成：Langmuir 已有运行级解析频率、场误差和最终守恒证据；uniform plasma 已有粒子数、能量统计和 checkpoint/restart 逐字段证据，但短时运行的总能量变化不能直接升级成热平衡守恒通过；FieldProbe 已确认 1/2-rank 输出一致，并通过 lambda/32 的 matched-time 解析 gate，但官方 lambda/16 coarse case 仍失败；reduced_diags 已有 60 项 compact observable 与 full-state reference 的 2-rank 逐项通过证据，并有 Heuristic/Timers 两条 LoadBalanceCosts efficiency improvement 证据；ColliderRelevant 已有 2-rank 的 chi/角度/ParticleExtrema/dL/dt 聚合合同证据；DifferentialLuminosity 已有 leptons、AMR 和 photons 三组 1D/2D 解析谱通过证据；laser-ion 已有 ParticleHistogram2D 的 2-rank openPMD writer 合同证据；BeamRelevant 已有最小 3D 的 schema/截断高斯束统计合同证据；完整 initial-distribution family 已有当前 checkout 的官方分布 analysis 通过证据，并在显式 $5e-3$ sampling tolerance 下通过 checksum，但不宣称 $1e-9$ 确定性相等；native Gaussian external-file 变体已有 1-rank 项目级束斑物理合同，但官方 CMake analysis 缺失仍保留为 upstream registration 缺口；RZ electrostatic sphere 又补充了官方场/能量 gate 与独立 rho-volume charge closure；RZ 三模 Langmuir sibling 又补充了 $m>0$ diagnostics writeback 和 theta=0 重建合同，但它是 project-level case-local evidence，不能替代官方单模 CMake analysis。JSON/Markdown 报告和脚本都保存在项目内，运行产物仍按 case-local 目录归档。

8.15 练习与复现实验

- 1. 证据分层题：从验证矩阵中各选一个 physics gate、writer/schema contract 和 performance gate，说明它们的 producer、analysis 量和“不能支持的结论”。
- 2. reader-side 复现题：使用 scripts/analyze_collider_relevant_contract.py 或 scripts/analyze_particle_histogram2 读取一个 case-local 产物，列出输入字段、输出文件和独立检查项。
- 3. 失败边界题：解释为什么 FieldProbe coarse failure、uniform-plasma reader-side 能量漂移和 initial-distribution binary mismatch 都应保留在书中，而不能简单从验证矩阵中删除。

本章后续扩写

- ☒ 加入第 8 章统一验证矩阵，列出 producer/MPI、项目级复现脚本、主要 gate 和 case-local 证据目录。
 - 继续把 ComputeDiagFunctors/、ParticleIO、WarpXOpenPMD 和 FlushFormats/ 的字段计算与 writer 细节拆开。
 - 继续把 FieldProbe、ParticleHistogram(2D)、LoadBalanceCosts 这类 reduced diagnostics 的最小输入文件和后处理示例补成可直接运行的小节。
 - 延长 uniform plasma 运行窗口，并结合 energy_conserving_thermal_plasma 的强 analysis 设计可解释的总能量 gate。
 - 为 Langmuir reader-side 拟合补充多模式/不同时间窗的敏感性检查，避免把单一窗口拟合误读为完整色散验证。

9. 文献路线与后续扩写计划

本书的文献不是装饰，也不是章节末尾统一贴一串 BibTeX。它真正承担三类职责：

- 1. 给物理结论提供一手来源。
- 2. 给数值算法和代码实现提供历史与方法边界。
- 3. 给 reader-side analysis、benchmark 和 regression 判据提供外部对照。

因此本章不再把文献简单列成“推荐阅读书单”，而是把当前项目里已经 materialize 的论文资产、仍未闭环的 acquisition 缺口，以及它们与各章的绑定关系写成一张可执行路线图。

9.1 当前项目的文献证据分层

当前本地文献证据有四层，强度不能混写：

层级	当前项目中的典型形态	可支持的写法	当前限制
A. 已 materialize 的正文资产	本地 PDF + MinerU Markdown + images/ + 中文讲解 + reading-log.md	可直接作为正文一手证据	仍需作者自己对照具体公式、图和段落，而不是只看中文摘要
B. 已取得 PDF 但未完成精读	本地 PDF 存在，但还没有完整中文讲解或章节回填	可作为“已获取、待精读”的明确线索	不能把具体公式或图表当成已核实正文
C. metadata / abstract 级线索	DOI、题名、摘要、访问审计、下载日志	可作为 acquisition 边界、章节缺口或后续计划	不能把摘要内容冒充成论文正文结论

层级	当前项目中的典型形态	可支持的写法	当前限制
D. 旁证或相关文献	主题相关但不是当前章的主引用，或并非同一 bibliographic item	可作背景、旁证、术语线索	不能替代主引用本身

当前项目的规则应保持为：

- 只有 A 层资产，才允许在正文里写成“已核实的一手证据”。
- B 层资产只能写成“已获取但尚待逐段讲解”。
- C 层和 D 层只能写成 acquisition / 背景边界，不能抬成正文论证。

这条规则尤其影响当前仍未闭环的 Hockney-Eastwood、Yee 1966、Esirkepov 2001、Villasenor-Buneman 1992 和 LeeCPC2015。

9.2 当前已 materialize 的核心文献树

按当前 references/ 目录，已经完成 materialization 的核心文献主要集中在三条主线。

9.2.1 PIC foundations

当前已 materialize：

- references/02_books_lecture_notes/1985_BirdsallLangdon_Plasma_physics_via_computer_simulation/
- references/03_pic_foundations/1979_TajimaDawson_Laser_Electron_Accelerator/
- references/03_pic_foundations/1983_Dawson_Particle_simulation_of_plasmas/

这三条线已经足以支撑：

- 第 1 章的 superparticle、weighted particles、finite-size particles、quiet start、噪声与 heating 讨论；
- 第 2 章的最小 PIC loop、electrostatic / EM / Darwin 模型边界；
- 第 8 章中 diagnostics、spectrum、correlation time、weak instability dynamic range 的讨论；
- LWFA 最早 scaling baseline 的历史入口。

其中 Birdsall 1985 因原书过长，当前项目采用分卷 PDF + 分卷 MinerU 的方式处理；这意味着它已经是 A 层资产，但仍不是“整本都已完全精读”。

9.2.2 Particle pusher

当前已 materialize：

- references/04_particle_pushers_deposition_shapes/2008_VayPOP2008_Simulation_of_beams_or_plasmas_crosss/
- references/04_particle_pushers_deposition_shapes/2017_HigueraPOP2017_Structure-preserving_second-order

这两条线当前已经足以支撑：

- 第 4 章对 Vay pusher 与 Higuera-Cary pusher 的源码讲解；
- Source/Particles/Pusher/UpdateMomentumVay.H 与 UpdateMomentumHigueraCary.H 的公式对表；
- “相对论精度”和“结构保持”两条不同的算法卖点。

但这条模块还没有完成 Boris 原始文献闭环，因此第 4 章仍不是“推进器历史谱系全闭环”。

9.2.3 PSATD / Galilean / boosted-frame / NCI

当前已 materialize：

- references/06_stability_filtering_nci/2014_GodfreyJCP2014_Numerical_stability_analysis_of_the_PSATD_P/
- references/06_stability_filtering_nci/2016_KirchenPOP2016_Stable_discrete_representation_of_relativist/
- references/06_stability_filtering_nci/2016_LehePRE2016_Elimination_of_NCI_by_Galilean_coordinates/

这三条线已经构成第 6 章目前最完整的一组 paper-backed 主干：

- Godfrey 2014: fixed-grid PSATD 的 NCI 策略分类；
- Lehe 2016: Galilean coordinates 消除 NCI 的核心离散论证；

- Kirchen 2016: boosted-frame workflow 与稳定离散表示之间的应用层连接。

因此第 6 章当前虽然仍有 runtime validation 和 upstream handoff 的工程缺口，但在文献层已经不再是空心章节。

9.3 当前最突出的未闭环文献缺口

如果按“哪一章会因为缺它而不够出版级”排序，当前最重要的缺口不是更多新论文，而是以下几条老而关键的 primary sources。

缺口	当前状态	主要影响章节	当前可替代程度
Hockney-Eastwood 原书	仅有 BibTeX 与 fallback article 线索；无本地合法 PDF	第 1、2、5、6 章	只能部分由 Birdsall 1985 与 Dawson 1983 顶住
Yee 1966	metadata/DOI 已清楚；无本地 PDF/MinerU	第 2、6 章	可暂由源码与后继 FDTD 文献支撑，但缺原始历史入口
Esirkepov 2001	已建立 paper-specific 目录、access audit，并已 materialize 作者 arXiv 预印本 + MinerU + 中文讲解；仍缺出版商 CPC PDF 对照	第 5 章	已从纯源码缺口推进到 preprint-backed，但还未完成 CPC 定稿核对
Villasenor-Buneman 1992	已建立 paper-specific 目录、access audit，并已 materialize 本机现成 PDF + MinerU + 中文讲解	第 5 章	已从纯源码缺口推进到 paper-backed，但中文讲解仍是第一轮结构精读
LeeCPC2015	已有 access audit、公式映射准备和核对清单；仍无授权 PDF/MinerU 正文	第 7 章	源码侧 C1-C25 和 regression 可继续推进，但论文闭环仍缺主文

这五条缺口里，LeeCPC2015 最特殊。它不是完全没工作，而是已经推进到：

- access-audit.md
- 公式映射准备.md
- 公式核对清单.md

也就是说，当前不是“不知道该怎么读”，而是“知道该对什么，但还拿不到可合法精读的正文 PDF”。

9.4 各章当前的文献成熟度

把全书按章节看，当前文献成熟度并不均匀。

章节	当前文献成熟度	主要已闭环来源	主要缺口
第 1 章 动理学模型	中等	Birdsall 1985、Dawson 1983	Hockney-Eastwood、更细的 particle-mesh heating 原始文献
第 2 章 PIC 总循环	中等	Birdsall 1985、Dawson 1983	Yee 1966 原始入口
第 3/3A 章 主循环与初始化	中低	以源码为主	需要把基础文献和工程论文绑定得更明确
第 4 章 粒子推进器	中高	Vay 2008、Higuera-Cary 2017	Boris 历史源头仍缺
第 5 章 沉积与形函数	中等	Esirkepov 与 Villasenor 两条 charge-conserving 主线都已有一轮 paper-backed 资产	仍需把两篇论文系统回写正文；Esirkepov 还缺 CPC 定稿对照
第 6 章 场求解器	高	Godfrey 2014、Lehe 2016、Kirchen 2016	更多 validation/engineering 线，而不是 paper 主干
第 7 章 边界、PML 与 AMR	中等偏低	Berenger 1994/1996 有 bibliographic anchor，源码和 regression 很强	LeeCPC2015 正文仍缺

章节	当前文献成熟度	主要已闭环来源	主要缺口
第 8 章 诊断、验证与案例	中等	Dawson 1983 diagnostics 思路已可直接服务正文	还缺更多 case-specific benchmark papers
第 9 章 文献路线	本章即路线图	当前 references/ 树和 docs/literature-map.md	需要持续同步，而不是一次性写完

这个表最重要的结论是：当前项目最缺 paper-backed 收口的不是第 6 章，而是第 5 章和第 7 章。

9.5 acquisition 优先级的重新排序

基于当前项目状态，后续 acquisition 不应再泛泛地“多找一些相关论文”，而应按成书影响排序：

1. Esirkepov 2001 的 CPC 定稿 PDF
 - 当前已有作者预印本，可支持第一轮论证；下一步是把预印本与 2001 CPC 发表版逐项对齐。
2. Yee 1966
 - 直接补第 2 / 6 章里的原始 FDTD 入口。
3. LeeCPC2015 正文 PDF
 - 直接补第 7 章的 PML paper closure。
4. Hockney-Eastwood 或其 article-level fallback
 - 继续补第 1 / 2 章的 particle-mesh foundations。
5. Boris 原始文献
 - 继续补第 4 章的 pusher 历史链。

这个顺序和早期版本相比已经变了。原因很简单：第 6 章现在已有较强 paper 主干，而第 5 / 7 章的 paper closure 反而更薄。

9.6 对 docs/literature-map.md 的使用边界

当前 docs/literature-map.md 已经不只是“列一下 BibTeX key”，而是承担三种作用：

1. 统计当前本地 PDF / topic 分布；
2. 记录哪些核心文献已经 materialize；
3. 记录哪些缺口当前只有 metadata / audit / fallback。

但它仍然是总索引，不适合直接拿来替代章节级写作清单。章节写作时更合理的做法是：

- 第 1 / 2 章先看 docs/foundations-literature-list.md
- 第 6 / 7 章结合 references/06_stability_filtering_nci/ 与 references/08_boundaries_pml_geometry/
- acquisition 计划再回到 docs/literature-map.md 和 references/00_index/books_to_locate.md

也就是说，literature-map 是总表，不是每章的最终操作手册。

9.7 下一轮最合理的文献推进目标

如果下一轮继续走“一个大模块一个版本”的节奏，那么最合理的文献模块不再是第 6 章，而是下面两条中的一条：

方案 A：第 5 章沉积文献闭环

目标：

- compare the current Esirkepov 2001 arXiv preprint against the 2001 CPC publication PDF
- deepen the current first-round Villasenor and Esirkepov Chinese notes into fuller formula-level walkthroughs
- 把第 5 章从源码校准推进到论文-源码-测试三线闭环

适合原因：

- 当前第 5 章是全书里最明显的“代码已读、文献未补”的章节之一；
- 这条线一旦闭合，会显著提升前半本书的基础可信度。

方案 B: 第 7 章 PML 论文闭环

目标:

- 继续推进 LeeCPC2015 正文获取
- 若正文仍不可得, 则至少把 Berenger 1994/1996 与 WarpX PsatdAlgorithmPml.cpp 的公式映射继续压实

适合原因:

- 当前第 7 章源码和 regression 已经很强, 只差 paper 正文闭环;
- 一旦拿到 LeeCPC2015 正文, 整章会从“强源码章”变成真正的 paper-backed 章节。

在这两者之间, 当前更推荐先走方案 A。原因是:

- 方案 A 不强依赖外部授权状态;
- 方案 B 仍可能被 PDF 获取问题卡住。

9.8 本章结论

当前项目的文献工作已经跨过了“只有书目, 没有正文资产”的阶段, 但还远未到“全书 primary sources fully closed”的阶段。可以更准确地概括成:

- foundations 线已有 Birdsall 1985、Dawson 1983、Tajima-Dawson 1979
- pusher 线已有 Vay 2008、Higuera-Cary 2017
- PSATD/NCI 线已有 Godfrey 2014、Lehe 2016、Kirchen 2016
- PML 线已有较强源码与审计资产, 但缺 LeeCPC2015 正文
- deposition 线当前已建立 Esirkepov 2001 与 Villasenor-Buneman 1992 的 paper-specific 目录与 access audit; 其中 Esirkepov 2001 已 materialize 作者 arXiv 预印本并完成第一轮 MinerU/中文讲解, Villasenor-Buneman 1992 也已从本机现成 PDF/MinerU 资产 materialize 到项目目录并完成第一轮中文讲解

因此, 这条路线图给后续推进的核心约束不是“再多下载一些论文”, 而是:

1. 优先补能直接改变章节可信度的 primary sources;
2. 严格区分 materialized 正文资产和 metadata-level 线索;
3. 把 acquisition、MinerU、中文精读和章节回填继续绑成同一条工作流。

做到这三点, 第 9 章才不是一个附录式书单, 而是真正控制全书证据质量的总调度章。

附录 A: 符号、时间层与源码变量

本附录服务于正文阅读和源码检索。公式中的符号采用连续模型或离散模型的常用记号; 反引号中的名称则优先对应 WarpX 源码、输入文件或 diagnostics 输出中的实际字段。除特别说明外, SI 制单位适用, 粒子权重 w_p 表示一个宏粒子代表的真实粒子数。

A.1 连续模型符号

符号	含义	常见单位或类型
s	species 索引, 例如 electron、ion、photon	无量纲整数
$f_s(\mathbf{x}, \mathbf{p}, t)$	species s 的相空间分布函数	依定义而定
q_s, m_s	species 电荷和静质量	C、kg
$\mathbf{x} = (x, y, z)$	物理空间位置	m
$\mathbf{p}$	物理动量	kg m s ⁻¹
$\mathbf{u} = \gamma \mathbf{v}$	归一化动量; WarpX 粒子分量通常写作 u_x, u_y, u_z	m s ⁻¹
$\mathbf{v}$	粒子速度	m s ⁻¹
γ	Lorentz 因子, $\gamma = (1 - v^2/c^2)^{-1/2}$	无量纲
c	真空光速	m s ⁻¹
$\mathbf{E}, \mathbf{B}$	电场和磁场	V m ⁻¹ 、T
$\rho, \mathbf{J}$	电荷密度和电流密度	C m ⁻³ 、A m ⁻²
ϵ_0, μ_0	真空介电常数和磁导率	SI 常数
w_p	宏粒子权重, 代表的真实粒子数	无量纲或按模型定义

符号	含义	常见单位或类型
S	粒子到网格的空间形函数	按离散归一化定义

A.2 网格、时间层与离散量

符号	含义	在程序中的对应
t^n	第 n 个整数时间层	<code>step</code> 、 <code>istep</code>
$t^{n+1/2}$	半时间层, 常用于 leapfrog 电流或磁场	$J^{n+1/2}$ 等时间层语义
Δt	时间步长	<code>dt</code> 、 <code>warpX.const_dt</code> 或派生时间步长
$\Delta x, \Delta y, \Delta z$	各方向网格间距	<code>amr.n_cell</code> 、几何 <code>cell_size</code>
i, j, k	网格单元或节点索引	AMReX <code>IntVect</code> 分量
ℓ	AMR level 索引, $\ell = 0$ 为最粗层	<code>lev</code> 、 <code>level</code>
r	MPI rank 或 rank 索引	<code>ParallelDescriptor::MyProc()</code> 等
V_i	网格单元体积	<code>Cartesian</code> 中常为 $\Delta x \Delta y \Delta z$
ρ_i^n	单元/节点 i 在时间层 n 的电荷密度	<code>rho</code> 、 <code>rho_fp</code> 、 <code>rho_buf</code>
$\mathbf{J}_i^{n+1/2}$	时间层 $n + 1/2$ 的电流密度	<code>current_fp</code> 、 <code>current_buf</code>
$\nabla_i \cdot \mathbf{J}$	离散散度	由 <code>staggering</code> 和差分 <code>stencil</code> 决定
$S_i(\mathbf{x}_p)$	粒子位置对网格量 i 的形函数值	<code>ShapeFactors</code> 中的 <code>shape</code> 权重

A.3 沉积、推进与场求解记号

符号或名称	含义	阅读提示
$\rho^n \rightarrow \mathbf{J}^{n+1/2} \rightarrow \rho^{n+1}$	电荷守恒 current-deposition 主线的时间层关系	不能把 charge deposition 和 current deposition 当成同一个 kernel
old / new	粒子在一个推进步或 segment 两端的形函数/位置状态	Esirkepov、Villasenor kernel 中常见
relative_time	在同一步内取样粒子位置的相对时间参数	影响 source 的时间层, 而不是额外推进粒子
icomp	charge component 或时间层分量选择	需结合调用入口解释, 不能只按名字猜测
depos_lev	charge 沉积目标 AMR level	coarse-buffer 粒子可能直接沉积到 <code>lev-1</code>
current_fp	fine-patch current buffer	主要对应 fine patch 粒子沉积
current_buf	coarse-buffer current buffer	后续还要经过同步/整理
rho_fp / rho_buf	fine-patch / coarse-buffer charge buffer	是 source route 的观测面, 不等于最终 <code>plotfile</code> 字段
Direct	直接速度加权电流沉积	简单但不自动满足离散连续性方程
Esirkepov	基于轨迹与形函数差分的 charge-conserving current deposition	重点看 <code>density decomposition</code> 和 <code>prefix accumulation</code>
Villasenor	基于 cell crossing segment 的 charge-conserving current deposition	重点看 <code>crossing</code> 、 <code>segment</code> 和局部 <code>face flux</code>
Vay	Vay current deposition / spectral 相关路径	常与 PSATD、current correction 组合讨论
FDTD	有限差分场域求解器	依赖 <code>staggered grid</code> 和 CFL 限制
PSATD	pseudo-spectral analytical time-domain 求解器	重点看谱空间系数、源项时间模型和周期边界假设
JRhom	PSATD 的多次 J/ρ 时间采样路径	不是多次粒子 push, 而是同一轨迹的多时刻源项
PML	吸收边界层	通过 <code>split-field</code> 或等价谱推进吸收出射波

A.4 诊断、文件与验证术语

名称	含义
plotfile	WarpX/AMReX 网格和粒子状态的可重启或后处理输出
openPMD	粒子、网格或 reduced diagnostics 的结构化输出接口
diagNNNNNNN	diagnostics 输出的迭代目录，例如 diag1000000
reduced_diags	不保存完整场/粒子，而输出聚合标量或低维量的 diagnostics
FieldProbe	沿指定几何路径采样场并做积分或线探针输出的诊断
BoundaryScrapingDiagnostics	记录穿过粒子边界的粒子及其统计量
producer	产生字段、粒子或 reduced output 的运行时路径
consumer	读取产物并执行物理、格式或性能断言的分析脚本
physics gate	对解析解、守恒量、谱或物理量的数值容差断言
writer gate	对文件、字段、维度、iteration、轴和有限非零数据的断言
checksum gate	对最终输出做确定性校验；不能自动等价于物理正确性
reader-side analysis	从已有 plotfile/openPMD 重新读取数据并验证合同的分析层

A.5 常用缩写

缩写	全称	本书中的语境
PIC	Particle-In-Cell	粒子-网格数值方法
AMR	Adaptive Mesh Refinement	多层网格和 coarse/fine 同步
CFL	Courant-Friedrichs-Lewy	显式场推进稳定性限制
NCI	Numerical Cherenkov Instability	boosted-frame / PSATD 中的数值不稳定性
RZ	cylindrical geometry with azimuthal modes	WarpX 的轴对称/模态几何
EB	Embedded Boundary	嵌入边界和 cut-cell 几何
MPI	Message Passing Interface	rank 间并行通信
GPU	Graphics Processing Unit	device kernel 和 GPU memory 路径
QED	Quantum Electrodynamics	高场量子辐射和 pair-production 模型
LWFA / PWFA	Laser / Plasma Wakefield Acceleration	激光/束流驱动尾场应用

A.6 使用规则

- 看到 `rho`、`current_fp` 或 `current_buf` 时，先判断它属于 local kernel、level buffer、同步后场，还是最终 diagnostics 输出；同名物理量可能处于不同生命周期阶段。
- 看到上标

$$n$$

、

$$n + 1/2$$

- 或 `relative_time` 时，先确认粒子位置、场、`current` 和 `charge` 是否处在同一个时间层；不能只根据变量名推断守恒性。
- 看到 `analysis.py`、`analysis_default_regression.py` 或 `checksum` 时，先区分 physics、writer 和 checksum 三类 gate；本书第 8 章明确保留这三种证据等级的边界。
 - 看到 `lev`、`fine`、`coarse` 或 `buf` 时，先回到 AMR route 和同步顺序，再解释某个字段的数值意义。